

ARC12 ▪ Architettura dei Calcolatori e Laboratorio

RISC-V sulla piattaforma PRG32

Università degli Studi di Napoli «Parthenope» • A.A. 2026/27

Architettura dei calcolatori

Prima programmare, poi formalizzare

Dalla prima istruzione RISC-V a un gioco pubblicato:
informazione, logica, memoria e processore, insegnati sulla piattaforma PRG32
attraverso Verm1, il progetto che accompagna tutto il corso

Raffaele Montella

Dipartimento di Scienze e Tecnologie
Università degli Studi di Napoli «Parthenope»

Implementazione di riferimento di Verm1: contributo di Giuseppe Coviello

Libro di testo delle lezioni di *Architettura dei Calcolatori e Laboratorio di Architettura dei Calcolatori* (ARC12),
Corso di Laurea in Informatica, primo anno, primo semestre.

I listati assembly di questo libro sono letti da file che vengono assemblati
e provati automaticamente.

Piattaforma: https://github.com/riscv-prg32/PRG32
Host desktop e debugger: https://github.com/riscv-prg32/PRG32-QT

Architettura dei calcolatori: prima programmare, poi formalizzare.
Libro di testo delle lezioni di ARC12, Architettura dei Calcolatori e Laboratorio di Architettura dei Calcolatori, Università degli Studi di Napoli «Parthenope», anno accademico 2026/27. Edizione italiana di *Computer Architecture: Program First, Formalise Later*.

Il libro segue, capitolo per capitolo, le venticinque lezioni del corso: il Capitolo N è la Lezione N. Ogni lezione ha due presentazioni, una per i concetti e una per il codice. Le presentazioni, il programma di ogni lezione, i programmi di riferimento del progetto Verm1 e i test che assemblano ed eseguono ogni listato sono conservati insieme ai sorgenti di questo libro; i listati stampati qui sono letti dagli stessi file che i test eseguono. Per questo i listati, con i loro commenti, sono in inglese in entrambe le edizioni: sono il codice provato, non una sua copia.

Il libro è costruito attorno a PRG32, un runtime didattico aperto per giochi in assembly RISC-V e in C, che ha come destinazioni il microcontrollore ESP32-C6, l'emulatore QEMU di Espressif e l'host desktop PRG32-QT. La piattaforma, il suo codice sorgente e la sua documentazione sono distribuiti con licenza MIT dai loro autori: Raffaele Montella (responsabile scientifico e del progetto), Ivan Cafiero e Simone Boscaglia (studenti contributori), Università degli Studi di Napoli «Parthenope». A chi usa PRG32 in esercitazioni, relazioni, tesi o materiale didattico si chiede di citarlo attraverso i metadati `CITATION.cff` del suo repository (Montella et al., 2026a). Il volume che lo accompagna, *Playing with RISC-V* (Montella et al., 2026b), è il manuale di laboratorio della piattaforma.

L'implementazione di riferimento di Verm1, il progetto che accompagna tutto il libro (i quattro programmi dell'Appendice D), è un contributo di Giuseppe Coviello.

Il testo, i programmi e le figure originali di questo libro, come tutto il materiale del corso, sono rilasciati con licenza Creative Commons Attribuzione – Condividi allo stesso modo 4.0 Internazionale (CC BY-SA 4.0). PRG32 e PRG32-QT sono progetti distinti, distribuiti con licenza MIT.

Crediti delle immagini. Le fotografie e gli schemi elencati qui provengono da Wikimedia Commons e conservano le proprie licenze; le altre figure sono state realizzate per questo libro. Nell'ordine dei nomi dei file: `abacus_chinese.jpg`: *Abacus 5*, Loadmaster (David R. Tribble), CC BY-SA 3.0; `abacus_roman.jpg`: *Number Stories, p51-illustration*, David Eugene Smith, Public domain; `analytical_engine.jpg`: *Babbages Analytical Engine, 1834-1871. (9660574685)*, Science Museum London / Science and Society Picture Library, CC BY-SA 2.0; `antikythera.jpg`: *NAMA Machine d'Anticythère 1*, Marsyas, CC BY 2.5; `antikythera_plate.jpg`: *Antikythera mechanism clockface, 1st-2nd century BC, Greece (model)*, Gts-tg, CC BY-SA 4.0; `astrolabe.jpg`: *Astrolabe planisférique closeup800x600x300*, Pom^2, CC BY-SA 3.0; `c64.jpg`: *Commodore-64-Computer-FL*, Evan-Amos, Public domain; `cdrom_drive.jpg`: *CD Player focusing lens assembly*, Zim 256, CC BY-SA 3.0; `chip_74273.jpg`: *Animation Technologies LifeView LR25 - Hitachi LS273-2523*, Raimond Spekking, CC BY-SA 4.0; `colossus.jpg`: *Dorothy Du Boisson*, unknown author (UK Government, 1943), Public domain; `core_memory.jpg`: *HP 9101A extended memory unit.- magnetic core memory module - macro detail*, Mister rf, CC BY-SA 4.0; `crt_scope.jpg`: *Tennis For Two on a DuMont Lab Oscilloscope Type 304-A*, Brookhaven National Laboratory, Public domain; `difference_engine.jpg`: *Babbage Difference Engine*, geni, CC BY-SA 4.0; `dvd_drive.jpg`: *Dvd-2010a hg*, Hannes Grobe, CC BY-SA 4.0; `eniac.png`: *Glen Beck and Betty Snyder program the ENIAC in building 328 at the Ballistic Research Laboratory*, U.S. Army, Public domain; `enigma.jpg`: *Enigma I cipher machine Museum of Communication Berlin cropped*, Museum für Kommunikation Berlin, CC BY 4.0; `eprom.jpg`: *Pair of BIOS chips*, Jud McCranie, CC BY-SA 4.0; `hard_disk.jpg`: *Open hard-drive*, Zzubnik, Public domain; `laser_printer.jpg`: *Laser printer-Writing*, DMahalko, traced by Stannered, CC BY 3.0; `lcd_layers.jpg`: *TN-LCD-schematic-MS-208kB*, see the file page, CC BY-SA 3.0; `lcd_pixels.jpg`: *Subpixel rendering LCD photo 3e composite*, ALexL33, CC BY-SA 3.0; `leibniz_reckoner.jpg`: *Leibniz-Rechenmaschine hg*, Hannes Grobe, CC BY 3.0; `mac_classic.jpg`: *Macintosh 128k transparency*, Grm wnr, CC BY-SA 3.0; `mainframe.jpg`: *Dept of Agriculture Computer Room*, Warren K. Leffler (U.S. News & World Report, Library of Congress), Public domain; `minicomputer.jpg`: *DEC PDP-11 Minicomputer in extended variant (edited)*, Loz Pycock, CC BY-SA 2.0; `pascaline.jpg`: *Arts et Metiers Pascaline dsc03869*, David.Monniaux, CC BY-SA 3.0; `pcb.jpg`: *Hand Etched PCB*, Tpdwkouaa, CC BY-SA 4.0; `pit_land.jpg`: *CD-ROM pressed pits*, Tycho,

CC0; `punched_cards.jpg`: *A Jacquard loom showing information punchcards, National Museum of Scotland*, Stephencdickson, CC BY-SA 4.0; `ram_modules.jpg`: *Platinen-001 hg*, Hannes Grobe, CC BY-SA 4.0; `ram_on_board.jpg`: *IBM PS2 MCA Model 55 SX, power supply, memory, riser base*, Dale Mahalko, CC BY-SA 3.0; `ssd.jpg`: *Vertex 2 Solid State Drive by OCZ-top oblique PNr°0307*, D-Kuru, CC BY-SA 3.0 AT; `storage_racks.jpg`: *StorageTek tape library (51993015872)*, Jorge Franganillo, CC BY 2.0; `workstation_board.jpg`: *IBM PC Server 320 motherboard front*, deksor, CC BY-SA 4.0; `zuse_z3.jpg`: *Z3 Deutsches Museum b*, Venusianer, CC BY-SA 3.0.

RISC-V è un marchio registrato di RISC-V International. ESP32, ESP32-C3, ESP32-C6 ed ESP-IDF sono prodotti e marchi di Espressif Systems. GitHub è un marchio di GitHub, Inc. Tutti gli altri marchi appartengono ai rispettivi titolari e sono usati qui al solo scopo di identificazione.

Edizione compilata il 17 ottobre 2026. Composto con LaTeX in Latin Modern.

Indice

Prefazione **xiii**

Come usare questo libro **xv**

Partire da zero **xix**

I Partenza **1**

0 Introduzione: prima programmare, poi formalizzare **3**
0.1 Il corso in breve . . . 4
0.2 Un approccio nuovo . . . 4
0.3 La macchina: PRG32 e RISC-V . . . 5
0.4 Il progetto: Verm1 . . . 9
0.5 Il corso in blocchi . . . 10
0.6 Programma e risultati di apprendimento . . . 10
0.7 Organizzazione e valutazione . . . 11
0.8 Materiale . . . 13
0.9 Primo contatto: chi siete? . . . 14
0.10 Prima del prossimo capitolo . . . 14
0.11 Il programma di questo capitolo . . . 14

1 PRG32 e PRG32-QT: allestire l'officina **17**
1.1 Quattro livelli: installate ciò che vi serve . . . 18
1.2 Prima di cominciare: le parole di oggi . . . 18
1.3 La piattaforma, in dettaglio . . . 19
1.4 Passo 1: installare PRG32-QT . . . 21
1.5 Passo 2: installare la toolchain . . . 23
1.6 Passo 3: la vostra prima compilazione . . . 26
1.7 Facoltativo: QEMU e il kit . . . 28
1.8 Quando qualcosa non funziona . . . 29
1.9 Il programma di questo capitolo . . . 29
1.10 Verificate di aver capito . . . 31
1.11 Esercizi . . . 32
1.12 Glossario . . . 32
1.13 Riepilogo e letture . . . 32

2 Primo contatto: assembly RISC-V su PRG32 **37**

2.1 La macchina . . . 38
2.2 La vostra prima cartuccia . . . 39
2.3 Leggere ciò che avete scritto . . . 44
2.4 Che cosa viene dopo . . . 46
2.5 Il programma di questo capitolo . . . 46
2.6 Verificate di aver capito . . . 49
2.7 Esercizi . . . 49
2.8 Glossario . . . 50
2.9 Riepilogo e letture . . . 50

II Informazione **51**

3 Storia della computazione **53**
3.1 Che cos'è l'informatica? . . . 54
3.2 Prima dell'elettricità . . . 54
3.3 Il Novecento . . . 56
3.4 Dagli home computer a RISC-V . . . 58
3.5 L'informatica come scienza applicata . . . 60
3.6 Il programma di questo capitolo . . . 61
3.7 Verificate di aver capito . . . 64
3.8 Esercizi . . . 64
3.9 Glossario . . . 65
3.10 Conclusioni e letture . . . 65

4 Rappresentare l'informazione digitale **67**
4.1 L'informazione . . . 68
4.2 Il testo . . . 71
4.3 Strutture dati . . . 73
4.4 Dati e algoritmi . . . 74
4.5 Il programma di questo capitolo . . . 76
4.6 Verificate di aver capito . . . 78
4.7 Esercizi . . . 79
4.8 Glossario . . . 79
4.9 Conclusioni e letture . . . 79

5 Rappresentazione dei numeri: gli interi **81**
5.1 Riscaldamento sulla macchina . . . 81
5.2 Basi e conversioni . . . 82
5.3 Numeri naturali e overflow . . . 83
5.4 Interi con segno . . . 85
5.5 Ritorno alla macchina . . . 89
5.6 Il programma di questo capitolo . . . 91
5.7 Verificate di aver capito . . . 94
5.8 Esercizi . . . 95
5.9 Glossario . . . 95
5.10 Conclusioni e letture . . . 95

6 Rappresentazione dei numeri: i reali **97**
6.1 Riscaldamento sulla macchina . . . 97

6.2 Altre codifiche per gli interi con segno . . . 98
6.3 Virgola fissa . . . 99
6.4 Virgola mobile . . . 101
6.5 Ritorno alla macchina . . . 104
6.6 Il programma di questo capitolo . . . 105
6.7 Verificate di aver capito . . . 106
6.8 Esercizi . . . 107
6.9 Glossario . . . 108
6.10 Conclusioni e letture . . . 108

7 Verm1, parte 1: una testa che attraversa il mondo 109
7.1 Il progetto . . . 110
7.2 Passo 0: il ciclo di lavoro . . . 111
7.3 Passo 1: tre funzioni che ritornano . . . 111
7.4 Passo 2: una griglia e una testa . . . 112
7.5 Passo 3: il movimento . . . 114
7.6 Passo 4: guardare dentro con PRG32-QT . . . 116
7.7 Verificate di aver capito . . . 117
7.8 Fatelo vostro . . . 118
7.9 Dove siamo, e che cosa viene dopo . . . 118

III Logica 121

8 Algebra di Boole: operatori e proprietà 123
8.1 Riscaldamento sulla macchina . . . 123
8.2 Il calcolatore come macchina logica . . . 124
8.3 L'algebra di Boole . . . 126
8.4 Ritorno alla macchina . . . 128
8.5 Il programma di questo capitolo . . . 130
8.6 Verificate di aver capito . . . 133
8.7 Esercizi . . . 133
8.8 Glossario . . . 134
8.9 Conclusioni e letture . . . 134

9 Algebra di Boole: funzioni, forme canoniche e minimizzazione 135
9.1 Riscaldamento sulla macchina . . . 135
9.2 Funzioni booleane . . . 136
9.3 Forme canoniche . . . 138
9.4 Mappe di Karnaugh . . . 140
9.5 Ritorno alla macchina . . . 142
9.6 Il programma di questo capitolo . . . 143
9.7 Verificate di aver capito . . . 145
9.8 Esercizi . . . 146
9.9 Glossario . . . 147
9.10 Conclusioni e letture . . . 147

10 Elettronica: di che cosa è fatto un bit 149
10.1 Riscaldamento sulla macchina . . . 149
10.2 L'elettricità in quattro grandezze . . . 150

10.3 Semiconduttori . . . 152
10.4 Dai volt ai bit . . . 153
10.5 Ritorno alla macchina . . . 156
10.6 Il programma di questo capitolo . . . 158
10.7 Verificate di aver capito . . . 160
10.8 Esercizi . . . 160
10.9 Glossario . . . 161
10.10 Conclusioni e letture . . . 161

11 Macchine combinatorie **163**
11.1 Riscaldamento sulla macchina . . . 163
11.2 Macchine combinatorie . . . 164
11.3 Codici . . . 165
11.4 Decodificatori e codificatori . . . 167
11.5 Multiplexer e demultiplexer . . . 169
11.6 Confrontare e controllare . . . 171
11.7 L'unità aritmetico-logica . . . 172
11.8 Il programma di questo capitolo . . . 173
11.9 Verificate di aver capito . . . 175
11.10 Esercizi . . . 176
11.11 Glossario . . . 177
11.12 Conclusioni e letture . . . 177

12 Verm1, parte 2: la guida, il cibo e un punteggio **179**
12.1 Partite da una parte 1 che funziona . . . 179
12.2 Passo 1: nomi per i bit . . . 180
12.3 Passo 2: la guida . . . 181
12.4 Passo 3: il cibo, un punteggio, un suono . . . 185
12.5 Passo 4: osservare una decisione . . . 188
12.6 Verificate di aver capito . . . 190
12.7 Fatelo vostro . . . 190
12.8 Dove siamo, e che cosa viene dopo . . . 191

IV Stato e memoria **193**

13 Macchine sequenziali **195**
13.1 Riscaldamento sulla macchina . . . 195
13.2 Tempo e stato . . . 196
13.3 Macchine a stati finiti . . . 197
13.4 Latch . . . 200
13.5 Flip-flop . . . 202
13.6 Ritorno alla macchina . . . 203
13.7 Il programma di questo capitolo . . . 206
13.8 Verificate di aver capito . . . 208
13.9 Esercizi . . . 208
13.10 Glossario . . . 209
13.11 Conclusioni e letture . . . 209

14 Macchine elementari: registri e bus **211**

14.1 Riscaldamento sulla macchina . . . 211
14.2 Porte di parola e registri . . . 212
14.3 Bus . . . 213
14.4 Buffer a tre stati . . . 213
14.5 Registri a scorrimento . . . 215
14.6 Ritorno alla macchina . . . 217
14.7 Il programma di questo capitolo . . . 218
14.8 Verificate di aver capito . . . 221
14.9 Esercizi . . . 221
14.10 Glossario . . . 222
14.11 Conclusioni e letture . . . 222

15 La memoria **223**
15.1 Riscaldamento sulla macchina . . . 223
15.2 Che cos'è una memoria? . . . 224
15.3 Un po' di storia: anelli di ferrite . . . 224
15.4 La memoria oggi . . . 225
15.5 La memoria centrale . . . 226
15.6 Processore e memoria . . . 228
15.7 Memoria e ingresso/uscita . . . 229
15.8 Il programma di questo capitolo . . . 231
15.9 Verificate di aver capito . . . 233
15.10 Esercizi . . . 234
15.11 Glossario . . . 234
15.12 Conclusioni e letture . . . 235

16 La memoria permanente **237**
16.1 Riscaldamento sulla macchina . . . 237
16.2 I componenti di un calcolatore . . . 238
16.3 Memorie non volatili . . . 239
16.4 La cache . . . 241
16.5 La memoria secondaria . . . 242
16.6 Dischi ottici . . . 243
16.7 La memorizzazione su più livelli . . . 244
16.8 Il programma di questo capitolo . . . 245
16.9 Verificate di aver capito . . . 248
16.10 Esercizi . . . 248
16.11 Glossario . . . 249
16.12 Conclusioni e letture . . . 249

17 Verm1, parte 3: un corpo, regole e stati **251**
17.1 Il progetto, prima del codice . . . 252
17.2 Passo 1: la memoria del gioco . . . 253
17.3 Passo 2: tre stati . . . 255
17.4 Passo 3: un verme in memoria . . . 258
17.5 Passo 4: le regole . . . 261
17.6 Passo 5: guardare la memoria che si muove . . . 266
17.7 Verificate di aver capito . . . 267
17.8 Fatelo vostro . . . 267
17.9 Due funzioni che non sono cambiate . . . 268

17.10Dove siamo, e che cosa viene dopo . . . 269

V La macchina 271

18 Il processore 273
18.1 Riscaldamento sulla macchina . . . 273
18.2 Il processore . . . 274
18.3 I registri . . . 277
18.4 Lo stack . . . 278
18.5 Modelli di processore . . . 279
18.6 La ALU e il quadro d'insieme . . . 280
18.7 Il programma di questo capitolo . . . 281
18.8 Verificate di aver capito . . . 283
18.9 Esercizi . . . 284
18.10Glossario . . . 285
18.11Conclusioni e letture . . . 285

19 Istruzioni e interruzioni 287
19.1 Riscaldamento sulla macchina . . . 287
19.2 Le istruzioni . . . 288
19.3 Il ciclo del processore . . . 290
19.4 Le interruzioni . . . 292
19.5 La soluzione di RISC-V . . . 294
19.6 Il programma di questo capitolo . . . 297
19.7 Verificate di aver capito . . . 300
19.8 Esercizi . . . 300
19.9 Glossario . . . 301
19.10Conclusioni e letture . . . 301

20 I bus e la scheda video 303
20.1 Riscaldamento sulla macchina . . . 303
20.2 Periferiche e controller . . . 304
20.3 I bus . . . 304
20.4 La scheda video . . . 305
20.5 GPU e GPGPU . . . 308
20.6 Il programma di questo capitolo . . . 309
20.7 Verificate di aver capito . . . 311
20.8 Esercizi . . . 311
20.9 Glossario . . . 312
20.10Conclusioni e letture . . . 312

21 Ingresso e uscita 313
21.1 Riscaldamento sulla macchina . . . 313
21.2 I display . . . 314
21.3 Tastiera e mouse . . . 316
21.4 Le stampanti . . . 317
21.5 Ritorno alla macchina . . . 318
21.6 Il programma di questo capitolo . . . 319
21.7 Verificate di aver capito . . . 321

21.8 Esercizi . . . 322
21.9 Glossario . . . 322
21.10 Conclusioni e letture . . . 322

22 Verm1, parte 4: il gioco finito 325
22.1 Passo 1: il punteggio in cifre . . . 326
22.2 Passo 2: i livelli . . . 328
22.3 Passo 3: `step` dice che cosa ha cambiato . . . 330
22.4 Passo 4: disegnare solo ciò che è cambiato . . . 334
22.5 Passo 5: misurare . . . 339
22.6 Verificate di aver capito . . . 339
22.7 Fatelo vostro . . . 340
22.8 Le funzioni che non sono cambiate . . . 340
22.9 Dove siamo, e che cosa viene dopo . . . 344

VI Pubblicazione e prospettive 345

23 Verm1 sul Cartridge Store 347
23.1 Perché pubblicare? . . . 347
23.2 Passo 1: un sorgente, due cartucce . . . 348
23.3 Passo 2: un'identità . . . 349
23.4 Passo 3: la cartuccia porta con sé i suoi documenti . . . 352
23.5 Passo 4: il bundle . . . 353
23.6 Passo 5: la pubblicazione . . . 353
23.7 Verificate di aver capito . . . 354
23.8 Il vostro rilascio . . . 355
23.9 Che cosa avete costruito . . . 355

24 Oltre un solo core: RISC-V superscalare, HPC, GPU e FPGA 357
24.1 Riscaldamento sulla macchina . . . 358
24.2 Processori superscalari . . . 358
24.3 Vettori: un'istruzione, molti dati . . . 361
24.4 Calcolo ad alte prestazioni . . . 362
24.5 Acceleratori GPU . . . 365
24.6 Acceleratori FPGA . . . 366
24.7 Ritorno alla macchina . . . 367
24.8 Il programma di questo capitolo . . . 368
24.9 Verificate di aver capito . . . 371
24.10 Esercizi . . . 371
24.11 Glossario . . . 372
24.12 Conclusioni e letture . . . 372

VII Appendici 373

A Riferimento di RISC-V 375
A.1 Il nome di un insieme di istruzioni . . . 375
A.2 I registri . . . 375
A.3 I formati delle istruzioni . . . 375

A.4 RV32I: l'insieme di istruzioni intero di base . . . 376
A.5 L'estensione M: moltiplicazione e divisione . . . 379
A.6 L'estensione A: istruzioni atomiche . . . 380
A.7 L'estensione C: istruzioni compresse . . . 380
A.8 Pseudoistruzioni . . . 381
A.9 L'assemblatore: direttive, etichette ed espressioni . . . 382
A.10 La convenzione di chiamata . . . 383
A.11 Privilegio, registri di controllo e stato, trap . . . 384
A.12 Codificare e decodificare a mano . . . 385
A.13 Idiomi . . . 386

B Riferimento di PRG32 **389**
B.1 La piattaforma in breve . . . 389
B.2 Il contratto . . . 390
B.3 Schermo, colori e ingresso . . . 391
B.4 L'interfaccia di programmazione . . . 392
B.5 Il file della cartuccia . . . 395
B.6 Gli strumenti a riga di comando . . . 396
B.7 Gli host . . . 398
B.8 Provare un programma sui tre host . . . 399
B.9 Il Cartridge Store . . . 401
B.10 Quando qualcosa non funziona . . . 402

C Riferimento di Git e GitHub **403**
C.1 Perché il controllo di versione . . . 403
C.2 Concetti . . . 403
C.3 Installare e configurare . . . 404
C.4 Ottenere un repository . . . 405
C.5 Il ciclo di ogni giorno . . . 405
C.6 Guardare la storia . . . 406
C.7 Annullare . . . 407
C.8 Branch . . . 407
C.9 Lavorare con GitHub . . . 408
C.10 Versioni, tag e release . . . 410
C.11 Questo libro e Overleaf . . . 411
C.12 Citare il software . . . 411
C.13 Riepilogo dei comandi . . . 411

D Verm1: i sorgenti completi **413**
D.1 Parte 1: una testa che attraversa il mondo . . . 413
D.2 Parte 2: la guida, il cibo e un punteggio . . . 415
D.3 Parte 3: un corpo, regole e stati . . . 419
D.4 Parte 4: il gioco finito . . . 427
D.5 I file del bundle dello Store . . . 437

E Risposte alle domande **441**

F Glossario inglese–italiano **447**

Prefazione

La maggior parte dei corsi di architettura dei calcolatori è costruita dal basso verso l'alto. Si comincia dal bit, si sale attraverso i sistemi di numerazione, l'algebra di Boole e le reti logiche, si assembla un processore a partire dalle porte, e solo nelle ultime settimane si lascia che lo studente scriva un programma per la macchina che i mesi precedenti hanno descritto. L'ordine è logicamente impeccabile, e per molti studenti del primo anno è anche la ragione per cui la materia sembra lontana: per dieci settimane ogni risposta arriva prima della sua domanda.

Questo libro è la forma scritta di un corso che rovescia l'ordine senza rinunciare in nulla al rigore. *Prima programmare, poi formalizzare*: nella prima settimana il lettore installa una piccola console, scrive una dozzina di righe di assembly RISC-V e vede un rettangolo muoversi su uno schermo. Da quel momento ogni argomento formale è introdotto da qualcosa che il programma del lettore ha già fatto. Il complemento a due arriva dopo che una posizione è uscita dal bordo sinistro dello schermo e il debugger ha mostrato `0xFFFFFFFF`. L'algebra di Boole arriva dopo che una maschera è servita a chiedere se un pulsante è premuto. Il flip-flop arriva dopo che un contatore è sopravvissuto da un frame al successivo, e il ciclo dell'istruzione dopo che il lettore ha premuto **Step** qualche centinaio di volte e ha visto il program counter avanzare di quattro.

Dal programma classico non viene tolto nulla. Il lettore troverà la notazione posizionale e i formati IEEE 754, assiomi e teoremi dell'algebra di Boole con le loro dimostrazioni, forme canoniche e mappe di Karnaugh, il transistor come interruttore, sommatori, multiplexer e decodificatori, latch e flip-flop, macchine a stati finiti, registri e bus, gerarchie di memoria e memorie permanenti, il datapath e l'unità di controllo, le interruzioni, la scheda video e le periferiche. Cambia la direzione del ragionamento: ogni capitolo si apre sulla macchina, la lascia per tutto il tempo che la teoria richiede, e vi ritorna con una sezione intitolata *Ritorno alla macchina*, in cui le stesse regole vengono rilette nelle istruzioni di un processore vero.

La macchina

Il processore è un core RISC-V: l'insieme di istruzioni RV32IMAC, così come lo realizza l'ESP32-C6 di Espressif. RISC-V è uno standard aperto, abbastanza piccolo da essere imparato per intero in un semestre e abbastanza reale da trovarsi in miliardi di dispositivi; sostituisce qui il Motorola 68000 delle edizioni precedenti del corso, che sopravvive solo in qualche confronto storico.

L'ambiente è PRG32, una piattaforma didattica aperta sviluppata all'Università degli Studi di Napoli «Parthenope». Un programma PRG32 è una *cartuccia*: un piccolo file di codice macchina nativo che fornisce tre funzioni, una chiamata una sola volta e due chiamate a

ogni frame. La stessa cartuccia gira su un kit su breadboard, sull'emulatore QEMU e su PRG32-QT, un'applicazione desktop il cui debugger integrato mostra i trentadue registri e la memoria mentre il programma viene eseguito un'istruzione alla volta. Per seguire il libro non serve una scheda: basta un portatile.

Il progetto

Una piattaforma usata solo per gli esercizi è una palestra. PRG32 è anche un'officina, e il libro ha un progetto che lo attraversa: Verm1, che si legge *vermone*, un omaggio a Snake scritto a partire da un file vuoto in assembly RISC-V. Dopo ogni blocco di quattro capitoli, un capitolo tutorial aggiunge una parte del gioco usando soltanto ciò che quei quattro capitoli hanno insegnato: prima una testa che attraversa un mondo senza bordi, poi la sterzata, il cibo e un punteggio, poi un corpo che cresce e le regole che chiudono una partita, poi cifre, livelli e suono. Un ultimo tutorial confeziona il gioco finito per il Cartridge Store di PRG32. Ogni tutorial stampa ogni riga del programma, dice quali righe sono nuove e quali sono cambiate, e si ferma a punti di controllo in cui il programma deve girare e comportarsi in un modo dichiarato.

Che cosa significa «verificato» in questo libro

Un listato che non si assembla insegna la lezione sbagliata. Il codice assembly di questo libro non è battuto dentro il testo: è letto da file che vengono assemblati, eseguiti sul QEMU ufficiale e sul runner senza interfaccia di PRG32-QT, e confrontati con i risultati attesi da test automatici; le quattro parti di Verm1 sono giocate da un giocatore programmato che scatta anche le schermate stampate nei capitoli tutorial. Dove un frammento non ha potuto essere eseguito sulla macchina dell'autore (i frammenti CUDA e Verilog dell'ultimo capitolo, i passi di installazione per sistemi operativi diversi da quello usato per scrivere il libro) il testo lo dice.

Ringraziamenti

Mariacarla Staffa tiene la metà di laboratorio del corso, in cui gli esercizi di questo libro si svolgono con matita, carta e tastiera. Ivan Cafiero e Simone Boscaglia hanno contribuito a PRG32 da studenti e hanno mostrato che la piattaforma può essere capita, ed estesa, dalle persone per cui è pensata. Giuseppe Coviello ha contribuito l'implementazione di riferimento di Verm1: i quattro programmi che i capitoli tutorial costruiscono riga per riga, e che i test automatici giocano, sono suoi. Gli studenti delle edizioni precedenti del corso hanno scritto, con le loro domande, la maggior parte dei paragrafi che cominciano con «Attenzione».

Raffaele Montella
Napoli, ottobre 2026

Come usare questo libro

Non si presuppone alcuna conoscenza. Chi non ha mai programmato, o non ha mai guardato dentro un computer, cominci dal capitolo *Partire da zero*, a pagina xix: poche pagine che spiegano le parole e le idee usate fin dalla prima lezione.

Un capitolo, una lezione

Il libro ha venticinque capitoli, numerati da 0 a 24 come le lezioni del corso, e come quasi tutto il resto in questo libro: bit, registri, righe e colonne partono tutti da zero. Il Capitolo N segue la Lezione N, che ha due presentazioni. La *presentazione della lezione*, di al più venti slide, contiene i concetti; la *presentazione del codice* contiene tutto ciò che si scrive e si esegue: ogni listato con la sua spiegazione, il programma completo della lezione, che cosa mostra, e il modo di provarlo su PRG32-QT, su QEMU e sulla scheda. Il capitolo è la forma estesa di entrambe: l'ordine dei paragrafi è l'ordine delle slide, i listati sono gli stessi file, le domande alla fine sono quelle mostrate in aula, e ciò per cui in venti slide non c'è spazio è qui. Chi ha seguito la lezione trova le frasi che una slide non può contenere; chi l'ha persa trova la lezione.

La maggior parte dei capitoli termina con un paragrafo, *Il programma di questo capitolo*, che stampa per intero il programma da cui sono tratti i listati del capitolo, mostra il frame che esso produce e dà i comandi per eseguirlo. I programmi sono nella cartella `code` del repository del corso, uno per lezione, e ciascuno è compilato ed eseguito dai test automatici; il Paragrafo B.8 è la guida passo per passo per i tre host.

Parte	**Capitoli**	**Poi, il progetto**
Partenza	0 Introduzione, 1 Installazione, 2 Primo contatto	
Informazione	3 Storia, 4 Informazione digitale, 5 Interi, 6 Reali	7 Verm1, parte 1
Logica	8–9 Algebra di Boole, 10 Elettronica, 11 Macchine combinatorie	12 Verm1, parte 2
Stato e memoria	13 Macchine sequenziali, 14 Registri e bus, 15 Memoria, 16 Memoria permanente	17 Verm1, parte 3
La macchina	18 Processore, 19 Istruzioni e interruzioni, 20 Scheda video, 21 Ingresso e uscita	22 Verm1, parte 4
Pubblicazione, prospettive	24 Oltre un solo core	23 Verm1 sul Cartridge Store

La forma di un capitolo di teoria

Ogni capitolo che presenta un argomento ha gli stessi cinque movimenti.

1. **Riscaldamento sulla macchina.** Poche righe di RISC-V su PRG32 che producono un fatto che chiede una spiegazione. Eseguitele prima di proseguire.
2. **La trattazione formale.** Definizioni, teoremi, metodi ed esempi svolti.
3. **Ritorno alla macchina.** Le stesse regole, come le realizza RISC-V.
4. **Verificate di aver capito.** Domande a risposta multipla nello stile della prova intercorso. Le risposte, ciascuna con una riga di spiegazione, sono nell'Appendice E: leggetele dopo aver risposto, non al posto di rispondere.
5. **Esercizi, glossario, letture.** Gli esercizi si svolgono prima su carta e poi su PRG32; il glossario affianca a ogni termine inglese quello italiano, perché la documentazione tecnica, i messaggi degli strumenti e i commenti dei listati sono in inglese.

La forma di un capitolo tutorial

I Capitoli 7, 12, 17, 22 e 23 costruiscono Verm1. Sono autonomi e pensati per essere seguiti alla tastiera. Ogni passo mostra poche righe, dice se sono **nuove**, **modificate** o **invariate**, e finisce con un punto di controllo *Eseguite* che dichiara che cosa deve mostrare lo schermo. Se all'inizio di un tutorial un programma si rifiuta di funzionare, ripartite dal file di riferimento della parte precedente (Appendice D): nessuno deve restare indietro per un errore.

I riquadri

Concetto

Un riquadro **Concetto** enuncia un'idea da cui dipende il resto del capitolo.

Definizione 0.1 (come appaiono le definizioni)

Definizioni formali e teoremi sono numerati all'interno del capitolo.

Provate voi

Un riquadro **Provate voi** chiede di fare qualcosa alla tastiera, e dice che cosa si deve osservare.

Attenzione

Un riquadro **Attenzione** segnala un errore che si ripete ogni anno.

Ritorno alla macchina

Un riquadro **Ritorno alla macchina** lega un risultato formale a un'istruzione, a un registro o a una schermata del debugger.

Nota storica

Un riquadro **Nota storica** racconta da dove viene un'idea.

Stato di apprendimento: comprendere oggi, utilizzare oggi, comprendere più avanti

Il corso prima programma e poi formalizza, quindi una lezione mostra sempre più di quanto spieghi. Per questo ogni capitolo si apre, dopo gli obiettivi, con un riquadro che divide ciò che contiene in tre stati, e le slide portano le stesse etichette sopra il titolo.

DA COMPRENDERE OGGI
: Alla fine della lezione sapete spiegarlo, seguirlo a mano passo passo e applicarlo da soli. Le domande e gli esercizi verificano questi.

DA UTILIZZARE OGGI
: Oggi seguite e usate la ricetta: una direttiva, un comando, una chiamata del runtime, una convenzione sui registri. Come funziona dentro lo spiega il capitolo indicato tra parentesi, oppure non è oggetto del corso (gli strumenti di PRG32 si usano, non si studiano).

DA COMPRENDERE PIÙ AVANTI
: Un'anticipazione: riconoscete il nome e lo scopo. Lo spiega il capitolo indicato tra parentesi; nulla è rinviato senza una destinazione.

Gli stati sono attese nel tempo, non livelli di difficoltà: la stessa idea passa spesso da *più avanti* a *utilizzare* a *comprendere* nel giro di qualche capitolo (la convenzione di chiamata si usa dal Capitolo 2 e si studia nel Capitolo 18). Una slide o un paragrafo senza etichetta è contesto: una storia, un esempio, una domanda, un esercizio. Il colore non è mai l'unico indizio: il libro, la chiave inglese e l'orologio, e le cornici continua, doppia e tratteggiata, distinguono i tre stati anche in una stampa in bianco e nero.

Notazione

I numeri senza indicazioni sono decimali, con la virgola decimale e, dove serve, il punto per le migliaia. I numeri esadecimali sono scritti come li vuole l'assemblatore, `0x07E0`; i numeri binari come `0b1010` nel codice e con un pedice, 1010_2, nelle formule. I bit si numerano da destra partendo da zero, così il bit i ha peso 2^i. Registri (`a0`, `sp`), istruzioni (`addi`) e nomi di file sono composti in carattere a spaziatura fissa. Nelle espressioni booleane il complemento di x è $\overline{x}$, la somma è $+$ e il prodotto si scrive per giustapposizione. I multipli del byte seguono i prefissi binari dove la precisione conta: 1 KiB $= 2^{10}$ byte, 1 MiB $= 2^{20}$ byte.

I listati e i loro commenti sono in inglese, come gli identificatori di PRG32, i comandi e le etichette dell'interfaccia di PRG32-QT (**Step**, **Resume**, **Memory**): sono ciò che compare davvero sullo schermo.

Le appendici

L'Appendice A è un riferimento completo dell'insieme di istruzioni RISC-V usato nel libro: registri, formati, ogni istruzione di RV32I e delle estensioni M, A, C e Zicsr, pseudo-istruzioni, direttive dell'assemblatore, la convenzione di chiamata e i registri di controllo del modo macchina. L'Appendice B è il riferimento di PRG32: il contratto, l'interfaccia di programmazione, il formato delle cartucce, gli strumenti a riga di comando, gli host e il Cartridge Store. L'Appendice C è un riferimento di Git e GitHub, gli strumenti con cui la piattaforma, questo libro e il progetto del lettore sono conservati e condivisi. L'Appendice D stampa per intero i quattro programmi di riferimento di Verm1; l'Appendice E le risposte a tutte le domande; l'Appendice F il glossario completo inglese–italiano.

Partire da zero

Questo corso non ha prerequisiti. La maggior parte dei suoi studenti non ha mai scritto un programma, e a nessuno si chiede di sapere come è fatto un computer. Questo capitolo è per loro. Spiega, con parole semplici, le poche idee e le poche decine di parole che le prime lezioni usano fin dal primo giorno; tutto ciò che contiene viene insegnato di nuovo, per bene, nel suo capitolo. Leggetelo prima della Lezione 01, e tornateci ogni volta che una parola di una slide non vi è familiare: il piccolo dizionario alla fine dice dove ciascuna è spiegata per intero.

Che cosa fa un computer

Un computer ha tre tipi di parti. Il *processore* fa il lavoro. La *memoria* conserva il programma e i dati finché il computer è acceso. I dispositivi di *ingresso e uscita* lo collegano al mondo: una tastiera, un joystick, uno schermo, un altoparlante, una rete.

Il processore fa una sola cosa, velocissimo: esegue *istruzioni*. Un'istruzione è un ordine piccolo ed esatto, come "somma questi due numeri", "copia questo valore dalla memoria" o "se questo numero è zero, prosegui da là". Ogni istruzione fa pochissimo; un processore ne esegue centinaia di milioni al secondo, e tutto ciò che fa un computer, da un gioco a una videochiamata, è una lunga sequenza di questi piccoli passi. Un *programma* è l'elenco di istruzioni che produce un certo comportamento. Il Capitolo 3 racconta come questa idea, un programma conservato in memoria come qualsiasi altro dato, sia nata nel 1945, e perché descriva ancora il computer sulla vostra scrivania.

Bit, byte e i numeri sulla pagina

Dentro un computer tutto, numeri, lettere, colori, istruzioni, è conservato come *bit*: cifre che possono valere solo 0 o 1. Otto bit formano un *byte*; trentadue bit, quattro byte, formano una *parola*, la dimensione dei numeri che il processore di questo corso tratta in un passo.

Un numero scritto con i bit è in *binario*, base 2: ogni posizione vale il doppio di quella alla sua destra, quindi $1010_2 = 8 + 2 = 10$. Il binario è esatto ma lungo, così i programmatori raggruppano i bit quattro alla volta e scrivono ogni gruppo con una cifra *esadecimale*, base 16: le cifre da 0 a 9 e poi da A a F per i valori da dieci a quindici. Il prefisso `0x` indica un numero esadecimale.

Decimale	Binario	Esadecimale
0	`0000`	`0x0`
1	`0001`	`0x1`
9	`1001`	`0x9`
10	`1010`	`0xA`
15	`1111`	`0xF`
16	`1 0000`	`0x10`
255	`1111 1111`	`0xFF`

Poiché una cifra esadecimale corrisponde esattamente a quattro bit, il colore verde di PRG32, `0x07E0`, è `0000 0111 1110 0000` in binario. Per le prime settimane basta riconoscere queste notazioni; il Capitolo 4 le spiega, con le conversioni nei due sensi.

Dal testo al codice macchina

Il processore legge le istruzioni come numeri. Questa forma numerica di un programma si chiama *codice macchina*. Nessuno la scrive a mano: le persone scrivono *codice sorgente*, un file di testo in un *linguaggio di programmazione*, e un traduttore lo trasforma in codice macchina. Questo corso usa l'*assembly*, il linguaggio più vicino alla macchina, in cui una riga diventa un'istruzione; il traduttore dell'assembly è l'*assemblatore*. Per il C, un linguaggio che anche gli esempi di PRG32 usano, il traduttore è un *compilatore*. L'insieme di questi traduttori e dei loro aiutanti è la *toolchain*, che la Lezione 01 installa. Ecco un'istruzione nelle sue due forme:

Voi scrivete (codice sorgente)	Il processore legge (codice macchina)
`addi a0, a0, 1`	`0x00150513`

Su PRG32 il risultato della traduzione è una *cartuccia*: un piccolo file, che termina in `.prg32`, con il codice macchina di un gioco, i suoi dati e una breve intestazione, come le cartucce delle console degli anni Ottanta.

Chi esegue il vostro programma

Un gioco non gira mai su una macchina vuota. Sul kit di PRG32 un altro programma, il *firmware*, parte quando la scheda viene accesa, resta in memoria e gestisce lo schermo, i pulsanti e il suono. È conservato nella memoria *flash*, che, come una chiavetta USB, mantiene il contenuto anche senza alimentazione. La parte del firmware con cui il gioco dialoga è il *runtime*: chiama il gioco una volta per ogni *frame*, ogni nuova immagine sullo schermo, e gli presta funzioni già pronte per disegnare e per suonare.

Ogni macchina che offre questo runtime è un *host*. Ce ne sono tre: la scheda; QEMU, un *emulatore*, un programma che imita un processore su un portatile; e PRG32-QT, un'applicazione il cui *interprete* legge ogni istruzione della cartuccia e la esegue via software. La stessa cartuccia gira su tutti e tre. PRG32-QT contiene anche il *debugger* del corso: uno strumento che mette in pausa un programma, lo esegue un'istruzione alla volta e mostra che cosa c'è dentro la macchina a ogni passo.

Il terminale, in dieci minuti

Installare e compilare richiedono il *terminale*: una finestra in cui i comandi si scrivono invece di fare clic. Il terminale mostra un *prompt*, una breve riga che termina con `$` o `>`, e attende. Si scrive un comando e si preme Invio; il computer lo esegue e stampa sotto la sua risposta. In ogni momento il terminale "si trova" in una cartella, la *cartella corrente*, e i comandi agiscono sui file che contiene. Un *percorso* nomina un file attraverso le cartelle che vi conducono, come `~/PRG32/examples/pong.S`, dove `~` è la cartella personale.

Per	**macOS e Linux**	**Windows PowerShell**
vedere in quale cartella siete	`pwd`	`pwd`
elencare i file della cartella	`ls`	`dir`
entrare nella cartella PRG32	`cd PRG32`	`cd PRG32`
risalire di una cartella	`cd ..`	`cd ..`
tornare alla cartella personale	`cd ~`	`cd ~`
fermare il programma in esecuzione	Ctrl+C	Ctrl+C

Il tasto Tab completa un nome di cui avete scritto l'inizio, e la freccia in su richiama il comando precedente. Un *repository* è una cartella di codice sorgente di cui un programma chiamato `git` conserva la storia; `git clone` ne copia uno da Internet al portatile.

Un comando che funziona spesso stampa poco o nulla. Un comando che fallisce stampa un messaggio, e il messaggio quasi sempre nomina la causa: leggetelo tutto, dalla prima riga che dice *error*, prima di cambiare qualsiasi cosa. Quattro messaggi coprono la maggior parte dei problemi della prima settimana. *command not found*: il programma non è installato, o non è attivo in questo terminale. *No such file or directory*: cartella o nome sbagliati, come rivelano `pwd` e `ls`. *Permission denied*: il comando richiede permessi che qui non avete. *Connection refused*: il programma a cui vi rivolgete, come PRG32-QT, non è in esecuzione. Quando chiedete aiuto, copiate il comando e il messaggio intero come testo.

Leggere una riga di assembly

Una riga di assembly ha al più quattro parti:

```
1 first_update:   addi t1, t1, 1      # one more tick
```

`first_update:` è un'*etichetta*, un nome per questo punto del programma, che termina con i due punti; spesso un'etichetta segna l'inizio di una *funzione*, un pezzo di programma con un nome che svolge un compito. `addi` è l'*istruzione*, qui "somma una costante". `t1, t1, 1` sono gli *operandi*: prima la destinazione, poi le sorgenti, quindi la riga si legge "metti in `t1` il valore di `t1` più 1". Tutto ciò che segue `#` è un *commento*, scritto per le persone e ignorato dall'assemblatore. Una riga che comincia con un punto, come `.section .text`, è una *direttiva*: un ordine all'assemblatore, non al processore.

Registri, memoria, indirizzi

Il processore calcola solo sui suoi *registri*: trentadue piccole celle di memorizzazione al suo interno, ciascuna con una parola. La memoria è molto più grande: una lunga fila di celle, un byte ciascuna, numerate a partire da zero; il numero di una cella è il suo *indirizzo*. Per

lavorare su un valore conservato in memoria, un programma prima lo *carica* in un registro (`lw`), poi ci calcola sopra, e infine lo *memorizza* di nuovo (`sw`). La differenza tra un indirizzo, dove un valore abita, e il valore stesso è l'idea più importante delle prime settimane; il Capitolo 2 la mostra con due frecce.

Per usare una funzione, un programma la *chiama*: `call` salta alla prima istruzione della funzione e annota nel registro `ra` dove tornare; `ret` torna lì. I valori di cui la funzione ha bisogno, i suoi *argomenti*, viaggiano nei registri da `a0` ad `a7`.

Un piccolo dizionario per le prime settimane

ABI l'accordo su come un programma e il runtime si chiamano a vicenda; Capitoli 1 e 18.

Bit, byte, parola una cifra binaria; otto bit; trentadue bit; Capitolo 4.

Breakpoint un segno su un'istruzione in cui il debugger ferma il programma; Capitolo 1.

Clock il segnale che scandisce il processore; 160 MHz significa 160 milioni di passi al secondo; Capitolo 13.

Frame un'immagine sullo schermo; un gioco ne disegna trenta o più al secondo; Capitolo 2.

Insieme di istruzioni (ISA) l'elenco delle istruzioni che un processore capisce; Capitoli 3 e 19.

Pixel un punto colorato dello schermo; Capitolo 4.

Stack una zona di memoria in cui le funzioni tengono valori per un po'; Capitoli 2 e 18.

Complemento a due il modo in cui i computer conservano gli interi negativi; Capitolo 5.

L'Appendice F elenca ogni termine del corso, in inglese e in italiano.

Come studiare quando si parte da zero

Tre abitudini contano più di qualsiasi conoscenza precedente. *Prevedere prima di eseguire*: prima di eseguire un programma, scrivete che cosa vi aspettate di vedere; una previsione sbagliata è la cosa più utile che possa capitare, perché mostra esattamente che cosa imparare dopo. *Leggere tutto il messaggio*: gli errori sono informazioni, non fallimenti, e quasi ognuno nomina la propria causa. *Chiedere presto*: una domanda fatta nelle prime settimane costa minuti, la stessa domanda a dicembre costa giorni. Le domande alla fine di ogni capitolo richiedono pochi minuti a settimana; sono la migliore preparazione all'esame, e il modo migliore per scoprire, presto, che cosa non è ancora chiaro.

Parte I

Partenza

CAPITOLO 0

Introduzione: prima programmare, poi formalizzare

"Matrix è un sistema, Neo. E quel sistema è nostro nemico."
— *Matrix*, Lana e Lilly Wachowski, 1999

In questo capitolo

- Che cos'è il corso, chi lo tiene e come sono usate le sue novantasei ore.
- Il metodo: prima viene un programma che gira, poi la sua teoria.
- La macchina: PRG32, RISC-V e il debugger PRG32-QT.
- Il progetto: Verm1, un gioco scritto da un file vuoto e pubblicato.
- Le regole: programma, risultati di apprendimento, valutazione, materiale.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI il metodo: prima il programma; le regole del corso.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI i componenti di PRG32 (Capitolo 1); RISC e un ISA aperto (Capitolo 3); il progetto Verm1 (Capitolo 7).

Un calcolatore è l'oggetto più complicato che la maggior parte delle persone usa ogni giorno, e quello di cui meno sono incoraggiate a guardare il funzionamento interno. I suoi costruttori hanno passato settant'anni a nascondere la macchina dietro strati ciascuno più facile da usare di quello sottostante: le porte logiche dietro le istruzioni, le istruzioni dietro i linguaggi, i linguaggi dietro le librerie, le librerie dietro le applicazioni, e ora le applicazioni dietro gli assistenti conversazionali. Sono gli strati a rendere utile l'informatica. Sono anche un sistema nel senso del film citato qui sopra: ci si può vivere dentro per una vita senza sapere che sotto c'è qualcosa.

L'architettura dei calcolatori è la disciplina che guarda sotto. Studia come una macchina capace soltanto di accendere e spegnere correnti arrivi a rappresentare numeri, testo, immagini e suono; come una manciata di operazioni logiche, ripetute miliardi di volte al secondo, diventino aritmetica, decisioni e memoria; e come il tutto sia organizzato in modo che una

sequenza di ordini semplicissimi, un *programma*, possa farle fare qualunque cosa sia calcolabile. Questo capitolo spiega come il corso affronta questo studio, con quale macchina e attraverso quale progetto, e poi enuncia le regole pratiche.

0.1 Il corso in breve

Architettura dei Calcolatori e Laboratorio di Architettura dei Calcolatori, codice ARC12, è un insegnamento del primo anno, primo semestre, del Corso di Laurea in Informatica dell'Università degli Studi di Napoli «Parthenope». Vale 12 CFU, appartiene al settore scientifico-disciplinare INFO-01/A (Informatica) e consiste di 96 ore, divise in parti uguali tra lezioni e laboratorio (Tabella 1).

Tabella 1. Il corso in breve.

Nome	Architettura dei Calcolatori e Laboratorio di Architettura dei Calcolatori (ARC12)
Collocazione	Primo anno, primo semestre, Corso di Laurea in Informatica
Crediti	12 CFU, 96 ore; settore INFO-01/A (Informatica)
Lezioni	Raffaele Montella, 6 CFU, 48 ore
Laboratorio	Mariacarla Staffa, 6 CFU, 48 ore
Ricevimento	Stanza 420, martedì 14:00–16:00, su appuntamento; anche su Microsoft Teams (codice del team `c7rf33g`)
Crediti aggiuntivi	Accenture SuperSkills! dà 2 CFU come ulteriori conoscenze (https://richiediaccesso.uniparthenope.it)
Prerequisiti	Nessuno

Non ci sono prerequisiti formali. Non si presuppone alcuna esperienza precedente di programmazione, di elettronica o di matematica oltre il programma scolastico: l'unico vero prerequisito è la passione per l'informatica e per le tecnologie che la rendono possibile.

Le due metà del corso sono due viste della stessa materia. Nelle **lezioni** si presentano le idee: informazione, numeri, algebra di Boole, reti logiche, memoria, processore, ingresso e uscita. Ogni argomento si apre con un programma che si può eseguire e viene poi formalizzato con definizioni, metodi e dimostrazioni; ogni lezione si chiude con domande a risposta multipla nello stile della prova intercorso; e ogni quattro lezioni un tutorial aggiunge la parte successiva di Verm1, il gioco del corso. Nel **laboratorio** le stesse idee vengono esercitate: rappresentazione dei numeri e tecniche di conversione; analisi, sintesi e minimizzazione di circuiti logici; programmazione in assembly RISC-V sulla piattaforma PRG32; e, infine, il progetto dello studente, una variante di Verm1 o un gioco di sua invenzione. Questo libro è il testo delle lezioni; i suoi esercizi sono il materiale naturale del laboratorio.

0.2 Un approccio nuovo

Prima programmare. Poi formalizzare.

Fin dai primi giorni lo studente scrive assembly RISC-V che gira su un processore vero. Ogni argomento formale risponde poi a una domanda che il programma dello studente ha già sollevato.

L'ordine tradizionale della materia è deduttivo: la rappresentazione, poi la logica, poi i circuiti, poi il processore, poi, finalmente, un programma. È l'ordine in cui la macchina viene *costruita*, e ha l'eleganza di ogni costruzione dai primi principi. Non è l'ordine in cui la macchina viene *capita*. Uno studente che incontra il complemento a due nella terza settimana di un corso deduttivo non ha mai visto un numero negativo dentro un calcolatore, e non ha alcuna ragione di interessarsi a come si scrive. Uno studente che, la settimana prima, ha spostato un rettangolo verso sinistra finché la sua coordinata, mostrata dal debugger, è passata da `0x00000000` a `0xFFFFFFFF` ha una domanda, e il complemento a due è la risposta.

Il corso è quindi organizzato come una spirale, ripetuta per ogni argomento (Figura 1).

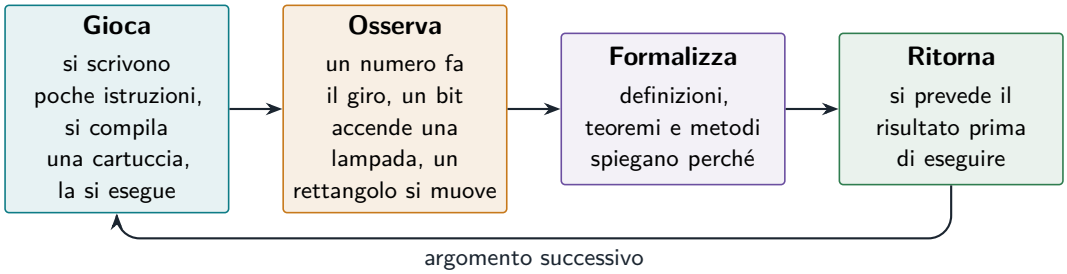


Figura 1. La spirale di apprendimento, ripetuta per ogni argomento del corso.

Prima si *gioca*: si scrivono poche istruzioni, si compila e si esegue una cartuccia. Poi si *osserva*: succede qualcosa che non ci si aspettava, oppure che ci si aspettava e che ora si può vedere. Poi si *formalizza*: definizioni e teoremi spiegano esattamente perché. Infine si *ritorna* alla macchina, e la misura della comprensione è la capacità di prevedere un risultato prima di eseguire il programma. Un voto è un'opinione; una racchetta che esce dallo schermo è un fatto. Il corso usa i fatti, e poi li spiega.

0.2.1 Gli obiettivi del corso

Gli obiettivi sono quelli di ogni corso serio sulla materia; cambia solo il percorso. Il corso

- presenta gli aspetti fondamentali dell'architettura dei calcolatori moderni;
- usa, in laboratorio, i concetti di base necessari a programmarli in modo efficiente con linguaggi di basso livello: linguaggio macchina e assembly;
- tratta con rigore gli strumenti con cui si costruisce un processore, cioè l'algebra di Boole con la sintesi di reti logiche combinatorie e sequenziali, e la rappresentazione binaria con l'aritmetica binaria;
- descrive in dettaglio la struttura funzionale e fisica della CPU, della memoria e del sottosistema di ingresso/uscita;
- sviluppa codice assembly che rende visibile come lavora davvero un processore e come interagiscono le unità fondamentali di un calcolatore.

0.3 La macchina: PRG32 e RISC-V

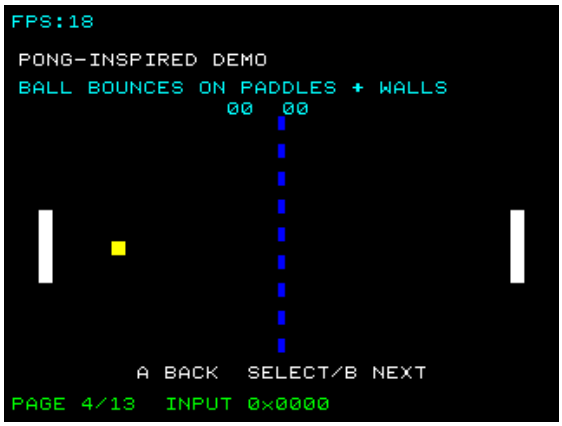
0.3.1 Una console rétro programmata in assembly

PRG32 è un runtime didattico open source per insegnare l'assembly RISC-V e il C, sviluppato all'Università degli Studi di Napoli «Parthenope» (Montella et al., 2026a). All'occhio è

una piccola console per videogiochi del tipo che si vendeva nei primi anni Ottanta (Figura 2); per il programmatore è un contratto volutamente sottile.



(a) Il menu del firmware residente.



(b) Pong: codice macchina RISC-V nativo.

Figura 2. PRG32, la macchina del corso.

- Il codice dello studente gira in modo *nativo*, come codice macchina RISC-V a 32 bit dell'insieme di istruzioni RV32IMAC. Non è interpretato da un simulatore didattico: le istruzioni scritte sono le istruzioni eseguite.
- Il contratto consiste di tre funzioni scritte dallo studente: `init` gira una volta; `update` e `draw` girano una volta per frame.
- I giochi viaggiano come piccole *cartucce*, file con estensione `.prg32`.
- La macchina offre un viewport di 320 per 200 pixel, un joystick con pochi pulsanti, e il suono: abbastanza piccola da tenerla tutta in mente, abbastanza ricca da mettere alla prova l'intero corso.

0.3.2 La filosofia di PRG32

Sei principi, presi dal libro di testo della piattaforma (Montella et al., 2026b), danno forma sia a PRG32 sia al modo in cui questo libro lo usa.

Abbastanza piccola da capirla.
Un processore, uno schermo di 320 per 200 pixel, un joystick, un piccolo altoparlante. Nulla è nascosto: l'intera macchina si può tenere in mente.

Vera, non simulata.
Le istruzioni scritte sono quelle che il chip esegue. Ciò che si impara qui è vero su ogni processore RISC-V, da un sensore a un supercalcolatore.

I limiti rendono creativi.
Pochi kilobyte e tre funzioni. I giochi degli anni Ottanta sono nati da limiti come questi: un vincolo è un invito.

Una cosa alla volta.
Si comincia con funzioni che si limitano a ritornare. Si aggiunge una cosa, si esegue, si guarda. `update` cambia lo stato, `draw` lo mostra. Il suono viene per ultimo.

Guardare dentro.
Quando qualcosa sorprende, si ferma il programma e si leggono registri e memoria. Il debugger fa parte del metodo, non è uno strumento d'emergenza.

Fare, poi condividere.
Una cartuccia è un file che si può dare a un amico. Il Cartridge Store è lo scaffale su cui il lavoro finito incontra i suoi giocatori.

La piattaforma ha quindi due usi nel corso. È una *palestra*: ogni capitolo si apre con un breve programma che gira e si chiude tornando alla macchina; un esercizio consiste di una previsione e di una verifica; e la macchina non lusinga mai, perché una maschera sbagliata accende la lampada sbagliata. Ed è un'*officina*: una cartuccia vuota e uno schermo, su cui regole, colori, suoni e livelli sono decisioni di progetto che spettano all'autore. Il progetto del corso comincia guidato e finisce libero, e ciò che è finito si può pubblicare. Gli studenti del primo anno di rado si aspettano che un corso di architettura chieda loro un'idea; questo lo fa.

0.3.3 PRG32-QT: il debugger del corso

PRG32-QT è PRG32 su un portatile: un'applicazione che esegue le stesse cartucce senza scheda e senza compilatore (Montella, 2026). In modalità debug apre la macchina (Figura 3): il programma può essere messo in pausa, ripreso e fatto avanzare di un'istruzione alla volta; si possono mettere breakpoint su qualunque istruzione di `init`, `update` o `draw`; vengono mostrati i trentadue registri e 128 byte di memoria a qualunque indirizzo; la velocità si può regolare tra 0,01 e 4 volte quella reale; e dei contatori riportano i frame, le istruzioni eseguite e i frame arrivati in ritardo.

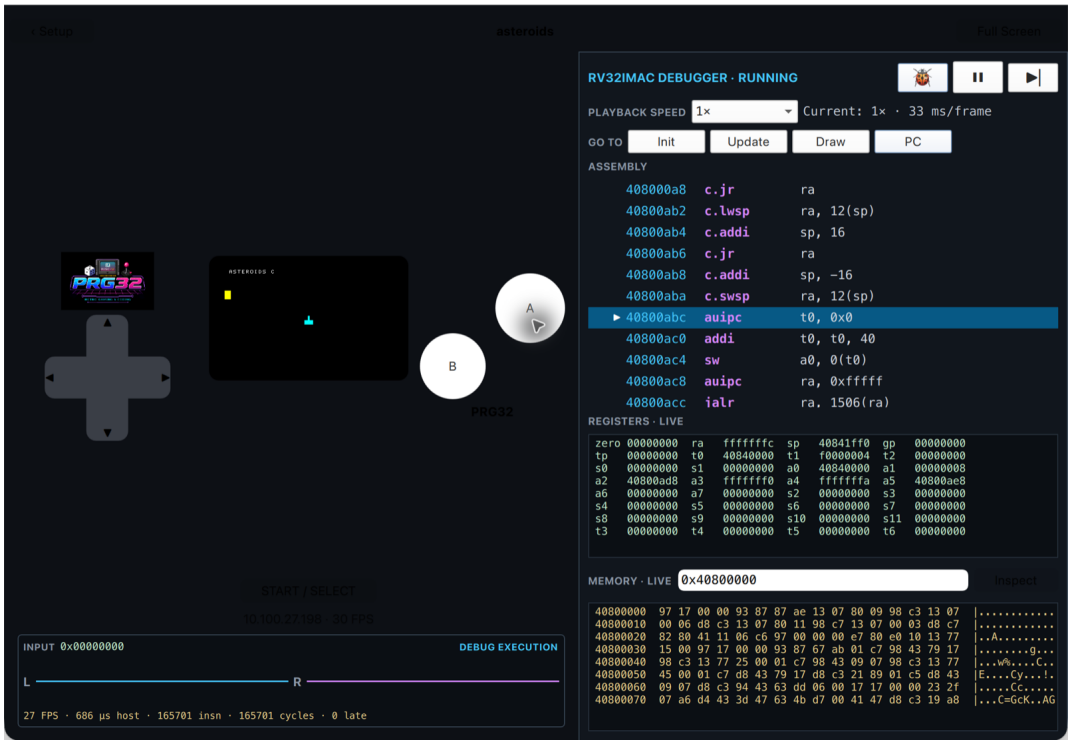


Figura 3. PRG32-QT in modalità debug: a sinistra il gioco, a destra la macchina.

Il debugger si usa in ogni capitolo: per verificare una previsione, per trovare un errore, per misurare. Fornisce anche la definizione operativa di comprensione adottata in questo libro.

Quando si è capito un programma?

Un programma è capito quando si sa dire che cosa conterrà ciascun registro *prima* di premere **Step**.

0.3.4 Perché RISC-V?

La scelta del processore non è neutra, e quattro ragioni hanno portato a RISC-V.

È aperto.
RISC-V è un insieme di istruzioni a standard aperto: chiunque può progettare, costruire e vendere un processore RISC-V senza pagare una licenza per l'architettura (RISC-V International, 2024c).

È piccolo.
L'insieme di istruzioni intero di base ha una quarantina di istruzioni e trentadue registri. Lo si può imparare tutto in un semestre, e l'Appendice A lo elenca tutto.

È reale.
`addi`, `lw`, `sw` e `beqz` non sono un dialetto didattico. L'industria consegna core RISC-V a miliardi, ed essi eseguono esattamente queste istruzioni.

È europeo.
L'Europa considera RISC-V una leva per la sovranità tecnologica: lo European Chips Act e progetti come DARE (Digital Autonomy with RISC-V in Europe, un progetto della EuroHPC Joint Undertaking) costruiscono processori su di esso (EuroHPC Joint Undertaking, 2025).

0.3.5 La mappa: ogni argomento si incontra due volte

La Tabella 2 è il corso in una pagina. Ogni argomento formale della colonna di sinistra si incontra prima sulla macchina, nella forma data dalla colonna di destra, e solo dopo nella sua forma generale. Vale la pena di tornare a questa tabella all'inizio di ogni parte del libro.

Tabella 2. Ogni argomento formale si incontra prima su PRG32.

Argomento formale	**Incontrato prima su PRG32 come…**
Notazione posizionale, esadecimale	il colore `0x07E0` e il valore mostrato dal debugger
Complemento a due, overflow	una posizione che va a sinistra oltre lo zero e mostra `0xFFFFFFFF`
Virgola fissa e virgola mobile	una palla che si muove di 0,375 pixel per frame su un chip senza unità in virgola mobile
Algebra di Boole, maschere di bit	`andi t3, t0, 1` per verificare se **LEFT** è premuto
Elettronica, porte logiche	il pulsante che collega un piedino del chip a massa
Reti combinatorie, sommatori	che cosa fa davvero `add` a trentadue bit
Reti sequenziali, registri	il contatore che sopravvive da un frame al successivo
CPU, datapath, ciclo dell'istruzione	l'esecuzione passo-passo di un'istruzione in PRG32-QT
Memoria e ingresso/uscita	`lw` e `sw`, lo stack, il joystick e il display

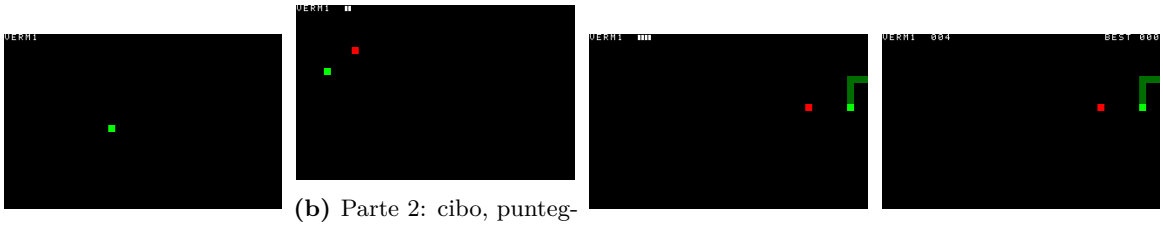
0.4 Il progetto: Verm1

Verm1, che si legge *vermone*, è un omaggio a Snake: il gioco del cabinato *Blockade* (1976) e dei telefoni Nokia (1997). Un verme si muove su una griglia, mangia, cresce, e non deve mordersi. Lo studente ne scrive ogni riga in assembly RISC-V, senza motore e senza librerie oltre alle chiamate di PRG32. Il gioco è abbastanza piccolo per uno studente del primo anno e abbastanza ricco da richiedere ogni argomento del corso; alla fine è pronto per il Cartridge Store con sopra il nome del suo autore. L'implementazione di riferimento, con cui ogni lettore può confrontare il proprio programma, è un contributo di Giuseppe Coviello (Appendice D).

Il progetto è il filo del libro. Si costruisce in quattro parti e una pubblicazione (Tabella 3, Figura 4), e ogni parte usa esattamente ciò che i quattro capitoli precedenti hanno insegnato. Ogni parte finisce con un gioco che gira: il lettore non torna mai a casa con un programma rotto.

Tabella 3. Verm1: quattro parti e una pubblicazione.

Capitolo	Dopo la parte su	Verm1 acquista	Con
7 Parte 1	informazione e numeri	una griglia, una testa che si muove, un mondo senza bordi	colori, scorrimenti, virgola fissa
12 Parte 2	logica e circuiti	la sterzata, il cibo, un punteggio	maschere, XOR, tabelle
17 Parte 3	stato e memoria	un corpo che cresce, le collisioni, tre schermate	una macchina a stati, vettori
22 Parte 4	la macchina	cifre, livelli, suono, disegno veloce	chiamate, divisione, il frame buffer
23 Lo Store	l'intero corso	un nome, un'icona, i crediti, un bundle	gli strumenti di PRG32



(a) Parte 1: una testa **(b)** Parte 2: cibo, punteggio **(c)** Parte 3: un corpo **(d)** Parte 4: finito

Figura 4. Verm1 cresce con il lettore: lo stesso file, quattro sedute. Le schermate sono prese dai programmi di riferimento dai test automatici del materiale del corso.

Un capitolo tutorial obbedisce a quattro regole. È *autonomo*: mostra l'intero programma, e ogni riga è nuova, modificata, oppure ristampata invariata. Procede *passo per passo*: poche righe alla volta, e dopo ogni passo il programma viene compilato, eseguito e confrontato con un risultato dichiarato. Usa *il debugger*: ogni tutorial ha una sessione in PRG32-QT, con un breakpoint, un registro da osservare e una regione di memoria da leggere. E il suo risultato appartiene al lettore: ogni tutorial finisce con modifiche da fare in autonomia, perché il gioco che viene pubblicato non dovrebbe essere identico a quello stampato qui.

0.5 Il corso in blocchi

L'ordine degli argomenti è stato scelto per l'apprendimento. Prima viene che cosa significa un bit; poi come i bit si combinano; poi come si ricordano; poi la macchina che fa tutte e tre le cose. Il processore è presentato prima dei dispositivi che pilota. Dopo ogni blocco il progetto lo usa. L'ultima lezione è uno sguardo in avanti, tenuta prima di una visita ai sistemi di calcolo del Dipartimento (Tabella 4).

Tabella 4. I blocchi del corso. I numeri sono quelli delle lezioni e dei capitoli di questo libro.

Blocco	Lezioni	Poi
Partenza	0 Introduzione; 1 PRG32 e PRG32-QT: installazione; 2 Primo contatto con RISC-V	
Informazione	3 Storia; 4 Informazione digitale; 5 Interi; 6 Reali	7 Verm1, parte 1
Logica	8 e 9 Algebra di Boole; 10 Elettronica; 11 Macchine combinatorie	12 Verm1, parte 2
Stato e memoria	13 Macchine sequenziali; 14 Registri e bus; 15 Memoria; 16 Memoria permanente	17 Verm1, parte 3
La macchina	18 Processore; 19 Istruzioni e interruzioni; 20 Scheda video; 21 Ingresso e uscita	22 Verm1, parte 4
Pubblicazione		23 Verm1 sul Cartridge Store
Prospettive	24 Oltre un solo core: RISC-V superscalare, HPC, GPU e FPGA	la visita ai sistemi di calcolo

0.6 Programma e risultati di apprendimento

0.6.1 Programma

Organizzazione del calcolatore.
Hardware e software; software applicativo e di sistema; il modello di von Neumann; principi di funzionamento di un calcolatore.

Algebra di Boole e logica.
Funzioni booleane, tabelle di verità; reti combinatorie e loro minimizzazione; sommatori; macchine a stati finiti; reti sequenziali; flip-flop, registri.

Rappresentazione dei dati.
Codifica; codici ridondanti, rilevazione e correzione degli errori; decodificatori, multiplexer; sistemi posizionali; naturali, interi, reali; aritmetica.

Struttura di un calcolatore.
La CPU: unità di controllo, ALU, registri; il ciclo dell'istruzione; il datapath; la memoria centrale; il sottosistema di I/O; architetture microprogrammate.

Assembly RISC-V.
Insieme di istruzioni; registri, indirizzamento; aritmetica e logica; salti; sottoprogrammi, convenzione di chiamata, stack; memoria e I/O.

Progetto.
Verm1: un gioco in assembly RISC-V su PRG32, costruito in quattro parti guidate, poi variato, documentato e confezionato dallo studente o da un piccolo gruppo.

Il programma dettagliato è pubblicato sul sistema informativo dell'ateneo e sul portale di e-learning, https://elearning.uniparthenope.it.

0.6.2 Risultati di apprendimento

I risultati sono enunciati, come d'uso nell'istruzione superiore europea, secondo i cinque descrittori di Dublino.

Conoscenza e capacità di comprensione.
Una solida conoscenza del sistema binario, della logica digitale e dei principi di funzionamento dei calcolatori. Non si presuppone alcuna formazione informatica precedente.

Capacità di applicare conoscenza e comprensione.
La capacità di applicare la teoria a esercizi e a programmi in assembly RISC-V, e di riconoscere sia le capacità sia i limiti dei calcolatori su problemi reali.

Autonomia di giudizio.
La capacità di scrivere programmi assembly che rappresentano ed elaborano in binario problemi reali, scegliendo rappresentazioni dei dati e strutture di controllo adeguate.

Abilità comunicative.
La capacità di partecipare attivamente a gruppi che progettano, realizzano e documentano applicazioni assembly, usando una terminologia tecnica corretta.

Capacità di apprendimento.
La capacità di collegare rappresentazione, reti logiche e processori in un unico quadro, e di aggiornarsi in autonomia da fonti tecniche affidabili.

0.7 Organizzazione e valutazione

Questa sezione riporta le regole dell'anno accademico 2026/27. Le date non ancora fissate al momento della compilazione del libro vengono comunicate in aula e sul portale di e-learning, che in caso di differenze prevalgono su questo testo.

0.7.1 Orario

Da settembre a dicembre 2026 il corso si tiene quattro volte alla settimana in Aula 1: lezioni il lunedì (14:00–16:00) e il mercoledì (09:00–11:00), laboratorio il martedì e il venerdì (09:00–11:00). Un calendario interno con l'effettiva distribuzione di lezioni e sessioni di laboratorio è condiviso con gli studenti.

0.7.2 Una prova intercorso

C'è una sola prova intercorso, svolta in presenza e su carta in circa 45 minuti: **dieci domande a risposta multipla più due esercizi**. Si tiene di lunedì, 14:00–16:00, in Aula 1, in una data che sarà comunicata; gli studenti possono essere divisi in gruppi e la prova svolta a turni. I candidati portano un documento di identità e una penna, e devono conoscere il proprio indirizzo di posta istituzionale e il numero di matricola.

La prova valuta entrambe le parti, studenti e docenti: serve a sapere dove sta andando il corso. Chiude i primi tre blocchi (Tabella 5); il quarto, la macchina, si esamina all'orale. Negli esempi della prova compaiono istruzioni RISC-V: il materiale del primo contatto del Capitolo 2 si dà per noto, e il codice di Verm1, parti da 1 a 3, è una fonte naturale di domande.

Ogni capitolo di questo libro finisce con domande dello stesso tipo, e la preparazione migliore è rispondere settimana per settimana.

Tabella 5. Che cosa copre la prova intercorso.

Blocco	Lezioni	Argomenti
Informazione	da 3 a 6	storia della computazione; informazione digitale; rappresentazione dei numeri: interi e reali
Logica	da 8 a 11	algebra di Boole; elettronica e porte logiche; macchine combinatorie
Stato e memoria	da 13 a 16	macchine sequenziali: latch, flip-flop, macchine a stati; registri e bus; memoria e memoria permanente
Non nella prova	da 18 a 24	il processore, istruzioni e interruzioni, la scheda video, ingresso e uscita, le prospettive: parte dell'esame orale

Niente tecnologia. Niente GPT.

La prova intercorso e l'esame sono *#nogpt*. Non è ammesso alcun aiuto tecnologico o umano di alcun tipo: niente dispositivi, niente ChatGPT o altri modelli linguistici, niente IA generativa. Copiare annulla subito la prova; comportamenti più gravi hanno conseguenze più gravi.

0.7.3 Valutazione

L'esame ha due parti, ed entrambe vanno superate.

- **Teoria (50%)**: un esame orale con domande estemporanee su tutto il programma; alcune risposte richiedono uno schema o un calcolo alla lavagna. L'orale può anche abbassare il voto ottenuto nella prova intercorso.
- **Pratica (50%)**: l'esame di laboratorio con la prof.ssa Staffa.

Il voto finale è la media del voto di teoria e del voto di pratica. I risultati della prova intercorso e degli esami valgono entro l'anno accademico corrente.

Per verbalizzare l'esame serve un voto positivo sia in teoria sia in pratica, e la verbalizzazione avviene solo durante un appello di teoria. L'esame di laboratorio si tiene di solito una settimana prima dell'appello di teoria, ed entrambi si prenotano su https://uniparthenope.esse3.cineca.it. Due domande vengono poste ogni anno. *Si può sostenere l'esame di teoria prima di aver superato quello pratico?* Sì: il risultato vale per un anno accademico, ma l'esame viene verbalizzato solo dopo aver superato la parte pratica. *Ci si può prenotare all'appello di teoria senza la prova pratica?* Sì, il caso è lo stesso.

0.7.4 La settimana della prova e l'evento finale

Durante la settimana della prova intercorso le lezioni sono sostituite da seminari sulle architetture di calcolo emergenti. Alla fine del corso, uno studente o un gruppo che presenta il proprio gioco alla cerimonia di chiusura, una variante di Verm1 o un gioco originale, può ottenere punti bonus per l'esame. La mattinata conclusiva ha due metà: l'ultima lezione, oltre un solo core, da RISC-V superscalare ai supercalcolatori (Capitolo 24), e una visita ai

sistemi di supercalcolo del Dipartimento di Scienze e Tecnologie. I giochi finiti si possono pubblicare sul Cartridge Store di PRG32, come mostra il Capitolo 23.

0.8 Materiale

Slide.
Ogni lezione ha due presentazioni: la presentazione della lezione, di al più venti slide, per i concetti, e la presentazione del codice, con ogni listato, il programma completo della lezione e la guida per provarlo su PRG32-QT, su QEMU e sulla scheda. Sono sempre disponibili in una cartella condivisa di Google Drive e vengono pubblicate periodicamente su https://elearning.uniparthenope.it. Sono generate da sorgente: correzioni e nuovi esercizi sono benvenuti come contributi.

Programmi.
Il programma di ogni lezione è nella cartella `code` del repository del corso, https://github.com/riscv-prg32/computer-architecture-with-prg32, con le presentazioni, questo libro e i test che eseguono ogni listato.

Piattaforma.
PRG32, libero e open source, su https://github.com/riscv-prg32/PRG32, con il suo libro di testo, *Playing with RISC-V: Assembly, C, and Retro Games on the PRG32 Platform* (Montella et al., 2026b).

Libri di testo.
Patterson e Hennessy (2020), *Computer Organization and Design, RISC-V Edition* (in italiano: *Struttura e progetto dei calcolatori*), e Tanenbaum e Austin (2013), *Structured Computer Organization* (in italiano: *Architettura dei calcolatori. Un approccio strutturale*), sono i due riferimenti del corso. Il programma elenca anche Stallings (2021) e Bucci (2017).

Standard.
La documentazione ufficiale di RISC-V su https://riscv.org (RISC-V International, 2024c).

Il materiale didattico vive in una cartella condivisa con gli studenti, che possono contribuirvi; per ottenere l'accesso lo studente invia al docente l'indirizzo di un account collegato al servizio. Ogni altra comunicazione con il docente passa per l'indirizzo istituzionale dello studente.

Cineforum

Trenta film e serie sull'informatica accompagnano il corso, per ricordare che la materia ha una cultura oltre che una tecnica: *The Social Network* (2010), *Steve Jobs* (2015), *Pirates of Silicon Valley* (1999), *The Imitation Game* (2014), *Hackers* (1995), *Ex Machina* (2014), *Her* (2013), *Tron* (1982), *Tron: Legacy* (2010), *The Matrix* (1999), *WarGames* (1983), *Sneakers* (1992), *Jobs* (2013), *Antitrust* (2001), *Blackhat* (2015), *The Fifth Estate* (2013), *The Circle* (2017), *Silicon Valley* (2014–2019), *Source Code* (2011), *Chappie* (2015), *Mr. Robot* (2015–2019), *Ghost in the Shell* (1995), *Minority Report* (2002), *Transcendence* (2014), *Ready Player One* (2018), *A.I. Artificial Intelligence* (2001), *Revolt* (2017), *The Thirteenth Floor* (1999), *Eagle Eye* (2008) e *Terminator 2: Judgment Day* (1991). *Ready Player One* è anche il riferimento culturale del libro di PRG32: la dimestichezza con gli strati fondamentali è un'alfabetizzazione che conserva

il suo valore.

0.9 Primo contatto: chi siete?

La prima lezione finisce con delle domande rivolte all'aula, e un lettore che lavora da solo può rivolgerle a sé stesso. Non servono a selezionare, ma a capire da dove cominciare.

- **Hardware.** Avete mai assemblato un calcolatore? Avete giocato con un Arduino o un ESP32?
- **Software.** Quanti linguaggi di programmazione conoscete? Avete mai letto una riga di assembly?
- **Cultura.** Che cosa sono il retrocomputing e il retrogaming? Avete mai visitato un museo dell'informatica?
- **Concetti.** Sapete dire la differenza tra ROM, RAM e memoria di massa? Che cos'è il cloud computing? E l'Internet delle cose?
- **RISC-V.** Avete sentito parlare di RISC-V? Quale processore c'è nel vostro telefono, nel vostro portatile, nel vostro router?

In aula le domande sono seguite da un breve questionario anonimo, il *Cappello Parlante*, su ciò che attrae di più ciascuno studente nel futuro dell'informatica e della scienza; i risultati vengono discussi e usati per calibrare esempi e progetti.

0.10 Prima del prossimo capitolo

Il Capitolo 1 installa tutto, passo per passo, su Windows, macOS e Linux. Si va più in fretta se i download lunghi sono già fatti.

Esercizio 0.1 (preparazione)

1. Procuratevi un portatile su cui vi sia permesso installare programmi.
2. Scaricate l'installer di PRG32-QT per il vostro sistema dalla pagina *Releases* di https://github.com/riscv-prg32/PRG32-QT.
3. Con una buona connessione, avviate il download lungo: ESP-IDF, seguendo il `README` di https://github.com/riscv-prg32/PRG32 per il vostro sistema.
4. Se qualcosa fallisce, non combattete da soli: conservate il messaggio di errore.

Risultato atteso. Cominciate il Capitolo 1 con i download fatti. In due ore avete PRG32-QT in funzione e una cartuccia compilata sul vostro portatile.

0.11 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l00.S` nel repository del corso, stampato per intero nel Listato 1. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 5 è il frame scritto dal test.

Sullo schermo.
La parola ARC12 in bianco su uno schermo blu, e sotto HELLO, RISC-V.

Che cosa premere.
Niente: il programma va da solo.



Figura 5. `l00.S` su PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 00: Course Introduction
# l00.S: the first program of the course, to read before you can write it
#
# On the screen: the word ARC12 in white on a blue screen, and under it HELLO, RISC-V.
# Build: python3 -m prg32 cartridge build l00.S --portable \
#            --entry-prefix l00 --name l00 --out l00.prg32
.option norelax

.equ BLUE,  0x001f           # a colour is a 16-bit number (Lecture 04)
.equ WHITE, 0xffff

.section .text
.global l00_init
.global l00_update
.global l00_draw

l00_init:                    # called once, when the cartridge starts
    ret

l00_update:                  # called once per frame: nothing changes here
    ret

l00_draw:                    # called once per frame: paint the screen
    addi sp, sp, -16         # a function that calls others saves ra
    sw   ra, 12(sp)
    li   a0, BLUE
    call prg32_gfx_clear     # fill the screen with one colour
    li   a0, 140             # x, in pixels
    li   a1, 88              # y
    la   a2, line1           # the ADDRESS of the text
    li   a3, WHITE           # foreground
    li   a4, BLUE            # background
    call prg32_gfx_text8
    li   a0, 104
    li   a1, 104
    la   a2, line2
    li   a3, WHITE
```

```
38      li   a4, BLUE
39      call prg32_gfx_text8
40      lw   ra, 12(sp)
41      addi sp, sp, 16
42      ret
43
44  .section .rodata            # constants: two strings, each ending with a zero
        byte
45  line1: .asciz "ARC12"
46  line2: .asciz "HELLO, RISC-V"
```

Listato 1. Il programma completo del capitolo, `code/l00.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 2; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l00.S --portable \
    --entry-prefix l00 --name l00 --out l00.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l00.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 2. Compilare `l00.S` ed eseguirlo su PRG32-QT.

CAPITOLO 1

PRG32 e PRG32-QT: allestire l'officina

"Datemi sei ore per abbattere un albero, e passerò le prime quattro ad affilare l'ascia."
— attribuita ad Abraham Lincoln

In questo capitolo

Alla fine di questo capitolo avrete

- un quadro chiaro dei pezzi di PRG32: runtime, cartuccia, strumenti, host, Store;
- PRG32-QT installato sul vostro portatile, per eseguire qualunque cartuccia e guardarci dentro;
- la toolchain RISC-V installata e verificata, per compilare cartucce vostre;
- una cartuccia compilata da voi, in esecuzione in PRG32-QT e fermata nel debugger;
- un elenco di che cosa fare quando qualcosa non funziona.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI programma, sorgente, codice macchina; i componenti di PRG32.

DA UTILIZZARE OGGI il terminale, gli strumenti di PRG32, il debugger di PRG32-QT, il programma completo (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI contratto init/update/draw (Capitolo 2); i registri (Capitolo 2).

Il primo lavoro di un artigiano in una nuova officina è mettere gli attrezzi dove la mano se li aspetta. Questo capitolo è quel lavoro. Non contiene teoria, ed è l'unico capitolo del libro il cui contenuto invecchierà: i numeri di versione sono quelli di ottobre 2026, e prima di seguire i passi conviene controllare le pagine dei rilasci dei progetti. Tutto il resto del libro dipende dall'ultima riga del riepilogo qui sopra: il ciclo che consiste nel compilare una cartuccia, inviarla a un host e fermarla nel debugger si ripete in ogni capitolo che segue.

I comandi della Sezione 1.6 sono stati eseguiti per questo libro su macOS con PRG32-QT. Le installazioni per Windows e Linux seguono le istruzioni ufficiali dei progetti e non sono state eseguite per questo testo.

1.1 Quattro livelli: installate ciò che vi serve

Non tutto va installato, e non tutto subito. La Tabella 1.1 ordina i pezzi secondo la necessità. Il corso, e il libro, si possono seguire con i soli livelli 1 e 2: *non serve alcuna scheda.*

Tabella 1.1. Quattro livelli di installazione.

Livello	Installate	Potete	Serve
1	PRG32-QT	eseguire cartucce, scaricarle dallo Store, usare il debugger	a tutti, subito
2	ESP-IDF e il repository di PRG32	compilare cartucce vostre da assembly o C	a tutti, prima del Capitolo 2
3	l'emulatore QEMU	eseguire il firmware vero sul portatile	facoltativo
4	il kit: una scheda ESP32-C6	eseguire sull'hardware vero	facoltativo

Il livello 1 richiede cinque minuti. Il livello 2 è un download voluminoso e richiede da mezz'ora a un'ora: conviene farlo a casa, con una buona connessione. I livelli 3 e 4 sono per i curiosi, e per il laboratorio.

1.2 Prima di cominciare: le parole di oggi

La maggior parte dei lettori di questo capitolo non ha mai scritto un programma, e l'installazione che segue usa alcune parole fin dal primo comando. Questa sezione le spiega; il capitolo *Partire da zero*, a pagina xix, dice lo stesso più per esteso.

1.2.1 Che cos'è un programma

Un *processore* fa una sola cosa, velocissimo: esegue *istruzioni*, piccoli ordini come "somma due numeri" o "copia un valore". Un *programma* è un elenco di istruzioni, conservato in memoria sotto forma di numeri; questa forma numerica si chiama *codice macchina*. Nessuno scrive quei numeri a mano. Si scrive *codice sorgente*, un file di testo in un linguaggio di programmazione come l'assembly o il C, e un traduttore lo trasforma in codice macchina: l'*assemblatore* per l'assembly, il *compilatore* per il C. La *toolchain* è l'insieme di questi traduttori e dei loro aiutanti, e installarla è il passo 2 di questo capitolo. L'assembly è il linguaggio più vicino alla macchina, una riga per un'istruzione, ed è per questo che il corso lo usa: ciò che scrivete è ciò che il processore fa.

Voi scrivete, come codice sorgente	Il processore legge, come codice macchina
`addi a0, a0, 1`	`0x00150513`

1.2.2 Firmware, runtime, host: chi esegue il vostro programma

Un gioco non è mai solo sulla macchina. Sul kit un altro programma, il *firmware*, parte per primo, resta in memoria e gestisce lo schermo, i pulsanti e il suono; è conservato nella memoria *flash* del chip, che mantiene il contenuto anche senza alimentazione. Il *runtime* è la parte del firmware con cui il gioco dialoga: chiama il gioco a ogni *frame*, ogni nuova immagine sullo schermo, e gli presta funzioni per disegnare e per suonare. Un *host* è qualsiasi macchina che offre questo runtime: la scheda, l'*emulatore* QEMU o l'applicazione

PRG32-QT. Un emulatore, o un *interprete*, è un programma che imita un processore via software: il portatile finge di essere il chip.

1.2.3 Il terminale: parlare al computer per iscritto

Il *terminale* è una finestra in cui si scrivono comandi invece di fare clic: su macOS l'applicazione Terminale, su Windows PowerShell, su Linux Terminale o Konsole. Mostra un *prompt*, che termina con `$` o `>`, e attende; si scrive un comando e si preme Invio. In ogni momento il terminale si trova in una cartella, e i comandi agiscono sui file che contiene.

Per	**macOS e Linux**	**Windows PowerShell**
vedere in quale cartella siete	`pwd`	`pwd`
elencare i file della cartella	`ls`	`dir`
entrare nella cartella PRG32	`cd PRG32`	`cd PRG32`
risalire di una cartella	`cd ..`	`cd ..`
tornare alla cartella personale	`cd ~`	`cd ~`
fermare il programma in esecuzione	Ctrl+C	Ctrl+C

Il tasto Tab completa un nome; la freccia in su richiama il comando precedente.

1.2.4 Leggere ciò che il computer risponde

Un *percorso* nomina un file attraverso le cartelle che vi conducono, come `~/PRG32/examples/pong.S`, dove `~` è la cartella personale. Un *repository* è una cartella di codice sorgente di cui `git` conserva la storia; `git clone` ne copia uno da Internet al portatile. Un comando che funziona spesso stampa poco o nulla; uno che fallisce stampa un messaggio, da leggere per intero prima di cambiare qualsiasi cosa, perché la prima riga che dice *error* di solito nomina la causa. Quattro messaggi coprono la maggior parte dei problemi di questo capitolo: *command not found* (il programma non è installato, o non è attivo in questo terminale), *No such file or directory* (cartella o nome sbagliati: controllate con `pwd` e `ls`), *Permission denied* (mancano i permessi; su Linux un'installazione richiede `sudo`) e *Connection refused* (il programma a cui ci si rivolge, come PRG32-QT, non è in esecuzione). Quando si chiede aiuto si copiano il comando e il messaggio intero, come testo.

1.3 La piattaforma, in dettaglio

1.3.1 Un runtime, un formato di file, un insieme di strumenti

Il nome PRG32 copre quattro cose.

- Un **firmware residente**, detto anche *runtime*: il programma che possiede lo schermo, i pulsanti e il suono.
- Un **formato di cartuccia**: un gioco come un unico piccolo file, `.prg32`.
- Uno **strumento** sul portatile: il comando `python3 -m prg32`, che compila, carica e confeziona.
- Un **Cartridge Store**, dove si condividono i giochi finiti.

Il codice di una cartuccia gira come vero codice macchina RISC-V a 32 bit, RV32IMAC, e il contratto tra un gioco e il runtime sono tre funzioni: `init`, `update` e `draw`. Il runtime

chiama `init` una volta e poi `update` e `draw` a ogni frame; il Capitolo 2 parte da questo contratto.

1.3.2 Da un file sorgente allo schermo

La Figura 1.1 segue un programma dall'editor allo schermo. Il sorgente, in assembly o in C, viene tradotto in codice macchina dalla toolchain RISC-V; il compilatore è `riscv32-esp-elf-gcc`, installato da ESP-IDF, il kit di sviluppo di Espressif. Il risultato viene impacchettato in una cartuccia, fatta di codice, dati e un'intestazione. La cartuccia viene inviata attraverso la rete a un *host*, dove il runtime la carica e comincia a chiamare le sue tre funzioni.

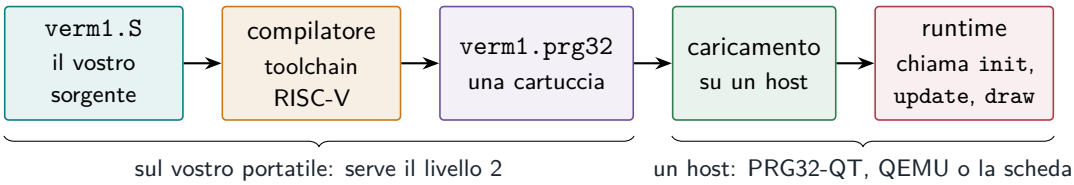


Figura 1.1. Da un file sorgente allo schermo.

Una cartuccia viene caricata all'indirizzo `0x40800000` e raggiunge il runtime soltanto attraverso una tabella di funzioni, l'*application binary interface* (ABI). È la ragione per cui lo stesso file gira su tre host diversi: non tocca mai direttamente l'hardware, e ogni host fornisce la stessa tabella.

1.3.3 Il kit: una console su breadboard

L'hardware di PRG32 (Figura 1.2) è costruito attorno a un ESP32-C6 di Espressif: un core RISC-V a 160 MHz, con Wi-Fi. Un display a colori da 2,8 pollici di 320 per 240 pixel (un controller ILI9341 su un bus SPI) mostra i giochi, che disegnano in un viewport di 320 per 200. Uno o due piccoli amplificatori audio (MAX98357A, su un bus I2S), un joystick e dei pulsanti sono collegati direttamente ai piedini del chip. Un gioco raggiunge la scheda via Wi-Fi: la scheda crea una rete chiamata `PRG32`, e la cartuccia vi viene caricata. L'elenco dei componenti e i collegamenti sono nella cartella `docs/hardware` del repository.

1.3.4 Tre modi di eseguire la stessa cartuccia

Il file della cartuccia è lo stesso su ogni host; cambia il modo in cui le sue istruzioni vengono eseguite e ciò che si può osservare (Tabella 1.2).

La regola pratica è: usate la scheda quando volete la verità sul *tempo*, e PRG32-QT quando volete la verità su un *registro*. L'emulatore modella un ESP32-C3 e non un C6 perché Espressif mantiene per quel chip un emulatore maturo con grafica; esegue le stesse istruzioni RISC-V, e la cartuccia non se ne accorge.

1.3.5 PRG32-QT: la console sul portatile

PRG32-QT è un'applicazione per Windows, macOS, Linux, Raspberry Pi, telefoni, tablet e televisori. Contiene un interprete di RV32IMAC, che esegue una cartuccia istruzione per istruzione e le offre le stesse funzioni del firmware. Ha tre velocità, scelte in *Settings*: *Accurate*, temporizzata come l'ESP32-C6, che è quella predefinita; *Optimal*, a 30 frame al secondo; e *Unlimited*, veloce quanto lo è il portatile. Il profilo accurato attribuisce a ogni

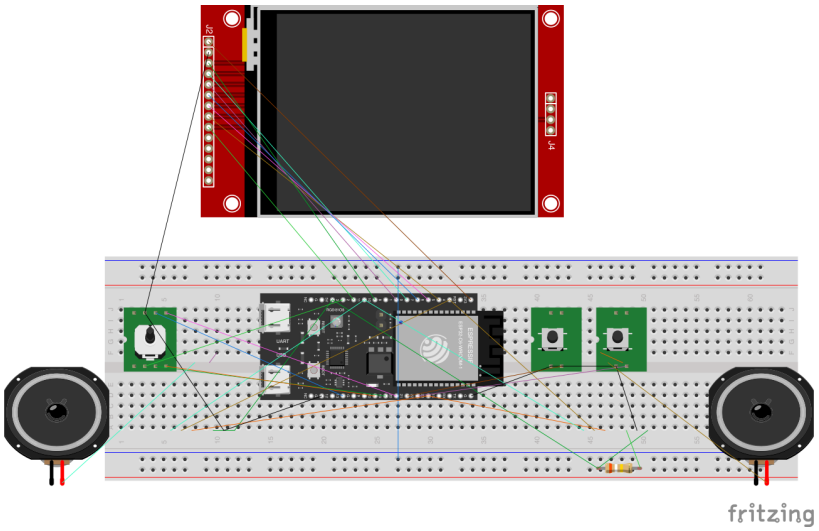


Figura 1.2. Il kit PRG32 su breadboard.

Tabella 1.2. Tre host per la stessa cartuccia.

	Scheda ESP32-C6	**QEMU di Espressif**	**PRG32-QT**
Come gira il codice	in modo nativo sul chip	in modo nativo su un ESP32-C3 emulato	guest RV32IMAC interpretato
Serve una toolchain	sì (ESP-IDF)	sì (ESP-IDF)	no, per giocare e fare debug
La cartuccia arriva con	caricamento via Wi-Fi, Store	scrittura nell'immagine della flash	importazione di file, Store, caricamento HTTP
Esecuzione passo-passo	con JTAG e GDB	con GDB	debugger integrato
Gira su	il kit hardware	Windows, Linux, macOS	desktop, Raspberry Pi, iOS, Android, TV

istruzione e a ogni chiamata un costo in cicli di un clock a 160 MHz: non è un modello esatto al ciclo del chip, ma è abbastanza vicino da far vedere quando un frame è in ritardo. PRG32-QT contiene anche un browser dello Store e il debugger descritto nel Capitolo 0.

I comandi della console corrispondono alla tastiera come nella Tabella 1.3. Un controller di gioco USB o Bluetooth viene riconosciuto da PRG32-QT quando lo si collega.

Attenzione

QEMU legge la tastiera dal terminale che lo ha lanciato, non dalla finestra con l'immagine. È la classica sorpresa della prima sessione.

1.4 Passo 1: installare PRG32-QT

Cinque minuti; nessun compilatore, nessun account. Dalla pagina https://github.com/riscv-prg32/PRG32-QT, sotto *Releases*, prendete l'ultima versione del file per il vostro siste-

Tabella 1.3. I comandi, su una tastiera.

Sul kit	In PRG32-QT	In QEMU
Joystick	tasti freccia, oppure **W A S D**	gli stessi, battuti nel terminale
Pulsante A	**Z** o **J**	**Z** o **J**
Pulsante B	**X** o **K**	**X** o **K**
Start / Select	**Invio** o **Spazio**	**Invio** o **Spazio**
Uscire dal gioco	**Esc**: ritorno a Setup	chiudere QEMU

ma (Tabella 1.4). Un file `SHA256SUMS-platforms.txt` sulla stessa pagina elenca le somme di controllo; lì ci sono anche le versioni per Android e per i televisori.

Tabella 1.4. Gli installer di PRG32-QT (versione 0.4.1, ottobre 2026; i nomi seguono questo schema con il numero corrente).

Sistema	File	Tipo
Windows 10 o 11	`PRG32-QT-0.4.1-Windows-X64-Setup.exe`	installer
macOS, Apple Silicon	`PRG32-QT-0.4.1-macOS-ARM64.dmg`	immagine disco
macOS, Intel	`PRG32-QT-0.4.1-macOS-Intel-X64.dmg`	immagine disco
Linux, PC a 64 bit	`PRG32-QT-0.4.1-Linux-X64.deb`	pacchetto Debian
Raspberry Pi OS, 64 bit	`PRG32-QT-0.4.1-Raspberry-Pi-OS-ARM64.deb`	pacchetto Debian

Windows. Scaricate il file che termina con `Windows-X64-Setup.exe` ed eseguitelo. Se Windows mostra un avviso blu di SmartScreen, scegliete *Ulteriori informazioni* e poi *Esegui comunque*. Accettate la cartella proposta: l'installer porta con sé le librerie Qt e il runtime del compilatore, e non c'è altro da installare. Avviate *PRG32* dal menu Start; se il firewall lo chiede, autorizzatelo sulle reti private. La domanda compare perché PRG32-QT ascolta sulla porta 8080 le cartucce inviate dagli strumenti.

macOS. Dal menu Apple scegliete *Informazioni su questo Mac*: «Chip Apple M…» significa ARM64, «Processore Intel» significa X64. Aprite il `.dmg` e trascinate *PRG32* sulla cartella Applicazioni. Al primo avvio, se macOS dice che lo sviluppatore non può essere verificato (l'applicazione è distribuita fuori dall'App Store), fate clic con il tasto destro su *PRG32* e scegliete *Apri*, oppure andate in *Impostazioni di Sistema, Privacy e sicurezza, Apri comunque*. Accettate le connessioni in entrata e l'accesso alla rete locale quando viene chiesto: servono all'interfaccia di dispositivo sulla porta 8080 e al suo annuncio sulla rete locale.

Linux. Il pacchetto richiede Qt 6.5 o successivo, che Debian 13 e Ubuntu 25.04 forniscono; sui sistemi più vecchi (Ubuntu 24.04 ha Qt 6.4) il programma si compila da sorgente (Listato 1.1). Su Raspberry Pi OS usate il pacchetto ARM64, oppure lo script `scripts/build-raspbian.sh`.

```
# Debian 13, Ubuntu 25.04 and later, or another system with Qt 6.5 or newer
cd ~/Downloads
sudo apt install ./PRG32-QT-0.4.1-Linux-X64.deb
PRG32 &
```

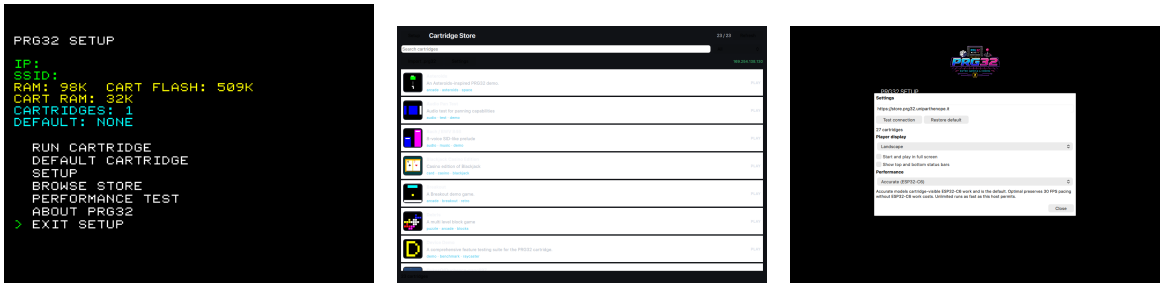
```
# On older systems (Ubuntu 24.04 has Qt 6.4: too old): build from source
sudo apt install git cmake ninja-build g++ qt6-base-dev qt6-declarative-dev \
    qt6-multimedia-dev qt6-websockets-dev
git clone https://github.com/riscv-prg32/PRG32-QT
cd PRG32-QT
./scripts/build-linux.sh
```

Listato 1.1. PRG32-QT su Linux.

Eseguite: la vostra prima cartuccia, dallo Store

1. Avviate PRG32-QT: compare la schermata *Setup*, la stessa che il firmware mostra sul kit.
2. Scegliete *Browse Store*: viene scaricato il catalogo del Cartridge Store.
3. Scegliete un gioco, scaricatelo, e scegliete *Run Cartridge*.
4. Giocate con i tasti freccia, **Z** e **X**. **Esc** vi riporta a Setup.
5. Aprite *Settings*: guardate i tre profili di prestazioni e le scelte di disposizione.

Risultato atteso. Un gioco dello Store gira sul vostro portatile (Figura 1.3). Questo è il livello 1: da ora potete eseguire qualunque cartuccia, e guardarci dentro. Se l'elenco dello Store resta vuoto, il portatile non raggiunge Internet oppure c'è di mezzo un proxy: *Settings* mostra l'indirizzo dello Store.



(a) Il menu Setup **(b)** Browse Store **(c)** Settings

Figura 1.3. Che cosa dovreste vedere: le stesse tre schermate su ogni sistema.

1.5 Passo 2: installare la toolchain

1.5.1 Un'idea dietro ogni installazione

Il compilatore per la famiglia ESP32 arriva con ESP-IDF, il kit di sviluppo di Espressif (Espressif Systems, 2026). PRG32 richiede ESP-IDF 5.4 o successivo, installato per due chip: `esp32c6`, il kit, ed `esp32c3`, l'emulatore. Dopo di che, un solo comando fa tutto: `python3 -m prg32`.

ESP-IDF non è attivo da solo

Ogni nuovo terminale deve accendere ESP-IDF prima che gli strumenti funzionino. Quasi ogni problema di installazione è ESP-IDF non attivo *nel terminale in cui state scrivendo*. Il sintomo è sempre lo stesso messaggio:

```
idf.py:  command not found
```

(oppure `riscv32-esp-elf-gcc` non trovato). Su Windows, attivare significa aprire la *ESP-IDF PowerShell*; su macOS e Linux, eseguire `export.sh`.

1.5.2 Windows

Installate Git for Windows e Python 3, poi eseguite l'ESP-IDF Tools Installer e lavorate nella finestra di PowerShell che esso crea (Listato 1.2). Usate la ESP-IDF PowerShell, mai un terminale qualunque; su Windows il comando è `python`, non `python3`; `pip` installa nell'ambiente di ESP-IDF, ed è voluto. Evitate le cartelle con spazi o lettere accentate nel percorso. Chi preferisce un editor può usare Visual Studio Code con l'estensione Espressif ESP-IDF, che installa la stessa toolchain.

```
# 1. Install Git for Windows (git-scm.com) and Python 3 (python.org)
# 2. Run the ESP-IDF Tools Installer (dl.espressif.com/dl/esp-idf):
#      - choose an ESP-IDF release 5.4 or newer
#      - select the targets esp32c3 and esp32c6
#      - optional: tick the QEMU RISC-V emulator
# 3. Open the shortcut "ESP-IDF PowerShell" that the installer created
# 4. In that window:

cd $HOME\Documents
git clone https://github.com/riscv-prg32/PRG32
cd PRG32
pip install pyaudio zeroconf
python -m prg32 doctor
```

Listato 1.2. La toolchain su Windows.

1.5.3 macOS

Prima viene Homebrew, perché fornisce `git`, `cmake` e `python` (Listato 1.3); `git` può chiedere, la prima volta, di installare gli strumenti a riga di comando di Xcode. Il clone ricorsivo e `install.sh` scaricano qualche gigabyte: abbiate pazienza. Il punto prima di `export.sh` conta, perché fa sì che lo script modifichi *questo* terminale. La procedura funziona su Apple Silicon e su Intel.

```
# 1. Homebrew (brew.sh), then the prerequisites
brew install git cmake ninja dfu-util ccache libusb python curl zip portaudio

# 2. ESP-IDF 5.4 for the two chips (a large download)
cd $HOME
git clone -b v5.4 --recursive https://github.com/espressif/esp-idf.git
cd esp-idf
./install.sh esp32c3,esp32c6
. ./export.sh
pip install pyaudio zeroconf

# 3. PRG32
cd $HOME
git clone https://github.com/riscv-prg32/PRG32
cd PRG32
python3 -m prg32 doctor
```

Listato 1.3. La toolchain su macOS.

1.5.4 Linux (Debian, Ubuntu)

I prerequisiti sono una lunga riga di `apt`, da copiare per intero (Listato 1.4); i passi 2 e 3 sono gli stessi di macOS. Altre distribuzioni hanno gli stessi pacchetti con altri nomi. Solo per il kit, l'utente deve appartenere al gruppo `dialout`, che dà accesso alla porta seriale della scheda: `sudo usermod -aG dialout $USER`, poi si esce e si rientra nella sessione.

```
# 1. Prerequisites
sudo apt update
sudo apt install -y git wget flex bison gperf python3 python3-venv python3-pip
    \
    python3-zeroconf cmake ninja-build ccache libffi-dev libssl-dev dfu-util \
    libusb-1.0-0 curl zip portaudio19-dev

# 2. ESP-IDF 5.4 for the two chips (a large download)
cd $HOME
git clone -b v5.4 --recursive https://github.com/espressif/esp-idf.git
cd esp-idf
./install.sh esp32c3,esp32c6
. ./export.sh
pip install pyaudio zeroconf

# 3. PRG32
cd $HOME
git clone https://github.com/riscv-prg32/PRG32
cd PRG32
python3 -m prg32 doctor
```

Listato 1.4. La toolchain su Linux.

1.5.5 A ogni nuovo terminale: accendere ESP-IDF

`export.sh` agisce sul terminale in cui viene eseguito, e solo su quello. Una nuova finestra è un nuovo terminale, e va attivata di nuovo. Due alias risparmiano battute (Listato 1.5); sono una comodità, non un obbligo. I comandi `prg32` si eseguono dalla cartella di PRG32.

```
# macOS and Linux: in every new terminal, before anything else
. $HOME/esp-idf/export.sh
cd $HOME/PRG32

# save typing: add these two lines to ~/.zshrc or ~/.bashrc
alias get_idf=". $HOME/esp-idf/export.sh"
alias prg="python3 -m prg32"

# then, in a new terminal
get_idf
prg doctor

# Windows: open "ESP-IDF PowerShell" from the Start menu, then
cd $HOME\Documents\PRG32
```

Listato 1.5. Attivare ESP-IDF in un nuovo terminale.

Vedete l'errore una volta, di proposito

Chiudete il terminale, apritene uno nuovo ed eseguite `python3 -m prg32 doctor` *senza* attivare ESP-IDF. Leggete il messaggio. È meglio incontrarlo qui che nel mezzo di un tutorial.

1.5.6 Il doctor

Il comando `doctor` controlla l'installazione (Listato 1.6). La riga su `idf.py` dice che ESP-IDF è attivo; quella su `riscv32-esp-elf-gcc` che il compilatore RISC-V è raggiungibile; quelle sulle partizioni che la cartella di PRG32 è completa. I quattro slot per le cartucce, da `cart0` a `cart3`, sono regioni della memoria flash della scheda, di 131 072 byte ciascuna. Ogni riga marcata `ERROR` dice che cosa fare. Se il doctor passa, il livello 2 è fatto.

```
$ python3 -m prg32 doctor
[OK] idf.py found
[OK] riscv32-esp-elf-gcc found
[OK] partitions file found: .../PRG32/profiles/partitions_prg32.csv
[OK] partition cart0 parsed (offset=0x210000, size=131072)
...

# and when ESP-IDF is not active in this terminal:
[ERROR] idf.py missing (you must source ESP-IDF export.sh)
```

Listato 1.6. Il doctor (messaggi di ottobre 2026).

1.6 Passo 3: la vostra prima compilazione

1.6.1 Compilare una cartuccia dagli esempi

Il repository contiene giochi completi. Compilarne uno dimostra che il compilatore funziona prima ancora di aver scritto una sola riga (Listato 1.7).

```
# in the PRG32 folder, ESP-IDF active
python3 -m prg32 cartridge build examples/games/pong/c/game.c \
    --portable --entry-prefix pong_c --name pong --out pong.prg32

[OK] built pong.prg32 name=pong load=0x40800000
[INFO] code=3004 mem=3028 audio=0

# what did we build?
python3 -m prg32 cartridge summary pong.prg32
```

Listato 1.7. Compilare Pong, in C, dagli esempi.

Il comando `cartridge build` prende un sorgente e produce un file `.prg32`. L'opzione `--entry-prefix` dà il nome che nel sorgente precede `_init`, `_update` e `_draw`; `--portable`

chiede una cartuccia che giri su ogni host, e non su un solo firmware. Un Pong intero sono tremila byte. Nel Capitolo 2 il sorgente sarà quello del lettore, in assembly, e il comando sarà lo stesso. Su Windows scrivete `python` e mettete il comando su una sola riga.

1.6.2 Inviarla a PRG32-QT

PRG32-QT risponde sulla porta 8080 esattamente come una scheda (Listato 1.8). Il comando `runtime` stampa lo stato dell'host ed è una prova rapida della connessione; `upload-and-run` memorizza la cartuccia in uno slot e la avvia. Il comando dice `esp32c6` perché, per lo strumento, PRG32-QT *è* una scheda. Se `localhost` non risponde, leggete l'indirizzo mostrato nella schermata Setup di PRG32-QT e usate quello. Senza gli strumenti, lo stesso risultato si ottiene in PRG32-QT importando il file `pong.prg32` e scegliendo *Run Cartridge*; con `--url http://192.168.4.1` lo stesso comando invia il gioco a una scheda vera.

```
# PRG32-QT must be running on this computer

# 1. is it listening?
python3 -m prg32 runtime --url http://localhost:8080

# 2. upload the cartridge to slot cart0 and start it
python3 -m prg32 esp32c6 upload-and-run pong.prg32 \
    --url http://localhost:8080 --slot cart0

{"loaded":false,"ok":true,"slot":"cart0","stored":true}
{"ok":true,"slot":"cart0"}
[OK] Finished upload and run on ESP32C6
```

Listato 1.8. Inviare la cartuccia a PRG32-QT, in esecuzione sullo stesso calcolatore.

1.6.3 Guardare dentro: il debugger

Aprite la macchina

1. In PRG32-QT aprite *Settings* e attivate *Run cartridges in RISC-V debug mode*.
2. Eseguite di nuovo la cartuccia: il gioco è a sinistra, la macchina a destra (Figura 1.4).
3. Premete **Pause** (il pulsante con due barre): il gioco si blocca, e nel listato è evidenziata un'istruzione.
4. Premete **Step** (il triangolo con una barra) qualche volta: l'evidenziazione si sposta, e alcuni registri cambiano valore.
5. Premete **Update**, poi **Draw**: il listato salta a quelle due funzioni della cartuccia.
6. Premete **Resume** (il triangolo): il gioco prosegue. Abbassate la velocità a 0.1x e guardatelo al rallentatore.

I tre pulsanti di trasporto del debugger sono disegnati come su un lettore video; questo libro li chiama **Pause**, **Resume** e **Step**. Questo è solo un primo sguardo: il prossimo capitolo spiega che cosa sono i registri. Qui importa che il debugger si apra e risponda. La modalità debug resta attiva finché non viene disattivata.

1.7 Facoltativo: QEMU e il kit

1.7.1 Livello 3: l'emulatore QEMU

QEMU emula un chip intero ed esegue il firmware vero (Listato 1.9). Il comando `qemu build` compila il firmware e la prima volta richiede parecchi minuti, perché scarica dei componenti; `qemu upload` scrive una cartuccia nell'immagine della flash, e mentre gira QEMU deve essere chiuso. Se il caricamento fallisce, eseguite QEMU una volta perché l'immagine della flash esista, chiudetelo, e caricate di nuovo. Da ottobre 2026 il firmware parte per impostazione predefinita dal menu *Setup*, su QEMU e sulla scheda: scegliete *Run Cartridge*, poi lo slot `cart0`. Questa procedura viene dalla documentazione di PRG32 (`docs/usage/qemu.md`) e non è stata eseguita per questo libro.

```
# libraries for the window of the emulator
brew install libgcrypt glib pixman sdl2 libslirp                        # macOS
sudo apt install -y libgcrypt20 libglib2.0-0 libpixman-1-0 \
    libsdl2-2.0-0 libslirp0                                              # Linux

# the emulator itself (Windows: tick it in the ESP-IDF installer)
python $IDF_PATH/tools/idf_tools.py install qemu-riscv32
. $IDF_PATH/export.sh

# build the firmware for QEMU, stage a cartridge, run
python3 -m prg32 qemu build
python3 -m prg32 qemu upload pong.prg32
python3 -m prg32 qemu run
```

Listato 1.9. Livello 3: l'emulatore QEMU.

1.7.2 Livello 4: il kit

Il firmware viene scritto sulla scheda una volta, attraverso un cavo USB; dopo di che i giochi arrivano via Wi-Fi e il cavo non serve più (Listato 1.10). Il comando di caricamento è lo stesso usato per PRG32-QT, con un altro indirizzo. Se la porta seriale non viene trovata la si indica esplicitamente: `COM`*x* su Windows, `/dev/ttyACM0` su Linux, `/dev/cu.usbmodem...` su macOS. Uno schermo nero dopo la scrittura significa di solito che è stato scritto il firmware per QEMU invece di quello per la scheda. Anche questa procedura viene dalla documentazione e non è stata eseguita per questo libro.

```
# connect the board with a USB cable; ESP-IDF active, in the PRG32 folder
python3 -m prg32 esp32c6 build-and-flash

# the board starts and creates a Wi-Fi network
#     name: PRG32     password: prg32game     address: 192.168.4.1
# join it with your laptop, then
python3 -m prg32 esp32c6 upload-and-run pong.prg32 \
    --url http://192.168.4.1 --slot cart0

# or let the board join the Wi-Fi of the classroom
python3 -m prg32 wifi set --ssid "NAME" --password "PASSWORD" --mode sta
```

Listato 1.10. Livello 4: il kit.

1.8 Quando qualcosa non funziona

La Tabella 1.5 elenca i sintomi che si incontrano più spesso; la prima riga copre la maggior parte dei casi. Il file `docs/usage/troubleshooting.md` del repository contiene l'elenco completo.

Tabella 1.5. Risoluzione dei problemi.

Vedete	Significa	Fate così
`idf.py: command not found`	ESP-IDF non è attivo in questo terminale	eseguite `export.sh`, o aprite la ESP-IDF PowerShell
manca `riscv32-esp-elf-gcc`	i compilatori non sono stati installati, o non sono attivi	`./install.sh esp32c3,esp32c6`, poi `export.sh`
`No module named prg32`	non siete nella cartella di PRG32	`cd` nella cartella che avete clonato
upload: connection refused	PRG32-QT non è in esecuzione, o un firewall blocca la porta 8080	avviatelo; autorizzatelo; usate l'indirizzo della sua schermata Setup
macOS non apre PRG32	l'applicazione viene da fuori dell'App Store	clic destro, Apri; oppure Privacy e sicurezza, Apri comunque
Linux: PRG32 non parte	il Qt del sistema è più vecchio della 6.5	una distribuzione più recente, o compilate da sorgente
QEMU ignora i tasti	la tastiera va al terminale	fate clic sul terminale che ha lanciato QEMU

Come chiedere aiuto

Incollate il comando e il messaggio *intero* come testo. Non mandate mai una fotografia dello schermo.

1.8.1 Pronti per il prossimo capitolo?

Cinque controlli, e l'officina è pronta.

1. PRG32-QT parte ed esegue un gioco dello Store (passo 1).
2. In un nuovo terminale, dopo aver attivato ESP-IDF, `doctor` mostra solo `OK` (passo 2).
3. `cartridge build` produce `pong.prg32` (passo 3).
4. `upload-and-run` lo avvia in PRG32-QT (passo 3).
5. In modalità debug, **Pause** e **Step** funzionano (passo 3).

1.9 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l01.S` nel repository del corso, stampato per intero nel Listato 1.11. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 1.5 è il frame scritto dal test.

Sullo schermo.
Quattro barre verticali, rossa, verde, blu e bianca, e le parole PRG32 OK.

Che cosa premere.

Niente: il programma va da solo.

```
# ARC12 Computer Architecture - Lecture 01: PRG32 and PRG32-QT: Setting Up Your Workshop
# l01.S: a test pattern that proves the installation works
#
# On the screen: four vertical bars, red, green, blue and white, and the words PRG32
# OK.
# Build: python3 -m prg32 cartridge build l01.S --portable \
#            --entry-prefix l01 --name l01 --out l01.prg32
.option norelax

.equ BLACK, 0x0000
.equ WHITE, 0xffff

.section .text
.global l01_init
.global l01_update
.global l01_draw

l01_init:
    ret

l01_update:
    ret

l01_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   s0, 0                # bar number, 0 to 3
1:  la   t0, bars             # the table of the four colours
    slli t1, s0, 2            # 4 bytes per word
    add  t0, t0, t1
    lw   a4, 0(t0)            # a4 = colour of bar s0
    slli a0, s0, 6            # 64 * bar
    addi a0, a0, 40           # x = 40 + 64 * bar
    li   a1, 40               # y
    li   a2, 48               # width
    li   a3, 100              # height
    call prg32_gfx_rect
    addi s0, s0, 1
    li   t0, 4
    blt  s0, t0, 1b
    li   a0, 124
    li   a1, 160
    la   a2, ok
    li   a3, WHITE
    li   a4, BLACK
    call prg32_gfx_text8
    lw   s0, 8(sp)
```

```
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

.section .rodata
bars: .word 0xf800, 0x07e0, 0x001f, 0xffff     # red, green, blue, white
ok:   .asciz "PRG32 OK"
```

Listato 1.11. Il programma completo del capitolo, `code/l01.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 1.12; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l01.S --portable \
    --entry-prefix l01 --name l01 --out l01.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l01.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 1.12. Compilare `l01.S` ed eseguirlo su PRG32-QT.

1.10 Verificate di aver capito

D1.1 In un nuovo terminale, `python3 -m prg32 doctor` dice: `idf.py missing`. Qual è la causa più probabile?

A. ESP-IDF va installato di nuovo
B. ESP-IDF non è attivo in questo terminale
C. PRG32-QT non è in esecuzione
D. la scheda non è collegata

D1.2 Quale host permette di eseguire una cartuccia e farne il debug senza installare un compilatore?

A. la scheda ESP32-C6
B. QEMU
C. PRG32-QT
D. nessuno

D1.3 Perché lo stesso `pong.prg32` gira sulla scheda, su QEMU e su PRG32-QT?

A. viene ricompilato a ogni avvio
B. contiene tre programmi
C. chiama il runtime attraverso la stessa tabella di funzioni su ogni host

D. PRG32-QT lo traduce in C

D1.4 Che cosa fa un assemblatore?

A. esegue il programma sul processore
B. traduce il sorgente assembly in codice macchina
C. invia la cartuccia a un host
D. mostra i registri

1.11 Esercizi

Esercizio 1.1 (l'officina sopravvive a un riavvio)

1. Completate i cinque controlli qui sopra sul vostro portatile.
2. Scaricate tre cartucce dallo Store, e in ciascuna trovate con il debugger la prima istruzione di `update`.
3. Compilate uno degli altri esempi in `examples/games`, ed eseguitelo.
4. Chiudete tutto, riavviate il calcolatore, e tornate a una cartuccia in esecuzione senza questo libro.
5. Facoltativo: installate QEMU ed eseguite lì la stessa cartuccia.

Risultato atteso. La vostra officina sopravvive a un riavvio, e sapete aprirla senza aiuto. Il quarto compito è la vera prova: mostra se l'attivazione di ESP-IDF è stata capita.

1.12 Glossario

Inglese	Italiano
toolchain	catena di strumenti di sviluppo
cartridge	cartuccia
emulator	emulatore
to build	compilare (costruire)
host	macchina ospite
firmware	firmware
runtime	ambiente di esecuzione
interpreter	interprete
to flash	scrivere nella memoria flash
breakpoint	punto di interruzione

1.13 Riepilogo e letture

PRG32 è un runtime, un formato di cartuccia, uno strumento e uno Store; una cartuccia gira senza modifiche sulla scheda, su QEMU e su PRG32-QT. PRG32-QT basta per giocare e per fare debug; ESP-IDF e il repository di PRG32 servono per compilare. ESP-IDF va attivato in ogni nuovo terminale: ricordatelo e la maggior parte dei problemi scompare. Avete compilato una cartuccia, l'avete inviata a PRG32-QT e l'avete fermata nel debugger: è il ciclo di tutto il libro.

Letture. Il `README` e la cartella `docs/usage` del repository di PRG32 (Montella et al., 2026a); il `README` e la documentazione di PRG32-QT (Montella, 2026); i capitoli sull'installazione e su PRG32-QT di *Playing with RISC-V* (Montella et al., 2026b). L'Appendice B di questo libro è un riferimento della piattaforma e dei suoi comandi.

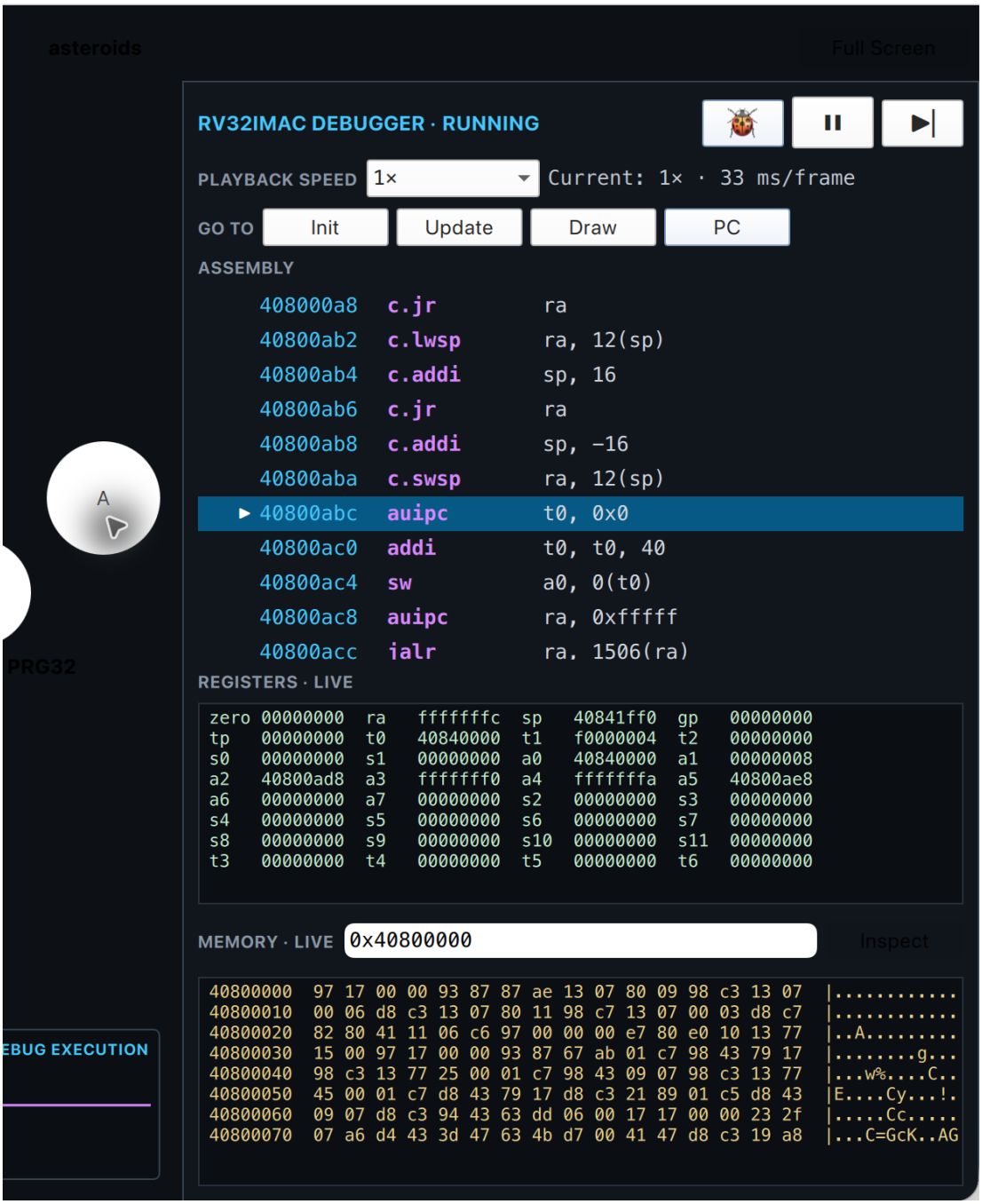


Figura 1.4. La macchina, aperta: l'assembly con l'istruzione corrente, i 32 registri, la memoria, e i contatori del frame. Se vedete questo con la vostra cartuccia, siete pronti per il resto del libro.

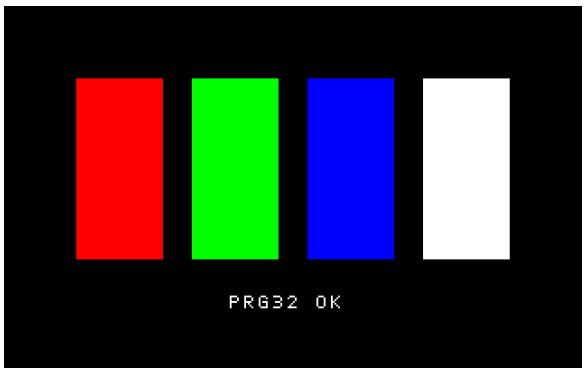


Figura 1.5. `101.S` su PRG32-QT.

CAPITOLO 2

Primo contatto: assembly RISC-V su PRG32

"Talk is cheap. Show me the code."
— Linus Torvalds, mailing list del kernel Linux, 2000: le chiacchiere stanno a zero, fatemi vedere il codice

In questo capitolo

Alla fine di questo capitolo saprete

- compilare una cartuccia da un file assembly ed eseguirla su PRG32;
- dire che cos'è un registro, e nominare quelli che si usano ogni giorno;
- chiamare una funzione del framework, passando gli argomenti nei registri;
- conservare un valore in memoria da un frame al successivo: caricare, modificare, memorizzare;
- leggere un'istruzione come il numero di 32 bit che è in realtà;
- elencare le domande che questo programma solleva, e a cui risponde il resto del libro.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI registri, calcolo, trasferimento, salto; i registri; funzioni, chiamata e ritorno; contratto init/update/draw; leggere una riga di assembly; gli argomenti in a0 ... a7; perché si salva ra; indirizzo, valore, load, store; le istruzioni di base; le pseudo-istruzioni; un'istruzione è un numero; seguire a mano l'esecuzione.

DA UTILIZZARE OGGI l'esadecimale (Capitolo 4); bit, byte, parola (Capitolo 4); la convenzione di chiamata (Capitolo 18); le maschere di bit (Capitolo 8) e, usati senza essere studiati nel corso, le direttive dell'assemblatore, gli strumenti di PRG32, le chiamate del runtime PRG32, il debugger di PRG32-QT, il programma completo.

DA COMPRENDERE PIÙ AVANTI la pila (Capitolo 18); i modi di indirizzamento (Capitolo 19); formati delle istruzioni (Capitolo 19); il complemento a due (Capitolo 5); la virgola fissa (Capitolo 6); l'algebra di Boole (Capitolo 8); il sommatore (Capitolo 9); dai volt ai bit (Capitolo 10); latch e flip-flop (Capitolo 13); il ciclo del processore (Capitolo 19); il progetto Verm1 (Capitolo 7).

Questo capitolo è il giro «gioca» della spirale per tutta la prima metà del libro: ogni capitolo successivo rimanda a qualcosa che è stato scritto qui. *Non* ci si aspetta che il lettore capisca

tutto ciò che contiene. Questo capitolo serve a vedere; i successivi servono a spiegare. Si scrivono quattro piccoli programmi, ciascuno completo e ciascuno funzionante; sono i passi verificati del primo laboratorio di assembly del libro di PRG32 (Montella et al., 2026b).

2.1 La macchina

2.1.1 Binario ed esadecimale: come leggere i numeri

Prima del primo programma, una parola su come sono scritti i suoi numeri, perché il codice assembly ne è pieno. Il computer conserva tutto come *bit*, cifre che valgono 0 o 1; un numero scritto con i bit è in *binario*, base 2, dove ogni posizione vale il doppio di quella alla sua destra: $1010_2 = 8 + 2 = 10$. Le lunghe file di bit si leggono male, così i programmatori le raggruppano quattro alla volta e scrivono ogni gruppo con una cifra *esadecimale*, base 16: da 0 a 9, poi da A a F per i valori da dieci a quindici. Il prefisso `0x` dice che un numero è esadecimale, quindi `0x10` vale sedici e non dieci.

Decimale	Binario	Esadecimale
0	`0000`	`0x0`
1	`0001`	`0x1`
9	`1001`	`0x9`
10	`1010`	`0xA`
15	`1111`	`0xF`
16	`1 0000`	`0x10`
255	`1111 1111`	`0xFF`

Una cifra esadecimale corrisponde esattamente a quattro bit: `0x07E0`, il verde di PRG32, è `0000 0111 1110 0000`. Oggi basta riconoscere queste notazioni; il Capitolo 4 le spiega, con le conversioni nei due sensi.

2.1.2 I registri, e tre tipi di istruzione

Un processore RISC-V, visto dal programmatore, è una cosa molto piccola.

Trentadue registri.
Ogni *registro* contiene un valore di 32 bit. Il processore può calcolare soltanto su valori che stanno nei registri.

Calcolare.
Un'operazione aritmetica o logica tra registri: `add`, `sub`, `and`, `or`, `xor`, e le loro forme *immediate*, come `addi`, in cui un operando è una piccola costante scritta nell'istruzione.

Spostare.
Un trasferimento tra un registro e la memoria: `lw` carica una parola, `sw` memorizza una parola. Nient'altro tocca la memoria.

Saltare.
Un cambiamento dell'istruzione che viene eseguita dopo: salti condizionati come `beqz`, chiamate e ritorni.

Questa è quasi tutta la macchina. La ricchezza dell'informatica non viene da un grande insieme di istruzioni, ma dal combinarne uno piccolo in lunghe sequenze.

2.1.3 I registri che si usano davvero

L'hardware chiama i registri da `x0` a `x31` e, con una sola eccezione, li tratta allo stesso modo. I programmatori li chiamano con altri nomi, che ricordano il *ruolo* svolto da ciascuno (Tabella 2.1). I ruoli non sono fissati dall'hardware ma da un accordo, la *convenzione di chiamata*, parte dell'application binary interface (ABI). Rispettarla è ciò che permette a una funzione scritta in assembly e al codice C del framework di chiamarsi a vicenda.

Tabella 2.1. I registri che si usano ogni giorno. La tabella completa è nell'Appendice A.

Registro	Nome	Ruolo	Conservato da una chiamata?
`a0`–`a7`	argomenti	valori passati a una chiamata; `a0` porta anche il valore di ritorno	no
`t0`–`t6`	temporanei	valori di lavoro	no
`s0`–`s11`	salvati	valori che una chiamata promette di conservare	sì
`ra`	indirizzo di ritorno	dove salta `ret`	no: salvatelo voi
`sp`	puntatore allo stack	cima dello stack, allineata a 16 byte	sì
`zero`	zero	si legge sempre come 0; le scritture sono ignorate	–

2.1.4 Le funzioni: un pezzo di programma con un nome

Una *funzione* è una sequenza di istruzioni che svolge un compito, come cancellare lo schermo, e comincia a un'etichetta, che le dà il nome. *Chiamare* una funzione significa saltare alla sua prima istruzione dopo aver annotato dove tornare: l'istruzione `call` fa entrambe le cose. Finito il compito, la funzione *ritorna*: `ret` salta all'istruzione che segue la chiamata. Gli *argomenti* sono i valori di cui la funzione ha bisogno; in RISC-V viaggiano nei registri da `a0` ad `a7`. Il codice che chiama è il *chiamante*, la funzione chiamata è il *chiamato*, e il punto a cui tornare è conservato nel registro `ra`, l'indirizzo di ritorno: per questo il passo 2 salva `ra` prima delle proprie chiamate e lo ripristina prima di `ret`. PRG32 offre funzioni già pronte come `prg32_gfx_clear`; voi ne scrivete tre, che il runtime chiama.

2.1.5 Il contratto: tre funzioni

Un gioco PRG32 sono tre funzioni che il programmatore scrive e il firmware chiama (Figura 2.1). `init` viene chiamata una volta, quando la cartuccia parte, e prepara lo stato del gioco. `update` viene chiamata una volta per frame: legge l'ingresso e cambia lo stato. `draw` viene chiamata una volta per frame: trasforma lo stato in un'immagine. Il firmware trova le tre funzioni per nome: un prefisso scelto dal programmatore, seguito da `_init`, `_update` e `_draw`. Questo è l'intero contratto tra un gioco e la macchina.

2.2 La vostra prima cartuccia

2.2.1 Come leggere una riga di assembly

Prima del primo listato, una guida per leggerlo. Una riga di assembly ha al più quattro parti:

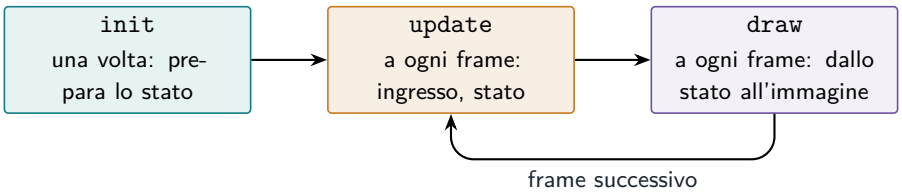


Figura 2.1. Il contratto tra una cartuccia e il runtime.

```
first_update:   addi t1, t1, 1      # one more tick
```

Un'*etichetta*, `first_update:`, è un nome per questo punto del programma e termina con i due punti. L'*istruzione*, `addi`, è l'operazione da eseguire, qui "somma una costante". Gli *operandi*, `t1, t1, 1`, sono ciò su cui lavora: prima la destinazione, poi le sorgenti, quindi la riga si legge "metti in `t1` il valore di `t1` più 1", l'assegnamento $t1 = t1 + 1$. Un *commento* va da `#` a fine riga ed è ignorato dall'assemblatore. Una riga che comincia con un punto, come `.section .text`, è una *direttiva*: un ordine all'assemblatore, non al processore. Righe vuote e rientri contano per chi legge, non per l'assemblatore.

2.2.2 Passo 1: il più piccolo programma che gira

Il Listato 2.1 è una cartuccia completa. Create una cartella, scrivete il programma in un file chiamato `step1_skeleton.S`, e resistete alla tentazione di copiarlo: imparano anche le dita.

```
# Assembly Lab 1: the smallest program that runs
.option norelax

.section .text
.global first_init
.global first_update
.global first_draw

first_init:
    ret

first_update:
    ret

first_draw:
    ret
```

Listato 2.1. Passo 1: tre funzioni che non fanno nulla, correttamente.

Le righe che cominciano con un punto sono *direttive*: ordini per l'assemblatore, non istruzioni per il processore. `.option norelax` mantiene prevedibile la disposizione della cartuccia, e ogni esempio di PRG32 comincia così. `.section .text` dice che ciò che segue è codice eseguibile. `.global` rende un'*etichetta* visibile fuori dal file, così che il firmware possa trovare le funzioni per nome. Un'etichetta, un nome seguito da due punti, sta per l'indirizzo di ciò che la segue. L'unica istruzione del programma, scritta tre volte, è `ret`: ritorna al chiamante.

La cartuccia si compila e si esegue con i comandi incontrati nel Capitolo 1 (Listato 2.2); il file della cartuccia è identico byte per byte per QEMU, per la scheda e per PRG32-QT, dove l'indirizzo è `http://localhost:8080`.

```
# 1. build a portable cartridge from one assembly file
python3 -m prg32 cartridge build step1_skeleton.S --portable \
    --entry-prefix first --name step1 --out build/step1.prg32

# 2a. run it in QEMU on your laptop
python3 -m prg32 qemu upload build/step1.prg32
python3 -m prg32 qemu run

# 2b. or send it to a board, or to PRG32-QT, over the network
python3 -m prg32 esp32c6 upload-and-run build/step1.prg32 \
    --url http://192.168.4.1 --slot cart0
```

Listato 2.2. Compilare ed eseguire una cartuccia.

Eseguite

Il risultato atteso è uno schermo nero. Il risultato è reale lo stesso: il framework ha trovato le tre funzioni, le ha chiamate frame dopo frame, ed è sopravvissuto.

2.2.3 Passo 2: dire ciao

Per mettere qualcosa sullo schermo il programma chiama due funzioni del framework: `prg32_gfx_clear`, che riempie lo schermo con un colore, e `prg32_gfx_text8`, che scrive una stringa con un font di 8 per 8 pixel (Listato 2.3, Figura 2.2). In cima al file due direttive `.equ` danno un nome a due costanti: `.equ BLACK, 0x0000` e `.equ WHITE, 0xffff`.

```
1  first_draw:
2      addi sp, sp, -16       # prologue: a 16-byte frame
3      sw   ra, 12(sp)        # save the return address
4
5      li   a0, BLACK
6      call prg32_gfx_clear
7
8      li   a0, 32            # x
9      li   a1, 40            # y
10     la   a2, title         # address of the string
11     li   a3, WHITE         # foreground
12     li   a4, BLACK         # background
13     call prg32_gfx_text8
14
15     lw   ra, 12(sp)        # epilogue: restore ra
16     addi sp, sp, 16        # release the frame
17     ret
18
19 .section .rodata
20 title:
21     .asciz "HELLO PRG32"
```

Listato 2.3. Passo 2: la nuova `first_draw`.

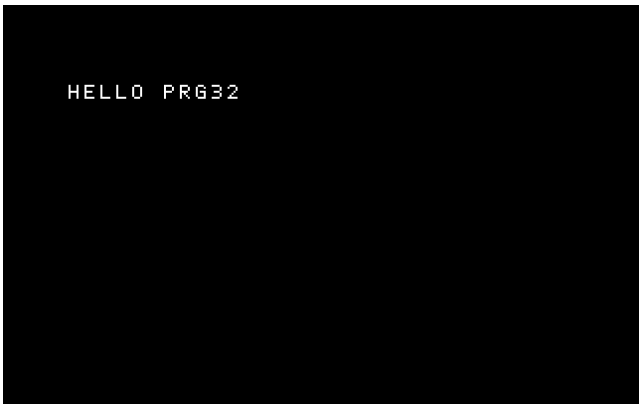


Figura 2.2. Il passo 2, compilato ed eseguito su PRG32.

La regola per passare gli argomenti è semplice: il primo argomento va in `a0`, il secondo in `a1`, e così via. `li` (*load immediate*) mette un numero in un registro; `la` (*load address*) vi mette l'indirizzo di un'etichetta. La stringa vive nella sezione `.rodata`, *read-only data*, dati di sola lettura, e la direttiva `.asciz` ne memorizza i caratteri seguiti da un byte di valore zero che ne segna la fine.

2.2.4 Perché salvare `ra`? Lo stack frame

La prima riga e le ultime tre di `first_draw` sono nuove, e sono il prezzo da pagare per chiamare un'altra funzione. L'istruzione `call` memorizza in `ra` l'indirizzo a cui tornare. Ma `first_draw` è stata a sua volta chiamata dal firmware, e `ra` conteneva già l'indirizzo del *suo* ritorno: la prima `call` lo sovrascriverebbe, e la `ret` finale salterebbe nel posto sbagliato.

Una funzione che ne chiama un'altra salva quindi per prima cosa `ra`. Abbassa `sp` di 16 per fare spazio sullo *stack*, una regione di memoria che ogni funzione può usare per le proprie necessità temporanee, e memorizza `ra` a `12(sp)`, cioè 12 byte sopra l'indirizzo contenuto in `sp` (Figura 2.3). Queste righe sono il *prologo* della funzione. Dopo l'ultima chiamata disfa tutto in ordine inverso, nell'*epilogo*: ricarica `ra`, alza `sp` di 16, e ritorna.

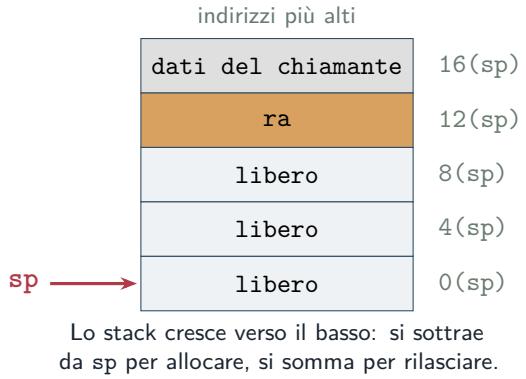


Figura 2.3. La memoria dello stack dopo il prologo di `first_draw`.

Il classico primo errore

Dimenticare la coppia `sw ra`/`lw ra` è il classico primo errore: la funzione ritorna nel posto sbagliato, e il programma va in crash o si blocca molto dopo la riga colpevole.

La convenzione richiede che `sp` resti allineato a un multiplo di 16 byte tra una chiamata e l'altra, ed è per questo che il frame è di 16 byte anche se ne vengono usati solo 4. Lo stack, con push e pop, è trattato formalmente nel Capitolo 18.

2.2.5 Passo 3: un valore che sopravvive tra i frame

I registri non conservano i loro valori tra una chiamata di `update` e la successiva: nel frattempo li usa il firmware. Ciò che deve durare deve vivere in memoria (Listato 2.4).

```
first_update:
    # A leaf function: it calls nothing,
    # so it needs no stack frame.
    la   t0, counter      # t0 = ADDRESS of counter
    lw   t1, 0(t0)        # t1 = VALUE stored there   (load)
    addi t1, t1, 1        # one tick per frame        (modify)
    sw   t1, 0(t0)        # write it back             (store)
    ret

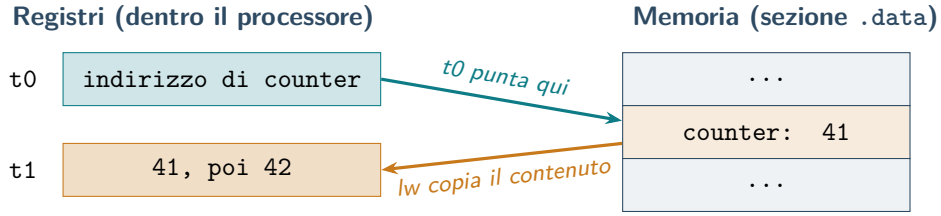
.section .data
counter: .word 0
```

Listato 2.4. Passo 3: la nuova `first_update` e la sua variabile.

`counter` è un'etichetta: dà un nome a una locazione di memoria, nella sezione `.data`, che contiene una parola di 32 bit, inizialmente zero. `la` mette in `t0` l'*indirizzo* di quella locazione. `lw` legge in `t1` il *valore* che si trova a quell'indirizzo; `sw` lo riscrive. Poiché il processore calcola solo nei registri, ogni modifica di una variabile ha gli stessi tre movimenti: *caricare, modificare, memorizzare*. La funzione è una *foglia*: non chiama nulla, quindi non ha bisogno di uno stack frame.

Indirizzo o valore? Il punto cruciale della memoria

Un'etichetta dà un nome a una *locazione*. `la` dà la locazione; `lw` dà il contenuto (Figura 2.4). È così che in ultima analisi funziona ogni variabile di ogni programma. Usare un indirizzo dove serve un valore, o il contrario, è uno degli errori più comuni dei principianti: commentate ogni registro come ADDRESS (indirizzo) o VALUE (valore).




```
la   t0, counter    # t0 = WHERE the value lives
lw   t1, 0(t0)      # t1 = WHAT is stored there
addi t1, t1, 1      # change the copy in the register
sw   t1, 0(t0)      # memory still says 41 until this line
```

Figura 2.4. Il registro `t0` contiene un indirizzo; `t1` contiene un valore.

Dopo questo passo non cambia nulla di visibile: il contatore avanza, ma nessuno lo disegna. Come si potrebbe dimostrare che sta avanzando? Con il debugger di PRG32-QT, oppure con il passo successivo.

2.2.6 Passo 4: rendere visibile il contatore

Otto righe aggiunte a `first_draw`, dopo la chiamata che scrive il testo, disegnano una barra verde la cui larghezza è il contatore (Listato 2.5, Figura 2.5). La costante `GREEN` è definita in cima al file con `.equ GREEN, 0x07e0`.

```
    la   t0, counter
    lw   t1, 0(t0)
    andi a2, t1, 255      # width = counter modulo 256
    li   a0, 32           # x
    li   a1, 60           # y
    li   a3, 8            # height
    li   a4, GREEN        # .equ GREEN, 0x07e0
    call prg32_gfx_rect
```

Listato 2.5. Passo 4: le righe aggiunte a `first_draw`.



Figura 2.5. Passo 4: la barra cresce da 0 a 255 pixel, poi ricomincia.

La funzione `prg32_gfx_rect` prende cinque argomenti: x, y, larghezza, altezza e colore. La larghezza è calcolata da `andi` con 255, che tiene soltanto gli otto bit bassi del contatore: la barra cresce da 0 a 255 pixel e poi ricomincia. In queste otto righe si nascondono tre capitoli futuri: l'uguaglianza 255 = `0xFF` (notazione posizionale), `andi` come maschera di bit (algebra di Boole), e `0x07E0` come colore (la codifica dell'informazione).

Il programma completo, `code/l02.S`, è il Listato 2.6 nel Paragrafo 2.5, con i comandi per compilarlo ed eseguirlo.

2.3 Leggere ciò che avete scritto

2.3.1 Le istruzioni appena usate

La Tabella 2.2 ordina per famiglia le istruzioni incontrate finora. In tutte il risultato va nel *primo* registro scritto. L'insieme di istruzioni intero di base di RISC-V, RV32I, ha una quarantina di istruzioni: un quarto è già stato usato.

Tabella 2.2. Le istruzioni usate in questo capitolo.

Famiglia	Istruzione	Che cosa fa	Esempio
Aritmetica	`add`, `sub`	combina due registri	`add t0, t1, t2`
Logica	`and`, `or`, `xor`	operazione bit a bit su due registri	`and t0, t1, t2`
Immediato	`addi`, `andi`, `ori`, `xori`	un registro e una piccola costante	`addi t1, t1, 1`
Memoria	`lw`, `sw`	carica o memorizza una parola di 32 bit	`lw t1, 0(t0)`
Controllo	`call`, `ret`	va a una funzione e ne ritorna	`call prg32_gfx_rect`

2.3.2 Pseudo-istruzioni

Parecchi degli mnemonici usati non sono affatto istruzioni del processore. Sono *pseudo-istruzioni*: scorciatoie che l'assemblatore espande in una o due istruzioni vere (Tabella 2.3). Una pseudo-istruzione non è un'operazione hardware distinta. È il registro `zero` a renderne possibili molte: un piccolo insieme di istruzioni, riusato con cura.

Tabella 2.3. Le pseudo-istruzioni e le loro espansioni.

Voi scrivete	L'assemblatore emette	Significato
`li t0, 42`	`addi t0, zero, 42`	carica una costante
`mv t0, t1`	`addi t0, t1, 0`	copia un registro
`not t0, t1`	`xori t0, t1, -1`	complemento bit a bit
`la t0, counter`	`auipc t0, ...` + `addi t0, t0, ...`	carica l'indirizzo di un'etichetta
`ret`	`jalr zero, 0(ra)`	salta all'indirizzo contenuto in `ra`
`beqz t0, skip`	`beq t0, zero, skip`	salta se `t0` vale zero

2.3.3 Un'istruzione è un numero di 32 bit

Il processore non legge testo. L'assemblatore traduce ogni riga in una parola di 32 bit, ed è quella parola a essere memorizzata nella cartuccia ed eseguita. La Figura 2.6 seziona `addi t1, t1, 1`: dodici bit contengono la costante, cinque bit indicano il registro sorgente (`t1` è `x6`), tre bit selezionano l'operazione, cinque bit indicano il registro di destinazione e sette bit, l'*opcode*, dicono di che tipo di istruzione si tratta. Scritti in esadecimale, i trentadue bit sono `0x00130313`.

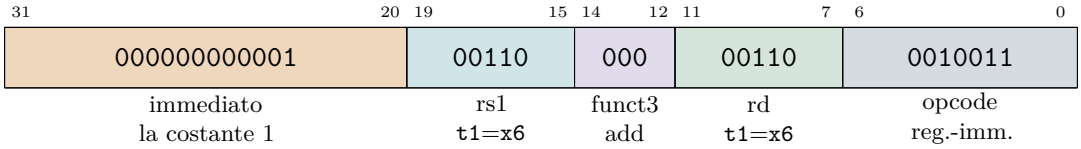


Figura 2.6. `addi t1, t1, 1` è il numero `0x00130313`.

Concetto

L'assembly è una scrittura leggibile del codice macchina. Un'istruzione è un dato: il processore ne legge i campi con circuiti logici, che verranno progettati nei capitoli sulla logica e sul processore.

2.3.4 PRG32-QT: un microscopio per i registri

PRG32-QT esegue la stessa cartuccia sul desktop un'istruzione alla volta, si ferma tra due istruzioni qualunque e mostra ogni registro.

Eseguite passo-passo `first_update`

Caricate il passo 3 in modalità debug, mettete un breakpoint su `first_update` e premete **Step** quattro volte. Prima di ogni pressione, dite che cosa conterranno `t0` e `t1`; dopo, verificate.

2.3.5 Tracciare a mano

Lo stesso esercizio si può, e si deve, fare su carta. La Tabella 2.4 segue `first_update` per alcuni frame. Al frame 256 il contatore continua a crescere, ma i suoi otto bit bassi tornano a zero, perché $256 = 1\,0000\,0000_2$: la barra ricomincia. Tracciare a mano è la più preziosa abilità di debug insegnata in questo libro.

Tabella 2.4. Tre frame, e uno in più, di `first_update`.

Frame	`lw`: `t1` diventa	`addi`: `t1` diventa	`sw`: counter diventa	Larghezza
1	0	1	1	1
2	1	2	2	2
256	255	256	256	0
257	256	257	257	1

2.4 Che cosa viene dopo

Il programma di questo capitolo ha sollevato più domande di quante ne abbia risolte. La Tabella 2.5 è la mappa in avanti del libro: ogni capitolo successivo si apre indicandone una riga.

2.5 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l02.S` nel repository del corso, stampato per intero nel Listato 2.6. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 2.7 è il frame scritto dal test.

Sullo schermo.
HELLO PRG32 e, sotto, una barra verde che cresce di un pixel per frame e ricomincia a 256.

Che cosa premere.
Niente: il programma va da solo.

```
# ARC12 Computer Architecture - Lecture 02: First Contact: RISC-V Assembly on
    PRG32
# l02.S: the four steps of the lecture in one file
```

Tabella 2.5. Le domande che il primo programma ha già sollevato.

Avete visto	La domanda	Risposta in
`0x07e0`, `0xffff`, 255	Perché i programmatori scrivono i numeri in esadecimale?	Capitoli 4, 5
una posizione sotto lo zero mostrata come `0xFFFFFFFF`	Come fa un registro a contenere un numero negativo?	Capitolo 5
una palla che si muove di meno di un pixel per frame	Come si rappresentano le frazioni senza un'unità in virgola mobile?	Capitolo 6
`andi a2, t1, 255`	Che cosa fa AND a trentadue bit tutti insieme?	Capitoli 8, 9
`addi t1, t1, 1`	Quale circuito somma davvero due numeri?	Capitolo 9
un pulsante premuto è un 1	Di che cosa è fatto, fisicamente, un bit?	Capitolo 10
`counter` sopravvive tra i frame	Come fa un circuito a ricordare un bit?	Capitoli 13, 15
`0x00130313`	Come fa il processore a decodificarla ed eseguirla?	Capitoli 18, 19
tre funzioni e un rettangolo	Se ne può fare un gioco intero?	Verm1: Capitoli 7, 12, 17, 22

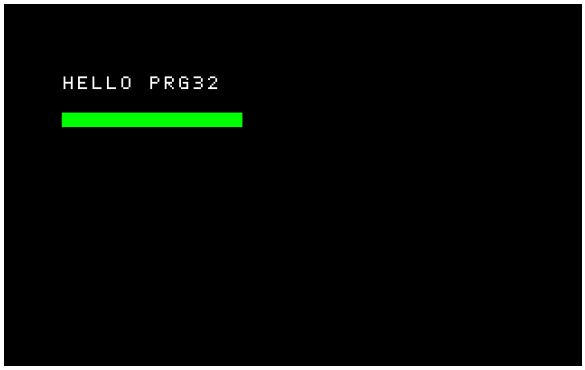


Figura 2.7. `l02.S` su PRG32-QT.

```
3  #
4  # On the screen: HELLO PRG32 and, under it, a green bar that grows by one pixel
       per
5  # frame and starts again at 256.
6  # Build: python3 -m prg32 cartridge build l02.S --portable \
7  #            --entry-prefix first --name first --out first.prg32
8  .option norelax
9
10 .equ BLACK, 0x0000
11 .equ WHITE, 0xffff
12 .equ GREEN, 0x07e0
13
14 .section .text
15 .global first_init
16 .global first_update
17 .global first_draw
18
19 first_init:                  # step 1: a function that returns
20     ret
```

```
21
22 first_update:                # step 3: a value that survives between frames
23     # A leaf function: it calls nothing, so it needs no stack frame.
24     la   t0, counter         # t0 = ADDRESS of counter
25     lw   t1, 0(t0)           # t1 = VALUE stored there   (load)
26     addi t1, t1, 1           # one tick per frame        (modify)
27     sw   t1, 0(t0)           # write it back             (store)
28     ret
29
30 first_draw:                  # step 2: say hello
31     addi sp, sp, -16         # prologue: a 16-byte frame
32     sw   ra, 12(sp)          # save the return address
33     li   a0, BLACK
34     call prg32_gfx_clear
35     li   a0, 32              # x
36     li   a1, 40              # y
37     la   a2, title           # address of the string
38     li   a3, WHITE           # foreground
39     li   a4, BLACK           # background
40     call prg32_gfx_text8
41
42     la   t0, counter         # step 4: make the counter visible
43     lw   t1, 0(t0)
44     andi a2, t1, 255         # width = counter modulo 256
45     li   a0, 32              # x
46     li   a1, 60              # y
47     li   a3, 8               # height
48     li   a4, GREEN
49     call prg32_gfx_rect
50
51     lw   ra, 12(sp)          # epilogue: restore ra
52     addi sp, sp, 16          # release the frame
53     ret
54
55 .section .rodata
56 title:
57     .asciz "HELLO PRG32"
58
59 .section .data
60 counter: .word 0
```

Listato 2.6. Il programma completo del capitolo, `code/l02.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 2.7; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l02.S --portable \
    --entry-prefix first --name l02 --out l02.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l02.prg32 \
    --url http://localhost:8080 --slot cart0
```

```
# 4. look, press, compare with the previous slide
```

Listato 2.7. Compilare `102.S` ed eseguirlo su PRG32-QT.

2.6 Verificate di aver capito

D2.1 Dopo `la t0, counter` e `lw t1, 0(t0)`, che cosa contiene `t0`?

A. Il valore del contatore
B. L'indirizzo del contatore
C. Zero, perché `la` azzera il registro
D. L'indirizzo di ritorno

D2.2 Una funzione che chiama `prg32_gfx_clear` deve salvare un registro prima della chiamata. Quale?

A. `a0`, perché porta l'argomento
B. `sp`, perché lo stack si sposta
C. `ra`, perché `call` lo sovrascrive
D. `t0`, perché è temporaneo

D2.3 `t1` contiene 300. Che cosa lascia in `a2` l'istruzione `andi a2, t1, 255`?

A. 300
B. 255
C. 45
D. 44

D2.4 Quale numero è `0x1F`?

A. il numero decimale 1F
B. 31: un sedici più quindici
C. il registro t1F
D. un commento

2.7 Esercizi

Esercizio 2.1 (su PRG32)

1. Cambiate la stringa del titolo e il suo colore in giallo (`0xffe0`); eseguite di nuovo.
2. Fate crescere la barra a velocità doppia, poi quadrupla, cambiando una sola istruzione.
3. Fate accorciare la barra invece di farla crescere. Che cosa succede quando il contatore passa per lo zero?
4. In PRG32-QT eseguite passo-passo `first_update` e annotate `t0` e `t1` dopo ogni istruzione.
5. Disegnate una seconda barra, rossa, la cui larghezza è il contatore modulo 64.

Risultato atteso. Nel terzo compito il contatore passa da 0 a `0xFFFFFFFF`: con `andi 255` la barra salta a 255 e poi si accorcia. Conservate l'osservazione: la spiega il Capitolo 5.

2.8 Glossario

Inglese	**Italiano**
register	registro
load / store	caricare / memorizzare
address	indirizzo
immediate	immediato (costante)
branch	salto condizionato
label	etichetta
stack	pila (stack)
stack pointer	puntatore alla pila
return address	indirizzo di ritorno
calling convention	convenzione di chiamata
machine code	codice macchina
cartridge	cartuccia

2.9 Riepilogo e letture

Un processore RISC-V sono trentadue registri e un piccolo insieme di istruzioni che calcolano, spostano e saltano. Un gioco PRG32 sono tre funzioni; gli argomenti viaggiano nei registri da `a0` ad `a7`; una funzione che ne chiama un'altra salva `ra`. Le variabili vivono in memoria: `la` per l'indirizzo, `lw` e `sw` per il valore. Un'istruzione è un numero di 32 bit; l'assembly ne è la scrittura leggibile.

Letture. *Playing with RISC-V*, capitolo 5, «First Contact» (Montella et al., 2026b); Patterson e Hennessy (2020), capitolo 2; il tutorial `docs/learn/tutorial_ascii_game.md` del repository di PRG32.

Parte II

Informazione

CAPITOLO 3

Storia della computazione

"printf("Hello, world!");"
— Brian Kernighan e Dennis Ritchie, *The C Programming Language*, 1978

In questo capitolo

Alla fine di questo capitolo saprete

- definire l'informatica e dire perché non è la scienza dei calcolatori;
- collocare su una linea del tempo le macchine e le persone chiave, dall'abaco al calcolatore a programma memorizzato;
- nominare le quattro unità dell'architettura di von Neumann e ritrovarle nella macchina programmata nel Capitolo 2;
- spiegare che cosa significa RISC e perché RISC-V è un insieme di istruzioni aperto;
- descrivere dove si colloca l'informatica tra scienza, ingegneria ed economia.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI che cos'è l'informatica; l'architettura di von Neumann; RISC e un ISA aperto.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI l'algebra di Boole (Capitolo 8).

Il lettore ha già eseguito un programma RISC-V. Questo capitolo racconta da dove viene quella macchina. Non è una cronaca: delle migliaia di macchine e di persone che la storia della computazione potrebbe elencare, conserva quelle che hanno aggiunto un'idea che si trova ancora dentro l'ESP32-C6 sulla breadboard. Il filo da seguire è che, a ogni tappa, qualcuno ha trovato un modo nuovo di fare una di due cose: *rappresentare* l'informazione in un oggetto fisico, oppure *prescrivere* una sequenza di operazioni da eseguire senza doverci più pensare.

3.1 Che cos'è l'informatica?

Definizione 3.1 (Informatica)

L'informatica è la scienza applicata che si occupa del trattamento dell'informazione mediante procedure automatizzate.

La parola *informatica* viene dal francese *informatique*, contrazione di *information automatique*. L'inglese dice *computer science* e, sempre più spesso, *informatics*. Lo strumento della disciplina si chiama *elaboratore* o *calcolatore* in italiano, *ordinateur* in francese, *computer* in inglese. Il nome inglese della disciplina ha il difetto di mettere lo strumento al posto dell'oggetto di studio, un difetto che una frase attribuita a Edsger W. Dijkstra corregge:

> *L'informatica non riguarda i calcolatori più di quanto l'astronomia riguardi i telescopi.*

La macchina è lo strumento, non l'oggetto di studio. In questo libro lo strumento viene studiato da vicino proprio perché smetta di essere un mistero.

La disciplina ha due anime. L'*informatica teorica* studia i fondamenti dell'informazione e della computazione a livello logico: che cosa si può calcolare, e a quale costo. Il suo oggetto centrale è l'*algoritmo*, un insieme finito di operazioni che descrive la trasformazione di un dato A in un risultato B. L'*informatica applicata* studia le tecnologie abilitanti e i sistemi informativi costruiti con esse. È pervasiva, è un motore dello sviluppo economico e dell'intero settore delle tecnologie dell'informazione e della comunicazione, è una disciplina STEM al centro di una rivoluzione industriale; e ha un costo sociale che va governato, il *divario digitale* tra chi ha accesso a queste tecnologie e chi non lo ha.

3.2 Prima dell'elettricità

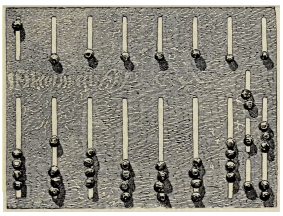
Da tremila anni si costruiscono macchine che aiutano a calcolare.

3.2.1 Le origini: l'abaco e il meccanismo di Anticitera

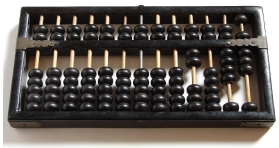
Babilonesi, Cinesi, Greci e Romani usavano tutti dei pallottolieri (Figura 3.1). In un abaco un numero *è* una posizione di palline, e un'addizione *è* un movimento delle dita: il dispositivo è una rappresentazione fisica di dati e di operazioni. Il meccanismo di Anticitera, recuperato da un relitto e costruito tra il secondo e il primo secolo avanti Cristo (la maggior parte delle stime cade tra il 205 e il 60 a.C. circa), è il più antico calcolatore a ingranaggi conosciuto: i suoi quadranti calcolavano posizioni astronomiche ed eclissi, e in esso un numero è l'angolo di una ruota dentata.

Concetto

La rappresentazione viene prima del calcolo. Prima di poter calcolare qualcosa, un oggetto fisico deve stare per un numero.



(a) Abaco romano tascabile



(b) *Suanpan* cinese



(c) Meccanismo di Antikythera



(d) Un modello moderno dei quadranti

Figura 3.1. Le origini: un numero diventa una posizione di palline o l'angolo di un ingranaggio.

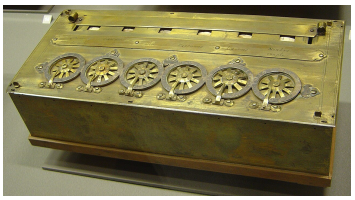
3.2.2 Il mondo islamico medievale

Tra il nono e il tredicesimo secolo gli studiosi del mondo islamico hanno conservato la scienza dell'antichità e vi hanno aggiunto tre cose che riguardano questa storia. La prima è una famiglia di strumenti astronomici, l'astrolabio meccanico (Figura 3.2a) e il torquetum, che sono calcolatori analogici per le posizioni degli astri. La seconda è la crittoanalisi: nel nono secolo al-Kindi descrisse l'analisi delle frequenze, il metodo per violare un cifrario contando quante volte compare ciascun simbolo. La terza è la macchina programmabile: il suonatore di flauto automatico dei fratelli Banū Mūsā, e gli orologi e gli automi a pesi di al-Jazari (1206), il cui comportamento si poteva cambiare spostando dei pioli.

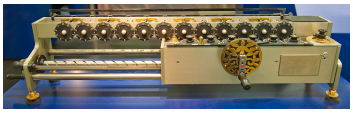
La stessa parola *algoritmo* viene dal nome del matematico al-Khwārizmī, come *algebra* viene dal titolo del suo libro.



(a) Un astrolabio di ottone



(b) Una Pascalina



(c) La calcolatrice di Leibniz

Figura 3.2. Strumenti e calcolatrici prima del 1700.

3.2.3 Dal 1600 al 1800: calcolatrici meccaniche e logica

Il Seicento costruì le prime calcolatrici meccaniche. Le tavole dei logaritmi di Nepero (1614) trasformarono le moltiplicazioni in addizioni. La *Pascalina* di Blaise Pascal (1642) sommava e sottraeva, e la sua vera invenzione fu il *riporto* automatico: ogni ruota è una cifra decimale (Figura 3.2b), e una ruota che passa da 9 a 0 spinge avanti di uno scatto la vicina, esattamente come farà con i bit il sommatore completo del Capitolo 11. La calcolatrice a cilindri scalati di Leibniz (1673) eseguiva le quattro operazioni.

Lo stesso Leibniz descrisse l'aritmetica binaria nel 1703, e un secolo e mezzo più tardi George Boole, in *The Laws of Thought* (1854), mostrò che la logica si può trattare come un'algebra (Boole, 1854): l'argomento dei Capitoli 8 e 9.

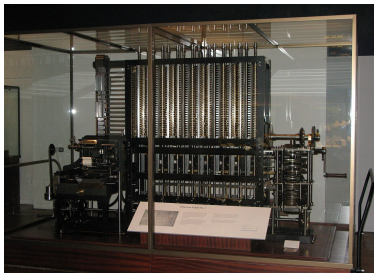
Nel frattempo una macchina che non aveva nulla a che fare con i numeri introdusse il programma memorizzato. Nel telaio di Jacquard (1804) il disegno del tessuto è registrato

su una catena di schede perforate: si cambiano le schede, e lo stesso telaio tesse un disegno diverso.

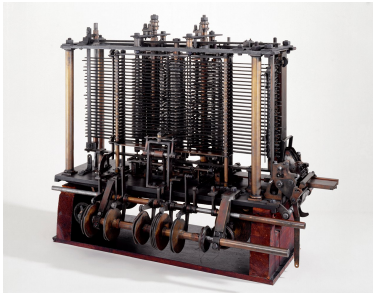
3.2.4 L'Ottocento: Charles Babbage e Ada Lovelace

Charles Babbage unì le due linee (Figura 3.3). La sua *Difference Engine*, la macchina alle differenze, calcolava automaticamente tavole matematiche. La sua *Analytical Engine*, la macchina analitica, progettata ma mai completata, era un calcolatore di uso generale in tutto tranne che nella tecnologia: aveva un *mill*, il mulino, cioè l'unità aritmetica; un'unità di controllo; uno *store*, il magazzino, cioè la memoria; e un dispositivo di uscita. Programmi e dati dovevano essere forniti su schede perforate, prese in prestito dal telaio di Jacquard.

Ada Lovelace ideò un metodo per programmare la macchina analitica. Le sue note del 1843 contengono quello che è considerato il primo programma pubblicato, per il calcolo dei numeri di Bernoulli, e lei è considerata la prima programmatrice della storia.



(a) Difference Engine



(b) Analytical Engine



(c) Schede perforate

Figura 3.3. Le macchine di Charles Babbage.

3.3 Il Novecento

Il Novecento ha prodotto l'idea di una macchina universale, e poi le macchine che l'hanno resa reale. La teoria è venuta prima.

3.3.1 Turing, Shannon e Nakajima

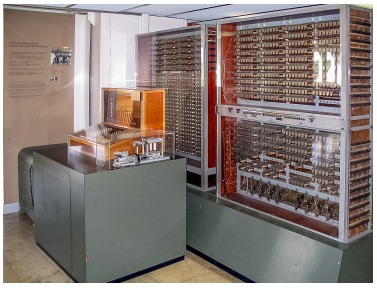
Nel 1936 Alan Turing definì una macchina astratta universale, e con essa la nozione di ciò che si può calcolare (Turing, 1936). Durante la guerra lavorò a Bletchley Park alla violazione del cifrario tedesco Enigma (Figura 3.4a).

Negli stessi anni Akira Nakajima in Giappone e Claude Shannon negli Stati Uniti fondarono la teoria dei circuiti a commutazione: l'algebra di Boole descrive i circuiti fatti di interruttori (Shannon, 1938). È il principio su cui poggia tutta la progettazione digitale, e il ponte tra la logica dei Capitoli 8–9 e i circuiti dei Capitoli 10–11.

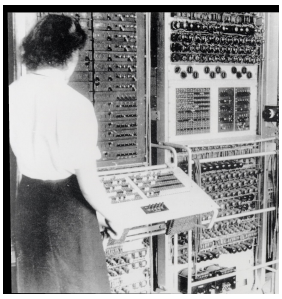
In Germania Konrad Zuse costruì lo Z3 (1941), il primo calcolatore funzionante controllato da un programma, di cui nel 1998 è stata dimostrata la completezza secondo Turing; il suo successore, lo Z4, fu uno dei primi calcolatori venduti commercialmente.



(a) Una macchina Enigma



(b) Zuse Z3 (replica)



(c) Colossus

Figura 3.4. Macchine degli anni Quaranta.

3.3.2 John von Neumann: il calcolatore a programma memorizzato

Nel 1945 il rapporto sull'EDVAC, firmato da John von Neumann, descrisse un'architettura hardware per calcolatori digitali programmabili (von Neumann, 1945). Il suo cuore è un'idea sola.

Il programma memorizzato

Programma e dati condividono la stessa memoria: *un programma è un dato.* Le istruzioni sono numeri conservati in memoria, prelevati ed eseguiti uno dopo l'altro.

L'architettura ha quattro unità essenziali (Figura 3.5): un'*unità di controllo*, che governa la sequenza delle operazioni; un'*unità aritmetico-logica* (ALU), con il suo registro accumulatore; una *memoria*; e dispositivi di *ingresso* e di *uscita*.

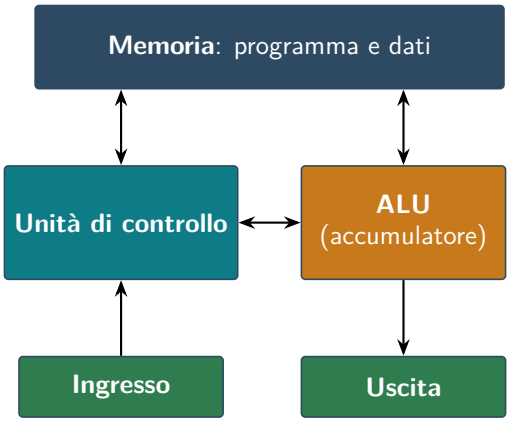


Figura 3.5. Le quattro unità dell'architettura di von Neumann.

Ritorno alla macchina

L'ESP32-C6 è ancora una macchina di von Neumann, e le sue quattro unità erano tutte al lavoro nel Capitolo 2. I registri e la ALU hanno eseguito `addi t1, t1, 1`; l'unità di controllo ha deciso che l'istruzione successiva era `sw`; la memoria conteneva la variabile `counter` *e le istruzioni stesse*, tra cui il numero `0x00130313`; il joystick e il display sono ingresso e uscita.

3.3.3 Otto decenni di calcolatori

Da allora ogni decennio ha reso il calcolatore più piccolo, più economico e più vicino alle persone (Tabella 3.1, Figura 3.6); nessuno ha cambiato l'idea del programma memorizzato.

Tabella 3.1. Otto decenni di calcolatori.

Decennio	Epoca	Che cosa è cambiato
Anni '40	La guerra	ENIAC (USA) e Colossus (Regno Unito), 1943–45
Anni '50	Militari e governi	EDVAC, 1949: il programma memorizzato
Anni '60	I mainframe	una macchina, molti utenti
Anni '70	I minicomputer	un calcolatore per reparto
Anni '80	Il personal computer	un calcolatore per persona
Anni '90	Internet	i calcolatori si collegano
Anni 2000	Smartphone e tablet	un calcolatore in tasca
Anni 2010	Il cloud computing	il calcolo come servizio
Anni 2020	Internet delle cose e IA	calcolatori ovunque



(a) ENIAC **(b)** Un mainframe **(c)** Un minicomputer **(d)** Commodore 64

Figura 3.6. Da una stanza a una scrivania in quarant'anni.

3.4 Dagli home computer a RISC-V

3.4.1 I microprocessori: da CISC a RISC a uno standard aperto

Il processore del kit PRG32 è fatto com'è per una sequenza di decisioni prese nell'arco di cinquant'anni (Tabella 3.2). I primi microprocessori sono cresciuti per accumulo: ogni generazione aggiungeva istruzioni, spesso complicate, per facilitare il lavoro dei compilatori e dei programmatori in assembly. Il risultato si chiama *complex instruction set computer* (CISC), e la famiglia x86, nata nel 1978, porta ancora questa eredità nei personal computer di oggi.

All'inizio degli anni Ottanta due progetti universitari, a Berkeley e a Stanford, misurarono che cosa fanno davvero i programmi, e scoprirono che un processore con istruzioni poche, semplici e regolari può essere reso più veloce di uno con molte istruzioni complicate. È l'idea del *reduced instruction set computer* (RISC), per la quale David Patterson e John Hennessy hanno ricevuto il premio Turing nel 2017. ARM l'ha applicata al basso consumo, e i core ARM sono oggi in quasi ogni telefono. Nel 2010 il quinto progetto RISC di Berkeley è diventato qualcosa di nuovo: un insieme di istruzioni pubblicato come *standard aperto*.

Tabella 3.2. I microprocessori: da CISC a RISC a uno standard aperto.

Anno	Tappa	Perché conta
1971	Intel 4004	il primo microprocessore commerciale: 2 300 transistor su un chip
1975	MOS 6502	una CPU a 8 bit economica: Apple II, Commodore 64, l'era degli home computer
1978	Intel 8086	l'inizio di x86, un insieme di istruzioni complesso (CISC) ancora nei PC di oggi
1980–82	RISC di Berkeley e MIPS di Stanford	istruzioni poche, semplici e regolari vanno più veloci: l'idea RISC
1985	ARM1	RISC per il basso consumo: oggi in quasi ogni telefono
2010	RISC-V nasce a UC Berkeley	il quinto progetto RISC di Berkeley diventa un insieme di istruzioni aperto
2015–20	RISC-V Foundation, poi RISC-V International	una specifica condivisa, governata dalla Svizzera
2023–25	European Chips Act; il progetto DARE	RISC-V come strategia per la sovranità tecnologica europea

Definizione 3.2 (RISC-V)

RISC-V è una *instruction set architecture*, un'architettura dell'insieme di istruzioni, a standard aperto: la specifica delle istruzioni, dei registri e della loro codifica è pubblica, e chiunque può progettare, costruire e vendere un processore che la realizza senza pagare una licenza per l'architettura.

RISC-V quindi non è un processore, e non è stato il primo RISC: il RISC di Berkeley, MIPS e ARM sono venuti prima. Ciò che lo distingue da x86 e da ARM è l'apertura.

3.4.2 La legge di Moore in un numero

L'Intel 4004 del 1971 aveva 2 300 transistor. Un processore per portatili di oggi ne ha decine di miliardi: una crescita di un fattore di circa dieci milioni. Per cinquant'anni il numero di transistor su un chip è raddoppiato circa ogni due anni, un'osservazione fatta da Gordon Moore nel 1965 e rivista nel 1975 (Moore, 1965). È un'osservazione su un'industria, non una legge di natura, e i suoi limiti fisici sono ormai vicini. Il progresso viene sempre più da architetture migliori, da acceleratori specializzati e da progetti aperti, ed è per questo che l'architettura, l'argomento di questo libro, conta più che mai. Il Capitolo 24 riprende la storia da qui.

3.4.3 Lo spirito degli home computer, su una CPU moderna e aperta

Gli home computer degli anni Ottanta avevano una qualità che le macchine successive hanno perso: una persona poteva capirne uno per intero. PRG32 è stato progettato per restituire quella qualità, sull'architettura che gli studenti incontreranno nell'industria (Tabella 3.3). Lo schermo ha la stessa dimensione, e la stessa immediatezza, a quarant'anni di distanza.

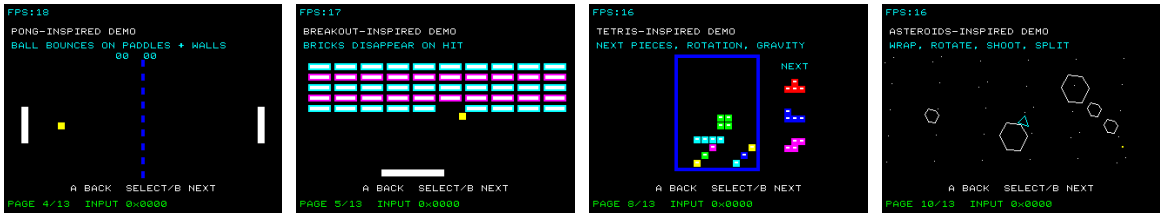
Il 6502 è ancora un piacere da esplorare, per esempio con il simulatore su https://skilldrick.github.io/easy6502.

Tabella 3.3. Un Commodore 64 e PRG32.

	Commodore 64 (1982), MOS 6510	**PRG32 su ESP32-C6, RISC-V**
Dimensione della parola	8 bit	32 bit
Registri	A, X, Y (più puntatore allo stack, stato, PC)	32 registri di uso generale, da `x0` a `x31`
Clock	circa 1 MHz	160 MHz
Spazio di indirizzamento	64 KB	4 GB
Insieme di istruzioni	56 istruzioni, proprietario	una quarantina di istruzioni di base più le estensioni, standard aperto
Schermo	320 per 200 pixel, 16 colori	viewport di 320 per 200, palette RGB565 di 256 voci
Si programma	scrivendo sulla macchina stessa	compilando una cartuccia e caricandola

3.4.4 I giochi rétro come esemplari

Un biologo studia il moscerino della frutta non perché i moscerini siano importanti, ma perché un moscerino è abbastanza piccolo da essere capito per intero e abbastanza ricco da mostrare come funziona l'ereditarietà. Un gioco rétro svolge qui la stessa parte (Figura 3.7): è abbastanza piccolo da tenerlo in mente e abbastanza ricco da mettere in esercizio stato, ingresso, aritmetica, decisioni e un ciclo, cioè l'intero corso. I quattro giochi della figura sono cartucce di esempio del repository di PRG32 (`examples/games`).



(a) Pong **(b)** Breakout **(c)** Tetris **(d)** Asteroids

Figura 3.7. I giochi rétro come esemplari.

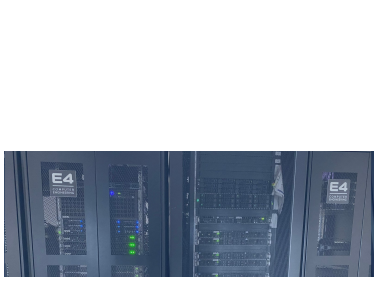
3.5 L'informatica come scienza applicata

Dove si colloca l'informatica tra le scienze? Un esempio dal Dipartimento in cui questo libro è stato scritto rende concreta la risposta (Figura 3.8). Sul mare si forma una tempesta: un fenomeno naturale. La fisica e la matematica, le scienze di base, lo descrivono con delle equazioni. L'informatica, una scienza applicata, trasforma le equazioni in algoritmi e simulazioni. L'ingegneria costruisce i supercalcolatori, le reti e i sensori che le eseguono in modo affidabile. E l'economia riceve previsioni e servizi che raggiungono le persone e creano valore. Le previsioni meteorologiche e marine di https://meteo.uniparthenope.it sono calcolate ogni giorno in questo modo, sui sistemi ad alte prestazioni del Dipartimento.

Le stesse idee che in questo libro si esercitano su un microcontrollore a 160 MHz scalano fino ai cluster (Figura 3.9): registri, memoria, istruzioni, solo in numero molto maggiore.



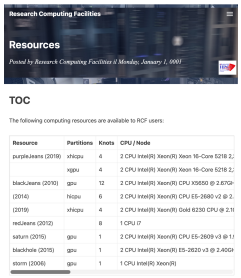
Figura 3.8. Dove si colloca l'informatica.



(a) Gli armadi



(b) Previsioni meteo



(c) rcf.uniparthenope.it

Figura 3.9. Il calcolo ad alte prestazioni alla Parthenope: gli armadi delle Research Computing Facilities, le previsioni di meteo@uniparthenope che calcolano ogni giorno, e la pagina che elenca le macchine.

3.6 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l03.S` nel repository del corso, stampato per intero nel Listato 3.1. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 3.10 è il frame scritto dal test.

Sullo schermo.
Una riga di 32 lampade che mostra 0x00130313, il codice macchina di addi t1, t1, 1, e un quadrato verde: la parola letta è quella attesa.

Che cosa premere.
Niente: il programma va da solo.

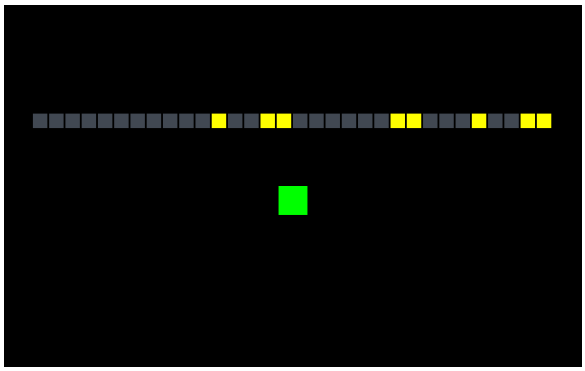


Figura 3.10. `l03.S` su PRG32-QT.

```
1 # ARC12 Computer Architecture - Lecture 03: History of Computing
2 # l03.S: the stored program: a program that reads one of its own instructions
      as data
3 #
```

```
# On the screen: a row of 32 lamps showing 0x00130313, the machine code of addi
    t1,
# t1, 1, and a green square: the word read is the one expected.
# Build: python3 -m prg32 cartridge build l03.S --portable \
#            --entry-prefix l03 --name l03 --out l03.prg32
.option norelax

.equ BLACK,  0x0000
.equ GREY,   0x4208
.equ YELLOW, 0xffe0
.equ GREEN,  0x07e0
.equ RED,    0xf800

.section .text
.global l03_init
.global l03_update
.global l03_draw

l03_init:
    ret

l03_update:
    ret

# One instruction of the program.  For von Neumann's machine it is a number in
# the same memory as the data: the program can load it like any other word.
.option push
.option norvc                # keep it 32 bits long: no compressed form
specimen:
    addi t1, t1, 1
.option pop
    ret

l03_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    la   t0, specimen        # the ADDRESS of an instruction ...
    lw   s0, 0(t0)           # ... and the instruction, read as DATA
    mv   a0, s0
    li   a1, 60
    call lamps32             # show its 32 bits
    li   t0, 0x00130313      # what the assembler should have produced
    li   a4, RED
    bne  s0, t0, 1f
    li   a4, GREEN           # equal: instructions are numbers
1:  li   a0, 152
    li   a1, 100
    li   a2, 16
    li   a3, 16
    call prg32_gfx_rect
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
```

```
59      ret
60
61  # lamps32: show the 32 bits of a word as a row of lamps, bit 31 on the left.
62  #   a0 = the word, a1 = y of the row.  A lamp is yellow for 1, grey for 0.
63  lamps32:
64      addi sp, sp, -32
65      sw   ra, 28(sp)
66      sw   s0, 24(sp)
67      sw   s1, 20(sp)
68      sw   s2, 16(sp)
69      mv   s0, a0             # the word
70      mv   s1, a1             # y
71      li   s2, 0              # lamp number, 0 on the left
72  1:  li   t0, 31
73      sub  t0, t0, s2         # lamp i shows bit 31 - i
74      srl  t1, s0, t0         # bring that bit down to bit 0
75      andi t1, t1, 1          # keep it, clear all the others
76      li   a4, GREY           # lamp off
77      beqz t1, 2f
78      li   a4, YELLOW         # lamp on
79  2:  slli a0, s2, 3
80      add  a0, a0, s2         # 9 * i
81      addi a0, a0, 16         # x = 16 + 9 * i
82      mv   a1, s1
83      li   a2, 8
84      li   a3, 8
85      call prg32_gfx_rect
86      addi s2, s2, 1
87      li   t0, 32
88      blt  s2, t0, 1b
89      lw   s2, 16(sp)
90      lw   s1, 20(sp)
91      lw   s0, 24(sp)
92      lw   ra, 28(sp)
93      addi sp, sp, 32
94      ret
```

Listato 3.1. Il programma completo del capitolo, `code/l03.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 3.2; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l03.S --portable \
    --entry-prefix l03 --name l03 --out l03.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l03.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 3.2. Compilare `l03.S` ed eseguirlo su PRG32-QT.

3.7 Verificate di aver capito

D3.1 Quale idea è il cuore dell'architettura di von Neumann?

A. Il processore ha esattamente quattro registri
B. Programma e dati sono memorizzati nella stessa memoria
C. Ogni istruzione richiede un ciclo di clock
D. Ingresso e uscita sono facoltativi

D3.2 Che cosa distingue RISC-V da x86 e ARM?

A. È l'insieme di istruzioni più veloce
B. È stata la prima architettura RISC
C. Il suo insieme di istruzioni è uno standard aperto, libero da realizzare
D. Funziona solo sui microcontrollori

D3.3 Chi è considerato il primo programmatore della storia?

A. Charles Babbage
B. Alan Turing
C. Ada Lovelace
D. Konrad Zuse

3.8 Esercizi

Esercizio 3.1 (prima su carta, poi su PRG32)

1. Disegnate le quattro unità dell'architettura di von Neumann e scrivete, dentro ciascuna, una cosa del programma del Capitolo 2 che vi risiede o vi accade.
2. Per ciascuna delle seguenti cose, dite se è un modo nuovo di *rappresentare* l'informazione o un modo nuovo di *prescrivere* operazioni: l'abaco, il telaio di Jacquard, la Pascalina, l'algebra di Boole, le schede perforate della macchina analitica, il programma memorizzato.
3. Scaricate dallo Store due dei quattro giochi della Figura 3.7, oppure compilateli da `examples/games`. Con i contatori del debugger, trovate quante istruzioni esegue ciascuno per frame, e confrontate i due giochi.

Risultato atteso. Registri e ALU: lì è stata eseguita `addi`. Memoria: il contatore, e le istruzioni stesse. Unità di controllo: il prelievo di ogni istruzione. Ingresso/uscita: la barra sullo schermo. Il programma memorizzato appartiene a entrambe le classi: un programma è esso stesso informazione rappresentata.

3.9 Glossario

Inglese	**Italiano**
computer science / informatics	informatica
computer	calcolatore, elaboratore
punched card	scheda perforata
loom	telaio
mainframe	mainframe, grande elaboratore
stored program	programma memorizzato
control unit	unità di controllo
arithmetic logic unit	unità aritmetico-logica
instruction set	insieme (set) di istruzioni
high-performance computing	calcolo ad alte prestazioni

3.10 Conclusioni e letture

La computazione è più antica dell'elettronica: rappresentazione e procedura sono venute prima. Il calcolatore a programma memorizzato del 1945 è ancora la macchina che si programma oggi. RISC ha reso i processori abbastanza semplici da essere capiti per intero; RISC-V li ha resi aperti. L'informatica è pervasiva: l'informazione digitale appartiene a ogni campo, i dati ne sono la moneta, e gli algoritmi trasformano i dati in decisioni.

Letture. Patterson e Hennessy (2020), capitolo 1; *Playing with RISC-V*, capitolo 1, «Why a Game, Why RISC-V, Why Now» (Montella et al., 2026b); Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*; per la storia in profondità, Ceruzzi (2003).

CAPITOLO 4

Rappresentare l'informazione digitale

"printf("Hello, world!");"
— Brian Kernighan e Dennis Ritchie, *The C Programming Language*, 1978

In questo capitolo

Alla fine di questo capitolo saprete

- spiegare la differenza tra informazione analogica e digitale;
- leggere una configurazione di bit come numero naturale, in binario e in esadecimale;
- decodificare un colore di PRG32 come `0x07E0` nei suoi campi rosso, verde e blu;
- dire come caratteri e stringhe sono memorizzati, in ASCII e in UTF-8;
- descrivere come un vettore vive in memoria e come l'assembly ne raggiunge un elemento;
- distinguere un algoritmo deterministico da uno probabilistico, e collocare tra essi l'apprendimento automatico.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI analogico e digitale; bit, byte, parola; l'esadecimale; la notazione posizionale; un colore è un numero; caratteri e stringhe; l'ordine little-endian; le strutture dati; gli array in memoria; gli algoritmi.

DA UTILIZZARE OGGI shift come moltiplicazione (Capitolo 5) e, usati senza essere studiati nel corso, il programma completo, gli strumenti di PRG32.

DA COMPRENDERE PIÙ AVANTI niente.

Nel Capitolo 2 il lettore ha scritto `0x07e0` e ha ottenuto il verde, ha scritto 255 e ha ottenuto una barra che ricomincia, ha scritto `.asciz` e ha ottenuto un saluto, ha scritto `.word` e ha ottenuto un contatore. Questo capitolo dà a quelle cose il loro nome. La sua tesi sta in una frase, che il resto del capitolo illustra quattro volte: *nella macchina tutto è una configurazione di bit, e il significato di una configurazione è una convenzione.* I due capitoli successivi trattano formalmente i numeri.

4.1 L'informazione

4.1.1 Il flusso dei dati

Ogni uso di un calcolatore ha la stessa forma. Dei *dati* entrano dal mondo: un tasto, un sensore, un file. Un *programma*, l'incarnazione di un algoritmo, li trasforma. Dei *risultati* escono: un'immagine, un suono, un numero. E qualcuno riceve *conoscenza*, cioè informazione in base alla quale è possibile agire. Su PRG32 le quattro fasi sono, nell'ordine, la funzione `prg32_input_read`, la `update` del programmatore, la `draw` del programmatore, e il giocatore che reagisce allo schermo.

L'ultima fase ha richiesto, finora, una persona. L'intelligenza artificiale, a cui torna la fine di questo capitolo, è un'intelligenza di un tipo suo proprio: un potente riconoscimento di schemi, senza autocoscienza e senza esperienza soggettiva.

4.1.2 Informazione analogica e digitale

Definizione 4.1 (Analogico e digitale)

Una grandezza *analogica* varia con continuità: una tensione, una pressione sonora, le lancette di un orologio. Una grandezza *digitale* è una sequenza di simboli presi da un alfabeto finito: le cifre di un display.

Digitalizzare una grandezza analogica significa farle due cose (Figura 4.1): *campionarla* nel tempo, leggendone il valore a istanti regolari, e *quantizzarla* nel valore, sostituendo ogni lettura con il più vicino tra un numero finito di livelli.

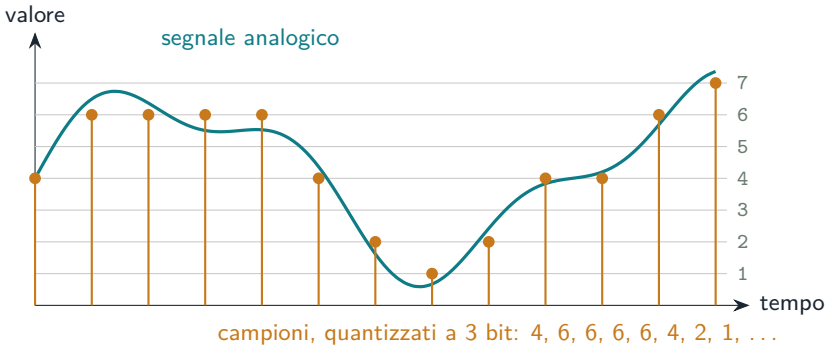


Figura 4.1. Digitalizzare: campionare nel tempo e quantizzare nel valore.

In entrambi i passaggi si perde qualcosa, e si guadagna qualcosa di decisivo. Un numero finito di livelli può essere *contato*, *copiato esattamente* e *controllato per rilevare errori*; una grandezza continua si può soltanto misurare, e ogni copia la degrada. Con due livelli il simbolo è il *bit*, che vale 0 oppure 1. Due livelli sono facili da realizzare in modo affidabile con i transistor, che sono spenti o accesi (Capitolo 10), ed è per questo che i calcolatori sono binari. L'algebra di Boole (Capitolo 8) è la matematica dei due livelli.

4.1.3 Bit, byte, parola

I bit si trattano a gruppi di dimensioni standard (Tabella 4.1). Il fatto fondamentale che li riguarda è un conteggio.

Teorema 4.1 (Numero di configurazioni)

Con n bit si possono scrivere esattamente 2^n configurazioni distinte.

Dimostrazione. Per induzione su n. Un bit ha $2 = 2^1$ configurazioni. Se n bit hanno 2^n configurazioni, ciascuna può essere seguita da uno 0 o da un 1, il che dà $2 \cdot 2^n = 2^{n+1}$ configurazioni di $n + 1$ bit. □

Tabella 4.1. Unità di informazione.

Unità	Dimensione	Configurazioni	Dove la si è incontrata
bit	1 cifra binaria	2	un pulsante del joystick
nibble	4 bit	16	una cifra esadecimale
byte	8 bit	256	un carattere ASCII; la larghezza della barra (`andi 255`)
mezza parola	16 bit	65 536	un colore RGB565
parola	32 bit	4 294 967 296	un registro RISC-V; un'istruzione; `.word`
KiB, MiB, GiB	2^{10}, 2^{20}, 2^{30} byte	–	dimensioni di memoria: 1 KiB = 1024 byte

Concetto

Il significato di una configurazione è una convenzione, non una proprietà dei bit. Che cosa significa una configurazione non c'è scritto dentro: è un accordo tra chi la scrive e chi la legge.

La dimensione di una parola dipende dall'architettura: su RV32 una parola è di 32 bit per definizione.

4.1.4 Contare in base 2 e in base 16

La Tabella 4.2 elenca le sedici configurazioni di quattro bit con i loro valori in decimale e le loro cifre esadecimali. Le quattro colonne di un numero binario pesano 8, 4, 2 e 1. Questa tabella va imparata a memoria: è la tavola pitagorica di questo libro.

Tabella 4.2. Le sedici configurazioni di quattro bit.

Dec	Binario	Esa	Dec	Binario	Esa
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

4.1.5 La notazione posizionale: che cosa significa `11011001`

In un sistema *posizionale* il valore di una cifra dipende dalla sua posizione: ogni posizione pesa una potenza della base. La base dieci usa le potenze di dieci, la base due le potenze di due, la base sedici le potenze di sedici. La regola è la stessa; cambia solo la base. Il byte `11011001` si legge come segue.

peso	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	128	64	32	16	8	4	2	1
bit	**1**	**1**	0	**1**	**1**	0	0	**1**
contributo	128	64	0	16	8	0	0	1

$$128 + 64 + 16 + 8 + 1 = 217$$

Il Capitolo 5 dà la formula generale e gli algoritmi di conversione. Qui lo scopo è soltanto leggere un byte.

4.1.6 Perché i programmatori scrivono in esadecimale

Una cifra esadecimale corrisponde esattamente a quattro bit, perché $16 = 2^4$. Convertire tra binario ed esadecimale è quindi una consultazione di tabella, cifra per cifra, senza aritmetica:

$$\underbrace{0000}_{0}\ \underbrace{0111}_{7}\ \underbrace{1110}_{E}\ \underbrace{0000}_{0} \quad \Longrightarrow \quad \texttt{0x07E0}.$$

Un registro di 32 bit sono otto cifre esadecimali: `0x00130313` è leggibile, trentadue zeri e uni non lo sono. Il prefisso `0x` indica l'esadecimale in assembly e in C, e il debugger di PRG32-QT mostra i registri in questo modo.

Il valore decimale di `0x07E0` è $7 \cdot 256 + 14 \cdot 16 = 2016$, e non serve assolutamente a nulla: ciò che conta in questo numero è la *configurazione di bit*, come mostra la prossima sezione.

4.1.7 Un colore è un numero: RGB565

Su PRG32 un colore sono sedici bit divisi in tre *campi*: cinque bit di rosso, sei di verde, cinque di blu (Figura 4.2). Il formato si chiama RGB565. La costante `.equ GREEN, 0x07e0` è la configurazione in cui il verde è al massimo, 63 su 63, e rosso e blu sono a zero. L'occhio è più sensibile al verde, ed è per questo che il verde riceve il bit in più.

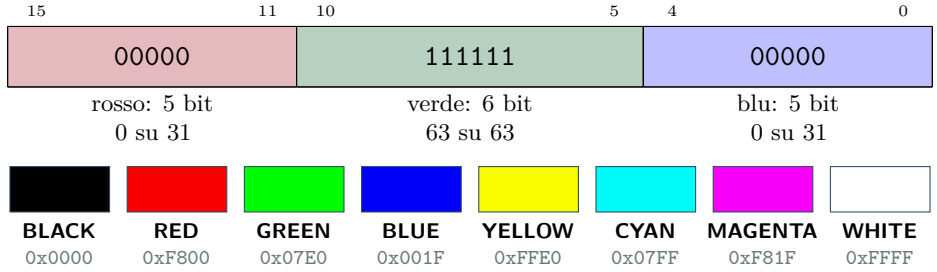


Figura 4.2. RGB565: `0x07E0` è il verde. Sotto, le costanti di colore di `prg32.h`.

Sedici bit, tre campi. Gli stessi sedici bit potrebbero essere un numero, due caratteri o mezza istruzione: il significato sta nel modo in cui il programma li usa.

4.1.8 I numeri: naturali, interi, reali

Gli stessi numeri sono rappresentati con tre convenzioni diverse, che i due capitoli successivi sviluppano.

- I **numeri naturali** sono la configurazione posizionale stessa: n bit coprono l'intervallo da 0 a $2^n - 1$.
- Gli **interi con segno** si scrivono in *complemento a due*: un registro di 32 bit copre da circa -2 miliardi a circa $+2$ miliardi (Capitolo 5).
- I **numeri reali** si scrivono in *virgola fissa* oppure nella *virgola mobile* dello standard IEEE 754, in cui il valore è $(-1)^s \times 1{,}f \times 2^{e-bias}$ (Capitolo 6). La precisione singola usa 32 bit, dà circa 7 cifre decimali e arriva a circa $3{,}4 \times 10^{38}$; la precisione doppia usa 64 bit, dà circa 16 cifre e arriva a circa $1{,}8 \times 10^{308}$.

4.2 Il testo

Anche i caratteri sono numeri.

4.2.1 ASCII: un carattere, un byte

Il codice ASCII (1963) assegna un numero tra 0 e 127, sette bit, a ciascuno di 128 caratteri: le lettere dell'inglese, le cifre, la punteggiatura e alcuni codici di controllo (Tabella 4.3).

Tabella 4.3. Alcuni caratteri ASCII.

Carattere	Decimale	Esa	Binario	Nota
`A`	65	`0x41`	`0100 0001`	le maiuscole cominciano da 65
`a`	97	`0x61`	`0110 0001`	come `A` con un bit in più: $97 - 65 = 32$
`0`	48	`0x30`	`0011 0000`	il carattere zero non è il numero zero
spazio	32	`0x20`	`0010 0000`	un carattere come un altro
a capo	10	`0x0A`	`0000 1010`	un carattere di controllo
NUL	0	`0x00`	`0000 0000`	segna la fine di una stringa C

La tabella è stata progettata con cura. Maiuscole e minuscole differiscono solo nel bit 5, così che `xori` con `0x20` scambia maiuscola e minuscola: un primo assaggio di algebra di Boole applicata al testo. E i caratteri delle cifre sono consecutivi, così che sottraendo 48 dal carattere di una cifra si ottiene il valore della cifra.

4.2.2 Stringhe: una sequenza di caratteri in memoria

Una *stringa* è una sequenza di caratteri memorizzati in byte consecutivi. L'unica questione è come fa chi legge la stringa a sapere dove finisce, e ci sono due risposte classiche (Figura 4.3). La *stringa C* segna la fine con un byte di valore zero. La *stringa Pascal* mette prima la lunghezza.

La `z` di `.asciz` significa *zero-terminated*, terminata da zero: per gli undici caratteri della stringa del Capitolo 2 l'assemblatore memorizza dodici byte. La funzione `prg32_gfx_text8` riceve in `a2` soltanto l'indirizzo del primo byte, e legge finché non trova lo zero. Se lo zero mancasse, la funzione continuerebbe a leggere ciò che segue in memoria, qualunque cosa sia.

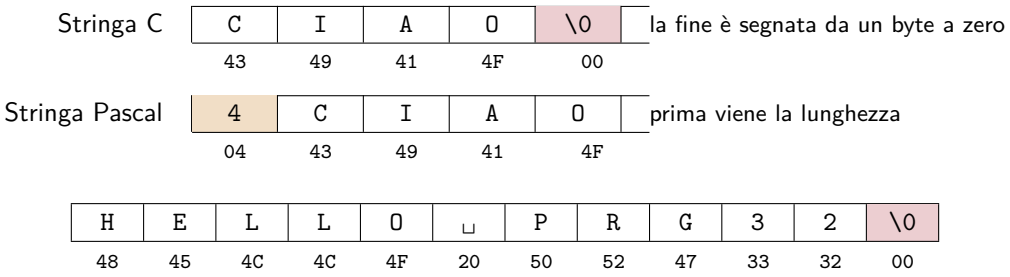


Figura 4.3. Stringhe in memoria; sotto ogni cella, il codice ASCII in esadecimale. Sotto: `.asciz "HELLO PRG32"`.

4.2.3 Unicode e UTF-8: ogni sistema di scrittura

Centoventotto caratteri bastano per l'inglese e per nient'altro. *Unicode* assegna un numero, detto *code point*, a ogni carattere di ogni scrittura; *UTF-8* memorizza quel numero in un numero di byte da uno a quattro (Tabella 4.4). La codifica è compatibile con ASCII: un testo ASCII puro è già UTF-8 valido. È la codifica del web e dei file sorgente che il lettore sta scrivendo.

Tabella 4.4. Code point Unicode e loro codifica UTF-8.

Carattere	Code point	Byte UTF-8	Lunghezza
A	U+0041	`41`	1 byte: identico ad ASCII
è	U+00E8	`C3 A8`	2 byte
€	U+20AC	`E2 82 AC`	3 byte
l'ideogramma per «sole», «giorno»	U+65E5	`E6 97 A5`	3 byte
l'emoji della «faccina sorridente»	U+1F600	`F0 9F 98 80`	4 byte

Attenzione

In UTF-8 la lunghezza di una stringa in byte non è la sua lunghezza in caratteri.

4.2.4 L'ordine dei byte: little-endian

Una parola sono quattro byte, e la memoria è indirizzata byte per byte. In che ordine vanno i quattro byte di una parola? La direttiva `.word 0x12345678` si può memorizzare in due modi (Figura 4.4).

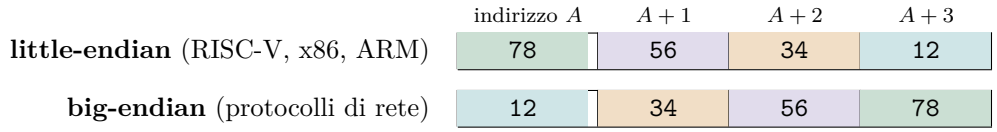


Figura 4.4. I due ordini dei byte della parola `0x12345678`.

Definizione 4.2 (Little-endian e big-endian)

Nell'ordine *little-endian* il byte meno significativo di una parola è memorizzato all'indirizzo più basso; nell'ordine *big-endian*, il byte più significativo.

RISC-V è little-endian: è ciò che fanno `lw` e `sw`. L'ordine dei byte si nota in due circostanze: quando si esamina la memoria in un debugger, e quando si scambiano dati con un'altra macchina. I nomi vengono dai *Viaggi di Gulliver*, dove due popoli si fanno la guerra per decidere da quale estremità vada rotto un uovo.

4.3 Strutture dati

Dai singoli valori si passa alle collezioni organizzate. Cinque modi di organizzare i dati ricorrono in tutta l'informatica.

Vettore (array).
Elementi dello stesso tipo, uno dopo l'altro, raggiunti tramite un indice: 32, 11, 34, 22.

Lista.
Elementi collegati ciascuno al successivo: economica per inserire e togliere, lenta da raggiungere per posizione.

Pila (stack).
L'ultimo a entrare è il primo a uscire: *push* aggiunge in cima, *pop* toglie dalla cima. Il frame che ha salvato `ra` nel Capitolo 2 stava su uno stack.

Dizionario.
Coppie di chiave e valore: `NA` dà `NAPOLI`, `PA` dà `PALERMO`, `MI` dà `MILANO`.

Albero.
Una gerarchia: un animale cammina o nuota, ha polmoni o branchie, due zampe o quattro.

Nella macchina sono tutti la stessa cosa: byte in memoria, più una regola per trovare l'elemento successivo.

4.3.1 Un vettore in memoria, un elemento in un registro

Per il vettore la regola è aritmetica (Listato 4.1). Un vettore è un indirizzo di base seguito da elementi di uguale dimensione; l'elemento i di un vettore di parole sta all'indirizzo $base + 4i$, perché una parola sono quattro byte.

```
.section .data
scores: .word 32, 11, 34, 22     # four 32-bit words, 16 bytes

.section .text
    la   t0, scores      # t0 = ADDRESS of element 0
    li   t1, 2           # index i = 2
    slli t1, t1, 2       # offset = i * 4 bytes (shift left by 2)
    add  t0, t0, t1      # ADDRESS of element i
    lw   a0, 0(t0)       # a0 = VALUE of scores[2] = 34
```

Listato 4.1. Leggere `scores[2]`.

L'istruzione `slli` fa scorrere a sinistra i bit di un registro; uno scorrimento di due posizioni moltiplica per quattro, come in decimale uno scorrimento di una posizione moltiplica per dieci. (La proprietà è dimostrata nel Capitolo 5.) L'espressione `scores[i]` di un linguaggio di alto livello è esattamente queste quattro istruzioni.

Attenzione

Che cosa legge `lw` se $i = 4$? Ciò che segue il vettore in memoria, qualunque cosa sia. La macchina non controlla i limiti: lo fa il programmatore.

4.4 Dati e algoritmi

Il capitolo si chiude allargando lo sguardo, dai bit di un programma ai dati del mondo e alle procedure che lavorano su di essi.

4.4.1 Dati, big data e data science

Definizione 4.3 (Big data)

I *big data* sono dati che superano i limiti degli strumenti tradizionali delle basi di dati.

Sono strutturati e non strutturati: l'economia, la geolocalizzazione, le serie temporali, le reti sociali. La loro dimensione si misura in petabyte e zettabyte, cioè in miliardi di terabyte. Vengono da quattro tipi di sorgente: i *dati da sensori* (l'ambiente, le posizioni GPS, l'Internet delle cose: ogni dispositivo misura qualcosa); le *transazioni* (gli acquisti al supermercato, le carte di credito, gli ordini online); i *dati pubblici* (atti amministrativi, territorio, consumi: gli open data delle istituzioni); e i *social media* (preferenze, politica, opinioni: ciò che le persone scrivono e condividono). La *data science* è l'analisi di grandi quantità di informazione, e le tecnologie che estraggono conoscenza e valore dai dati sono il calcolo parallelo e le GPU.

Definizione 4.4 (Data mining)

Il *data mining* è l'insieme delle tecniche e delle metodologie per estrarre informazione o conoscenza da grandi quantità di dati.

Tra le sue tecniche ci sono gli alberi di decisione e la classificazione, l'apprendimento di regole e i metodi bayesiani, le reti neurali e il riconoscimento di schemi.

4.4.2 GPGPU: il calcolo generico sui processori grafici

Un processore grafico (GPU) ha migliaia di core semplici costruiti per fare la stessa operazione su molti dati insieme. La somma di N elementi, che richiede N passi uno dopo l'altro, ha bisogno di soli $\log_2 N$ passi paralleli (Figura 4.5): gli elementi si sommano a coppie, poi le somme parziali a coppie, e così via. Lo stesso hardware che disegna i videogiochi oggi addestra le reti neurali. Sotto ogni core ci sono sempre registri, istruzioni e bit; il Capitolo 24 torna sull'argomento.

4.4.3 Algoritmi e calcolabilità

Un problema è *calcolabile* se può essere risolto da un algoritmo. La *macchina di Turing* è un calcolatore ideale con un programma, espresso matematicamente tramite insiemi, relazioni e funzioni. La *macchina di von Neumann* è l'architettura di calcolo capace di eseguire un programma che realizza un algoritmo per la macchina di Turing. Un problema che

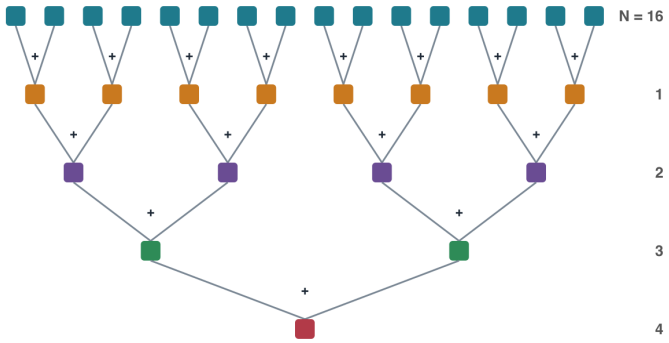


Figura 4.5. Una somma parallela sui core di una GPU: $\log_2 N$ passi.

nessun algoritmo su una macchina di Turing può risolvere non può essere risolto da alcun calcolatore: questo enunciato, nella sua versione informale, è la tesi di Church–Turing. Non ogni problema ha un algoritmo; l'esempio classico di problema non calcolabile è il *problema della fermata*, quello di decidere se un programma arbitrario prima o poi si fermerà.

Definizione 4.5 (Algoritmo)

Un algoritmo è una procedura con cinque proprietà. È **atomico**: i suoi passi sono elementari e non ulteriormente scomponibili. È **non ambiguo**: ogni passo è interpretato in un solo modo diretto dall'esecutore, umano o macchina. È **finito**: ha un numero finito di passi e una quantità finita di dati in ingresso. È **terminante**: la sua esecuzione finisce dopo un tempo finito. È **effettivo**: la sua esecuzione porta a un risultato unico e ben definito.

Un'istruzione RISC-V è il modello di un passo atomico e non ambiguo: il suo significato è fissato dalla specifica, bit per bit.

4.4.4 Algoritmi deterministici e non deterministici

Un algoritmo è *deterministico* se per ogni istruzione esiste, a parità di ingresso, esattamente un passo successivo: il percorso dell'algoritmo esiste ed è unico, $y = f(x)$. Convertire un'unità di misura è deterministico; lo è anche `first_update`, che somma sempre uno.

Un algoritmo è *non deterministico* se, a parità di ingresso, può comportarsi in modo diverso a ogni esecuzione. Gli algoritmi *probabilistici* usano un generatore di numeri casuali; gli algoritmi *concorrenti* possono dipendere dai tempi (le cosiddette race condition). Un gioco che colloca un nemico chiamando `prg32_random_number` è non deterministico.

Il Listato 4.2 mostra due algoritmi probabilistici in pseudo-codice. Il primo verifica se un numero è primo: la risposta «composto» è sempre giusta, la risposta «probabilmente primo» è giusta con probabilità schiacciante. Scambiare una minuscola possibilità di errore con un enorme guadagno di velocità è il modo in cui la crittografia reale verifica i numeri primi. (Il test di Fermat mostrato è il più semplice; le librerie reali usano il test di Miller–Rabin, che ne chiude le falle.) Il secondo è una passeggiata aleatoria, che dà un risultato diverso a ogni esecuzione: le simulazioni ne fanno la media su molte.

```
is_probably_prime(n):
```

```
    repeat 30 times:
        a = random(2, n - 2)
        if a^(n-1) mod n != 1:
            return COMPOSITE          # certain
    return PROBABLY PRIME             # almost certain

random_walk():
    while t < t_final:
        r = random(-1, 1)
        x = x + r * u                 # a different path every run
        y = y + r * v
        t = t + 1
```

Listato 4.2. Due algoritmi probabilistici.

4.4.5 L'apprendimento automatico

L'*apprendimento automatico* (machine learning) è un metodo di analisi dei dati che automatizza la costruzione di un modello analitico: il calcolatore trova schemi nascosti senza essere stato esplicitamente programmato su dove cercare. In un programma classico la funzione $y = f(x)$ è scritta da una persona; nell'apprendimento automatico f viene appresa dagli esempi. I modelli generativi di oggi sono la stessa idea su scala enorme: un grande modello linguistico predice il prossimo pezzo di testo a partire da miliardi di numeri appresi.

Quei numeri sono bit in memoria, moltiplicati e sommati da processori: l'argomento di questo libro. E un modello può sbagliare con grande sicurezza, quindi per controllarlo servono i fondamenti. È la ragione della regola enunciata nel Capitolo 0: l'esame verifica che i fondamenti stiano nella testa dello studente.

4.5 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l04.S` nel repository del corso, stampato per intero nel Listato 4.3. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 4.6 è il frame scritto dal test.

Sullo schermo.
Quattro barre orizzontali, una per elemento dell'array (32, 11, 34, 22, quattro pixel per unità); l'elemento 2, quello che il listato carica, è giallo.

Che cosa premere.
Niente: il programma va da solo.

```
# ARC12 Computer Architecture - Lecture 04: Representing Digital Information
# l04.S: an array in memory, an element in a register
#
# On the screen: four horizontal bars, one per element of the array (32, 11,
    34, 22,
# four pixels per unit); element 2, the one the listing loads, is yellow.
# Build: python3 -m prg32 cartridge build l04.S --portable \
#            --entry-prefix l04 --name l04 --out l04.prg32
.option norelax

```



Figura 4.6. `l04.s` su PRG32-QT.

```
.equ BLACK,  0x0000
.equ WHITE,  0xffff
.equ YELLOW, 0xffe0

.section .text
.global l04_init
.global l04_update
.global l04_draw

l04_init:
    ret

l04_update:
    ret

l04_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   s0, 0              # index i, 0 to 3
1:  la   t0, scores         # t0 = ADDRESS of element 0
    mv   t1, s0             # index i
    slli t1, t1, 2          # offset = i * 4 bytes (shift left by 2)
    add  t0, t0, t1         # ADDRESS of element i
    lw   a2, 0(t0)          # VALUE of scores[i]
    slli a2, a2, 2          # width: four pixels per unit
    li   a4, WHITE
    li   t0, 2
    bne  s0, t0, 2f
    li   a4, YELLOW         # scores[2] = 34: the element of the slide
2:  li   a0, 20             # x
    slli a1, s0, 5
    addi a1, a1, 30         # y = 30 + 32 * i
    li   a3, 16             # height
    call prg32_gfx_rect
    addi s0, s0, 1
    li   t0, 4
    blt  s0, t0, 1b
    lw   s0, 8(sp)
```

```
51      lw   ra, 12(sp)
52      addi sp, sp, 16
53      ret
54
55  .section .data
56  scores: .word 32, 11, 34, 22     # four 32-bit words, 16 bytes
```

Listato 4.3. Il programma completo del capitolo, `code/l04.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 4.4; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l04.S --portable \
    --entry-prefix l04 --name l04 --out l04.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l04.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 4.4. Compilare `l04.S` ed eseguirlo su PRG32-QT.

4.6 Verificate di aver capito

D4.1 Quanti valori diversi può rappresentare un byte?

A. 8
B. 16
C. 255
D. 256

D4.2 Il colore PRG32 `0xF81F` è:

A. rosso
B. magenta
C. ciano
D. giallo

D4.3 Quanti byte occupa in memoria `.asciz "CIAO"`?

A. 3
B. 4
C. 5
D. 8

D4.4 Su RISC-V, `.word 0x0000ABCD` è memorizzata all'indirizzo A. Quale byte si trova all'indirizzo A?

A. `0x00`
B. `0xAB`
C. `0xCD`
D. `0xDC`

4.7 Esercizi

Esercizio 4.1 (su PRG32)

1. Cambiate la barra del Capitolo 2 in magenta, poi in un grigio scuro a vostra scelta: calcolate prima a mano il valore RGB565.
2. Scrivete il vostro nome con `prg32_gfx_text8`. Quanti byte occupa la stringa?
3. Dichiarate un vettore di quattro colori con `.word` e disegnate quattro barre, una per colore, leggendo ogni colore con `lw`.
4. In PRG32-QT aprite la vista della memoria sulla vostra stringa e sul vostro vettore: trovate lo zero finale e verificate l'ordine dei byte.

Risultato atteso. Quattro barre in quattro colori; nella vista della memoria la stringa finisce con `00` e ogni `.word` mostra per primo il byte meno significativo. Un grigio scuro, per confronto: rosso 8 su 31, verde 16 su 63, blu 8 su 31 è `01000 010000 01000`, cioè `0x4208`.

4.8 Glossario

Inglese	Italiano
bit / byte / word	bit / byte / parola
positional notation	notazione posizionale
hexadecimal	esadecimale
sampling	campionamento
quantisation	quantizzazione
byte order (endianness)	ordinamento dei byte
string	stringa
array	vettore (array)
linked list	lista concatenata
stack	pila
tree	albero
machine learning	apprendimento automatico

4.9 Conclusioni e letture

L'informazione digitale è una configurazione di bit più una convenzione per leggerla. Gli stessi sedici bit possono essere un numero, un colore o due caratteri: lo decide il programma. L'esadecimale è una scrittura compatta del binario, quattro bit per cifra. Il testo è fatto di numeri (ASCII, UTF-8); le strutture sono byte più una regola per trovare l'elemento successivo. I dati sono la moneta di questo regno, e gli algoritmi trasformano i dati in informazione adatta a decidere.

Letture. Patterson e Hennessy (2020), paragrafi 2.4 e 2.9; Charles Petzold, *Code*, i capitoli su bit, byte e testo; per l'apprendimento automatico, Sebastian Raschka, *Machine Learning with Python*.

CAPITOLO 5

Rappresentazione dei numeri: gli interi

"Conterò poco, è vero: — diceva l'Uno ar Zero — ma tu che vali? Gnente: propio gnente."
— Trilussa, *Nummeri*, 1944

In questo capitolo

Alla fine di questo capitolo saprete

- convertire interi e frazioni tra le basi 2, 8, 10 e 16;
- dire quali numeri naturali stanno in n bit e riconoscere un overflow;
- codificare e decodificare interi con segno in segno e modulo e in complemento a due;
- sommare, sottrarre e cambiare di segno in complemento a due, e rilevare l'overflow;
- spiegare perché $0 - 1$ appare come `0xFFFFFFFF` in un registro RISC-V;
- scegliere l'istruzione RISC-V giusta per dati con segno e senza segno.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI il complemento a due; i cambi di base; naturali e overflow; shift come moltiplicazione; segno e modulo; istruzioni con o senza segno; l'estensione del segno.

DA UTILIZZARE OGGI le chiamate del runtime PRG32, il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI niente.

5.1 Riscaldamento sulla macchina

Un marcatore sta sullo schermo, alla posizione 2 di una linea orizzontale. Il programma legge il joystick e, finché **LEFT** è premuto, sottrae uno dalla posizione a ogni frame (Listato 5.1); la funzione `draw` stampa la posizione in esadecimale con `prg32_console_hex32`.

```
steer_update:
    ...
    call prg32_input_read       # a0 = button mask
    mv   t0, a0
```

```
5
6       la   t1, marker_x
7       lw   t2, 0(t1)              # t2 = marker_x, starts at 2
8       andi t3, t0, BTN_LEFT       # is LEFT pressed?
9       beqz t3, 1f
10      addi t2, t2, -1             # one step to the left
11  1:  sw   t2, 0(t1)
12      ...
13  # draw prints marker_x with prg32_console_hex32
```

Listato 5.1. Tenete premuto **LEFT** e guardate il numero (adattato dal laboratorio su stato e ingresso del libro di PRG32).

Provate voi

Tenete premuto **LEFT**. Il valore mostrato fa `00000002`, `00000001`, `00000000` … e poi `FFFFFFFF`, e il marcatore scompare. (Sulla scheda e in QEMU la console è disegnata sullo schermo; PRG32-QT la raccoglie nel suo log.) È un errore? `0xFFFFFFFF` è il numero 4 294 967 295 oppure il numero -1?

La risposta è: entrambi. Dipende da come si leggono i bit, e questo capitolo dà le regole.

5.2 Basi e conversioni

5.2.1 Sistemi posizionali

Definizione 5.1 (Rappresentazione posizionale)

Sia $b \geq 2$ un intero, la *base*. Una stringa di cifre $d_{n-1}\, d_{n-2} \cdots d_1\, d_0$, con $0 \leq d_i \leq b-1$ per ogni i, rappresenta in base b il numero naturale

$$N \;=\; \sum_{i=0}^{n-1} d_i\, b^{\,i} \;=\; d_{n-1} b^{\,n-1} + \cdots + d_1 b + d_0. \tag{5.1}$$

La cifra d_{n-1} è la *più significativa*, d_0 la *meno significativa*.

In informatica si usano quattro basi (Tabella 5.1). Ogni numero naturale ha esattamente una rappresentazione in ciascuna base, a parte gli zeri aggiunti a sinistra.

Tabella 5.1. Le quattro basi dell'informatica.

Base b	Cifre	Nome	Tredici si scrive
2	0, 1	binaria	`1101`
8	0, 1, …, 7	ottale	`15`
10	0, 1, …, 9	decimale	`13`
16	0, …, 9, A, …, F	esadecimale	`D`

5.2.2 Le basi che sono potenze di due condividono gli stessi bit

Se ogni cifra è scritta in binario puro, i bit di un numero sono gli stessi in ogni base $b = 2^k$: basta raggrupparli da destra, k alla volta, completando con zeri a sinistra. Per il numero 127:

$$\texttt{01111111}_2 = \underbrace{\texttt{0111}}_{7}\,\underbrace{\texttt{1111}}_{F} = \texttt{0x7F} \qquad \text{e} \qquad \texttt{01111111}_2 = \underbrace{\texttt{01}}_{1}\,\underbrace{\texttt{111}}_{7}\,\underbrace{\texttt{111}}_{7} = 177_8.$$

La ragione è che nella (5.1) con $b = 2$ i termini si possono raccogliere k alla volta, e ogni gruppo è una cifra moltiplicata per una potenza di 2^k.

5.2.3 Convertire un intero: divisioni successive

Per scrivere un numero naturale N in base r: si divide N per r; il resto è la cifra successiva, a partire dalla meno significativa; si ripete sul quoziente finché non è zero. I resti si leggono poi dall'ultimo al primo (Tabella 5.2).

Tabella 5.2. Il numero 127 convertito per divisioni successive.

$r = 8$

N	$N/8$	$N \bmod 8$
127	15	7
15	1	7
1	0	1

$127 = 177_8$

$r = 16$

N	$N/16$	$N \bmod 16$
127	7	$15 = \text{F}$
7	0	7

$127 = \texttt{0x7F}$

$r = 2$

N	$N/2$	$N \bmod 2$
127	63	1
63	31	1
31	15	1
15	7	1
7	3	1
3	1	1
1	0	1

$127 = 1111111_2$

Il metodo è corretto perché, per la (5.1), $N = r\,(d_{n-1}r^{\,n-2} + \cdots + d_1) + d_0$: il resto della divisione di N per r è d_0, e il quoziente è il numero le cui cifre sono $d_{n-1}\cdots d_1$, a cui si applica lo stesso ragionamento. In C le due operazioni sono `/` e `%`; in RISC-V, con l'estensione M, sono `divu` e `remu`, e il Capitolo 22 usa esattamente questo algoritmo per stampare un punteggio.

5.2.4 Convertire una frazione: moltiplicazioni successive

Un numero $0 \le N < 1$ si scrive in base r come $N = d_{-1}r^{-1} + d_{-2}r^{-2} + \cdots$. Moltiplicando per r si ottiene $d_{-1} + (d_{-2}r^{-1} + \cdots)$: la parte intera del prodotto è la cifra successiva, a partire dalla più significativa, e il procedimento si ripete sulla parte frazionaria (Tabella 5.3).

Attenzione

Una frazione finita in base 10 può essere periodica in base 2. Con un numero finito di bit va troncata: da qui vengono gli errori di approssimazione dei numeri reali (Capitolo 6).

5.3 Numeri naturali e overflow

Tabella 5.3. Due frazioni convertite in base 2.

0,875: esatta

Prodotto	Frazione	Cifra
$0{,}875 \times 2$	0,75	1
$0{,}75 \times 2$	0,5	1
$0{,}5 \times 2$	0	1

$0{,}875 = 0{,}111_2$

0,8: periodica

Prodotto	Frazione	Cifra
$0{,}8 \times 2$	0,6	1
$0{,}6 \times 2$	0,2	1
$0{,}2 \times 2$	0,4	0
$0{,}4 \times 2$	0,8	0
$0{,}8 \times 2$	0,6	1 …

$0{,}8 = 0{,}\overline{1100}_2$

5.3.1 Strategie di codifica

Definizione 5.2 (Rappresentazione)

Una *rappresentazione* è una legge $X = R(x)$ che associa a un numero x una stringa di bit X. Qui x è il numero che si intende; X è la configurazione memorizzata nel registro.

Il numero di bit dipende da quanti numeri si vogliono rappresentare; di solito è un multiplo di 8, tipicamente 16, 32 o 64. La legge R si sceglie secondo il tipo di numero, naturale, con segno o razionale, e una buona legge è quella che rende semplici da costruire come circuiti le operazioni aritmetiche e i confronti.

5.3.2 Numeri naturali su n bit

Con n bit ci sono 2^n configurazioni, e la scelta naturale è associare a ogni numero naturale da 0 a $2^n - 1$ la stringa di bit che lo rappresenta nel sistema posizionale binario. Con $n = 4$ l'intervallo è [0,15]. Le dimensioni trattate da RISC-V sono nella Tabella 5.4.

Tabella 5.4. Numeri naturali su 8, 16 e 32 bit.

Bit	Intervallo	In RISC-V	Massimo
8	$0 \dots 2^8 - 1$	byte (`lbu`)	255
16	$0 \dots 2^{16} - 1$	mezza parola (`lhu`)	65 535
32	$0 \dots 2^{32} - 1$	parola, registro	4 294 967 295

Il numero 4 294 967 295 è la lettura di `0xFFFFFFFF` come numero naturale: la prima metà della risposta al riscaldamento.

5.3.3 Proprietà delle stringhe binarie

Dalla (5.1) seguono subito tre proprietà.

1. *L'estensione a sinistra con zeri non cambia il valore*: $5 = 101_2 = 00101_2$.
2. *Moltiplicare per 2^k è uno scorrimento a sinistra di k posizioni*: $00101_2 \times 2 = 01010_2$, cinque diventa dieci. Ogni cifra d_i passa alla posizione $i + k$, dove pesa 2^k volte tanto.
3. *La divisione intera per 2^k è uno scorrimento a destra di k posizioni*: $00101_2/2 = 00010_2$, cinque diventa due, e il resto va perso.

Gli scorrimenti costano molto meno delle moltiplicazioni, e i compilatori li usano ogni volta che possono (Listato 5.2). L'istruzione `slli t1, t1, 2` del Capitolo 4 moltiplicava così un indice per quattro. Gli scorrimenti sono anche la base dell'aritmetica in virgola fissa (Capitolo 6).

```
# multiply by 4
slli t0, t0, 2

# divide by 2 (unsigned)
srli t0, t0, 1

# x * 10 = x*8 + x*2
slli t1, t0, 3
slli t2, t0, 1
add  t0, t1, t2
```

Listato 5.2. Gli scorrimenti come moltiplicazioni e divisioni.

5.3.4 Overflow

Definizione 5.3 (Overflow)

Si ha un *overflow* quando il risultato di un'operazione non è contenuto nel sottoinsieme finito di numeri che i bit disponibili possono rappresentare.

I registri hanno una dimensione finita, e ce n'è un numero finito. Gli insiemi numerici sono infiniti, e di ciascuno si può rappresentare solo un sottoinsieme finito. Gli operatori aritmetici sono chiusi sull'intero insieme, non sul sottoinsieme finito. Con 7 bit si codifica l'intervallo [0,127]: 100 e 38 sono rappresentabili, la loro somma 138 no.

5.3.5 Aritmetica su quattro bit

L'aritmetica binaria segue le stesse regole colonna per colonna dell'aritmetica decimale, con l'unica tabellina $0+0=0$, $0+1=1$, $1+1=0$ con riporto di 1.

Somma		Somma con overflow		Differenza		Prodotto	
6 +	0110 +	14 +	1110 +	11 −	1011 −	5 x	0101 x
8 =	1000 =	3 =	0011 =	5 =	0101 =	3 =	0011 =
14	1110	17	1 0001	6	0110	15	000 1111

Nella seconda somma serve un quinto bit: il registro di quattro bit conserva `0001` e il riporto va perso. I processori segnalano un riporto perso in modi diversi: molti hanno un *flag di riporto* (carry flag); RISC-V non ne ha e lascia che sia il programma a calcolarlo con un confronto, come mostra la Sezione 5.5. Il prodotto di due numeri di n bit richiede fino a $2n$ bit: su RV32 l'istruzione `mul` dà i 32 bit bassi e `mulhu` i 32 alti.

5.4 Interi con segno

Ci sono tre modi di scrivere un segno meno con i bit. Due sono presentati qui e il terzo, con una variante, nel Capitolo 6.

5.4.1 Segno e modulo

Il bit più significativo è il segno (1 per i negativi) e gli altri $n-1$ bit sono il modulo:

$$X = |x| + 2^{n-1}s, \qquad s = \begin{cases} 1 & x < 0 \\ 0 & x \geq 0. \end{cases}$$

L'intervallo va da $-(2^{n-1}-1)$ a $+(2^{n-1}-1)$. Lo zero ha due rappresentazioni, $+0$ e -0, quindi si rappresentano solo $2^n - 1$ numeri. E l'aritmetica deve lavorare separatamente su segno e modulo: per sommare due numeri di segno opposto bisogna trovare il modulo maggiore, sottrarne il minore, e dare al risultato il segno dell'operando con il modulo maggiore. La rappresentazione è semplice per le persone e scomoda per i circuiti. Sopravvive nella mantissa dei numeri in virgola mobile (Capitolo 6).

5.4.2 Complemento a due

Definizione 5.4 (Complemento a due)

Su n bit, la rappresentazione in *complemento a due* di un intero x con $-2^{n-1} \leq x \leq 2^{n-1}-1$ è il numero naturale

$$X = \begin{cases} x & \text{se } 0 \leq x < 2^{n-1}, \\ 2^n - |x| & \text{se } -2^{n-1} \leq x < 0, \end{cases}$$

scritto in binario su n bit. In modo equivalente, $X = x \bmod 2^n$.

L'intervallo va da -2^{n-1} a $+2^{n-1}-1$, e lo zero ha una sola rappresentazione. Il bit più significativo vale 1 se e solo se il numero è negativo, perché i numeri negativi occupano le configurazioni da 2^{n-1} a $2^n - 1$. È la rappresentazione usata da ogni processore moderno, RISC-V compreso: un registro di 32 bit letto come intero con segno copre $-2^{31} \ldots 2^{31}-1$, da circa meno due miliardi a circa più due miliardi. L'intervallo è asimmetrico, con un numero negativo in più rispetto ai positivi, e questo ha una conseguenza che comparirà tra poco.

5.4.3 Quattro bit, tre letture

La Tabella 5.5 legge ciascuna delle sedici configurazioni di quattro bit nei tre modi visti finora. Quando il bit più significativo vale 0 le tre letture coincidono; quando vale 1 differiscono.

Tabella 5.5. Quattro bit, tre letture. S-M: segno e modulo; C2: complemento a due.

Bit	Senza segno	S-M	C2	Bit	Senza segno	S-M	C2
0000	0	+0	0	1000	8	−0	−8
0001	1	+1	+1	1001	9	−1	−7
0010	2	+2	+2	1010	10	−2	−6
0011	3	+3	+3	1011	11	−3	−5
0100	4	+4	+4	1100	12	−4	−4
0101	5	+5	+5	1101	13	−5	−3
0110	6	+6	+6	1110	14	−6	−2
0111	7	+7	+7	1111	15	−7	−1

Concetto

I bit non dicono quale sia la colonna giusta. Lo dice il programma, scegliendo le sue istruzioni.

5.4.4 La ruota dei numeri

Con n bit l'aritmetica è *modulo* 2^n: la retta dei numeri si chiude in un cerchio (Figura 5.1). Sommare 1 sposta di un passo in senso orario, qualunque sia la lettura. Le due letture differiscono solo nel punto in cui la ruota «si spezza». Nella lettura senza segno si spezza tra `1111` e `0000`, e attraversare quel punto è un riporto: $15 + 1 = 0$. Nella lettura in complemento a due si spezza tra `0111` e `1000`, e attraversare quel punto è un overflow con segno: $7 + 1 = -8$. Un passo indietro da `0000` è `1111`: ecco perché $0 - 1$ dà tutti uni.

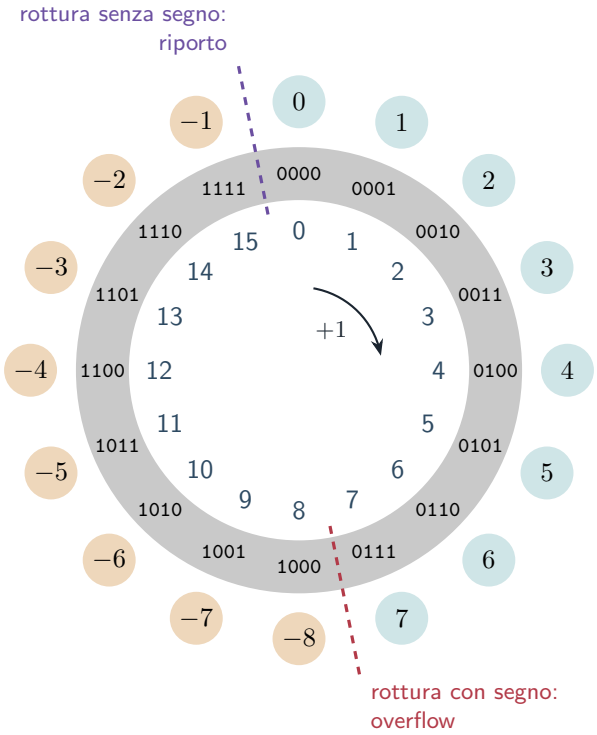


Figura 5.1. La ruota dei numeri per quattro bit. Numeri interni: valore senza segno; anello: bit; dischi esterni: valore in complemento a due.

Lo stesso sommatore serve a entrambe le letture; cambia solo il punto in cui la ruota si spezza.

5.4.5 Aritmetica in complemento a due

Teorema 5.1 (Addizione in complemento a due)

Siano x, y e $x + y$ tutti in $[-2^{n-1}, 2^{n-1} - 1]$, e siano X, Y le rappresentazioni di x e y. Allora la rappresentazione di $x+y$ è $(X+Y) \bmod 2^n$: la somma delle due configurazioni come numeri naturali, scartando il riporto uscente dal bit più significativo.

Dimostrazione. $X \equiv x$ e $Y \equiv y \pmod{2^n}$ per definizione, quindi $X + Y \equiv x + y \pmod{2^n}$; e la rappresentazione di $x + y$ è, per definizione, $(x + y) \bmod 2^n$. □

L'addizione non ha quindi bisogno di conoscere i segni: il sommatore per i numeri naturali dà già la risposta giusta.

Segni opposti		Due negativi		Prodotto	
2 +	0010 +	-2 +	1110 +	2 x	0010 x
-6 =	1010 =	-3 =	1101 =	-3 =	1101 =
-4	1100	-5	1 1011	-6	001 1010

Lo stesso ragionamento vale per la moltiplicazione: $XY \equiv xy \pmod{2^n}$, quindi gli n bit bassi di un prodotto sono corretti qualunque siano i segni. Nel terzo esempio $2 \times 13 = 26 = 1\,1010_2$, e i quattro bit bassi `1010` si leggono -6. Per questo RISC-V ha una sola `mul` per operandi con segno e senza segno.

5.4.6 Cambiare di segno un numero

Per definizione $-x$ è rappresentato da $2^n - X$. Due metodi lo calcolano senza una sottrazione.

Metodo 1: complementare ogni bit, poi sommare 1.
$6 =$ `0110`, complemento `1001`, più uno: `1010` $= -6$. Funziona perché il complemento di ogni bit di X è il numero $(2^n - 1) - X$, una sottrazione da tutti uni in cui non può esserci alcun prestito; sommando 1 si ottiene $2^n - X$.

Metodo 2: scorrere da destra.
Partendo da destra, si copiano i bit fino al primo 1 compreso, e si complementano tutti gli altri. $5 =$ `0101` dà `1011` $= -5$; $1 =$ `0001` dà `1111` $= -1$. È il metodo 1 con l'addizione svolta a mente: il $+1$ finale si propaga attraverso gli uni finali del complemento, che sono gli zeri finali dell'originale.

Cambiare di segno due volte restituisce il numero originale, con un'eccezione: il numero più negativo, -2^{n-1}, non ha un compagno positivo, e il suo cambio di segno va in overflow e restituisce il numero stesso.

```
# neg t0, t1 is a pseudo-instruction:
sub  t0, zero, t1

# the same thing by hand:
not  t0, t1        # complement
addi t0, t0, 1     # plus one
```

Listato 5.3. Il cambio di segno in RISC-V.

La sottrazione è un'addizione

$y - x = y + (2^n - x) \pmod{2^n}$. Un sommatore e un complementatore sono gli unici componenti necessari per costruire addizione, sottrazione, moltiplicazione e divisione.

È la ragione ingegneristica del complemento a due: un solo circuito, il sommatore del Capitolo 11, fa tutto il lavoro. Nel datapath di RISC-V `sub` è `add` con il secondo operando complementato e un riporto entrante di 1.

5.4.7 Rilevare l'overflow

Senza segno, 8 bit		Con segno, 8 bit	
128 +	1000 0000 +	127 +	0111 1111 +
130 =	1000 0010 =	127 =	0111 1111 =
258	1 0000 0010	254	1111 1110

Nella prima somma c'è un riporto uscente, e il registro contiene 2, la somma modulo 256. Nella seconda non c'è riporto uscente, ma il risultato si legge -2.

Teorema 5.2 (Condizioni di overflow)

In un'addizione su n bit:

1. si ha un overflow *senza segno* se e solo se c'è un riporto uscente dal bit più significativo;
2. si ha un overflow *con segno* se e solo se due operandi con lo stesso segno danno una somma di segno opposto: due positivi danno un negativo, o due negativi danno un positivo.

La somma di due numeri di segno opposto non va mai in overflow.

L'ultima affermazione vale perché una tale somma è compresa tra i due operandi, entrambi rappresentabili. Le due condizioni sono indipendenti: la stessa addizione può far scattare l'una, entrambe o nessuna. In termini di circuiti la condizione con segno equivale a questa: il riporto *entrante* nel bit più significativo è diverso dal riporto *uscente* da esso.

5.5 Ritorno alla macchina

5.5.1 Il riscaldamento, spiegato

La variabile `marker_x` conteneva 0. L'istruzione `addi t2, t2, -1` ha fatto un passo indietro su una ruota con 2^{32} posizioni, e il registro ora contiene trentadue uni: `0xFFFFFFFF`. Letto senza segno è 4 294 967 295; letto in complemento a due è -1. Il sommatore ha fatto la stessa cosa in entrambi i casi. La funzione `prg32_console_hex32` stampa la configurazione, non un'interpretazione: non ha né ragione né torto. E il marcatore scompare perché a `prg32_gfx_rect` si chiede di disegnare a una x enorme, oppure a -1: fuori dallo schermo in ogni caso (Figura 5.2). Non è un errore del processore; è una decisione che manca nel programma.

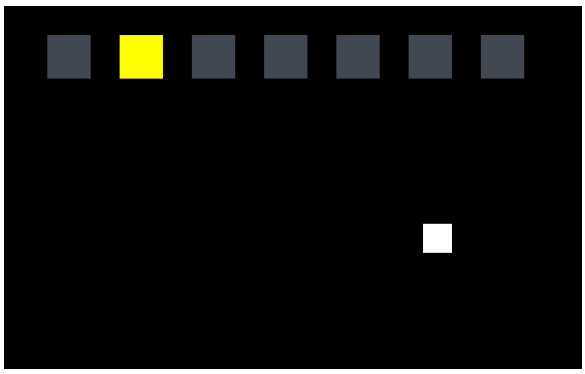


Figura 5.2. Il programma del laboratorio: sette lampade per i bit dei pulsanti e un marcatore.

5.5.2 Con segno o senza segno? Sceglie l'istruzione

I registri di RISC-V non hanno un tipo. Gli stessi 32 bit sono con segno o senza segno solo agli occhi dell'istruzione che li legge (Tabella 5.6).

Tabella 5.6. Istruzioni con segno e senza segno.

Operazione	Con segno	Senza segno	Perché differiscono
somma, sottrazione	`add`, `addi`, `sub`	le stesse	complemento a due: un sommatore per entrambi
prodotto (parola bassa)	`mul`	la stessa	i 32 bit bassi non dipendono dal segno
confronto	`slt`, `slti`	`sltu`, `sltiu`	`0xFFFFFFFF` è minore di 0? sì, se con segno
salto condizionato	`blt`, `bge`	`bltu`, `bgeu`	la stessa domanda, come salto
scorrimento a destra	`sra`, `srai`	`srl`, `srli`	copia il bit di segno, o inserisce zeri
caricamento di un byte	`lb`	`lbu`	estensione del segno o con zeri
divisione, resto	`div`, `rem`	`divu`, `remu`	i moduli si leggono in modo diverso

5.5.3 Tenere il marcatore sullo schermo

La decisione che mancava è presa nel Listato 5.4: dopo lo spostamento, la posizione viene *limitata* (clamp) all'intervallo $[0, 304]$.

```
    andi t3, t0, BTN_LEFT
    beqz t3, 1f
    addi t2, t2, -SPEED
1:  andi t3, t0, BTN_RIGHT
    beqz t3, 2f
    addi t2, t2, SPEED
2:
    bgez t2, 3f             # x < 0 ?  clamp to 0
    li   t2, 0
3:  li   t3, MAX_X          # .equ MAX_X, 304
    ble  t2, t3, 4f         # x > 304 ?  clamp to 304
    mv   t2, t3
4:
    sw   t2, 0(t1)          # one store, after all adjustments
```

Listato 5.4. Limitare la posizione (laboratorio su stato e ingresso, passo 3).

`bgez` legge `t2` come numero con segno: esamina il bit di segno. Anche `ble` è un confronto con segno: -1 è minore di 304, mentre con `bleu` la configurazione `0xFFFFFFFF` sarebbe di gran lunga maggiore. La costante `-SPEED` è un immediato: dodici bit in complemento a due, da -2048 a $+2047$.

Concetto

Un gioco non rileva l'overflow a cose fatte: mantiene i suoi valori dentro un intervallo in cui l'overflow non può avvenire.

5.5.4 Rilevare l'overflow senza flag

RISC-V non ha un flag di riporto né un flag di overflow: meno stati nascosti, un processore più semplice. Quando un programma ha davvero bisogno di saperlo, lo chiede con un confronto (Listato 5.5). Per i numeri senza segno, una somma che ha fatto il giro è sempre minore di ciascuno dei due operandi. Per i numeri con segno, sommare un numero negativo deve rendere la somma più piccola, e sommarne uno non negativo no. Sulla ruota, `sltu` rileva l'attraversamento della cima e il test con segno l'attraversamento del fondo.

```
# unsigned: did the sum wrap around?
    add  t2, t0, t1        # t2 = t0 + t1   (modulo 2^32)
    sltu t3, t2, t0        # t3 = 1 if the sum is smaller than
                           #        an operand: a carry was lost

# signed: did the sign come out wrong?
    add  t2, t0, t1
    slti t3, t1, 0         # t3 = 1 if t1 is negative
    slt  t4, t2, t0        # t4 = 1 if the sum is less than t0
    bne  t3, t4, overflow  # they must agree
```

Listato 5.5. I due test di overflow suggeriti dalla specifica RISC-V.

5.5.5 Estensione del segno: lo stesso numero su più bit

Teorema 5.3 (Estensione del segno)

Un numero in complemento a due su n bit si scrive su $m > n$ bit copiando il suo bit di segno in tutte le nuove posizioni a sinistra.

Dimostrazione. Se $x \geq 0$ il bit di segno è 0 e l'enunciato è l'estensione con zeri dei numeri naturali. Se $x < 0$ la sua configurazione su n bit è $X = 2^n - |x|$ e la configurazione voluta è $2^m - |x| = X + (2^m - 2^n)$; il numero $2^m - 2^n$ è esattamente $m - n$ uni seguiti da n zeri. □

-1 su 4 bit	`1111`	`= -1`
-1 su 8 bit	`1111 1111`	`= -1`
$+5$ su 4 bit	`0101`	`= +5`
$+5$ su 8 bit	`0000 0101`	`= +5`

L'estensione con zeri trasformerebbe `1111` (-1) in `0000 1111` ($+15$). L'istruzione `addi t2, t2, -1` memorizza -1 in dodici bit, e il processore ne estende il segno a trentadue prima di sommare. `lb` estende il segno del byte che carica, `lbu` riempie con zeri: la scelta dipende da che cosa significa il byte.

5.6 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l05.S` nel repository del corso, stampato per intero nel Listato 5.6. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 5.3 è il frame scritto dal test.

Sullo schermo.

I 32 bit di marker_x come lampade. Parte da 2 (una lampada); ogni frame con LEFT premuto sottrae 1: dopo 2, 1 e 0 si accendono tutte le lampade, 0xFFFFFFFF, cioè -1.

Che cosa premere.

Tenete premuto LEFT per qualche frame.

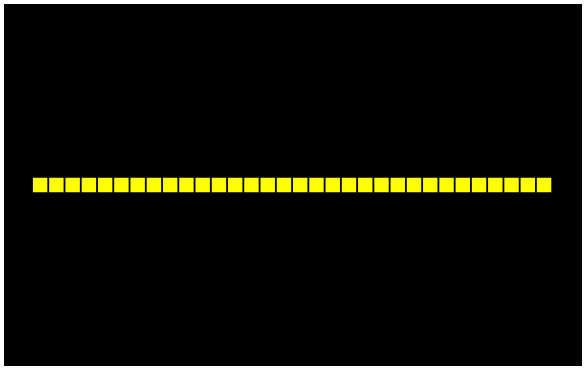


Figura 5.3. `l05.S` su PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 05: Number Representation: Integers
# l05.S: hold LEFT and watch a number go below zero
#
# On the screen: the 32 bits of marker_x as lamps. It starts at 2 (one lamp); each
# frame with LEFT held subtracts 1: after 2, 1 and 0 every lamp lights up, 0xFFFFFFFF,
# which is -1.
# Build: python3 -m prg32 cartridge build l05.S --portable \
#            --entry-prefix l05 --name l05 --out l05.prg32
.option norelax

.equ BLACK,  0x0000
.equ GREY,   0x4208
.equ YELLOW, 0xffe0
.equ BTN_LEFT, 1

.section .text
.global l05_init
.global l05_update
.global l05_draw

l05_init:
    ret

l05_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    call prg32_input_read      # a0 = button mask
    mv   t0, a0

    la   t1, marker_x
    lw   t2, 0(t1)             # t2 = marker_x, starts at 2
    andi t3, t0, BTN_LEFT      # is LEFT pressed?
```

```
33      beqz t3, 1f
34      addi t2, t2, -1            # one step to the left
35  1:  sw   t2, 0(t1)
36      lw   ra, 12(sp)
37      addi sp, sp, 16
38      ret
39
40  l05_draw:
41      addi sp, sp, -16
42      sw   ra, 12(sp)
43      li   a0, BLACK
44      call prg32_gfx_clear
45      call prg32_console_clear   # the number in hexadecimal, on the console
46      la   t0, marker_x          # (the board and QEMU show it on the screen,
47      lw   a0, 0(t0)             #  PRG32-QT in its log)
48      call prg32_console_hex32
49      la   t0, marker_x          # and the same number, bit by bit
50      lw   a0, 0(t0)
51      li   a1, 96
52      call lamps32
53      lw   ra, 12(sp)
54      addi sp, sp, 16
55      ret
56
57  # lamps32: show the 32 bits of a word as a row of lamps, bit 31 on the left.
58  #   a0 = the word, a1 = y of the row.  A lamp is yellow for 1, grey for 0.
59  lamps32:
60      addi sp, sp, -32
61      sw   ra, 28(sp)
62      sw   s0, 24(sp)
63      sw   s1, 20(sp)
64      sw   s2, 16(sp)
65      mv   s0, a0              # the word
66      mv   s1, a1              # y
67      li   s2, 0               # lamp number, 0 on the left
68  1:  li   t0, 31
69      sub  t0, t0, s2          # lamp i shows bit 31 - i
70      srl  t1, s0, t0          # bring that bit down to bit 0
71      andi t1, t1, 1           # keep it, clear all the others
72      li   a4, GREY            # lamp off
73      beqz t1, 2f
74      li   a4, YELLOW          # lamp on
75  2:  slli a0, s2, 3
76      add  a0, a0, s2          # 9 * i
77      addi a0, a0, 16          # x = 16 + 9 * i
78      mv   a1, s1
79      li   a2, 8
80      li   a3, 8
81      call prg32_gfx_rect
82      addi s2, s2, 1
83      li   t0, 32
84      blt  s2, t0, 1b
85      lw   s2, 16(sp)
86      lw   s1, 20(sp)
87      lw   s0, 24(sp)
88      lw   ra, 28(sp)
```

```
89      addi sp, sp, 32
90      ret
91
92  .section .data
93  marker_x: .word 2
```

Listato 5.6. Il programma completo del capitolo, `code/l05.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 5.7; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l05.S --portable \
    --entry-prefix l05 --name l05 --out l05.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l05.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 5.7. Compilare `l05.S` ed eseguirlo su PRG32-QT.

5.7 Verificate di aver capito

D5.1 Quale numero rappresenta in complemento a due la configurazione di 8 bit `1111 1100`?

A. 252
B. -4
C. -3
D. -124

D5.2 Su 8 bit, quale addizione produce un overflow con segno?

A. $100 + 27$
B. $100 + 28$
C. $-100 + 28$
D. $-1 + 1$

D5.3 `t0` contiene `0xFFFFFFFF` e `t1` contiene 1. Dopo `slt t2, t0, t1` e `sltu t3, t0, t1`:

A. `t2` $= 1$, `t3` $= 1$
B. `t2` $= 0$, `t3` $= 0$
C. `t2` $= 1$, `t3` $= 0$
D. `t2` $= 0$, `t3` $= 1$

D5.4 Come si scrive 0,625 in binario?

A. 0,101
B. 0,110
C. 0,625
D. 0,011

5.8 Esercizi

Esercizio 5.1 (prima su carta, poi su PRG32)

1. Convertite 2026 in binario, ottale ed esadecimale; convertite 0,1 in binario fino a otto bit frazionari.
2. Scrivete -100 su 8 bit in segno e modulo e in complemento a due.
3. Calcolate su 8 bit in complemento a due: $90 + 60$, $-90 - 60$, $90 - 60$. Quali vanno in overflow?
4. Nel codice del clamp sostituite `bgez` e `ble` con le loro versioni senza segno, prevedete il comportamento, poi eseguite.
5. Scrivete una `update` che somma `0x10000000` a un contatore a ogni frame e accende un rettangolo rosso quando la somma con segno va in overflow.

Risultato atteso. Su carta: $2026 = \texttt{0x7EA} = 111\,1110\,1010_2$. Su PRG32: il rettangolo rosso lampeggia una volta ogni sedici frame, quando il contatore passa da `0x7...` a `0x8...`.

5.9 Glossario

Inglese	**Italiano**
base (radix)	base
natural number	numero naturale
signed integer	numero relativo (intero con segno)
most significant bit	bit più significativo
remainder	resto
shift	scorrimento (shift)
two's complement	complemento a due (alla base)
sign and magnitude	segno e modulo
overflow	overflow (traboccamento)
carry / borrow	riporto / prestito
sign extension	estensione del segno
to clamp	limitare a un intervallo

5.10 Conclusioni e letture

Ogni base ha i suoi vantaggi: la base 2 per i circuiti, la base 16 per le persone che leggono i bit. n bit rappresentano 2^n numeri: quali, dipende dalla legge di codifica. Il complemento a due trasforma la sottrazione in addizione, così che un solo sommatore fa tutto. L'overflow è il prezzo dei registri finiti: lo si rileva, oppure lo si evita per progetto. In RISC-V i bit non hanno un tipo: è l'istruzione a scegliere tra con segno e senza segno.

Letture. Patterson e Hennessy (2020), paragrafi 2.4 e 3.1–3.2; *Playing with RISC-V*, capitolo 6, «State and Input» (Montella et al., 2026b); Stallings (2021), i capitoli sui sistemi di numerazione e sull'aritmetica del calcolatore.

CAPITOLO 6

Rappresentazione dei numeri: i reali

"Conterò poco, è vero: — diceva l'Uno ar Zero — ma tu che vali? Gnente: propio gnente."
— Trilussa, *Nummeri*, 1944

In questo capitolo

Alla fine di questo capitolo saprete

- codificare interi con segno in complemento a uno e in eccesso-k, e dire a che cosa serve ciascuna codifica;
- rappresentare una grandezza frazionaria in virgola fissa e calcolare con essa usando istruzioni intere;
- spiegare mantissa, esponente e normalizzazione nella virgola mobile;
- codificare e decodificare un numero in IEEE 754 a precisione singola;
- riconoscere zero, numeri denormalizzati, infinito e NaN;
- dire perché l'ESP32-C6 non ha istruzioni in virgola mobile e che cosa fanno invece i giochi PRG32.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI la virgola fissa; complemento a uno ed eccesso k; la virgola mobile IEEE 754; il nome RV32IMAC.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI niente.

6.1 Riscaldamento sulla macchina

Una palla deve attraversare lentamente lo schermo. Un pixel per frame è troppo veloce; zero pixel per frame non è un movimento. Alla palla serve una via di mezzo: 0,375 pixel per frame. Ma il processore del kit è RV32IMAC: ha solo istruzioni intere, e nessuna virgola mobile. Il Listato 6.1 risolve il problema con un trucco.

```
1 .equ SPEED_Q8, 96            # 96 / 256 = 0.375 pixel per frame
2
```

```
ball_update:
    la   t0, ball_x_q8
    lw   t1, 0(t0)          # position, in 1/256 of a pixel
    addi t1, t1, SPEED_Q8   # add a fraction of a pixel
    sw   t1, 0(t0)
    ret

ball_draw:
    ...
    la   t0, ball_x_q8
    lw   t1, 0(t0)
    srai a0, t1, 8          # x in whole pixels: drop 8 bits
    ...
    call prg32_gfx_rect
```

Listato 6.1. Tre ottavi di pixel per frame, con i soli interi.

La posizione è memorizzata *moltiplicata per 256*, e viene divisa per 256 solo quando la palla viene disegnata. In unità di 1/256 di pixel la velocità è $0{,}375 \times 256 = 96$, un onesto intero.

Provate voi

Eseguite il programma con `SPEED_Q8` uguale a 96, poi 16, poi 1.

Il trucco ha un nome, *virgola fissa*. Questo capitolo dice da dove viene e quando non basta, nel qual caso subentra la risposta generale, la *virgola mobile*. Prima dell'una e dell'altra servono altre due codifiche degli interi con segno, perché la virgola mobile è costruita con esse.

6.2 Altre codifiche per gli interi con segno

6.2.1 Complemento a uno

Definizione 6.1 (Complemento a uno)

Su n bit, un numero non negativo $0 \le x < 2^{n-1}$ è rappresentato da sé stesso, $X = x$; un numero negativo è rappresentato da $X = 2^n - 1 - |x|$.

L'intervallo va da $-(2^{n-1}-1)$ a $+(2^{n-1}-1)$. Lo zero ha due rappresentazioni, `0000` e `1111`, quindi si rappresentano $2^n - 1$ numeri, come in segno e modulo. La ragione per usarlo è che il cambio di segno è banale: poiché $2^n - 1$ è una stringa di uni, sottrarre da esso complementa ogni bit,

$$-x \;=\; \texttt{not}\ x.$$

Quindi $6 =$ `0110` e $-6 =$ `1001`; $5 =$ `0101` e $-5 =$ `1010`; $1 =$ `0001` e $-1 =$ `1110`. La `not` bit a bit di RISC-V calcola esattamente questo.

L'aritmetica in complemento a uno lavora modulo $2^n - 1$ invece che modulo 2^n, e questo costa una correzione.

Nessuna correzione		Due negativi		Risultato positivo	
3 +	0011 +	−2 +	1101 +	5 +	0101 +
−4 =	1011 =	−3 =	1100 =	−2 =	1101 =
−1	1110		1 1001 + 1 =		1 0010 + 1 =
		−5	1010	3	0011

Ogni volta che c'è un riporto uscente dal bit più significativo, al risultato va sommato 1: il *riporto circolare* (end-around carry). La correzione serve quando entrambi gli operandi sono negativi, oppure quando gli operandi hanno segni opposti e la somma è positiva. Un sommatore e un complementatore bastano ancora; ma il passo in più e il doppio zero hanno fatto vincere il complemento a due.

6.2.2 Rappresentazione in eccesso-k

Definizione 6.2 (Eccesso-k)

Nella rappresentazione in *eccesso-k* ogni numero è traslato verso l'alto di una costante k, il *bias*: $X = x + k$. Tipicamente $k = 2^{n-1}$.

Su quattro bit con $k = 8$ la configurazione `0000` significa -8, `1000` significa 0 e `1111` significa $+7$. La proprietà decisiva è che l'ordine è conservato: se $x_1 > x_2$ allora $X_1 > X_2$, quindi i numeri con segno si confrontano come se fossero naturali. Il prezzo si paga nell'aritmetica: una somma va corretta sottraendo k, perché $X_1 + X_2 = (x_1 + x_2) + 2k$, e moltiplicazione e divisione sono molto più scomode che con i complementi. L'eccesso-k è quindi preferito dove servono solo somme e confronti, ed è esattamente la situazione dell'esponente di un numero in virgola mobile.

6.2.3 Quattro bit, cinque letture

La Tabella 6.1 completa la tabella del Capitolo 5. Segno e modulo e complemento a uno hanno due zeri. Complemento a due ed eccesso-8 coprono lo stesso intervallo, da -8 a $+7$, con un solo zero; differiscono soltanto nel bit più significativo, che è invertito. Solo l'eccesso-k mantiene le configurazioni nello stesso ordine dei numeri.

Tabella 6.1. Quattro bit, cinque letture. S-M: segno e modulo; C1, C2: complemento a uno e a due; Ecc-8: eccesso-8.

Bit	Nat.	S-M	C1	C2	Ecc-8	Bit	Nat.	S-M	C1	C2	Ecc-8
0000	0	+0	+0	0	−8	1000	8	−0	−7	−8	0
0001	1	+1	+1	+1	−7	1001	9	−1	−6	−7	+1
0010	2	+2	+2	+2	−6	1010	10	−2	−5	−6	+2
0011	3	+3	+3	+3	−5	1011	11	−3	−4	−5	+3
0100	4	+4	+4	+4	−4	1100	12	−4	−3	−4	+4
0101	5	+5	+5	+5	−3	1101	13	−5	−2	−3	+5
0110	6	+6	+6	+6	−2	1110	14	−6	−1	−2	+6
0111	7	+7	+7	+7	−1	1111	15	−7	−0	−1	+7

6.3 Virgola fissa

6.3.1 Numeri in virgola fissa

In virgola fissa la parte intera e la parte frazionaria di un numero sono memorizzate una accanto all'altra in una sola parola. La posizione della virgola è fissa e implicita: non è memorizzata, la conosce il programma.

Definizione 6.3 (Virgola fissa)

Con f bit frazionari un numero x è memorizzato come l'intero $X = x \cdot 2^f$ (arrotondato). La *risoluzione* è 2^{-f}. La notazione $Qm.f$ indica un formato con m bit interi e f bit frazionari.

Di solito l'intera parola è in complemento a due. La virgola fissa è il modo in cui giochi, audio e software di controllo trattano le frazioni sui processori privi di un'unità in virgola mobile. È anche il modo in cui andrebbe memorizzato il denaro: come numero intero di centesimi.

6.3.2 La palla, bit per bit: Q24.8

La palla del riscaldamento è nel formato Q24.8 (Figura 6.1). Quando la sua variabile contiene `0x00002A60` = 10848, la palla si trova a 10848/256 = 42,375 pixel: i 24 bit alti, `0x00002A`, sono i 42 pixel interi e gli 8 bassi, `0x60`, sono 96/256 = 0,375.

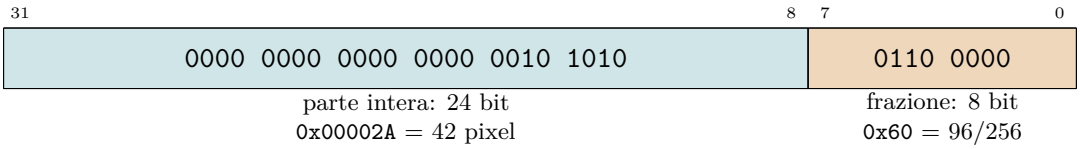


Figura 6.1. `ball_x_q8 = 0x00002A60`: la palla si trova a 42,375 pixel.

Dopo un altro frame il valore è `0x2AC0` (42,75), poi `0x2B20` (43,125): la palla attraversa il confine di un pixel ogni due o tre frame, e in media avanza di 0,375.

L'aritmetica in virgola fissa è aritmetica intera con un po' di attenzione (Tabella 6.2). Somme e differenze non ne richiedono, perché le virgole dei due operandi sono allineate. Il prodotto di due numeri scalati di 2^f è scalato di 2^{2f}, e va riportato indietro con uno scorrimento di f. Lo scorrimento che riporta ai pixel interi è quello *aritmetico*, `srai`, che conserva il segno (Capitolo 5).

Tabella 6.2. Le operazioni in virgola fissa.

Operazione	In virgola fissa	RISC-V
somma, sottrazione	come tra interi: le virgole sono allineate	`add`, `sub`, `addi`
ai pixel interi	scorrimento a destra di f, conservando il segno	`srai a0, t1, 8`
prodotto	moltiplicare, poi scorrere a destra di f	`mul`, poi `srai`

6.3.3 Numeri razionali: l'errore di approssimazione

I numeri reali sono densi: anche in un intervallo limitato, non ogni numero si può rappresentare con un numero finito di bit. A ogni numero si associa quindi il più vicino tra quelli

rappresentabili. In Q24.8 il numero 0,1 diventa $26/256 = 0{,}1015625$, perché $0{,}1 \times 256 = 25{,}6$ viene arrotondato a 26: un errore di circa l'1,6%.

L'errore di approssimazione è al massimo metà della risoluzione: 1/512 con otto bit frazionari. L'*underflow* è il caso di un numero vicino a zero, ma non zero, che viene rappresentato da zero: con una velocità di 0,001 pixel per frame la costante verrebbe arrotondata a 0, e la palla non si muoverebbe mai.

Concetto

In virgola fissa l'errore *assoluto* è lo stesso ovunque sulla retta dei numeri. La risoluzione di otto bit frazionari è 1/256: nulla di più piccolo si distingue da zero.

6.4 Virgola mobile

La virgola fissa spende tutti i suoi bit su un unico intervallo fissato. La virgola mobile ne spende alcuni per dire *dove* si trova l'intervallo: scambia precisione con ampiezza dell'intervallo.

6.4.1 Numeri reali in virgola mobile

Definizione 6.4 (Virgola mobile)

Un numero reale x è rappresentato in virgola mobile da una coppia (m,e) tale che

$$x = m \cdot b^{e},$$

dove m è la *mantissa*, e l'*esponente* e b la base.

La virgola è «mobile» perché è l'esponente a dire dove si trova. È la notazione scientifica del laboratorio di fisica. Infinite coppie rappresentano lo stesso numero,

$$346{,}09801 \;=\; (346{,}09801;\,0) \;=\; (346098{,}01;\,-3) \;=\; (0{,}034609801;\,4),$$

e la *normalizzazione* sceglie un unico rappresentante: la virgola va subito dopo la prima cifra significativa, (3,4609801; 2).

6.4.2 L'intervallo di un piccolo formato decimale

Si consideri un formato in base 10 con quattro cifre per la mantissa e due per l'esponente (Figura 6.2). Rappresenta i numeri da $-9{,}999 \times 10^{99}$ a $-1{,}000 \times 10^{-99}$, lo zero, e i numeri da $+1{,}000 \times 10^{-99}$ a $+9{,}999 \times 10^{99}$. Si ha un *overflow* quando il modulo è troppo grande per l'esponente; un *underflow* quando è troppo piccolo, e il numero collassa a zero.

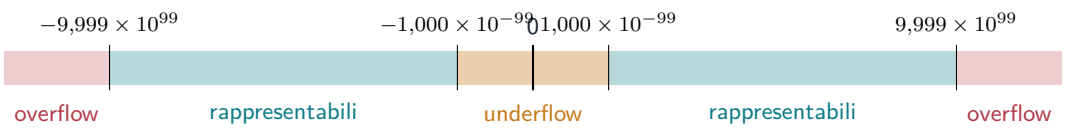


Figura 6.2. Un formato decimale con quattro cifre di mantissa e due di esponente (non in scala).

6.4.3 Approssimazione: errore assoluto ed errore relativo

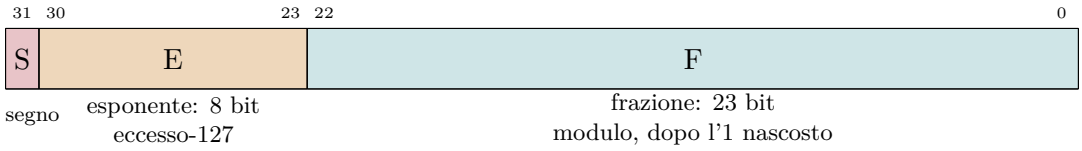
In un formato del genere l'errore assoluto non è costante. Vicino a zero la distanza tra due numeri consecutivi è $1{,}001 \times 10^{-99} - 1{,}000 \times 10^{-99} = 0{,}001 \times 10^{-99}$; vicino al limite superiore è $9{,}999 \times 10^{99} - 9{,}998 \times 10^{99} = 0{,}001 \times 10^{99}$. L'errore assoluto è piccolo sui numeri piccoli e grande sui numeri grandi. Ciò che è costante lungo tutto l'asse è l'errore *relativo*, perché il numero di cifre significative, quattro, è lo stesso ovunque.

Concetto

Virgola fissa: errore assoluto costante, errore relativo che esplode vicino a zero. Virgola mobile: errore relativo costante, errore assoluto che cresce con il numero.

6.4.4 IEEE 754 a precisione singola

Fino al 1985 ogni costruttore aveva un proprio formato in virgola mobile, e i programmi davano risultati diversi su macchine diverse. Lo standard IEEE 754 (1985, rivisto nel 2008 e nel 2019) vi ha posto fine (IEEE, 2019). Il suo formato a precisione singola divide 32 bit in tre campi (Figura 6.3).



$$\text{valore} \;=\; (-1)^{S} \times 1{,}F \times 2^{\,E-127}$$

Figura 6.3. IEEE 754 a precisione singola.

- La mantissa è in segno e modulo: un bit di segno e 23 bit di modulo.
- Una mantissa binaria normalizzata comincia sempre con 1. Quel bit è *nascosto*: non viene memorizzato, e si guadagna un bit di precisione.
- L'esponente è in eccesso-127 (non 128): gli esponenti veri vanno da -126 a $+127$. Grazie alla codifica in eccesso, e alla posizione dell'esponente prima della frazione, due numeri positivi in virgola mobile si possono confrontare come se fossero naturali.
- Le configurazioni $E = 0$ ed $E = 255$ sono riservate: zero e numeri denormalizzati, infinito e NaN.

Esempio 6.1 ($-5{,}125$ in IEEE 754)

1. Si convertono le due parti: $5 = 101_2$ e $0{,}125 = 1/8 = 0{,}001_2$, quindi $5{,}125 = 101{,}001_2$.
2. Si normalizza: $101{,}001_2 = 1{,}01001_2 \times 2^2$.
3. Segno $S = 1$; esponente $E = 2 + 127 = 129 = \texttt{1000 0001}$.
4. Frazione $F = \texttt{01001}$ seguita da zeri: l'1 iniziale è nascosto.

$$\underbrace{\texttt{1}}_{S}\ \underbrace{\texttt{1000 0001}}_{E}\ \underbrace{\texttt{0100 1000 0000 0000 0000 000}}_{F}$$

Raggruppato a nibble, `1100 0000 1010 0100 0000 ...` è `0xC0A40000`.

6.4.5 IEEE 754 in sintesi

La Tabella 6.3 confronta i due formati di uso comune, e la Tabella 6.4 elenca il significato di tutte le configurazioni.

Tabella 6.3. IEEE 754 a precisione singola e doppia.

	Precisione singola	**Precisione doppia**
Bit totali	32	64
Bit di segno	1	1
Bit di esponente	8	11
Bit di frazione	23	52
Codifica dell'esponente	eccesso-127	eccesso-1023
Intervallo dell'esponente	$-126 \ldots +127$	$-1022 \ldots +1023$
Minimo normalizzato	$2^{-126} \approx 1{,}2 \times 10^{-38}$	$2^{-1022} \approx 2{,}2 \times 10^{-308}$
Massimo	circa $2^{128} \approx 3{,}4 \times 10^{38}$	circa $2^{1024} \approx 1{,}8 \times 10^{308}$
Minimo denormalizzato	circa $1{,}4 \times 10^{-45}$	circa $4{,}9 \times 10^{-324}$
Cifre decimali	circa 7	circa 16

Tabella 6.4. IEEE 754: che cosa significano le configurazioni.

Tipo	**Segno**	**Esponente** E	**Frazione** F	**Valore**
Normalizzato	0 o 1	$0 < E <$ max	qualunque	$(-1)^S \times 1{,}F \times 2^{E-bias}$
Denormalizzato	0 o 1	tutti zeri	non zero	$(-1)^S \times 0{,}F \times 2^{1-bias}$
Zero	0 o 1	tutti zeri	tutti zeri	$+0$ o -0
Infinito	0 o 1	tutti uni	tutti zeri	$+\infty$ o $-\infty$
Not a Number	0 o 1	tutti uni	non zero	NaN: un risultato non valido

I numeri *denormalizzati* (o subnormali) riempiono il vuoto dell'underflow attorno allo zero con una precisione che diminuisce gradualmente. La divisione $1/0$ dà infinito; $0/0$ e la radice quadrata di -1 danno NaN. NaN non è uguale a nulla, nemmeno a sé stesso.

6.4.6 La virgola mobile non è l'insieme dei numeri reali

Un numero in virgola mobile è un numero razionale scelto da un insieme finito, e l'aritmetica su tali numeri obbedisce a leggi che non sono quelle del campo reale (Goldberg, 1991).

- 0,1 non ha un'espansione binaria finita: è memorizzato in modo approssimato, come 0,8 nel Capitolo 5.
- Due risultati in virgola mobile non vanno mai confrontati per uguaglianza: se ne confronta la differenza con una tolleranza (Listato 6.2).
- L'addizione non è associativa: $(a + b) + c$ può differire da $a + (b + c)$.
- Sommare un numero molto piccolo a uno molto grande può lasciare invariato quello grande.
- Per il denaro e per le posizioni di un gioco, gli interi scalati sono spesso lo strumento giusto.

```
double a = 0.1 + 0.2;
```

```
// a == 0.3 ?   false
// a = 0.30000000000000004

// better:
// fabs(a - 0.3) < 1e-9
```

Listato 6.2. La somma più famosa dell'informatica.

6.4.7 Oltre la singola e la doppia

La scelta di una rappresentazione è una decisione ingegneristica che si prende ancora oggi. La *mezza precisione*, aggiunta allo standard nel 2008, ha sedici bit: 1 di segno, 5 di esponente, 10 di frazione; dà circa tre cifre decimali in metà della memoria della precisione singola. Conta perché le reti neurali contengono miliardi di numeri: dimezzare ciascuno dimezza memoria, banda ed energia. Molti modelli vanno oltre e vengono memorizzati, per *quantizzazione*, come interi a 8 bit con un fattore di scala: di nuovo virgola fissa, su scala industriale. I bit sono un bilancio; intervallo, precisione e costo tirano in direzioni diverse, su una GPU come su una palla in Q24.8.

6.5 Ritorno alla macchina

6.5.1 La virgola mobile su RISC-V, e sul kit

RISC-V è modulare: una piccola base obbligatoria più estensioni facoltative (Tabella 6.5). Un processore RISC-V con le estensioni F e D ha altri trentadue registri, da `f0` a `f31`, per i valori in virgola mobile, e istruzioni come `fadd.s`, `fmul.s`, `flw` e `fsw` che lavorano su numeri IEEE 754; `fcvt.w.s` converte in intero e `fcvt.s.w` da intero. Processori così si trovano in portatili, server e supercalcolatori.

L'ESP32-C6 è RV32IMAC: non ha né F né D, e quindi nessuna istruzione in virgola mobile. Un programma C può comunque usare il tipo `float`: il compilatore chiama routine software come `__addsf3`, che realizzano esattamente l'algoritmo di questo capitolo con scorrimenti, maschere e somme intere, al costo di decine o centinaia di istruzioni intere per operazione. I giochi PRG32 usano interi e virgola fissa: veloci, abbastanza esatti, e visibili nel debugger.

Tabella 6.5. Leggere il nome RV32IMAC.

Lettera	Estensione	Che cosa aggiunge	ESP32-C6
RV32I	insieme intero di base	registri a 32 bit, somma, logica, load e store, salti	sì
M	moltiplicazione e divisione	`mul`, `mulh`, `div`, `rem` e le loro forme senza segno	sì
A	atomiche	istruzioni per la condivisione sicura tra task	sì
C	compresse	codifiche a 16 bit delle istruzioni più comuni	sì
F	precisione singola	da `f0` a `f31` e istruzioni IEEE 754 a precisione singola	no
D	precisione doppia	istruzioni IEEE 754 a precisione doppia	no

Un microcontrollore sceglie le estensioni che gli servono. È una delle ragioni per cui la stessa architettura scala da un chip che costa due euro a un supercalcolatore.

6.6 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l06.S` nel repository del corso, stampato per intero nel Listato 6.3. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 6.4 è il frame scritto dal test.

Sullo schermo.
Un quadrato bianco che attraversa lentamente lo schermo da sinistra: un pixel ogni 2,67 frame.

Che cosa premere.
Niente: il programma va da solo.



Figura 6.4. `l06.S` su PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 06: Number Representation: Real Numbers
# l06.S: three-eighths of a pixel per frame, with integers only
#
# On the screen: a white square that crosses the screen slowly from the left: one
# pixel every 2.67 frames.
# Build: python3 -m prg32 cartridge build l06.S --portable \
#            --entry-prefix l06 --name l06 --out l06.prg32
.option norelax

.equ SPEED_Q8, 96            # 96 / 256 = 0.375 pixel per frame

.section .text
.global l06_init
.global l06_update
.global l06_draw

l06_init:
    ret

l06_update:
ball_update:
    la   t0, ball_x_q8
    lw   t1, 0(t0)           # position, in 1/256 of a pixel
    addi t1, t1, SPEED_Q8    # add a fraction of a pixel
    sw   t1, 0(t0)
    ret
```

```
27
28 l06_draw:
29 ball_draw:
30     addi sp, sp, -16
31     sw   ra, 12(sp)
32     li   a0, 0
33     call prg32_gfx_clear
34     la   t0, ball_x_q8
35     lw   t1, 0(t0)
36     srai a0, t1, 8           # x in whole pixels: drop 8 bits
37     li   a1, 100
38     li   a2, 8
39     li   a3, 8
40     li   a4, 0xffff
41     call prg32_gfx_rect
42     lw   ra, 12(sp)
43     addi sp, sp, 16
44     ret
45
46 .section .data
47 ball_x_q8: .word 0           # Q24.8: 24 bits of pixels, 8 bits of fraction
```

Listato 6.3. Il programma completo del capitolo, `code/l06.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 6.4; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l06.S --portable \
    --entry-prefix l06 --name l06 --out l06.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l06.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 6.4. Compilare `l06.S` ed eseguirlo su PRG32-QT.

6.7 Verificate di aver capito

D6.1 Quale codifica con segno mantiene le configurazioni di bit nello stesso ordine dei numeri?

A. Segno e modulo
B. Complemento a uno
C. Complemento a due
D. Eccesso-k

D6.2 In virgola fissa Q24.8, quale numero reale rappresenta la parola `0x00000180`?

A. 384
B. 1,5
C. 1,8
D. 0,375

D6.3 Quale numero reale è la configurazione IEEE 754 a precisione singola `0x40400000`?

A. 2,0
B. 3,0
C. 4,0
D. 0,75

D6.4 In IEEE 754, un campo esponente di tutti uni con una frazione di tutti zeri significa:

A. zero
B. un numero denormalizzato
C. infinito
D. NaN

6.8 Esercizi

Esercizio 6.1 (prima su carta, poi su PRG32)

1. Scrivete -6 su quattro bit in segno e modulo, complemento a uno, complemento a due ed eccesso-8.
2. Codificate 10,5 e 0,15625 in IEEE 754 a precisione singola; date il risultato in esadecimale.
3. Decodificate `0xBF800000`.
4. Eseguite la palla con `SPEED_Q8` uguale a 96, 16 e 1. Dopo quanti frame si sposta di un pixel in ciascun caso?
5. Date alla palla anche una velocità verticale, e fatela rimbalzare: cambiate di segno la velocità con `neg` quando raggiunge un bordo.

Risultato atteso. Su carta: `1110`, `1001`, `1010`, `0010`; `0x41280000` e `0x3E200000`; $-1{,}0$. Su PRG32: la palla avanza di un pixel ogni 2,67, 16 e 256 frame; con il rimbalzo si muove in diagonale e resta sullo schermo.

6.9 Glossario

Inglese	**Italiano**
one's complement	complemento a uno (alla base diminuita)
excess-k (biased)	eccesso-k
fixed point	virgola fissa
floating point	virgola mobile
single / double precision	singola / doppia precisione
rounding	arrotondamento
mantissa / fraction	mantissa
exponent	esponente
normalisation	normalizzazione
hidden bit	bit nascosto
denormalised	denormalizzato
Not a Number (NaN)	non un numero (NaN)

6.10 Conclusioni e letture

Gli interi hanno più rappresentazioni: 8, 16, 32 o 64 bit, con segno o senza, complementi o eccesso. La virgola fissa memorizza una frazione come intero scalato: errore assoluto costante, la velocità degli interi. La virgola mobile memorizza una mantissa e un esponente: un intervallo enorme, errore relativo costante. IEEE 754 è lo standard universale per la virgola mobile, con configurazioni speciali per zero, infinito e NaN. Su un processore senza l'estensione F, la virgola fissa è la scelta naturale.

Letture. Patterson e Hennessy (2020), paragrafo 3.5; Goldberg (1991); *Playing with RISC-V*, l'appendice sull'interfaccia del runtime, per playfield e parallasse in Q8 (Montella et al., 2026b).

Il primo blocco è completo. Il prossimo capitolo mette al lavoro le sue quattro lezioni in un programma vero: la prima parte di Verm1.

CAPITOLO 7

Verm1, parte 1: una testa che attraversa il mondo

"Un viaggio di mille miglia comincia con un solo passo."
— Laozi, *Tao Te Ching*

In questo capitolo

Alla fine di questo capitolo avrete

- una cartuccia vostra, `verm1.prg32`, costruita da un solo file assembly scritto da voi;
- un titolo, una griglia di 40 per 24 celle, e la testa del verme disegnata in una di esse;
- la testa che si muove da sola, una cella alla volta, a una velocità che controllate in virgola fissa;
- un mondo senza bordi: oltre l'ultima colonna la testa rientra dalla prima;
- l'abitudine al ciclo di lavoro: modificare, compilare, eseguire, guardare, e fermare il programma nel debugger.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI il progetto Verm1; contratto init/update/draw; un colore è un numero; gli array in memoria; .text, .rodata, .data; indirizzo, valore, load, store; perché si salva ra; shift come moltiplicazione; la virgola fissa; le pseudo-istruzioni; l'ordine little-endian.

DA UTILIZZARE OGGI gli strumenti di PRG32, il debugger di PRG32-QT (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI formati delle istruzioni (Capitolo 19).

Questo è il primo tutorial del progetto. Usa soltanto ciò che i Capitoli da 2 a 6 hanno insegnato, ed è pensato per essere seguito alla tastiera: il programma lo scrivete voi. Tutto ciò che è stampato nei listati è letto dal file di riferimento `verm1_part1.S`, che viene compilato ed eseguito automaticamente; il file intero è nell'Appendice D.

7.1 Il progetto

7.1.1 Verm1: un omaggio a Snake

Verm1, che si legge *vermone*, è un verme che si muove su una griglia, mangia, cresce, e non deve mordersi. Il gioco ha una lunga storia: apparve come *Blockade*, una macchina da sala giochi della Gremlin, nel 1976, e come *Snake* sul Nokia 6110 nel 1997, scritto da Taneli Armanto, divenne uno dei giochi più giocati di sempre. Verm1 è un omaggio, non una copia: il nome, le regole di dettaglio e l'aspetto sono suoi. È abbastanza piccolo da essere scritto da zero in assembly e abbastanza ricco da richiedere ogni argomento del corso, e ogni sua riga la scrivete voi: non c'è un motore, e non c'è una libreria oltre le chiamate di PRG32. L'implementazione di riferimento che i capitoli tutorial seguono, e con la quale potete sempre confrontare un programma vostro, è un contributo di Giuseppe Coviello (Appendice D). La Figura 7.1 mostra la destinazione.

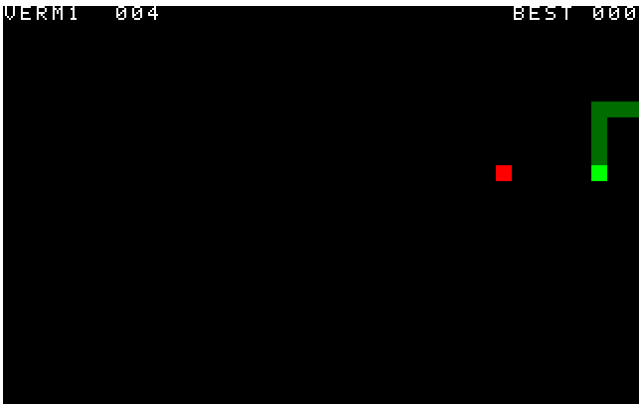


Figura 7.1. Verm1 alla fine del libro (parte 4).

7.1.2 Il piano

Tabella 7.1. Quattro parti e una pubblicazione.

Capitolo	Aggiungete	Usa i Capitoli
7 Parte 1	una griglia, una testa, il movimento automatico, il rientro dal bordo opposto	da 3 a 6: informazione e numeri
12 Parte 2	la guida con il joystick, il cibo, un punteggio	da 8 a 11: logica e circuiti
17 Parte 3	un corpo che cresce, le collisioni, le schermate del titolo e di fine partita	da 13 a 16: stato e memoria
22 Parte 4	il disegno veloce, il punteggio in cifre, i livelli, il suono	da 18 a 21: la macchina
23 Lo Store	un bundle con manifesto, icona e crediti	tutti

Ogni parte termina con un programma che funziona (Tabella 7.1).

Attenzione

Non cominciate mai una parte con un programma che non funziona. Se il vostro è rotto, ripartite dal file di riferimento della parte precedente.

La parte 1 prende dai capitoli letti finora ciò che segue.

Da	Idea	Dove compare
Capitolo 2	i registri, `la`/`lw`/`sw`, `call` e lo stack frame	dappertutto
Capitolo 3	i giochi retro come esemplari	la scelta del gioco
Capitolo 4	i colori come numeri RGB565; le stringhe terminate da zero	`GREEN`, il titolo
Capitolo 5	gli scorrimenti come moltiplicazioni; i confronti; il rientro dal bordo	da cella a pixel; il bordo del mondo
Capitolo 6	la virgola fissa: una frazione come intero scalato	la velocità del verme

7.2 Passo 0: il ciclo di lavoro

Create una cartella `~/verm1` e teneteci un solo file, `verm1.S`, che crescerà di parte in parte. I due comandi del Listato 7.1 compilano la cartuccia e la inviano a PRG32-QT; sono gli stessi in ogni tutorial. PRG32-QT ascolta sulla porta 8080 e mostra il suo indirizzo nella schermata Setup: usate quell'indirizzo se `localhost` non risponde. La stessa cartuccia gira su QEMU (`qemu upload`, poi `qemu run`) e sulla scheda (`--url http://192.168.4.1`).

```
# 1. build the cartridge (from your PRG32 folder, toolchain active)
python3 -m prg32 cartridge build ~/verm1/verm1.S --portable \
    --entry-prefix verm1 --name verm1 --out ~/verm1/verm1.prg32

# 2. send it to PRG32-QT, running on the same computer
python3 -m prg32 esp32c6 upload-and-run ~/verm1/verm1.prg32 \
    --url http://localhost:8080 --slot cart0

# or: in PRG32-QT, import the file verm1.prg32 and choose Run Cartridge
```

Listato 7.1. Compilare ed eseguire: modificare, compilare, eseguire, guardare.

7.3 Passo 1: tre funzioni che ritornano

```
# verm1.S - my Verm1
.option norelax

.section .text
.global verm1_init
.global verm1_update
.global verm1_draw

verm1_init:
    ret

verm1_update:
    ret

verm1_draw:
    ret
```

Listato 7.2. Passo 1 **(nuovo)**: lo scheletro.

È lo scheletro del Capitolo 2 con un altro prefisso: `verm1` è il nome passato a `--entry-prefix`. `init` viene eseguita una volta; `update` e `draw` una volta per frame.

Eseguite

Scrivetelo, compilatelo, eseguitelo: uno schermo nero è la risposta giusta. Se qui la compilazione fallisce il problema è l'installazione, non il programma: risolvetelo prima di proseguire. D'ora in poi: aggiungete una cosa, eseguite, guardate. Mai due cose insieme.

7.4 Passo 2: una griglia e una testa

7.4.1 Costanti: i colori e la griglia

Sostituite le prime righe dello scheletro con quelle del Listato 7.3.

```
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
.equ GREEN, 0x07e0
.equ CELL,  8               # a cell is 8 x 8 pixels
.equ COLS,  40              # 320 / 8 cells across
.equ ROWS,  24              # 192 / 8 cells down
.equ TOP,   8               # the top 8 pixels are the title line
.equ SPEED_Q8, 40           # 40/256 of a cell per frame (Q24.8)

.section .text
.global verm1_init
.global verm1_update
.global verm1_draw
```

Listato 7.3. Passo 2 **(modificato)**: l'inizio del file.

I colori sono numeri RGB565: `0x07e0` è verde puro. Lo schermo è largo 320 pixel e una cella è larga 8 pixel, il che fa 40 colonne. Gli 8 pixel in alto contengono il titolo, e ne restano 192: 24 righe. La direttiva `.equ` dà un nome a un numero e non occupa memoria. La costante `SPEED_Q8` si usa nel passo 3.

7.4.2 Il mondo di Verm1

Concetto

Il verme non vive nei pixel: vive nelle celle. Una posizione è una colonna, da 0 a 39, e una riga, da 0 a 23.

Per disegnare una cella le sue coordinate vengono trasformate in pixel (Figura 7.2): $x = \mathit{colonna} \times 8$, uno scorrimento a sinistra di 3, e $y = \mathit{riga} \times 8 + 8$, perché prima viene la riga

del titolo. La cella (0,0) è al pixel (0,8); la cella (39,23) è a (312,192). Pensare in celle rende semplice ogni regola successiva: muoversi è sommare 1.

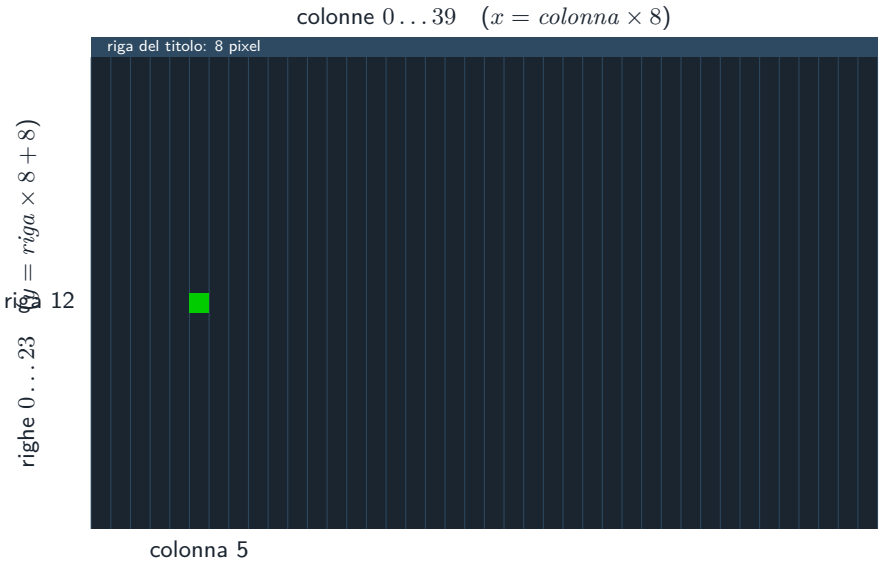


Figura 7.2. Il mondo di Verm1: 40 per 24 celle di 8 per 8 pixel. La testa parte dalla colonna 5 della riga 12.

7.4.3 Variabili e una stringa

Mettete il Listato 7.4 alla fine del file. La sezione `.rodata` contiene le costanti: il titolo termina con un byte a zero. La sezione `.data` contiene le variabili, una parola di 32 bit ciascuna. I loro valori iniziali sono solo un punto di partenza: `init` li imposterà di nuovo.

```
.section .rodata
title:   .asciz "VERM1"

.section .data
head_x:  .word 5                # column of the head, 0 .. COLS-1
head_y:  .word 12               # row of the head, 0 .. ROWS-1
step_q8: .word 0                # fraction of a step, in 1/256
```

Listato 7.4. Passo 2 **(nuovo)**: i dati, alla fine del file.

7.4.4 `init`: mettere la testa al suo posto

```
verm1_init:
    la   t0, head_x
    li   t1, 5
    sw   t1, 0(t0)              # the head starts in column 5 ...
    la   t0, head_y
    li   t1, 12
    sw   t1, 0(t0)              # ... of row 12
    la   t0, step_q8
    sw   zero, 0(t0)            # no fraction of a step yet
    ret
```

Listato 7.5. Passo 2 **(modificato)**: `verm1_init`.

`la` prende un *indirizzo*, `sw` vi memorizza un *valore*. La funzione imposta la partenza in modo esplicito: un programma non dovrebbe mai fidarsi di ciò che la memoria conteneva. `zero` è il registro che si legge sempre 0. La funzione è una foglia: non fa chiamate, quindi non le serve uno stack frame.

7.4.5 `draw`: il titolo e la testa

```
verm1_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 0                  # x
    li   a1, 0                  # y
    la   a2, title              # address of the string
    li   a3, WHITE
    li   a4, BLACK
    call prg32_gfx_text8
    la   t0, head_x
    lw   a0, 0(t0)
    slli a0, a0, 3              # x = column * 8
    la   t0, head_y
    lw   a1, 0(t0)
    slli a1, a1, 3
    addi a1, a1, TOP            # y = row * 8 + 8
    li   a2, CELL               # width
    li   a3, CELL               # height
    li   a4, GREEN
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 7.6. Passo 2 **(modificato)**: `verm1_draw`.

Cancellare lo schermo, scrivere il titolo, disegnare un rettangolo: tre chiamate, con gli argomenti che viaggiano da `a0` ad `a4`, in ordine. Le firme sono `prg32_gfx_text8(x, y, stringa, primo piano, sfondo)` e `prg32_gfx_rect(x, y, larghezza, altezza, colore)`. `slli` di 3 moltiplica per 8: da cella a pixel. Poiché `draw` fa chiamate, salva `ra` in uno stack frame.

Eseguite: una testa sulla griglia

Risultato atteso ora: la parola `VERM1` in alto a sinistra e un quadrato verde fermo in colonna 5, riga 12 (la Figura 7.3 lo mostra più tardi, quando ha cominciato a muoversi).

7.5 Passo 3: il movimento

7.5.1 Quanto deve andare veloce un verme?

`update` viene eseguita circa trenta volte al secondo. Una cella per frame attraverserebbe lo schermo in poco più di un secondo: ingiocabile. Ciò che serve è una frazione di cella per

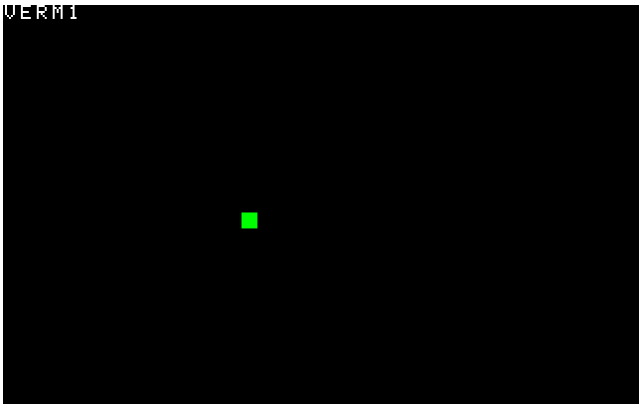


Figura 7.3. Il titolo e la testa. Questa immagine è stata presa dai test automatici 64 frame dopo l'avvio della parte 1 finita: la testa è in colonna 15.

frame, e il processore del kit ha solo interi. La risposta è la virgola fissa del Capitolo 6, nel formato Q24.8: il programma tiene una frazione in unità di 1/256 di cella, e fa un passo ogni volta che si è accumulata una cella intera. Con `SPEED_Q8` = 40 il verme avanza di $40/256 \approx 0{,}156$ celle per frame: circa un passo ogni 6 frame.

È la palla del Capitolo 6 trasformata in un timer. Il resto non viene buttato via, quindi la velocità media è esatta: 256 frame danno esattamente 40 passi.

7.5.2 `update`: accumulare, fare un passo, rientrare

```
verm1_update:
    la   t0, step_q8
    lw   t1, 0(t0)
    addi t1, t1, SPEED_Q8       # add a fraction of a cell
    li   t2, 256
    blt  t1, t2, 2f             # less than one whole cell: wait
    sub  t1, t1, t2             # one cell done: keep the remainder
    sw   t1, 0(t0)

    la   t0, head_x
    lw   t3, 0(t0)
    addi t3, t3, 1              # one cell to the right
    li   t4, COLS
    blt  t3, t4, 1f
    li   t3, 0                  # past the right edge: back to column 0
1:  sw   t3, 0(t0)
    ret
2:  sw   t1, 0(t0)
    ret
```

Listato 7.7. Passo 3 **(modificato)**: `verm1_update`.

La velocità viene sommata alla frazione; sotto 256, la funzione aspetta. A 256 sottrae 256, conserva il resto, e sposta la testa di una cella a destra. La colonna 40 non esiste: oltre il bordo destro la testa torna alla colonna 0. La funzione ha due uscite, e ciascuna memorizza ciò che ha cambiato. `blt` confronta numeri con segno, il che qui è corretto perché ogni valore

è piccolo e positivo. Le etichette locali `1:` e `2:` si richiamano come `1f` e `2f`: «il prossimo 1, o 2, *in avanti*» (*forward*).

7.5.3 Tracciatela a mano

Tabella 7.2. Una traccia di `verm1_update`.

Frame	`step_q8` prima	+ 40	Cella intera?	`step_q8` dopo	`head_x`
1	0	40	no	40	5
2	40	80	no	80	5
6	200	240	no	240	5
7	240	280	sì: $280 - 256$	24	6
13	224	264	sì: $264 - 256$	8	7
64	–	–	–	0	15

I passi arrivano ogni 6 o 7 frame (Tabella 7.2): il resto porta con sé la differenza, come fa un anno bisestile. Dopo 64 frame la somma è $64 \times 40 = 2560 = 10 \times 256$: dieci passi, e la testa è in colonna 15, esattamente come nella Figura 7.3. Completate i frame da 8 a 12 per esercizio.

Eseguite: la testa attraversa il mondo

1. Compilate ed eseguite: la testa si muove verso destra, circa cinque celle al secondo.
2. Aspettate il bordo destro: la testa esce dalla colonna 39 e rientra dalla colonna 0.
3. Cambiate `SPEED_Q8` in 8, poi in 128, poi in 256: prevedete la velocità prima di eseguire.
4. Cambiate la riga di partenza in `init`, e il colore della testa.

Risultato atteso. La testa attraversa le quaranta colonne in 256 frame, circa otto secondi e mezzo a trenta frame al secondo, per sempre. Con `SPEED_Q8` = 256 si muove di una cella per frame; valori più grandi danno comunque un passo per frame, perché `update` fa al più un passo: un limite che vale la pena scoprire.

7.6 Passo 4: guardare dentro con PRG32-QT

Il debugger fa parte del metodo, non è uno strumento di emergenza.

Fermate il programma e leggete la macchina

1. In PRG32-QT aprite *Settings* e abilitate *Run cartridges in RISC-V debug mode*.
2. Eseguite la vostra cartuccia: il gioco è a sinistra, il debugger a destra.
3. Premete **Update**: il listato salta a `verm1_update`. Fate clic sul margine per mettere un breakpoint sulla sua prima istruzione.
4. Il programma si ferma lì al frame successivo. Premete **Step** e guardate i registri cambiare, un'istruzione alla volta.
5. Dopo le due istruzioni di `la`, `t0` contiene un indirizzo: scrivetelo nel campo *Memory* per vedere `step_q8` in memoria.

6. Premete **Resume**, e guardate i byte di `step_q8` crescere di 40 (`0x28`) a ogni fermata.

Resume ignora una volta il breakpoint corrente, così che ogni pressione esegua un frame intero.

7.6.1 Che cosa mostra il listato

Il debugger non mostra il file sorgente: mostra il codice come l'assemblatore lo ha lasciato (Listato 7.8).

```
verm1_update:
  auipc   t0, 0x0          # la t0, step_q8   is two instructions:
  addi    t0, t0, ...      #   t0 = address of step_q8
  lw      t1, 0(t0)
  addi    t1, t1, 40       # SPEED_Q8 is gone: only the number is left
  addi    t2, zero, 256    # li t2, 256
  blt     t1, t2, ...
  sub     t1, t1, t2
  sw      t1, 0(t0)
  ...
  c.addi  t3, 1            # addi t3, t3, 1 in a 16-bit form
  ...
  c.jr    ra               # ret
```

Listato 7.8. `verm1_update`, disassemblata (ridotta; gli offset esatti dipendono dalla compilazione).

- I nomi sono spariti: le etichette sono diventate indirizzi, e le costanti sono diventate numeri.
- Le pseudoistruzioni sono espanse: `la` è `auipc` più `addi`, `li` è `addi` da `zero`, `ret` è un salto a `ra`.
- Alcune istruzioni cominciano con `c.`: sono le forme *compresse*, a 16 bit, delle stesse istruzioni (la C di RV32IMAC).
- La cartuccia è caricata all'indirizzo `0x40800000`: ogni indirizzo mostrato parte da lì.

I formati delle istruzioni sono l'argomento del Capitolo 19; le forme compresse vi sono solo nominate, e sono elencate nell'Appendice A.

7.6.2 Leggere la memoria: little-endian, dal vivo

La vista *Memory* mostra 128 byte a partire dall'indirizzo scritto, in esadecimale (Figura 7.4). La variabile `step_q8` è una parola: i suoi quattro byte arrivano con il byte meno significativo per primo, così che con `step_q8` = 240 si legge `F0 00 00 00`. Nel file l'ordine delle variabili è `head_x`, `head_y`, `step_q8`, quattro byte ciascuna: se `step_q8` è all'indirizzo A, `head_x` è ad $A - 8$.

Il debugger osserva soltanto. Per cambiare il programma: modificare, compilare, eseguire.

7.7 Verificate di aver capito

D7.1 Con `SPEED_Q8` = 64, quanti frame passano tra due passi della testa?

A. 2
B. 4
C. 64
D. 256

D7.2 Quale pixel è l'angolo in alto a sinistra della cella in colonna 10, riga 3?

A. $(10, 3)$
B. $(80, 24)$
C. $(80, 32)$
D. $(88, 32)$

D7.3 Perché `verm1_draw` salva `ra` e `verm1_update` no?

A. `draw` è più lunga
B. `draw` chiama altre funzioni, che sovrascrivono `ra`
C. `update` non usa registri
D. in `update` `ra` è salvato automaticamente

7.8 Fatelo vostro

Esercizio 7.1 (su PRG32)

1. Fate muovere la testa verso sinistra: quale istruzione cambia, e quale confronto rileva il bordo?
2. Fatela muovere invece verso il basso, rientrando dalla riga 23 alla riga 0.
3. Disegnate una seconda cella, in un verde più scuro, una colonna dietro la testa: un primo accenno di corpo.
4. Scrivete il vostro nome accanto al titolo, in un colore che avete calcolato a mano in RGB565.

Risultato atteso. Ogni modifica è di poche righe. Tenete una copia della parte 1 funzionante prima di sperimentare: la parte 2 parte da lì. (Suggerimenti: per andare a sinistra, `addi t3, t3, -1`, poi `bgez t3` per saltare il rientro, altrimenti `li t3, COLS-1`: il confronto è sul segno. Un verde più scuro è `0x03e0`.)

7.9 Dove siamo, e che cosa viene dopo

Avete una cartuccia che funziona, costruita da un file scritto e capito riga per riga. Ha uno stato in memoria, una regola che cambia lo stato a ogni frame, e una funzione che lo disegna: la forma di ogni gioco. Manca il giocatore: la testa va dove vuole. I Capitoli da 8 a 11 danno gli strumenti: bit e maschere per leggere il joystick, regole booleane, tabelle che selezionano. La parte 2 aggiunge la guida, il cibo e un punteggio.

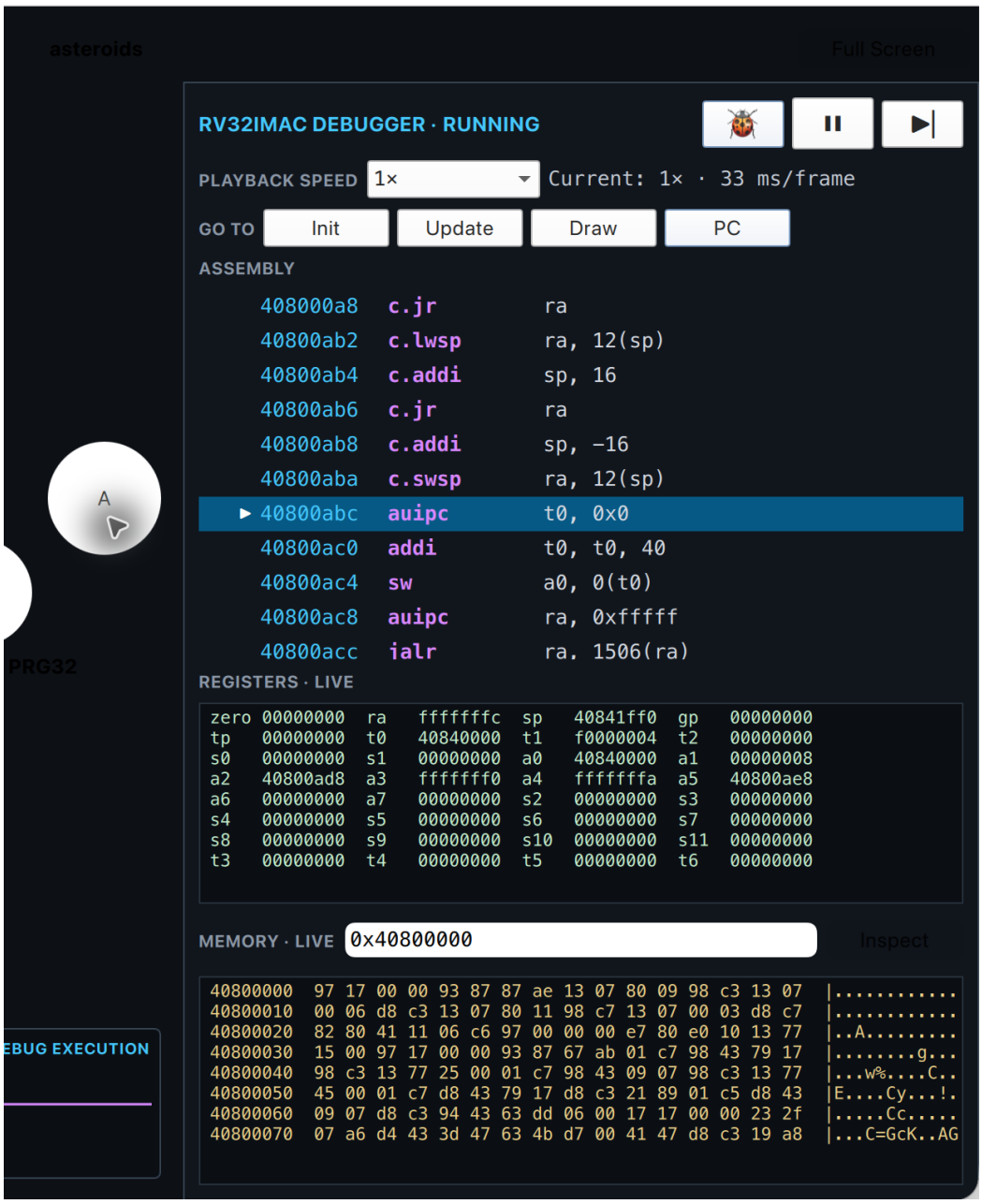


Figura 7.4. PRG32-QT: listato, registri, memoria.

Parte III

Logica

CAPITOLO 8

Algebra di Boole: operatori e proprietà

"Povero me! Povero me! Arriverò troppo tardi!"
— Lewis Carroll, *Alice nel Paese delle Meraviglie*, 1865

In questo capitolo

Alla fine di questo capitolo saprete

- descrivere un calcolatore come processore, memoria e ingresso/uscita, collegati da un bus;
- spiegare la differenza tra l'analisi e la sintesi di un circuito logico;
- scrivere le tabelle di verità di AND, OR, NOT, XOR, NAND e NOR e riconoscerne i simboli;
- applicare le proprietà dell'algebra di Boole, leggi di De Morgan comprese, per semplificare un'espressione;
- usare `and`, `or`, `xor` e `not` su RISC-V per esaminare, porre a 1, azzerare e invertire singoli bit.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI le maschere di bit; l'architettura di von Neumann; analisi e sintesi; l'algebra di Boole; le porte logiche.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI minimizzazione, Karnaugh (Capitolo 9).

8.1 Riscaldamento sulla macchina

State facendo algebra di Boole fin dai primi giorni. Il Listato 8.1 è il modo in cui ogni gioco PRG32 chiede se una direzione è premuta.

```
1     call prg32_input_read     # a0 = all the buttons, one bit each
2     mv   t0, a0               # keep our own copy
3
4     andi t3, t0, 0x01         # keep only bit 0: LEFT
```

```
    beqz t3, 1f             # zero: LEFT is not pressed
    addi t2, t2, -2         # move left
1:
    andi t3, t0, 0x02       # keep only bit 1: RIGHT
    beqz t3, 2f
    addi t2, t2, 2          # move right
2:
```

Listato 8.1. **LEFT** è premuto?

La funzione `prg32_input_read` restituisce tutti i pulsanti insieme, un bit per pulsante (Tabella 8.1). Tenere premuti insieme **RIGHT** e **A** dà `0x12` = $0001\,0010_2$. L'istruzione `andi` con `0x01` conserva il bit 0 e forza a zero ogni altro bit, così che il risultato è diverso da zero esattamente quando **LEFT** è premuto. La costante usata con `andi` si chiama *maschera*.

Tabella 8.1. Un bit per pulsante (le costanti `PRG32_BTN_*` di `prg32.h`).

Pulsante	LEFT	RIGHT	UP	DOWN	A	B	SELECT
Bit	0	1	2	3	4	5	6
Maschera	`0x01`	`0x02`	`0x04`	`0x08`	`0x10`	`0x20`	`0x40`



Figura 8.1. Sette lampade, una per bit della maschera dei pulsanti: una lampada è accesa quando il suo bit vale 1.

Ogni domanda sul joystick è un'espressione booleana su queste sette variabili. E `andi` è trentadue porte AND che lavorano fianco a fianco. Che cos'è, esattamente, un AND?

8.2 Il calcolatore come macchina logica

Questa parte del libro scende di un livello sotto le istruzioni.

8.2.1 Tre sottosistemi e un bus

In termini odierni il modello di von Neumann del Capitolo 3 si disegna come nella Figura 8.2: le unità del 1945, collegate da fili condivisi. Il *processore* (CPU) contiene l'unità di controllo, la ALU e i registri, ed esegue le istruzioni. La *memoria centrale* contiene le istruzioni e i dati. Il sottosistema di *ingresso/uscita* collega joystick, display, suono, rete e memoria di massa. Il *bus* è l'insieme dei fili che li unisce: linee degli indirizzi, linee dei dati e linee di controllo. Tutto ciò che viaggia sul bus è una configurazione di bit. Sull'ESP32-C6 i tre sottosistemi e il bus stanno tutti dentro un solo chip.



Figura 8.2. Il calcolatore elettronico: tre sottosistemi e un bus.

8.2.2 Il processore

La capacità di elaborazione di un calcolatore risiede nel processore. Esso può eseguire un insieme di azioni elementari, le *istruzioni*: comandi espliciti che governano il trasferimento dell'informazione all'interno del calcolatore e tra il calcolatore e i dispositivi di I/O, e che specificano le operazioni aritmetiche e logiche da compiere. La memoria centrale contiene i dati di ingresso e di uscita dell'elaborazione e la sequenza delle istruzioni; una sequenza di istruzioni registrata in memoria centrale è un *programma*. Il processore preleva le istruzioni dalla memoria e le esegue una per una: *prelievo, esecuzione, e da capo*.

Un calcolatore elettronico è, di conseguenza, un sistema

numerico:
: un sistema digitale, in cui tutta l'informazione trattata è espressa come stringhe di bit;

automatico:
: una volta avviato procede da solo, istruzione dopo istruzione, senza intervento umano;

a programma memorizzato:
: il programma è registrato in memoria e può essere sostituito, così che la stessa macchina fa lavori diversi;

elettronico:
: è costruito con circuiti elettronici che memorizzano i bit e li elaborano.

8.2.3 Circuiti logici

I circuiti elettronici che memorizzano ed elaborano i bit si chiamano *circuiti di commutazione* o *circuiti logici*. Un circuito logico forma una *rete di commutazione*, e progettarne una si dice *progetto logico*. Due livelli di tensione su un filo, due stati di un interruttore, due valori di una variabile: per descrivere il comportamento di questi circuiti si usa un'algebra, una notazione matematica che ne semplifica sia l'analisi sia la sintesi. L'algebra è quella di Boole (1854), applicata agli interruttori da Nakajima e da Shannon (1938).

Definizione 8.1 (Analisi e sintesi)

Analisi: data la descrizione della *struttura* di un circuito (come è fatto), trovarne il *comportamento* (che cosa fa). *Sintesi*: data la descrizione del *comportamento* (che cosa deve fare), trovare una *struttura* che lo realizzi (come costruirlo).

Leggere uno schema e ricavarne la tabella di verità è analisi; lo è anche leggere `andi t3, t0, 1` e dire che cosa finisce in `t3`. Passare dalla tabella di verità di una somma a un sommatore è sintesi; lo è anche passare da «muoviti se **LEFT** o **RIGHT** è premuto» a una maschera e

un salto. La sintesi ha di solito molte soluzioni, e scegliere la meno costosa è il problema della *minimizzazione*, argomento del Capitolo 9.

8.3 L'algebra di Boole

Definizione 8.2 (Algebra di Boole)

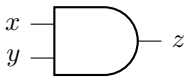
L'*algebra di Boole* è l'algebra delle variabili che possono assumere solo due valori, 0 e 1, con tre operazioni di base:

- AND, il *prodotto logico* o congiunzione, scritto $x \cdot y$ o semplicemente xy;
- OR, la *somma logica* o disgiunzione, scritta $x + y$;
- NOT, il *complemento* o negazione, scritto $\overline{x}$ (nei programmi e sulla tastiera, `x'`).

8.3.1 Le tre operazioni fondamentali

Un'operazione è definita dalla sua *tabella di verità*, che elenca l'uscita per ogni combinazione degli ingressi (Figura 8.3). AND vale 1 solo quando entrambi gli ingressi valgono 1; OR vale 0 solo quando entrambi gli ingressi valgono 0; NOT inverte. Ogni operazione ha un simbolo grafico, la *porta logica*, che sta per il circuito che la calcola.

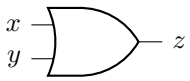
AND (congiunzione)



$z = x \cdot y$

x	y	z
0	0	0
0	1	0
1	0	0
1	1	1

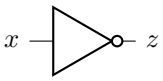
OR (disgiunzione)



$z = x + y$

x	y	z
0	0	0
0	1	1
1	0	1
1	1	1

NOT (negazione)



$z = \overline{x}$

x	z
0	1
1	0

Figura 8.3. Le tre operazioni fondamentali.

8.3.2 Altre tre porte

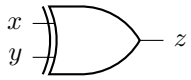
Altre tre operazioni si incontrano dappertutto (Figura 8.4). XOR, l'*or esclusivo*, vale 1 quando gli ingressi sono diversi: è il bit di somma di un'addizione, e RISC-V lo ha come istruzione. NAND e NOR sono i complementi di AND e OR; il cerchietto su un simbolo significa negazione.

8.3.3 Proprietà

La Tabella 8.2 raccoglie le proprietà dell'algebra. Ogni proprietà si presenta in coppia: una forma per AND e una per OR.

Per la proprietà associativa, AND e OR si estendono a un numero qualsiasi di operandi: $x \cdot y \cdot z$, $x + y + z$.

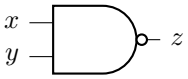
XOR (or esclusivo)



$z = \overline{x}y + x\overline{y} = x \oplus y$

x	y	z
0	0	0
0	1	1
1	0	1
1	1	0

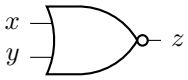
NAND (not and)



$z = \overline{x \cdot y}$

x	y	z
0	0	1
0	1	1
1	0	1
1	1	0

NOR (not or)



$z = \overline{x + y}$

x	y	z
0	0	1
0	1	0
1	0	0
1	1	0

Figura 8.4. XOR, NAND e NOR.

Tabella 8.2. Proprietà dell'algebra di Boole.

Proprietà	**Forma AND**	**Forma OR**
Commutativa	$x \cdot y = y \cdot x$	$x + y = y + x$
Associativa	$(x \cdot y) \cdot z = x \cdot (y \cdot z)$	$(x + y) + z = x + (y + z)$
Idempotenza	$x \cdot x = x$	$x + x = x$
Assorbimento	$x \cdot (x + y) = x$	$x + (x \cdot y) = x$
Distributiva	$x \cdot (y + z) = x \cdot y + x \cdot z$	$x + y \cdot z = (x + y) \cdot (x + z)$
Elemento neutro	$x \cdot 1 = x$	$x + 0 = x$
Elemento nullo	$x \cdot 0 = 0$	$x + 1 = 1$
Complemento	$x \cdot \overline{x} = 0$	$x + \overline{x} = 1$
Involuzione	$\overline{\overline{x}} = x$	

Attenzione

Anche OR è distributivo rispetto ad AND: $x + yz = (x + y)(x + z)$. Questo è falso per la somma e il prodotto ordinari, ed è la proprietà che si dimentica più spesso.

Come si dimostra una proprietà? Poiché una variabile ha solo due valori, ogni identità può essere verificata provando tutti i casi: è il metodo dell'*induzione perfetta*. Due espressioni sono *equivalenti* se hanno la stessa tabella di verità. In alternativa una proprietà può essere derivata algebricamente da altre; l'assorbimento, per esempio, segue da elemento neutro, distributività ed elemento nullo:

$$x + xy \;=\; x \cdot 1 + x \cdot y \;=\; x\,(1 + y) \;=\; x \cdot 1 \;=\; x.$$

8.3.4 Le leggi di De Morgan

Teorema 8.1 (De Morgan, 1847)

$$\overline{x \cdot y} = \overline{x} + \overline{y}, \qquad \overline{x + y} = \overline{x} \cdot \overline{y}.$$

Il complemento di un prodotto è la somma dei complementi, e viceversa.

Dimostrazione della prima legge, per induzione perfetta: le colonne di $\overline{xy}$ e di $\overline{x} + \overline{y}$ sono identiche.

x	y	xy	$\overline{xy}$	$\overline{x}$	$\overline{y}$	$\overline{x}+\overline{y}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

La seconda legge si dimostra allo stesso modo (Esercizio 8.1). □

In termini di porte: un NAND è un OR degli ingressi negati, e un NOR è un AND degli ingressi negati. Le leggi si generalizzano a un numero qualsiasi di variabili.

8.3.5 Dualità e precedenza

Teorema 8.2 (Principio di dualità)

Ogni identità dell'algebra di Boole resta vera se si scambiano AND con OR e 0 con 1.

Così $x \cdot 1 = x$ diventa $x + 0 = x$, e ciascuna delle leggi di De Morgan è la duale dell'altra. La dualità dimezza il lavoro: basta dimostrare una legge di ogni coppia.

Quanto alla *precedenza*, NOT viene per primo, poi AND, poi OR, come per potenze, prodotti e somme: $x + y\,\overline{z}$ significa $x + (y \cdot (\overline{z}))$. Le parentesi cambiano l'ordine esattamente come nell'algebra ordinaria.

8.3.6 Semplificare con l'algebra

Le proprietà sono strumenti per trasformare un'espressione in una equivalente e meno costosa.

Esempio 8.1 (Tre porte diventano un filo)

$$\begin{aligned} x\,y + x\,\overline{y} &= x\,(y + \overline{y}) && \text{proprietà distributiva, letta da destra a sinistra} \\ &= x \cdot 1 && \text{legge del complemento} \\ &= x && \text{elemento neutro.} \end{aligned}$$

Due porte AND, una porta OR e un invertitore sono diventati un pezzo di filo.

Allo stesso modo, usando la proprietà distributiva di OR rispetto ad AND,

$$x + \overline{x}\,y \;=\; (x + \overline{x})(x + y) \;=\; 1 \cdot (x + y) \;=\; x + y, \tag{8.1}$$

un'identità chiamata talvolta *teorema di semplificazione*, che sarà usata di nuovo nel Capitolo 9.

8.4 Ritorno alla macchina

8.4.1 Istruzioni bit a bit: l'algebra su un intero registro

Ogni istruzione bit a bit di RISC-V applica una porta a tutte le trentadue coppie di bit insieme (Tabella 8.3, dove sono mostrati otto bit). Le forme immediate `andi`, `ori` e `xori`

prendono una costante di 12 bit. `not` è una pseudoistruzione: `xori t0, a, -1`, perché -1 è tutti uni e $x \oplus 1 = \overline{x}$.

Tabella 8.3. Istruzioni bit a bit, con $a =$ `1100 1010` e $b =$ `0000 1111`.

Istruzione	Operazione	Risultato	In C
`and t0, a, b`	AND bit a bit	`0000 1010`	`a & b`
`or  t0, a, b`	OR bit a bit	`1100 1111`	`a | b`
`xor t0, a, b`	XOR bit a bit	`1100 0101`	`a ^ b`
`not t0, a`	complementa ogni bit	`0011 0101`	`~a`

8.4.2 Quattro cose da fare con una maschera

Le leggi dell'elemento neutro e dell'elemento nullo sono esattamente ciò che fa funzionare una maschera (Tabella 8.4). Con AND, un 1 nella maschera lascia passare il bit e uno 0 lo forza a 0; con OR, uno 0 lascia passare il bit e un 1 lo forza a 1; con XOR, uno 0 lascia passare il bit e un 1 lo inverte.

Tabella 8.4. Quattro usi di una maschera (il bit 4 è il pulsante **A**).

Scopo	Legge usata	Istruzione	Altri bit
Esaminare un bit	$x \cdot 1 = x$, $x \cdot 0 = 0$	`andi t1, t0, 0x10`	forzati a 0 nel risultato
Porre un bit a 1	$x + 1 = 1$, $x + 0 = x$	`ori  t0, t0, 0x10`	invariati
Azzerare un bit	$x \cdot 0 = 0$, $x \cdot 1 = x$	`andi t0, t0, ~0x10`	invariati
Invertire un bit	$x \oplus 1 = \overline{x}$, $x \oplus 0 = x$	`xori t0, t0, 0x10`	invariati

La maschera `~0x10` è `0xFFFFFFEF`, che l'assemblatore codifica come immediato -17.

8.4.3 De Morgan al joystick

Il verme è «fermo» quando né **LEFT** né **RIGHT** è premuto. Il Listato 8.2 scrive il test in due modi.

```
# "idle" when NEITHER LEFT NOR RIGHT is pressed
    andi t3, t0, 0x03        # 0x03 = LEFT | RIGHT: keep both bits
    beqz t3, idle            # all zero: (LEFT + RIGHT)' is true

# the long way: test each button, the result is the same
    andi t3, t0, 0x01
    bnez t3, moving          # LEFT pressed: not idle
    andi t3, t0, 0x02
    bnez t3, moving          # RIGHT pressed: not idle
    j    idle                # LEFT' . RIGHT'
```

Listato 8.2. $\overline{L + R} = \overline{L} \cdot \overline{R}$, come due programmi.

Una maschera con due bit esamina due pulsanti in una sola istruzione: il risultato è zero solo se entrambi i bit sono zero, cioè quando $\overline{L + R}$ è vero. Per De Morgan questo è uguale a $\overline{L} \cdot \overline{R}$, che è la via lunga scritta sotto. L'algebra dice che i due programmi sono equivalenti, e quale costa meno: due istruzioni invece di cinque. La minimizzazione non riguarda solo l'hardware.

8.4.4 Una lampada per bit: scorrere e mascherare

Le lampade della Figura 8.1 sono disegnate da un ciclo il cui corpo è il Listato 8.3. Per leggere il bit numero n di una parola, si fa scorrere la parola a destra di n e si maschera con 1: è l'idioma standard.

```
    # s0 = bit number, 0 to 6
    la   t0, buttons
    lw   t1, 0(t0)          # t1 = the button mask
    srl  t1, t1, s0         # bring bit number s0 down to bit 0
    andi t1, t1, 1          # keep it, clear all the others
    li   a4, GREY           # lamp off
    beqz t1, 1f
    li   a4, YELLOW         # lamp on
1:  call prg32_gfx_rect
```

Listato 8.3. Lettura del bit numero `s0` della maschera dei pulsanti.

8.5 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l08.S` nel repository del corso, stampato per intero nel Listato 8.4. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 8.5 è il frame scritto dal test.

Sullo schermo.
Sette lampade, una per pulsante (LEFT a sinistra), accese mentre il pulsante è premuto; un segnalino bianco mosso da LEFT e RIGHT; un quadrato verde quando né LEFT né RIGHT è premuto, rosso altrimenti.

Che cosa premere.
LEFT e RIGHT muovono il segnalino; provate anche A e B.

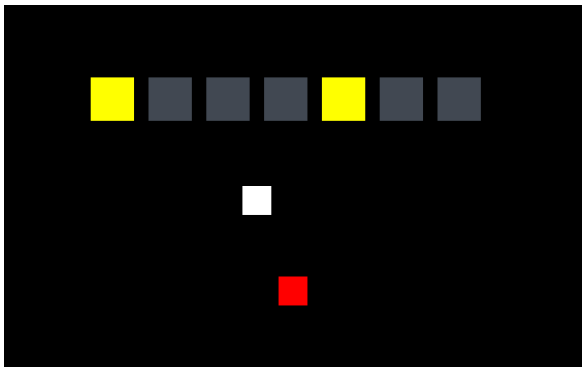


Figura 8.5. `l08.S` su PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 08: Elements of Boolean Algebra
# l08.S: one lamp per button, a marker that moves, and De Morgan at the
    joystick
#
# On the screen: seven lamps, one per button (LEFT on the left), lit while the
    button
```

```
# is held; a white marker that LEFT and RIGHT move; a square that is green when
# neither LEFT nor RIGHT is pressed and red otherwise.
# Build: python3 -m prg32 cartridge build l08.S --portable \
#            --entry-prefix l08 --name l08 --out l08.prg32
.option norelax

.equ BLACK,  0x0000
.equ WHITE,  0xffff
.equ GREY,   0x4208
.equ YELLOW, 0xffe0
.equ GREEN,  0x07e0
.equ RED,    0xf800

.section .text
.global l08_init
.global l08_update
.global l08_draw

l08_init:
    ret

l08_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    call prg32_input_read    # a0 = all the buttons, one bit each
    mv   t0, a0              # keep our own copy
    la   t1, buttons
    sw   t0, 0(t1)           # remember them for draw

    la   t1, marker_x
    lw   t2, 0(t1)
    andi t3, t0, 0x01        # keep only bit 0: LEFT
    beqz t3, 1f              # zero: LEFT is not pressed
    addi t2, t2, -2          # move left
1:
    andi t3, t0, 0x02        # keep only bit 1: RIGHT
    beqz t3, 2f
    addi t2, t2, 2           # move right
2:
    sw   t2, 0(t1)

# "idle" when NEITHER LEFT NOR RIGHT is pressed
    li   t4, 1               # idle = 1
    andi t3, t0, 0x03        # 0x03 = LEFT | RIGHT: keep both bits
    beqz t3, 3f              # all zero: (LEFT + RIGHT)' is true
    li   t4, 0               # idle = 0
3:  la   t1, idle
    sw   t4, 0(t1)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

l08_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
```

```
61      li   a0, BLACK
62      call prg32_gfx_clear
63      li   s0, 0              # s0 = bit number, 0 to 6
64  1:  la   t0, buttons
65      lw   t1, 0(t0)          # t1 = the button mask
66      srl  t1, t1, s0         # bring bit number s0 down to bit 0
67      andi t1, t1, 1          # keep it, clear all the others
68      li   a4, GREY           # lamp off
69      beqz t1, 2f
70      li   a4, YELLOW         # lamp on
71  2:  slli a0, s0, 5
72      addi a0, a0, 48         # x = 48 + 32 * bit
73      li   a1, 40
74      li   a2, 24
75      li   a3, 24
76      call prg32_gfx_rect
77      addi s0, s0, 1
78      li   t0, 7
79      blt  s0, t0, 1b
80
81      la   t0, marker_x       # the marker
82      lw   a0, 0(t0)
83      li   a1, 100
84      li   a2, 16
85      li   a3, 16
86      li   a4, WHITE
87      call prg32_gfx_rect
88
89      la   t0, idle           # the result of De Morgan's test
90      lw   t1, 0(t0)
91      li   a4, RED
92      beqz t1, 3f
93      li   a4, GREEN
94  3:  li   a0, 152
95      li   a1, 150
96      li   a2, 16
97      li   a3, 16
98      call prg32_gfx_rect
99      lw   s0, 8(sp)
100     lw   ra, 12(sp)
101     addi sp, sp, 16
102     ret
103
104 .section .data
105 buttons:  .word 0
106 marker_x: .word 152
107 idle:     .word 1
```

Listato 8.4. Il programma completo del capitolo, `code/l08.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 8.5; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)
```

```
# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l08.S --portable \
    --entry-prefix l08 --name l08 --out l08.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l08.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 8.5. Compilare `l08.S` ed eseguirlo su PRG32-QT.

8.6 Verificate di aver capito

D8.1 Per quali ingressi x NAND y vale 0?

A. $x = 0$, $y = 0$
B. $x = 0$, $y = 1$
C. $x = 1$, $y = 1$
D. mai

D8.2 Per De Morgan, $\overline{a \cdot \overline{b}}$ è uguale a:

A. $\overline{a} \cdot b$
B. $\overline{a} + b$
C. $a + \overline{b}$
D. $\overline{a} + \overline{b}$

D8.3 `t0` $=$ `0x0000005A`. Che cosa c'è in `t1` dopo `andi t1, t0, 0x0F`?

A. `0x0000005A`
B. `0x00000050`
C. `0x0000000A`
D. `0x0000000F`

D8.4 Quale istruzione porta a 1 il bit 5 di `t0` e lascia invariati tutti gli altri bit?

A. `andi t0, t0, 0x20`
B. `ori t0, t0, 0x20`
C. `xori t0, t0, 0x20`
D. `addi t0, t0, 0x20`

8.7 Esercizi

Esercizio 8.1 (prima su carta, poi su PRG32)

1. Dimostrate la seconda legge di De Morgan, $\overline{x + y} = \overline{x} \cdot \overline{y}$, con una tabella di verità.
2. Semplificate con l'algebra $x\,y + x\,\overline{y}\,z + \overline{x}\,y$, nominando la legge usata a ogni passo.
3. Scrivete la maschera e le istruzioni che verificano se **A** e **B** sono premuti *insieme*.

4. Modificate il programma delle lampade in modo che le lampade mostrino il complemento dei pulsanti (accese quando rilasciati).
5. Convertite una lettera da maiuscola a minuscola e viceversa usando una sola istruzione bit a bit (suggerimento: la tabella ASCII del Capitolo 4).

Risultato atteso. Su carta, il secondo compito dà $y\,(x+\overline{x})+x\,\overline{y}\,z = y+x\,\overline{y}\,z = y+x\,z$, l'ultimo passo per la (8.1). Per il terzo: `andi t1, t0, 0x30` e poi un confronto *con* `0x30`; verificare che sia diverso da zero significherebbe **A** *oppure* **B**. Su PRG32: tutte e sette le lampade accese a riposo, ciascuna che si spegne mentre il suo pulsante è tenuto premuto. Per la lettera, `xori` con `0x20` inverte il bit 5.

8.8 Glossario

Inglese	**Italiano**
logic circuit	circuito logico
switching network	rete di commutazione
analysis / synthesis	analisi / sintesi
logical product (AND)	prodotto logico (congiunzione)
logical sum (OR)	somma logica (disgiunzione)
involution	involuzione
truth table	tabella di verità
logic gate	porta logica
complement	complemento (negazione)
duality	dualità
mask	maschera
to set / clear / toggle	porre a 1 / azzerare / invertire

8.9 Conclusioni e letture

L'architettura di von Neumann rappresenta una macchina reale in forma semplificata: processore, memoria, ingresso/uscita. L'algebra di Boole rappresenta funzioni logiche che si possono realizzare con circuiti elettronici. Tre operazioni, una manciata di proprietà e le leggi di De Morgan bastano a trasformare qualsiasi espressione. Un'istruzione bit a bit è l'algebra applicata a trentadue bit insieme, e una maschera è la legge dell'elemento neutro al lavoro.

Letture. Patterson e Hennessy (2020), appendice A, «The Basics of Logic Design»; *Playing with RISC-V*, capitolo 6, «State and Input: Branches, Bitmasks, and Boundaries» (Montella et al., 2026b); Bucci (2017), i capitoli sull'algebra di Boole.

CAPITOLO 9

Algebra di Boole: funzioni, forme canoniche e minimizzazione

"Povero me! Povero me! Arriverò troppo tardi!"
— Lewis Carroll, *Alice nel Paese delle Meraviglie*, 1865

In questo capitolo

Alla fine di questo capitolo saprete

- definire letterale, clausola, fattore, mintermine e maxtermine;
- dire quante funzioni booleane di n variabili esistono, e nominare quelle di due variabili;
- scrivere una funzione in forma canonica somma di prodotti e prodotto di somme a partire dalla sua tabella di verità;
- minimizzare una funzione fino a quattro variabili con una mappa di Karnaugh;
- ricavare il semisommatore e il sommatore completo, e spiegare come trentadue di essi eseguono `add`.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI il sommatore; le funzioni booleane; le forme canoniche; minimizzazione, Karnaugh.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI niente.

9.1 Riscaldamento sulla macchina

Due registri contengono un bit ciascuno, 0 o 1. La loro somma può essere 0, 1 o 2, e richiede quindi due bit. Il Listato 9.1 la calcola senza alcuna istruzione aritmetica.

```
# a0 and a1 hold one bit each: 0 or 1
# goal: compute a0 + a1, which needs two bits (0, 1 or 2)

```

```
    xor  t0, a0, a1      # low bit of the sum:  1 when the bits differ
    and  t1, a0, a1      # high bit, the carry: 1 when both are 1

#   a0 a1 | t1 t0
#    0  0 |  0  0        0 + 0 = 0
#    0  1 |  0  1        0 + 1 = 1
#    1  0 |  0  1        1 + 0 = 1
#    1  1 |  1  0        1 + 1 = 2  (binary 10)
```

Listato 9.1. Somma di un bit senza `add`.

Due istruzioni logiche sommano due bit. Il commento del listato è una tabella di verità con due uscite, e la coppia di porte che essa descrive si chiama *semisommatore*. Il Capitolo 5 affermava che a un processore, per l'aritmetica, basta un sommatore: questo è il suo primo bit. Ma come si trovano le porte quando è data solo la tabella? È il metodo di questo capitolo: tabella di verità, forma canonica, minimizzazione.

9.2 Funzioni booleane

9.2.1 Il vocabolario delle espressioni

Definizione 9.1 (Letterale, clausola, fattore)

Un *letterale* è una variabile booleana, o il suo complemento, così come compare in un'espressione: a, $\overline{b}$, x. Una *clausola*, o *termine prodotto*, è il prodotto logico di due o più letterali: $a\,\overline{b}\,x$. Un *fattore*, o *termine somma*, è la somma logica di due o più letterali: $(a + \overline{x})$, $(b + c)$.

Clausole e fattori con più di un letterale sono funzioni di livello 1: uno strato di porte. Ogni occorrenza di una variabile conta come un letterale, e il numero dei letterali misura il costo di un'espressione.

9.2.2 Quante funzioni di n variabili?

Definizione 9.2 (Funzione booleana)

Una *funzione booleana* di n variabili è un'applicazione $f : B^n \to B$, con $B = \{0,1\}$: associa un'uscita binaria a ogni combinazione di n ingressi binari.

Teorema 9.1 (Numero delle funzioni booleane)

Esistono esattamente 2^{2^n} funzioni booleane distinte di n variabili.

Dimostrazione. Le combinazioni degli ingressi sono 2^n, e una funzione è determinata dalla scelta di un'uscita, 0 o 1, per ciascuna di esse: è una stringa di 2^n bit, e di tali stringhe ce ne sono 2^{2^n}. □

Il numero è finito, e cresce in fretta: 4 funzioni di una variabile, 16 di due, 256 di tre, 65 536 di quattro, e più di quattro miliardi di cinque. Le funzioni booleane sono isomorfe ai circuiti

digitali: ogni circuito si può scrivere come espressione, e ogni espressione si può costruire come circuito.

9.2.3 Le sedici funzioni di due variabili

La Tabella 9.1 elenca tutte le funzioni di due variabili. L'indice di ogni funzione è la sua colonna di uscite letta come numero binario. Con una variabile le quattro funzioni sono la costante 0, x, $\overline{x}$ e la costante 1.

Tabella 9.1. Le sedici funzioni di due variabili. Le colonne 00, 01, 10, 11 danno l'uscita per ogni coppia di ingressi $x\,y$.

f	00	01	10	11	**nome**	f	00	01	10	11	**nome**
f_0	0	0	0	0	costante 0	f_8	1	0	0	0	**NOR**
f_1	0	0	0	1	**AND**: $x\,y$	f_9	1	0	0	1	**XNOR** (equivalenza)
f_2	0	0	1	0	$x\,\overline{y}$ (inibizione)	f_{10}	1	0	1	0	NOT y
f_3	0	0	1	1	x (identità)	f_{11}	1	0	1	1	$x + \overline{y}$ (implicazione $y \to x$)
f_4	0	1	0	0	$\overline{x}\,y$ (inibizione)	f_{12}	1	1	0	0	NOT x
f_5	0	1	0	1	y (identità)	f_{13}	1	1	0	1	$\overline{x} + y$ (implicazione $x \to y$)
f_6	0	1	1	0	**XOR**	f_{14}	1	1	1	0	**NAND**
f_7	0	1	1	1	**OR**: $x + y$	f_{15}	1	1	1	1	costante 1

9.2.4 Insiemi funzionalmente completi

Qualsiasi funzione booleana si può realizzare usando solo AND, OR e NOT; per esempio $x \oplus y = \overline{x}\,y + x\,\overline{y}$. (Il Paragrafo 9.3 dimostra l'enunciato generale per costruzione.) L'insieme {AND,OR,NOT} si dice perciò *funzionalmente completo*.

Per De Morgan, AND si può costruire con OR e NOT, e OR con AND e NOT: anche gli insiemi {AND,NOT} e {OR,NOT} sono completi. E basta una sola porta.

Teorema 9.2 (Porte universali)

Il NAND da solo è funzionalmente completo, e così il NOR da solo.

Dimostrazione per il NAND, scrivendo $x \uparrow y = \overline{xy}$:

$$\overline{x} = x \uparrow x, \qquad x\,y = (x \uparrow y) \uparrow (x \uparrow y), \qquad x + y = (x \uparrow x) \uparrow (y \uparrow y),$$

l'ultima per De Morgan. La dimostrazione per il NOR è duale. □

Un solo tipo di porta basta a costruire qualsiasi circuito digitale, un intero processore compreso. I chip reali sono costruiti in gran parte con NAND e NOR perché sono le porte che richiedono meno transistor (Capitolo 10).

9.2.5 Tabelle di verità

Una funzione booleana può essere definita dalla sua tabella di verità, che dà il valore della variabile dipendente per ogni combinazione delle variabili indipendenti. Una funzione di n variabili è definita su 2^n punti, e la sua tabella ha 2^n righe, elencate nell'ordine del conteggio: 000, 001, 010, … La funzione F della Tabella 9.2, che vale 1 sulle righe 3, 4, 5, 6 e 7, sarà seguita per tutto il capitolo.

Tabella 9.2. La funzione F, con il mintermine o il maxtermine di ogni riga.

riga	A	B	C	F	**mintermine** ($F=1$)	**maxtermine** ($F=0$)
0	0	0	0	0		$M_0 = A + B + C$
1	0	0	1	0		$M_1 = A + B + \overline{C}$
2	0	1	0	0		$M_2 = A + \overline{B} + C$
3	0	1	1	1	$m_3 = \overline{A}\,B\,C$	
4	1	0	0	1	$m_4 = A\,\overline{B}\,\overline{C}$	
5	1	0	1	1	$m_5 = A\,\overline{B}\,C$	
6	1	1	0	1	$m_6 = A\,B\,\overline{C}$	
7	1	1	1	1	$m_7 = A\,B\,C$	

9.2.6 Il semisommatore

Il *semisommatore* (ingl. *half adder*) somma due bit A e B e produce un bit di somma S e un riporto C. In binario $1 + 1$ dà somma 0 e riporto 1; dalla tabella, S vale 1 quando gli ingressi sono diversi e C vale 1 quando entrambi valgono 1 (Figura 9.1):

$$S = A \oplus B, \qquad C = A \cdot B.$$

Il circuito è esattamente la coppia di istruzioni del riscaldamento. È «mezzo» sommatore perché non ha un riporto in ingresso: può sommare soltanto i bit meno significativi di due numeri.

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

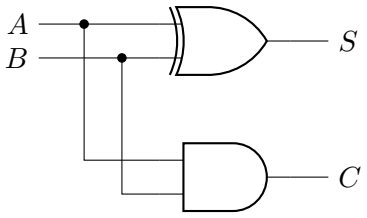


Figura 9.1. Il semisommatore.

9.2.7 Il sommatore completo

Il *sommatore completo* (ingl. *full adder*) somma due bit e un riporto entrante, C_{in}, e produce un bit di somma e un riporto uscente, C_{out} (Figura 9.2). S vale 1 quando un numero dispari di ingressi vale 1; C_{out} vale 1 quando almeno due ingressi valgono 1:

$$S = A \oplus B \oplus C_{in}, \qquad C_{out} = A\,B + C_{in}\,(A \oplus B).$$

Il circuito è fatto di due semisommatori e un OR.

9.3 Forme canoniche

Le forme canoniche sono una via sistematica dalla tabella a un'espressione.

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

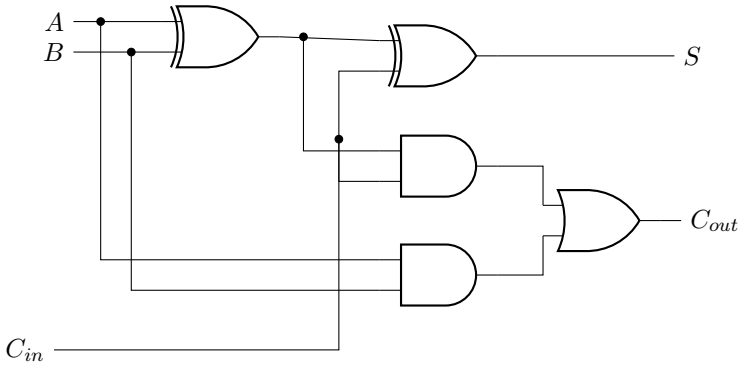


Figura 9.2. Il sommatore completo: due semisommatori e un OR. I fili che si incrociano senza un punto non sono collegati.

9.3.1 Forme elementari: funzioni di livello 2

Definizione 9.3 (Somma di prodotti e prodotto di somme)

Un'espressione di *tipo P*, o *somma di prodotti* (SOP), è una somma logica di clausole, come $y = a\,\overline{b} + \overline{a}\,d + b\,\overline{d}\,x$. Si costruisce come uno strato di porte AND che alimenta una porta OR. Un'espressione di *tipo S*, o *prodotto di somme* (POS), è un prodotto logico di fattori, come $y = (a + \overline{b})(\overline{a} + d)(b + \overline{d} + x)$. Si costruisce come uno strato di porte OR che alimenta una porta AND.

9.3.2 Mintermini e maxtermini

Definizione 9.4 (Mintermine e maxtermine)

Un *mintermine* di n variabili è un prodotto che contiene un letterale di ogni variabile. Un *maxtermine* è una somma che contiene un letterale di ogni variabile.

Un mintermine vale 1 per una sola riga della tabella di verità, e un maxtermine vale 0 per una sola riga. Il mintermine m_i della riga i contiene una variabile complementata dove la riga ha uno 0; il maxtermine M_i contiene una variabile complementata dove la riga ha un 1 (Tabella 9.2). Per De Morgan, $M_i = \overline{m_i}$.

9.3.3 Dalla tabella di verità alle forme canoniche

Teorema 9.3 (Forme canoniche)

Ogni funzione booleana è uguale
1. alla somma dei mintermini delle righe in cui vale 1 (*forma canonica P*);
2. al prodotto dei maxtermini delle righe in cui vale 0 (*forma canonica S*).

Ciascuna delle due forme è unica, a meno dell'ordine dei termini.

Dimostrazione. Su una riga in cui $f = 1$, il mintermine di quella riga vale 1 e così la somma; su una riga in cui $f = 0$, tutti i mintermini della somma valgono 0. Il secondo enunciato è il duale. □

Per la funzione F:

$$F = \overline{A}BC + A\overline{B}\,\overline{C} + A\overline{B}C + AB\overline{C} + ABC \;=\; \sum m(3,4,5,6,7),$$
$$F = (A+B+C)\,(A+B+\overline{C})\,(A+\overline{B}+C) \;=\; \prod M(0,1,2).$$

Il teorema dimostra anche che AND, OR e NOT sono funzionalmente completi. È una sintesi per ricetta: sempre possibile, raramente economica.

9.3.4 Minimizzazione algebrica di F

La forma canonica P di F ha 5 clausole e 15 letterali. L'algebra del Capitolo 8 la riduce:

$$\begin{aligned}
F &= \overline{A}BC + A\overline{B}\,\overline{C} + A\overline{B}C + AB\overline{C} + ABC \\
&= \overline{A}BC + A\,(\overline{B}\,\overline{C} + \overline{B}C + B\overline{C} + BC) && \text{distributiva: si raccoglie } A \\
&= \overline{A}BC + A\,(\overline{B}\,(\overline{C}+C) + B\,(\overline{C}+C)) && \text{distributiva, due volte} \\
&= \overline{A}BC + A\,(\overline{B}+B) && \text{complemento: } \overline{C}+C=1 \\
&= \overline{A}BC + A && \text{complemento: } \overline{B}+B=1 \\
&= A + BC && \text{semplificazione: } x+\overline{x}\,y = x+y.
\end{aligned}$$

Quindici letterali sono diventati tre: da cinque porte AND a tre ingressi e un OR a cinque, a un AND e un OR. Funziona, ma ogni passo richiede un'idea. Ciò che serve è un metodo.

9.3.5 Lo scopo della minimizzazione

Lo scopo è trovare, tra tutte le possibili forme a due livelli di tipo P, una forma elementare *minima*. Porte meno numerose e più piccole significano costo inferiore, meno potenza e un circuito più veloce. Tre metodi sono classici.

- **Mappe di Karnaugh**: grafico, per gruppi di celle adiacenti di area massima; pratico fino a quattro o cinque variabili (Karnaugh, 1953).
- **Metodo di Quine**: algebrico, per confronto sistematico dei termini.
- **Metodo di McCluskey**: tabellare, adatto a un programma, e base degli odierni strumenti di sintesi logica. La procedura di Quine–McCluskey è esatta ma esponenziale; gli strumenti industriali usano euristiche.

9.4 Mappe di Karnaugh

9.4.1 L'idea

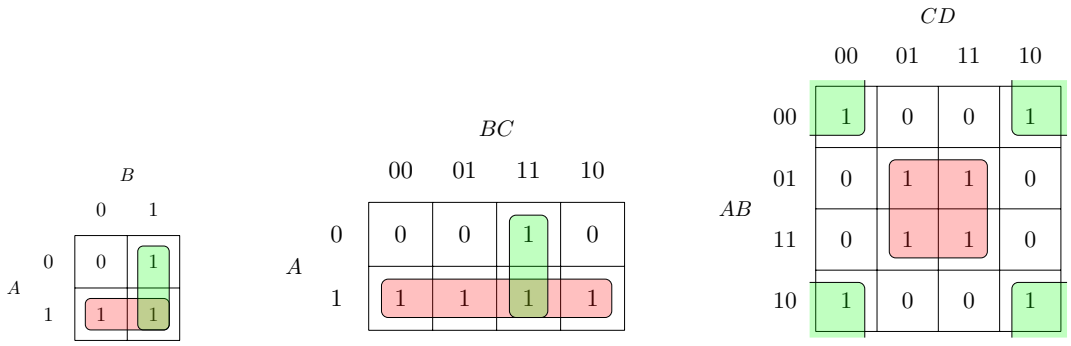
Una mappa di Karnaugh è la tabella di verità di una funzione di n variabili ridisegnata su una griglia di 2^n celle, una cella per riga. Le celle sono ordinate in modo che due vicine differiscano in una sola variabile: di qui l'ordine 00, 01, 11, 10 delle colonne, che è un codice Gray (Capitolo 11), e di qui il fatto che in una mappa m_3 viene prima di m_2. Anche i bordi sinistro e destro sono adiacenti: la mappa si richiude su sé stessa.

L'adiacenza grafica è così adiacenza logica: due 1 adiacenti sono due mintermini che differiscono in una variabile, e per $x\,y + x\,\overline{y} = x$ si fondono in una clausola con un letterale in meno.

La regola della mappa

Raggruppate gli 1 in rettangoli di 1, 2, 4 o 8 celle, i più grandi possibile, finché ogni 1 è coperto. Un gruppo di 2^k celle elimina k variabili: la clausola di un gruppo contiene solo le variabili che al suo interno non cambiano.

9.4.2 Due, tre e quattro variabili



(a) $F = A + B$ **(b)** $F = A + BC$ **(c)** $F = BD + \overline{B}\,\overline{D}$

Figura 9.3. Mappe di Karnaugh di due, tre e quattro variabili.

Due variabili (Figura 9.3a). La funzione vale 1 per $AB = 01$, 10 e 11; la sua forma canonica $\overline{A}B + A\overline{B} + AB$ ha sei letterali. Un gruppo è la riga $A = 1$: al suo interno B cambia, quindi B sparisce e la clausola è A. L'altro è la colonna $B = 1$: cambia A, e la clausola è B. Dunque $F = A + B$: la funzione è semplicemente OR. La cella AB appartiene a entrambi i gruppi; usare un 1 due volte è la legge di idempotenza, $x + x = x$.

Tre variabili (Figura 9.3b). È la funzione F del capitolo: un 1 nelle celle dei mintermini 3, 4, 5, 6 e 7. Un gruppo di quattro copre l'intera riga $A = 1$: B e C cambiano entrambe, e la clausola è A. Un gruppo di due copre la colonna $BC = 11$: cambia A, e la clausola è BC. Ogni 1 è coperto e nessun gruppo può essere ingrandito: $F = A + BC$, lo stesso risultato dell'algebra, trovato a occhio in pochi secondi.

Quattro variabili (Figura 9.3c). La funzione $F = \sum m(0,2,5,7,8,10,13,15)$ ha otto mintermini e trentadue letterali. Un gruppo di quattro al centro ha dappertutto $B = 1$ e $D = 1$: la clausola è BD. Anche i quattro angoli sono un solo gruppo, perché la mappa si richiude sia in orizzontale sia in verticale, come se fosse disegnata su un toro: lì $B = 0$ e $D = 0$, e la clausola è $\overline{B}\,\overline{D}$. Il risultato ha quattro letterali, e si riconosce: F è lo XNOR di B e D.

La regola pratica è: prendere sempre prima i gruppi più grandi, poi coprire ciò che resta.

9.4.3 Condizioni di indifferenza

A volte una combinazione degli ingressi non può mai presentarsi, oppure quando si presenta l'uscita non importa. Una cifra in decimale codificato in binario, per esempio, usa le configurazioni da 0000 a 1001, e le sei configurazioni da 1010 a 1111 non compaiono mai. Quelle

celle si segnano nella mappa con un trattino (o, in molti testi, con una croce): *condizione di indifferenza* (ingl. *don't care*). Una cella di indifferenza può essere trattata come un 1 quando rende un gruppo più grande, e come uno 0 altrimenti. Più libertà significa gruppi più grandi e un circuito meno costoso. L'esercizio classico è il decodificatore per un display a sette segmenti, dove ogni segmento è una funzione di quattro variabili con sei condizioni di indifferenza.

9.5 Ritorno alla macchina

9.5.1 Trentadue sommatori completi fanno una `add`

L'istruzione `add t2, t0, t1` porta i due registri a trentadue sommatori completi, uno per posizione di bit (Figura 9.4). Il riporto uscente di ogni posizione è il riporto entrante della successiva: un *sommatore a propagazione del riporto* (ingl. *ripple-carry adder*), in cui il riporto si propaga verso sinistra come in una somma fatta a mano.

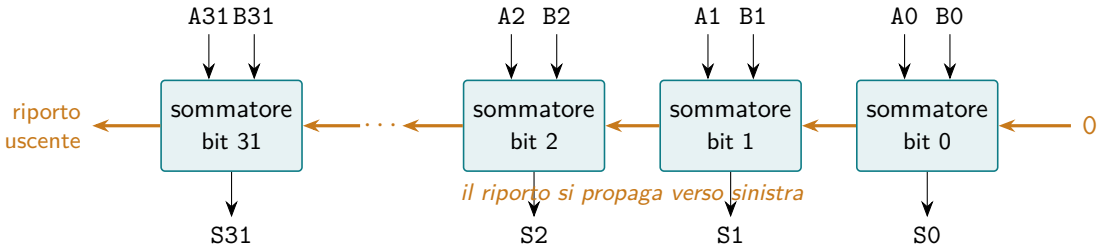


Figura 9.4. Un sommatore a propagazione del riporto di 32 bit.

Il riporto uscente del bit 31 è l'overflow senza segno del Capitolo 5, che RISC-V semplicemente scarta. I processori reali usano schemi più veloci per il riporto, come il carry look-ahead (Capitolo 11), ma la funzione calcolata è la stessa.

9.5.2 Un sommatore completo, una porta per istruzione

```
# a0 = A, a1 = B, a2 = Cin   (each 0 or 1)
# result: a3 = S, a4 = Cout

    xor  t0, a0, a1      # t0   = A xor B
    xor  a3, t0, a2      # S    = A xor B xor Cin
    and  t1, a0, a1      # t1   = A . B
    and  t2, t0, a2      # t2   = (A xor B) . Cin
    or   a4, t1, t2      # Cout = A . B + (A xor B) . Cin
```

Listato 9.2. Il sommatore completo come programma.

Cinque istruzioni, cinque porte: il Listato 9.2 è lo schema della Figura 9.2, letto da sinistra a destra. Eseguirlo passo-passo in PRG32-QT e compilare la tabella di verità è *analisi*; ricavare queste porte dalla tabella con il metodo di questo capitolo era *sintesi*. E con registri interi come ingressi, le stesse cinque righe lavorano su tutte le trentadue posizioni di bit insieme.

9.6 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l09.S` nel repository del corso, stampato per intero nel Listato 9.3. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 9.5 è il frame scritto dal test.

Sullo schermo.
Otto righe, una per combinazione di A, B e Cin (000 in alto): tre lampade per gli ingressi a sinistra, poi S e Cout a destra. È la tabella di verità del sommatore completo.

Che cosa premere.
Niente: il programma va da solo.

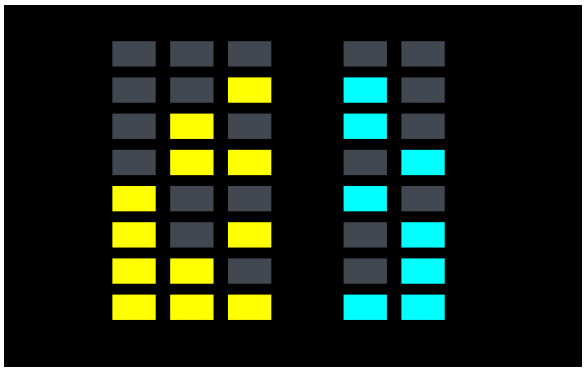


Figura 9.5. `l09.S` su PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 09: Elements of Boolean Algebra
# l09.S: a full adder, one gate per instruction, on its eight input combinations
#
# On the screen: eight rows, one per combination of A, B and Cin (000 at the top):
# three lamps for the inputs on the left, then S and Cout on the right. It is the
# truth table of the full adder.
# Build: python3 -m prg32 cartridge build l09.S --portable \
#            --entry-prefix l09 --name l09 --out l09.prg32
.option norelax

.equ BLACK,  0x0000
.equ GREY,   0x4208
.equ YELLOW, 0xffe0
.equ CYAN,   0x07ff

.section .text
.global l09_init
.global l09_update
.global l09_draw

l09_init:
    ret

l09_update:
```

```
    ret

# full_adder: a0 = A, a1 = B, a2 = Cin (each 0 or 1); result: a3 = S, a4 = Cout
full_adder:
    xor  t0, a0, a1      # t0   = A xor B
    xor  a3, t0, a2      # S    = A xor B xor Cin
    and  t1, a0, a1      # t1   = A . B
    and  t2, t0, a2      # t2   = (A xor B) . Cin
    or   a4, t1, t2      # Cout = A . B + (A xor B) . Cin
    ret

# lamp: a0 = column, a1 = row, a2 = bit (0 or 1), a3 = colour when the bit is 1
lamp:
    li   a4, GREY
    beqz a2, 1f
    mv   a4, a3
1:  slli a0, a0, 5
    addi a0, a0, 60          # x = 60 + 32 * column
    slli a1, a1, 4
    slli t0, a1, 1
    srli a1, a1, 1
    add  a1, a1, t0          # 40 * row ...
    srli a1, a1, 1           # ... / 2 = 20 * row
    addi a1, a1, 20          # y = 20 + 20 * row
    li   a2, 24
    li   a3, 14
    tail prg32_gfx_rect      # a tail call: rect returns to our caller

l09_draw:
    addi sp, sp, -32
    sw   ra, 28(sp)
    sw   s0, 24(sp)
    sw   s1, 20(sp)
    sw   s2, 16(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   s0, 0               # row = the three inputs as a number, 0 to 7
1:  srli a0, s0, 2
    andi a0, a0, 1           # A   = bit 2 of the row
    srli a1, s0, 1
    andi a1, a1, 1           # B   = bit 1
    andi a2, s0, 1           # Cin = bit 0
    call full_adder
    mv   s1, a3              # keep S and Cout: they are in a registers,
    mv   s2, a4              # which the next calls destroy

    li   a0, 0               # column 0: A
    mv   a1, s0
    srli a2, s0, 2
    andi a2, a2, 1
    li   a3, YELLOW
    call lamp
    li   a0, 1               # column 1: B
    mv   a1, s0
    srli a2, s0, 1
    andi a2, a2, 1
```

```
    li   a3, YELLOW
    call lamp
    li   a0, 2              # column 2: Cin
    mv   a1, s0
    andi a2, s0, 1
    li   a3, YELLOW
    call lamp
    li   a0, 4              # column 4: S
    mv   a1, s0
    mv   a2, s1
    li   a3, CYAN
    call lamp
    li   a0, 5              # column 5: Cout
    mv   a1, s0
    mv   a2, s2
    li   a3, CYAN
    call lamp

    addi s0, s0, 1
    li   t0, 8
    blt  s0, t0, 1b
    lw   s2, 16(sp)
    lw   s1, 20(sp)
    lw   s0, 24(sp)
    lw   ra, 28(sp)
    addi sp, sp, 32
    ret
```

Listato 9.3. Il programma completo del capitolo, `code/l09.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 9.4; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l09.S --portable \
    --entry-prefix l09 --name l09 --out l09.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l09.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 9.4. Compilare `l09.S` ed eseguirlo su PRG32-QT.

9.7 Verificate di aver capito

D9.1 Quante funzioni booleane distinte di tre variabili esistono?

A. 8
B. 9

C. 64
D. 256

D9.2 Quale mintermine di tre variabili A, B, C corrisponde alla riga 5 della tabella di verità?

A. $\overline{A}\,B\,\overline{C}$
B. $A\,\overline{B}\,C$
C. $A + \overline{B} + C$
D. $A\,B\,\overline{C}$

D9.3 In una mappa di Karnaugh, un gruppo rettangolare di quattro 1 adiacenti elimina:

A. una variabile
B. due variabili
C. tre variabili
D. quattro variabili

D9.4 In un sommatore completo, il riporto uscente vale 1 quando:

A. esattamente un ingresso vale 1
B. un numero dispari di ingressi vale 1
C. almeno due ingressi valgono 1
D. solo quando tutti e tre gli ingressi valgono 1

9.8 Esercizi

Esercizio 9.1 (prima su carta, poi su PRG32)

1. Scrivete le forme canoniche P e S delle uscite S e C_{out} del sommatore completo.
2. Minimizzate C_{out} con una mappa di Karnaugh di tre variabili. Quante porte si risparmiano?
3. Minimizzate $F = \sum m(0,1,2,5,8,9,10)$ con una mappa di quattro variabili.
4. Costruite NOT, AND e OR usando solo NAND, e verificate ciascuno con una tabella di verità.
5. Programmate il sommatore completo su PRG32 e mostrate S e C_{out} come lampade per le otto combinazioni degli ingressi.

Risultato atteso. Su carta: la forma canonica di C_{out} ha quattro mintermini (3, 5, 6, 7) e la mappa dà tre clausole di due letterali, $C_{out} = AB + AC_{in} + BC_{in}$, la funzione di *maggioranza*. Il terzo compito dà $F = \overline{B}\,\overline{D} + \overline{B}\,\overline{C} + \overline{A}\,\overline{C}\,D$. Su PRG32: otto coppie di lampade che riproducono la tabella di verità del sommatore completo.

9.9 Glossario

Inglese	**Italiano**
literal	letterale
clause, product term	clausola, termine prodotto
factor, sum term	fattore, termine somma
minterm / maxterm	mintermine / maxtermine
half adder / full adder	semisommatore / sommatore completo
carry in / carry out	riporto entrante / uscente
canonical form	forma canonica
sum of products (SOP)	somma di prodotti (tipo P)
product of sums (POS)	prodotto di somme (tipo S)
Karnaugh map	mappa di Karnaugh
don't care	condizione di indifferenza
ripple-carry adder	sommatore a propagazione del riporto

9.10 Conclusioni e letture

Una funzione booleana è una tabella di verità, e una tabella di verità dà sempre un'espressione canonica. La minimizzazione è il processo che ottimizza una rete logica per semplificarne la realizzazione. Le mappe di Karnaugh trasformano l'algebra in un disegno: raggruppare gli 1, leggere le clausole. Il semisommatore e il sommatore completo sono i primi veri circuiti di questo libro, e trentadue sommatori completi in fila sono ciò che esegue la `add` scritta nel Capitolo 2.

Letture. Patterson e Hennessy (2020), appendice A; Bucci (2017), i capitoli sulle reti combinatorie; Charles Petzold, *Code*, i capitoli sulle porte logiche e sulle macchine addizionatrici binarie.

CAPITOLO 10

Elettronica: di che cosa è fatto un bit

"Qualunque tecnologia sufficientemente avanzata è indistinguibile dalla magia."
— Arthur C. Clarke, *Profiles of the Future*, 1973

In questo capitolo

Alla fine di questo capitolo saprete

- usare tensione, corrente, resistenza e potenza, e applicare la legge di Ohm a un circuito semplice;
- dimensionare la resistenza di un LED e calcolare un partitore di tensione;
- spiegare che cos'è un transistor e come un transistor MOS funziona da interruttore;
- dire come le tensioni diventano valori logici, e che cos'è un margine di rumore;
- disegnare l'invertitore, il NAND e il NOR in CMOS, e contarne i transistor;
- spiegare le resistenze di pull-up, gli ingressi flottanti e perché una porta impiega tempo ed energia per commutare.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI gli ingressi con pull-up; tensione, corrente, legge di Ohm; il transistor come interruttore; dai volt ai bit; le porte CMOS.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI il clock (Capitolo 13).

Le porte degli ultimi due capitoli erano simboli sulla carta. Questo capitolo dà loro un corpo fisico, prima che il prossimo le componga in macchine. Non si presuppone alcuna conoscenza di fisica. I numeri citati per il kit vengono dal datasheet dell'ESP32-C6 (Espressif Systems, 2025) e dalla documentazione hardware di PRG32.

10.1 Riscaldamento sulla macchina

```
1     call prg32_input_read    # a0 = one bit per button
```

```
    andi t3, a0, 0x01        # bit 0: LEFT
    beqz t3, 1f              # 0: not pressed
    ...                      # 1: pressed
1:

# on the kit:
#   LEFT is a switch between pin GPIO18 and ground
#   pressed  -> the pin is at 0 volts
#   released -> the pin is at 3.3 volts
```

Listato 10.1. Un pulsante, un bit.

Fin dai primi capitoli un pulsante premuto è un 1 in un registro (Listato 10.1). Sulla scheda il pulsante è un pezzo di metallo che unisce un piedino del chip a massa. Premuto significa 0 volt sul piedino, eppure il programma legge 1. Che cos'è un volt? Perché il piedino rilasciato è a 3,3 V? E chi decide che 0 V è un bit? Il capitolo segue il pulsante dal dito al bit, e il suo ultimo paragrafo risponde alle tre domande.

10.2 L'elettricità in quattro grandezze

10.2.1 Tensione, corrente, resistenza, potenza

La carica elettrica è portata dagli elettroni, e quattro grandezze descrivono ciò che essa fa in un circuito (Tabella 10.1). Un paragone con l'acqua nei tubi aiuta l'intuizione.

Tabella 10.1. Quattro grandezze.

Grandezza	Simbolo, unità	Analogia con l'acqua	Sul kit
tensione	V, volt (V)	la pressione che spinge l'acqua	3,3 V tra la linea di alimentazione e la massa
corrente	I, ampere (A)	la portata: litri al secondo	circa 10 mA attraverso un LED acceso
resistenza	R, ohm (Ω)	un tubo stretto che ostacola il flusso	45 kΩ: il pull-up dentro ogni piedino di ingresso
potenza	P, watt (W)	il lavoro che il flusso può compiere ogni secondo	al più 2,5 W da una porta USB 2.0 (5 V, 0,5 A)

Una tensione si misura sempre *tra due punti*. Quando si dice che «il piedino è a 3,3 V» si intende rispetto alla *massa* (GND), il riferimento comune dell'intero circuito, che è a 0 V per definizione. I prefissi usuali sono milli (m, 10^{-3}), micro (μ, 10^{-6}), nano (n, 10^{-9}), pico (p, 10^{-12}), kilo (k, 10^{3}) e mega (M, 10^{6}).

10.2.2 La legge di Ohm, e la resistenza di un LED

Teorema 10.1 (Legge di Ohm, 1827)

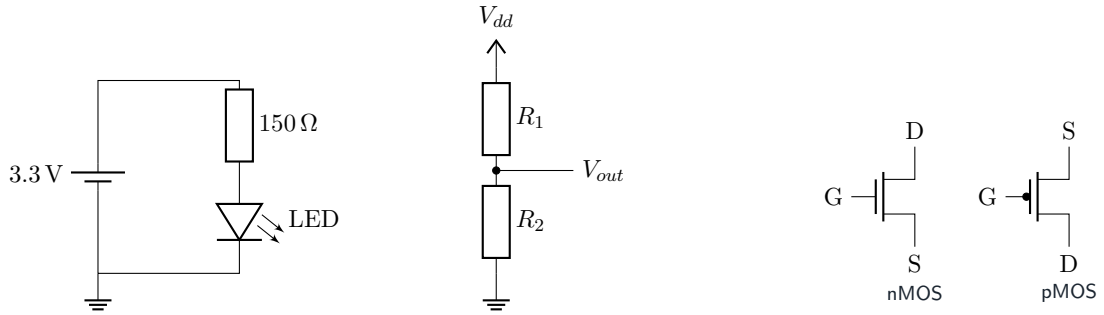
La corrente attraverso un resistore è proporzionale alla tensione ai suoi capi:

$$V = R \cdot I.$$

Un circuito è un anello chiuso: la corrente esce dalla sorgente, attraversa i componenti e ritorna. Un LED (Figura 10.1a) si accende quando è attraversato da corrente, e quando lo è ai suoi capi c'è una caduta di circa 2 V; senza una resistenza in serie la corrente sarebbe di gran lunga troppo grande. La resistenza prende ciò che resta, $3{,}3 - 2 = 1{,}3$ V. Per una corrente di 10 mA,

$$R = \frac{1{,}3 \text{ V}}{0{,}010 \text{ A}} = 130 \ \Omega,$$

e il valore commerciale più vicino è 150 Ω, che dà $1{,}3/150 = 8{,}7$ mA. La tensione diretta di un LED dipende dal suo colore: circa 1,8–2,2 V per il rosso, circa 3 V per il blu e il bianco.



(a) Un LED e la sua resistenza **(b)** Un partitore di tensione **(c)** I due transistor MOS

Figura 10.1. Tre circuiti elementari.

10.2.3 Serie, parallelo e il partitore di tensione

Due resistori sono in *serie* quando la stessa corrente li attraversa entrambi: le resistenze si sommano, $R = R_1 + R_2$. Sono in *parallelo* quando ai capi di entrambi c'è la stessa tensione: la corrente si divide, e la resistenza totale, $1/R = 1/R_1 + 1/R_2$, è minore di ciascuna delle due.

Due resistori in serie dividono la tensione (Figura 10.1b):

$$V_{out} = V_{dd} \frac{R_2}{R_1 + R_2}. \tag{10.1}$$

Resistori uguali danno metà dell'alimentazione; una R_2 molto più piccola porta V_{out} vicino alla massa. La formula segue dalla legge di Ohm, poiché la corrente $I = V_{dd}/(R_1 + R_2)$ attraversa R_2, e dalle leggi di Kirchhoff: le correnti che entrano in un nodo sommate danno quelle che ne escono, e le tensioni lungo una maglia sommate danno zero. Il partitore è la chiave del circuito del pulsante, dove R_2 è un interruttore: una resistenza che è o infinita o quasi nulla.

10.2.4 Potenza ed energia

La *potenza* è tensione per corrente, $P = V \cdot I$, in watt; in un resistore diventa calore, $P = R \cdot I^2$. L'*energia* è potenza per tempo, in joule; una batteria immagazzina energia. Il LED di prima consuma $3{,}3 \text{ V} \times 8{,}7 \text{ mA} = 29$ mW, di cui 11 mW scaldano la resistenza. Ogni transistor che commuta dentro un processore dissipa un po' di energia, e con miliardi di transistor il calore è il limite dei chip moderni: il processore di un portatile dissipa decine di watt in pochi centimetri quadrati.

10.2.5 Il condensatore: perché i segnali impiegano tempo

Un *condensatore* immagazzina carica elettrica come un serbatoio immagazzina acqua; la sua capacità si misura in farad (F). La sua tensione non può saltare: il condensatore va riempito, o svuotato, attraverso una resistenza, e il tempo che ciò richiede è dell'ordine del prodotto

$$\tau = R \cdot C,$$

la *costante di tempo*, trascorsa la quale la tensione ha percorso circa il 63% della strada. Un piedino di ingresso del kit ha una capacità di circa 2 pF e sale attraverso il suo pull-up di 45 kΩ in circa 45 kΩ × 2 pF = 90 ns (i collegamenti e il pulsante ne aggiungono altra, quindi il valore reale è maggiore).

Ogni filo e ogni ingresso di un transistor è un piccolo condensatore. È da qui che viene il ritardo di propagazione di una porta: resistenza per capacità è un tempo, ed è la ragione per cui in un circuito niente è istantaneo. Un condensatore carico o scarico è anche un modo di memorizzare un bit: la cella DRAM del Capitolo 15.

10.3 Semiconduttori

10.3.1 Conduttori, isolanti, semiconduttori

I conduttori, come il rame, lasciano scorrere liberamente la carica; gli isolanti, come il vetro, no. I *semiconduttori*, il silicio sopra tutti, stanno in mezzo, e la loro conduzione può essere controllata. Il controllo si ottiene con il *drogaggio*: l'aggiunta di pochi atomi di un altro elemento. Il silicio di tipo n ha elettroni in più, che sono portatori di carica negativi; il silicio di tipo p ha elettroni mancanti, chiamati *lacune*, che si comportano come portatori positivi. Dove una regione p incontra una regione n c'è una *giunzione*, il mattone di ogni dispositivo. Un chip è una fetta di silicio con miliardi di regioni drogate, collegate da strati di metallo. Il silicio è il secondo elemento più abbondante nella crosta terrestre, l'elemento della sabbia, e un'intera industria poggia sul purificarlo e drogarlo con estrema precisione.

10.3.2 Il diodo e il LED

Una sola giunzione fa un *diodo*, una valvola a senso unico per la corrente. In senso diretto, da p a n, conduce una volta che la tensione supera una soglia, circa 0,7 V per il silicio; in senso inverso blocca. Un diodo a emissione di luce emette luce quando conduce; il suo colore dipende dal materiale, e così la sua soglia. I diodi trasformano la corrente alternata in continua in ogni alimentatore. I LED sono dappertutto in un calcolatore: indicatori, la retroilluminazione del display del Capitolo 20, collegamenti ottici; e anche il laser dei dischi ottici del Capitolo 16 è un diodo.

10.3.3 Il transistor: un interruttore senza parti mobili

Un *transistor* è un dispositivo con tre terminali in cui un piccolo segnale su uno controlla la corrente tra gli altri due. Usato come amplificatore ha reso portatile la radio; usato come interruttore ha fatto il calcolatore (Tabella 10.2). Valvole termoioniche e relè facevano lo stesso lavoro nei primi calcolatori del Capitolo 3, un milione di volte più grandi e più lenti.

10.3.4 Il transistor MOS come interruttore

Il nome MOS elenca i tre strati del dispositivo: metallo, ossido, semiconduttore. I suoi tre terminali sono il *gate*, che controlla se la corrente può passare, e il *source* e il *drain*, tra i

Tabella 10.2. Il transistor in sei date.

Anno	Chi	Che cosa
1947	Bardeen, Brattain e Shockley, Bell Labs	il primo transistor: sostituisce la valvola termoionica
1958–59	Jack Kilby; Robert Noyce	il circuito integrato: più transistor su un solo pezzo di semiconduttore
1959–60	Atalla e Kahng, Bell Labs	il transistor MOS: piccolo, semplice da costruire, quasi nessuna corrente di controllo
1963	Wanlass e Sah, Fairchild	il CMOS: coppie di transistor MOS complementari che assorbono potenza solo quando commutano
1971	Intel 4004	2 300 transistor MOS: un processore su un chip
oggi	l'intera industria	decine di miliardi di transistor CMOS su un chip

quali essa passa (Figura 10.1c). Il gate è isolato: non assorbe praticamente corrente (al più 50 nA su un piedino dell'ESP32-C6), solo la minuscola carica della sua capacità.

Definizione 10.1 (nMOS e pMOS)

Un transistor *nMOS* conduce (l'interruttore è chiuso) quando il suo gate è alto, e blocca quando è basso. Un transistor *pMOS* conduce quando il suo gate è basso, e blocca quando è alto.

I due tipi sono complementari: ciò che chiude l'uno apre l'altro. Il cerchietto sul gate del pMOS significa, come sui simboli logici, «attivo quando basso». Un altro fatto decide come vengono usati: un nMOS lascia passare un buon 0, ed è quindi usato per collegare a massa; un pMOS lascia passare un buon 1, ed è usato per collegare all'alimentazione.

10.4 Dai volt ai bit

10.4.1 Livelli logici: quando una tensione è un 1?

Un circuito digitale usa due *intervalli* di tensione: basso è 0, alto è 1 (Figura 10.2). Un'uscita garantisce livelli puliti: sul kit, sopra 2,64 V per un 1 e sotto 0,33 V per uno 0. Un ingresso accetta intervalli più ampi: sopra 2,475 V legge 1, sotto 0,825 V legge 0. (Con $V_{DD} = 3{,}3$ V il datasheet dà $V_{OH} \geq 0{,}8\,V_{DD}$, $V_{OL} \leq 0{,}1\,V_{DD}$, $V_{IH} \geq 0{,}75\,V_{DD}$, $V_{IL} \leq 0{,}25\,V_{DD}$.)

Definizione 10.2 (Margine di rumore)

Il *margine di rumore* è la differenza tra il livello che un'uscita garantisce e il livello che un ingresso richiede. Un disturbo più piccolo del margine di rumore non può cambiare il bit.

Sul kit i margini sono $2{,}64 - 2{,}475 = 0{,}165$ V per il livello alto e $0{,}825 - 0{,}33 = 0{,}495$ V per il livello basso. Tra le due soglie di ingresso il valore è indefinito: i segnali devono attraversare quella fascia in fretta e non restarci mai.

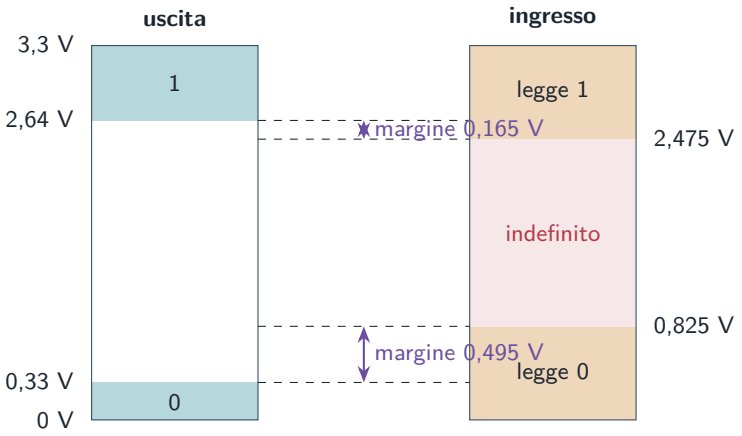


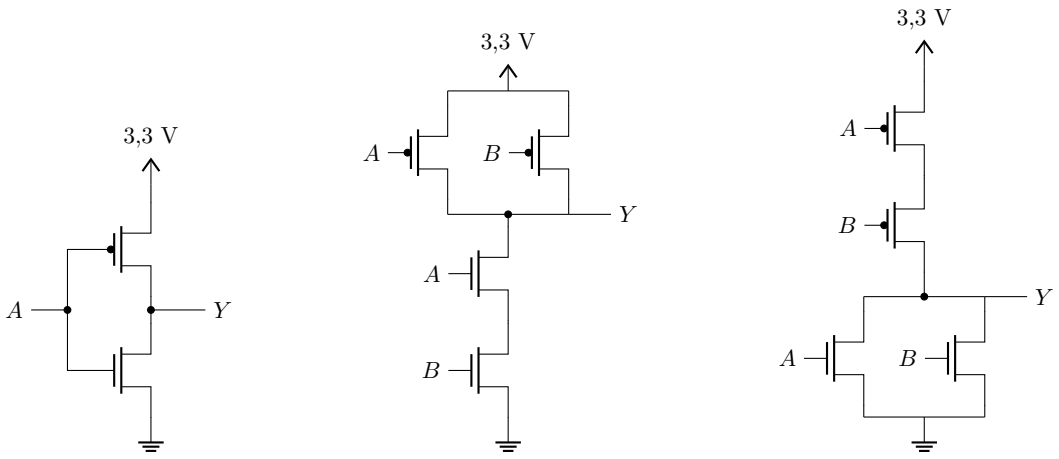
Figura 10.2. Livelli logici dell'ESP32-C6 con un'alimentazione di 3,3 V.

10.4.2 Perché digitale?

In un circuito *analogico* il valore esatto della tensione è l'informazione. Ogni stadio aggiunge un po' di rumore e di distorsione, che si accumulano, e le copie peggiorano a ogni generazione. I circuiti analogici restano indispensabili ai bordi di un sistema: sensori, radio, suono. In un circuito *digitale* conta solo l'intervallo, alto o basso. Ogni porta ripristina i livelli, così che il rumore non si accumula; le copie sono perfette, e gli errori possono essere rilevati e corretti (Capitolo 11). E due livelli sono facili da distinguere con transistor veloci, minuscoli ed economici. È la storia del Capitolo 4, vista dal filo.

10.4.3 L'invertitore CMOS: una porta NOT con due transistor

La porta più semplice è fatta di un pMOS verso l'alimentazione e di un nMOS verso massa, pilotati dallo stesso ingresso (Figura 10.3a).



(a) NOT: $Y = \overline{A}$ **(b)** NAND: $Y = \overline{A \cdot B}$ **(c)** NOR: $Y = \overline{A + B}$

Figura 10.3. Tre porte CMOS. Nel NAND e nel NOR i gate dei transistor segnati con la stessa lettera sono collegati insieme.

A	**pMOS**	**nMOS**	Y
0	conduce	interdetto	1: unita a 3,3 V
1	interdetto	conduce	0: unita a massa

Esattamente uno dei due transistor conduce: l'uscita è sempre collegata a una linea, mai a entrambe. Con l'ingresso fermo, nessuna corrente scorre dall'alimentazione a massa, così che il CMOS (MOS *complementare*) non consuma quasi niente a riposo. E l'uscita è un livello nuovo e pulito qualunque sia la qualità dell'ingresso: la porta *rigenera* il segnale.

10.4.4 NAND e NOR in CMOS

Serie è AND, parallelo è OR

Due interruttori in serie conducono quando il primo *e* il secondo sono chiusi; due interruttori in parallelo conducono quando il primo *o* il secondo è chiuso. È l'idea di Nakajima e Shannon.

Nel NAND (Figura 10.3b) due nMOS in serie raggiungono la massa solo se A e B valgono entrambi 1, e due pMOS in parallelo raggiungono l'alimentazione se uno dei due vale 0. Il NOR (Figura 10.3c) è il duale: il parallelo diventa serie e la serie diventa parallelo, esattamente come le leggi di De Morgan scambiano AND e OR. In ogni porta CMOS la rete di pMOS che porta l'uscita in alto e la rete di nMOS che la porta in basso sono l'una la duale dell'altra, nel senso del principio di dualità del Capitolo 8. Ciascuna delle due porte ha quattro transistor; verificate sui disegni le quattro combinazioni degli ingressi.

10.4.5 Quanto costano le porte, in transistor

Tabella 10.3. Il costo di alcuni circuiti in CMOS statico convenzionale; i conteggi variano con lo stile di progetto.

Circuito	**Transistor**	**Nota**
NOT	2	l'invertitore CMOS
NAND, NOR (2 ingressi)	4	le porte naturali del CMOS
AND, OR (2 ingressi)	6	un NAND o un NOR seguito da un invertitore
XOR (2 ingressi)	da 8 a 12 circa	secondo il progetto
sommatore completo	circa 28	il classico sommatore CMOS statico
cella SRAM, un bit	6	due invertitori ad anello e due transistor di accesso
flip-flop D	circa 20	due latch, master e slave

Un AND costa più di un NAND (Tabella 10.3): in CMOS le porte invertenti vengono per prime. È per questo che il NAND è la porta universale del Capitolo 9 in pratica oltre che in teoria. Minimizzare una funzione significa risparmiare transistor, e quindi area, energia e tempo. Il sommatore che sta dietro l'istruzione `add`, trentadue sommatori completi, è fatto di circa 900 transistor.

10.4.6 Commutare costa tempo ed energia

L'uscita di una porta pilota gli ingressi delle porte successive, che sono piccoli condensatori. Per cambiare livello l'uscita deve caricarli o scaricarli attraverso la resistenza dei suoi tran-

sistor: è il *ritardo di propagazione* della porta. E ogni cambio di livello costa energia: la potenza spesa nella commutazione è circa

$$P_{dyn} \approx C \cdot V^2 \cdot f,$$

dove C è la capacità commutata, V la tensione di alimentazione e f la frequenza di commutazione. Abbassare la tensione di alimentazione è ciò che fa risparmiare di più, perché compare al quadrato: è la ragione per cui i chip sono passati da 5 V a 3,3 V e a circa 1 V all'interno. Alzare la frequenza del clock alza il consumo in proporzione: la velocità si paga in calore. Il ritardo di propagazione ritorna nel Capitolo 11 come cammino critico, e nel Capitolo 13 come la ragione del clock.

10.4.7 Scale di integrazione

Tabella 10.4. Scale di integrazione.

Scala	Transistor per chip	Esempi
SSI, piccola	fino a circa 100	poche porte: la serie 74, come quattro NAND in un contenitore
MSI, media	centinaia	decodificatori, multiplexer, contatori, registri: le macchine dei Capitoli da 11 a 14
LSI, grande	migliaia	i primi microprocessori e le prime memorie: Intel 4004
VLSI, molto grande	centinaia di migliaia e oltre	i processori degli anni Ottanta e Novanta
oggi	decine di miliardi	sistemi su un chip: processore, memoria, radio e periferiche insieme

L'ESP32-C6 del kit è un sistema su un chip in un contenitore di pochi millimetri (Tabella 10.4). I chip della serie 74 ricompaiono nei prossimi capitoli come la forma storica di ogni blocco costruttivo.

10.5 Ritorno alla macchina

10.5.1 Pull-up: un ingresso che non resta mai flottante

Un ingresso collegato a niente è *flottante*: raccoglie rumore e legge 0 o 1 a caso. Una *resistenza di pull-up* lega il piedino a 3,3 V (Figura 10.4). Con il pulsante rilasciato non scorre corrente, e il piedino legge un 1 pulito. Con il pulsante premuto l'interruttore unisce il piedino a massa: 0 V, uno 0 pulito, mentre attraverso la resistenza scorre una corrente di 3,3 V/45 kΩ = 73 μA, troppo piccola per contare. Il circuito è il partitore di tensione della (10.1) con un resistore inferiore che è o infinito o nullo.

Il firmware abilita il pull-up interno di ogni piedino dei pulsanti, legge i piedini e inverte i bit, così che nel programma premuto è 1. Di un segnale il cui stato attivo è il livello basso si dice che usa una logica *attiva bassa* (ingl. *active-low*).

10.5.2 Dal dito al bit

Le domande del riscaldamento hanno ora una risposta, e la Tabella 10.5 le mette in ordine. Alcuni dei suoi passi appartengono a capitoli che devono ancora venire.

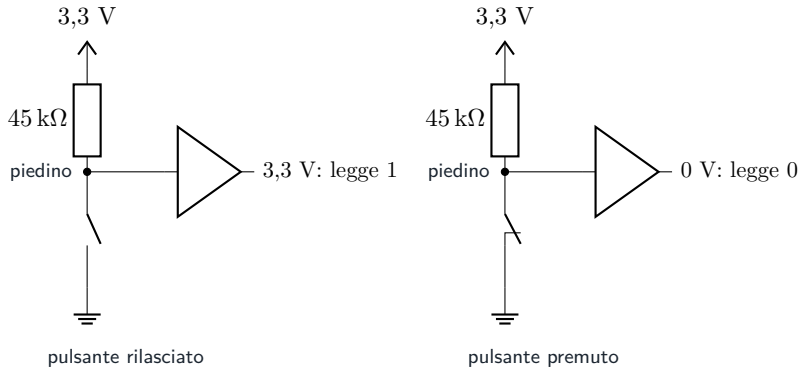


Figura 10.4. Il pulsante del kit con la sua resistenza di pull-up. Il triangolo è il buffer di ingresso del piedino.

Tabella 10.5. Dal dito al bit.

Passo	Che cosa succede	Dove è spiegato
1	il dito chiude l'interruttore tra GPIO18 e massa	questo capitolo: il circuito
2	il piedino scende da 3,3 V a 0 V, in molto meno di un microsecondo	questo capitolo: $R \cdot C$
3	il buffer di ingresso vede una tensione sotto 0,825 V: uno 0 logico	questo capitolo: i livelli logici
4	il livello è campionato in un flip-flop del registro di ingresso	Capitolo 13
5	il firmware legge quel registro con un'istruzione di load	Capitolo 15: I/O mappato in memoria
6	inverte il bit e lo impacchetta con gli altri in `a0`	Capitoli 8 e 21
7	il programma lo esamina: `andi t3, a0, 0x01`	Capitolo 2

I contatti di un interruttore reale *rimbalzano* per qualche millisecondo quando si chiudono. Leggere i pulsanti una volta per frame, ogni poche decine di millisecondi, nasconde il rimbalzo (Capitolo 21).

10.5.3 Il kit come circuito

Il kit della Figura 1.2 può ora essere letto come un circuito. Una *basetta sperimentale* (ingl. *breadboard*) ha fori uniti a file nella parte inferiore, così che i componenti si collegano senza saldature. Due lunghi bus portano l'alimentazione, 3,3 V e massa; un bus separato porta 5 V per gli amplificatori audio. Ogni parte condivide la stessa massa: senza un riferimento comune le tensioni di due parti non significano niente l'una per l'altra. I pulsanti vanno da un piedino a massa; il display e gli amplificatori ricevono segnali digitali da altri piedini. Un piedino di uscita può accendere un LED attraverso una resistenza; un piedino di ingresso ascolta soltanto.

Attenzione

Un piedino dell'ESP32-C6 può fornire alcune decine di milliampere: a un LED serve la sua resistenza in serie. Non unite mai direttamente a massa un piedino pilotato alto.

10.6 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l10.S` nel repository del corso, stampato per intero nel Listato 10.2. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 10.5 è il frame scritto dal test.

Sullo schermo.
Un grande quadrato, grigio mentre LEFT è rilasciato e giallo mentre è premuto. Sul kit LEFT è un interruttore tra GPIO18 e massa.

Che cosa premere.
Premete e rilasciate LEFT.

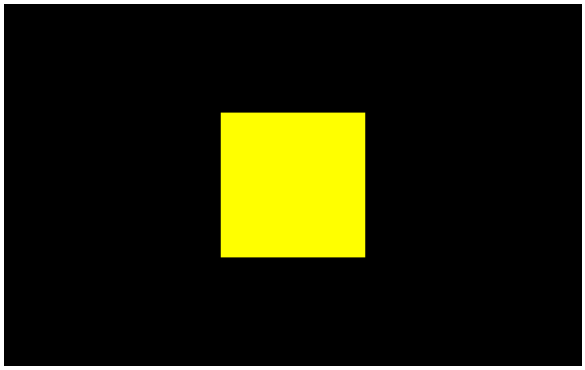


Figura 10.5. `l10.S` su PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 10: Introduction to Electronics and Digital Electronics
# l10.S: one button, one bit: from the finger to a register
#
# On the screen: a large square, grey while LEFT is released and yellow while it is
# pressed. On the kit LEFT is a switch between GPIO18 and ground.
# Build: python3 -m prg32 cartridge build l10.S --portable \
#            --entry-prefix l10 --name l10 --out l10.prg32
.option norelax

.equ BLACK,  0x0000
.equ GREY,   0x4208
.equ YELLOW, 0xffe0

.section .text
.global l10_init
.global l10_update
.global l10_draw

l10_init:
    ret

l10_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    call prg32_input_read    # a0 = one bit per button
```

```
    andi t3, a0, 0x01        # bit 0: LEFT
    la   t0, pressed
    sw   t3, 0(t0)           # 0: not pressed, 1: pressed
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
# on the kit:
#   LEFT is a switch between pin GPIO18 and ground
#   pressed  -> the pin is at 0 volts
#   released -> the pin is at 3.3 volts
# the runtime inverts the level: in the program, pressed is 1

l10_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    la   t0, pressed
    lw   t1, 0(t0)
    li   a4, GREY
    beqz t1, 1f
    li   a4, YELLOW
1:  li   a0, 120
    li   a1, 60
    li   a2, 80
    li   a3, 80
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

.section .data
pressed: .word 0
```

Listato 10.2. Il programma completo del capitolo, `code/l10.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 10.3; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l10.S --portable \
    --entry-prefix l10 --name l10 --out l10.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l10.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 10.3. Compilare `l10.S` ed eseguirlo su PRG32-QT.

10.7 Verificate di aver capito

D10.1 Un LED fa cadere 2 V e deve portare 5 mA da un piedino a 3,3 V. Quale resistenza serve?

A. 26 Ω
B. 260 Ω
C. 400 Ω
D. 660 Ω

D10.2 In un invertitore CMOS con l'ingresso a 0:

A. entrambi i transistor conducono
B. il pMOS conduce e l'uscita vale 1
C. l'nMOS conduce e l'uscita vale 0
D. nessuno conduce e l'uscita è flottante

D10.3 Quanti transistor richiede un AND a due ingressi in CMOS statico?

A. 2
B. 4
C. 6
D. 8

D10.4 Perché l'ingresso del pulsante del kit ha bisogno di una resistenza di pull-up?

A. Per limitare la corrente del LED
B. Perché il piedino legga un 1 definito quando il pulsante è rilasciato
C. Per rendere il pulsante più veloce
D. Per proteggere il chip dai 5 V

10.8 Esercizi

Esercizio 10.1 (prima su carta, poi sul kit)

1. Un LED blu fa cadere 3,0 V: quale resistenza dà 2 mA da 3,3 V? Quale potenza dissipa la resistenza?
2. Un partitore con $R_1 = 10$ kΩ e $R_2 = 20$ kΩ su 3,3 V: calcolate V_{out}. Un ingresso del kit lo legge come 0, come 1, o è indefinito?
3. Disegnate un NAND a tre ingressi in CMOS e contatene i transistor.
4. Verificate il disegno del NOR per le quattro combinazioni degli ingressi, segnando quali transistor conducono.
5. Su PRG32, eseguite la cartuccia delle lampade e collegate con un filo un piedino dei pulsanti a massa: spiegate ciò che vedete, e che cosa succederebbe con il pull-up disabilitato.

Risultato atteso. Su carta: 150 Ω e 0,6 mW; $V_{out} = 2{,}2$ V, nella fascia indefinita; sei transistor, tre pMOS in parallelo e tre nMOS in serie. Sul kit: la lampada si accende esattamente come se il pulsante fosse premuto.

10.9 Glossario

Inglese	**Italiano**
voltage	tensione
current	corrente
resistor / resistance	resistore / resistenza
capacitor	condensatore
ground	massa
semiconductor, doping	semiconduttore, drogaggio
gate, source, drain	gate, source, drain
logic level	livello logico
noise margin	margine di rumore
pull-up resistor	resistenza di pull-up
floating input	ingresso flottante
propagation delay	ritardo di propagazione
integrated circuit	circuito integrato
breadboard	basetta sperimentale (breadboard)

10.10 Conclusioni e letture

La tensione spinge, la corrente scorre, la resistenza ostacola: $V = R \cdot I$ descrive la maggior parte di ciò che serve a un progettista digitale. Un transistor MOS è un interruttore controllato da una tensione, e nMOS e pMOS sono complementari. Un bit è un intervallo di tensioni; i margini di rumore e la rigenerazione sono ciò che rende affidabili i circuiti digitali. Le porte CMOS sono coppie di reti di interruttori, e le porte invertenti NAND e NOR sono le meno costose. Commutare costa tempo ($R \cdot C$) ed energia ($C \cdot V^2 \cdot f$): i limiti fisici dietro ogni clock.

Letture. Charles Petzold, *Code*, i capitoli dalla torcia elettrica alle porte logiche; Patterson e Hennessy (2020), paragrafi 1.5 e 1.7 e appendice A; `docs/hardware/hardware.md` nel repository di PRG32.

CAPITOLO 11

Macchine combinatorie

"... non è colpa mia, le carote hanno toccato i piselli..."
— Alan Turing da ragazzo, nel film *The Imitation Game*, 2014

In questo capitolo

Alla fine di questo capitolo saprete

- distinguere una macchina combinatoria da una sequenziale;
- calcolare quanti bit richiede un codice a lunghezza fissa, e riconoscere i codici incompleti e ridondanti;
- progettare un decodificatore e un codificatore, e comporre decodificatori piccoli in decodificatori più grandi;
- spiegare che cosa fanno un multiplexer e un demultiplexer, e costruirli con le porte;
- costruire un comparatore e una rete di parità, e dire che cosa un controllo di parità può e non può rilevare;
- indicare ciascuna di queste macchine dentro un processore RISC-V.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI i multiplexer; le reti combinatorie; i codici; decoder ed encoder; comparatori e parità; l'ALU.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI macchine a stati finiti (Capitolo 13).

11.1 Riscaldamento sulla macchina

```
# a0 = selector, 0 to 3      result: a4 = the selected colour

    la   t0, palette         # four words: the data inputs
    slli t1, a0, 2           # selector * 4 bytes
    add  t0, t0, t1          # address of the selected word
    lw   a4, 0(t0)           # the selected input reaches the output
```

```
7
8  .section .rodata
9  # red, green, blue, yellow
10 palette: .word 0xf800, 0x07e0, 0x001f, 0xffe0
```

Listato 11.1. Scegliere uno di quattro colori.

Il Listato 11.1 è l'accesso a un array del Capitolo 4, usato come selezione. Lo stesso selettore dà sempre lo stesso colore: il risultato dipende solo dagli ingressi presenti, e nulla viene ricordato da un uso al successivo. In hardware questo è un *multiplexer*: quattro ingressi dati, due bit di selezione, un'uscita. Un processore ne contiene decine, e fa la selezione con le porte in molto meno di un nanosecondo, dove il programma impiega quattro istruzioni. Come sono costruiti?

11.2 Macchine combinatorie

11.2.1 Una tassonomia dei circuiti digitali

Definizione 11.1 (Circuiti combinatori e sequenziali)

In un circuito *combinatorio* il valore delle uscite in un dato istante dipende solo dal valore degli ingressi in quello stesso istante. In un circuito *sequenziale* le uscite dipendono dagli ingressi presenti e dalla storia passata degli ingressi.

Un circuito combinatorio non ha memoria né anelli di retroazione, ed è completamente descritto da una tabella di verità: sommatori, decodificatori, multiplexer e la ALU sono combinatori. Un circuito sequenziale richiede i concetti di *tempo* e di *stato*, ed è descritto da una tabella degli stati o da un diagramma degli stati: latch, flip-flop, registri, contatori e il processore nel suo insieme sono sequenziali. Sono l'argomento del Capitolo 13.

11.2.2 Progettare una macchina combinatoria

Il progetto segue il metodo del Capitolo 9 (Figura 11.1): da una specifica a parole a una tabella di verità, con una riga per combinazione degli ingressi e con le condizioni di indifferenza; dalla tabella a espressioni minime, un'uscita alla volta; dalle espressioni a una rete di porte, o di blocchi standard. L'analisi va nell'altro verso, dallo schema alla tabella di verità. Per più di quattro o cinque ingressi nessuno disegna mappe: i progettisti compongono blocchi standard, ed è per questo che il capitolo ne è un catalogo.

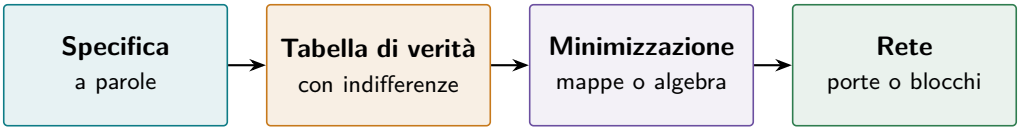


Figura 11.1. La sintesi di una macchina combinatoria.

11.2.3 Le porte reali impiegano tempo

Una porta non risponde all'istante: la sua uscita si assesta dopo un ritardo di propagazione (Capitolo 10). Il ritardo di una rete è quello del suo percorso più lento da un in-

gresso a un'uscita, il *cammino critico*. Mentre i segnali viaggiano, le uscite possono essere temporaneamente sbagliate.

Concetto

Una macchina combinatoria è corretta solo dopo che si è assestata. Qualcosa deve dire quando guardare le sue uscite: quel qualcosa è il clock, ed è lì che cominciano le macchine sequenziali.

Nel sommatore a propagazione del riporto del Capitolo 9 il cammino critico attraversa tutti i trentadue sommatori completi, perché l'ultimo riporto dipende dal primo. I sommatori più veloci, come il sommatore *carry look-ahead*, calcolano i riporti con porte aggiuntive disposte su pochi livelli: spendono area per accorciare il cammino critico. La funzione calcolata è la stessa. Il periodo di clock di un processore deve essere più lungo del suo cammino critico più lungo.

11.3 Codici

Prima di costruire macchine che trattano codici, deve essere chiaro che cos'è un codice.

11.3.1 Il concetto di informazione

Un'informazione ha tre aspetti. L'*attributo* è il significato che le è associato: per esempio, il voto dell'esame di Architettura dei Calcolatori. Il *tipo* è l'insieme degli elementi tra i quali il valore è scelto: i voti da 18 a 30, e 30 e lode. Il *valore* è l'elemento effettivamente assunto: 30. Dare un'informazione è scegliere un elemento in un insieme definito.

La *cardinalità* N del tipo è il numero dei suoi elementi, qui $N = 14$; un tipo con più elementi porta più informazione. Lo stesso valore si può scrivere in molti modi: 3, III, tre; e A, B, C, D possono stare per 30 e lode, 30, 29, 28. «30 e lode» non è un numero: è un quattordicesimo elemento del tipo, e come scriverlo è un problema di codifica. Infine, i tipi sono *semplici* quando sono atomici, come gli interi e i caratteri, e *strutturati* quando combinano tipi semplici: una posizione è una latitudine e una longitudine.

11.3.2 Alfabeti e stringhe

Definizione 11.2 (Alfabeto e stringa)

Un *alfabeto* è un insieme finito e non vuoto di simboli. Una *stringa*, o parola, su un alfabeto è una sequenza finita di suoi simboli. La *lunghezza* $|x|$ di una stringa x è il numero dei suoi simboli; la stringa di lunghezza 0 è la stringa vuota. L'insieme di tutte le stringhe su un alfabeto S, quella vuota compresa, si scrive S^*.

L'alfabeto binario è $\{0,1\}$, quello decimale $\{0,\ldots,9\}$; `01000010` è una stringa binaria di lunghezza 8. Una sequenza infinita $010101\ldots$ non è una stringa, e non appartiene a $\{0,1\}^*$.

11.3.3 Codici a lunghezza fissa

Definizione 11.3 (Codice)

Un *codice* associa a ogni elemento di un tipo sorgente T, di cardinalità N, una stringa di simboli di un alfabeto di codice E con k simboli, la *parola codice*. In un codice a *lunghezza fissa* ogni parola codice ha la stessa lunghezza m.

Teorema 11.1 (Lunghezza minima)

Con un alfabeto di k simboli ci sono k^m parole di lunghezza m. Un codice a lunghezza fissa per N elementi esiste se e solo se $k^m \geq N$, così che la lunghezza minima è

$$m = \lceil \log_k N \rceil, \qquad \text{e in binario} \qquad m = \lceil \log_2 N \rceil.$$

I quattordici voti dell'esame richiedono 4 bit; i 32 registri di RISC-V ne richiedono esattamente 5, e quei cinque bit sono i campi `rd`, `rs1` e `rs2` di un'istruzione. Un codice è *incompleto* quando N non è una potenza di k: allora $k^m - N$ parole restano non assegnate. I quattordici voti lasciano non assegnate $16 - 14 = 2$ parole.

11.3.4 Un codice incompleto: il decimale codificato in binario

Le dieci cifre decimali richiedono $\lceil \log_2 10 \rceil = 4$ bit, e sei delle sedici parole non sono mai usate (Tabella 11.1). Nel codice BCD 8421, in cui ogni cifra è scritta come il suo valore in binario, le parole non usate vanno da 1010 a 1111. Le parole non usate diventano condizioni di indifferenza quando si progetta un circuito.

Tabella 11.1. Due codici per le cifre decimali.

Cifra	BCD 8421	Eccesso-3	Cifra	BCD 8421	Eccesso-3
0	0000	0011	5	0101	1000
1	0001	0100	6	0110	1001
2	0010	0101	7	0111	1010
3	0011	0110	8	1000	1011
4	0100	0111	9	1001	1100

11.3.5 Codici ridondanti

Definizione 11.4 (Codice ridondante)

Un codice è *ridondante* quando usa più simboli dello stretto necessario: la sua lunghezza è maggiore del minimo m.

In un codice ridondante solo alcune delle parole possibili sono lecite; le altre non possono essere prodotte dal codificatore. Se arriva una parola che non è lecita, qualcosa l'ha alterata lungo il percorso: l'errore è *rilevato*. La ridondanza è il prezzo pagato per rilevare, e talvolta correggere, gli errori.

Elemento	Codice 1	Codice 2
A	00	000
B	01	011
C	10	101
D	11	110

Il Codice 1 non è ridondante: ogni parola di due bit è lecita, così che un bit alterato trasforma un elemento in un altro senza che nessuno se ne accorga. Il Codice 2 aggiunge un terzo bit che rende pari il numero degli 1. Le parole 001, 010, 100 e 111 non sono lecite, e un singolo bit alterato è sempre rilevato. Il Codice 2 è il Codice 1 più un bit di *parità pari*; le macchine che lo calcolano vengono più avanti nel capitolo.

11.3.6 Codici a lunghezza variabile

In un codice a *lunghezza variabile* la lunghezza della parola codice dipende dall'elemento: gli elementi frequenti ricevono parole corte. Un tale codice deve avere la *proprietà del prefisso*: nessuna parola codice è l'inizio di una più lunga, così che chi legge sa sempre dove finisce una parola. Conviene quando gli elementi non hanno tutti la stessa probabilità: meno spazio in memoria, meno tempo in trasmissione. Il codice Morse e la codifica di Huffman sono gli esempi classici, e ne avete già incontrati altri due.

- **UTF-8**: da uno a quattro byte per carattere (Capitolo 4).
- **Le istruzioni RISC-V sul kit**: con l'estensione C ogni istruzione è lunga 16 o 32 bit. Un'istruzione i cui due bit meno significativi sono `11` è lunga 32 bit; ogni altra configurazione segnala un'istruzione di 16 bit. La lunghezza è scritta nella parola codice stessa.

11.4 Decodificatori e codificatori

11.4.1 La rappresentazione decodificata

All'estremo opposto del codice compatto c'è la rappresentazione *decodificata*, detta anche *one-hot*: un codice in cui la lunghezza della parola è uguale alla cardinalità dell'insieme, $m = N$, e ogni parola codice ha esattamente un bit a 1. C'è una linea per elemento: `0010` significa che il terzo di quattro elementi è quello scelto. La rappresentazione è semplice da usare, perché la linea attiva seleziona direttamente qualcosa, ma costosa in fili. La regola d'uso è: un codice compatto dentro una parola, una rappresentazione decodificata dove su una scelta bisogna agire. Le sette lampade del Capitolo 8 sono una rappresentazione decodificata di un numero di bit.

11.4.2 Il decodificatore

Definizione 11.5 (Decodificatore)

Un *decodificatore* riceve una parola codice di m bit e attiva la sola linea di uscita, tra $N = 2^m$, che le corrisponde.

Ogni uscita è un mintermine degli ingressi: un decodificatore li genera tutti (Figura 11.2). Con un ingresso di *abilitazione* EN, nessuna uscita è attiva quando EN $= 0$.

B	A	U_0	U_1	U_2	U_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

$$U_0 = \overline{B}\,\overline{A} \quad U_1 = \overline{B}\,A$$
$$U_2 = B\,\overline{A} \quad U_3 = B\,A$$

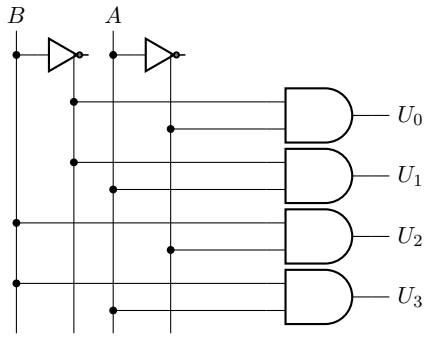


Figura 11.2. Il decodificatore da 2 a 4 (A è il bit meno significativo).

11.4.3 Comporre i decodificatori

I decodificatori grandi si costruiscono con quelli piccoli. La Figura 11.3 costruisce un decodificatore da 4 a 16 con cinque decodificatori da 2 a 4 con abilitazione: il primo decodifica i due bit alti e abilita uno degli altri quattro; quello abilitato decodifica i due bit bassi, che vanno a tutti e quattro, e attiva una delle sue quattro linee. Cinque decodificatori piccoli selezionano una linea su sedici. Per questo esistevano componenti standard: l'SN74139 contiene due decodificatori da 2 a 4 con abilitazione in un solo contenitore. (Nel componente reale l'abilitazione e le uscite sono attive basse, un dettaglio qui omesso.)

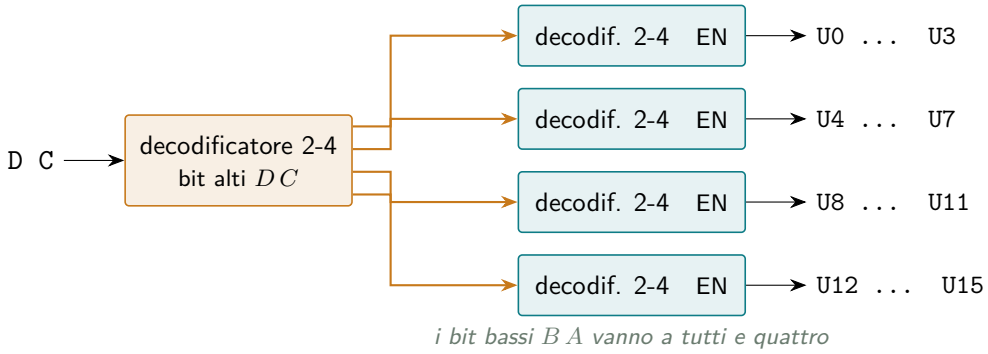


Figura 11.3. Composizione modulare: un decodificatore da 4 a 16 con cinque decodificatori da 2 a 4.

11.4.4 Nel processore: quale registro viene scritto?

Ritorno alla macchina

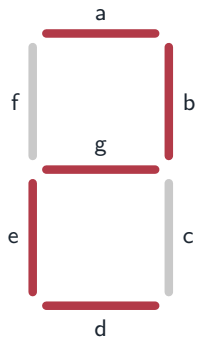
L'istruzione `addi t1, t1, 1`, il numero `0x00130313` del Capitolo 2, contiene il campo `rd` = `00110`: una parola codice di cinque bit, perché i registri sono trentadue e $\lceil \log_2 32 \rceil = 5$. Un decodificatore da 5 a 32 la trasforma in trentadue linee di *abilitazione alla scrittura*, delle quali è attiva solo quella di `x6`, cioè `t1`: solo quel registro riceve il risultato. L'abilitazione alla scrittura di `x0` semplicemente non è collegata, e `x0` si legge sempre zero. Altre due selezioni, su `rs1` e `rs2`, scelgono quali registri vengono letti: le fanno dei multiplexer.

11.4.5 Il codificatore

Il *codificatore* è la macchina inversa: riceve una rappresentazione decodificata, N linee di cui una attiva, e dà in uscita la corrispondente parola codice di $m = \lceil \log_2 N \rceil$ bit. Ogni bit di uscita è l'OR delle linee di ingresso il cui numero ha un 1 in quella posizione; per $N = 4$, $C_0 = U_1 + U_3$ e $C_1 = U_2 + U_3$. Se più linee possono essere attive insieme, serve un *codificatore di priorità*, che dà in uscita il numero della linea attiva con la priorità più alta. Una tastiera con un filo per tasto diventa, in questo modo, un numero di tasto compatto; e i codificatori di priorità ricompariranno con le interruzioni (Capitolo 19), dove più dispositivi chiedono attenzione e un solo numero raggiunge il processore.

11.4.6 Transcodificatori: il display a sette segmenti

Un *transcodificatore* trasforma ogni parola di un codice C_1 nella corrispondente parola di un codice C_2 definito sullo stesso insieme. L'esempio classico passa dal BCD alle sette linee che accendono i segmenti da a a g di un display numerico (Figura 11.4); ogni segmento è un LED comandato da un segnale digitale. Il transcodificatore ha sette uscite: sette funzioni di quattro variabili da sintetizzare, tutte con le stesse sei condizioni di indifferenza.



cifra	BCD	a	b	c	d	e	f	g
0	0000	1	1	1	1	1	1	0
1	0001	0	1	1	0	0	0	0
2	0010	1	1	0	1	1	0	1
3	0011	1	1	1	1	0	0	1
4	0100	0	1	1	0	0	1	1
5	0101	1	0	1	1	0	1	1
6	0110	1	0	1	1	1	1	1
7	0111	1	1	1	0	0	0	0
8	1000	1	1	1	1	1	1	1
9	1001	1	1	1	1	0	1	1

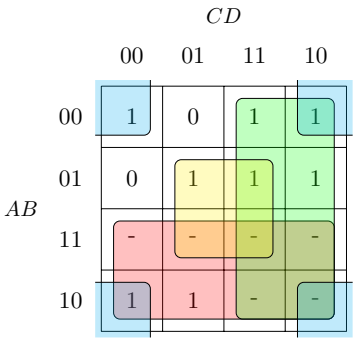


Figura 11.4. Il transcodificatore da BCD a sette segmenti: la cifra 2 accende a, b, g, e e d. A destra: la mappa del segmento a, con la cifra BCD su $A\,B\,C\,D$ e A più significativo.

Esempio 11.1 (Sintesi del segmento a)

Il segmento a è spento solo per le cifre 1 e 4. Le parole da 1010 a 1111 non si presentano mai, e sono segnate come condizioni di indifferenza nella mappa della Figura 11.4. Usando quelle celle come 1 i gruppi diventano grandi: le due righe inferiori danno A; le due colonne di destra danno C; il quadrato centrale dà BD; i quattro angoli danno $\overline{B}\,\overline{D}$:

$$a = A + C + B\,D + \overline{B}\,\overline{D}.$$

Senza le condizioni di indifferenza i gruppi sarebbero più piccoli e l'espressione più lunga. Come verifica, la cifra 0 (`0000`) è coperta dagli angoli e la cifra 9 (`1001`) da A. Altre sei mappe danno i segmenti da b a g, e un transcodificatore completo.

11.5 Multiplexer e demultiplexer

11.5.1 Il multiplexer

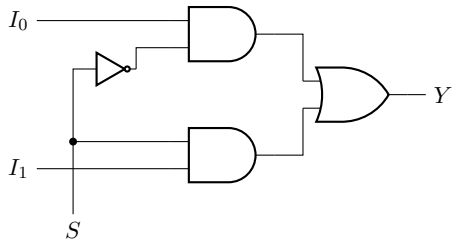
Definizione 11.6 (Multiplexer)

Un *multiplexer* è un circuito combinatorio con 2^n ingressi dati, n linee di selezione e un'uscita. Seleziona uno dei 2^n segnali di ingresso e lo trasferisce all'uscita.

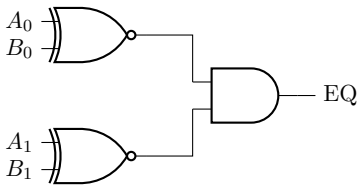
Ogni ingresso dati è moltiplicato per un mintermine delle linee di selezione, e un OR raccoglie i risultati (Figura 11.5a):

$$\begin{aligned} &\text{2 ingressi:} \quad Y = \overline{S}\, I_0 + S\, I_1, \\ &\text{4 ingressi:} \quad Y = \overline{S_1}\,\overline{S_0}\, I_0 + \overline{S_1}\, S_0\, I_1 + S_1\, \overline{S_0}\, I_2 + S_1\, S_0\, I_3. \end{aligned}$$

In altre parole, un decodificatore sulle linee di selezione abilita una porta AND: il dato è scelto da un *indirizzo*, non da una linea decodificata. Un multiplexer può anche realizzare qualsiasi funzione delle sue variabili di selezione: basta mettere la tabella di verità sugli ingressi dati.



(a) Un multiplexer a due ingressi



(b) Un comparatore per parole di due bit

Figura 11.5. Due macchine costruite con le porte.

11.5.2 Il demultiplexer

Il *demultiplexer* è la macchina inversa: un ingresso dati D, n linee di selezione, 2^n uscite. Instrada l'ingresso verso l'uscita scelta dalle linee di selezione, e le altre uscite restano a 0:

$$F_i = D \cdot U_i,$$

dove U_i sono le uscite di un decodificatore sulle linee di selezione. Con D fisso a 1 un demultiplexer è esattamente un decodificatore, in cui D fa la parte dell'abilitazione. Un multiplexer seguito da un demultiplexer porta molti segnali su un solo filo, uno alla volta: l'idea che sta dietro ogni linea condivisa.

11.5.3 Nel processore: registro o costante?

Ritorno alla macchina

Il secondo operando della ALU è o il valore del registro `rs2` o la costante scritta nell'istruzione, estesa in segno (Figura 11.6). Un multiplexer a due ingressi, largo trentadue bit, sceglie: la sua linea di selezione vale 0 per `add` e 1 per `addi`. Nel datapath di Patterson e Hennessy (2020) il segnale si chiama `ALUSrc`. Una porta applicata a ogni bit di una parola sotto un controllo comune si chiama *porta di parola*

(ingl. *word gate*).

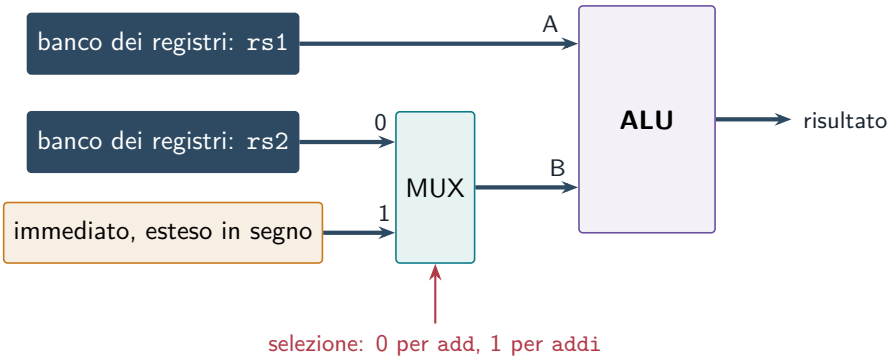


Figura 11.6. Un multiplexer sceglie il secondo operando della ALU.

11.6 Confrontare e controllare

11.6.1 Il comparatore

Un *comparatore* ha due dati A e B in ingresso e un'uscita booleana EQ che vale 1 quando $A = B$. Due bit sono uguali quando il loro XNOR vale 1; due parole sono uguali quando tutte le loro coppie di bit sono uguali, un AND di XNOR (Figura 11.5b):

$$\mathrm{EQ} = \overline{(A_0 \oplus B_0)} \cdot \overline{(A_1 \oplus B_1)} \cdots$$

Nel processore queste sono `beq` e `bne`: i due registri sono confrontati bit per bit, e il salto viene preso o no secondo EQ. Molti datapath riusano invece la ALU: sottraggono e verificano se il risultato è zero. I confronti d'ordine (`blt`, `slt`) richiedono la sottrazione e le regole di segno del Capitolo 5.

11.6.2 Parità

Quando i dati viaggiano da una sorgente a una destinazione, i codici ridondanti permettono di controllare gli errori. Il controllo più semplice è un solo *bit di parità*: le parole codice lecite hanno, per esempio, un numero pari di bit a 1.

Definizione 11.7 (Rete di parità)

Una *rete di parità* calcola la parità degli n bit al suo ingresso,

$$p = x_0 \oplus x_1 \oplus \cdots \oplus x_{n-1},$$

che vale 1 quando il numero degli 1 è dispari.

XOR è la somma modulo 2, ed è per questo che una catena di XOR conta gli 1 modulo 2. La rete serve due volte (Figura 11.7): alla sorgente, per calcolare il bit da aggiungere a un codice non ridondante, e alla destinazione, per verificare all'arrivo la parità complessiva del codice ridondante.

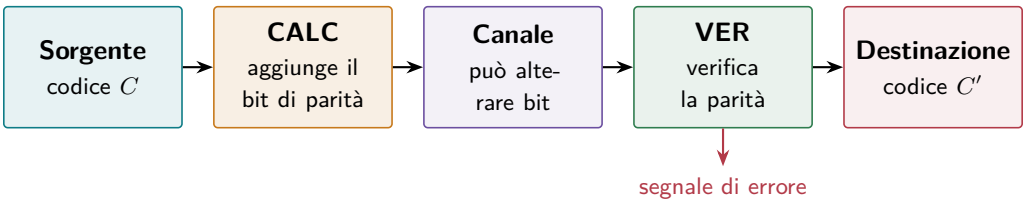


Figura 11.7. Trasmissione con un controllo di parità.

Teorema 11.2 (Potere di un bit di parità)

Un solo bit di parità rileva ogni alterazione di un numero dispari di bit. Non rileva l'alterazione di un numero pari di bit, e non può correggere alcun errore.

Dimostrazione. Ogni bit alterato cambia di uno il numero degli 1, e quindi inverte la parità: un numero dispari di inversioni la cambia, un numero pari la lascia com'era. E il controllo dà un solo bit di informazione, giusto o sbagliato, che non basta a dire *quale* bit è stato alterato. □

Il codice C' che raggiunge la destinazione è dunque *probabilmente* uguale a C se non è stato segnalato alcun errore. Codici più forti aggiungono più bit di controllo: i codici di Hamming (1950), con più bit di parità su gruppi di bit che si sovrappongono, localizzano e correggono un errore singolo.

11.6.3 Una rete di parità in sette istruzioni

```
# a0 = one byte        result: a0 = 1 if the number of 1 bits is odd

    srli t0, a0, 4
    xor  a0, a0, t0      # fold eight bits onto four
    srli t0, a0, 2
    xor  a0, a0, t0      # fold four bits onto two
    srli t0, a0, 1
    xor  a0, a0, t0      # fold two bits onto one
    andi a0, a0, 1       # keep the parity bit
```

Listato 11.2. La parità di un byte.

Ogni `xor` del Listato 11.2 combina la metà superiore dei bit con la metà inferiore: tre livelli per otto bit, la versione software di un albero di sette porte XOR. Il valore calcolato, aggiunto al byte, ne rende pari la parità totale. Tracciatelo con `0x53` $= 0101\,0011_2$, che ha quattro 1 e dà 0, e con `0x13` $= 0001\,0011_2$, che ne ha tre e dà 1.

11.7 L'unità aritmetico-logica

Tutti i blocchi di questo capitolo lavorano insieme in una sola macchina.

Definizione 11.8 (Unità aritmetico-logica)

L'*unità aritmetico-logica* (ALU) è una macchina combinatoria che riceve due operandi e un codice di operazione, e produce il risultato dell'operazione selezionata.

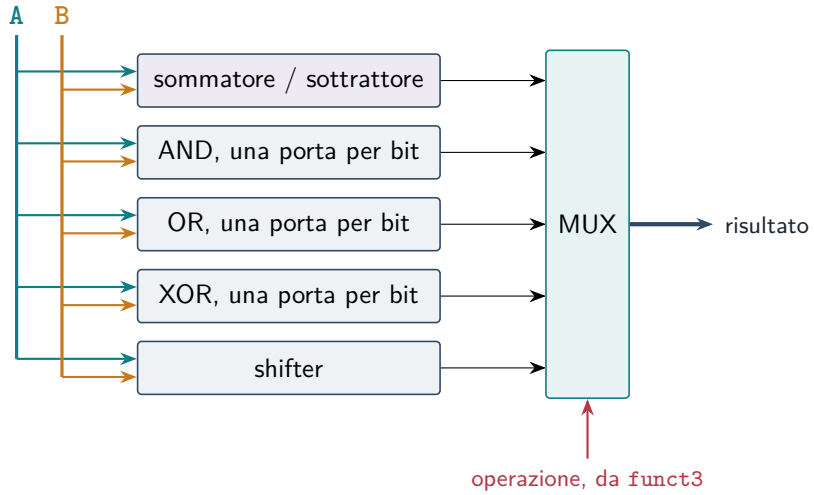


Figura 11.8. La ALU è un insieme di macchine, tutte al lavoro sugli stessi due operandi di 32 bit, e un multiplexer che sceglie un risultato.

La struttura della Figura 11.8 è di una semplicità disarmante: *ogni* unità calcola il suo risultato a ogni istruzione, e un multiplexer ne lascia passare uno. Il sommatore è il sommatore a propagazione del riporto del Capitolo 9, con il trucco del complemento e del riporto entrante del Capitolo 5 per la sottrazione; i confronti (`slt`) riusano il sottrattore.

La selezione è scritta nell'istruzione (Tabella 11.2): il campo `funct3`, bit da 14 a 12, è un codice compatto di tre bit, otto parole codice, che un piccolo decodificatore trasforma nelle linee di selezione del multiplexer della ALU.

Tabella 11.2. Le istruzioni RISC-V e la ALU.

Istruzione	funct3	Operazione della ALU	Unità che produce il risultato
`add`, `sub`	`000`	somma o differenza	sommatore / sottrattore (sceglie un altro bit, in `funct7`)
`sll`	`001`	scorrimento a sinistra	shifter
`slt`, `sltu`	`010`, `011`	pone a 1 se minore	sottrattore, poi le regole di segno
`xor`	`100`	XOR bit a bit	porte XOR
`srl`, `sra`	`101`	scorrimento a destra	shifter (`funct7` sceglie logico o aritmetico)
`or`	`110`	OR bit a bit	porte OR
`and`	`111`	AND bit a bit	porte AND

11.8 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l11.S` nel repository del corso, stampato per intero nel Listato 11.3. I test automatici del materiale del cor-

so lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 11.9 è il frame scritto dal test.

Sullo schermo.
Quattro barre orizzontali, rossa, verde, blu e gialla dall'alto: la barra k è disegnata nel colore che il selettore k sceglie nella tabella.

Che cosa premere.
Niente: il programma va da solo.



Figura 11.9. `l11.S` su PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 11: Combinational Machines
# l11.S: a multiplexer in software: a selector chooses one of four colours
#
# On the screen: four horizontal bars, red, green, blue and yellow from the top
    : bar k
# is drawn in the colour that the selector k chooses in the table.
# Build: python3 -m prg32 cartridge build l11.S --portable \
#            --entry-prefix l11 --name l11 --out l11.prg32
.option norelax

.section .text
.global l11_init
.global l11_update
.global l11_draw

l11_init:
    ret

l11_update:
    ret

l11_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   a0, 0
    call prg32_gfx_clear
    li   s0, 0                # the selector, 0 to 3
bars:
    mv   a0, s0               # a0 = selector, 0 to 3
    la   t0, palette          # four words: the data inputs
```

```
    slli t1, a0, 2          # selector * 4 bytes
    add  t0, t0, t1         # address of the selected word
    lw   a4, 0(t0)          # the selected input reaches the output
    li   a0, 20
    slli a1, s0, 5
    addi a1, a1, 20         # y = 20 + 32 * selector
    li   a2, 200
    li   a3, 16
    call prg32_gfx_rect
    addi s0, s0, 1
    li   t0, 4
    blt  s0, t0, bars
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

.section .rodata
# red, green, blue, yellow
palette: .word 0xf800, 0x07e0, 0x001f, 0xffe0
```

Listato 11.3. Il programma completo del capitolo, `code/l11.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 11.4; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l11.S --portable \
    --entry-prefix l11 --name l11 --out l11.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l11.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 11.4. Compilare `l11.S` ed eseguirlo su PRG32-QT.

11.9 Verificate di aver capito

D11.1 Quanti bit richiede un codice binario a lunghezza fissa per le 26 lettere dell'alfabeto?

A. 4
B. 5
C. 6
D. 26

D11.2 Un decodificatore ha 3 linee di ingresso e nessuna abilitazione. Quante uscite ha, e quante valgono 1 in ogni istante?

A. 3 uscite, una a 1
B. 6 uscite, due a 1
C. 8 uscite, una a 1
D. 8 uscite, tre a 1

D11.3 Di quante linee di selezione ha bisogno un multiplexer con 8 ingressi dati?

A. 2
B. 3
C. 4
D. 8

D11.4 Una parola con un bit di parità pari viene ricevuta con due bit alterati. Il controllo di parità:

A. rileva l'errore e lo corregge
B. rileva l'errore ma non può correggerlo
C. non rileva l'errore
D. segnala un errore solo se entrambi i bit valevano 1

11.10 Esercizi

Esercizio 11.1 (prima su carta, poi su PRG32)

1. Sintetizzate il segmento g del transcodificatore a sette segmenti con una mappa di Karnaugh, usando le condizioni di indifferenza.
2. Costruite un decodificatore da 3 a 8 con due decodificatori da 2 a 4 con abilitazione e un invertitore.
3. Realizzate $F = \sum m(1,2,4,7)$ con un multiplexer a 8 ingressi. Quale funzione è?
4. Scrivete un comparatore in assembly: `a0` $= 1$ se `a0` e `a1` sono uguali, usando solo `xor` e `sltiu`.
5. Disegnate una cifra a sette segmenti su PRG32: una tabella di dieci byte, uno per cifra, e sette rettangoli accesi esaminandone i bit.

Risultato atteso. Il terzo compito mette 0, 1, 1, 0, 1, 0, 0, 1 sugli ingressi dati: è lo XOR di tre ingressi, il bit di somma del sommatore completo. Il quarto è `xor a0, a0, a1` seguita da `sltiu a0, a0, 1` (la pseudoistruzione `seqz`). Su PRG32: una grande cifra che conta da 0 a 9, un passo al secondo, disegnata con sette rettangoli; è un transcodificatore software seguito da sette test di bit, e la sua tabella è la tabella di verità della Figura 11.4, una riga per byte.

11.11 Glossario

Inglese	Italiano
combinational network	rete combinatoria
propagation delay	ritardo di propagazione
cardinality	cardinalità
code word	parola codice
fixed / variable length	lunghezza fissa / variabile
redundant code	codice ridondante
decoder / encoder	decodificatore / codificatore
transcoder	transcodificatore
multiplexer / demultiplexer	multiplexer / demultiplexer
enable	abilitazione
comparator	comparatore
parity bit	bit di parità

11.12 Conclusioni e letture

L'informazione è un attributo, un tipo e un valore; un codice scrive il valore come una stringa su un alfabeto. I codici a lunghezza fissa richiedono $\lceil \log_2 N \rceil$ bit; i codici ridondanti spendono di più per rilevare gli errori. Codifica, decodifica e transcodifica sono svolte da macchine combinatorie. I multiplexer selezionano una sorgente, i demultiplexer una destinazione; comparatori e reti di parità controllano. Un processore RISC-V le usa tutte: decodificatori sui campi dell'istruzione, multiplexer sugli operandi, e una ALU che è essa stessa costruita con questi blocchi.

Ciò che qui manca è la memoria. Prima delle macchine sequenziali del Capitolo 13, il prossimo capitolo mette al lavoro la logica di questa parte nella seconda parte di Verm1.

Letture. Patterson e Hennessy (2020), appendice A e paragrafo 4.1; Bucci (2017), i capitoli sulle reti combinatorie; Stallings (2021), il capitolo sulla logica digitale.

CAPITOLO 12

Verm1, parte 2: la guida, il cibo e un punteggio

"Un gioco è una serie di scelte interessanti."
— Sid Meier

In questo capitolo

Alla fine di questo capitolo avrete

- la testa di un verme che guidate con il joystick, in quattro direzioni;
- una regola che vieta l'inversione a U, scritta come uno XOR e un confronto;
- il movimento preso da due piccole tabelle invece che da quattro salti;
- il cibo che compare in una cella casuale, viene mangiato, e ricompare;
- un punteggio, mostrato come tacche nella riga del titolo, e un suono quando il verme mangia.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI le maschere di bit; i codici; gli array in memoria; gli ingressi con pull-up; l'algebra di Boole; la virgola fissa; i multiplexer; istruzioni con o senza segno; comparatori e parità; gli argomenti in a0 ... a7.

DA UTILIZZARE OGGI la convenzione di chiamata (Capitolo 18) e, usati senza essere studiati nel corso, gli strumenti di PRG32, le chiamate del runtime PRG32, il debugger di PRG32-QT.

DA COMPRENDERE PIÙ AVANTI niente.

In questo secondo tutorial il giocatore entra nel gioco. Ogni idea usata è già stata vista come circuito nei Capitoli da 8 a 11; qui le stesse idee sono istruzioni (Tabella 12.1). Il file di riferimento è `verm1_part2.S` (Appendice D).

12.1 Partite da una parte 1 che funziona

Aprite il vostro `verm1.S` della parte 1, compilatelo ed eseguitelo con i due comandi del Listato 7.1: la testa deve attraversare lo schermo. Se non lo fa, copiate il file di riferimento `verm1_part1.S` e partite da quello. Il file viene modificato in tre passi e viene eseguito dopo

Tabella 12.1. Che cosa la parte 2 prende dai capitoli sulla logica.

Da	Idea	Dove compare
Capitolo 8	AND come maschera: conservare un bit, azzerare gli altri	un pulsante nella parola dei pulsanti
Capitolo 8	tabelle di verità: un codice per ogni caso	le quattro direzioni in due bit
Capitolo 9	XOR vale 1 dove due bit sono diversi	la regola dell'inversione a U; il test di uguaglianza
Capitolo 10	un pulsante è un interruttore verso massa con un pull-up	perché «premuto» arriva come 1 solo dopo che il runtime lo ha invertito
Capitolo 11	comparatore, multiplexer, ROM come tabella	la testa sul cibo; `dx` e `dy`

il secondo e dopo il terzo. I listati segnati **modificato** sostituiscono codice che il file già contiene; i listati segnati **nuovo** aggiungono codice. Tenete una copia di ogni parte finita, e tenete aperto il debugger di PRG32-QT: il passo 4 lo usa per guardare una decisione mentre viene presa.

12.2 Passo 1: nomi per i bit

12.2.1 Costanti: pulsanti e direzioni

Aggiungete alle costanti della parte 1 le righe di `RED` e quelle da `BTN_LEFT` a `DIR_UP` (Listato 12.1). C'è un colore nuovo, il rosso, per il cibo. Ogni pulsante è un bit di una parola, e una maschera lo seleziona. Una direzione è un numero da 0 a 3: due bit.

```
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
.equ GREEN, 0x07e0
.equ RED,   0xf800
.equ CELL,  8               # a cell is 8 x 8 pixels
.equ COLS,  40
.equ ROWS,  24
.equ TOP,   8
.equ SPEED_Q8, 40           # 40/256 of a cell per frame
.equ BTN_LEFT,  0x01        # one bit per button
.equ BTN_RIGHT, 0x02
.equ BTN_UP,    0x04
.equ BTN_DOWN,  0x08
.equ DIR_RIGHT, 0           # a direction is a 2-bit code:
.equ DIR_DOWN,  1           #   bit 0 = vertical
.equ DIR_LEFT,  2           #   bit 1 = backwards
.equ DIR_UP,    3

.section .text
.global verm1_init
.global verm1_update
.global verm1_draw
```

Listato 12.1. Passo 1 **(modificato)**: le costanti.

12.2.2 Un codice scelto con cura

I codici delle quattro direzioni non sono arbitrari (Tabella 12.2). Il bit 0 dice «verticale» e il bit 1 dice «all'indietro», così che una direzione e la sua opposta differiscono nel bit 1 e solo lì: lo XOR di un codice con il suo opposto è sempre $10_2 = 2$.

Tabella 12.2. Le quattro direzioni. Sullo schermo y cresce verso il basso: DOWN è +1, UP è −1.

Direzione	Codice	Bit 1	Bit 0	dx	dy	Opposta
RIGHT	0	0	0	+1	0	LEFT = 2
DOWN	1	0	1	0	+1	UP = 3
LEFT	2	1	0	−1	0	RIGHT = 0
UP	3	1	1	0	−1	DOWN = 1

Concetto

Una buona codifica trasforma una regola in una sola porta. È la lezione del Capitolo 9, e vale più di qualsiasi trucco di programmazione.

12.2.3 Dati: due tabelle e cinque nuove variabili

Sostituite i dati della parte 1 con il Listato 12.2. Le tabelle `dx` e `dy` hanno quattro parole ciascuna, una per direzione; sono costanti, e perciò stanno in `.rodata`. La variabile `dir` è la direzione dell'ultimo passo, `want` quella chiesta dal giocatore; `score`, `food_x` e `food_y` si usano nel passo 3.

```
.section .rodata
title:   .asciz "VERM1"
dx:      .word  1, 0, -1,  0    # right, down, left, up
dy:      .word  0, 1,  0, -1

.section .data
head_x:  .word 5
head_y:  .word 12
dir:     .word 0                # direction of the last step
want:    .word 0                # direction asked by the player
step_q8: .word 0
score:   .word 0
food_x:  .word 20
food_y:  .word 12
```

Listato 12.2. Passo 1 **(modificato)**: i dati.

12.3 Passo 2: la guida

Il piano è: leggere i pulsanti, rifiutare l'inversione a U, cercare il movimento in una tabella.

12.3.1 Dal dito al bit

Il piedino di un pulsante legge 0 quando il pulsante è premuto: è attivo basso (Capitolo 10). La funzione `prg32_input_read` inverte e impacchetta: una parola, un bit per pulsante, con

1 = premuto. **UP** e **RIGHT** insieme danno `0x04` + `0x02` = `0x06`, e `a0` · `0x04` vale zero se e solo se **UP** non è premuto. La cartuccia non vede mai il livello elettrico: il runtime dà la stessa parola sulla scheda, su QEMU e su PRG32-QT.

12.3.2 `update`: dai pulsanti a un desiderio

```
verm1_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    call prg32_input_read       # a0 = one bit per button
    la   t0, dir
    lw   t1, 0(t0)              # t1 = direction of the last step
    la   t0, want
    lw   t2, 0(t0)              # t2 = wanted direction, so far
    andi t3, a0, BTN_RIGHT
    beqz t3, 1f
    li   t2, DIR_RIGHT
1:  andi t3, a0, BTN_DOWN
    beqz t3, 2f
    li   t2, DIR_DOWN
2:  andi t3, a0, BTN_LEFT
    beqz t3, 3f
    li   t2, DIR_LEFT
3:  andi t3, a0, BTN_UP
    beqz t3, 4f
    li   t2, DIR_UP
4:  xor  t3, t2, t1             # opposite directions differ in bit 1 only
    li   t4, 2
    beq  t3, t4, 5f             # a U-turn: ignore the request
    sw   t2, 0(t0)              # remember it for the next step
5:
```

Listato 12.3. Passo 2 **(modificato)**: l'inizio di `verm1_update`.

Il Listato 12.3 sostituisce l'inizio di `verm1_update`, che ora chiama una funzione e quindi ha bisogno di uno stack frame. `andi` conserva un bit, e zero significa «non premuto». Ogni pulsante premuto sovrascrive il desiderio in `t2`; con due pulsanti premuti vince l'ultimo test, così che **UP** batte **LEFT**, che batte **DOWN**, che batte **RIGHT**. Poi viene la regola: lo XOR del desiderio e della direzione corrente vale 2 esattamente quando sono opposti, e in quel caso il desiderio non viene memorizzato e sopravvive quello vecchio.

Il desiderio è memorizzato in `want` e diventa la direzione solo al passo successivo. In questo modo due pressioni rapide tra un passo e il successivo non possono ripiegare il verme su sé stesso.

La regola è uno XOR e un confronto con la costante 2 (Tabella 12.3). Completate le sedici righe per esercizio e contate i rifiuti: sono quattro, uno per direzione. Senza la codifica accurata la regola richiederebbe quattro confronti.

12.3.3 `update`: il timer chiama `step`

```
    la   t0, step_q8
    lw   t1, 0(t0)
    addi t1, t1, SPEED_Q8       # add a fraction of a cell
```

Tabella 12.3. La regola dell'inversione a U come tabella di verità: sei delle sedici righe.

dir	want	dir ⊕ want	= 2?	Decisione
0 RIGHT	0 RIGHT	00	no	prosegue
0 RIGHT	1 DOWN	01	no	gira
0 RIGHT	2 LEFT	10	sì	rifiuta: inversione a U
0 RIGHT	3 UP	11	no	gira
1 DOWN	3 UP	10	sì	rifiuta: inversione a U
3 UP	2 LEFT	01	no	gira

```
4       li   t2, 256
5       blt  t1, t2, 6f            # less than one whole cell: wait
6       sub  t1, t1, t2
7       sw   t1, 0(t0)
8       call step                  # one whole cell: move
9       j    7f
10  6:  sw   t1, 0(t0)
11  7:  lw   ra, 12(sp)
12      addi sp, sp, 16
13      ret
```

Listato 12.4. Passo 2 **(modificato)**: il resto di `verm1_update`.

Il timer in virgola fissa della parte 1 è invariato (Listato 12.4). Le righe che spostavano `head_x` sono sparite: muoversi è ora una funzione a sé, `step`. Le etichette locali sono 6 e 7 perché i numeri da 1 a 5 sono usati sopra.

12.3.4 `step`: cercare il movimento

```
1   step:
2       addi sp, sp, -16
3       sw   ra, 12(sp)
4       la   t0, want
5       lw   t1, 0(t0)
6       la   t0, dir
7       sw   t1, 0(t0)             # the wanted direction is now the direction
8       slli t1, t1, 2             # table offset = direction * 4 bytes
9       la   t0, dx
10      add  t0, t0, t1
11      lw   t2, 0(t0)             # t2 = dx[direction]: -1, 0 or +1
12      la   t0, dy
13      add  t0, t0, t1
14      lw   t3, 0(t0)             # t3 = dy[direction]
```

Listato 12.5. Passo 2 **(nuovo)**: `step`, prima metà.

Il desiderio diventa la direzione, una volta per passo (Listato 12.5). Poi la direzione seleziona il movimento: una parola è di 4 byte, quindi lo scostamento è il codice fatto scorrere a sinistra di 2, e l'indirizzo della tabella più lo scostamento è l'indirizzo della parola da caricare. Non c'è alcun salto: il codice seleziona, come in un multiplexer. Una tabella in memoria di sola

lettura indirizzata da un codice è una ROM (Capitolo 16). L'alternativa sono quattro salti; la tabella è più corta, e cambiare le regole significa cambiare dati, non codice.

12.3.5 `step`: muoversi e rientrare, nei due versi

```
    la   t0, head_x
    lw   t4, 0(t0)
    add  t4, t4, t2
    bgez t4, 1f
    li   t4, COLS-1             # off the left edge: wrap
1:  li   t5, COLS
    blt  t4, t5, 2f
    li   t4, 0                  # off the right edge: wrap
2:  sw   t4, 0(t0)              # t4 = new column
    la   t0, head_y
    lw   t6, 0(t0)
    add  t6, t6, t3
    bgez t6, 3f
    li   t6, ROWS-1
3:  li   t5, ROWS
    blt  t6, t5, 4f
    li   t6, 0
4:  sw   t6, 0(t0)              # t6 = new row
```

Listato 12.6. Passo 2 **(nuovo)**: `step`, seconda metà.

Nella parte 1 esisteva solo il bordo destro. Ora `dx` può essere -1 e la colonna può scendere sotto 0: una colonna negativa è un numero con segno, e `bgez` e `blt` sono i confronti con segno del Capitolo 5 (Listato 12.6). Lo stesso si fa per la riga, con `ROWS`. I registri `t4` e `t6` conservano la nuova cella per il passo 3.

Per il momento la funzione si chiude con le tre righe del Listato 12.7, che ripristinano `ra` e ritornano. Il passo 3 metterà prima di esse il test per il cibo.

```
    lw   ra, 12(sp)             # temporary end of step:
    addi sp, sp, 16             # Step 3 will replace these three lines
    ret
```

Listato 12.7. Passo 2: una fine provvisoria per `step`.

Eseguite: state guidando

1. Compilate ed eseguite: la testa parte ancora verso destra, dalla colonna 5 della riga 12.
2. Tenete premuto **DOWN**: la testa gira al suo passo successivo e scende, rientrando dalla riga 23 alla riga 0.
3. Mentre va a destra, tenete premuto **LEFT**: non succede niente. L'inversione a U è rifiutata.
4. Tenete premuto **DOWN** e poi **LEFT**: due svolte, e ora la testa va a sinistra.

Risultato atteso. Tenere premuto **DOWN** dall'inizio per 64 frame porta la testa in colonna 5, riga 22: dieci passi, tutti verso il basso. Tenere premuto **LEFT** per 64 frame

la porta in colonna 15, riga 12, come se non fosse stato premuto niente. (Entrambi i casi sono stati verificati sull'esecutore senza interfaccia di PRG32-QT.)

12.4 Passo 3: il cibo, un punteggio, un suono

12.4.1 `init`: una partenza completa

```
verm1_init:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, head_x
    li   t1, 5
    sw   t1, 0(t0)
    la   t0, head_y
    li   t1, 12
    sw   t1, 0(t0)
    la   t0, dir
    sw   zero, 0(t0)            # moving right
    la   t0, want
    sw   zero, 0(t0)
    la   t0, step_q8
    sw   zero, 0(t0)
    la   t0, score
    sw   zero, 0(t0)
    call place_food
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 12.8. Passo 3 **(modificato)**: `verm1_init`.

Ogni variabile riceve ora il suo valore di partenza (Listato 12.8). `init` chiama una funzione, per collocare il primo cibo, e quindi ha bisogno di uno stack frame.

12.4.2 `place_food`: una cella casuale

```
place_food:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0
    li   a1, COLS-1
    call prg32_random_number    # a0 = a column, 0 .. 39
    la   t0, food_x
    sw   a0, 0(t0)
    li   a0, 0
    li   a1, ROWS-1
    call prg32_random_number    # a0 = a row, 0 .. 23
    la   t0, food_y
    sw   a0, 0(t0)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 12.9. Passo 3 **(nuovo)**: `place_food`.

La funzione `prg32_random_number(min, max)` restituisce un numero compreso tra i suoi due argomenti, estremi inclusi; gli argomenti vanno in `a0` e `a1` e il risultato torna in `a0` (Listato 12.9). I numeri casuali vengono dal runtime, e sulla scheda dal generatore hardware dell'ESP32-C6: due esecuzioni del gioco sono diverse. Il cibo può cadere sulla testa; la parte 3 lo corregge.

12.4.3 `step`: la testa è sul cibo?

```
    la   t0, food_x
    lw   t1, 0(t0)
    xor  t1, t1, t4            # 0 if same column
    la   t0, food_y
    lw   t2, 0(t0)
    xor  t2, t2, t6            # 0 if same row
    or   t1, t1, t2            # 0 only if both are equal
    bnez t1, 5f                # not on the food
    la   t0, score
    lw   t1, 0(t0)
    addi t1, t1, 1
    sw   t1, 0(t0)
    call place_food
    call eat_sound
5:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 12.10. Passo 3 **(modificato)**: la fine di `step`.

Sostituite la fine provvisoria con il Listato 12.10. Un comparatore di uguaglianza è una fila di porte XOR seguita da un OR (Capitolo 11); qui sono due `xor` e una `or`: ogni `xor` vale 0 quando due numeri sono uguali, e la `or` dei due risultati vale 0 solo se entrambe le coppie sono uguali. Mangiare significa un punto in più, nuovo cibo e un suono. I registri `t4` e `t6` contengono ancora la nuova colonna e la nuova riga; dopo le chiamate non servono più, ed è per questo che tenerli in registri temporanei è sicuro.

12.4.4 `eat_sound`: una nota

```
eat_sound:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0                 # channel
    li   a1, 0                 # instrument
    li   a2, 72                # note: C5
    li   a3, 255               # volume
    li   a4, 60                # milliseconds
    call prg32_audio_note
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 12.11. Passo 3 **(nuovo)**: `eat_sound`.

La firma è

```
prg32_audio_note(canale, strumento, nota, volume, millisecondi)
```

con cinque argomenti, da `a0` ad `a4`, poi la chiamata. La nota 72 è il Do5 nella numerazione MIDI. Su PRG32-QT la nota è suonata dal computer, sulla scheda dall'amplificatore e dal suo altoparlante. Il suono viene per ultimo: il gioco funzionava anche senza.

12.4.5 `draw`: il punteggio come tacche

```
verm1_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 0
    li   a1, 0
    la   a2, title
    li   a3, WHITE
    li   a4, BLACK
    call prg32_gfx_text8
    li   s0, 0                  # s0 = i: one mark per point
1:  la   t0, score
    lw   t1, 0(t0)
    bge  s0, t1, 2f
    slli a0, s0, 2
    addi a0, a0, 56             # x = 56 + 4 * i
    li   a1, 1
    li   a2, 3
    li   a3, 6
    li   a4, WHITE
    call prg32_gfx_rect
    addi s0, s0, 1
    j    1b
2:
```

Listato 12.12. Passo 3 **(modificato)**: l'inizio di `verm1_draw`.

Il Listato 12.12 disegna un piccolo rettangolo per punto, a 4 pixel di distanza, in un ciclo che termina quando il contatore raggiunge il punteggio. Il contatore è tenuto in `s0`, perché deve sopravvivere alle chiamate: un registro `t` può essere distrutto da qualsiasi chiamata, mentre un registro `s` è preservato, a condizione che la funzione che lo usa lo salvi e lo ripristini. Quindi `s0` è salvato nel frame, con `ra`. La convenzione, incontrata nel Capitolo 2, è studiata nel Capitolo 18. Le cifre arrivano nella parte 4, quando sarà disponibile la divisione.

12.4.6 `draw`: il cibo e la testa

```
    la   t0, food_x
    lw   a0, 0(t0)
```

```
    la   t0, food_y
    lw   a1, 0(t0)
    li   a2, RED
    call draw_cell
    la   t0, head_x
    lw   a0, 0(t0)
    la   t0, head_y
    lw   a1, 0(t0)
    li   a2, GREEN
    call draw_cell
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 12.13. Passo 3 **(modificato)**: il resto di `verm1_draw`.

Le stesse tre righe compaiono due volte, per colonna, riga e colore (Listato 12.13); la conversione in pixel è lasciata a una funzione. La testa è disegnata per ultima, così che copra il cibo.

```
draw_cell:                      # a0 = column, a1 = row, a2 = colour
    addi sp, sp, -16
    sw   ra, 12(sp)
    slli a0, a0, 3              # x = column * 8
    slli a1, a1, 3
    addi a1, a1, TOP            # y = row * 8 + 8
    mv   a4, a2                 # the colour is the fifth argument
    li   a2, CELL
    li   a3, CELL
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 12.14. Passo 3 **(nuovo)**: `draw_cell`.

`draw_cell` è l'aritmetica della parte 1, scritta una volta sola (Listato 12.14). È una funzione con un contratto: `a0` = colonna, `a1` = riga, `a2` = colore. Il colore arriva in `a2` e deve uscire in `a4`: viene spostato per primo, perché `a2` sta per essere sovrascritto. Scrivere il contratto in un commento fa parte del lavoro.

Eseguite: Verm1, parte 2

Una cella rossa è da qualche parte sullo schermo. Guidateci sopra la testa: suona una nota, compare una tacca accanto a `VERM1`, il cibo si sposta (Figura 12.1).

12.5 Passo 4: osservare una decisione

In questa sessione PRG32-QT è usato come sonda logica.



Figura 12.1. La testa, il cibo, e due tacche dopo il titolo: due punti. La schermata è prodotta dai test, in cui un giocatore programmato guida la testa sul cibo due volte.

Cogliete la regola dell'inversione a U al lavoro

1. Eseguite la cartuccia in modalità debug e premete **Update**: il listato mostra `verm1_update`.
2. Guardate la telemetria mentre tenete premuto un pulsante: la maschera di ingresso mostra `0x04` per **UP**, `0x01` per **LEFT**.
3. Trovate l'istruzione `xor t3, t2, t1` e fate clic sul margine: un breakpoint.
4. Tenete premuto **LEFT** mentre la testa va a destra, e premete **Resume**: il programma si ferma sulla `xor`.
5. Leggete i registri: `t1` vale 0, la direzione; `t2` vale 2, il desiderio.
6. Premete **Step**: `t3` diventa 2. Fate altri due passi: il salto viene preso e la `sw` viene saltata.

Abbassate la velocità del debugger a 0.25x se vi serve tempo per premere.

La Tabella 12.4 elenca ciò che si legge nei registri, sempre con la testa che va a destra. La sua ultima riga mostra la priorità dei test: **UP** è esaminato per ultimo e vince. `a0` viene sovrascritto più avanti nel frame, e va letto alla prima istruzione dopo la chiamata.

Tabella 12.4. Che cosa mostrano i registri al breakpoint.

Tenete premuto	`a0`	`t1` (dir)	`t2` (want)	`t3` = `t2`⊕`t1`	Memorizzato?
niente	`0x00`	0	0	0	sì: 0
DOWN	`0x08`	0	1	1	sì: 1
LEFT	`0x01`	0	2	2	no: inversione a U
UP	`0x04`	0	3	3	sì: 3
UP + **RIGHT**	`0x06`	0	3	3	sì: 3

12.5.1 Leggere le variabili in memoria

Dopo `la t0, dir` il registro `t0` contiene l'indirizzo di `dir`: scrivetelo nel campo *Memory*. Le variabili si susseguono, 4 byte ciascuna, nell'ordine del file: dall'indirizzo di `dir`, a +0 c'è `dir`, a +4 `want`, a +8 `step_q8`, a +12 `score`, a +16 `food_x` e a +20 `food_y`; `head_x` e `head_y` sono le due parole prima di `dir`. Mangiate una volta, mettete in pausa, e leggete

`score`: `01 00 00 00`, il byte meno significativo per primo. Poi leggete `food_x` e `food_y` e confrontateli con lo schermo: sono la colonna e la riga della cella rossa.

12.6 Verificate di aver capito

D12.1 Il giocatore tiene premuti insieme **DOWN** e **RIGHT**: `prg32_input_read` restituisce

A. `0x02`
B. `0x08`
C. `0x0a`
D. `0x10`

D12.2 Il verme va verso UP (3). Quale desiderio viene rifiutato?

A. RIGHT (0)
B. DOWN (1)
C. LEFT (2)
D. UP (3)

D12.3 Perché `slli t1, t1, 2` prima di sommare all'indirizzo di `dx`?

A. per rendere positivo il codice
B. ogni elemento della tabella è di 4 byte
C. per azzerare il bit 1
D. le tabelle cominciano all'indirizzo 4

D12.4 `xor t1, t1, t4` dà zero esattamente quando

A. `t1` vale zero
B. `t4` vale zero
C. `t1` e `t4` sono uguali
D. `t1` e `t4` sono opposti

12.7 Fatelo vostro

Esercizio 12.1 (su PRG32)

1. Fate del pulsante **A** un freno: mentre è tenuto premuto, il timer non avanza.
2. Date due punti per un pezzo di cibo mangiato nella riga più in alto.
3. Disegnate la testa in un colore diverso per ogni direzione, usando una terza tabella di quattro colori.
4. Cambiate la codifica in modo che RIGHT = 0, LEFT = 1, DOWN = 2, UP = 3: che cosa diventa la regola dell'inversione a U?

Risultato atteso. Ogni modifica è di poche righe. Tenete una copia della parte 2 funzionante prima di sperimentare: la parte 3 parte da lì. (Suggerimenti: per il freno, `andi` con `BTN_A`, `0x10`, all'inizio del timer e un salto all'uscita. La tabella dei colori è fatta di quattro `.word` in `.rodata`, indicizzata come `dx`. Con la nuova codifica gli opposti differiscono nel bit 0, quindi il test diventa $\oplus = 1$.)

12.8 Dove siamo, e che cosa viene dopo

Il giocatore è nel gioco: un desiderio, una regola, un movimento, una ricompensa. Bit e maschere, XOR come comparatore, una tabella come selettore: la logica è stata trasformata in istruzioni. Ma il verme non ha un corpo, niente può andare storto, e il gioco non finisce mai. I Capitoli da 13 a 16 danno gli strumenti: macchine a stati, registri, la memoria come array, e ciò che sopravvive quando si toglie l'alimentazione. La parte 3 aggiunge un corpo che cresce, le collisioni, una schermata del titolo e una fine partita.

Parte IV

Stato e memoria

CAPITOLO 13

Macchine sequenziali

"Steve, ogni automobile ha un volante, ma nessuno prova a sostenere che il volante sia una sua invenzione."

— Bill Gates, nel film *I pirati di Silicon Valley*, 1999

In questo capitolo

Alla fine di questo capitolo saprete

- spiegare perché un circuito sequenziale ha bisogno dei concetti di tempo e di stato;
- definire una macchina a stati finiti e distinguere le macchine di Mealy dalle macchine di Moore;
- descrivere una macchina con una tabella degli stati e con un diagramma degli stati, e passare dall'una all'altro;
- spiegare come un latch SR ricorda un bit, e perché $S = R = 1$ non è ammesso;
- distinguere i latch dai flip-flop, la sensibilità al livello da quella al fronte;
- riconoscere i registri, i contatori e il program counter di RISC-V come macchine sequenziali.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI macchine a stati finiti; il clock; latch e flip-flop; i registri come flip-flop.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI niente.

13.1 Riscaldamento sulla macchina

Un gioco ha una schermata del titolo, una schermata su cui si gioca, e una schermata che dice «game over». Il Listato 13.1 è la parte della sua `update` che passa dall'una all'altra.

```
1 .equ TITLE, 0
2 .equ PLAY,  1
```

```
.equ OVER,  2

game_update:
    ...
    call prg32_input_read      # a0 = buttons: the INPUT
    la   t0, state
    lw   t1, 0(t0)             # t1 = the present STATE
    andi t2, a0, 0x10          # button A
    li   t3, TITLE
    bne  t1, t3, 1f
    beqz t2, done              # TITLE: stay until A is pressed
    li   t1, PLAY              # TITLE --A--> PLAY
    j    store
1:  li   t3, OVER
    bne  t1, t3, done          # PLAY is handled elsewhere
    beqz t2, done              # OVER: stay until A is pressed
    li   t1, TITLE             # OVER --A--> TITLE
store:
    sw   t1, 0(t0)             # the next state becomes the present state
done:
    ...
```

Listato 13.1. Un gioco è una macchina a stati.

Premere **A** avvia il gioco dalla schermata del titolo e riporta al titolo dalla schermata di fine partita: *lo stesso pulsante ha effetti diversi*, e l'effetto dipende dallo stato. Lo stato è una parola in memoria che sopravvive da un frame all'altro. `update` calcola lo stato prossimo dallo stato presente e dall'ingresso; `draw` mostra una schermata diversa per ogni stato, l'uscita.

Il programma ha un difetto di cui vale la pena discutere: tenere premuto **A** sulla schermata di fine partita porta al titolo e, un frame dopo, al gioco. Per correggerlo il programma deve ricordare il valore precedente del pulsante: un bit di stato in più. Il capitolo ci tornerà.

13.2 Tempo e stato

13.2.1 Circuiti sequenziali

In un circuito combinatorio le uscite in un dato istante dipendono solo dagli ingressi in quello stesso istante. In un circuito sequenziale dipendono dagli ingressi in quell'istante e dalla storia passata degli ingressi. La definizione implica due concetti che una macchina combinatoria non ha: il *tempo* (quando si guardano i segnali, e quando le cose cambiano?) e lo *stato* (che cosa ricorda il circuito del passato?). Un circuito sequenziale è un circuito combinatorio più qualcosa che ricorda. Analisi e sintesi valgono ancora: dalla struttura al comportamento, e da una specifica funzionale a una struttura.

13.2.2 Il tempo: dal continuo al discreto

In un intervallo qualsiasi un segnale fisico passa per infiniti istanti, e una tensione per infiniti valori. Un segnale binario viene letto, o *campionato*, in un numero finito di istanti scelti: 0 se la tensione è sotto una soglia, 1 se è sopra (il Capitolo 10 ha precisato la soglia). Tra due campioni il segnale può fare qualunque cosa: contano solo i campioni. È il campionamento e la quantizzazione del Capitolo 4, applicato ai fili del circuito stesso. In un sistema digitale

il tempo è quindi una sequenza di istanti discreti t_0, t_1, t_2, … Chi decide quando sono gli istanti? Un segnale periodico dedicato: il clock.

13.2.3 Il clock

Definizione 13.1 (Clock)

Il *clock* è un segnale binario periodico che scandisce gli istanti discreti di un sistema digitale. Il suo *periodo* T è la durata di un ciclo; la sua *frequenza* è il numero di periodi al secondo, $f = 1/T$; il suo *duty cycle* è il rapporto tra il tempo in cui è alto e il periodo. Il *fronte di salita* è il passaggio da 0 a 1, il *fronte di discesa* il passaggio da 1 a 0.

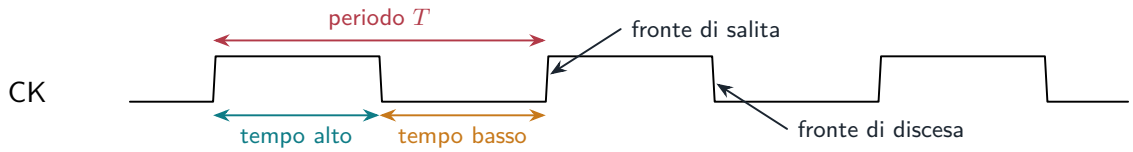


Figura 13.1. Il clock.

I circuiti sincroni cambiano stato su uno dei due fronti. L'ESP32-C6 lavora a 160 MHz: un periodo dura 6,25 nanosecondi, e ogni rete combinatoria dentro il chip deve assestarsi entro quel tempo, perché il periodo deve superare il cammino critico (Capitolo 11). In un gioco PRG32 il frame fa la parte di un clock molto più lento.

13.2.4 Il concetto di stato

Definizione 13.2 (Stato)

Lo *stato* di un circuito sequenziale è un riassunto dell'intera storia precedente dei suoi ingressi, sufficiente a determinare, insieme agli ingressi presenti, le uscite e lo stato prossimo. È tenuto da un insieme di *variabili di stato*.

Una *funzione di transizione* calcola lo stato prossimo dallo stato presente e dagli ingressi presenti, e lo stato viene aggiornato a ogni istante del tempo discreto, cioè a ogni ciclo di clock:

$$S(k) = f\big(S(k-1), I(k)\big).$$

Per memorizzare lo stato si usano elementi di memoria chiamati *bistabili*: latch e flip-flop. Nel riscaldamento c'è una sola variabile di stato, la parola `state`; la funzione di transizione è `game_update`; il clock è il frame.

13.3 Macchine a stati finiti

13.3.1 L'automa a stati finiti

Il numero di stati di una rete sequenziale che si possa costruire fisicamente è necessariamente finito: con k variabili di stato ci sono al più 2^k stati. Il modello che ne descrive il comportamento è l'automa a stati finiti.

Definizione 13.3 (Macchina sequenziale)

Una *macchina sequenziale*, o macchina a stati finiti, è una quintupla (I,U,S,Ω,w) dove
- I è l'alfabeto di ingresso e U l'alfabeto di uscita, insiemi finiti di simboli;
- S è l'insieme finito e non vuoto degli stati;
- $\Omega : S \times I \to S$ è la *funzione stato prossimo*: a ogni coppia di stato e ingresso corrisponde, se specificato, uno e un solo stato prossimo;
- w è la *funzione di uscita*, che genera il simbolo di uscita.

13.3.2 Macchine di Mealy e macchine di Moore

I due modelli classici differiscono nella funzione di uscita (Mealy, 1955; Moore, 1956).

Definizione 13.4 (Mealy e Moore)

In una *macchina di Mealy* l'uscita dipende dallo stato e dall'ingresso, $w : S \times I \to U$. In una *macchina di Moore* l'uscita è associata al solo stato, $w : S \to U$.

Nel diagramma di una macchina di Mealy l'uscita è scritta sulle transizioni; la macchina può reagire a un ingresso nello stesso ciclo, e spesso richiede meno stati. Nel diagramma di una macchina di Moore l'uscita è scritta dentro gli stati; cambia solo quando cambia lo stato, così che nessun disturbo degli ingressi la raggiunge. Ogni macchina di Mealy ha una macchina di Moore equivalente e viceversa, eventualmente con un diverso numero di stati. Il gioco del riscaldamento è una macchina di Moore: ogni stato ha la sua schermata.

13.3.3 Architettura generale di una macchina sequenziale

Ogni macchina sequenziale ha la struttura della Figura 13.2: una rete combinatoria calcola lo stato prossimo e le uscite dagli ingressi e dallo stato presente; un *registro di stato* di k flip-flop tiene lo stato presente e, a ogni fronte del clock, lo sostituisce con il prossimo. Il riscaldamento ha esattamente questa forma: `game_update` è il riquadro in alto, la parola `state` è il registro, il frame è il clock.

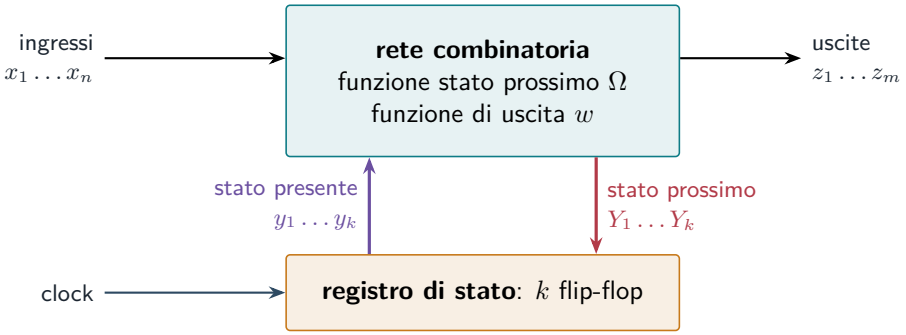


Figura 13.2. L'architettura generale di una macchina sequenziale.

13.3.4 Sintesi di una rete sequenziale

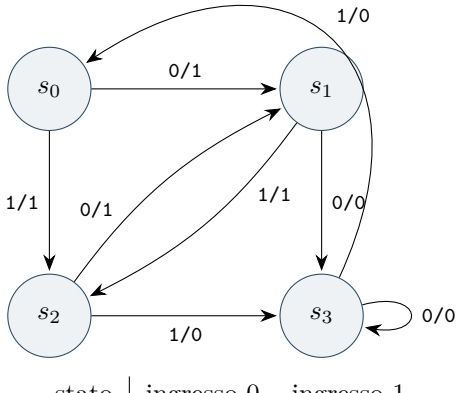
1. Definire gli insiemi I, U e S: ingressi, uscite e stati.

2. Definire la funzione stato prossimo Ω e la funzione di uscita w, con una tabella degli stati o un diagramma degli stati.
3. Assegnare un codice binario a ogni stato: k variabili di stato per un massimo di 2^k stati.
4. Sintetizzare le due reti combinatorie che realizzano Ω e w, con i metodi dei Capitoli 9 e 11.
5. Scegliere gli elementi di memoria: la rete dello stato prossimo dipende dal tipo di bistabile, la rete di uscita no.

Con i flip-flop D la rete dello stato prossimo calcola semplicemente il codice dello stato prossimo, ed è per questo che oggi sono la scelta abituale.

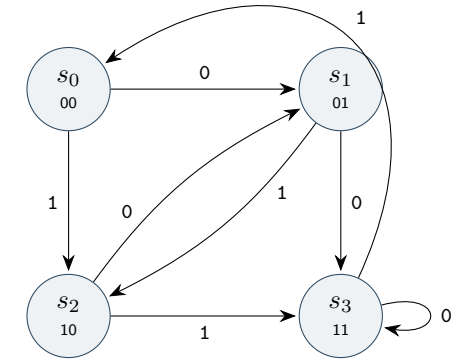
13.3.5 Tabelle degli stati e diagrammi degli stati

Una macchina si descrive in due modi equivalenti. La *tabella degli stati* ha una riga per stato presente e una colonna per simbolo di ingresso; ogni cella contiene lo stato prossimo e, per una macchina di Mealy, l'uscita. Il *diagramma degli stati* è un grafo orientato con un nodo per stato e un arco per transizione, etichettato con l'ingresso e, per una macchina di Mealy, l'uscita. La Figura 13.3 mostra le stesse transizioni come macchina di Mealy e come macchina di Moore in cui a ogni stato è associata un'uscita di due bit, qui semplicemente il codice binario dello stato.



stato	ingresso 0	ingresso 1
s_0	s_1/1	s_2/1
s_1	s_3/0	s_2/1
s_2	s_1/1	s_3/0
s_3	s_3/0	s_0/0

(a) Una macchina di Mealy: gli archi portano ingresso/uscita



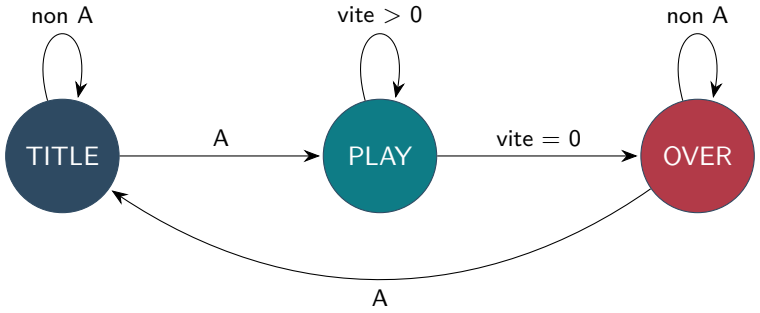
stato	ingresso 0	ingresso 1	uscita
s_0	s_1	s_2	00
s_1	s_3	s_2	01
s_2	s_1	s_3	10
s_3	s_3	s_0	11

(b) Una macchina di Moore: l'uscita è nello stato

Figura 13.3. Diagrammi degli stati e tabelle degli stati.

13.3.6 Il riscaldamento come diagramma degli stati

Il gioco del riscaldamento è la macchina di Moore della Figura 13.4. Ha tre stati: due bit di stato basterebbero, e il programma usa un'intera parola.



Stato presente	Ingresso che lo cambia	Stato prossimo	Uscita (`draw`)
`TITLE`	**A** premuto	`PLAY`	la schermata del titolo
`PLAY`	vite = 0	`OVER`	il campo di gioco
`OVER`	**A** premuto	`TITLE`	il punteggio finale

Figura 13.4. Il gioco del riscaldamento come macchina di Moore.

Concetto

Disegnare il diagramma degli stati *prima* di scrivere il codice è l'abitudine di progetto più utile in assoluto per un gioco. Il Capitolo 17 la applica a Verm1.

13.4 Latch

13.4.1 Elementi di memoria

I circuiti elettronici capaci di memorizzare un singolo bit sono essenzialmente di due tipi: latch e flip-flop. Entrambi sono *bistabili*: hanno due stati stabili, 0 e 1, che mantengono finché niente li cambia; ed entrambi hanno due uscite, Q e $\overline{Q}$, che sono sempre l'una il complemento dell'altra.

Nota storica

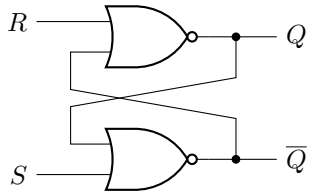
Nel 1918 Eccles e Jordan costruirono il primo flip-flop elettronico, con due valvole termoioniche. Nel 1943 lo stesso circuito era al cuore di Colossus. Poi vennero i relè, i transistor, e oggi le porte logiche su un chip. Il nome «flip-flop» viene dal suono delle versioni elettromeccaniche.

13.4.2 Il latch SR

Il *latch SR* è fatto di due porte NOR, ciascuna con l'uscita collegata a un ingresso dell'altra (Figura 13.5): una *retroazione*. L'ingresso S (*set*) porta Q nello stato stabile 1; l'ingresso R (*reset*) porta Q nello stato stabile 0. Con $S = R = 0$ il latch conserva l'ultimo valore: ricorda quale dei due ingressi è stato a 1 più di recente.

Dal circuito, $Q = \overline{R + \overline{Q}}$ e $\overline{Q} = \overline{S + Q}$, da cui

$$Q = \overline{R} \cdot (S + Q),$$



S	R	Q prossimo
0	0	Q: mantiene
0	1	0: reset
1	0	1: set
1	1	non ammesso

Figura 13.5. Il latch SR costruito con porte NOR.

dove la Q a destra è il valore di un ritardo di propagazione prima. A causa della retroazione Q dipende da sé stessa: in una rete con anelli ogni variabile è una forma d'onda nel tempo, non un singolo valore. Interrompere il filo di retroazione trasforma il latch in una semplice rete combinatoria di due NOR, ed è così che lo si analizza, caso per caso. Lo stesso latch si può costruire con due porte NAND e ingressi attivi bassi.

13.4.3 Il latch SR nel tempo

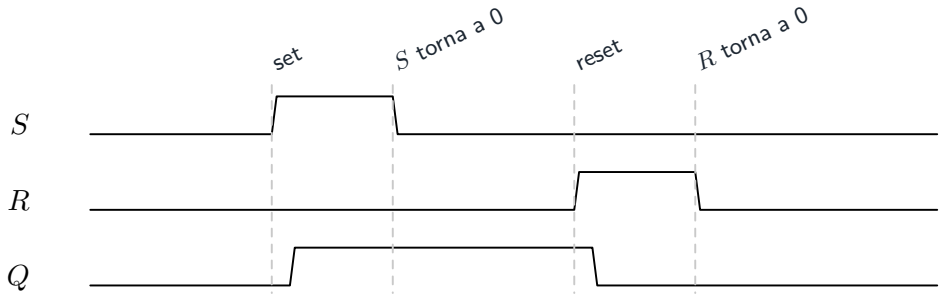


Figura 13.6. Un diagramma temporale del latch SR.

La Figura 13.6 segue il latch nel tempo. Al *set*, S va a 1 e, dopo il ritardo di due porte, Q diventa 1. Poi S torna a 0 e non cambia niente: la retroazione tiene Q a 1. *Il latch ricorda.* Al *reset*, R va a 1 e, dopo il ritardo, Q diventa 0; quando R torna a 0, Q resta lì. Il latch è un automa di Moore la cui uscita è il suo stato.

13.4.4 Perché $S = R = 1$ non è ammesso

Con entrambi gli ingressi a 1 entrambe le porte NOR danno 0: Q e $\overline{Q}$ non sono più complementari. Peggio, se poi S e R tornano a 0 insieme le due porte gareggiano, e lo stato finale dipende da quale è più veloce: l'uscita è imprevedibile, e il latch può perfino oscillare. S e R si chiamano *ingressi di eccitazione*, e il progettista deve garantire che

$$S \cdot R = 0.$$

Gli elementi JK e D esistono per eliminare questo problema.

13.4.5 Latch sincroni: aggiungere un clock

In un sistema *asincrono* le uscite possono cambiare ogni volta che cambia un ingresso. In un sistema *sincrono* l'istante in cui le uscite possono cambiare è deciso dal clock. In un latch SR con clock (Figura 13.7) S e R passano attraverso due porte AND controllate dal clock: con $CK = 0$ entrambe le porte AND danno 0 e il latch non può cambiare stato; con $CK = 1$ il latch è sensibile a S e R esattamente come quello asincrono. Il clock decide *quando gli ingressi vengono ascoltati*.

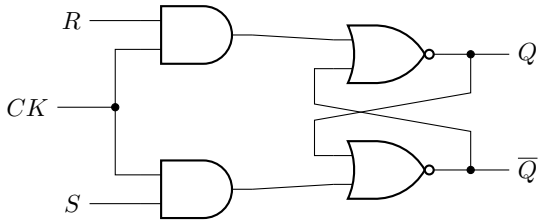


Figura 13.7. Un latch SR con clock.

13.4.6 I latch JK e D

Due varianti eliminano la combinazione proibita (Tabella 13.1). Nel *latch JK* le uscite sono riportate alle porte di ingresso, $S = J \cdot \overline{Q}$ e $R = K \cdot Q$; poiché Q e $\overline{Q}$ sono complementari, S e R non possono mai valere entrambi 1, e la quarta combinazione degli ingressi acquista un significato: *inverti* (toggle). Nel *latch D* (D per *delay*, ritardo, o *data*, dato) un solo ingresso va a S e, attraverso un invertitore, a R, così che sono possibili solo i casi $S = 0$, $R = 1$ e $S = 1$, $R = 0$: l'uscita semplicemente ripete l'ingresso mentre il clock è attivo.

Tabella 13.1. I latch JK e D.

Ingressi	Latch JK	Ingresso	Latch D
$J = 0$, $K = 0$	mantiene: Q invariato	$D = 0$	Q diventa 0
$J = 0$, $K = 1$	reset: Q diventa 0	$D = 1$	Q diventa 1
$J = 1$, $K = 0$	set: Q diventa 1		
$J = 1$, $K = 1$	toggle: Q viene invertito		

13.4.7 La sensibilità al livello e il suo problema

Un latch è *sensibile al livello*: è sensibile per tutto il tempo in cui il clock è al suo livello attivo. Mentre il clock è alto un latch è *trasparente*: ogni cambiamento degli ingressi raggiunge le uscite, così che durante un ciclo di clock sono possibili più cambiamenti di stato. In una macchina con retroazione, come quella della Figura 13.2, questo è fatale: il nuovo stato percorre l'anello e cambia di nuovo lo stato, nello stesso ciclo. Un latch JK con $J = K = 1$ continua a invertirsi finché il clock resta alto.

Concetto

Un registro di stato deve cambiare *al più una volta per ciclo*, e non deve lasciar passare lo stato prossimo mentre viene ancora calcolato. Un latch trasparente non può garantirlo.

13.5 Flip-flop

13.5.1 Il flip-flop master-slave

La soluzione è mettere due latch in fila e farli ascoltare in fasi opposte del clock (Figura 13.8). Il primo, il *master*, è attivo quando $CK = 0$: segue l'ingresso D, ma il secondo, lo *slave*, è chiuso e l'uscita non si muove. Quando il clock sale il master si chiude, congelando il valore

che D aveva in quell'istante, e lo slave si apre e lo mostra. In nessun momento i due latch sono entrambi trasparenti: l'uscita cambia una volta per ciclo, sul fronte di salita.

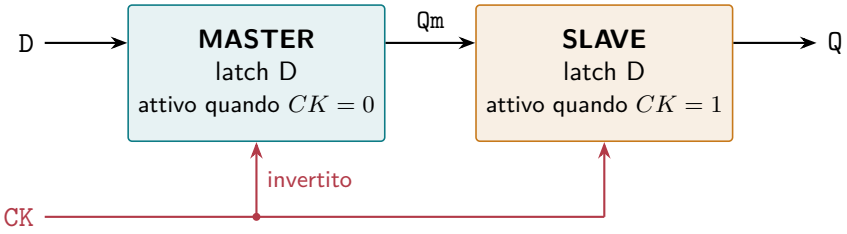


Figura 13.8. Un flip-flop D master-slave.

Definizione 13.5 (Flip-flop)

Un *flip-flop* è un bistabile *sensibile al fronte*: campiona i suoi ingressi solo su un fronte del clock, e le sue uscite cambiano solo allora.

13.5.2 Latch o flip-flop? Guardate l'uscita

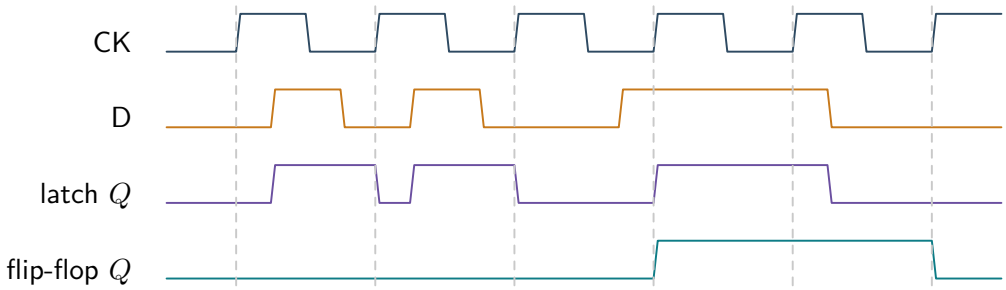


Figura 13.9. Lo stesso clock e lo stesso ingresso D, visti da un latch D e da un flip-flop D. Le linee tratteggiate sono i fronti di salita.

Nella Figura 13.9 il latch D segue D per tutto il tempo in cui il clock è alto, e mantiene l'ultimo valore quando il clock scende: la sua uscita cambia nel mezzo di una fase alta. Il flip-flop D guarda D solo sul fronte di salita: gli impulsi brevi tra due fronti sono ignorati. L'uscita del flip-flop è ferma per un intero periodo, ed è ciò che serve a un registro di stato.

13.5.3 Elementi di memoria: un riepilogo

La Tabella 13.2 dà, per ogni elemento, l'*equazione caratteristica* che esprime lo stato prossimo Q^+. Un flip-flop T è un JK con $J = K = T$. Il flip-flop D è l'elemento di memoria standard dei processori di oggi.

13.6 Ritorno alla macchina

Tabella 13.2. Elementi di memoria.

Elemento	Ingressi	Stato prossimo	Cambia quando
latch SR ($S\,R = 0$)	S, R	$Q^+ = S + \overline{R}\,Q$	ogni volta che cambia un ingresso
latch SR con clock	S, R, CK	come sopra	mentre il clock è al livello attivo
latch D	D, CK	$Q^+ = D$	mentre il clock è al livello attivo
flip-flop JK	J, K, CK	$Q^+ = J\,\overline{Q} + \overline{K}\,Q$	sul fronte del clock
flip-flop D	D, CK	$Q^+ = D$	sul fronte del clock
flip-flop T	T, CK	$Q^+ = T \oplus Q$	sul fronte del clock

13.6.1 Un registro è una fila di flip-flop

Definizione 13.6 (Registro)

Un *registro* è un dispositivo capace di memorizzare un gruppo di bit: un flip-flop per bit, e un clock comune.

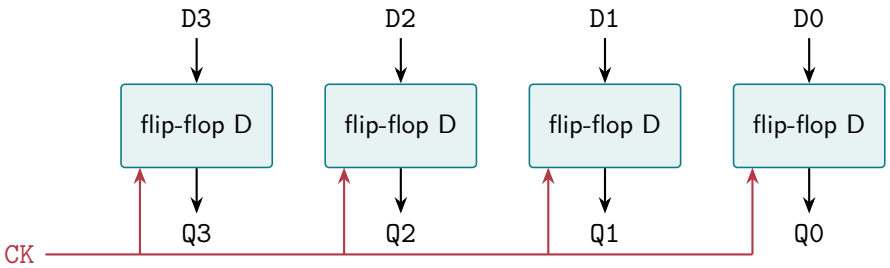


Figura 13.10. Un registro di quattro bit.

Con un segnale di *abilitazione al caricamento* il registro prende il nuovo valore solo quando il segnale è attivo, e altrimenti conserva il vecchio. I trentadue registri di RISC-V sono trentadue file di trentadue flip-flop, e le linee di abilitazione alla scrittura del decodificatore del Capitolo 11 sono le loro abilitazioni al caricamento. Il componente classico della serie 74 è il 74273: otto flip-flop D, un clock e un azzeramento. Registri, registri a scorrimento, bus e buffer a tre stati sono l'argomento del prossimo capitolo.

13.6.2 Il program counter: un registro in un anello

Il *program counter* (PC), o contatore di programma, è il registro che contiene l'indirizzo dell'istruzione in esecuzione. Il suo valore successivo è calcolato da una piccola rete combinatoria (Figura 13.11): un sommatore calcola $PC + 4$, perché le istruzioni distano 4 byte (con l'estensione compressa il sommatore aggiunge 2 dopo un'istruzione di 16 bit); un multiplexer sceglie tra quel valore e la destinazione di un salto; il comparatore del Capitolo 11 produce, per `beq` e `bne`, il segnale che pilota il multiplexer. A ogni fronte del clock il registro prende il valore scelto. È l'architettura generale della Figura 13.2 nel suo più piccolo esemplare utile.

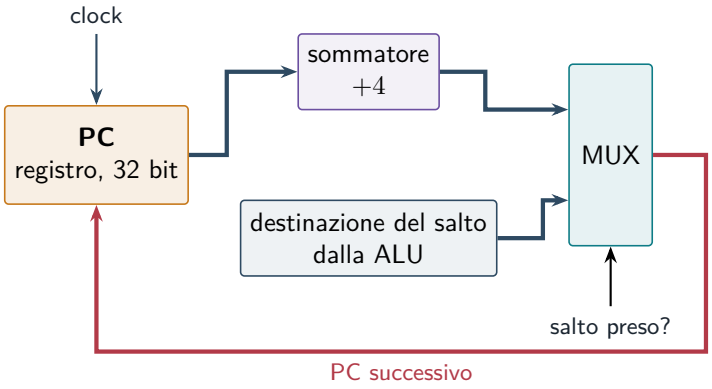


Figura 13.11. Il program counter e la rete che calcola il suo valore successivo.

13.6.3 Esempio di sintesi: un contatore a due bit

Esempio 13.1 (Un contatore modulo 4)

Stati: quattro, uno per valore del conteggio; non c'è ingresso, e l'uscita è lo stato stesso (una macchina di Moore). *Codice degli stati*: due variabili di stato Q_1Q_0, il conteggio in binario. *Tabella degli stati*: accanto a ogni stato presente, il prossimo. Con i flip-flop D l'eccitazione è lo stato prossimo, $D_1D_0 = Q_1^+Q_0^+$.

Q_1Q_0 presente	Q_1Q_0 prossimo	D_1	D_0
0 0	0 1	0	1
0 1	1 0	1	0
1 0	1 1	1	1
1 1	0 0	0	0

D_0 vale 1 nelle righe 00 e 10, quindi $D_0 = \overline{Q_0}$. D_1 vale 1 nelle righe 01 e 10, quindi $D_1 = \overline{Q_1}Q_0 + Q_1\overline{Q_0} = Q_1 \oplus Q_0$. Due flip-flop D, un invertitore e una porta XOR: una macchina che conta 0, 1, 2, 3, 0, ..., un passo per fronte del clock.

Gli stessi cinque passi, con più stati e più ingressi, progettano qualsiasi macchina sequenziale. E il risultato merita un confronto con il sommatore completo: un contatore è un sommatore che somma 1 al proprio registro.

13.6.4 Il processore è una macchina sequenziale

Ritorno alla macchina

Lo *stato* di un processore è il program counter, i trentadue registri e la memoria. Il suo *ingresso* è l'istruzione all'indirizzo contenuto nel PC, con i dati che essa legge. La sua *rete dello stato prossimo* è fatta di decodificatori, multiplexer e della ALU, che calcolano i nuovi valori. A ogni *fronte del clock* i registri nominati dall'istruzione prendono i loro nuovi valori.
Eseguire un programma non è altro che una lunga passeggiata tra gli stati di una macchina a stati finiti molto grande, un fronte del clock alla volta. L'esecuzione passo-passo in PRG32-QT mostra quella passeggiata, uno stato dopo l'altro.

Questo è il ponte verso la Parte V: il datapath (la parte combinatoria e i registri) e il controllo, che è esso stesso una macchina a stati finiti.

13.7 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l13.S` nel repository del corso, stampato per intero nel Listato 13.2. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 13.12 è il frame scritto dal test.

Sullo schermo.
Un colore per stato. Blu è TITLE: premete A e lo schermo diventa verde, PLAY. (OVER, rosso, si raggiunge solo quando una partita finisce: questo riscaldamento non ha ancora un gioco.)

Che cosa premere.
Premete A.

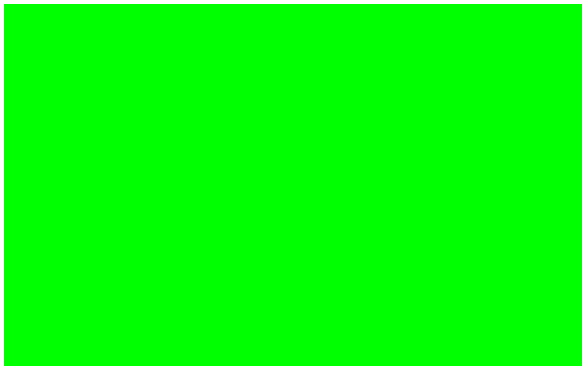


Figura 13.12. `l13.S` su PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 13: Sequential Machines
# l13.S: a game is a state machine
#
# On the screen: one colour per state. Blue is TITLE: press A and the screen turns
# green, PLAY. (OVER, red, is reached only when a game ends: this warm-up has no game
# yet. Hold A on the red screen and you meet the bug discussed in the lecture.)
# Build: python3 -m prg32 cartridge build l13.S --portable \
#            --entry-prefix l13 --name l13 --out l13.prg32
.option norelax

.equ TITLE, 0
.equ PLAY,  1
.equ OVER,  2

.section .text
.global l13_init
.global l13_update
.global l13_draw

l13_init:
```

```
    ret

l13_update:
game_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    call prg32_input_read      # a0 = buttons: the INPUT
    la   t0, state
    lw   t1, 0(t0)             # t1 = the present STATE
    andi t2, a0, 0x10          # button A
    li   t3, TITLE
    bne  t1, t3, 1f
    beqz t2, done              # TITLE: stay until A is pressed
    li   t1, PLAY              # TITLE --A--> PLAY
    j    store
1:  li   t3, OVER
    bne  t1, t3, done          # PLAY is handled elsewhere
    beqz t2, done              # OVER: stay until A is pressed
    li   t1, TITLE             # OVER --A--> TITLE
store:
    sw   t1, 0(t0)             # the next state becomes the present state
done:
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

l13_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, state
    lw   t1, 0(t0)
    la   t0, colours         # the OUTPUT depends on the state only:
    slli t1, t1, 2           # a Moore machine
    add  t0, t0, t1
    lw   a0, 0(t0)
    call prg32_gfx_clear
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

.section .rodata
colours: .word 0x001f, 0x07e0, 0xf800     # TITLE blue, PLAY green, OVER red

.section .data
state: .word TITLE
```

Listato 13.2. Il programma completo del capitolo, `code/l13.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 13.3; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l13.S --portable \
```

```
    --entry-prefix l13 --name l13 --out l13.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l13.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 13.3. Compilare `l13.S` ed eseguirlo su PRG32-QT.

13.8 Verificate di aver capito

D13.1 In una macchina di Moore l'uscita dipende:

A. dal solo ingresso presente
B. dal solo stato presente
C. dallo stato presente e dall'ingresso presente
D. dallo stato prossimo e dall'ingresso

D13.2 Un latch SR fatto di porte NOR ha $Q = 1$. S e R valgono entrambi 0. Che cosa accade?

A. Q diventa 0
B. Q oscilla
C. Q resta a 1
D. lo stato è indefinito

D13.3 Che cosa distingue un flip-flop D da un latch D?

A. Il flip-flop ha due ingressi
B. Il flip-flop memorizza due bit
C. Il flip-flop cambia solo su un fronte del clock
D. Al flip-flop non serve un clock

D13.4 Quanti flip-flop servono per una macchina a stati finiti con 10 stati?

A. 3
B. 4
C. 5
D. 10

13.9 Esercizi

Esercizio 13.1 (prima su carta, poi su PRG32)

1. Disegnate il diagramma degli stati di una macchina che dà in uscita 1 quando gli ultimi due ingressi valevano entrambi 1; scrivetela come macchina di Moore e come macchina di Mealy.

2. Completate il diagramma temporale di un latch SR per una sequenza di S e R a vostra scelta, segnando dove mantiene.
3. Sintetizzate un contatore modulo 3 (0, 1, 2, 0, ...) con due flip-flop D; che cosa accade nello stato non usato?
4. Aggiungete al gioco del riscaldamento uno stato `PAUSE`, in cui si entra e da cui si esce con il pulsante **SELECT**, e disegnate prima il nuovo diagramma.
5. Correggete il riscaldamento in modo che **A** debba essere rilasciato e premuto di nuovo: memorizzate lo stato precedente dei pulsanti e reagite solo a un cambiamento da 0 a 1.

Risultato atteso. La macchina di Moore del primo compito richiede tre stati (nessun 1 visto, un 1 visto, due o più), la macchina di Mealy due. Per il contatore: $D_1 = Q_0$ e $D_0 = \overline{Q_1}\,\overline{Q_0}$, con lo stato 11 come condizione di indifferenza o progettato per tornare a 00. Su PRG32: quattro schermate, e un pulsante tenuto premuto che causa esattamente una transizione. L'ultimo compito è il *rilevamento del fronte* in software: tenete `prev` in memoria, calcolate $pressed_now = buttons \cdot \overline{prev}$, poi memorizzate i pulsanti in `prev`. Il programma si comporta allora come un flip-flop invece che come un latch.

13.10 Glossario

Inglese	**Italiano**
sequential network	rete sequenziale
finite-state machine	automa a stati finiti
state table / diagram	tabella / diagramma degli stati
next-state function	funzione stato prossimo
present / next state	stato presente / prossimo
clock period	periodo di clock
latch / flip-flop	latch / flip-flop (bistabile)
feedback	retroazione
rising / falling edge	fronte di salita / di discesa
level- / edge-triggered	sensibile al livello / al fronte
register	registro
program counter	contatore di programma

13.11 Conclusioni e letture

I circuiti digitali sono combinatori o sequenziali; per quelli sequenziali contano il tempo e lo stato. Una macchina sequenziale realizza un automa a stati finiti, descritto come macchina di Mealy o di Moore. Latch e flip-flop sono le memorie elementari: la retroazione fa sì che un circuito ricordi. Il clock decide quando: i latch ascoltano durante un livello, i flip-flop solo su un fronte. I registri, i contatori, il program counter e il processore stesso sono macchine sequenziali costruite attorno a reti combinatorie.

Letture. Patterson e Hennessy (2020), appendice A (clock, elementi di memoria, macchine a stati finiti); Bucci (2017), i capitoli sulle reti sequenziali; *Playing with RISC-V*, capitolo 8, il gioco conclusivo in assembly (Montella et al., 2026b).

CAPITOLO 14

Macchine elementari: registri e bus

"Strade? Dove stiamo andando non c'è bisogno di strade."
— Dr. Emmett Brown, *Ritorno al futuro*, Robert Zemeckis, 1985

In questo capitolo

Alla fine di questo capitolo saprete

- descrivere un registro come un insieme di flip-flop con un clock comune, e spiegarne il segnale di caricamento;
- spiegare che cos'è un bus, e confrontare i bus dedicati con un bus comune;
- dire perché a un bus comune servono i buffer a tre stati, e che cosa significa alta impedenza;
- descrivere un registro a scorrimento e le sue quattro combinazioni di ingresso e uscita seriali e paralleli;
- convertire un dato da parallelo a seriale e viceversa, in hardware e in assembly RISC-V.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI i registri a scorrimento; i registri come flip-flop; bus e porte three-state.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI l'I/O mappato in memoria (Capitolo 15); memorie e indirizzi (Capitolo 15).

14.1 Riscaldamento sulla macchina

Il display e il chip audio del kit sono collegati al processore da pochi fili, non da trentadue. Una parola deve quindi viaggiare un bit dopo l'altro: un trasferimento *seriale*. Il Listato 14.1 invia un byte in questo modo.

```
1 # a0 = the byte to send, most significant bit first
2
```

```
    li   t0, 8              # eight bits to go
1:  srli t1, a0, 7          # bring bit 7 down to bit 0
    andi t1, t1, 1          # t1 = the bit to put on the wire
    ...                     # drive the data line with t1,
    ...                     # then pulse the clock line
    slli a0, a0, 1          # shift left: the next bit moves up
    addi t0, t0, -1
    bnez t0, 1b
```

Listato 14.1. Inviare un byte un bit alla volta, il bit più significativo per primo.

Il ciclo è un registro a scorrimento in software: scorrere, prendere un bit, ripetere. (Le due righe omesse scriverebbero nei registri del chip che ne controllano i piedini, un argomento del Capitolo 15.) L'ESP32-C6 contiene hardware che fa esattamente questo, milioni di volte al secondo: ogni pixel del display del kit arriva un bit alla volta. Questo capitolo costruisce le macchine che lo fanno.

14.2 Porte di parola e registri

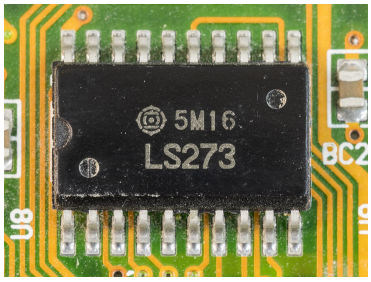
14.2.1 Porte di parola

Una *parola* è un vettore di n bit, $V = (v_0, v_1, \ldots, v_{n-1})$. Una *porta di parola* agisce su tutti i bit di una parola insieme: con un segnale di controllo a, calcola $a \cdot v_i$ per ogni i. Con $a = 1$ la parola passa invariata; con $a = 0$ ogni bit è forzato a 0. Sono n porte identiche che condividono un ingresso di controllo, il mattone di ogni datapath. Il multiplexer su parole del Capitolo 11 è costruito con porte di parola, e le istruzioni bit a bit `and`, `or` e `xor` di RISC-V sono porte di parola larghe trentadue bit.

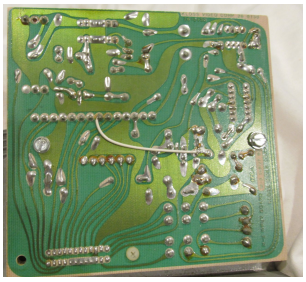
14.2.2 Il registro

Un *registro* è un dispositivo capace di memorizzare un gruppo di bit. Nella sua forma più semplice è un gruppo di flip-flop con un segnale di sincronizzazione comune, un clock o un'abilitazione (Capitolo 13). *Caricare* un registro significa memorizzare il dato presente ai suoi ingressi, il che avviene quando il segnale di *caricamento* (ingl. *load*) è attivo, tipicamente in sincronia con il clock.

L'esempio classico è il 74273, un registro di 8 bit in un solo chip (Figura 14.1a): otto flip-flop D, un clock e un azzeramento, ingressi da D_0 a D_7 e uscite da Q_0 a Q_7. In RISC-V ciascuno dei registri da `x1` a `x31` è fatto di trentadue di questi flip-flop.



(a) Un 74LS273: otto flip-flop D



(b) Piste di rame su un circuito stampato

Figura 14.1. Un registro e un bus, come oggetti.

14.3 Bus

14.3.1 I trasferimenti di dati e il bus

Definizione 14.1 (Bus)

Un *bus* è un canale di comunicazione che permette ai componenti di un sistema elettronico di scambiarsi dati trasmettendo e ricevendo segnali: l'insieme delle linee che trasporta un dato.

Il nome è una contrazione del latino *omnibus*, «per tutti». Un bus di n linee trasporta una parola di n bit in un solo passo. Il trasferimento di un dato da un registro X a un registro Y è l'operazione fondamentale dell'architettura di un calcolatore: quasi ogni istruzione del Capitolo 2 è, in fondo, una sequenza di trasferimenti tra registri.

14.3.2 Due soluzioni tecniche

Con i *bus dedicati* c'è un insieme di linee per ogni possibile trasferimento. Più trasferimenti possono avvenire nello stesso momento; la soluzione è efficiente ma costosa, perché i fili crescono con il quadrato del numero dei registri. Con un *bus comune* un unico percorso raccoglie molti trasferimenti potenziali: ogni volta si scelgono una sorgente e una destinazione, e un trasferimento richiede due passi, dalla sorgente al bus e dal bus alla destinazione. Il bus comune è economico e diffuso; è meno efficiente, perché i trasferimenti non possono essere simultanei. I processori reali usano soluzioni intermedie: più bus, ciascuno condiviso da un gruppo di registri.

14.3.3 Trasferimenti paralleli, seriali e misti

In un trasferimento *parallelo* gli n bit della parola viaggiano insieme su n linee, in un solo passo. In un trasferimento *seriale* i bit viaggiano uno dopo l'altro su una sola linea, in n passi. Un trasferimento può anche essere *misto*: seriale sul byte, per esempio, con otto linee che trasportano un byte alla volta.

I bus hanno dimensioni fisiche molto diverse (Figura 14.1b). Dentro un chip sono paralleli. Tra chip e tra macchine oggi sono quasi sempre seriali: USB, SATA, PCI Express, HDMI ed Ethernet sono tutti seriali, perché a velocità molto alte è più facile tenere pulita una sola linea veloce che tenere in sincronia trentadue linee.

14.3.4 Dal bus al registro, e dal registro al bus

La Figura 14.2 mostra le due operazioni elementari su un bus comune. *Dal bus al registro*: il dato è caricato nel registro R quando il segnale di caricamento c è attivo (uno *strobe*, di solito in sincronia con il clock). *Dal registro al bus*: il contenuto di R raggiunge il bus attraverso un *buffer* controllato da un segnale di abilitazione; quando l'abilitazione non è attiva il registro deve lasciare il bus libero per gli altri. Come lo faccia è l'argomento del prossimo paragrafo.

14.4 Buffer a tre stati



Figura 14.2. I due trasferimenti elementari su un bus comune.

14.4.1 La porta a tre stati

Se due registri pilotassero lo stesso filo, uno con 0 e uno con 1, il risultato sarebbe un cortocircuito tra l'alimentazione e la massa.

Definizione 14.2 (Uscita a tre stati)

Un'uscita *a tre stati* (o *tristate*) ha una terza condizione oltre a 0 e 1: l'*alta impedenza*, scritta z. In alta impedenza l'uscita è elettricamente scollegata: non pilota il filo e non lo carica.

Un segnale di abilitazione a decide: con $a = 1$ la porta funziona normalmente, con $a = 0$ la sua uscita è z. Elettricamente, entrambi i transistor di uscita della porta (Capitolo 10) sono interdetti. La tabella seguente descrive un AND a tre stati con abilitazione attiva alta.

a	x	y	uscita
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1
0	qualsiasi	qualsiasi	z

È la tecnica più diffusa per collegare più registri sorgente a un bus comune.

14.4.2 Abilitare un bus

Una porta *di parola* a tre stati posta tra un bus X e un bus Y, con un segnale di abilitazione a, fa sì che Y riproduca X se a è attivo; altrimenti tutte le linee di Y sono in alta impedenza, e Y assume il valore neutro. Si dice che Y è *sostenuto* da un buffer a tre stati.

Ritorno alla macchina

I piedini GPIO dell'ESP32-C6 funzionano in questo modo: ogni piedino può essere un'uscita, oppure può essere rilasciato e usato come ingresso. Sul kit i piedini del joystick sono configurati come ingressi, con i loro buffer di uscita in alta impedenza, e i piedini del display come uscite.

14.4.3 Collegare i bus: l'OR di bus

Più registri sorgente si collegano a un bus semplicemente unendo le loro uscite, ciascuna attraverso un buffer a tre stati (Figura 14.3).

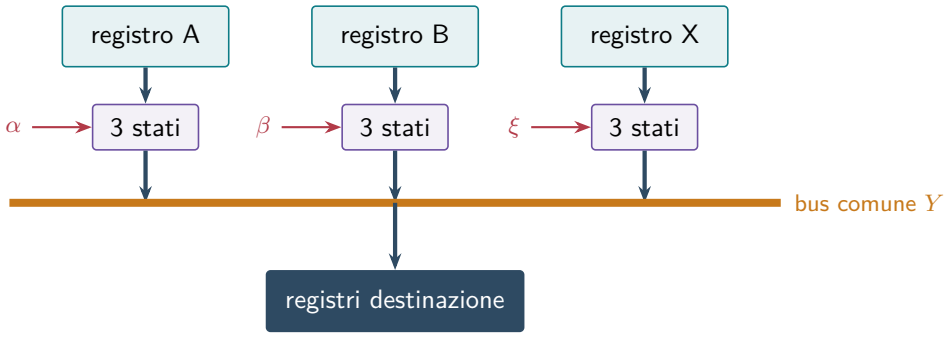


Figura 14.3. Tre sorgenti su un bus comune, ciascuna dietro un buffer a tre stati con la propria abilitazione.

La regola del bus comune

Delle n sorgenti che confluiscono in un bus, al più una è abilitata alla volta: un segnale attivo, e gli altri $n-1$ neutri.

Dentro i chip moderni le lunghe linee condivise sono di solito sostituite da multiplexer, che fanno la stessa selezione senza stati di alta impedenza; tra chip, i bus a tre stati sono ancora la regola.

14.5 Registri a scorrimento

14.5.1 Definizione

Definizione 14.3 (Registro a scorrimento)

Un *registro a scorrimento* (ingl. *shift register*) è una catena di celle di memoria di un bit collegate tra loro: a ogni impulso di clock ogni bit si sposta da una cella a quella adiacente.

Ogni cella è un flip-flop D il cui ingresso è l'uscita del suo vicino (Figura 14.4). Lo scorrimento può andare in un verso fisso o in un verso scelto da una linea di controllo (registri bidirezionali). I flip-flop sensibili al fronte sono qui essenziali: con latch trasparenti un bit percorrerebbe l'intera catena durante un solo impulso di clock.

14.5.2 Quattro tipi di registro a scorrimento

L'ingresso e l'uscita di un registro a scorrimento possono essere ciascuno seriale o parallelo, il che dà quattro tipi (Tabella 14.1). Rendono più facili la lettura e la scrittura dei dati, perché trasformano dati paralleli in dati seriali e viceversa.

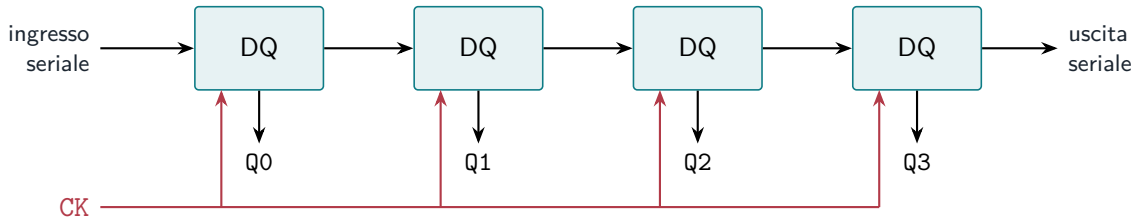


Figura 14.4. Un registro a scorrimento di quattro bit costruito con flip-flop D, con ingresso seriale e uscite sia seriale sia parallele.

Tabella 14.1. Quattro tipi di registro a scorrimento.

Tipo	Nome	Uso
ingresso seriale, uscita seriale	SISO	un ritardo di n impulsi di clock; un collegamento seriale tra due registri
ingresso seriale, uscita parallela	SIPO	conversione da seriale a parallelo: il ricevitore
ingresso parallelo, uscita seriale	PISO	conversione da parallelo a seriale: il trasmettitore
ingresso parallelo, uscita parallela	PIPO	un registro ordinario che può anche scorrere

14.5.3 Trasferimenti tra registri

In un trasferimento seriale l'uscita seriale di un registro a scorrimento alimenta l'ingresso seriale di un altro: n impulsi di clock spostano una parola di n bit, e basta un filo. In un trasferimento parallelo le uscite parallele di un registro alimentano gli ingressi paralleli di un altro: un solo impulso di clock sposta l'intera parola, e servono n fili.

Concetto

Spostare una parola di n bit costa o tempo o rame: n impulsi o n fili. È lo stesso compromesso che c'è tra bus dedicati e bus comune.

14.5.4 Da seriale a parallelo e ritorno

Un collegamento seriale completo funziona in cinque passi (Figura 14.5). Il dato di n bit è posto in parallelo in un registro R. R è un registro a scorrimento, e gli n bit ne escono uno dopo l'altro, in n passi successivi. Una sola linea li porta all'apparato a valle. Lì un altro registro viene caricato serialmente in n passi, ricostruendo il dato bit per bit. Infine il dato ricostruito è passato oltre in parallelo.



Figura 14.5. La conversione di un dato da parallelo a seriale e ritorno.

Questo è l'hardware del riscaldamento. Il display del kit riceve ogni pixel in questo modo su un bus chiamato SPI, e la tastiera del Capitolo 21 invia ogni codice di tasto in questo modo. UART, SPI, I2C, I2S e USB poggiano tutti su una coppia di registri a scorrimento, uno a ciascun capo.

14.5.5 Registri con un indirizzo: una prima memoria

Prendete quattro registri di tre bit e metteteci davanti un decodificatore (Figura 14.6). Due linee di indirizzo, A_1 e A_0, vanno al decodificatore da 2 a 4, che seleziona un registro. Tre ingressi dati, $I_2I_1I_0$, raggiungono tutti i registri, ma viene caricato solo quello selezionato; tre uscite dati, $O_2O_1O_0$, mostrano il contenuto del registro selezionato attraverso buffer a tre stati. Tre linee di controllo governano l'insieme: una *selezione del chip* (ingl. *chip select*) (CS), che abilita il dispositivo, una linea che sceglie tra lettura e scrittura, e un'*abilitazione dell'uscita* (OE). Il risultato è una memoria di 4 parole per 3 bit.

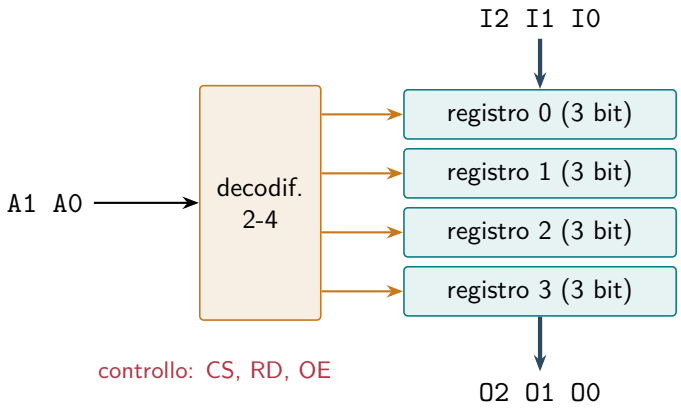


Figura 14.6. Quattro registri e un decodificatore: una memoria di 4 parole per 3 bit.

Concetto

Un decodificatore davanti a un gruppo di registri fa una memoria. Il prossimo capitolo porta questa idea alla scala della memoria centrale di un calcolatore.

14.6 Ritorno alla macchina

14.6.1 I bus sul kit PRG32

Il kit contiene un esempio di quasi ogni tipo di bus di questo capitolo (Tabella 14.2). La selezione del chip del display è il CS della piccola memoria appena vista: più dispositivi possono condividere lo stesso bus seriale perché ascolta solo quello selezionato.

14.6.2 Gli scorrimenti in RISC-V: un registro a scorrimento in una istruzione

`slli` e `srli` spostano tutti i bit di un registro di un dato numero di posizioni, riempiendo con zeri; `srai` riempie con copie del bit di segno (Capitolo 5); `sll`, `srl` e `sra` prendono la quantità da un secondo registro. Con esse le due conversioni sono due istruzioni ciascuna (Listato 14.2).

```
# parallel to serial, LSB first
andi t1, a0, 1      # take bit 0
srli a0, a0, 1      # next bit

# serial to parallel, MSB first
slli a1, a1, 1      # make room
or   a1, a1, t1     # insert bit
```

Tabella 14.2. I bus sul kit PRG32 (da `docs/hardware` del repository).

Collegamento	Tipo	Linee	Che cosa vi viaggia
dentro l'ESP32-C6	parallelo	32 linee dati, indirizzi e controllo	istruzioni, dati e trasferimenti tra registri
display (ILI9341)	seriale, SPI	clock, dati in uscita, selezione del chip, dato/comando	ogni pixel di ogni frame, bit per bit
amplificatore audio	seriale, I2S	clock di bit, selezione di parola, dati	campioni audio, 22 050 al secondo
joystick e pulsanti	un filo per pulsante	un piedino di ingresso ciascuno	una rappresentazione decodificata: una linea per pulsante
USB verso il portatile	seriale, coppia differenziale	due fili dati	il firmware e la console di debug

Listato 14.2. Da parallelo a seriale, e da seriale a parallelo.

A un vero registro a scorrimento servono n impulsi di clock per scorrere di n posizioni. Il processore non funziona così: in hardware uno scorrimento di una quantità qualsiasi in un solo passo è fatto da un *barrel shifter*, che è fatto di strati di multiplexer, non di una catena di flip-flop. È combinatorio, fa il suo lavoro entro un periodo di clock, ed è una delle unità della ALU del Capitolo 11.

14.7 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l14.S` nel repository del corso, stampato per intero nel Listato 14.3. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 14.7 è il frame scritto dal test.

Sullo schermo.
Riga in alto: gli otto bit di 0xA5 nell'ordine in cui escono, il più significativo per primo (accesa, spenta, accesa, spenta, spenta, accesa, spenta, accesa). Riga in basso: il byte ricostruito dal ricevitore. Un quadrato verde dice che è uguale al byte inviato.

Che cosa premere.
Niente: il programma va da solo.

```
# ARC12 Computer Architecture - Lecture 14: Elementary Machines: Registers and Buses
# l14.S: a byte sent one bit at a time, and rebuilt at the other end
#
# On the screen: top row: the eight bits of 0xA5 in the order in which they leave,
# most significant first (lit, dark, lit, dark, dark, lit, dark, lit). Bottom row: the
# byte rebuilt by the receiver. A green square says that it is equal to the byte sent.
```

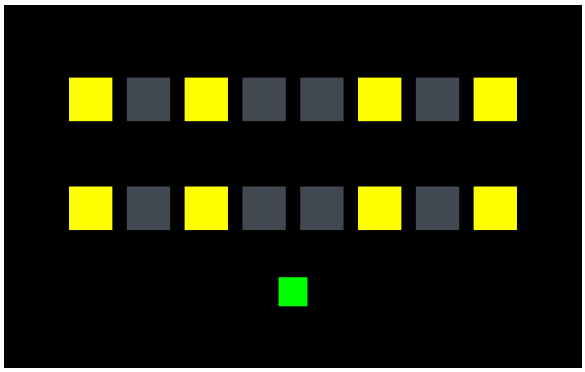


Figura 14.7. `l14.S` su PRG32-QT.

```
# Build: python3 -m prg32 cartridge build l14.S --portable \
#            --entry-prefix l14 --name l14 --out l14.prg32
.option norelax

.equ BLACK,  0x0000
.equ GREY,   0x4208
.equ YELLOW, 0xffe0
.equ GREEN,  0x07e0
.equ RED,    0xf800
.equ BYTE,   0xA5            # the byte to send: 1010 0101

.section .text
.global l14_init
.global l14_update
.global l14_draw

l14_init:
    ret

l14_update:
    ret

# lamp: a0 = position 0 to 7, a1 = y, a2 = bit
lamp:
    li   a4, GREY
    beqz a2, 1f
    li   a4, YELLOW
1:  slli a0, a0, 5
    addi a0, a0, 36          # x = 36 + 32 * position
    li   a2, 24
    li   a3, 24
    tail prg32_gfx_rect

l14_draw:
    addi sp, sp, -32
    sw   ra, 28(sp)
    sw   s0, 24(sp)
    sw   s1, 20(sp)
    sw   s2, 16(sp)
    sw   s3, 12(sp)
    li   a0, BLACK
```

```
48      call prg32_gfx_clear
49
50      li   s0, BYTE            # a0 of the slide: the byte to send
51      li   s1, 0               # the receiver's shift register
52      li   s2, 0               # how many bits have gone
53  1:  srli t1, s0, 7           # bring bit 7 down to bit 0
54      andi s3, t1, 1           # s3 = the bit to put on the wire
55      mv   a0, s2              # ... here the wire is a lamp:
56      li   a1, 40              #     position = the order of sending
57      mv   a2, s3
58      call lamp
59      slli s1, s1, 1           # receiver: make room (serial to parallel)
60      or   s1, s1, s3          #           insert the bit
61      slli s0, s0, 1           # sender: shift left, the next bit moves up
62      andi s0, s0, 255         #         (keep eight bits)
63      addi s2, s2, 1
64      li   t0, 8               # eight bits to go
65      blt  s2, t0, 1b
66
67      li   s2, 0               # show the byte that the receiver rebuilt
68  2:  li   t0, 7
69      sub  t0, t0, s2
70      srl  a2, s1, t0
71      andi a2, a2, 1
72      mv   a0, s2
73      li   a1, 100
74      call lamp
75      addi s2, s2, 1
76      li   t0, 8
77      blt  s2, t0, 2b
78
79      li   a4, RED             # is it the byte that was sent?
80      li   t0, BYTE
81      bne  s1, t0, 3f
82      li   a4, GREEN
83  3:  li   a0, 152
84      li   a1, 150
85      li   a2, 16
86      li   a3, 16
87      call prg32_gfx_rect
88      lw   s3, 12(sp)
89      lw   s2, 16(sp)
90      lw   s1, 20(sp)
91      lw   s0, 24(sp)
92      lw   ra, 28(sp)
93      addi sp, sp, 32
94      ret
```

Listato 14.3. Il programma completo del capitolo, `code/l14.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 14.4; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)
```

```
# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l14.S --portable \
    --entry-prefix l14 --name l14 --out l14.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l14.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 14.4. Compilare `l14.S` ed eseguirlo su PRG32-QT.

14.8 Verificate di aver capito

D14.1 Perché a un bus comune servono i buffer a tre stati?

A. Per rendere più veloci i trasferimenti
B. Perché solo la sorgente abilitata piloti le linee
C. Per memorizzare il dato sul bus
D. Per convertire i dati da seriali a paralleli

D14.2 Quanti impulsi di clock richiede il trasferimento seriale di una parola di 32 bit?

A. 1
B. 5
C. 32
D. 64

D14.3 Una memoria ha 10 linee di indirizzo e 8 linee di dati. Quanti bit memorizza?

A. 80
B. 1 024
C. 8 192
D. 10 240

14.9 Esercizi

Esercizio 14.1 (prima su carta, poi su PRG32)

1. Disegnate un registro a scorrimento di 4 bit con ingresso seriale e uscita parallela con flip-flop D, e compilatene il contenuto lungo quattro impulsi di clock per l'ingresso 1, 0, 1, 1.
2. Tre registri condividono un bus comune: scrivete i segnali di abilitazione e di caricamento necessari per copiare il registro B nel registro A.
3. Completate il riscaldamento: scrivete il ciclo che riceve otto bit da una linea e ricostruisce il byte.
4. Su PRG32, mostrate un byte come otto lampade e fatelo scorrere a sinistra di una posizione ogni mezzo secondo.

5. Fate rientrare da destra il bit che esce a sinistra (una rotazione): un *contatore ad anello*.

Risultato atteso. Il terzo compito è `slli a1, a1, 1` e poi `or` con il bit ricevuto, otto volte. L'ultimo è `srli t1, a0, 7`; `andi t1, t1, 1`; `slli a0, a0, 1`; `or a0, a0, t1`; `andi a0, a0, 255`. Su PRG32: una configurazione di lampade che cammina verso sinistra e, nell'ultimo compito, gira in tondo per sempre.

14.10 Glossario

Inglese	**Italiano**
word gate	porta di parola
register	registro
bus	bus
dedicated / common bus	bus dedicato / comune
load, strobe	caricamento, strobe
three-state (tri-state)	a tre stati (tristate)
high impedance	alta impedenza
shift register	registro a scorrimento
serial / parallel transfer	trasferimento seriale / parallelo
chip select	selezione del chip

14.11 Conclusioni e letture

I registri sono una forma evoluta di memoria con un funzionamento elementare. La loro lettura e la loro scrittura avvengono attraverso bus di tipi diversi. Perché un bus comune funzioni servono circuiti a tre stati: 0, 1, e scollegato. I dati possono essere trasferiti in serie o in parallelo, e i registri a scorrimento convertono tra i due modi. Un decodificatore davanti a un gruppo di registri fa una memoria: l'argomento del prossimo capitolo.

Letture. Patterson e Hennessy (2020), appendice A; Bucci (2017), i capitoli su registri e bus; `docs/hardware/hardware.md` nel repository di PRG32.

CAPITOLO 15

La memoria

"...i computer e i programmi cominceranno a pensare e le persone smetteranno."
— Dr. Walter Gibbs, *Tron*, Steven Lisberger, 1982

In questo capitolo

Alla fine di questo capitolo saprete

- classificare le memorie come volatili o persistenti, in linea o fuori linea, e collocarle nella gerarchia delle memorie;
- descrivere come SRAM e DRAM memorizzano un bit, e perché sono usate per compiti diversi;
- spiegare che cosa sono le parole, gli indirizzi e la memoria indirizzabile al byte;
- distinguere l'ordine little-endian da quello big-endian, e dire quale usa RISC-V;
- descrivere passo per passo come il processore legge e scrive una parola;
- spiegare l'ingresso/uscita mappato in memoria e a che cosa serve il DMA.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI memorie e indirizzi; l'ordine little-endian; processore e memoria; l'I/O mappato in memoria.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI niente.

15.1 Riscaldamento sulla macchina

```
    la   t0, value
    lw   t1, 0(t0)       # the whole word        t1 = 0x12345678
    lbu  t2, 0(t0)       # the byte at address A     t2 = 0x78
    lbu  t3, 1(t0)       # the byte at address A+1   t3 = 0x56
    lbu  t4, 3(t0)       # the byte at address A+3   t4 = 0x12

.section .data
value: .word 0x12345678
```

Listato 15.1. La stessa memoria letta come parola e come byte.

Nel Listato 15.1 una parola di trentadue bit occupa quattro indirizzi consecutivi: la memoria è indirizzata byte per byte. Il byte all'indirizzo più basso è `0x78`, il meno significativo: l'ordine è little-endian. `lw` preleva quattro byte in una sola operazione, `lbu` ne preleva uno.

Provate voi

Eseguitelo in PRG32-QT e aprite la vista della memoria all'indirizzo contenuto in `t0`: i byte appaiono come `78 56 34 12`. L'ordine dei byte del Capitolo 4 è diventato un esperimento.

Di che cosa è fatta una memoria, e che cosa succede tra il processore e la memoria quando viene eseguita `lw`?

15.2 Che cos'è una memoria?

Definizione 15.1 (Memoria)

La *memoria* è l'elemento di un calcolatore, o di un suo sottosistema, dedicato alla persistenza dei dati e delle istruzioni dei programmi.

Le memorie sono i componenti capaci di memorizzare informazione: dati, programmi e risultati. Si classificano lungo due assi. Una memoria è *volatile* se l'informazione è conservata solo finché il calcolatore è alimentato, e *persistente* se è conservata anche a calcolatore spento. È *in linea* se i dati sono sempre accessibili, e *fuori linea* se il supporto deve essere montato prima che i dati possano essere raggiunti.

15.2.1 La gerarchia delle memorie

Nessuna tecnologia è allo stesso tempo veloce, capiente ed economica. Un calcolatore ne usa quindi diverse, disposte in una *gerarchia* (Figura 15.1): salendo, le memorie sono più veloci e più costose per bit; scendendo, più capienti e meno costose per bit. I tempi di accesso tipici vanno da meno di un nanosecondo a pochi nanosecondi per registri e cache, decine di nanosecondi per la DRAM, decine di microsecondi per i dischi a stato solido, millisecondi per i dischi magnetici, da secondi a minuti per il nastro.

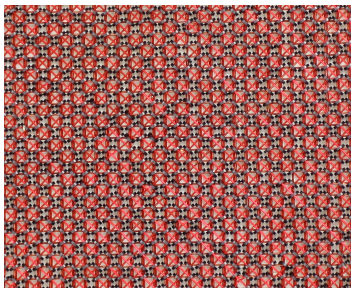
15.3 Un po' di storia: anelli di ferrite

15.3.1 La memoria a nuclei magnetici

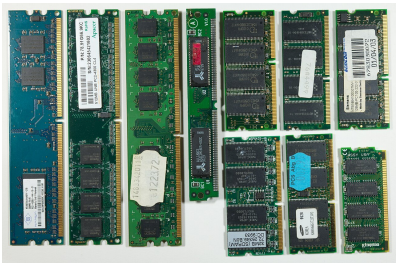
Nei primi calcolatori, prima che si diffondessero le memorie a semiconduttore, un bit era il verso del campo magnetico di un piccolo anello di ferrite, una ceramica magnetica (Figura 15.2a). Una memoria a nuclei è statica: conserva il suo contenuto senza alimentazione. Le prime ricerche furono condotte da An Wang e Way-Dong Woo a Harvard nel 1949; all'inizio degli anni Cinquanta Jay Forrester al MIT costruì una memoria a nuclei per Whirlwind, un calcolatore pensato come simulatore di volo.



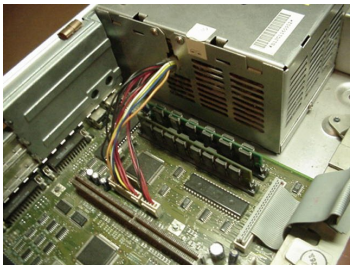
Figura 15.1. La gerarchia delle memorie.



(a) Nuclei di ferrite e i loro fili



(b) Moduli di memoria di più generazioni



(c) Moduli nei loro alloggiamenti

Figura 15.2. La memoria, ieri e oggi.

15.3.2 Leggere e scrivere un nucleo

Ogni nucleo è attraversato da fili. Due linee di indirizzo, X e Y, selezionano un nucleo: solo il nucleo al loro incrocio riceve corrente sufficiente per cambiare.

La lettura è distruttiva. Il bit selezionato viene posto a 0; se conteneva 1, compare un impulso su un terzo filo, la linea di *sense*, e altrimenti non succede niente. Il dato è stato letto e perso. La soluzione di Wang fu che ogni lettura è seguita da una scrittura che memorizza di nuovo il dato.

La scrittura. Tutti i bit partono da 0. X e Y selezionano il nucleo, che viene posto a 1; se il bit deve essere 0, una linea di *inhibit* impedisce il cambiamento. (Sense e inhibit condividono un solo filo.) Il contributo di Forrester fu la disposizione che gestisce un gran numero di nuclei con pochi collegamenti.

La lettura distruttiva seguita da una riscrittura è tornata oggi nella DRAM.

15.4 La memoria oggi

15.4.1 SRAM e DRAM

Definizione 15.2 (SRAM e DRAM)

In una *RAM statica* (SRAM) ogni bit è tenuto da un circuito bistabile, come in un flip-flop. In una *RAM dinamica* (DRAM) ogni bit è una carica in un minuscolo condensatore, dietro un solo transistor.

Le celle di una SRAM sono disposte in una matrice e raggiunte per riga e colonna; ciascuna richiede sei transistor (Capitolo 10) e conserva il suo contenuto finché è alimentata. La SRAM è veloce e costosa, ed è usata per le memorie cache; è anche la memoria centrale di un microcontrollore come l'ESP32-C6, che ha 512 KB di SRAM e nessuna DRAM.

Il condensatore di una cella DRAM si scarica, e il suo contenuto deve essere *rinfrescato* periodicamente da un circuito interno alla memoria. La DRAM è economica e densa, ed è la memoria centrale di portatili, telefoni e server. Una *SDRAM* è una DRAM sincrona, il cui accesso è governato da un clock; DDR4 e DDR5 ne sono generazioni.

15.4.2 Altre memorie

Una *RAM ferroelettrica* (FeRAM) conserva il suo stato senza alimentazione. La memoria *flash* è persistente e si cancella a blocchi: è la memoria delle chiavette USB, dei telefoni e dei dischi a stato solido, trattati nel prossimo capitolo. Il programma di un microcontrollore vive nella flash e le sue variabili nella SRAM.

I moduli di memoria hanno cambiato forma e dimensione negli anni (Figura 15.2b): SIMM, SDRAM PC100 e PC133, Rambus e DDR contenevano da decine a centinaia di megabyte ciascuno; i moduli DDR5 di oggi contengono da 16 a 64 GB.

15.4.3 Come riconoscere la memoria

Sulla scheda madre di un desktop o di un server la memoria sta in file di lunghi alloggiamenti accanto al processore, che contengono i moduli (Figura 15.2c). Su un portatile o un telefono sta in chip saldati vicino al processore. Sul kit PRG32 non c'è niente da vedere: la SRAM è dentro l'ESP32-C6 stesso, e la flash è un piccolo chip al suo fianco.

Due parametri caratterizzano una memoria: la sua *dimensione*, il numero di celle, che oggi è di gigabyte su un PC e di kilobyte su un microcontrollore; e il suo *tempo di accesso*, il ritardo dopo il quale il dato è disponibile, decine di nanosecondi per la DRAM. Per confronto, a 160 MHz un periodo di clock dell'ESP32-C6 è di 6,25 ns.

15.5 La memoria centrale

15.5.1 Parole e indirizzi

Definizione 15.3 (Memoria centrale)

La *memoria centrale* è un array di *parole*; una parola è una stringa di m bit, alla quale il processore accede con una sola operazione di lettura o di scrittura. Ogni parola è identificata da un *indirizzo*: un intero compreso tra 0 e $N-1$, rappresentato su n bit, con $N = 2^n$.

L'indirizzo è una parola codice, e la memoria la decodifica: è il decodificatore del Capitolo 11 e la piccola memoria del Capitolo 14, in scala maggiore. Trentadue bit di indirizzo danno 2^{32} indirizzi, 4 GiB. Una memoria si dice ad *accesso casuale* (RAM) quando il tempo di accesso non dipende dalla posizione del dato. I processori accedono alla memoria con un parallelismo uguale alla lunghezza della loro parola: 16, 32 o 64 bit.

15.5.2 Memoria indirizzabile al byte

In una memoria *indirizzabile al byte* il byte è la più piccola unità che ha un indirizzo (Figura 15.3). Una parola di 32 bit occupa quattro indirizzi consecutivi, e il suo indirizzo è quello del suo primo byte. Parole consecutive distano 4 indirizzi: è per questo che il program counter avanza di 4, e che l'indice di un array di parole è moltiplicato per 4 (Capitolo 4).

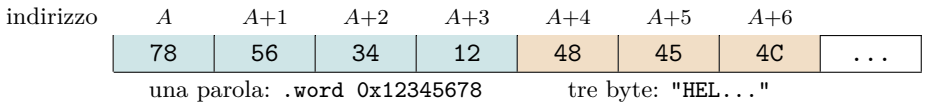


Figura 15.3. Una memoria indirizzabile al byte: ogni cella è un byte.

15.5.3 Big-endian e little-endian

I due nomi dicono come i byte di una parola sono disposti in memoria (Figura 15.4). Nell'ordine *little-endian* il byte meno significativo va all'indirizzo più basso: è l'ordine di RISC-V, di x86 e di ARM. Nell'ordine *big-endian* il byte più significativo va all'indirizzo più basso: è l'ordine del Motorola 68000, il processore di riferimento delle edizioni precedenti di questo corso, e dei protocolli di Internet.

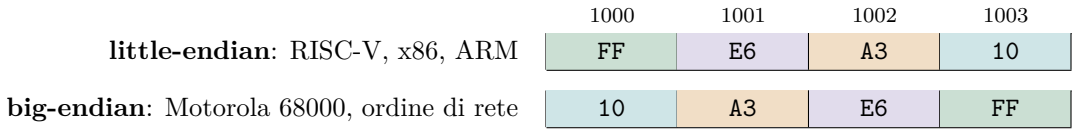


Figura 15.4. La parola `0x10A3E6FF` memorizzata all'indirizzo 1000.

15.5.4 Parole allineate e non allineate

Una parola che comincia a un indirizzo multiplo della sua dimensione è *allineata*: multipli di 4 per parole di 32 bit. Una parola allineata è prelevata con un solo accesso alla memoria; una parola non allineata sta a cavallo di due parole di memoria, e servono due accessi. La specifica RISC-V lascia all'implementazione il trattamento degli accessi non allineati: un load o uno store a un indirizzo non allineato è, secondo il processore, eseguito più lentamente

o rifiutato con un'eccezione. I programmi corretti tengono allineati i loro dati. L'assemblatore allinea i dati dichiarati con `.word`, e la direttiva `.balign 4` forza l'allineamento dopo una stringa o un byte (Listato 15.2). Le istruzioni devono sempre essere allineate.

```
.section .data
name:   .asciz "PRG32"    # 6 bytes
        .balign 4           # skip 2
score:  .word 0             # aligned
```

Listato 15.2. Forzare l'allineamento di una parola.

15.5.5 Parametri di una memoria

La *capacità* di una memoria è il numero N dei suoi registri e la lunghezza in bit di ciascuno: 1 MB sono 2^{20} registri di 8 bit; «16 kW di 32 bit» sono 16×2^{10} registri di 32 bit. Il tempo di accesso è il tempo necessario per compiere una lettura o una scrittura: il tempo per selezionare la locazione, più il tempo per la lettura o la scrittura vera e propria.

Ritorno alla macchina

L'ESP32-C6 ha 512 KB di SRAM per i dati, 320 KB di ROM, e una flash esterna di alcuni megabyte per i programmi. Il menu di PRG32 mostra ciò che resta al programmatore: circa 98 KB di RAM e 509 KB di flash per le cartucce. La memoria centrale del chip è quella di otto Commodore 64, o un trentamillesimo di quella di un portatile.

15.6 Processore e memoria

15.6.1 L'interfaccia

Il processore e la memoria si parlano attraverso tre bus e due registri (Figura 15.5). Il *registro indirizzo di memoria* (MA) contiene l'indirizzo e pilota il *bus indirizzi*. Il *registro buffer di memoria* (MB) contiene il dato che viaggia, in un verso o nell'altro, sul *bus dati*. L'unità di controllo pilota le *linee di controllo*, tra le quali ci sono l'abilitazione alla lettura e l'abilitazione alla scrittura. MA e MB sono registri interni del modello di processore del Capitolo 18: non sono tra i trentadue registri che un programmatore RISC-V vede.

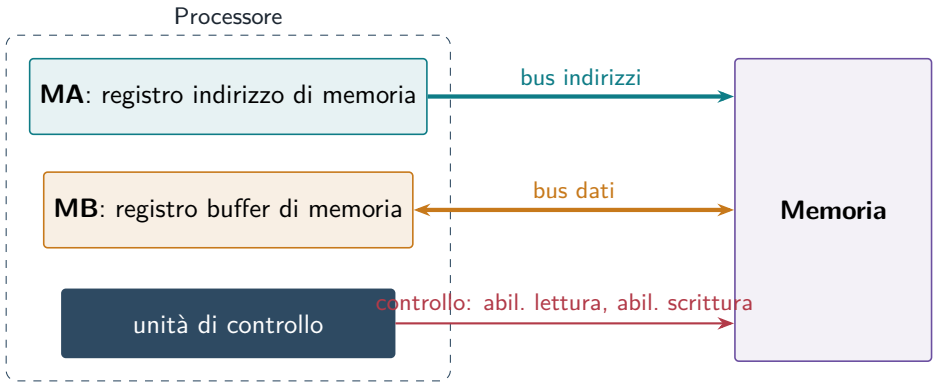


Figura 15.5. L'interfaccia tra processore e memoria.

15.6.2 Prelevare una parola: `lw t1, 0(t0)`

Tabella 15.1. I passi di un load.

Passo	Che cosa succede	Dove
1	l'indirizzo, `t0` + 0, è calcolato dalla ALU e posto in MA	processore
2	MA pilota il bus indirizzi	bus indirizzi
3	l'unità di controllo attiva l'abilitazione alla lettura	linee di controllo
4	la memoria decodifica l'indirizzo e mette i quattro byte della parola sul bus dati	memoria, bus dati
5	il dato sul bus dati è catturato in MB	processore
6	MB è copiato nel registro destinazione `t1`	processore

Uno store, `sw t1, 0(t0)`, è simmetrico (Tabella 15.1): l'indirizzo va in MA e il dato in MB, viene attivata l'abilitazione alla scrittura, e la memoria memorizza ciò che trova sul bus dati. Il tempo di accesso è il tempo che passa tra il passo 2 e il passo 5.

15.6.3 Sincronizzare processore e memoria

Come fa il processore a sapere che la memoria ha finito? Con la tecnica *asincrona* aspetta finché la memoria segnala, su una linea di *acknowledge* (conferma), che l'operazione è completa: il processore si adatta alla velocità della memoria, e il progetto della memoria diventa indipendente da quello del processore, al prezzo di una maggiore complessità. Con la tecnica *sincrona* una delle linee di controllo porta gli impulsi di un clock a frequenza fissa, e una lettura o una scrittura si completa entro un ciclo di clock, che va quindi dimensionato sulla velocità della memoria. La seconda tecnica è più semplice, ed è la regola dentro un singolo chip come un microcontrollore.

15.7 Memoria e ingresso/uscita

15.7.1 Collegare processore, memoria e I/O

Il sottosistema di ingresso/uscita permette al processore di interagire con dispositivi di ingresso (tastiera, mouse), dispositivi di uscita (schermo, stampante) e dispositivi che sono entrambe le cose (memoria di massa, schede di rete, porte seriali). Può essere collegato in due modi.

Definizione 15.4 (I/O mappato in memoria)

Con l'*I/O mappato in memoria* (ingl. *memory-mapped I/O*) la memoria e i dispositivi condividono lo stesso bus e lo stesso spazio degli indirizzi: un registro di un dispositivo si legge e si scrive come una locazione di memoria, e non servono istruzioni speciali.

L'alternativa è l'*I/O isolato*: due bus fisici, o due spazi degli indirizzi, uno per la memoria e uno per l'I/O, e istruzioni dedicate per raggiungere i dispositivi. La famiglia Intel x86 ha entrambi: le istruzioni `in` e `out` oltre ai dispositivi mappati in memoria.

Ritorno alla macchina

L'I/O mappato in memoria è la via di RISC-V: RISC-V non ha affatto istruzioni di ingresso/uscita. Scrivere una parola in un registro di una periferica con `sw` è il modo in cui un programma RISC-V accende un pixel o legge un pulsante.

15.7.2 Dove stanno le cose sull'ESP32-C6

La Tabella 15.2 è una mappa semplificata dello spazio degli indirizzi del chip del kit. Un solo spazio degli indirizzi serve per tutto: le stesse trentadue linee di indirizzo raggiungono ROM, RAM, flash e ogni dispositivo, e le stesse `lw` e `sw` li fanno funzionare. (La tabella è semplificata a partire dal manuale tecnico di riferimento (Espressif Systems, 2025), che va consultato prima di fare affidamento su un indirizzo.)

Tabella 15.2. Lo spazio degli indirizzi dell'ESP32-C6, semplificato.

L'intervallo comincia con	Che cosa c'è	Raggiunto da
`0x4000...`	ROM: il codice che avvia il chip	prelievo delle istruzioni
`0x4080...`	SRAM: 512 KB per dati, stack e cartucce caricate in RAM	`lw`, `sw`, prelievo delle istruzioni
`0x4200...`	flash esterna, vista attraverso una cache: firmware e costanti	`lw`, prelievo delle istruzioni
`0x6000...`	registri delle periferiche: GPIO, SPI, I2S, timer, Wi-Fi	`lw`, `sw`

L'indirizzo `0x40800000`, al quale viene caricata ogni cartuccia (Capitolo 1), è l'inizio della SRAM.

15.7.3 I tre bus componenti, e il DMA

Il bus che collega processore, memoria e I/O è dunque fatto di tre bus: il *bus dati*, che trasporta i dati; il *bus indirizzi*, che trasporta l'indirizzo, di memoria o di I/O, da cui o verso cui il dato è diretto; e le *linee di controllo*: abilitazione alla lettura e alla scrittura, acknowledge, e altre.

Definizione 15.5 (Accesso diretto alla memoria)

Con l'*accesso diretto alla memoria* (DMA, *direct memory access*), blocchi di dati passano direttamente tra un dispositivo e la memoria. Il processore avvia il trasferimento e viene avvisato quando è terminato.

Se più unità condividono il bus e possono accedere direttamente alla memoria, un controller DMA fa da arbitro tra esse. Sull'ESP32-C6 le periferiche SPI e I2S usano il DMA per inviare il frame buffer al display e i campioni audio all'amplificatore: il processore non copia i pixel uno per uno.

15.8 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l15.S` nel repository del corso, stampato per intero nel Listato 15.3. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 15.6 è il frame scritto dal test.

Sullo schermo.
Cinque righe di lampade. La prima è la parola 0x12345678 letta con lw. Le altre quattro sono i byte agli indirizzi A, A+1, A+2, A+3 letti con lbu: 0x78, 0x56, 0x34, 0x12. L'indirizzo più basso contiene il byte meno significativo: little-endian.

Che cosa premere.
Niente: il programma va da solo.

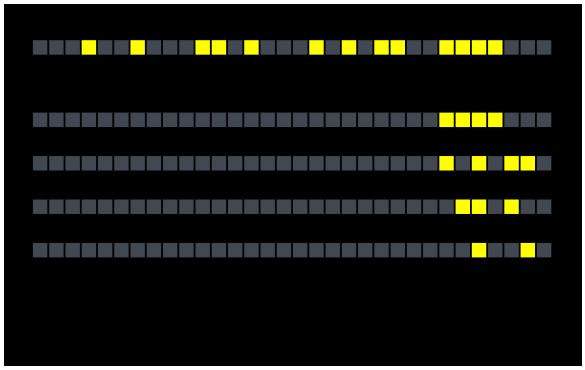


Figura 15.6. `l15.S` su PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 15: Memory
# l15.S: the same memory read as a word and as bytes
#
# On the screen: five rows of lamps. The first is the word 0x12345678 read with lw.
# The other four are the bytes at addresses A, A+1, A+2, A+3 read with lbu: 0x78,
# 0x56, 0x34, 0x12. The lowest address holds the least significant byte: little-
# endian.
# Build: python3 -m prg32 cartridge build l15.S --portable \
#            --entry-prefix l15 --name l15 --out l15.prg32
.option norelax

.equ BLACK,  0x0000
.equ GREY,   0x4208
.equ YELLOW, 0xffe0

.section .text
.global l15_init
.global l15_update
.global l15_draw

l15_init:
    ret

```

```
l15_update:
    ret

l15_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    call prg32_console_clear
    la   t0, value
    lw   a0, 0(t0)       # the whole word
    call prg32_console_hex32

    la   t0, value
    lw   a0, 0(t0)       # the whole word: 0x12345678
    li   a1, 20
    call lamps32
    li   s0, 0           # byte number, 0 to 3
1:  la   t0, value
    add  t0, t0, s0      # address A + s0: one byte further each time
    lbu  a0, 0(t0)       # the byte at that address, zero-extended
    slli a1, s0, 4
    slli t1, s0, 3
    add  a1, a1, t1      # 24 * s0
    addi a1, a1, 60      # y = 60 + 24 * byte number
    call lamps32
    addi s0, s0, 1
    li   t0, 4
    blt  s0, t0, 1b
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

# lamps32: show the 32 bits of a word as a row of lamps, bit 31 on the left.
#   a0 = the word, a1 = y of the row.  A lamp is yellow for 1, grey for 0.
lamps32:
    addi sp, sp, -32
    sw   ra, 28(sp)
    sw   s0, 24(sp)
    sw   s1, 20(sp)
    sw   s2, 16(sp)
    mv   s0, a0              # the word
    mv   s1, a1              # y
    li   s2, 0               # lamp number, 0 on the left
1:  li   t0, 31
    sub  t0, t0, s2          # lamp i shows bit 31 - i
    srl  t1, s0, t0          # bring that bit down to bit 0
    andi t1, t1, 1           # keep it, clear all the others
    li   a4, GREY            # lamp off
    beqz t1, 2f
    li   a4, YELLOW          # lamp on
2:  slli a0, s2, 3
    add  a0, a0, s2          # 9 * i
    addi a0, a0, 16          # x = 16 + 9 * i
```

```
80      mv   a1, s1
81      li   a2, 8
82      li   a3, 8
83      call prg32_gfx_rect
84      addi s2, s2, 1
85      li   t0, 32
86      blt  s2, t0, 1b
87      lw   s2, 16(sp)
88      lw   s1, 20(sp)
89      lw   s0, 24(sp)
90      lw   ra, 28(sp)
91      addi sp, sp, 32
92      ret
93
94  .section .data
95  value: .word 0x12345678
```

Listato 15.3. Il programma completo del capitolo, `code/l15.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 15.4; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l15.S --portable \
    --entry-prefix l15 --name l15 --out l15.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l15.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 15.4. Compilare `l15.S` ed eseguirlo su PRG32-QT.

15.9 Verificate di aver capito

D15.1 Una memoria ha 16 linee di indirizzo ed è indirizzabile al byte. Quanto è grande?

A. 16 byte
B. 32 KiB
C. 64 KiB
D. 16 MiB

D15.2 Su RISC-V, `value: .word 0xCAFE0001` sta all'indirizzo A. Che cosa legge `lbu t0, 0(t0)` se `t0` $= A$?

A. `0xCA`
B. `0xFE`
C. `0x00`
D. `0x01`

D15.3 Perché la DRAM va rinfrescata?

A. Perché i suoi condensatori perdono la carica
B. Perché la lettura è lenta
C. Perché non è ad accesso casuale
D. Perché il clock è troppo veloce

D15.4 Come scrive un programma RISC-V su un dispositivo di uscita?

A. Con l'istruzione `out`
B. Con `sw` all'indirizzo di un registro del dispositivo
C. Con un registro speciale `x32`
D. Non può: solo il firmware può farlo

15.10 Esercizi

Esercizio 15.1 (prima su carta, poi su PRG32)

1. Una memoria di 4 KiB parole di 16 bit: quante linee di indirizzo, quante linee di dati, quanti bit in tutto?
2. Scrivete i byte memorizzati, in ordine di indirizzo, da `.word 0xDEADBEEF` e da `.asciz "Ciao"` su una macchina little-endian.
3. Elencate i passi di `sw t1, 8(t0)` sull'interfaccia processore–memoria.
4. In PRG32-QT, trovate la vostra variabile contatore nella vista della memoria e guardate i suoi quattro byte cambiare frame per frame.
5. Memorizzate una parola con `sw` e rileggetela con quattro istruzioni `lbu`; ricostruitela con scorrimenti e `or` e verificate che coincida.

Risultato atteso. Su carta: 12 linee di indirizzo, 16 linee di dati, 65 536 bit; `EF BE AD DE` e `43 69 61 6F 00`. Su PRG32: il byte meno significativo del contatore cambia a ogni frame, il successivo ogni 256 frame. Nell'ultimo compito i byte sono fatti scorrere di 8, 16 e 24 posizioni e poi uniti.

15.11 Glossario

Inglese	Italiano
main memory	memoria centrale
volatile / persistent	volatile / persistente
magnetic core	nucleo magnetico (di ferrite)
refresh	rinfresco (refresh)
byte order	ordinamento dei byte
access time	tempo di accesso
aligned word	parola allineata
address / data bus	bus indirizzi / bus dati
read / write enable	abilitazione lettura / scrittura
memory-mapped I/O	I/O mappato in memoria

15.12 Conclusioni e letture

La memoria è un componente fondamentale di un calcolatore: un dispositivo capace di conservare lo stato dei dati. Tipi diversi di memoria servono a scopi diversi, perché nessuna tecnologia è insieme veloce, capiente ed economica. I dati sono memorizzati come parole di bit, ciascuna con un indirizzo; RISC-V indirizza i byte e li ordina in little-endian. L'accesso avviene attraverso la selezione (il bus indirizzi), la lettura o la scrittura (il bus dati) e le linee di controllo. In RISC-V, i dispositivi di ingresso e di uscita si raggiungono esattamente come la memoria.

Letture. Patterson e Hennessy (2020), paragrafi 2.3 e 5.1–5.2; Stallings (2021), il capitolo sulla memoria interna; il manuale tecnico di riferimento dell'ESP32-C6, «System and Memory» (Espressif Systems, 2025).

CAPITOLO 16

La memoria permanente

"... un dato non esiste se non è stato salvato..."
— Prof. Almerico Murli, anni Novanta

In questo capitolo

Alla fine di questo capitolo saprete

- nominare le unità funzionali di un calcolatore e riconoscerle su una scheda madre e sul kit PRG32;
- distinguere ROM, PROM, EPROM, EEPROM e flash, e dire che cos'è il firmware;
- spiegare perché una cache rende più veloce un calcolatore, e che cos'è un cache miss;
- descrivere tracce, settori e blocchi di un disco magnetico e le componenti del suo tempo di accesso;
- spiegare come un disco ottico memorizza i bit, e confrontare CD e DVD;
- collocare flash, dischi a stato solido, dischi rigidi e nastri in un sistema di memorizzazione a più livelli.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI ROM, flash, firmware; l'architettura di von Neumann; la cache; la memoria secondaria.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI niente.

16.1 Riscaldamento sulla macchina

```
1 .section .rodata            # part of the cartridge file
2 title: .asciz "HIGH SCORE"  # stored in flash: it survives
3
4 .section .data              # copied into RAM at start
5 best:  .word 0              # lives in RAM: it is lost
6
```

```
7  game_update:
8      ...
9      la   t0, best
10     lw   t1, 0(t0)
11     ble  a0, t1, 1f         # new score not better: skip
12     sw   a0, 0(t0)          # remember the best score ... in RAM
13 1:  ...
```

Listato 16.1. Spegnere e riaccendere il kit.

Un gioco tiene il punteggio migliore in una variabile (Listato 16.1). Dopo che il kit è stato spento e riacceso il titolo è ancora lì, ma il punteggio migliore è 0. La ragione sta nelle due direttive di sezione. La stringa è in `.rodata`, che fa parte del file della cartuccia, e la cartuccia è un file nella memoria *flash*, che è non volatile. La variabile è in `.data`, che viene copiata in RAM all'avvio e vive nella SRAM, che è volatile.

Per conservare un punteggio per sempre, bisogna scriverlo in una memoria permanente. Quali esistono? (Su PRG32, i punteggi persistenti sono un servizio del Cartridge Store; qui il punto è dove vive ciascuna sezione di un programma.)

16.2 I componenti di un calcolatore

16.2.1 Unità funzionali

Processore.
Esegue le istruzioni. Sul kit: il core RISC-V dell'ESP32-C6.

Memoria centrale.
Contiene i programmi in esecuzione e i loro dati. Sul kit: 512 KB di SRAM dentro il chip.

Memoria secondaria.
Conserva programmi e dati quando l'alimentazione è spenta. Sul kit: il chip flash che contiene il firmware e le cartucce.

Dispositivi di ingresso/uscita.
Scheda video e monitor, tastiera, mouse, stampante, memoria di massa. Sul kit: display, joystick, amplificatore.

Linee di comunicazione.
I bus: linee di dati e di indirizzi condivise da tutte le unità.

Un personal computer distribuisce queste unità su una scheda madre; un microcontrollore le mette quasi tutte in un solo contenitore.

16.2.2 La scheda madre

Una *scheda madre* (Figura 16.1) porta gli zoccoli per uno o più processori e, vicino a essi, gli alloggiamenti per i moduli di memoria; un *chipset* che collega processore, memoria e periferiche; bus di espansione, oggi alloggiamenti PCI Express per schede video e altre schede; connessioni per la memoria di massa (Serial ATA, M.2) e per il mondo esterno (USB, Ethernet, audio); e un chip flash con il firmware che avvia la macchina. Dal punto di vista architetturale una scheda del 2002 e una di oggi hanno molto in comune.

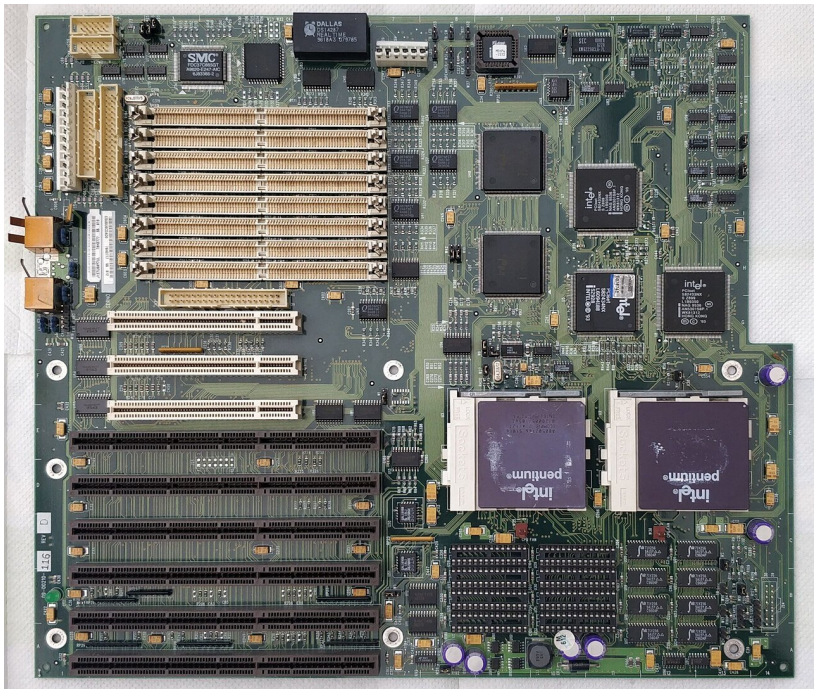


Figura 16.1. La scheda madre di un server biprocessore.

16.2.3 Il firmware: il BIOS e i suoi eredi

Definizione 16.1 (Firmware)

Il *firmware* è il software di base di una macchina, conservato in una memoria non volatile e usato dal processore dal momento in cui viene acceso.

In un personal computer si chiamava BIOS: un programma in un chip non volatile, specifico di una data scheda madre. Le sue istruzioni e i suoi dati sono divisi in quattro parti: il *POST* (power-on self test), che controlla che i dispositivi funzionino; il *SETUP*, la configurazione dei dispositivi; il *BOOT*, che avvia il sistema operativo; e i *DRIVER*, funzioni di base a basso livello per la tastiera, i dischi e altri dispositivi. I PC di oggi usano il firmware UEFI, che svolge gli stessi compiti.

Ritorno alla macchina

Sull'ESP32-C6 un programma di avvio in ROM avvia un bootloader in flash, che avvia il firmware PRG32, che carica la cartuccia scelta dal suo menu *Setup*. Ogni stadio trova, controlla e avvia il successivo: una catena che va dalla prima istruzione dopo il reset, attraverso strati di memoria permanente, alla prima istruzione della vostra `init`.

16.3 Memorie non volatili

16.3.1 Dalla ROM alla flash

Le memorie a semiconduttore che conservano il loro contenuto senza alimentazione si sono evolute verso un modo sempre più facile di scriverle (Tabella 16.1). Leggerle richiede un

tempo dell'ordine dei nanosecondi.

Tabella 16.1. Dalla ROM alla flash.

Memoria	Scritta da	Cancellata	Uso tipico
ROM	il costruttore, una volta per tutte	mai	codice di avvio, vecchie cartucce di gioco
PROM	l'utente, una volta	mai	piccole serie di produzione
EPROM	l'utente, molte volte	tutta insieme, con luce ultravioletta attraverso una finestra di quarzo (circa 15 minuti)	sviluppo, BIOS dei primi PC
EEPROM	l'utente, sul posto	elettricamente, un byte alla volta	dati di configurazione
Flash	l'utente, sul posto	elettricamente, a blocchi	firmware, chiavette USB, schede di memoria, dischi a stato solido

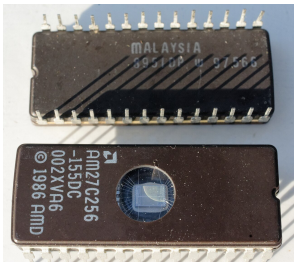
Le sigle stanno per *read-only memory* (memoria di sola lettura), ROM *programmabile*, PROM *cancellabile* e PROM *cancellabile elettricamente*. Una ROM è, dal punto di vista logico, una macchina combinatoria: una tabella che associa una parola fissa a ogni indirizzo, come fanno le tabelle `dx` e `dy` di Verm1.

16.3.2 La memoria flash

La flash è il tipo più recente di EEPROM: può essere riprogrammata sul posto, senza essere rimossa dalla scheda, ed è oggi la memoria a semiconduttore meno costosa. Ha tre peculiarità.

- Si cancella a *blocchi* di byte, non un byte alla volta. La cancellazione pone a 1 tutti i bit di un blocco, e la scrittura può solo trasformare degli 1 in 0.
- Ogni blocco sopravvive a un numero limitato di cicli di cancellazione, così che il controller distribuisce le scritture in modo uniforme sui blocchi.
- La lettura è veloce; la scrittura e, soprattutto, la cancellazione sono lente.

La flash del kit contiene il firmware e le cartucce: caricare una cartuccia cancella e riscrive alcuni dei suoi blocchi. Ed è per questo che il punteggio migliore del riscaldamento non viene semplicemente riscritto nella cartuccia a ogni frame: la flash non è un posto per le variabili.



(a) Una EPROM con la sua finestra di quarzo



(b) Un disco rigido aperto



(c) Un disco a stato solido

Figura 16.2. Tre memorie permanenti.

16.4 La cache

16.4.1 Perché una cache?

Durante i cicli di clock necessari per leggere o scrivere la memoria il processore aspetta; più lenta è la memoria, più cicli si perdono. Perché non costruire memorie veloci quanto i registri del processore? Perché dovrebbero stare sullo stesso chip, dato che un collegamento attraverso un bus è più lento; e allora non potrebbero essere molto grandi, o il chip crescerebbe in dimensioni e in costo. Il compromesso è una memoria piccola e veloce sul chip del processore, e una o più memorie più grandi e più lente raggiunte attraverso il bus.

16.4.2 Come funziona una cache

Definizione 16.2 (Cache)

Una *cache* (dal francese *cacher*, nascondere) è una memoria di piccola capacità ma veloce, posta tra il processore e la memoria centrale, che tiene una copia di parte del contenuto di quest'ultima.

Il processore cerca prima nella cache (Figura 16.3). C'è un *successo* (ingl. *hit*) se il contenuto dell'indirizzo richiesto è lì, e arriva subito. C'è un *fallimento* (ingl. *miss*) se non c'è, e allora deve essere copiato dalla RAM nella cache; con esso viene copiato un numero fisso di locazioni vicine. Il programma non si ferma e non se ne accorge: l'accesso è semplicemente più lento.

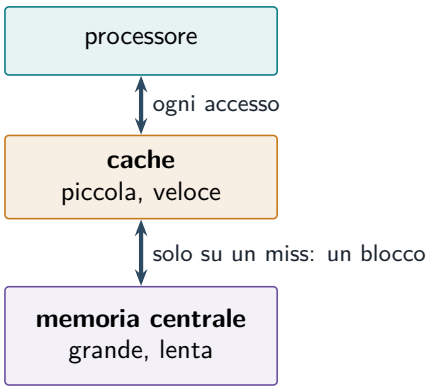


Figura 16.3. Una cache tra il processore e la memoria centrale.

Perché copiare i vicini? Per una proprietà empirica dei programmi.

Il principio di località

Un programma che ha appena usato un indirizzo probabilmente lo userà di nuovo presto (*località temporale*) e probabilmente userà i suoi vicini (*località spaziale*). I cicli e gli array ne sono la ragione.

Più successi ci sono, più breve è il tempo medio di accesso. Se una frazione h degli accessi sono successi, la cache risponde in un tempo t_c e un fallimento costa in più il tempo t_m della memoria, la media è

$$t = h\,t_c + (1-h)\,(t_c + t_m).$$

Con $t_c = 1$ ns, $t_m = 50$ ns e il 95% di successi, $t = 0{,}95 \times 1 + 0{,}05 \times 51 = 3{,}5$ ns: il sistema si comporta quasi come se l'intera memoria fosse veloce quanto la cache.

16.4.3 Livelli di cache e macchine tipiche

I processori reali hanno più livelli di cache, tutti costruiti con SRAM (Tabella 16.2). Il primo livello, interno, mantiene un flusso continuo dentro il processore; il secondo limita il rallentamento dovuto alla memoria centrale; il terzo, da 8 a 32 MB e oltre, è condiviso dai core dello stesso processore.

Tabella 16.2. I livelli di cache su tre macchine.

	PC del 2002	**PC del 2020**	**ESP32-C6 (il kit)**
Processore	1 core	4 core	1 core RISC-V, 160 MHz
Cache L1	64 KB	64 KB per core	nessuna per la SRAM
Cache L2	512 KB	512 KB per core	nessuna
Cache L3	nessuna	8 MB, condivisa dai core	nessuna
Memoria centrale	256 MB	16 GB	512 KB di SRAM
Memoria di massa	disco rigido da 40 GB	1 TB + 1 TB, in mirror	pochi MB di flash, letta attraverso una piccola cache

Sull'ESP32-C6 la SRAM è già veloce quanto serve al core; solo la flash esterna sta dietro una cache.

16.5 La memoria secondaria

16.5.1 Caratteristiche

La *memoria secondaria*, o *memoria di massa*, conserva tutti i programmi e i dati di un calcolatore, mentre la memoria centrale contiene i programmi in esecuzione e i dati di cui hanno bisogno. Rispetto alla memoria centrale è

- non volatile: i dati non si perdono quando il calcolatore viene spento;
- di grande capacità: più ordini di grandezza in più;
- di basso costo per bit: più ordini di grandezza in meno;
- più lenta: di alcuni ordini di grandezza.

La sua informazione è organizzata in *blocchi*, di solito di un kilobyte o più, non in singoli byte. Alcuni supporti permettono l'accesso *diretto* a qualsiasi blocco; altri solo l'accesso *sequenziale*.

16.5.2 Dischi magnetici: tracce, settori, blocchi

Un disco magnetico (Figura 16.2b) è fatto di piatti rivestiti di un materiale magnetizzabile e tenuti in rotazione a velocità costante, e di testine che leggono e scrivono l'informazione. La superficie è divisa in *tracce* concentriche e in *settori*, spicchi del disco (Figura 16.4). Un settore taglia ogni traccia in blocchi: il blocco è la più piccola unità indirizzabile, identificata dal numero della sua traccia e del suo settore. *Formattare* un disco significa dividerne la superficie in tracce e settori. L'accesso è diretto: la testina può essere posizionata su qualsiasi

blocco senza passare su tutti i precedenti. Più settori formano un *cluster*; le tracce allo stesso raggio su tutti i piatti formano un *cilindro*.

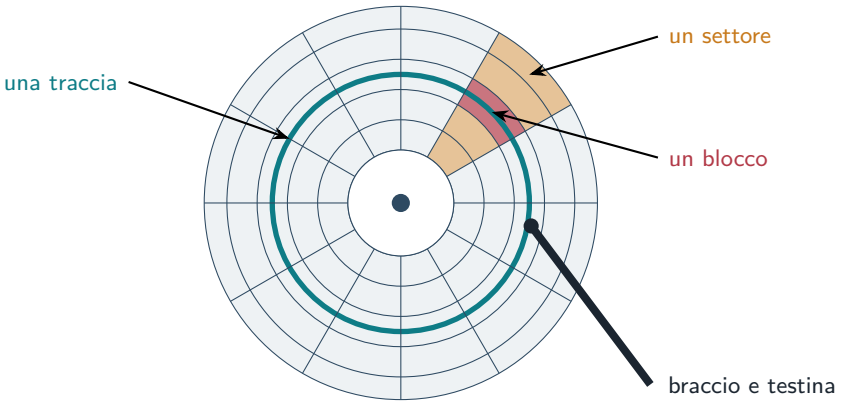


Figura 16.4. La superficie di un disco magnetico.

16.5.3 Tempo di accesso di un disco magnetico

Per leggere o scrivere un blocco la testina deve essere portata all'indirizzo giusto; durante il trasferimento la testina è ferma e il disco si muove. Il tempo di accesso è la somma di tre componenti.

Tempo di posizionamento.
Il tempo per spostare la testina in senso radiale fino al cilindro giusto, e quindi alla traccia giusta.

Tempo di latenza.
Il tempo di attesa perché il settore desiderato passi sotto la testina; in media, mezzo giro.

Tempo di trasferimento.
Il tempo perché il blocco passi sotto la testina.

Un disco che gira a 7 200 giri al minuto impiega $60/7200 = 8{,}33$ ms per giro, e la sua latenza media è di circa 4,2 ms. I dischi rigidi sono sigillati contro la polvere, hanno capacità di terabyte e tempi di accesso di millisecondi. Un millisecondo è un milione di nanosecondi: durante un solo accesso a un disco un processore a 160 MHz potrebbe eseguire più di centomila istruzioni.

16.5.4 Dischi rigidi e dischi a stato solido

Un *disco rigido* (HDD, *hard disk drive*) ha piatti magnetici e testine mobili; il suo tempo di accesso, di millisecondi, è dominato dalla meccanica; ha il costo più basso per terabyte ed è ancora la scelta per i grandi archivi e per i backup. Un *disco a stato solido* (SSD) è memoria flash con un controller, e non ha parti mobili; il suo tempo di accesso è di decine di microsecondi; si collega attraverso SATA o, più velocemente, attraverso NVMe su PCI Express. È la memoria di massa standard di portatili e telefoni, ed è la stessa tecnologia del chip flash del kit.

16.6 Dischi ottici

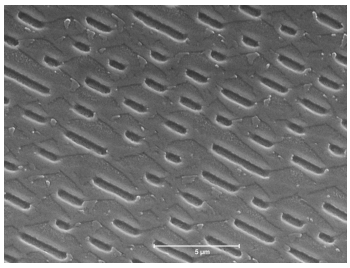
16.6.1 Bit letti con la luce

Un disco ottico è un disco rigido sul quale viene proiettato un raggio laser di pochi milliwatt; la riflessione cambia tra i *pit* (le fossette) e la superficie, il *land* (Figura 16.5a).

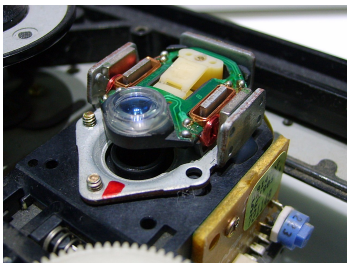
Attenzione

0 e 1 *non* sono pit e land. Un 1 è un *cambiamento*, da pit a land o da land a pit; uno 0 è nessun cambiamento.

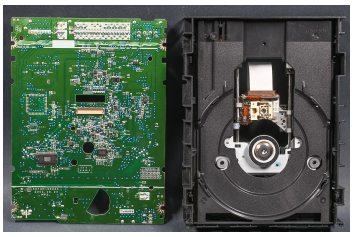
Codificare un 1 come una transizione è un'idea usata anche in molti collegamenti seriali, perché permette al ricevitore di recuperare il ritmo dei bit dal segnale stesso. I dischi ottici di solito si scrivono una volta sola, perché i cambiamenti fisici sono irreversibili: sono usati per distribuire programmi e come archivi che non devono essere modificati. In un CD-Recordable, due strati di policarbonato racchiudono un sottile foglio riflettente coperto da un colorante traslucido, e la registrazione brucia dei punti nel colorante; i dischi riscrivibili usano un materiale che può tornare indietro.



(a) Pit e land (microscopio elettronico)



(b) Dentro un lettore di CD-ROM



(c) Dentro un lettore di DVD

Figura 16.5. Memorie ottiche.

16.6.2 Il compact disc

Su un compact disc l'informazione è memorizzata su un'unica traccia a spirale, con densità di registrazione costante; la distanza tra due giri successivi della spirale, il *passo della traccia*, è di 1,6 micrometri. Sono possibili due modi di far girare il disco. Con la *velocità lineare costante* (CLV) il disco gira più in fretta quando la testina legge vicino al centro, da 200 a 500 giri al minuto, così che la traccia passa sotto la testina sempre alla stessa velocità (1,2 m/s alla velocità di base, fissata dalla velocità di bit necessaria per riprodurre la musica). Con la *velocità angolare costante* (CAV) i giri sono costanti e così il numero di bit per settore, con il risultato che la superficie esterna è sottoutilizzata.

16.6.3 DVD e CD a confronto

Il DVD (*digital versatile disk*) ha pit più piccoli e una spirale più fitta, e fino a due strati e due facce (Tabella 16.3).

16.7 La memorizzazione su più livelli

Un moderno sistema di calcolo ha più livelli di memoria permanente (Figura 16.6), spesso gestiti automaticamente: i dati usati spesso migrano verso i livelli veloci, e il resto scende

Tabella 16.3. DVD e CD.

	DVD	**CD**
Diametro del disco	120 mm	120 mm
Spessore del disco	1,2 mm (0,6 mm × 2)	1,2 mm
Facce / strati	1 o 2 / 1 o 2	1 / 1
Dimensione minima dei pit	0,4 μm	0,83 μm
Passo della traccia	0,74 μm	1,6 μm
Velocità media di trasferimento	4,7 MB/s	0,15 MB/s
Capacità, 1 strato, 1 faccia	circa 5 GB	circa 0,68 GB
Capacità, 2 strati, 2 facce	17 GB	–

verso quelli economici. È l'idea della cache, applicata alla memoria di massa; e da un chip di pochi megabyte sul kit a sale di petabyte (Figura 16.7) la gerarchia è la stessa a ogni scala.



Figura 16.6. Una memoria permanente a più livelli.



Figura 16.7. Una libreria di nastri in un centro di calcolo.

16.8 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/116.S` nel repository del corso, stampato per intero nel Listato 16.2. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 16.8 è il frame scritto dal test.

Sullo schermo.

Il titolo HIGH SCORE e due barre: il punteggio di questa prova, che cresce mentre A

è premuto e torna a zero quando A è rilasciato, e il punteggio migliore, che non scende mai. Riavviate la cartuccia: il titolo c'è ancora, il punteggio migliore è di nuovo zero.

Che cosa premere.

Tenete premuto A, rilasciatelo, poi riavviate la cartuccia.



Figura 16.8. `l16.S` su PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 16: Permanent Memory
# l16.S: what stays in the file and what lives in RAM
#
# On the screen: the title HIGH SCORE and two bars: the score of this run, which grows
# while A is held and falls back to zero when A is released, and the best score, which
# never goes down. Restart the cartridge: the title is still there, the best score is
# zero again.
# Build: python3 -m prg32 cartridge build l16.S --portable \
#            --entry-prefix l16 --name l16 --out l16.prg32
.option norelax

.equ BLACK,  0x0000
.equ WHITE,  0xffff
.equ GREEN,  0x07e0
.equ YELLOW, 0xffe0

.section .text
.global l16_init
.global l16_update
.global l16_draw

l16_init:
    ret

l16_update:
game_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    call prg32_input_read
    andi t2, a0, 0x10        # button A
    la   t0, score
    lw   a0, 0(t0)
    addi a0, a0, 1           # A held: one more point per frame
```

```
34      bnez t2, 2f
35      li   a0, 0                # A released: the run is over
36  2:  sw   a0, 0(t0)
37
38      la   t0, best
39      lw   t1, 0(t0)
40      ble  a0, t1, 1f          # new score not better: skip
41      sw   a0, 0(t0)           # remember the best score ... in RAM
42  1:  lw   ra, 12(sp)
43      addi sp, sp, 16
44      ret
45
46  l16_draw:
47      addi sp, sp, -16
48      sw   ra, 12(sp)
49      li   a0, BLACK
50      call prg32_gfx_clear
51      li   a0, 20
52      li   a1, 20
53      la   a2, title            # read from .rodata: it is always there
54      li   a3, WHITE
55      li   a4, BLACK
56      call prg32_gfx_text8
57      la   t0, score
58      lw   a2, 0(t0)
59      andi a2, a2, 255          # width = score, at most 255 pixels
60      li   a0, 20
61      li   a1, 60
62      li   a3, 12
63      li   a4, GREEN
64      call prg32_gfx_rect
65      la   t0, best
66      lw   a2, 0(t0)
67      andi a2, a2, 255
68      li   a0, 20
69      li   a1, 100
70      li   a3, 12
71      li   a4, YELLOW
72      call prg32_gfx_rect
73      lw   ra, 12(sp)
74      addi sp, sp, 16
75      ret
76
77  .section .rodata             # part of the cartridge file
78  title: .asciz "HIGH SCORE"   # stored in flash: it survives
79
80  .section .data               # copied into RAM at start
81  best:  .word 0               # lives in RAM: it is lost
82  score: .word 0
```

Listato 16.2. Il programma completo del capitolo, `code/l16.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 16.3; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l16.S --portable \
    --entry-prefix l16 --name l16 --out l16.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l16.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 16.3. Compilare `l16.S` ed eseguirlo su PRG32-QT.

16.9 Verificate di aver capito

D16.1 Quale memoria si può cancellare elettricamente, a blocchi, senza rimuoverla dalla scheda?

A. ROM
B. PROM
C. EPROM
D. Flash

D16.2 Che cosa accade su un cache miss?

A. Il programma si ferma con un errore
B. Il dato viene prelevato dalla memoria centrale e copiato nella cache, con i suoi vicini
C. La cache viene cancellata
D. Il processore salta l'istruzione

D16.3 Quale componente del tempo di accesso di un disco rigido dipende dalla velocità di rotazione?

A. Il tempo di posizionamento
B. La latenza
C. Il tempo di formattazione
D. Il tempo di avvio

D16.4 Su un disco ottico, un 1 è codificato come:

A. un pit
B. un land
C. una transizione tra pit e land
D. un doppio pit

16.10 Esercizi

Esercizio 16.1 (su carta e su PRG32)

1. Un disco gira a 5 400 giri al minuto: qual è la sua latenza media?
2. Una cache risponde in 2 ns e la memoria in 80 ns: qual è il tempo medio di accesso con il 90% di successi? E con il 99%?
3. Dite dove vive sul kit PRG32 ciascuna di queste cose: il codice di avvio, il firmware, una cartuccia, la variabile `counter`, il registro `t0`.
4. Trovate le specifiche della memoria di massa del vostro portatile o del vostro telefono: tecnologia, capacità, interfaccia.
5. Su PRG32, cronometrate due cicli che sommano un array grande: uno in ordine, uno che salta con un passo grande. C'è una differenza, e perché su questo chip potrebbe non esserci?

Risultato atteso. Latenza: metà di $60/5400 = 11{,}1$ ms, cioè circa 5,6 ms. Tempo di accesso: $0{,}9 \times 2 + 0{,}1 \times 82 = 10$ ns, e $0{,}99 \times 2 + 0{,}01 \times 82 = 2{,}8$ ns. Le cinque cose vivono in ROM, in flash, in flash (o in RAM una volta caricata), in SRAM, e in un registro di flip-flop dentro il processore. Nell'ultimo compito la SRAM dell'ESP32-C6 non ha una cache davanti, così che lo schema di accesso conta poco; su un portatile la differenza è grande.

16.11 Glossario

Inglese	**Italiano**
motherboard	scheda madre
firmware	firmware
non-volatile	non volatile
cache hit / miss	successo / fallimento di cache
mass storage	memoria di massa
track / sector / block	traccia / settore / blocco
seek time	tempo di posizionamento
latency	tempo di latenza
solid-state disk	disco a stato solido
tape library	libreria a nastri

16.12 Conclusioni e letture

Le memorie non volatili contengono di solito il software di base di una macchina, il firmware. La cache migliora le prestazioni tenendo vicino al processore una copia di parte della memoria centrale, e la stessa idea, un livello piccolo e veloce davanti a uno grande e lento, è usata di nuovo nei sistemi di memorizzazione. Le memorie permanenti possono essere a semiconduttore (flash), magnetiche o ottiche, e un sistema moderno ne ha più livelli, spesso gestiti automaticamente.

Letture. Patterson e Hennessy (2020), paragrafi 5.1–5.4; Stallings (2021), i capitoli sulla cache e sulla memoria esterna; Tanenbaum e Austin (2013), capitolo 2.

Il terzo blocco è completo. Il prossimo capitolo lo usa nella terza parte di Verm1: un corpo, regole e stati.

CAPITOLO 17

Verm1, parte 3: un corpo, regole e stati

"Mostratemi le vostre tabelle, e di solito non avrò bisogno dei vostri diagrammi di flusso: saranno ovvi."

— Fred Brooks, *The Mythical Man-Month*

In questo capitolo

Alla fine di questo capitolo avrete

- un verme con un corpo che segue la testa e cresce di una cella a ogni pasto;
- un gioco che si può perdere: il verme non deve mordersi;
- tre schermate, un titolo, il gioco e una fine partita, collegate dal pulsante **A**;
- il cibo che non compare mai sul verme;
- un punteggio migliore, conservato finché la cartuccia resta in memoria.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI gli array in memoria; macchine a stati finiti; .text, .rodata, .data; latch e flip-flop; la virgola fissa.

DA UTILIZZARE OGGI la convenzione di chiamata (Capitolo 18) e, usati senza essere studiati nel corso, gli strumenti di PRG32, il debugger di PRG32-QT.

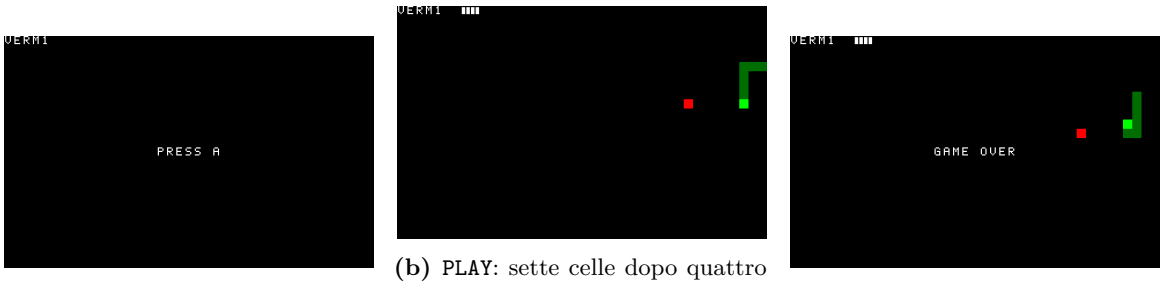
DA COMPRENDERE PIÙ AVANTI niente.

In questo terzo tutorial la memoria entra nel gioco: un anello di byte, una mappa del mondo, una macchina con tre stati. Con «strutture dati» al posto di «tabelle», la frase di Brooks citata sopra è il motto del capitolo: una volta progettati bene i dati, il codice è una trascrizione. La Tabella 17.1 elenca ciò che si prende dai Capitoli da 13 a 16. Il frame è il clock della macchina: `update` calcola lo stato prossimo dallo stato presente e dagli ingressi. Il file di riferimento è `verm1_part3.S` (Appendice D).

Partite da una parte 2 che funziona; se la vostra non funziona, copiate `verm1_part2.S`. Il ciclo di lavoro è quello del Listato 7.1. In questa parte il file cambia molto: compilate dopo ogni *Eseguite*, non solo alla fine.

Tabella 17.1. Che cosa la parte 3 prende dai capitoli su stato e memoria.

Da	Idea	Dove compare
Capitolo 13	una macchina a stati finiti: stati, ingressi, transizioni	`TITLE`, `PLAY`, `OVER`
Capitolo 13	un fronte è un cambiamento, non un livello	**A** premuto ora e non un frame fa
Capitolo 14	un registro conserva un valore per il clock successivo	`prev_btn`: i pulsanti dell'ultimo frame
Capitolo 15	la memoria è un array di byte: indirizzo = base + indice	il corpo, la mappa del mondo
Capitolo 16	che cosa resta nel file e che cosa vive in RAM	`.rodata` e `.data`; il punteggio migliore



(a) `TITLE` **(b)** `PLAY`: sette celle dopo quattro pasti **(c)** `OVER`

Figura 17.1. Dove va questo capitolo: tre schermate, un solo programma.

17.1 Il progetto, prima del codice

Per prime si progettano due strutture dati e un diagramma.

17.1.1 Il corpo è un anello di 256 elementi

Il corpo è tenuto in due array di byte, `body_x` e `body_y`: l'elemento k contiene la cella di un segmento. Niente viene mai fatto scorrere: un passo scrive un nuovo elemento e sposta un indice (Figura 17.2). Due numeri descrivono il verme: `head_i`, l'elemento della testa, e `length`, il numero di elementi in uso. La coda è l'elemento

$$\texttt{head_i} - \texttt{length} + 1.$$

Dopo l'elemento 255 viene l'elemento 0, e il rientro è una sola istruzione: l'indice AND 255. La struttura è un *anello*, o buffer circolare: la struttura che sta dietro ogni coda first-in, first-out.

Muovere un verme di 200 celle costa quindi quanto muoverne uno di 3. La dimensione 256 è scelta perché è una potenza di due, così che il rientro è una maschera.

17.1.2 Il mondo è una mappa di 960 byte

La nuova testa tocca il corpo? Guardare ogni segmento costerebbe fino a 255 confronti. Il programma tiene invece una *mappa*: un byte per cella, 1 se c'è un segmento e 0 se no. La cella in colonna c della riga r è il byte numero

$$r \times 40 + c, \qquad \text{da 0 a 959},$$

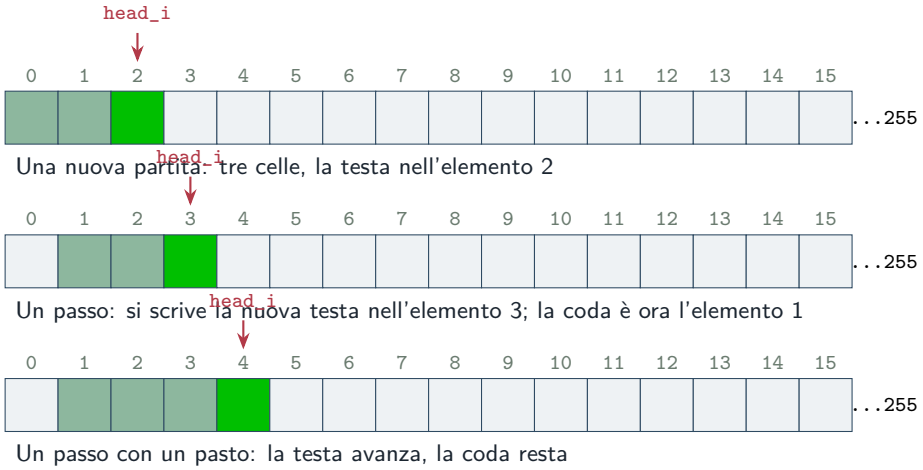


Figura 17.2. I primi sedici elementi dell'anello in tre momenti successivi.

e il suo indirizzo è `grid` $+ r \times 40 + c$: un mondo a due dimensioni in una memoria a una dimensione, il calcolo dell'indirizzo del Capitolo 15. Un passo pone a 1 un byte, quello della nuova testa, e ne azzera uno, quello della vecchia coda. Il test di collisione è un solo load, qualunque sia la lunghezza del verme.

È il classico scambio di memoria in cambio di tempo. La mappa e l'anello dicono la stessa cosa in due modi, e il programma deve tenerli coerenti: ogni scrittura nell'una è accoppiata a una scrittura nell'altro.

17.1.3 Il gioco è una macchina con tre stati

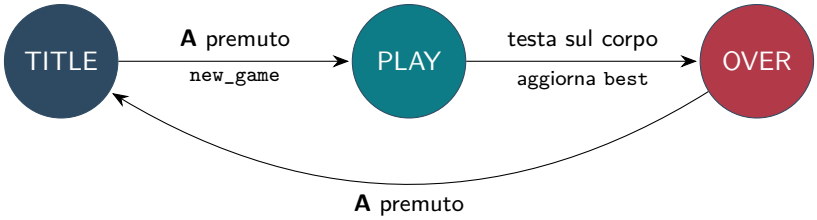


Figura 17.3. Il diagramma degli stati di Verm1. In ogni altro caso lo stato non cambia.

Lo stato è una parola in memoria (Figura 17.3). `update` guarda prima lo stato e poi gli ingressi. E «**A** premuto» significa: abbassato ora, e alzato un frame fa. Tenere premuto **A** non deve far saltare il titolo: ciò che serve è un fronte, non un livello.

17.2 Passo 1: la memoria del gioco

Sostituite le costanti, la `.rodata` e la `.data` della parte 2 con i Listati 17.1, 17.2 e 17.3.

```
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
.equ GREEN, 0x07e0          # the head
.equ BODY,  0x03e0          # the body: a darker green
.equ RED,   0xf800          # the food
.equ CELL,  8               # a cell is 8 x 8 pixels
```

```
.equ COLS,  40
.equ ROWS,  24
.equ TOP,   8
.equ SPEED_Q8, 40          # 40/256 of a cell per frame
.equ BTN_LEFT,  0x01
.equ BTN_RIGHT, 0x02
.equ BTN_UP,    0x04
.equ BTN_DOWN,  0x08
.equ BTN_A,     0x10
.equ DIR_RIGHT, 0          # bit 0 = vertical, bit 1 = backwards
.equ DIR_DOWN,  1
.equ DIR_LEFT,  2
.equ DIR_UP,    3
.equ TITLE, 0              # the three states of the game
.equ PLAY,  1
.equ OVER,  2
```

Listato 17.1. Passo 1 **(modificato)**: le costanti.

C'è un verde più scuro, `0x03e0`, per il corpo; `BTN_A` è il bit 4 della parola dei pulsanti; gli stati sono numeri, 0, 1 e 2.

```
.section .rodata               # constants: they stay in the cartridge file
title:     .asciz "VERM1"
press_a:   .asciz "PRESS A"
game_over: .asciz "GAME OVER"
dx:        .word  1, 0, -1,  0  # right, down, left, up
dy:        .word  0, 1,  0, -1
```

Listato 17.2. Passo 1 **(modificato)**: `.rodata`, ciò che resta nel file.

Tre stringhe e le due tabelle della parte 2: dati di sola lettura, nei quali il programma non scrive mai, e che sulla scheda potrebbero restare in flash.

```
.section .data                 # variables: they live in RAM
state:     .word TITLE
prev_btn:  .word 0             # the buttons one frame ago
dir:       .word 0
want:      .word 0
step_q8:   .word 0
score:     .word 0
best:      .word 0             # lost when the power goes
food_x:    .word 0
food_y:    .word 0
head_i:    .word 0             # ring index of the head
length:    .word 0             # number of cells of the worm
body_x:    .space 256          # the ring: one byte per segment
body_y:    .space 256
grid:      .space COLS*ROWS    # one byte per cell: 1 = worm
```

Listato 17.3. Passo 1 **(modificato)**: `.data`, ciò che vive in RAM.

`head_x` e `head_y` sono sparite: la testa è nell'anello. La direttiva `.space` riserva dei byte senza nominarli uno per uno: $256 + 256 + 960$ byte, il verme e il suo mondo. Tutto ciò

che è qui si perde quando la cartuccia viene scaricata (Capitolo 16): `best` sopravvive a una partita, non a uno spegnimento. Un punteggio migliore che sopravviva richiederebbe una memoria permanente, che una cartuccia portabile non ha: un limite onesto.

17.3 Passo 2: tre stati

Per prima si costruisce la macchina, con funzioni che non fanno ancora niente.

17.3.1 `init`: partire dal titolo

Sostituite `verm1_init` con il Listato 17.4, e cancellate dal file `verm1_update`, `step` e `verm1_draw` della parte 2: i prossimi listati le sostituiscono. Tenete `place_food`, `eat_sound` e `draw_cell`.

```
.section .text
.global verm1_init
.global verm1_update
.global verm1_draw

verm1_init:
    la   t0, state
    sw   zero, 0(t0)            # start on the title screen
    la   t0, prev_btn
    sw   zero, 0(t0)
    ret
```

Listato 17.4. Passo 2 **(modificato)**: `verm1_init`.

`init` sceglie soltanto il primo stato; la partita vera e propria è preparata da `new_game`.

17.3.2 `update`: i pulsanti, e i loro fronti

```
verm1_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    sw   s1, 4(sp)
    call prg32_input_read
    mv   s0, a0                 # s0 = buttons held in this frame
    la   t0, prev_btn
    lw   t1, 0(t0)              # t1 = buttons held one frame ago
    sw   s0, 0(t0)
    not  t1, t1
    and  s1, s0, t1             # s1 = buttons pressed just now
```

Listato 17.5. Passo 2 **(modificato)**: `verm1_update`, prima metà.

Nel Listato 17.5 `s0` contiene i pulsanti di ora e `prev_btn` i pulsanti di un frame fa. La variabile `prev_btn` è un registro nel senso del Capitolo 14: ritarda il suo ingresso di un clock, e qui il clock è il frame. L'espressione «ora AND NOT prima» vale 1 solo dove è cominciata una pressione:

$$\texttt{s1} = \texttt{s0} \cdot \overline{\texttt{prev_btn}}.$$

È il rilevatore di fronte del Capitolo 13: un flip-flop e una porta AND con un ingresso invertito. `s0` e `s1` devono sopravvivere alle chiamate, e sono salvati nel frame.

17.3.3 `update`: un ramo per stato

```
    la   t0, state
    lw   t1, 0(t0)              # t1 = present state
    li   t2, PLAY
    beq  t1, t2, u_play
    andi t3, s1, BTN_A          # TITLE or OVER: wait for a new press of A
    beqz t3, u_done
    li   t2, TITLE
    beq  t1, t2, u_start
    sw   t2, 0(t0)              # OVER --A--> TITLE
    j    u_done
u_start:
    call new_game               # TITLE --A--> PLAY
    j    u_done
u_play:
    mv   a0, s0
    call play_update
u_done:
    lw   s1, 4(sp)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 17.6. Passo 2 **(modificato)**: `verm1_update`, seconda metà.

Il Listato 17.6 è la tabella delle transizioni, riga per riga. Lo stato viene letto per primo: decide chi guarda gli ingressi. `TITLE` e `OVER` reagiscono solo a una nuova pressione di **A**; `PLAY` passa a `play_update` i pulsanti tenuti premuti. La transizione da `OVER` a `TITLE` memorizza soltanto il nuovo stato; quella da `TITLE` a `PLAY` chiama `new_game`, che memorizza essa stessa `PLAY`.

17.3.4 `draw`: che cosa mostrare in ogni stato

```
verm1_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 0
    li   a1, 0
    la   a2, title
    call draw_text
    la   t0, state
    lw   t1, 0(t0)
    li   t2, TITLE
    bne  t1, t2, 1f
    li   a0, 132                # TITLE: only an invitation
    li   a1, 96
    la   a2, press_a
```

```
    call draw_text
    j    2f
1:  call draw_score             # PLAY and OVER: the whole scene
    call draw_worm
    la   t0, food_x
    lw   a0, 0(t0)
    la   t0, food_y
    lw   a1, 0(t0)
    li   a2, RED
    call draw_cell
```

Listato 17.7. Passo 2 **(modificato)**: `verm1_draw`, prima metà.

La riga del titolo è sempre disegnata (Listato 17.7). In `TITLE` si aggiunge solo l'invito `PRESS A`, a $x = 132$, che centra 7 caratteri di 8 pixel; in `PLAY` e `OVER` il punteggio, il verme e il cibo. L'uscita dipende solo dallo stato e dalla memoria: `draw` non cambia niente. È una macchina di Moore.

```
    la   t0, state
    lw   t1, 0(t0)
    li   t2, OVER
    bne  t1, t2, 2f
    li   a0, 124
    li   a1, 96
    la   a2, game_over
    call draw_text
2:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret

draw_text:                      # a0 = x, a1 = y, a2 = address of the string
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a3, WHITE
    li   a4, BLACK
    call prg32_gfx_text8
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 17.8. Passo 2 **(modificato)**: la fine di `verm1_draw`, e `draw_text`.

`OVER` aggiunge due parole sopra la scena (Listato 17.8). La funzione di servizio `draw_text` scrive bianco su nero: riempie `a3` e `a4` e lascia passare da `a0` ad `a2`, togliendo due righe a ogni chiamata che scrive testo.

17.3.5 Quattro funzioni che non fanno niente

```
new_game:                       # temporary: only change the state
    la   t0, state
    li   t1, PLAY
    sw   t1, 0(t0)
    ret

```

```
play_update:                    # temporary: nothing moves yet
    ret

draw_score:                     # temporary
    ret

draw_worm:                      # temporary
    ret
```

Listato 17.9. Passo 2: stub provvisori.

Gli stub del Listato 17.9 fanno esistere i nomi, così che il programma si compili; `new_game` cambia soltanto lo stato. I passi 3 e 4 li sostituiscono uno alla volta. È il metodo del Capitolo 2: cominciare con ritorni che non fanno niente, poi aggiungere una cosa alla volta.

Eseguite: una macchina con niente dentro

1. Compilate ed eseguite: la riga del titolo, e `PRESS A` al centro dello schermo.
2. Premete **A**: l'invito scompare. Una cella rossa è nell'angolo in alto a sinistra: il cibo, alla colonna 0 della riga 0, perché `food_x` e `food_y` partono da zero e lo stub di `new_game` non chiama `place_food`.
3. Niente si muove e la partita non può finire: `PLAY` non ha ancora regole.
4. Nel debugger, trovate la variabile `state` e guardatela cambiare da 0 a 1 quando premete **A**.

17.4 Passo 3: un verme in memoria

17.4.1 `new_game`: svuotare la mappa, posare il verme

Sostituite lo stub di `new_game` con i Listati 17.10 e 17.11.

```
new_game:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, grid
    li   t1, COLS*ROWS
1:  sb   zero, 0(t0)            # empty the grid, one byte per cell
    addi t0, t0, 1
    addi t1, t1, -1
    bnez t1, 1b
    li   t1, 0                  # i = 0, 1, 2: three cells in row 12
2:  la   t0, body_x
    add  t0, t0, t1
    addi t2, t1, 3
    sb   t2, 0(t0)              # columns 3, 4, 5
    la   t0, body_y
    add  t0, t0, t1
    li   t3, 12
    sb   t3, 0(t0)
    la   t0, grid+12*COLS
    add  t0, t0, t2
    li   t3, 1
    sb   t3, 0(t0)              # mark the cell as occupied
```

```
    addi t1, t1, 1
    li   t3, 3
    blt  t1, t3, 2b
```

Listato 17.10. Passo 3 **(nuovo)**: `new_game`, prima metà.

I 960 byte della mappa vengono posti a zero, una `sb` alla volta. Poi tre segmenti vengono posati nelle colonne 3, 4 e 5 della riga 12, e ciascuno è scritto due volte: nell'anello e nella mappa. L'espressione `grid+12*COLS` è calcolata dall'assemblatore: è l'indirizzo del primo byte della riga 12. `sb` memorizza un byte; l'indice è sommato all'indirizzo di base senza scorrimento, perché ogni elemento è un byte.

```
    la   t0, head_i
    li   t1, 2
    sw   t1, 0(t0)              # the head is entry 2 of the ring
    la   t0, length
    li   t1, 3
    sw   t1, 0(t0)
    la   t0, dir
    sw   zero, 0(t0)
    la   t0, want
    sw   zero, 0(t0)
    la   t0, step_q8
    sw   zero, 0(t0)
    la   t0, score
    sw   zero, 0(t0)
    call place_food
    la   t0, state
    li   t1, PLAY
    sw   t1, 0(t0)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 17.11. Passo 3 **(nuovo)**: `new_game`, seconda metà.

`head_i` $= 2$ e `length` $= 3$ descrivono il verme; direzione, timer e punteggio partono da zero. Lo stato è memorizzato per ultimo: la transizione da `TITLE` a `PLAY` è completa solo quando tutto è pronto.

17.4.2 `draw_worm`: percorrere l'anello all'indietro

```
draw_worm:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   s0, 0                  # s0 = i: segment 0 is the head
1:  la   t0, length
    lw   t1, 0(t0)
    bge  s0, t1, 3f
    la   t0, head_i
    lw   t1, 0(t0)
    sub  t1, t1, s0
    andi t1, t1, 255            # ring index of segment i
```

```
    la   t0, body_x
    add  t0, t0, t1
    lbu  a0, 0(t0)
    la   t0, body_y
    add  t0, t0, t1
    lbu  a1, 0(t0)
    li   a2, BODY
    bnez s0, 2f
    li   a2, GREEN              # the head is brighter
2:  call draw_cell
    addi s0, s0, 1
    j    1b
3:  lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 17.12. Passo 3 **(nuovo)**: `draw_worm`.

Il segmento i è l'elemento (`head_i` $- i$) AND 255 (Listato 17.12). `lbu` carica un byte come numero senza segno, e non `lb`: una colonna è un numero da 0 a 39, mai negativo. Qui funzionerebbero entrambe, dato che con `lb` solo un byte di 128 o più diventerebbe negativo (Capitolo 5); ma `lbu` dice ciò che si intende. Il segmento 0 è la testa, in un verde più brillante.

17.4.3 `draw_score`

```
draw_score:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   s0, 0                  # s0 = i: one mark per point
1:  la   t0, score
    lw   t1, 0(t0)
    bge  s0, t1, 2f
    slli a0, s0, 2
    addi a0, a0, 56             # x = 56 + 4 * i
    li   a1, 1
    li   a2, 3
    li   a3, 6
    li   a4, WHITE
    call prg32_gfx_rect
    addi s0, s0, 1
    j    1b
2:  lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 17.13. Passo 3 **(nuovo)**: `draw_score`.

È il ciclo che stava dentro `verm1_draw` nella parte 2, ora una funzione (Listato 17.13). Usa `s0`, quindi lo salva e lo ripristina. Resta uno stub (Listato 17.14): il verme esiste, e sta fermo.

```
play_update:                    # temporary: nothing moves yet
    ret
```

Listato 17.14. Passo 3: l'ultimo stub.

Eseguite: un verme che sta fermo

1. Compilate ed eseguite, premete **A**: un verme di tre celle nella riga 12, colonne da 3 a 5, con la testa, più brillante, a destra.
2. Il cibo è ora in una cella casuale: `new_game` chiama `place_food`.
3. Nel debugger, fermatevi all'inizio di `draw_worm` e procedete passo-passo finché è stata eseguita `la t0, body_x`: `t0` contiene l'indirizzo di `body_x`. Scrivetelo nel campo *Memory*: i primi byte sono `03 04 05`.
4. Trovate `grid` $+ 12 \times 40 + 3$: tre byte a `01` in un mare di zeri. (`grid` è 512 byte dopo `body_x`, e la riga 12 comincia 480 byte dopo `grid`.)

17.5 Passo 4: le regole

Un passo del verme ha tre movimenti: decidere, guardare, scrivere.

17.5.1 `play_update`: la update della parte 2

Sostituite l'ultimo stub con i Listati 17.15 e 17.16. Sono il corpo della vecchia `verm1_update` sotto un nuovo nome; i pulsanti arrivano in `a0`, così che non c'è alcuna chiamata per leggerli.

```
play_update:                    # a0 = buttons held
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, dir
    lw   t1, 0(t0)              # t1 = direction of the last step
    la   t0, want
    lw   t2, 0(t0)              # t2 = wanted direction, so far
    andi t3, a0, BTN_RIGHT
    beqz t3, 1f
    li   t2, DIR_RIGHT
1:  andi t3, a0, BTN_DOWN
    beqz t3, 2f
    li   t2, DIR_DOWN
2:  andi t3, a0, BTN_LEFT
    beqz t3, 3f
    li   t2, DIR_LEFT
3:  andi t3, a0, BTN_UP
    beqz t3, 4f
    li   t2, DIR_UP
4:  xor  t3, t2, t1             # opposite directions differ in bit 1 only
    li   t4, 2
    beq  t3, t4, 5f             # a U-turn: ignore the request
    sw   t2, 0(t0)
5:
```

Listato 17.15. Passo 4 **(modificato)**: `play_update`, la guida.

```
    la   t0, step_q8
    lw   t1, 0(t0)
    addi t1, t1, SPEED_Q8       # add a fraction of a cell
    li   t2, 256
    blt  t1, t2, 6f             # less than one whole cell: wait
    sub  t1, t1, t2
    sw   t1, 0(t0)
    call step                   # one whole cell: move
    j    7f
6:  sw   t1, 0(t0)
7:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 17.16. Passo 4 **(modificato)**: `play_update`, il timer.

17.5.2 `step`: tre registri da conservare

```
step:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)              # s0 = column of the new head
    sw   s1, 4(sp)              # s1 = row of the new head
    sw   s2, 0(sp)              # s2 = 0 when the worm eats
    la   t0, want
    lw   t1, 0(t0)
    la   t0, dir
    sw   t1, 0(t0)              # the wanted direction is now the direction
    slli t1, t1, 2              # table offset = direction * 4 bytes
    la   t0, dx
    add  t0, t0, t1
    lw   t2, 0(t0)              # t2 = dx[direction]
    la   t0, dy
    add  t0, t0, t1
    lw   t3, 0(t0)              # t3 = dy[direction]
```

Listato 17.17. Passo 4 **(modificato)**: `step`, l'inizio.

A `step` servono tre valori che devono sopravvivere a una chiamata a `place_food`: la cella della nuova testa, in `s0` e `s1`, e un indicatore in `s2` che vale zero quando il verme mangia. Sono tenuti in registri `s`, e il frame ha spazio per quattro parole: `ra` e i tre registri salvati (Listato 17.17). La ricerca nella tabella è quella della parte 2.

17.5.3 `step`: dove sarà la testa?

```
    la   t0, head_i
    lw   t5, 0(t0)              # t5 = ring index of the head
    la   t0, body_x
    add  t0, t0, t5
    lbu  s0, 0(t0)              # one byte: the column of the head
    add  s0, s0, t2
    bgez s0, 1f
    li   s0, COLS-1             # off the left edge: wrap
```

```
1:  li   t4, COLS
    blt  s0, t4, 2f
    li   s0, 0
2:  la   t0, body_y
    add  t0, t0, t5
    lbu  s1, 0(t0)
    add  s1, s1, t3
    bgez s1, 3f
    li   s1, ROWS-1
3:  li   t4, ROWS
    blt  s1, t4, 4f
    li   s1, 0
4:
```

Listato 17.18. Passo 4 **(modificato)**: `step`, la nuova cella.

La testa attuale è l'elemento `head_i` dell'anello: un byte ciascuno per colonna e riga, caricati con `lbu`. Si sommano `dx` e `dy` e il risultato viene fatto rientrare come nella parte 2 (Listato 17.18). Non viene ancora scritto niente: prima il programma deve guardare.

17.5.4 `step`: mangiare, o spostare la coda

```
    la   t0, food_x
    lw   t1, 0(t0)
    xor  t1, t1, s0
    la   t0, food_y
    lw   t2, 0(t0)
    xor  t2, t2, s1
    or   s2, t1, t2            # s2 = 0 if the new head is on the food
    la   t0, length
    lw   t1, 0(t0)
    bnez s2, 5f                # not eating: the tail moves on
    li   t2, 255
    bge  t1, t2, 5f            # the ring is full: no more growth
    addi t1, t1, 1
    sw   t1, 0(t0)             # eating: one cell longer, the tail stays
    j    6f
5:  la   t0, head_i
    lw   t2, 0(t0)
    sub  t2, t2, t1
    addi t2, t2, 1
    andi t2, t2, 255           # ring index of the tail
    la   t0, body_x
    add  t0, t0, t2
    lbu  t3, 0(t0)
    la   t0, body_y
    add  t0, t0, t2
    lbu  t4, 0(t0)
```

Listato 17.19. Passo 4 **(modificato)**: `step`, la coda.

Il comparatore della parte 2 lascia il suo risultato in `s2` (Listato 17.19). Se il verme mangia, la lunghezza cresce di uno e la coda resta dov'è. Se non mangia, la coda si trova all'elemento

`head_i` − `length` + 1, e la sua colonna e la sua riga vengono messe in `t3` e `t4`. A 255 celle il verme smette di crescere.

Concetto

Crescere e non spostare la coda sono la stessa cosa. È tutto il trucco di Snake.

17.5.5 `step`: liberare la coda, esaminare la testa

```
    li   t5, COLS
    mul  t4, t4, t5
    add  t4, t4, t3            # cell number of the tail = row * 40 + column
    la   t0, grid
    add  t0, t0, t4
    sb   zero, 0(t0)           # the tail cell is free again
6:  li   t5, COLS
    mul  t4, s1, t5
    add  t4, t4, s0            # cell number of the new head
    la   t0, grid
    add  t0, t0, t4
    lbu  t1, 0(t0)
    beqz t1, 7f                # a free cell: go on
    la   t0, state
    li   t1, OVER
    sw   t1, 0(t0)             # the worm bit itself: PLAY --> OVER
    la   t0, score
    lw   t1, 0(t0)
    la   t0, best
    lw   t2, 0(t0)
    ble  t1, t2, 9f
    sw   t1, 0(t0)             # a new best score
    j    9f
```

Listato 17.20. Passo 4 **(nuovo)**: `step`, la collisione.

Il numero di un byte della mappa è *riga* × 40 + *colonna*: una `mul` e una `add` (Listato 17.20); `mul` appartiene all'estensione M di RISC-V, che l'ESP32-C6 ha. Il byte della coda viene azzerato *per primo*, e poi si esamina il byte della testa. L'ordine conta: in questo modo la testa può entrare nella cella che la coda ha appena lasciato, e il verme può inseguire la propria coda, come nel gioco originale. Se la cella è occupata lo stato diventa `OVER`, e `best` viene aggiornato.

17.5.6 `step`: scrivere la nuova testa

```
7:  li   t1, 1
    sb   t1, 0(t0)             # occupy the new cell
    la   t0, head_i
    lw   t1, 0(t0)
    addi t1, t1, 1
    andi t1, t1, 255           # the ring wraps after entry 255
    sw   t1, 0(t0)
    la   t0, body_x
    add  t0, t0, t1
```

```
    sb   s0, 0(t0)              # write the new head into the ring
    la   t0, body_y
    add  t0, t0, t1
    sb   s1, 0(t0)
    bnez s2, 9f
    la   t0, score
    lw   t1, 0(t0)
    addi t1, t1, 1
    sw   t1, 0(t0)
    call place_food
    call eat_sound
9:  lw   s2, 0(sp)
    lw   s1, 4(sp)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 17.21. Passo 4 **(nuovo)**: la fine di `step`.

La cella è segnata nella mappa, mentre `t0` punta ancora a essa; l'indice `head_i` diventa (`head_i` + 1) AND 255, e l'anello si chiude; colonna e riga sono scritte nel nuovo elemento (Listato 17.21). Se `s2` vale zero ci sono un punto, nuovo cibo e un suono. Dopo una collisione la funzione salta direttamente alla sua ultima etichetta: non viene scritto niente, e lo schermo mostra il verme com'era prima del passo fatale.

17.5.7 `place_food`: non sul verme

```
place_food:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
1:  li   a0, 0
    li   a1, COLS-1
    call prg32_random_number
    mv   s0, a0                 # s0 = a column
    li   a0, 0
    li   a1, ROWS-1
    call prg32_random_number    # a0 = a row
    li   t1, COLS
    mul  t2, a0, t1
    add  t2, t2, s0
    la   t0, grid
    add  t0, t0, t2
    lbu  t3, 0(t0)
    bnez t3, 1b                 # on the worm: try again
    la   t0, food_x
    sw   s0, 0(t0)
    la   t0, food_y
    sw   a0, 0(t0)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 17.22. Passo 4 **(modificato)**: `place_food`.

Si estrae a caso una cella, e poi si consulta la mappa: se la cella è occupata se ne estrae un'altra, finché ne arriva una libera (Listato 17.22). La colonna aspetta in `s0` durante la seconda chiamata. Il ciclo termina in fretta: anche con 255 celle occupate su 960, la media è inferiore a due tentativi.

Eseguite: Verm1, parte 3

Il verme segue la testa e cresce (Figura 17.1b); svoltare dentro il corpo fa finire la partita (Figura 17.1c); **A** riporta al titolo, e **A** di nuovo avvia una nuova partita.

17.6 Passo 5: guardare la memoria che si muove

Guardate l'anello girare

1. Eseguite in modalità debug, premete **A** per avviare una partita, poi mettete in pausa.
2. Premete **Update** e mettete un breakpoint sulla prima istruzione di `step`.
3. **Resume**: il programma si ferma una volta per passo del verme, non una volta per frame.
4. Scrivete l'indirizzo di `body_x` nel campo *Memory*: i byte `03 04 05` della partenza, poi `06`, `07`, …, un nuovo byte per passo.
5. Scendete per qualche passo: `body_x` ripete la stessa colonna, e `body_y`, 256 byte più avanti, conta le righe.
6. Mangiate, e confrontate `head_i` e `length` prima e dopo: entrambi crescono di uno.

I byte delle vecchie posizioni non vengono mai cancellati: l'anello conserva l'intera storia degli ultimi 256 passi, e `length` dice quanta di essa è ancora verme. Dopo 256 passi la scrittura ricomincia dal primo byte.

17.6.1 Cogliere il fronte del pulsante A

Mettete un breakpoint su `and s1, s0, t1` in `verm1_update`, tenete premuto **A** e premete **Resume** tre volte. La Tabella 17.2 mostra ciò che si legge: `s1` vale `0x10` in un solo frame, il frame in cui il pulsante si abbassa. (Il test del file di riferimento tiene premuto **A** per 28 frame e verifica che avvenga esattamente una transizione.)

Tabella 17.2. Il fronte del pulsante **A**.

Frame	A è	`s0` (ora)	`prev_btn`	`s1` = `s0` AND NOT prev	Effetto
n	alzato	`0x00`	`0x00`	`0x00`	–
$n+1$	abbassato	`0x10`	`0x00`	`0x10`	transizione
$n+2$	abbassato	`0x10`	`0x10`	`0x00`	–
$n+3$	alzato	`0x00`	`0x10`	`0x00`	–

17.6.2 Tre domande per il debugger

- *Dov'è `state`?* Dopo `la t0, state` in `verm1_update`, `t0` ne contiene l'indirizzo. Guardate `00`, `01`, `02` mentre giocate e perdete.
- *La mappa è coerente con l'anello?* Mettete in pausa, leggete la testa da `body_x` e `body_y`, calcolate *riga* $\times 40 +$ *colonna*, e guardate quel byte di `grid`.
- *Che cosa ha visto il passo fatale?* Fermatevi sulla `lbu` che segue la seconda `mul` in `step`: `t1` vale 1 esattamente una volta, al morso. In quel momento `s0` e `s1` contengono la cella in cui la testa voleva entrare, e lo schermo mostra che fa parte del corpo.

Concetto

Un programma è giusto quando la sua memoria è giusta. Imparate a leggerla.

17.7 Verificate di aver capito

D17.1 `head_i` $= 1$ e `length` $= 4$. Quali elementi dell'anello contengono il verme?

A. 1, 2, 3, 4
B. 0, 1, 2, 3
C. 254, 255, 0, 1
D. 253, 254, 255, 0

D17.2 Quale byte di `grid` descrive la cella in colonna 7 della riga 3?

A. il byte 37
B. il byte 127
C. il byte 283
D. il byte 307

D17.3 Il giocatore tiene premuto **A** per due secondi sulla schermata del titolo. Che cosa accade?

A. la partita comincia e finisce subito
B. la partita comincia una volta
C. nulla, finché **A** non viene rilasciato
D. gli stati si alternano a ogni frame

D17.4 Perché il byte della coda viene azzerato prima di esaminare quello della testa?

A. per risparmiare un'istruzione
B. perché la testa possa entrare nella cella lasciata dalla coda
C. perché `sb` è più veloce di `lbu`
D. l'ordine non conta

17.8 Fatelo vostro

Esercizio 17.1 (su PRG32)

1. Muri: il verme muore quando esce dallo schermo invece di rientrare.
2. Partite con un verme di cinque celle; poi fate dipendere la lunghezza iniziale dal punteggio migliore.
3. Veleno: una seconda cella, blu, che accorcia il verme di una cella.
4. Nella schermata di fine partita mostrate il punteggio migliore come tacche sulla seconda riga.

Risultato atteso. Ogni modifica tocca una o due funzioni. Tenete una copia della parte 3 funzionante: la parte 4 parte da lì. (Suggerimenti: per i muri, sostituite ogni rientro in `step` con un salto al codice che memorizza `OVER`. Per il veleno, spostate la coda due volte in un passo, azzerando entrambi i byte della mappa, e non scendete mai sotto la lunghezza 1.)

17.9 Due funzioni che non sono cambiate

`eat_sound` e `draw_cell` sono quelle della parte 2 e restano come sono. Sono ristampate qui perché questo capitolo contenga l'intero programma: il file completo è la somma dei listati segnati nuovo e modificato, più questi due.

```
eat_sound:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0                  # channel
    li   a1, 0                  # instrument
    li   a2, 72                 # note: C5
    li   a3, 255                # volume
    li   a4, 60                 # milliseconds
    call prg32_audio_note
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 17.23. Invariato: `eat_sound`.

```
draw_cell:                      # a0 = column, a1 = row, a2 = colour
    addi sp, sp, -16
    sw   ra, 12(sp)
    slli a0, a0, 3              # x = column * 8
    slli a1, a1, 3
    addi a1, a1, TOP            # y = row * 8 + 8
    mv   a4, a2                 # the colour is the fifth argument
    li   a2, CELL
    li   a3, CELL
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 17.24. Invariato: `draw_cell`.

17.10 Dove siamo, e che cosa viene dopo

Verm1 è ora un gioco: ha uno scopo, un rischio, un inizio e una fine. Una macchina con tre stati, un anello di byte, una mappa: logica sequenziale e memoria scritte come programma, in circa 450 righe di assembly che avete scritto e sapete spiegare. Mancano tre cose: il programma ridisegna l'intero schermo a ogni frame, il punteggio è una fila di tacche, e la velocità non cambia mai. I prossimi quattro capitoli aprono la macchina che esegue quelle righe: come il processore esegue, le chiamate e la divisione, come funzionano video e suono. La parte 4 è il gioco finito.

Parte V

La macchina

CAPITOLO 18

Il processore

"Un Anello per domarli tutti..."
— J. R. R. Tolkien, *Il Signore degli Anelli*, 1954–1955

In questo capitolo

Alla fine di questo capitolo saprete

- nominare i componenti di un processore: unità di controllo, ALU, registri, collegamenti con la memoria e con l'ingresso/uscita;
- distinguere i registri interni dai registri di macchina, e i registri generici da quelli speciali;
- spiegare come funziona uno stack e come RISC-V realizza push e pop;
- confrontare i modelli ad accumulatore, a stack e a registri generali sullo stesso calcolo;
- descrivere RISC-V come un processore load-store a registri generali, e leggere il nome RV32IMAC;
- far corrispondere ogni elemento del modello generale al processore del kit PRG32.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI accumulatore, pila, registri; il modello del processore; i registri di RISC-V; la convenzione di chiamata; la pila; il processore RV32.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI interruzioni e trap (Capitolo 19).

Programmate un processore fin dal Capitolo 2. Questo capitolo presenta il modello generale di un processore, di cui ogni processore reale è una variante, e usa RISC-V come riferimento: ogni elemento astratto ha un nome che conoscete già.

18.1 Riscaldamento sulla macchina

```
1     la   t0, x
```

```
    lw   t1, 0(t0)       # bring x into a register
    la   t0, y
    lw   t2, 0(t0)       # bring y into a register
    add  t3, t1, t2      # compute, between registers
    la   t0, z
    sw   t3, 0(t0)       # take the result back to memory

.section .data
x: .word 5
y: .word 7
z: .word 0
```

Listato 18.1. Una riga di un linguaggio ad alto livello: `z = x + y`.

Sette righe per una somma (Listato 18.1). La memoria è toccata solo da `lw` e `sw`, e l'aritmetica avviene solo tra registri: è il ciclo «carica, modifica, memorizza» del Capitolo 2, visto ora come una scelta di progetto. Altri processori fanno lo stesso lavoro con tre istruzioni, o con una. Perché RISC-V è fatto così? Per rispondere serve un modello del processore.

18.2 Il processore

18.2.1 Il calcolatore e il suo processore

Un calcolatore elettronico è un sistema numerico, automatico, a programma memorizzato, costruito con circuiti elettronici (Capitolo 8), con tre sottosistemi: processore, memoria centrale e ingresso/uscita, collegati da bus. Il processore può eseguire solo un insieme fisso di istruzioni, da alcune decine a poche centinaia, più o meno complesse. Il *linguaggio macchina* è l'insieme delle stringhe di bit che codificano le istruzioni che i circuiti di un processore possono eseguire. Le istruzioni richiedono trasferimenti di informazione, all'interno del calcolatore e da o verso i dispositivi di I/O, e operazioni aritmetiche e logiche. I dati e le istruzioni di un programma sono memorizzati nella memoria centrale.

RV32I, la base di RISC-V, ha circa quaranta istruzioni; con le estensioni M, A e C del kit, circa un centinaio di codifiche. Il Motorola 68000 aveva cinquantasei tipi di istruzione, con molte varianti di ciascuno.

18.2.2 Un modello di riferimento

La Figura 18.1 è il modello di riferimento di un processore. I suoi elementi sono i seguenti.

- La *rete di controllo*, o unità di controllo, che produce i comandi per tutto il resto.
- Il *program counter* PC, che contiene l'indirizzo della prossima istruzione, e il *registro di decodifica* D, che contiene l'istruzione in esecuzione.
- I *registri di macchina* $R_0, \ldots, R_{n-1}$.
- La ALU, con i suoi registri di ingresso O_1 e O_2 e il suo registro di uscita U.
- Il collegamento con la memoria: il registro indirizzo MA e il registro buffer MB del Capitolo 15.
- Il collegamento con l'ingresso/uscita: un registro di ingresso RI e un registro di uscita RU.
- Un *bus interno* che li unisce tutti.

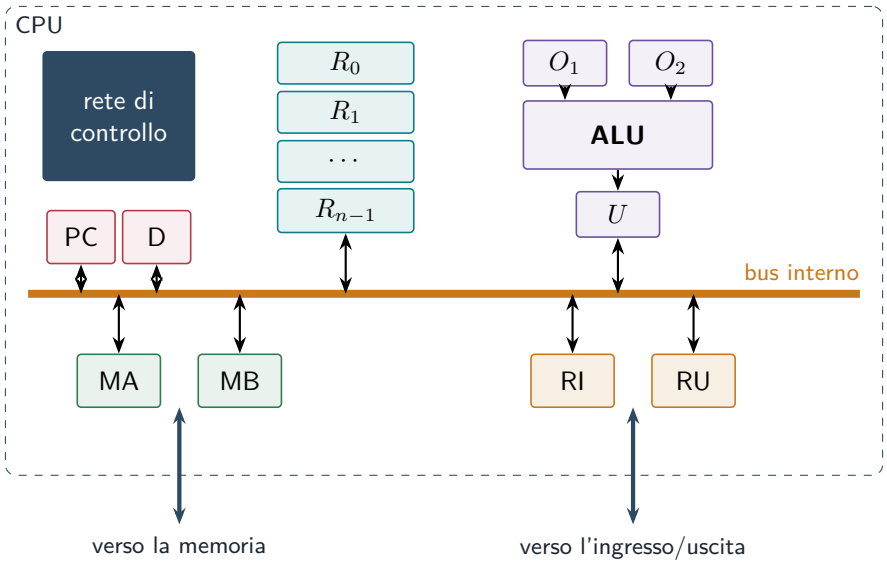


Figura 18.1. Il modello di riferimento di un processore.

Pur essendo basati sullo stesso schema logico, e quindi fatti di componenti piuttosto simili, i processori in commercio possono essere molto diversi l'uno dall'altro. La Tabella 18.1 mette fianco a fianco il modello e RISC-V. Le due righe segnate «assente» sono semplificazioni deliberate di RISC-V, discusse più avanti e nel Capitolo 15. L'MC68000 delle edizioni precedenti di questo corso aveva tutti gli elementi del modello: otto registri dati e otto registri indirizzi, un registro di stato con i codici di condizione, e una circuiteria separata per le interruzioni.

Tabella 18.1. Il modello e RISC-V.

Elemento del modello	**In RISC-V**	**Visto dal programmatore?**
PC, program counter	`pc`, 32 bit	indirettamente: lo usano i salti e `auipc`
D, registro di decodifica	il registro istruzione	no
$R_0 \dots R_{n-1}$, registri di macchina	`x0`...`x31`, trentadue registri di 32 bit	sì: `zero`, `ra`, `sp`, `t0`, `a0`, `s0`, ...
O_1, O_2, U, registri della ALU	gli ingressi e l'uscita della ALU	no
MA, MB, collegamento con la memoria	l'indirizzo e il dato di un load o di uno store	no
RI, RU, collegamento con l'ingresso/uscita	assente: i dispositivi sono mappati in memoria	–
registro di stato con i flag	assente: nessun codice di condizione	–
registri speciali	i registri di controllo e stato (CSR)	solo dal software privilegiato

18.2.3 Registri interni e registri di macchina

Definizione 18.1 (Registri interni e di macchina)

I *registri interni* non sono direttamente visibili al programmatore, e sono necessari al funzionamento del processore: PC, MA, MB, D, O_1, O_2, U. I *registri di macchina* definiscono il modello di programmazione; si dividono in registri *generici* e registri *speciali*.

Un processore tratta tre tipi di informazione: istruzioni, indirizzi di memoria e dati. La dimensione di MA e del PC dipende dalla dimensione degli indirizzi; la dimensione dei registri della ALU dipende dalla dimensione dei dati. In RV32 tutte e tre sono larghe 32 bit: una sola dimensione per registri, indirizzi e istruzioni. Il «32» del nome è la larghezza dei registri: RV64 ha registri e indirizzi di 64 bit, e le stesse istruzioni di 32 bit.

18.2.4 In che cosa i processori reali differiscono dal modello

In generale, i processori reali possono avere un insieme di registri generici diverso e di solito più ricco; i loro registri di macchina possono essere specifiche locazioni di memoria; la sezione di I/O può mancare del tutto ed essere realizzata da chip dedicati; e unità aritmetiche molto semplici possono essere estese da chip dedicati che migliorano la ALU, come facevano i coprocessori in virgola mobile 68881 per la famiglia 68000 e 8087 per l'8086.

RISC-V sul kit PRG32 ha trentadue registri generici in un banco dentro il processore; nessuna sezione di I/O, dato che i controller delle periferiche sono sullo stesso chip e si raggiungono come memoria; *estensioni* opzionali al posto di chip aggiuntivi, tra le quali M aggiunge moltiplicazione e divisione; e nessuna estensione F, così che la virgola mobile andrebbe fatta in software (Capitolo 6).

18.2.5 Parole o byte: la lunghezza delle parole di macchina

In un *processore a parola* tutte le parole di macchina hanno la stessa lunghezza, dimensionata sull'informazione più lunga; questo è scomodo con dati di natura intrinsecamente variabile. Ogni istruzione RISC-V è lunga 32 bit (16 nella forma compressa). In un processore *a carattere*, o a byte, la lunghezza delle parole di macchina è variabile, un multiplo di un byte, il che si adatta al trattamento delle stringhe: un'istruzione Intel x86 occupa da 1 a 15 byte, e un'istruzione MC68000 da una a cinque parole di 16 bit. Un parametro caratteristico di un processore è il numero di bit dei suoi registri, il parallelismo dei suoi trasferimenti interni: 8, 16, 32 o 64.

18.2.6 Stato utente e stato supervisore

Un processore è in *stato supervisore* quando esegue il sistema operativo, e può allora eseguire ogni istruzione del suo insieme; è in *stato utente* quando esegue un programma utente, e può allora eseguire tutto tranne le *istruzioni privilegiate*.

RISC-V definisce tre *modi di privilegio*. Il modo macchina (M), il più privilegiato e l'unico obbligatorio, esegue il firmware. Il modo supervisore (S) esegue il kernel di un sistema operativo come Linux. Il modo utente (U) esegue le applicazioni. L'ESP32-C6 realizza M e U. Il passaggio da un modo all'altro avviene attraverso le trap, argomento del Capitolo 19.

18.3 I registri

18.3.1 Tipi di registri

Un registro può essere specializzato, o può fare più lavori. I registri specializzati richiedono codici operativi specializzati per la stessa operazione su banchi diversi; concentrare più funzioni negli stessi registri dà maggiore complessità interna e maggiore flessibilità d'uso. I registri di macchina, diversi in numero e spesso raggruppati in banchi, possono essere organizzati come una memoria indirizzabile (un banco R, di cui R_i è un registro) o come registri autonomi, ciascuno con il proprio nome. Alcuni processori collocavano i loro registri generici nelle prime locazioni di memoria: la «pagina zero» del 6502.

RISC-V ha un solo banco di trentadue registri, indirizzato da 5 bit dentro l'istruzione; i nomi `t0`, `a0`, `sp` sono solo convenzioni.

18.3.2 Le funzioni dei registri

Transito dei dati.
Dalla memoria a un registro, da registro a registro, da un registro alla memoria: load, move, store.

Accumulo dei risultati.
Necessario per l'aritmetica e la logica: $R := \mathrm{op}\, R$, oppure $R := R \mathrm{\ op\ } X$.

Indirizzamento.
Trovare l'indirizzo di un operando: registri indice, registri base.

Puntatori allo stack.
Gestire uno stack realizzato in memoria.

Indicatori, o flag.
Informazione binaria su eventi: zero, segno, overflow, riporto, indirizzi o codici illegali. Prodotti durante l'esecuzione di un'istruzione, possono condizionare le successive.

In RISC-V le prime quattro funzioni sono tutte svolte dai registri generici; la quinta non esiste come registro.

18.3.3 I trentadue registri di RISC-V

Tabella 18.2. I registri di RISC-V e i loro ruoli convenzionali.

Registro	Nome	Funzione	Registro	Nome	Funzione
`x0`	`zero`	sempre 0	`x8`–`x9`	`s0`–`s1`	salvati
`x1`	`ra`	indirizzo di ritorno	`x10`–`x11`	`a0`–`a1`	argomenti, risultati
`x2`	`sp`	puntatore allo stack	`x12`–`x17`	`a2`–`a7`	argomenti
`x3`	`gp`	puntatore globale	`x18`–`x27`	`s2`–`s11`	salvati
`x4`	`tp`	puntatore al thread	`x28`–`x31`	`t3`–`t6`	temporanei
`x5`–`x7`	`t0`–`t2`	temporanei	`pc`	–	program counter

L'hardware tratta in modo identico i registri da `x1` a `x31`, e solo `x0` è speciale (Tabella 18.2); il program counter non è nel banco. I ruoli vengono dalla convenzione di chiamata, incontrata nel Capitolo 2, la cui regola è questa. Una chiamata può distruggere `ra` e i registri `t` e `a`: sono *salvati dal chiamante*, e un chiamante che ne ha ancora bisogno li salva prima della chiamata. Una chiamata deve restituire invariati i registri `s` e `sp`: sono *salvati dal chiamato*,

e una funzione che ne usa uno lo salva nel suo frame sullo stack e lo ripristina prima di `ret`. L'Appendice A dà la convenzione completa. L'MC68000, invece, separava i registri dati D0–D7 dai registri indirizzi A0–A7, con A7 come puntatore allo stack.

18.3.4 I flag: il registro di stato, e la sua assenza

Molti processori hanno un *registro di stato*: bit, chiamati flag, accessibili dal programma. Quelli usuali sono Z (zero), N (negativo), V (overflow) e C (riporto), talvolta con un bit di parità. Un'istruzione li imposta, e un successivo salto condizionato li esamina. L'MC68000 ha il suo registro dei codici di condizione, e x86 il suo registro dei flag.

RISC-V non ha flag. Un salto confronta due registri e salta, in una sola istruzione: `beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`. Le istruzioni `slt` e `sltu` scrivono il risultato di un confronto in un registro generico. Riporto e overflow si calcolano quando servono (Capitolo 5). C'è meno stato nascosto, il che rende più semplici le pipeline e le interruzioni.

18.3.5 I registri speciali di RISC-V: i CSR

I registri speciali di RISC-V sono i *registri di controllo e stato* (CSR, *control and status registers*): un banco separato di un massimo di 4 096 registri, letti e scritti solo con le istruzioni `csrrw`, `csrrs` e `csrrc`, e solo dal codice privilegiato (Tabella 18.3). Sono i «registri delle interruzioni» del modello generale, e l'argomento del Capitolo 19.

Tabella 18.3. Alcuni CSR del modo macchina.

Registro	Nome	Che cosa contiene
`mstatus`	machine status	l'abilitazione globale delle interruzioni e il modo di privilegio precedente
`mtvec`	trap vector	l'indirizzo della routine che gestisce interruzioni ed eccezioni
`mepc`	exception program counter	l'indirizzo dell'istruzione che è stata interrotta
`mcause`	trap cause	un codice che dice perché è avvenuta la trap
`mie`, `mip`	interrupt enable, interrupt pending	un bit per sorgente di interruzione

18.4 Lo stack

18.4.1 L'ultimo a entrare è il primo a uscire

Definizione 18.2 (Stack)

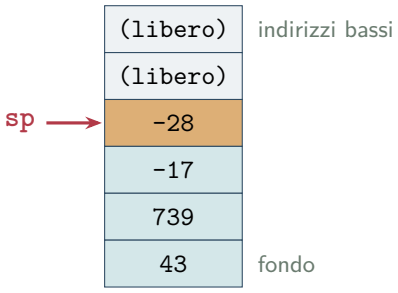
Uno *stack*, o pila, è una struttura dati astratta con due operazioni: *push* memorizza un elemento come ultimo della pila, e *pop* toglie l'ultimo elemento. L'ultimo valore inserito è il primo estratto (LIFO, *last in, first out*).

Uno stack lavora sempre e solo sulla sua cima: non c'è bisogno di dire dove memorizzare o da dove prelevare. Si usa per mettere da parte il contenuto dei registri che vanno liberati per svolgere un compito, e per ripristinarlo dopo: accade nei sottoprogrammi e nelle interruzioni. È realizzato con un puntatore, in memoria o in un registro dedicato, che indica in ogni

istante la posizione dell'ultimo elemento. Le operazioni su uno stack dovrebbero controllare l'overflow su un push e lo stack vuoto su un pop.

18.4.2 Push e pop in RISC-V

In RISC-V lo stack vive in memoria, cresce verso gli indirizzi bassi, e il registro `sp` punta alla sua cima (Figura 18.2). Non ci sono istruzioni chiamate push e pop: ciascuna è fatta di due istruzioni ordinarie.




```
# push t0
addi sp, sp, -4       # make room
sw   t0, 0(sp)        # store on the new top

# pop t0
lw   t0, 0(sp)        # read the top
addi sp, sp, 4        # release it
```

Figura 18.2. Uno stack in memoria, e le due operazioni in RISC-V.

In pratica un programma RISC-V non inserisce ed estrae valori singoli: uno *stack frame* è allocato una volta per funzione, sedici byte alla volta, ed è riempito con store ordinari, come ha fatto ogni funzione di Verm1 che fa una chiamata. Altri processori hanno forme dedicate: sul 68000, `MOVE (SP)+, D0` estrae con un post-incremento.

18.5 Modelli di processore

I processori si classificano secondo il luogo in cui vivono gli operandi di un'operazione.

18.5.1 Quattro architetture

A registri generali.
Il processore ha un insieme di registri che possono essere usati in modo intercambiabile.

Ad accumulatore.
Tutte le istruzioni aritmetiche, logiche e di confronto hanno un operando implicito: un registro speciale, l'accumulatore.

A stack.
Non ci sono né accumulatori né registri generali, solo il puntatore allo stack; le istruzioni lavorano sui dati in cima allo stack.

Ortogonale.
Un solo banco di registri generali, alcuni dei quali servono, quando occorre, da puntatore allo stack o da altri registri speciali.

18.5.2 `z = x + y` in quattro processori

Gli accessi alla memoria sono dappertutto gli stessi tre (Tabella 18.4): ciò che cambia è quante istruzioni ci sono, e quanto è complessa ciascuna. Nel modello registro-memoria un'istruzione come `ADD y, D0` legge la memoria *e* calcola; nel modello load-store ogni istruzione fa una cosa sola. Le macchine a stack con lo stack in memoria sono generalmente più lente delle macchine a registri generali. (Nella tabella, `lw t1, x` è un'abbreviazione che l'assemblatore accetta per la coppia `la` e `lw` del riscaldamento.)

Tabella 18.4. Lo stesso calcolo in quattro modelli.

Modello	Esempio	Programma	Accessi
accumulatore	Motorola 6809, MOS 6502	`LDA x; ADD y; STA z`	3
stack	macchina virtuale Java, WebAssembly	`PUSH x; PUSH y; ADD; POP z`	3 + stack
registri generali, operandi in memoria	MC68000, Intel x86	`MOVE x,D0; ADD y,D0; MOVE D0,z`	3
registri generali, load-store	RISC-V	`lw t1,x; lw t2,y; add t3,t1,t2; sw t3,z`	3

18.5.3 RISC-V: un processore load-store a registri generali

Caricare, calcolare, memorizzare

Solo i load e gli store toccano la memoria; ogni altra istruzione lavora da registro a registro. È il ritmo di ogni programma RISC-V, dal primo contatore all'ultimo gioco.

Le ragioni sono quattro. Le istruzioni che lavorano sui registri sono più veloci di quelle che lavorano su locazioni di memoria, così che i dati più usati si tengono nei registri. Un'istruzione ha tre operandi, due sorgenti e una destinazione, così che nessun valore viene distrutto a meno che il programmatore non lo voglia. Ogni istruzione fa una cosa semplice e ha una dimensione fissa: è facile da decodificare, e facile da eseguire in una pipeline. È questo il significato di RISC, *reduced instruction set computer*: le forme che vanno dalla memoria a un registro e da un registro alla memoria, che esistono nei processori CISC, sono esattamente ciò a cui il RISC rinuncia.

18.6 La ALU e il quadro d'insieme

18.6.1 L'unità aritmetico-logica

La ALU del Capitolo 11, vista dal processore, sovrintende alle operazioni aritmetiche e logiche tra operandi. Nel modello ha due registri di ingresso, O_1 e O_2, e un registro di uscita U. È fisicamente inclusa nel processore nelle architetture elementari ed è un apparato indipendente in quelle più complesse; una soluzione diffusa è un'unità aritmetica ridotta dentro il processore con apparati indipendenti per le operazioni più sofisticate. In RISC-V quell'apparato sono le estensioni: M per moltiplicazione e divisione, F e D per la virgola mobile. Le operazioni disponibili dipendono dai tipi di dati e dalla loro rappresentazione: valori logici, stringhe di caratteri, puntatori, e numeri nelle codifiche dei Capitoli 5 e 6.

18.6.2 L'architettura di un processore RV32

La Figura 18.3 riduce ai suoi blocchi il classico datapath RISC-V a ciclo singolo di Patterson e Hennessy (2020). Il PC dà l'indirizzo dell'istruzione; l'istruzione viene prelevata dalla memoria; il decodificatore ne separa i campi e l'unità di controllo produce i comandi; il banco dei registri fornisce gli operandi; la ALU calcola un risultato o forma un indirizzo; la memoria dati viene letta o scritta; e il risultato torna al banco dei registri. La memoria

istruzioni e la memoria dati sono lo stesso spazio degli indirizzi, disegnato due volte per chiarezza.

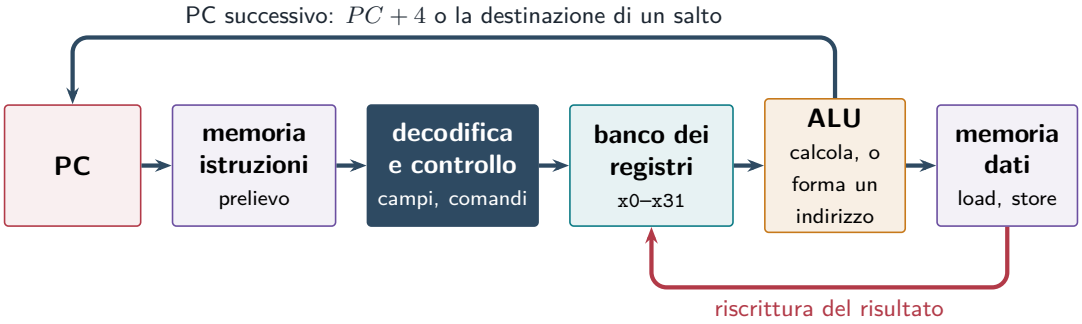


Figura 18.3. L'architettura di un processore RV32, ridotta ai suoi blocchi.

18.6.3 Il processore del kit PRG32

Tabella 18.5. L'ESP32-C6 (dal datasheet di Espressif).

Caratteristica	ESP32-C6
Insieme di istruzioni	RV32IMAC: insieme intero di base, moltiplicazione e divisione, operazioni atomiche, istruzioni compresse
Core principale	un core RISC-V a 32 bit, fino a 160 MHz, pipeline a quattro stadi
Secondo core	un core RISC-V a basso consumo, fino a 20 MHz, per i compiti che girano mentre il core principale dorme
Modi di privilegio	macchina e utente
Virgola mobile	nessuna: né estensione F né D
Memoria	512 KB di SRAM, 320 KB di ROM, flash esterna
Sullo stesso chip	radio Wi-Fi 6, Bluetooth, Thread e Zigbee; periferiche SPI, I2S, GPIO e timer

Il chip della Tabella 18.5 è un calcolatore completo su pochi millimetri quadrati che costa circa due euro: processore, memoria e ingresso/uscita ci sono tutti, come nel modello con cui il capitolo è cominciato.

18.7 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l18.S` nel repository del corso, stampato per intero nel Listato 18.2. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 18.4 è il frame scritto dal test.

Sullo schermo.
Tre barre, otto pixel per unità: x = 5 (bianca), y = 7 (bianca) e z = 12 (verde), calcolata dalle sette istruzioni della slide.

Che cosa premere.
Niente: il programma va da solo.

```
1 # ARC12 Computer Architecture - Lecture 18: The Processor
```



Figura 18.4. `l18.S` su PRG32-QT.

```
# l18.S: z = x + y on a load-store machine
#
# On the screen: three bars, eight pixels per unit: x = 5 (white), y = 7 (white) and z
# = 12 (green), computed by the seven instructions of the slide.
# Build: python3 -m prg32 cartridge build l18.S --portable \
#            --entry-prefix l18 --name l18 --out l18.prg32
.option norelax

.equ BLACK, 0x0000
.equ WHITE, 0xffff
.equ GREEN, 0x07e0

.section .text
.global l18_init
.global l18_update
.global l18_draw

l18_init:
    ret

l18_update:
    la   t0, x
    lw   t1, 0(t0)       # bring x into a register
    la   t0, y
    lw   t2, 0(t0)       # bring y into a register
    add  t3, t1, t2      # compute, between registers
    la   t0, z
    sw   t3, 0(t0)       # take the result back to memory
    ret

# bar: a0 = ADDRESS of a variable, a1 = y of the bar, a4 = colour
bar:
    lw   a2, 0(a0)       # the VALUE of the variable
    slli a2, a2, 3       # width: eight pixels per unit
    li   a0, 20
    li   a3, 16
    tail prg32_gfx_rect

l18_draw:
    addi sp, sp, -16
```

```
42      sw   ra, 12(sp)
43      li   a0, BLACK
44      call prg32_gfx_clear
45      la   a0, x
46      li   a1, 40
47      li   a4, WHITE
48      call bar
49      la   a0, y
50      li   a1, 72
51      li   a4, WHITE
52      call bar
53      la   a0, z
54      li   a1, 120
55      li   a4, GREEN
56      call bar
57      lw   ra, 12(sp)
58      addi sp, sp, 16
59      ret
60
61  .section .data
62  x: .word 5
63  y: .word 7
64  z: .word 0
```

Listato 18.2. Il programma completo del capitolo, `code/l18.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 18.3; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l18.S --portable \
    --entry-prefix l18 --name l18 --out l18.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l18.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 18.3. Compilare `l18.S` ed eseguirlo su PRG32-QT.

18.8 Verificate di aver capito

D18.1 Quale registro del modello generale non ha corrispettivo in RISC-V?

A. il program counter
B. il registro di stato con i flag
C. il puntatore allo stack
D. il registro istruzione

D18.2 In RISC-V, quali istruzioni possono leggere o scrivere la memoria centrale?

A. tutte
B. `add` e `sub`
C. solo load e store
D. solo i salti

D18.3 Dopo `addi sp, sp, -4` e `sw t0, 0(sp)`, che cosa è successo?

A. `t0` è stato estratto dallo stack
B. `t0` è stato inserito sullo stack
C. lo stack è stato svuotato
D. `sp` è stato salvato in `t0`

D18.4 In una macchina ad accumulatore, l'istruzione `ADD y` significa:

A. $y := y +$ accumulatore
B. accumulatore := accumulatore + $[y]$
C. push di y, poi somma
D. $y :=$ accumulatore

18.9 Esercizi

Esercizio 18.1 (prima su carta, poi su PRG32)

1. Scrivete $z = (a + b) - (c + d)$ per una macchina ad accumulatore, per una macchina a stack e per RISC-V; contate le istruzioni e gli accessi alla memoria.
2. Scrivete il push e il pop di due registri, `t0` e `t1`, tenendo `sp` multiplo di 16 come richiede la convenzione di chiamata.
3. Per ogni registro usato nel vostro ultimo programma PRG32, dite quale funzione svolge: transito, accumulo, indirizzamento, stack.
4. In PRG32-QT, eseguite passo-passo una chiamata e il suo ritorno: guardate `pc`, `ra` e `sp`, e la parola salvata sullo stack.
5. Trovate nel datasheet del processore del vostro telefono o portatile: insieme di istruzioni, numero di core, clock, livelli di privilegio.

Risultato atteso. Accumulatore: `LDA a; ADD b; STA tmp; LDA c; ADD d; STA tmp2; LDA tmp; SUB tmp2; STA z`. Stack: `PUSH a; PUSH b; ADD; PUSH c; PUSH d; ADD; SUB; POP z`. RISC-V: quattro load, due `add`, una `sub`, uno store. In PRG32-QT: `sp` scende di 16 al prologo e ritorna all'epilogo; `ra` contiene l'indirizzo dopo la chiamata.

18.10 Glossario

Inglese	Italiano
control unit	unità di controllo
program counter	contatore di programma
machine register	registro di macchina
general / special register	registro generico / speciale
instruction set	insieme (set) di istruzioni
accumulator	accumulatore
stack pointer	puntatore allo stack
status register, flags	registro di stato, indicatori
supervisor / user state	stato supervisore / utente
privileged instruction	istruzione privilegiata

18.11 Conclusioni e letture

Il processore comanda e controlla, appoggiandosi alla memoria e ai sistemi di ingresso/uscita. I registri sono locazioni di memorizzazione che possono trovarsi fisicamente dentro o fuori il processore, e i tipi di registri e il modo in cui sono usati determinano il modello del processore: accumulatore, stack, registri generali. La ALU svolge le operazioni tra operandi. RISC-V è un processore load-store a registri generali, senza flag e senza istruzioni di I/O: pochi elementi, ciascuno semplice.

Letture. Patterson e Hennessy (2020), capitoli 2 e 4; il manuale dell'insieme di istruzioni RISC-V, volume I (RISC-V International, 2024c); Stallings (2021), i capitoli sulla struttura e sul funzionamento del processore. L'Appendice A è il riferimento completo.

CAPITOLO 19

Istruzioni e interruzioni

"Perché la CPU non riesce mai a combinare niente? Perché viene interrotta continuamente."
— Battuta anonima sull'architettura dei calcolatori

In questo capitolo

Alla fine di questo capitolo saprete

- descrivere un'istruzione come un codice operativo e i suoi operandi, e nominare i tipi di operando;
- decodificare a mano un'istruzione RISC-V, riconoscendone il formato e i campi;
- seguire un'istruzione attraverso il ciclo del processore: prelievo, decodifica, esecuzione, memoria, scrittura del risultato;
- spiegare che cos'è una pipeline e perché rende più veloce un processore;
- definire interruzioni, eccezioni e trap, ed elencarne le cause;
- descrivere che cosa fa RISC-V quando avviene una trap e come riprende il programma interrotto.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI formati delle istruzioni; i modi di indirizzamento; il ciclo del processore; interruzioni e trap.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI niente.

19.1 Riscaldamento sulla macchina

```
indirizzo   codice macchina  che cosa avete scritto

0x000       0x0002A303       lw   t1, 0(t0)
0x004       0x00130313       addi t1, t1, 1
0x008       0x0062A023       sw   t1, 0(t0)
0x00C       0x00008067       ret
```

Listato 19.1. Quattro parole in memoria.

Il Listato 19.1 è la funzione `first_update` del Capitolo 2 così come sta in memoria: quattro numeri, a quattro indirizzi di distanza. (La `la` che li precede è omessa, e le codifiche sono quelle non compresse di 32 bit: con l'estensione C l'assemblatore può scegliere per alcune di esse forme di 16 bit.) Seguono tre domande. Come fa il processore a sapere che il primo numero è un load e il secondo un'addizione? Quali passi compie per ciascuno di essi? E mentre li esegue, come si accorge che è stato premuto un pulsante o che è scaduto un timer?

19.2 Le istruzioni

19.2.1 Di che cosa è fatta un'istruzione

Le istruzioni tipiche di un processore sono trasferimenti di dati tra memoria e registri, operazioni aritmetiche e logiche, istruzioni per il controllo del flusso, e trasferimenti di dati da e verso l'ingresso/uscita. (In RISC-V la quarta famiglia non esiste in quanto tale, perché i dispositivi sono memoria.)

Definizione 19.1 (Istruzione)

Un'*istruzione* è composta da un *codice operativo*, l'opcode, e da zero o più *operandi*. Un operando può essere una costante (un *immediato*), una locazione di memoria, o un registro; può essere esplicito o implicito.

Le istruzioni possono avere zero, uno, due o tre operandi: `add` ne ha tre, tutti registri; `ret` non ne ha di visibili.

19.2.2 I formati delle istruzioni RISC-V

Ogni istruzione RISC-V dell'insieme di base è lunga 32 bit, ed è disposta secondo uno di sei *formati* (Figura 19.1). I formati sono stati progettati in modo che lo stesso campo sia sempre nello stesso posto: l'opcode nei bit da 6 a 0, il registro destinazione `rd` nei bit da 11 a 7, i registri sorgente `rs1` e `rs2` nei bit da 19 a 15 e da 24 a 20, il campo `funct3` nei bit da 14 a 12. I formati B e J differiscono da S e U solo nel modo in cui sono disposti i bit dell'immediato.

Questa regolarità è il cuore dell'idea RISC. Il processore sa sempre dove guardare per prima cosa, e può leggere i registri nominati da un'istruzione mentre sta ancora scoprendo di quale istruzione si tratta.

19.2.3 Decodificare a mano

Il metodo

Scrivete in binario le otto cifre esadecimali; staccate gli ultimi sette bit, l'opcode; cercate il formato; poi tagliate i campi.

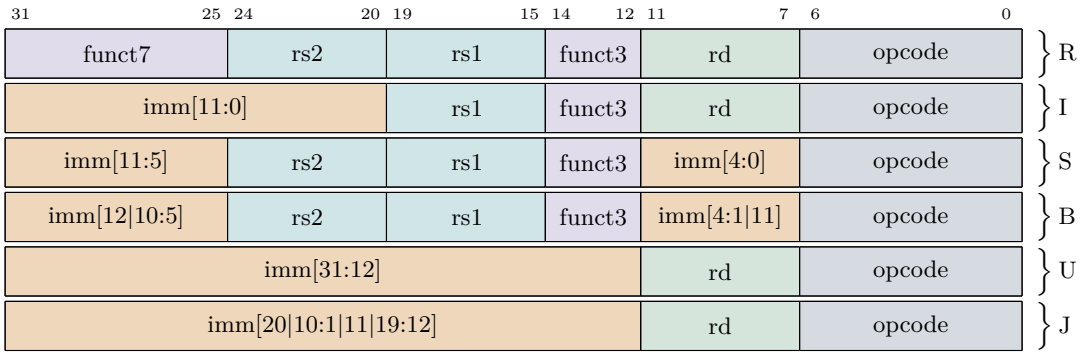


Figura 19.1. I sei formati di RISC-V. R: da registro a registro (`add`, `sub`, `and`, `sll`, ...). I: immediati e load (`addi`, `andi`, `lw`, `jalr`, ...). S: store. B: salti condizionati. U: immediati superiori (`lui`, `auipc`). J: salti (`jal`).

Esempio 19.1 (`0x006283B3`)

`0x006283B3` = $0000\,0000\,0110\,0010\,1000\,0011\,1011\,0011_2$. Gli ultimi sette bit, `0110011`, sono l'opcode delle operazioni da registro a registro: il formato è R.

0000000	00110	00101	000	00111	0110011
funct7	rs2 = x6 = t1	rs1 = x5 = t0	funct3	rd = x7 = t2	opcode

Con `funct3` = 000 e `funct7` = 0000000 l'operazione è un'addizione (con `funct7` = 0100000 sarebbe una sottrazione): l'istruzione è `add t2, t0, t1`.

Esempio 19.2 (`0x0002A303`)

`0x0002A303` = $0000\,0000\,0000\,0010\,1010\,0011\,0000\,0011_2$. L'opcode `0000011` è quello dei load: il formato è I.

000000000000	00101	010	00110	0000011
immediato = 0	rs1 = x5 = t0	funct3 = parola	rd = x6 = t1	opcode

L'istruzione è `lw t1, 0(t0)`, la prima del riscaldamento.

I decodificatori del Capitolo 11 fanno la stessa cosa in una frazione di nanosecondo. Le tabelle necessarie per decodificare qualsiasi istruzione sono nell'Appendice A.

19.2.4 Istruzioni a lunghezza fissa o variabile

In una macchina con istruzioni a *lunghezza fissa* tutte le istruzioni hanno lo stesso numero di bit, l'istruzione successiva è sempre al PC più una costante, e prelievo e decodifica sono semplici. I processori RISC sono di questo tipo: RISC-V usa 32 bit e, con l'estensione compressa, 16 o 32, distinti dai due bit più bassi (Capitolo 11). In una macchina con istruzioni a *lunghezza variabile* la lunghezza è nota solo dopo che la prima parte è stata decodificata: i programmi sono compatti, e la decodifica è complessa. I processori CISC sono di questo tipo: l'MC68000 usa da una a cinque parole di 16 bit, e Intel x86 da uno a quindici byte. In entrambi i casi la lunghezza dell'istruzione decide di quanto avanza il PC.

19.2.5 Modi di indirizzamento: dove si trova un operando

RISC-V ha solo i primi quattro modi della Tabella 19.1. Un processore CISC come l'MC68000 ne ha una dozzina, compresi i modi indiretti in memoria e con autoincremento.

Tabella 19.1. Modi di indirizzamento.

Modo	L'operando è	Esempio RISC-V
immediato	una costante dentro l'istruzione	`addi t1, t1, 1`
a registro	il contenuto di un registro	`add t2, t0, t1`
base più scostamento	la parola di memoria a registro + costante	`lw t1, 8(sp)`
relativo al PC	un indirizzo a una certa distanza dall'istruzione stessa	`beq t0, t1, loop`
indiretto in memoria	la parola di memoria il cui indirizzo è in memoria	non disponibile: due load

L'operazione $R := M[M[A]]$ diventa in RISC-V `lw t0, 0(t1)` seguita da `lw t0, 0(t0)`. Il prezzo di modi di indirizzamento ricchi è un'ulteriore fase nel ciclo del processore, in cui si assemblano gli operandi.

19.2.6 Chiamate, ritorni e la convenzione di chiamata

Le istruzioni per il controllo del flusso meritano una parola in più, perché ogni funzione di Verm1 dipende da esse. `jal` (*jump and link*, formato J) salta a un indirizzo relativo al PC e scrive nel suo registro destinazione l'indirizzo dell'istruzione seguente; `jalr` (formato I) fa lo stesso con una destinazione presa da un registro. La pseudoistruzione `call` è una `jal` (o una `auipc` con una `jalr`, per destinazioni lontane) la cui destinazione è `ra`; `ret` è `jalr zero, 0(ra)`.

La *convenzione di chiamata* è l'accordo che permette a funzioni scritte separatamente di chiamarsi l'una con l'altra (RISC-V International, 2024a). Dice che gli argomenti viaggiano in `a0`–`a7` e i risultati in `a0` e `a1`; che una funzione può distruggere i registri `t0`–`t6` e `a0`–`a7`, che sono quindi *salvati dal chiamante*; che deve restituire i registri `s0`–`s11` e `sp` come li ha trovati, che sono quindi *salvati dal chiamato*; e che `sp` è sempre multiplo di 16. È per questo che, fin dal Capitolo 12, un valore che deve sopravvivere a una chiamata è stato tenuto in un registro `s` e salvato nel frame. La convenzione è enunciata per intero nell'Appendice A.

19.3 Il ciclo del processore

19.3.1 Le fasi del ciclo

Dopo una fase di avvio, il *bootstrap*, un processore ripete per sempre la stessa sequenza: il suo algoritmo fondamentale. Nella formulazione RISC classica la sequenza ha cinque fasi (Figura 19.2).

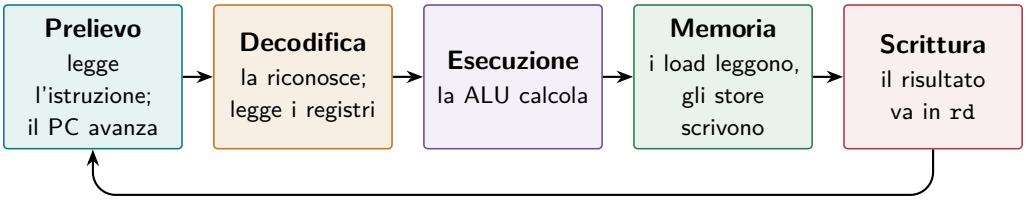


Figura 19.2. Il ciclo del processore.

Prelievo (fetch).
L'istruzione all'indirizzo contenuto nel PC viene letta dalla memoria; il PC avanza.

Decodifica (decode).
L'opcode viene riconosciuto; i registri nominati dall'istruzione vengono letti.

Esecuzione (execute).
La ALU calcola: un risultato, un indirizzo o un confronto.

Memoria.
I load leggono la memoria e gli store la scrivono; le altre istruzioni saltano questa fase.

Scrittura del risultato (write back).
Il risultato viene scritto nel registro destinazione.

19.3.2 La fase di prelievo, trasferimento per trasferimento

Nei termini del modello di riferimento del Capitolo 18, il prelievo è fatto di quattro *trasferimenti tra registri* sul bus interno (Tabella 19.2). In RV32 una parola è di quattro byte e gli indirizzi contano i byte, così che il PC avanza di 4 (di 2 dopo un'istruzione compressa).

Tabella 19.2. La fase di prelievo.

Passo	Trasferimento tra registri	Significato
1	$MA := PC$	l'indirizzo dell'istruzione va al registro indirizzo di memoria
2	abil. lettura; $MB := M[MA]$	la memoria mette l'istruzione sul bus dati; viene catturata nel buffer
3	$D := MB$	l'istruzione è copiata nel registro di decodifica
4	$PC := PC + 4$	il PC punta ora all'istruzione successiva

19.3.3 Il resto del ciclo per due istruzioni

Tabella 19.3. Le fasi restanti per un'addizione e per un load.

Fase	`add t2, t0, t1`	`lw t1, 0(t0)`
decodifica	$O_1 := R[\texttt{t0}]$; $O_2 := R[\texttt{t1}]$	$O_1 := R[\texttt{t0}]$; $O_2 := 0$ (l'immediato)
esecuzione	$U := O_1 + O_2$	$U := O_1 + O_2$ (un indirizzo)
memoria	–	$MA := U$; abil. lettura; $MB := M[MA]$
scrittura	$R[\texttt{t2}] := U$	$R[\texttt{t1}] := MB$

La stessa ALU somma due registri per `add` e forma un indirizzo per `lw` (Tabella 19.3): differiscono solo i segnali di controllo. L'unità di controllo, decodificando l'opcode, decide quali trasferimenti avvengono. Uno store termina alla fase di memoria; un salto condizionato termina alla fase di esecuzione, scegliendo il PC successivo.

19.3.4 La pipeline

Cinque fasi sono cinque pezzi di hardware. Mentre un'istruzione viene eseguita, la successiva può essere decodificata e una terza prelevata: è una *pipeline* (Tabella 19.4).

Ogni istruzione impiega ancora cinque cicli, ma a ogni ciclo di clock si completa un'istruzione: la produttività è cinque volte maggiore. In generale, n istruzioni su una pipeline di k stadi si completano in $k + n - 1$ cicli. I processori *superscalari* mettono due o più pipeline fianco a fianco, e completano più di un'istruzione per ciclo (Capitolo 24). Il core principale dell'ESP32-C6 ha una pipeline a quattro stadi. I salti, e le dipendenze tra istruzioni

Tabella 19.4. Una pipeline lineare a cinque stadi: IF prelievo dell'istruzione, ID decodifica, EX esecuzione, MEM memoria, WB scrittura del risultato.

ciclo di clock	1	2	3	4	5	6	7	8
istruzione 1	IF	ID	EX	MEM	WB			
istruzione 2		IF	ID	EX	MEM	WB		
istruzione 3			IF	ID	EX	MEM	WB	
istruzione 4				IF	ID	EX	MEM	WB

vicine, complicano il quadro: si chiamano *hazard* (criticità), e sono l'argomento di un corso avanzato.

19.4 Le interruzioni

19.4.1 Il problema del ciclo semplice

Se il processore facesse sempre e solo prelievo, decodifica ed esecuzione, nascerebbero tre problemi. L'interazione con il mondo esterno attraverso le unità di I/O sarebbe inefficiente, perché il programma dovrebbe continuare a chiedere. Non ci sarebbe modo di rimuovere forzatamente un'applicazione che, per errore, entra in un ciclo infinito: chi toglie il processore a un programma bloccato in `j loop`? E il sistema operativo avrebbe un controllo limitato sul sistema.

19.4.2 Interruzioni, eccezioni, trap

Definizione 19.2 (Interruzione)

Le *interruzioni* sono eventi eccezionali, di solito asincroni rispetto all'esecuzione del programma in corso, che richiedono l'attenzione del processore.

La specifica RISC-V usa tre parole in modo preciso (RISC-V International, 2024d). Un'*interruzione* (ingl. *interrupt*) viene dall'esterno del programma, in un momento qualsiasi: un timer, un dispositivo. Un'*eccezione* è causata dall'istruzione in esecuzione: un opcode illegale, un indirizzo proibito. Una *trap* è il trasferimento del controllo a un gestore che segue l'una o l'altra. (I processori più vecchi chiamano «trap» gli eventi sincroni e usano «interruzione» per tutto.)

19.4.3 Il vero ciclo del processore

Il ciclo reale ha quindi un passo in più (Figura 19.3): dopo ogni istruzione il processore controlla se c'è un'interruzione in attesa e, in tal caso, passa al gestore invece di prelevare l'istruzione successiva.

19.4.4 Possibili cause di interruzione

Interruzione periodica.
Un timer, per esempio ogni 10 ms: per condividere il processore tra i programmi e per misurare il tempo che ciascuno usa.

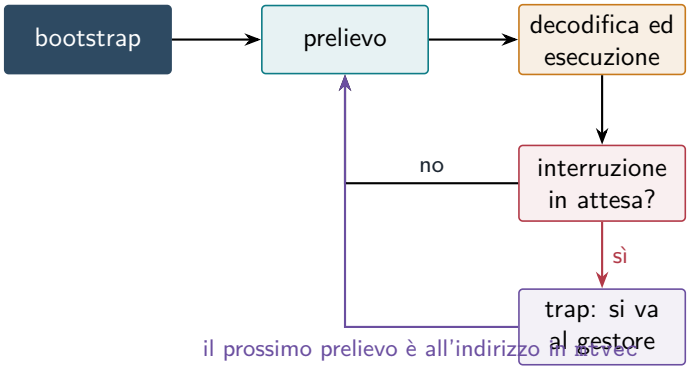


Figura 19.3. Il vero ciclo del processore.

Interruzione di I/O.
Una periferica comunica il proprio stato, per esempio che è pronta a ricevere dati, per sincronizzarsi con il processore.

Errori nel programma.
Un'istruzione che non esiste o è privilegiata, un indirizzo proibito: eventi sincroni, le eccezioni. (In RISC-V un overflow intero *non* solleva un'eccezione.)

Guasti.
Malfunzionamenti del sistema rilevati da sonde dedicate: alimentazione, temperatura, errori di memoria.

Reset.
Riporta l'intero sistema a uno stato noto.

Interruzione software.
Generata da un programma che ha bisogno di una risorsa condivisa e chiede al sistema operativo di fare da mediatore: in RISC-V, l'istruzione `ecall`, *environment call*, che è sincrona con il programma che la esegue.

19.4.5 Gestire un'interruzione

1. Salvare il contesto, parte hardware: almeno l'indirizzo da cui riprendere.
2. Identificare la causa dell'interruzione.
3. Saltare al punto di ingresso della *routine di servizio dell'interruzione* (ISR).
4. Salvare il contesto, parte software: i registri che la routine sta per usare.
5. Servire l'interruzione.
6. Se l'interruzione lo consente, ripristinare il contesto e riprendere il programma sospeso dall'istruzione che segue l'ultima eseguita.

Il *contesto* da salvare e ripristinare è il PC, lo stato e i registri. Il salvataggio può essere fatto in parte o del tutto in hardware o in software; RISC-V fa il minimo in hardware.

19.4.6 Chi ha interrotto? Due soluzioni

Con più dispositivi il processore deve identificare chi ha interrotto. Con *registri di stato e polling*, ogni dispositivo ha un registro di stato: un dispositivo che richiede un'interruzione vi pone a 1 un bit e alza una linea di richiesta comune; la routine di servizio interroga i dispositivi in ordine di priorità e serve il primo che trova con il bit alto. Lo schema è semplice da costruire ma richiede tempo, perché interroga anche dispositivi che non hanno chiesto

niente. Con le *interruzioni vettorizzate* l'identità della sorgente è fornita al processore al momento della richiesta, e il processore salta direttamente alla routine che le corrisponde: un meccanismo molto efficiente, in cui il numero di bit del vettore limita il numero di sorgenti diverse.

La *latenza di interruzione* è il ritardo tra l'istante in cui la linea di richiesta sale e l'istante in cui la routine di servizio comincia a fare lavoro utile; comprende la fine dell'istruzione corrente, il controllo e la commutazione.

19.5 La soluzione di RISC-V

RISC-V risolve l'intero problema con pochi registri e un'istruzione.

19.5.1 I registri del meccanismo delle trap

Tabella 19.5. I CSR del modo macchina per il meccanismo delle trap.

CSR	Scritto da	Contenuto
`mtvec`	il firmware, una volta, all'avvio	l'indirizzo del gestore delle trap, e se si usa il modo vettorizzato
`mstatus`	hardware e firmware	MIE, l'abilitazione globale delle interruzioni, e MPIE, il suo valore precedente
`mie`	il firmware	un bit di abilitazione per ogni sorgente: software, timer, esterna
`mip`	l'hardware	un bit di richiesta in attesa per ogni sorgente
`mepc`	l'hardware, a ogni trap	l'indirizzo dell'istruzione interrotta
`mcause`	l'hardware, a ogni trap	un bit che dice interruzione o eccezione, e un codice per la causa

Un'interruzione viene accolta solo se il suo bit è a 1 sia in `mip` sia in `mie`, e MIE vale 1 (Tabella 19.5).

19.5.2 Che cosa fa l'hardware su una trap

1. `mepc` := l'indirizzo dell'istruzione interrotta.
2. `mcause` := la causa: il bit più alto vale 1 per un'interruzione e 0 per un'eccezione, e gli altri bit contengono un codice.
3. In `mstatus`: MPIE := MIE, poi MIE := 0, così che ulteriori interruzioni sono disabilitate.
4. Il processore passa al modo macchina.
5. PC := l'indirizzo contenuto in `mtvec`; oppure, nel modo vettorizzato e per le interruzioni, quell'indirizzo più 4 volte la causa.

Alla fine del gestore l'istruzione `mret` disfa tutto: MIE := MPIE e PC := `mepc`.

L'hardware non inserisce niente su uno stack: il salvataggio dei registri generici è lasciato al gestore. Il «salvataggio hardware del contesto» dello schema generale è ridotto a due registri e un bit.

Tabella 19.6. Alcuni valori standard di `mcause`; i chip ne aggiungono di propri.

Tipo	Codice	Causa
interruzione	3	interruzione software di macchina
interruzione	7	interruzione del timer di macchina
interruzione	11	interruzione esterna di macchina: un dispositivo, attraverso il controller delle interruzioni
eccezione	2	istruzione illegale
eccezione	3	breakpoint (`ebreak`): il modo in cui un debugger ferma un programma
eccezione	5, 7	errore di accesso in load o in store: un indirizzo proibito
eccezione	8, 11	environment call (`ecall`) dal modo utente, dal modo macchina

19.5.3 La causa di una trap

Lo stesso codice 3 significa cose diverse per le interruzioni e per le eccezioni (Tabella 19.6): le distingue il bit più alto di `mcause`. Con `mtvec` nel modo vettorizzato, un'interruzione del timer entra a $base + 4 \times 7$.

19.5.4 Un gestore delle trap, in schema

```
trap_entry:
    addi sp, sp, -16
    sw   t0, 0(sp)            # software context: save what we use
    sw   t1, 4(sp)

    csrr t0, mcause           # why are we here?
    bgez t0, exception        # top bit 0: an exception
    ...                       # top bit 1: find the device, serve it

    lw   t0, 0(sp)            # restore the context
    lw   t1, 4(sp)
    addi sp, sp, 16
    mret                      # MIE := MPIE, PC := mepc
```

Listato 19.2. Lo schema di un gestore delle trap.

Il programma interrotto non deve accorgersi di niente: ogni registro che il gestore tocca viene salvato e ripristinato (Listato 19.2). Letto come numero con segno, `mcause` è negativo per le interruzioni, così che `bgez` separa le due famiglie. `mret` è l'unica via di ritorno: riabilita le interruzioni e ricarica il PC in un solo passo. L'istruzione `csrr t0, mcause` è la pseudoistruzione per `csrrs t0, mcause, zero`.

Il listato è uno schema, non il sorgente di PRG32: un gestore reale salva tutti i registri salvati dal chiamante, di solito su uno stack separato raggiunto attraverso il registro `mscratch`. Il firmware del kit contiene esattamente una routine di questo tipo, e una cartuccia viene interrotta molte volte per frame.

19.5.5 Priorità e annidamento

Per gestire le interruzioni *annidate* si stabilisce una priorità: mentre una routine di servizio è in esecuzione, si accettano solo richieste da dispositivi di priorità più alta, e una richiesta

da un dispositivo ad alta priorità, come il clock, deve essere accettata anche mentre se ne sta servendo una di priorità più bassa. La Tabella 19.7 confronta la soluzione classica dell'MC68000 con quella di RISC-V, dove le priorità appartengono a un controller delle interruzioni esterno al core (sui chip Espressif vanno da 1 a 15).

Tabella 19.7. Priorità e annidamento in due processori.

	Motorola MC68000	**RISC-V**
priorità di una richiesta	tre linee codificano un livello di priorità dell'interruzione, da 1 a 7	il controller delle interruzioni dà a ogni sorgente una priorità
priorità del processore	tre bit nel registro di stato: la maschera	un registro di soglia nel controller delle interruzioni
una richiesta è accettata se	il suo livello è più alto della maschera	la sua priorità è più alta della soglia, ed è abilitata
mentre la si serve	la maschera è alzata a quel livello	le interruzioni sono disabilitate (MIE = 0) a meno che il gestore non le riabiliti
non mascherabile	il livello 7	un'interruzione non mascherabile, definita da ogni chip
identità della sorgente	un numero di vettore di 8 bit dal dispositivo: 256 vettori	`mcause`, poi il controller dice quale dispositivo
ritorno	`RTE`, che estrae PC e stato dallo stack	`mret`, che usa `mepc` e `mstatus`

Attenzione

A volte si dice che «un'interruzione è un segnale che non può essere mascherato». È vero il contrario: la maggior parte delle interruzioni può essere mascherata, e solo poche non lo possono.

19.5.6 Le interruzioni sul kit PRG32

I tre problemi del ciclo semplice, e la loro soluzione, sono tutti visibili sul kit.

- Un timer interrompe il processore cento volte al secondo: il firmware lo usa per tenere il tempo e per scandire i frame.
- Le periferiche del display e del suono interrompono quando un blocco di dati è stato inviato e serve il successivo.
- La radio Wi-Fi interrompe quando arriva un pacchetto: una cartuccia può essere caricata mentre un gioco è in esecuzione.
- Le funzioni `init`, `update` e `draw` di una cartuccia sono funzioni ordinarie chiamate dal firmware tra un'interruzione e la successiva.
- Una cartuccia bloccata in un ciclo non ferma le interruzioni. PRG32-QT termina il frame con un errore («instruction budget exhausted»); sulla scheda un watchdog segnala lo stallo, e l'utente preme reset.

In PRG32-QT un breakpoint è l'eccezione con codice 3.

19.6 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l19.S` nel repository del corso, stampato per intero nel Listato 19.3. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 19.4 è il frame scritto dal test.

Sullo schermo.
Quattro righe di 32 lampade, il codice macchina di lw t1, 0(t0) / addi t1, t1, 1 / sw t1, 0(t0) / ret, e accanto a ciascuna un quadrato verde: la parola in memoria è quella decodificata a mano a lezione.

Che cosa premere.
Niente: il programma va da solo.

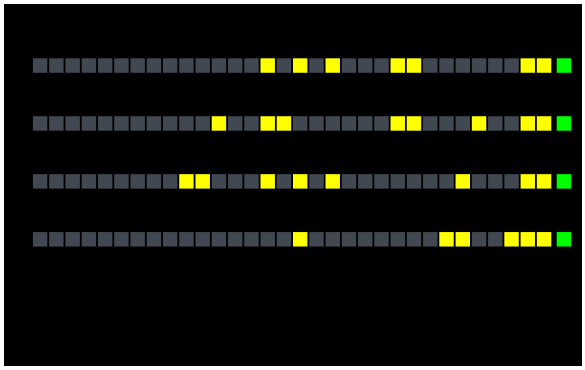


Figura 19.4. `l19.S` su PRG32-QT.

```
# ARC12 Computer Architecture - Lecture 19: Instructions and Interrupts
# l19.S: four words in memory: a function read back as machine code
#
# On the screen: four rows of 32 lamps, the machine code of lw t1, 0(t0) / addi t1,
# t1, 1 / sw t1, 0(t0) / ret, and beside each a green square: the word in memory is
# the one decoded by hand in the lecture.
# Build: python3 -m prg32 cartridge build l19.S --portable \
#            --entry-prefix l19 --name l19 --out l19.prg32
.option norelax

.equ BLACK,  0x0000
.equ GREY,   0x4208
.equ YELLOW, 0xffe0
.equ GREEN,  0x07e0
.equ RED,    0xf800

.section .text
.global l19_init
.global l19_update
.global l19_draw

l19_init:
    ret

```

```
l19_update:
    ret

# The function of Lecture 02, kept in its uncompressed 32-bit forms.
.option push
.option norvc
sample:
    lw   t1, 0(t0)          # 0x0002A303
    addi t1, t1, 1          # 0x00130313
    sw   t1, 0(t0)          # 0x0062A023
    ret                     # 0x00008067
.option pop

l19_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    sw   s1, 4(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   s0, 0              # which instruction, 0 to 3
1:  la   t0, sample
    slli t1, s0, 2          # instructions are 4 bytes apart
    add  t0, t0, t1
    lw   s1, 0(t0)          # the instruction, fetched as a datum
    mv   a0, s1
    slli a1, s0, 5
    addi a1, a1, 30         # y = 30 + 32 * s0
    call lamps32
    la   t0, expected       # compare with the encoding worked out by hand
    slli t1, s0, 2
    add  t0, t0, t1
    lw   t2, 0(t0)
    li   a4, RED
    bne  s1, t2, 2f
    li   a4, GREEN
2:  li   a0, 306
    slli a1, s0, 5
    addi a1, a1, 30
    li   a2, 8
    li   a3, 8
    call prg32_gfx_rect
    addi s0, s0, 1
    li   t0, 4
    blt  s0, t0, 1b
    lw   s1, 4(sp)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

# lamps32: show the 32 bits of a word as a row of lamps, bit 31 on the left.
#   a0 = the word, a1 = y of the row.  A lamp is yellow for 1, grey for 0.
lamps32:
    addi sp, sp, -32
    sw   ra, 28(sp)
```

```
81      sw   s0, 24(sp)
82      sw   s1, 20(sp)
83      sw   s2, 16(sp)
84      mv   s0, a0              # the word
85      mv   s1, a1              # y
86      li   s2, 0               # lamp number, 0 on the left
87  1:  li   t0, 31
88      sub  t0, t0, s2          # lamp i shows bit 31 - i
89      srl  t1, s0, t0          # bring that bit down to bit 0
90      andi t1, t1, 1           # keep it, clear all the others
91      li   a4, GREY            # lamp off
92      beqz t1, 2f
93      li   a4, YELLOW          # lamp on
94  2:  slli a0, s2, 3
95      add  a0, a0, s2          # 9 * i
96      addi a0, a0, 16          # x = 16 + 9 * i
97      mv   a1, s1
98      li   a2, 8
99      li   a3, 8
100     call prg32_gfx_rect
101     addi s2, s2, 1
102     li   t0, 32
103     blt  s2, t0, 1b
104     lw   s2, 16(sp)
105     lw   s1, 20(sp)
106     lw   s0, 24(sp)
107     lw   ra, 28(sp)
108     addi sp, sp, 32
109     ret
110
111 .section .rodata
112 expected: .word 0x0002A303, 0x00130313, 0x0062A023, 0x00008067
```

Listato 19.3. Il programma completo del capitolo, `code/l19.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 19.4; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l19.S --portable \
    --entry-prefix l19 --name l19 --out l19.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l19.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 19.4. Compilare `l19.S` ed eseguirlo su PRG32-QT.

19.7 Verificate di aver capito

D19.1 In quali bit di un'istruzione RISC-V sta l'opcode?

A. i sette più significativi
B. i sette meno significativi
C. i bit da 14 a 12
D. dipende dal formato

D19.2 Durante il prelievo di un'istruzione RV32 non compressa, il PC viene incrementato di:

A. 1
B. 2
C. 4
D. 32

D19.3 Quale registro contiene l'indirizzo da cui riprenderà un programma RISC-V interrotto?

A. `ra`
B. `mtvec`
C. `mepc`
D. `mcause`

D19.4 Quale di questi è un'eccezione e non un'interruzione, nei termini di RISC-V?

A. scade un timer
B. viene premuto un tasto
C. viene prelevata un'istruzione illegale
D. arriva un pacchetto dalla rete

19.8 Esercizi

Esercizio 19.1 (prima su carta, poi su PRG32)

1. Decodificate a mano `0x00A28293` e `0x40628333`: formato, campi, assembly.
2. Codificate `sub t0, t1, t2` e `sw a0, 8(sp)`.
3. Scrivete i trasferimenti tra registri dell'intero ciclo per `sw t1, 0(t0)` e per `beq t0, t1, target`.
4. Disegnate il diagramma della pipeline per sei istruzioni su una pipeline a quattro stadi: quando si completa la sesta?
5. In PRG32-QT mettete un breakpoint in `first_update`, procedete passo-passo, e confrontate il codice macchina mostrato con la vostra decodifica del riscaldamento.

Risultato atteso. Su carta: `addi t0, t0, 10` (immediato 10, `rs1` = `x5`, `funct3` = 000, `rd` = `x5`, opcode 0010011) e `sub t1, t0, t1` (`funct7` = 0100000, `rs2` = `x6`, `rs1` = `x5`, `rd` = `x6`); `0x407302B3` e `0x00A12423`; la sesta istruzione si completa al ciclo 9. In PRG32-QT: le quattro parole del riscaldamento, o i loro equivalenti compressi.

19.9 Glossario

Inglese	Italiano
operation code (opcode)	codice operativo
operand	operando
addressing mode	modo di indirizzamento
fetch / decode / execute	prelievo / decodifica / esecuzione
pipeline stage	stadio della pipeline
context	contesto
interrupt	interruzione
exception / trap	eccezione / trap
interrupt service routine	routine di servizio dell'interruzione
interrupt latency	latenza di interruzione
to mask an interrupt	mascherare un'interruzione
nesting	annidamento

19.10 Conclusioni e letture

Un'istruzione è un codice operativo e zero, uno o più operandi; in RISC-V è sempre di 32 bit, in sei formati regolari. La lunghezza delle istruzioni, fissa o variabile, determina come si comporta il PC. Il processore ha un ciclo: preleva un'istruzione, la decodifica, la esegue e scrive il risultato. Gli eventi eccezionali rompono il ciclo: il processore salva dov'era, serve l'evento e riprende. Le interruzioni possono essere abilitate, mascherate e ordinate per priorità; RISC-V lo fa con una manciata di registri e l'istruzione `mret`.

Letture. Patterson e Hennessy (2020), paragrafi 2.5, 4.1–4.6 e 4.10; il manuale dell'insieme di istruzioni RISC-V, volume II, «Privileged Architecture» (RISC-V International, 2024d); *Playing with RISC-V*, capitolo 4, su PRG32-QT e il suo debugger (Montella et al., 2026b).

CAPITOLO 20

I bus e la scheda video

"... niente è più serio del gioco..."
— attribuita a Jean Cocteau (1889–1963)

In questo capitolo

Alla fine di questo capitolo saprete

- spiegare che cos'è il controller di una periferica e perché i dispositivi hanno bisogno di sincronizzarsi con il processore;
- caratterizzare un bus per larghezza, velocità e velocità di trasferimento, ed elencarne vantaggi e limiti;
- descrivere il compito di una scheda video: frame buffer, risoluzione, profondità di colore, interfaccia di uscita;
- calcolare la memoria che serve a un frame e il tempo necessario per inviarlo su un bus;
- spiegare che cosa fa una GPU e che cos'è il calcolo di uso generale sulle GPU.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI il frame buffer; i controllori delle periferiche; bus e banda; la scheda video; GPU e GPGPU.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI niente.

20.1 Riscaldamento sulla macchina

```
    li   a0, 100            # x
    li   a1, 50             # y
    li   a2, 0xf800         # red, in RGB565
    call prg32_gfx_pixel    # ... where does it go?

# what the runtime does, in principle:
    li   t0, 320
```

```
8     mul  t1, a1, t0          # y * 320
9     add  t1, t1, a0          # + x  =  position in the frame
10    la   t2, framebuffer
11    add  t2, t2, t1          # ADDRESS of that pixel
12    sb   t3, 0(t2)           # store its palette index (one byte)
```

Listato 20.1. Un pixel, per favore: la chiamata, e che cosa fa in linea di principio il runtime.

Le prime quattro righe del Listato 20.1 disegnano un pixel rosso. Il resto è uno schizzo di ciò che succede dietro la chiamata (è uno schizzo del principio, non il sorgente del runtime: `t3` sta per l'indice di palette che il runtime ricava dal colore in `a2`).

Concetto

Lo schermo è un array in memoria, con un elemento per pixel: il *frame buffer*. Disegnare un pixel è un'istruzione di store all'indirizzo $base + y \times 320 + x$.

Non compare niente finché il frame non viene inviato al display, cosa che fa la funzione `prg32_gfx_present`. Su un personal computer questi due compiti, tenere il frame e inviarlo al monitor, appartengono alla scheda video. Il kit non ha una scheda video: il suo processore fa tutto, il che rende visibile ogni passo. Il capitolo segue il pixel dal frame buffer al display, e poi generalizza.

20.2 Periferiche e controller

20.2.1 Dispositivi di ingresso/uscita

Le *periferiche* servono all'interazione tra l'utente e il sistema. Hanno un'autonomia limitata: sono controllate e coordinate dal processore. E lavorano in modo asincrono rispetto a esso, così che servono meccanismi di sincronizzazione: per sapere quando sarà disponibile un dato di ingresso, e per sapere quando un dispositivo di uscita ha finito di produrre i suoi dati. Il terminale, lo strumento di interazione più comune, è fatto di due dispositivi indipendenti, la tastiera e il video; non sono collegati direttamente, ed è il processore che riproduce sul video ciò che arriva dalla tastiera (Capitolo 21).

20.2.2 Controller

Definizione 20.1 (Controller)

Un *controller* è l'unità che interfaccia una periferica al bus: riceve i comandi dal processore, pilota il dispositivo, e contiene i registri attraverso i quali si scambiano dati e stato.

Le periferiche sono interfacciate al processore attraverso controller dedicati (Figura 20.1). Sull'ESP32-C6 i controller (SPI, I2S, GPIO, USB) sono sullo stesso chip del processore, e i loro registri sono mappati in memoria (Capitolo 15).

20.3 I bus

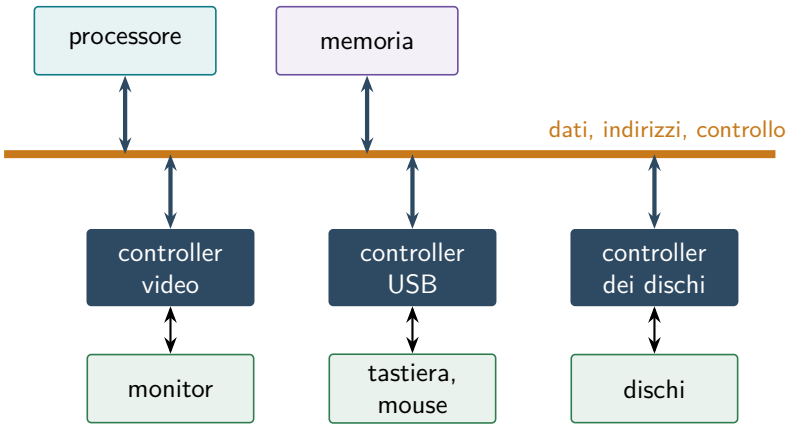


Figura 20.1. Le periferiche raggiungono il bus attraverso i loro controller.

20.3.1 Che cosa caratterizza un bus

Un bus (Capitolo 14) è un canale di comunicazione condiviso da più unità, e le sue caratteristiche influenzano in modo decisivo le prestazioni dell'intero sistema. Lo descrivono tre numeri.

Larghezza.
Il numero di bit del bus dati.

Velocità.
La frequenza di lavoro del bus; con la larghezza, dà il numero di byte per ciclo di clock.

Velocità di trasferimento.
I megabyte trasmessi al secondo: larghezza per frequenza.

Un bus di 32 bit a 66 MHz, il bus PCI, trasporta 4 byte $\times$ $66 \cdot 10^6$ = 264 MB/s. Un bus è anche uno *standard*: segnali, connettori e un protocollo su cui tutte le unità concordano. L'*interfaccia* è il modulo hardware che codifica i dati elettricamente o otticamente; il *protocollo* è la descrizione di più alto livello di come si svolge il dialogo.

20.3.2 Vantaggi e svantaggi

Con un bus si possono aggiungere facilmente nuovi dispositivi; le periferiche possono essere spostate da un calcolatore a un altro che usa lo stesso bus; un solo insieme di fili collega più periferiche; e il costo per byte trasmesso è basso. D'altra parte la larghezza di banda del bus limita la velocità di trasferimento; la velocità massima dipende dalla lunghezza del bus e dal numero di dispositivi a esso collegati; e il bus è una risorsa condivisa, su cui può avvenire un solo trasferimento alla volta.

20.3.3 I bus negli anni

La storia della Tabella 20.1 è il passaggio dal parallelo al seriale spiegato nel Capitolo 14. (Le sigle: ISA, *industry standard architecture*; PCI, *peripheral component interconnect*; AGP, *accelerated graphics port*; IDE, *integrated device electronics*; SCSI, *small computer system interface*; USB, *universal serial bus*.)

20.4 La scheda video

Tabella 20.1. I bus negli anni.

Bus	Tipo	Usato per	Oggi
ISA, EISA, VESA	parallelo, da 16 a 32 bit	schede di espansione dei primi PC	in disuso
PCI	parallelo, 32 o 64 bit	schede di espansione	sostituito da PCI Express
AGP	parallelo, porta dedicata	la sola scheda video	sostituito da PCI Express
PCI Express	seriale, da 1 a 16 linee (lane)	schede video, memoria di massa veloce	attuale: ogni generazione raddoppia la velocità
IDE (ATA), SCSI	parallelo	dischi	sostituiti da Serial ATA e NVMe
USB	seriale	quasi ogni dispositivo esterno	attuale: da 12 Mbit/s a decine di Gbit/s
SPI, I2C, I2S	seriale, pochi fili	chip sulla stessa scheda	attuali: i bus del kit PRG32

20.4.1 Che cosa fa una scheda video

La funzione di una *scheda video* è inviare al monitor l'informazione da visualizzare. Traduce la rappresentazione dell'immagine prodotta dal processore in un formato che il monitor può mostrare. Contiene una memoria propria, la *memoria video*, che contiene il frame: il frame buffer. Ogni volta che lo schermo va ridisegnato, legge la rappresentazione digitale dell'immagine e invia al monitor il segnale per il colore di ogni pixel. E ha una propria capacità di elaborazione. Su PRG32 la «scheda video» è una regione di RAM più la periferica SPI: nessun hardware separato.

20.4.2 Monitor analogici e digitali

Per un monitor CRT, che ha un ingresso analogico, la scheda converte l'immagine digitale in segnali analogici; la conversione è fatta da un componente chiamato RAMDAC (*random access memory digital-analog converter*). L'uscita VGA del 1987 fu progettata per i tubi a raggi catodici, e soffre di rumore elettrico, distorsione ed errori nel posizionamento dei pixel. Per un monitor LCD, che ha un ingresso digitale, la scheda adatta i colori e la risoluzione dell'immagine, fornisce memoria locale da usare come buffer e supporta l'accelerazione grafica, per esempio in tre dimensioni; con DVI ogni pixel dell'uscita corrisponde a un pixel dello schermo, alla sua risoluzione nativa.

L'ILI9341 dell'ultima riga della Tabella 20.2 è il chip sul modulo display del kit: ha una propria memoria per il frame e pilota i cristalli liquidi (Capitolo 21).

20.4.3 Risoluzione: la dimensione della griglia di pixel

La *risoluzione* determina la definizione dell'immagine, e quanti pixel vanno memorizzati e inviati per ogni frame (Tabella 20.3). Il tradizionale rapporto 4:3 ha ceduto il posto al panoramico 16:9.

Tabella 20.2. Uscite video.

Interfaccia	Anno	Segnale	Note
VGA	1987	analogico	per monitor CRT; mantenuta per anni per compatibilità
DVI	1999	digitale (e analogico)	introdotta con i monitor LCD
HDMI	2003	digitale	sostituisce gli standard precedenti; trasporta anche l'audio
DisplayPort	2006	digitale	video, audio e altri dati; anche su connettori USB-C
SPI verso un controller di display	–	digitale, seriale	il kit PRG32: un pannello ILI9341 di 320 per 240 pixel

Tabella 20.3. Alcune risoluzioni.

Risoluzione	Pixel	Rapporto	Dove
320×200	64 000	8:5	Commodore 64, giochi VGA, la viewport di PRG32
640×480	307 200	4:3	VGA
1024×768	786 432	4:3	monitor degli anni Novanta
1920×1080	2 073 600	16:9	Full HD
3840×2160	8 294 400	16:9	4K

20.4.4 Profondità di colore: bit per pixel

La *profondità di colore* è il numero di bit usati per ogni pixel (Tabella 20.4). In True Color ogni colore si ottiene da 256 sfumature di ciascuno dei tre primari additivi, rosso, verde e blu. Nel modo a 8 bit il valore di un pixel non è un colore ma un *indice* in una tabella di 256 colori, la *palette* (tavolozza).

Tabella 20.4. Profondità di colore.

Profondità	Colori	Byte per pixel	Nome
4 bit	16	0,5	VGA standard
8 bit	256	1	modo a 256 colori: un indice in una palette
16 bit	65 536	2	High Color: RGB565
24 bit	16 777 216	3	True Color

PRG32 usa entrambe le righe centrali: un frame buffer a 8 bit per pixel che contiene indici di palette, e una palette di 256 colori in RGB565 (Capitolo 4).

Teorema 20.1 (Memoria di un frame)

Un frame di $W \times H$ pixel con una profondità di d bit per pixel occupa $W \cdot H \cdot d/8$ byte.

Per esempio 640×480 in True Color richiede $307\,200 \times 3 = 921\,600$ byte.

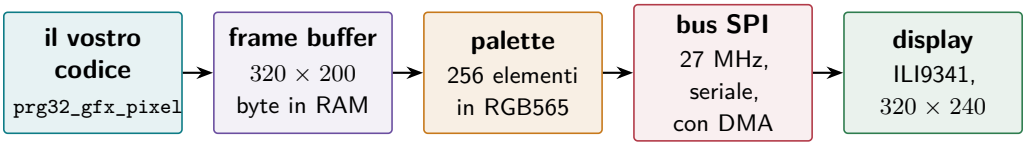


Figura 20.2. Il viaggio di un pixel sul kit PRG32.

20.4.5 Seguire il pixel su PRG32

La Figura 20.2 è il viaggio del pixel del riscaldamento, e la Tabella 20.5 ne dà i numeri (da `docs/hardware/ili9341.md` del repository).

Tabella 20.5. L'aritmetica di un frame su PRG32.

Grandezza	Valore
memoria per un frame, un byte per pixel	$320 \times 200 \times 1 = 64\,000$ byte
lo stesso frame a due byte per pixel	128 000 byte: un quarto di tutta la RAM del chip
bit inviati al display per un intero frame	64 000 pixel × 16 bit = 1 024 000 bit
tempo sul bus a 27 MHz	circa 38 ms: al più circa 26 frame interi al secondo

L'ultima riga segue da $1\,024\,000/27\,000\,000 = 37{,}9$ ms. La piattaforma dà a questi numeri due risposte ingegneristiche: un frame buffer *indicizzato*, che dimezza la memoria, e il tracciamento delle righe che sono cambiate, così che vengano inviate solo quelle. Il Capitolo 22 sfrutterà la seconda.

20.4.6 Memoria e larghezza di banda: la pressione sull'hardware video

La memoria e il tempo di processore richiesti da una grafica sempre più esigente, i videogiochi tridimensionali sopra tutti, hanno imposto alle schede video una rapida evoluzione. Al crescere della definizione delle immagini cresce la memoria necessaria per rappresentare un frame, e il flusso di dati dalla memoria alla scheda video rischia di saturare la capacità del bus: un frame 4K in True Color a 60 frame al secondo sono $8{,}3 \cdot 10^6 \times 3 \times 60$ byte, circa 1,5 GB di pixel ogni secondo. Le linee di attacco furono due: sempre più memoria sulla scheda stessa, e un bus più largo o dedicato, AGP ieri e PCI Express oggi. Si potevano perfino collegare insieme più schede per dividere il lavoro; oggi una sola scheda, o molte schede in un server, sono la regola.

20.5 GPU e GPGPU

20.5.1 L'unità di elaborazione grafica

La quota di tempo di processore spesa a calcolare immagini è cresciuta con l'uso delle interfacce grafiche. Per alleggerire la CPU, alle schede video furono dati processori dedicati.

Definizione 20.2 (GPU)

Un'*unità di elaborazione grafica* (GPU, *graphics processing unit*) è un processore specializzato nelle operazioni della grafica, che esegue in parallelo su molti pixel.

Una GPU accelera la grafica raster, cioè la copia di blocchi di pixel (*blitting*) e le operazioni sulle texture, e la grafica tridimensionale: trasformazioni, applicazione delle texture, illuminazione. I programmi la usano attraverso librerie: OpenGL, Direct3D, e oggi Vulkan e Metal. Un *driver di periferica* adatta il software che gira sulla CPU alle primitive offerte dalla GPU; ciò che non è supportato viene emulato dalla CPU, lentamente.

Ritorno alla macchina

Su PRG32 non c'è una GPU: `prg32_gfx_rect` è un ciclo di istruzioni di store eseguito dal core RISC-V. È per questo che la frequenza dei frame di una cartuccia impegnativa cala, e che il costo di ogni pixel si può vedere.

20.5.2 Calcolo di uso generale sulle GPU

Le GPU moderne eseguono la stessa operazione in parallelo su intere matrici di pixel, con un numero molto grande di core semplici: centinaia, migliaia. Il *GPGPU* è l'uso delle GPU per applicazioni di tipo generale, non necessariamente grafiche (la Figura 4.5 ne mostrava l'esempio più semplice, una somma parallela). I modelli di programmazione sono CUDA (*Compute Unified Device Architecture*, di NVIDIA) e OpenCL, uno standard aperto. La simulazione scientifica, l'analisi dei dati e, oggi soprattutto, l'addestramento e l'uso delle reti neurali girano su GPU; così i modelli meteorologici e marini del Dipartimento. Il Capitolo 24 torna sull'argomento.

20.6 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l20.S` nel repository del corso, stampato per intero nel Listato 20.2. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 20.3 è il frame scritto dal test.

Sullo schermo.
Un solo pixel rosso a x = 100, y = 50 su uno schermo nero. Guardate da vicino.

Che cosa premere.
Niente: il programma va da solo.

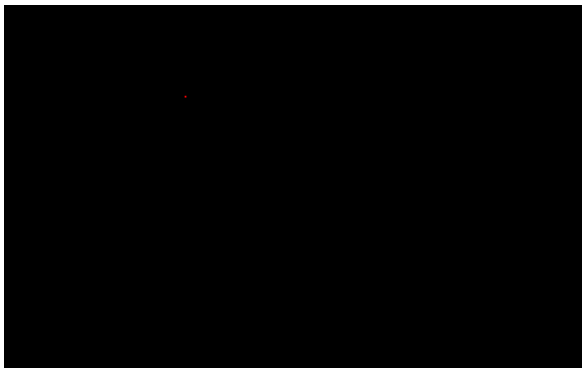


Figura 20.3. `l20.S` su PRG32-QT.

```
1 # ARC12 Computer Architecture - Lecture 20: Buses and the Video Card
```

```
# l20.S: one pixel, please
#
# On the screen: a single red pixel at x = 100, y = 50 on a black screen. Look
# closely.
# Build: python3 -m prg32 cartridge build l20.S --portable \
#            --entry-prefix l20 --name l20 --out l20.prg32
.option norelax

.section .text
.global l20_init
.global l20_update
.global l20_draw

l20_init:
    ret

l20_update:
    ret

l20_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0
    call prg32_gfx_clear
    li   a0, 100             # x
    li   a1, 50              # y
    li   a2, 0xf800          # red, in RGB565
    call prg32_gfx_pixel     # ... where does it go?
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 20.2. Il programma completo del capitolo, `code/l20.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 20.3; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l20.S --portable \
    --entry-prefix l20 --name l20 --out l20.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l20.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 20.3. Compilare `l20.S` ed eseguirlo su PRG32-QT.

20.7 Verificate di aver capito

D20.1 Quanta memoria serve a un frame di 640×480 pixel in True Color (24 bit)?

A. 307 200 byte
B. 614 400 byte
C. 921 600 byte
D. 7 372 800 byte

D20.2 Un bus a 16 bit lavora a 50 MHz, un trasferimento per ciclo. Qual è la sua velocità di trasferimento?

A. 50 MB/s
B. 100 MB/s
C. 400 MB/s
D. 800 MB/s

D20.3 In un frame buffer indicizzato a 8 bit, il byte di un pixel contiene:

A. la sua componente rossa
B. la sua luminosità
C. la posizione del suo colore in una palette
D. la sua coordinata x

D20.4 Che cos'è il GPGPU?

A. Un'uscita video più veloce
B. L'uso dei processori grafici per calcoli non grafici
C. Un bus per schede video
D. Un tipo di memoria video

20.8 Esercizi

Esercizio 20.1 (prima su carta, poi su PRG32)

1. Calcolate la memoria per un frame 4K in True Color, e i byte al secondo a 60 frame al secondo.
2. Il bus del display di PRG32 lavora a 27 MHz: quanto tempo serve per inviare 20 righe di 320 pixel a 16 bit per pixel?
3. Disegnate una linea diagonale pixel per pixel, con `prg32_gfx_pixel` in un ciclo.
4. Riempite lo schermo una volta con `prg32_gfx_clear` e una volta con 64 000 chiamate a `prg32_gfx_pixel`: confrontate le frequenze dei frame.
5. Cambiate un elemento della palette con `prg32_palette_set` a ogni frame, e guardate cambiare insieme tutti i pixel che lo usano.

Risultato atteso. Su carta: 24 883 200 byte per frame, circa 1,5 GB al secondo; 102 400 bit, circa 3,8 ms. Su PRG32: la rotazione della palette anima lo schermo senza ridisegnare un solo pixel. È la ragione per cui il colore indicizzato era amato dai programmatori di giochi: uno store cambia migliaia di pixel.

20.9 Glossario

Inglese	Italiano
peripheral	periferica
controller	controller (unità di controllo della periferica)
bus width	larghezza del bus
transfer rate	velocità di trasferimento
video card	scheda video
resolution	risoluzione
colour depth	profondità di colore
palette	tavolozza (palette)
video memory	memoria video
device driver	driver di periferica

20.10 Conclusioni e letture

I dispositivi di ingresso/uscita sono un componente fondamentale dei calcolatori, e ciascuno è gestito da un controller che lo interfaccia al bus. Un bus è caratterizzato da larghezza, velocità e velocità di trasferimento, e la sua larghezza di banda è un limite attorno al quale progettare. Le schede video rendono possibile visualizzare grafica e testo: un frame buffer, una risoluzione, una profondità di colore, un'uscita. I loro processori, le GPU, sono usati anche per il calcolo ad alte prestazioni.

Letture. Tanenbaum e Austin (2013), capitoli 2 e 3 (bus, display); *Playing with RISC-V*, capitolo 7, il gioco grafico (Montella et al., 2026b); `docs/hardware/ili9341.md` nel repository di PRG32.

CAPITOLO 21

Ingresso e uscita

"Input! Ho bisogno di input!"
— Numero 5, nel film *Corto circuito*, John Badham, 1986

In questo capitolo

Alla fine di questo capitolo saprete

- descrivere come si forma un'immagine su un CRT e su un LCD, e che cosa aggiunge una matrice attiva TFT;
- spiegare il percorso della pressione di un tasto: matrice, codificatore, registro a scorrimento, interfaccia seriale;
- dire che cosa invia un mouse al calcolatore, e quanto spesso;
- descrivere le fasi della stampa laser;
- spiegare come il kit PRG32 legge il suo joystick, e confrontare polling, interruzioni e DMA.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI polling, interruzioni, DMA; display CRT e LCD; tastiera e mouse; le stampanti.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI niente.

21.1 Riscaldamento sulla macchina

```
steer_update:
    addi sp, sp, -16
    sw   ra, 12(sp)

    sw   s0, 8(sp)

    call prg32_input_read       # a0 = one bit per button
    mv   s0, a0                 # keep it in a saved register
```

```
9
10      andi t0, s0, 0x10          # button A
11      beqz t0, 1f
12      li   a0, 0                 # channel
13      li   a1, 0                 # instrument
14      li   a2, 69                # note: A4, 440 Hz
15      li   a3, 255               # volume
16      li   a4, 120               # milliseconds
17      call prg32_audio_note
18  1:  andi t0, s0, 0x01          # LEFT: s0 survived the call
19      beqz t0, 2f
20      ...                        # move left
21  2:  ...
```

Listato 21.1. Leggere il mondo, una volta per frame.

Leggete il joystick fin dai primi capitoli (Listato 21.1). Ogni pulsante è un interruttore su un piedino di ingresso del chip: premuto o rilasciato, un bit. Il runtime legge i piedini, che sono registri mappati in memoria configurati come ingressi, e li impacchetta in una parola: una rappresentazione decodificata, nel senso del Capitolo 11. E il gioco chiede una volta per frame: questo si chiama *polling* (interrogazione ciclica).

Il listato ripete anche una lezione del Capitolo 19. La maschera dei pulsanti è tenuta in `s0`, un registro salvato, e quindi sopravvive alla chiamata che suona la nota: il segnalino si muove con **LEFT** anche mentre suona **A**. In `t0` la maschera andrebbe persa alla chiamata. (Il programma è stato compilato come cartuccia ed eseguito sull'esecutore senza interfaccia di PRG32-QT.)

Una tastiera ha un centinaio di tasti, un mouse si muove in due dimensioni, un display ha migliaia di pixel: come parlano con un processore?

21.2 I display

21.2.1 Il video: un'introduzione

I primi display mostravano soprattutto caratteri, in bianco e nero; oggi sono grafici e a colori. Un display grafico è una matrice di punti, i *pixel*, ciascuno acceso con un'intensità diversa, e un'immagine si compone accendendo e spegnendo i pixel. Una diversa densità di pixel dà una diversa risoluzione; i colori vanno da 256 (8 bit) a 16 milioni (24 bit) e oltre. La dimensione si misura in pollici lungo la diagonale. Il display del kit PRG32 ha una diagonale di 2,8 pollici, 320 per 240 pixel e 65 536 colori.

21.2.2 Il tubo a raggi catodici

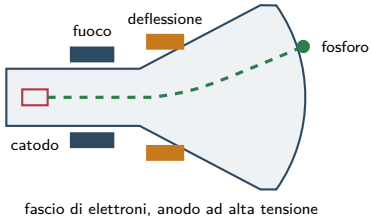
In un *tubo a raggi catodici* (CRT; Tabella 21.1, Figura 21.1) il catodo è un piccolo elemento metallico riscaldato fino all'incandescenza, che emette elettroni per effetto termoionico. Dentro il tubo gli elettroni sono diretti in un fascio da un'alta differenza di potenziale tra catodo e anodo, e altri campi elettromagnetici mettono a fuoco il fascio. Il fascio, il «pennello elettronico», è deflesso da campi magnetici (la forza di Lorentz) o da campi elettrici, e colpisce un punto sulla superficie interna dello schermo, che è rivestita di fosforo e si illumina. Far scorrere il fascio riga per riga, molte volte al secondo, dipinge l'immagine.

Tabella 21.1. Breve storia del tubo a raggi catodici.

Anno	Chi	Che cosa
primi anni 1870	William Crookes	il tubo di Crookes: il primo tubo a raggi catodici, uno strumento di laboratorio
1897	Karl Ferdinand Braun	il tubo di Braun e il primo oscilloscopio a raggi catodici: la prima applicazione pratica
1907	Boris Rosing	un CRT mostra semplici forme geometriche da un segnale video ricevuto: la radice della televisione
1934	Telefunken	il primo televisore con un CRT sul mercato, in Germania
anni 1970–1990	l'industria dei calcolatori	il monitor CRT: il display di ogni terminale e personal computer



(a) Un Macintosh compatto: un CRT da 9 pollici



(b) Lo schema di un CRT



(c) Un oscilloscopio usato come display

Figura 21.1. Il tubo a raggi catodici.

21.2.3 Display a cristalli liquidi

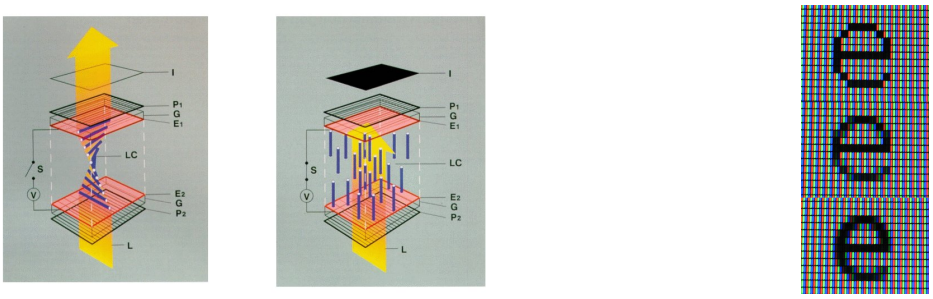
I monitor LCD hanno sostituito quasi del tutto il tubo a raggi catodici: danno un'immagine migliore, con meno ingombro, peso e potenza, e affaticano meno gli occhi. I *cristalli liquidi* sono molecole organiche viscose che scorrono come un liquido ma hanno una struttura spaziale. Un campo elettrico cambia l'orientamento delle molecole, e con esso una proprietà ottica del cristallo: l'angolo di polarizzazione della luce che lo attraversa. Un cristallo è una valvola per la luce, aperta e chiusa da una tensione.

21.2.4 Dentro un pannello LCD

Lo schermo è fatto di molte celle, spesso una per pixel, in cui sono intrappolati i cristalli liquidi (Figura 21.2). Ogni cella ha contatti elettrici, così che si può applicare un campo al liquido che contiene. Le celle stanno tra due filtri polarizzatori i cui assi sono perpendicolari tra loro, con una luce, la *retroilluminazione*, dietro quello posteriore. La polarizzazione decide se la luce passa: quando i cristalli non sono pilotati la luce viene ruotata e attraversa il polarizzatore anteriore; quando sono pilotati viene completamente bloccata. In uno schermo a colori ogni cella è divisa in tre sezioni con un filtro rosso, uno verde e uno blu: i tre campi del colore RGB565 del Capitolo 4 finiscono qui, come tre tensioni.

21.2.5 Pilotare le celle: dai segmenti alla matrice attiva

Nelle calcolatrici e negli orologi digitali c'è un contatto elettrico per segmento, il che diventa impossibile al crescere del numero di elementi. Una *matrice passiva* ha un gruppo di contatti



(a) Gli strati di un LCD

(b) Sottopixel rossi, verdi e blu

Figura 21.2. Display a cristalli liquidi.

per riga e uno per colonna, non per pixel; viene pilotato un solo pixel alla volta, e gli altri devono ricordare il proprio stato. Le conseguenze sono la *persistenza*, le scie dietro gli oggetti in movimento, e la diafonia tra celle vicine.

Gli schermi ad alta risoluzione usano una *matrice attiva*. Il pannello contiene una pellicola sottile di transistor (TFT, *thin-film transistor*) su uno strato semiconduttore trasparente; il campo è applicato esattamente dove serve da un transistor in ogni cella, e il transistor memorizza lo stato elettrico del suo pixel mentre gli altri vengono aggiornati. Ogni pixel è, in effetti, una cella di memoria analogica. Il display del kit è un pannello TFT con un controller ILI9341. Nei più recenti display OLED ogni pixel emette la propria luce, e non serve alcuna retroilluminazione.

21.3 Tastiera e mouse

21.3.1 La tastiera

La tastiera deriva da quella delle macchine da scrivere, ed è il principale dispositivo per l'ingresso di caratteri. I suoi tasti sono raggruppati: tasti alfanumerici; tasti speciali come invio e backspace; tasti freccia e tasti funzione. Tastiera e video non sono collegati direttamente tra loro: è il processore che riproduce sul video l'informazione battuta sulla tastiera. L'eco di un tasto sullo schermo è un programma: leggere un codice, disegnare un carattere. Le tastiere di oggi si collegano attraverso USB, o senza fili.

21.3.2 Il percorso della pressione di un tasto

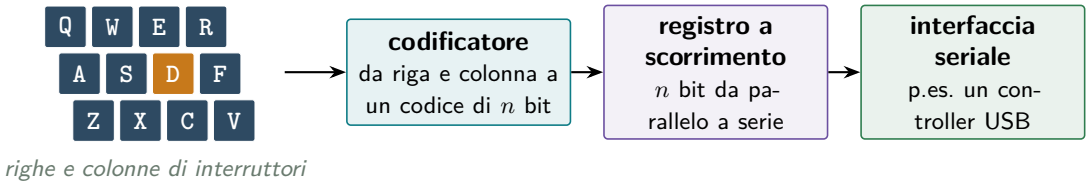


Figura 21.3. Il percorso della pressione di un tasto.

I tasti di una tastiera sono interruttori disposti in righe e colonne (Figura 21.3). Un tasto premuto unisce una riga a una colonna; un *codificatore* trasforma quella combinazione di riga e colonna in un codice di tasto di n bit; un *registro a scorrimento* porta gli n bit da parallelo a seriale; e un'*interfaccia seriale*, per esempio un controller USB, li invia al calcolatore.

Concetto

Ogni blocco della Figura 21.3 è una macchina che avete già progettato. È il frutto dei Capitoli da 11 a 14.

21.3.3 Il mouse

Il mouse fu inventato negli anni Sessanta da Douglas Engelbart (il primo prototipo è del 1964, il brevetto fu depositato nel 1967 e la celebre dimostrazione pubblica è del 1968): fu il primo dispositivo di puntamento della storia. Era una scatola di legno con due ruote sotto e un filo che usciva dal retro, come la coda di un topo. Le due ruote rilevavano i movimenti lungo l'asse orizzontale e quello verticale, e l'elettronica li trasformava in segnali elettrici. In un mouse *meccanico* una pallina fa girare due rulli, ciascuno dei quali aziona un contatore; in un mouse *ottico* una sorgente di luce e una minuscola telecamera misurano il movimento della superficie sottostante.

21.3.4 Che cosa invia un mouse

Ogni volta che si è spostato di una distanza minima, circa un centesimo di pollice, chiamata *mickey*, il mouse invia tre byte: di quante unità si è spostato lungo x dall'ultimo rapporto; lo spostamento lungo y; e lo stato dei pulsanti. I due spostamenti sono numeri con segno in complemento a due (Capitolo 5). I mouse moderni riferiscono fino a mille volte al secondo. Dentro, il mouse è fatto di due contatori, un registro a scorrimento che mette in serie i tre byte, e un'interfaccia seriale; dal lato del calcolatore il software di controllo, il *driver di periferica*, sovrintende alla comunicazione con il processore.

21.4 Le stampanti

21.4.1 La stampante

Una stampante è un dispositivo di uscita che stampa informazione su carta. La sua velocità si misura in righe al minuto, caratteri al secondo o pagine al minuto; la sua risoluzione, cioè la qualità della stampa, in punti per pollice. Le stampanti a margherita e a catena sono ormai in disuso; una stampante *ad aghi* colpisce con degli aghi un nastro inchiostrato; una stampante *a getto d'inchiostro* lancia minuscole gocce di inchiostro; e poi c'è la stampante laser. Una stampante è anche un piccolo calcolatore: ha processore, memoria e firmware propri.

21.4.2 La stampante laser

La stampante laser deriva dalla fotocopiatrice (Figura 21.4). Funziona in sei fasi.

1. Un tamburo viene caricato elettricamente, a circa 1 000 volt.
2. Un raggio laser, riflesso da uno specchio rotante, scandisce la lunghezza del tamburo; il raggio è modulato in modo da produrre una configurazione di punti chiari e scuri.
3. I punti colpiti dal raggio perdono la carica. Dopo ogni riga il tamburo ruota di una frazione di grado.
4. Il toner è attratto dai punti ancora carichi: si forma un'immagine della riga.

5. Il tamburo viene premuto contro la carta e vi trasferisce il toner; rulli riscaldati lo fissano in modo permanente.
6. Il tamburo viene scaricato e pulito.

La pagina viene prima costruita come bitmap nella memoria della stampante: un frame buffer per la carta.

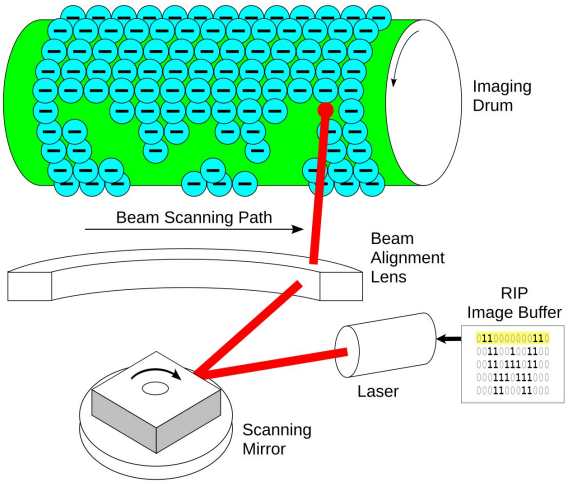


Figura 21.4. Il laser scrive la pagina sul tamburo, riga per riga.

21.5 Ritorno alla macchina

21.5.1 I dispositivi del kit PRG32

Tabella 21.2. I dispositivi del kit PRG32.

Dispositivo	**Direzione**	**Come è collegato**	**Come lo usa una cartuccia**
joystick e pulsanti	ingresso	un interruttore per piedino di ingresso (GPIO)	`prg32_input_read`: un bit per pulsante
display, TFT 320×240	uscita	bus seriale SPI, con DMA	le chiamate `prg32_gfx_*` disegnano nel frame buffer
amplificatore e altoparlante	uscita	bus seriale I2S	le chiamate `prg32_audio_*`: note, campioni, musica
radio Wi-Fi	ingresso e uscita	dentro il chip	caricamento delle cartucce, lo Store, multigiocatore
porta seriale USB	ingresso e uscita	USB	programmazione del firmware, messaggi di debug

Un dettaglio della Tabella 21.2 era stato promesso nel Capitolo 10: un interruttore rimbalza per qualche millisecondo quando viene premuto, e campionarlo una volta per frame, ogni poche decine di millisecondi, nasconde il rimbalzo.

21.5.2 Tre modi di trattare un dispositivo

Polling (I/O programmato).
Il programma chiede al dispositivo, ancora e ancora, se ha qualcosa. È semplice, ed è ciò che fa un gioco una volta per frame; è uno spreco quando non succede niente.

Interruzioni.
Il dispositivo chiede attenzione quando gli serve; il processore sospende ciò che sta facendo, serve il dispositivo e riprende (Capitolo 19).

Accesso diretto alla memoria.
Il dispositivo scambia da solo interi blocchi di dati con la memoria; il processore avvia soltanto il trasferimento e viene avvisato quando termina (Capitolo 15).

Ritorno alla macchina

Sul kit tutti e tre sono al lavoro nello stesso momento: il gioco interroga il joystick, il timer che scandisce i frame interrompe, e il display e il suono sono alimentati dal DMA.

21.6 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l21.S` nel repository del corso, stampato per intero nel Listato 21.2. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 21.5 è il frame scritto dal test.

Sullo schermo.
Un quadrato giallo al centro dello schermo. LEFT lo sposta a sinistra; A suona una nota, e il quadrato continua a muoversi mentre la nota suona.

Che cosa premere.
LEFT muove il quadrato; A suona una nota.

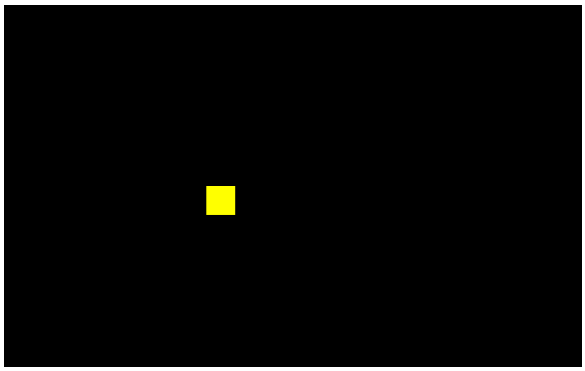


Figura 21.5. `l21.S` su PRG32-QT.

```
1 # ARC12 Computer Architecture - Lecture 21: Input and Output
2 # l21.S: read the world, once per frame
3 #
4 # On the screen: a yellow square in the middle of the screen. LEFT moves it to
      the
5 # left; A plays a note, and the square keeps moving while the note sounds.
```

```
# Build: python3 -m prg32 cartridge build l21.S --portable \
#            --entry-prefix l21 --name l21 --out l21.prg32
.option norelax

.section .text
.global l21_init
.global l21_update
.global l21_draw

l21_init:
    ret

l21_update:
steer_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)

    call prg32_input_read      # a0 = one bit per button
    mv   s0, a0                # keep it in a saved register

    andi t0, s0, 0x10          # button A
    beqz t0, 1f
    li   a0, 0                 # channel
    li   a1, 0                 # instrument
    li   a2, 69                # note: A4, 440 Hz
    li   a3, 255               # volume
    li   a4, 120               # milliseconds
    call prg32_audio_note
1:  andi t0, s0, 0x01          # LEFT: s0 survived the call
    beqz t0, 2f
    la   t1, marker_x
    lw   t2, 0(t1)
    addi t2, t2, -2
    sw   t2, 0(t1)
2:  lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

l21_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0
    call prg32_gfx_clear
    la   t0, marker_x
    lw   a0, 0(t0)
    li   a1, 100
    li   a2, 16
    li   a3, 16
    li   a4, 0xffe0
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

```

```
.section .data
marker_x: .word 152
```

Listato 21.2. Il programma completo del capitolo, `code/l21.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 21.3; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l21.S --portable \
    --entry-prefix l21 --name l21 --out l21.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l21.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 21.3. Compilare `l21.S` ed eseguirlo su PRG32-QT.

21.7 Verificate di aver capito

D21.1 Che cosa fa il codificatore di un controller di tastiera?

A. Illumina il tasto premuto
B. Trasforma una combinazione di riga e colonna in un codice di tasto
C. Invia il codice al monitor
D. Conta per quanto tempo un tasto resta premuto

D21.2 In un LCD a colori, un pixel è fatto di:

A. una cella che cambia colore
B. tre celle con filtri rosso, verde e blu
C. un punto di fosforo colpito da elettroni
D. un transistor che emette luce

D21.3 Perché un LCD a matrice passiva mostra scie dietro gli oggetti in movimento?

A. La retroilluminazione è troppo debole
B. I pixel sono pilotati uno alla volta e nel frattempo devono mantenere il proprio stato
C. Ha troppo pochi colori
D. La scheda video è lenta

D21.4 Un gioco PRG32 legge il joystick una volta per frame. Questa tecnica si chiama:

A. accesso diretto alla memoria
B. ingresso a interruzioni
C. polling

D. mappatura in memoria

21.8 Esercizi

Esercizio 21.1 (prima su carta, poi su PRG32)

1. Una tastiera ha 104 tasti: quanti bit richiede il codice di tasto? Quante righe e colonne potrebbe avere la matrice?
2. Una pagina di 8 per 11 pollici a 600 punti per pollice, un bit per punto: quanta memoria serve alla stampante per la bitmap?
3. Muovete un puntatore sullo schermo di PRG32 con il joystick, come farebbe un mouse: tenete x e y, e limitateli allo schermo.
4. Fate agire il pulsante **A** una sola volta per pressione (rilevamento del fronte, Capitolo 13).
5. Disegnate un punteggio a sette segmenti con dei rettangoli, usando la tabella del transcodificatore del Capitolo 11.

Risultato atteso. Su carta: 7 bit e, per esempio, 8 righe per 13 colonne; $4\,800 \times 6\,600$ punti, cioè 31 680 000 bit, circa 4 MB. Su PRG32: un puntatore che resta sullo schermo, e un pulsante che scatta una volta per pressione.

21.9 Glossario

Inglese	Italiano
cathode-ray tube	tubo a raggi catodici
liquid crystals	cristalli liquidi
backlight	retroilluminazione
active matrix	matrice attiva
keyboard / key	tastiera / tasto
pointing device	periferica di puntamento
dot-matrix / ink-jet printer	stampante ad aghi / a getto d'inchiostro
drum / toner	tamburo / toner
polling	interrogazione ciclica
device driver	driver di periferica

21.10 Conclusioni e letture

I dispositivi di ingresso/uscita sono un componente fondamentale dei calcolatori. La tecnologia dei display ha fatto progressi enormi, dal fascio di elettroni a un transistor dietro ogni pixel. Tastiera, mouse e stampante sono cambiati poco nella forma, e dentro sono costruiti con codificatori, contatori e registri a scorrimento. Un dispositivo raggiunge il processore attraverso un controller e un bus, ed è trattato con il polling, con le interruzioni o con il DMA. Sul kit PRG32 l'intera catena, da un interruttore a un pixel, può essere seguita istruzione per istruzione.

Letture. Tanenbaum e Austin (2013), paragrafo 2.4 (ingresso/uscita); Stallings (2021), il capitolo sull'ingresso/uscita; *Playing with RISC-V*, capitolo 6, «State and Input» (Montella et al., 2026b).

Il quarto blocco è completo. Il prossimo capitolo conclude Verm1, e quello successivo pubblica il gioco.

CAPITOLO 22

Verm1, parte 4: il gioco finito

"L'ottimizzazione prematura è la radice di tutti i mali."
— Donald Knuth, 1974

In questo capitolo

Alla fine di questo capitolo avrete

- il punteggio e il punteggio migliore scritti in cifre, convertiti dalla vostra divisione per dieci;
- un verme che va più veloce al crescere del punteggio, fino a un limite;
- due suoni: una nota breve per un pasto, una lunga e grave per la fine;
- una funzione `draw` che tocca una manciata di celle per passo invece dell'intero schermo;
- Verm1 finito, pronto per essere impacchettato nel Capitolo 23.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI quoziente e resto; caratteri e stringhe; la virgola fissa; la convenzione di chiamata; gli array in memoria; macchine a stati finiti; il frame buffer; il ciclo del processore.

DA UTILIZZARE OGGI gli strumenti di PRG32, il debugger di PRG32-QT (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI niente.

In quest'ultimo tutorial sul programma entra nel gioco la macchina stessa. La frase di Knuth ne è il motto: il gioco è stato fatto *funzionare* nella parte 3, e solo ora, con una misura in mano, viene reso *veloce*. La Tabella 22.1 elenca ciò che si prende dai Capitoli da 18 a 21; torna anche il Capitolo 4, perché una cifra diventa un carattere sommando 48, il codice ASCII di `0`. Il file di riferimento è `verm1_part4.S` (Appendice D).

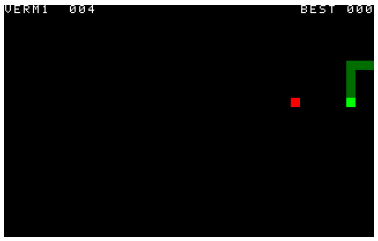
Partite da una parte 3 che funziona; se la vostra non funziona, copiate `verm1_part3.S`. Il capitolo ha quattro passi, e il programma funziona dopo ciascuno.

Tabella 22.1. Che cosa la parte 4 prende dai capitoli sulla macchina.

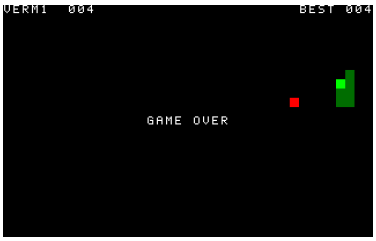
Da	Idea	Dove compare
Capitolo 18	la convenzione di chiamata: registri `a`, `t`, `s`, `ra`, lo stack	piccole funzioni con contratti; funzioni foglia
Capitoli 5, 6	`divu` e `remu` (senza segno), la M di RV32IMAC: quoziente e resto	le cifre del punteggio
Capitolo 19	il ciclo del processore: ogni istruzione costa tempo	la misura prima e dopo il passo 4
Capitolo 20	lo schermo è memoria: scrivere pixel è il costo	disegnare solo ciò che è cambiato
Capitolo 21	i dispositivi appartengono al runtime: il programma chiede, il runtime fa	`prg32_audio_note`, con una durata



(a) `TITLE`, con il punteggio migliore



(b) `PLAY`: il punteggio in cifre



(c) `OVER`

Figura 22.1. Dove va questo capitolo.

Prima di cominciare: misurate

Eseguite la parte 3 in modalità debug e annotate ciò che la telemetria mostra durante una partita: istruzioni completate, frame al secondo, frame in ritardo. I numeri serviranno nel passo 5.

22.1 Passo 1: il punteggio in cifre

22.1.1 Un numero non è le sue cifre

La variabile `score` contiene un numero binario: 137 è `0x89`, un byte. Lo schermo vuole tre caratteri, `'1'`, `'3'`, `'7'`, cioè i byte `0x31`, `0x33`, `0x37`. La conversione è l'algoritmo del Capitolo 5, eseguito dal programma invece che a mano: dividere per dieci, tre volte.

$$\begin{aligned} 137/10 &= 13, && \text{resto 7: le unità;} \\ 13/10 &= 1, && \text{resto 3: le decine;} \\ 1/10 &= 0, && \text{resto 1: le centinaia.} \end{aligned}$$

Le cifre escono da destra, e sono memorizzate dall'ultimo carattere al primo. Una cifra d diventa un carattere sommando 48.

22.1.2 `to_decimal`

```
to_decimal:                     # a0 = value 0..999, a1 = address of 3 chars
    li   t0, 10
    remu t1, a0, t0             # the units: the remainder of value / 10
    addi t1, t1, 48             # 48 is the ASCII code of '0'
    sb   t1, 2(a1)
    divu a0, a0, t0
    remu t1, a0, t0             # the tens
    addi t1, t1, 48
    sb   t1, 1(a1)
    divu a0, a0, t0
    remu t1, a0, t0             # the hundreds
    addi t1, t1, 48
    sb   t1, 0(a1)
    ret                         # a leaf function: no stack frame needed
```

Listato 22.1. Passo 1 **(nuovo)**: `to_decimal`.

Il contratto del Listato 22.1 è: `a0` = valore, `a1` = dove vanno i tre caratteri. `remu` dà la cifra e `divu` ciò che resta: due istruzioni dell'estensione M. `sb` memorizza un byte agli scostamenti 2, 1 e 0 da `a1`. La funzione è una foglia che usa solo registri `t`: non le serve alcun frame. Valori sopra 999 mostrerebbero solo le ultime tre cifre, un limite che nessun giocatore incontrerà presto. Le istruzioni di divisione sono lente, decine di cicli di clock ciascuna su un processore piccolo; dieci per passo del verme non sono niente.

22.1.3 `draw_score`: due numeri, due testi

```
draw_score:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, score
    lw   a0, 0(t0)
    la   a1, score_txt
    call to_decimal             # the score, as three characters
    li   a0, 56
    li   a1, 0
    la   a2, score_txt
    call draw_text
    la   t0, best
    lw   a0, 0(t0)
    la   a1, best_txt+5         # after the letters "BEST "
    call to_decimal
    li   a0, 256
    li   a1, 0
    la   a2, best_txt
    call draw_text
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.2. Passo 1 **(modificato)**: `draw_score`.

Sostituite la `draw_score` della parte 3 con il Listato 22.2: le tacche spariscono. Le cifre vengono scritte in una stringa in RAM, e poi la stringa viene disegnata come qualsiasi altra; l'indirizzo `best_txt+5` salta la parola `BEST` e lo spazio, cinque caratteri. Il testo è

disegnato con il suo colore di sfondo, così che le nuove cifre coprono le vecchie senza alcuna cancellazione.

22.1.4 Le variabili della parte 4

```
.section .data                  # variables: they live in RAM
state:     .word TITLE
prev_btn:  .word 0              # the buttons one frame ago
dir:       .word 0
want:      .word 0
step_q8:   .word 0
score:     .word 0
best:      .word 0              # lost when the power goes
food_x:    .word 0
food_y:    .word 0
head_i:    .word 0              # ring index of the head
length:    .word 0              # number of cells of the worm
body_x:    .space 256           # the ring: one byte per segment
body_y:    .space 256
grid:      .space COLS*ROWS     # one byte per cell: 1 = worm
dirty:     .word 2              # 0 nothing to draw, 1 a step, 2 everything
old_x:     .word 0              # where the head was before the step
old_y:     .word 0
erase_on:  .word 0              # 1 if the tail left a cell
erase_x:   .word 0
erase_y:   .word 0
score_txt: .asciz "000"         # filled in by to_decimal
best_txt:  .asciz "BEST 000"
```

Listato 22.3. Passo 1 **(modificato)**: `.data`.

Sostituite la `.data` della parte 3 con il Listato 22.3, e scrivetela tutta ora. Le due stringhe `score_txt` e `best_txt` sono in `.data` e non in `.rodata`, perché il programma vi scrive. Le sei variabili da `dirty` a `erase_y` non sono usate fino al passo 3.

Eseguite: le cifre

1. Compilate ed eseguite, premete **A**: la riga in alto dice `VERM1 000` e, a destra, `BEST 000`.
2. Giocate e mangiate: `001`, `002`, ...
3. Perdete: nella schermata di fine partita `BEST` mostra il vostro punteggio.
4. Nel debugger trovate `score_txt` in memoria e leggetene i byte: `30 30 33 00` dopo tre pasti.

22.2 Passo 2: i livelli

La velocità diventa una funzione del punteggio.

```
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
.equ GREEN, 0x07e0          # the head
```

```
.equ BODY,  0x03e0          # the body: a darker green
.equ RED,   0xf800          # the food
.equ CELL,  8               # a cell is 8 x 8 pixels
.equ COLS,  40
.equ ROWS,  24
.equ TOP,   8
.equ SPEED_MIN, 32          # 32/256 of a cell per frame at the start
.equ SPEED_MAX, 96          # the worm never goes faster than this
.equ BTN_LEFT,  0x01
.equ BTN_RIGHT, 0x02
.equ BTN_UP,    0x04
.equ BTN_DOWN,  0x08
.equ BTN_A,     0x10
.equ DIR_RIGHT, 0           # bit 0 = vertical, bit 1 = backwards
.equ DIR_DOWN,  1
.equ DIR_LEFT,  2
.equ DIR_UP,    3
.equ TITLE, 0               # the three states of the game
.equ PLAY,  1
.equ OVER,  2
```

Listato 22.4. Passo 2 **(modificato)**: le costanti. Solo le due righe `SPEED` differiscono dalla parte 3.

`SPEED_Q8` è sostituita da un minimo e un massimo (Listato 22.4): 32/256, un passo ogni 8 frame, all'inizio, e 96/256, tre passi ogni 8 frame, al massimo.

```
    la   t0, score
    lw   t3, 0(t0)
    slli t3, t3, 2
    addi t3, t3, SPEED_MIN      # speed = 32 + 4 * score, in 1/256 of a cell
    li   t4, SPEED_MAX
    ble  t3, t4, 8f
    mv   t3, t4                 # ... but never more than 96
8:  la   t0, step_q8
    lw   t1, 0(t0)
    add  t1, t1, t3             # add a fraction of a cell
    li   t2, 256
    blt  t1, t2, 6f             # less than one whole cell: wait
    sub  t1, t1, t2
    sw   t1, 0(t0)
    call step                   # one whole cell: move
    j    7f
6:  sw   t1, 0(t0)
7:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.5. Passo 2 **(modificato)**: il timer alla fine di `play_update`.

La velocità è $32 + 4 \times \textit{score}$: uno scorrimento e una somma (Listato 22.5). Poi `ble` e `mv` prendono il minore tra la velocità e 96. È una *saturazione*: il valore segue una retta e poi resta al limite, come la limitazione del Capitolo 5. Il limite è raggiunto a 16 punti (Tabella 22.2). Con la virgola fissa la velocità cambia di un sessantaquattresimo di cella per frame a ogni pasto: nessuna velocità intera potrebbe essere così fine.

Tabella 22.2. I livelli, in numeri. 256/*velocità* dà i frame per passo.

Punteggio	Velocità (1/256 cella/frame)	Frame per passo	Celle/s a 30 fps
0	32	8	3,75
4	48	5,3	5,6
8	64	4	7,5
12	80	3,2	9,4
16 e oltre	96	2,7	11,25

Le frazioni di frame sono reali: a 48/256 i passi arrivano ogni 5 o 6 frame, 5,33 in media. Calcolate la riga del punteggio 2 per esercizio.

Eseguite: sempre più veloce

1. Compilate ed eseguite: il verme parte più lentamente che nella parte 3, e acquista velocità a ogni pasto.
2. Contate con il debugger i frame tra due passi a punteggio 0: otto.
3. Arrivate a 16 punti: la velocità non cambia più.
4. Provate `SPEED_MAX` = 256, se ne avete il coraggio.

22.3 Passo 3: `step` dice che cosa ha cambiato

Questo passo aggiunge i suoni, e le annotazioni che serviranno a `draw` nel passo 4.

22.3.1 `play_note`: una funzione, qualsiasi nota

```
play_note:                      # a2 = MIDI note, a4 = milliseconds
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0                  # channel
    li   a1, 0                  # instrument
    li   a3, 255                # volume
    call prg32_audio_note
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.6. Passo 3 **(modificato)**: `play_note` sostituisce `eat_sound`.

Il chiamante sceglie la nota e la durata, in `a2` e `a4`; la funzione riempie gli altri tre argomenti (Listato 22.6). Gli argomenti restano nei registri in cui `prg32_audio_note` li aspetta: progettare un contratto attorno al chiamato fa risparmiare spostamenti.

22.3.2 `step`: che cosa lascia a `draw`

La funzione `step` della parte 3 viene modificata in cinque punti.

```
    la   t0, head_i
    lw   t5, 0(t0)              # t5 = ring index of the head
```

```
    la   t0, body_x
    add  t0, t0, t5
    lbu  s0, 0(t0)              # one byte: the column of the head
    la   t0, old_x
    sw   s0, 0(t0)              # remember where the head was
    add  s0, s0, t2
    bgez s0, 1f
    li   s0, COLS-1             # off the left edge: wrap
1:  li   t4, COLS
    blt  s0, t4, 2f
    li   s0, 0
2:  la   t0, body_y
    add  t0, t0, t5
    lbu  s1, 0(t0)
    la   t0, old_y
    sw   s1, 0(t0)
    add  s1, s1, t3
    bgez s1, 3f
    li   s1, ROWS-1
3:  li   t4, ROWS
    blt  s1, t4, 4f
    li   s1, 0
4:
```

Listato 22.7. Passo 3 **(modificato)**: `step` ricorda la vecchia testa.

Si aggiungono due store (Listato 22.7): `old_x` e `old_y`, la cella che la testa sta lasciando, che `draw` ridipingerà nel colore del corpo.

```
    la   t0, food_x
    lw   t1, 0(t0)
    xor  t1, t1, s0
    la   t0, food_y
    lw   t2, 0(t0)
    xor  t2, t2, s1
    or   s2, t1, t2             # s2 = 0 if the new head is on the food
    la   t0, length
    lw   t1, 0(t0)
    bnez s2, 5f                 # not eating: the tail moves on
    li   t2, 255
    bge  t1, t2, 5f             # the ring is full: no more growth
    addi t1, t1, 1
    sw   t1, 0(t0)              # eating: one cell longer, the tail stays
    la   t0, erase_on
    sw   zero, 0(t0)            # ... so there is no cell to erase
    j    6f
5:  la   t0, head_i
    lw   t2, 0(t0)
    sub  t2, t2, t1
    addi t2, t2, 1
    andi t2, t2, 255            # ring index of the tail
    la   t0, body_x
    add  t0, t0, t2
    lbu  t3, 0(t0)
    la   t0, body_y
```

```
    add  t0, t0, t2
    lbu  t4, 0(t0)
```

Listato 22.8. Passo 3 **(modificato)**: `step`, quando il verme mangia.

Quando il verme mangia, la coda resta e non c'è niente da cancellare: `erase_on` = 0 lo dice a `draw` (Listato 22.8).

```
    la   t0, erase_x
    sw   t3, 0(t0)              # remember the cell the tail leaves
    la   t0, erase_y
    sw   t4, 0(t0)
    la   t0, erase_on
    li   t5, 1
    sw   t5, 0(t0)
    li   t5, COLS
    mul  t4, t4, t5
    add  t4, t4, t3             # cell number of the tail = row * 40 + column
    la   t0, grid
    add  t0, t0, t4
    sb   zero, 0(t0)            # the tail cell is free again
6:  li   t5, COLS
    mul  t4, s1, t5
    add  t4, t4, s0             # cell number of the new head
    la   t0, grid
    add  t0, t0, t4
    lbu  t1, 0(t0)
    beqz t1, 7f                 # a free cell: go on
    call game_is_over           # the worm bit itself
    j    9f
```

Listato 22.9. Passo 3 **(modificato)**: `step` annota la coda, e chiama la fine.

Quando non mangia, la coda lascia una cella, e `step` ricorda quale: `erase_on` = 1 significa che `draw` deve dipingerla di nero (Listato 22.9). La fine di questo frammento è più corta che nella parte 3, perché la collisione è ora una sola chiamata, a `game_is_over`.

```
7:  li   t1, 1
    sb   t1, 0(t0)              # occupy the new cell
    la   t0, head_i
    lw   t1, 0(t0)
    addi t1, t1, 1
    andi t1, t1, 255            # the ring wraps after entry 255
    sw   t1, 0(t0)
    la   t0, body_x
    add  t0, t0, t1
    sb   s0, 0(t0)              # write the new head into the ring
    la   t0, body_y
    add  t0, t0, t1
    sb   s1, 0(t0)
```

Listato 22.10. Passo 3 **(modificato)**: `step` scrive la nuova testa (come nella parte 3).

```
    la   t0, dirty
```

```
    li   t1, 1
    sw   t1, 0(t0)              # the screen must follow: a few cells
    bnez s2, 9f
    la   t0, score
    lw   t1, 0(t0)
    addi t1, t1, 1
    sw   t1, 0(t0)
    call place_food
    li   a2, 72                 # a short high note
    li   a4, 60
    call play_note
9:  lw   s2, 0(sp)
    lw   s1, 4(sp)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.11. Passo 3 **(nuovo)**: la fine di `step`.

Alla fine (Listato 22.11) `dirty` = 1 registra che è avvenuto un passo e che `draw` ha del lavoro da fare. Un pasto è un punto, nuovo cibo e una nota breve e acuta: la nota 72 per 60 millisecondi, il suono della parte 2 attraverso la nuova funzione.

22.3.3 `game_is_over`: la transizione, in un solo posto

```
game_is_over:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, state
    li   t1, OVER
    sw   t1, 0(t0)              # PLAY --> OVER
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # a new screen: draw everything
    la   t0, score
    lw   t1, 0(t0)
    la   t0, best
    lw   t2, 0(t0)
    ble  t1, t2, 1f
    sw   t1, 0(t0)              # a new best score
1:  li   a2, 36                 # a long low note
    li   a4, 400
    call play_note
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.12. Passo 3 **(nuovo)**: `game_is_over`.

Una transizione della macchina a stati merita una funzione tutta sua (Listato 22.12): lo stato, le uscite e il suono stanno in un solo posto. La funzione memorizza `OVER`, aggiorna il punteggio migliore come nella parte 3, pone `dirty` = 2 perché va disegnata una nuova schermata, e suona una nota lunga e grave: la 36, per 400 millisecondi.

Eseguite: il suono della sconfitta

1. Compilate ed eseguite: il gioco ha esattamente l'aspetto che aveva dopo il passo 2.
2. Un pasto suona come prima; mordersi dà una nota lunga e grave.
3. Nel debugger guardate `erase_x` ed `erase_y` seguire la coda, un passo indietro.
4. `dirty` resta a 1 o 2: nessuno lo legge ancora.

Risultato atteso. Nessun cambiamento visibile: lo schermo è ancora disegnato per intero a ogni frame. Le nuove variabili sono scritte e non ancora usate.

22.4 Passo 4: disegnare solo ciò che è cambiato

22.4.1 Quanto costa un frame?

La parte 3 cancella 64 000 pixel e ridisegna il titolo, il punteggio, l'intero verme e il cibo, trenta volte al secondo. Ma tra due passi sullo schermo non cambia niente, e un passo cambia pochissimo: al più la cella che la coda ha lasciato, che diventa nera; la vecchia testa, che diventa corpo; la nuova testa; e forse il cibo e il punteggio.

Concetto

Ricordate che cosa è cambiato, e disegnate solo quello. L'immagine resta sullo schermo da sola: il frame buffer è memoria (Capitolo 20).

Un verme di 200 celle costa 200 rettangoli per frame nella parte 3, e tre per passo nella parte 4. È ciò che facevano i vecchi home computer, perché non avevano scelta.

22.4.2 `draw`: niente, poco, o tutto

```
verm1_draw:
    la   t0, dirty
    lw   t1, 0(t0)
    beqz t1, 3f                 # nothing changed: nothing to draw
    sw   zero, 0(t0)
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   t2, 2
    beq  t1, t2, 1f
    call draw_changes           # one step: a few cells
    j    2f
1:  call draw_full              # a new screen: everything
2:  lw   ra, 12(sp)
    addi sp, sp, 16
3:  ret
```

Listato 22.13. Passo 4 **(modificato)**: `verm1_draw`.

La variabile `dirty` ha tre valori (Listato 22.13). Con 0 la funzione ritorna subito, senza frame e senza chiamate: il caso comune, in cui non è cambiato niente, costa cinque istruzioni macchina. Con 1 è avvenuto un passo, e si disegnano i cambiamenti. Con 2 c'è una nuova schermata, e si disegna tutto. L'indicatore viene azzerato appena è stato letto.

22.4.3 `draw_changes`: tre celle e il punteggio

```
draw_changes:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, erase_on
    lw   t1, 0(t0)
    beqz t1, 1f
    la   t0, erase_x
    lw   a0, 0(t0)
    la   t0, erase_y
    lw   a1, 0(t0)
    li   a2, BLACK
    call draw_cell              # the cell the tail has left
1:  la   t0, old_x
    lw   a0, 0(t0)
    la   t0, old_y
    lw   a1, 0(t0)
    li   a2, BODY
    call draw_cell              # the old head is now body
    call draw_head
    call draw_food
    call draw_score
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.14. Passo 4 **(nuovo)**: `draw_changes`.

L'ordine conta (Listato 22.14): prima la cella che la coda ha lasciato, se c'è, in nero; poi la vecchia testa, nel colore del corpo; poi la nuova testa, il cibo e il punteggio. Poiché la testa può entrare nella cella che la coda ha appena lasciato, cancellare prima e disegnare la testa dopo dà l'immagine giusta.

22.4.4 `draw_full`: un'intera schermata

```
draw_full:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 0
    li   a1, 0
    la   a2, title
    call draw_text
    call draw_score
    la   t0, state
    lw   t1, 0(t0)
    li   t2, TITLE
    bne  t1, t2, 1f
    li   a0, 132                # TITLE: only an invitation
    li   a1, 96
    la   a2, press_a
    call draw_text
```

```
    j    2f
```

Listato 22.15. Passo 4 **(nuovo)**: `draw_full`, prima metà.

```
1:  call draw_worm              # PLAY and OVER: the whole scene
    call draw_food
    la   t0, state
    lw   t1, 0(t0)
    li   t2, OVER
    bne  t1, t2, 2f
    li   a0, 124
    li   a1, 96
    la   a2, game_over
    call draw_text
2:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.16. Passo 4 **(nuovo)**: `draw_full`, `PLAY` e `OVER`.

`draw_full` è il corpo della vecchia `verm1_draw`, ed è chiamata una volta per ogni cambio di stato (Listati 22.15 e 22.16). Il punteggio è disegnato anche nella schermata del titolo.

22.4.5 Tre piccole funzioni

```
draw_head:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, head_i
    lw   t1, 0(t0)
    la   t0, body_x
    add  t0, t0, t1
    lbu  a0, 0(t0)
    la   t0, body_y
    add  t0, t0, t1
    lbu  a1, 0(t0)
    li   a2, GREEN
    call draw_cell
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.17. Passo 4 **(nuovo)**: `draw_head`.

```
draw_food:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, food_x
    lw   a0, 0(t0)
    la   t0, food_y
    lw   a1, 0(t0)
    li   a2, RED
    call draw_cell
```

```
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.18. Passo 4 **(nuovo)**: `draw_food`.

```
draw_text:                      # a0 = x, a1 = y, a2 = address of the string
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a3, WHITE
    li   a4, BLACK
    call prg32_gfx_text8
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.19. Passo 4: `draw_text`, la funzione di servizio della parte 3.

22.4.6 Ogni transizione chiede una schermata

Tre piccole aggiunte completano il meccanismo: dovunque cambi lo stato, `dirty` diventa 2.

```
.section .text
.global verm1_init
.global verm1_update
.global verm1_draw

verm1_init:
    la   t0, state
    sw   zero, 0(t0)            # start on the title screen
    la   t0, prev_btn
    sw   zero, 0(t0)
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # draw the whole title screen, once
    ret
```

Listato 22.20. Passo 4 **(modificato)**: `verm1_init` chiede la prima schermata.

```
    la   t0, state
    lw   t1, 0(t0)              # t1 = present state
    li   t2, PLAY
    beq  t1, t2, u_play
    andi t3, s1, BTN_A          # TITLE or OVER: wait for a new press of A
    beqz t3, u_done
    li   t2, TITLE
    beq  t1, t2, u_start
    sw   t2, 0(t0)              # OVER --A--> TITLE
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # a new screen: draw everything
    j    u_done
u_start:
```

```
    call new_game               # TITLE --A--> PLAY
    j    u_done
u_play:
    mv   a0, s0
    call play_update
u_done:
    lw   s1, 4(sp)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.21. Passo 4 **(modificato)**: `verm1_update`, da `OVER` a `TITLE`.

```
    la   t0, head_i
    li   t1, 2
    sw   t1, 0(t0)              # the head is entry 2 of the ring
    la   t0, length
    li   t1, 3
    sw   t1, 0(t0)
    la   t0, dir
    sw   zero, 0(t0)
    la   t0, want
    sw   zero, 0(t0)
    la   t0, step_q8
    sw   zero, 0(t0)
    la   t0, score
    sw   zero, 0(t0)
    call place_food
    la   t0, state
    li   t1, PLAY
    sw   t1, 0(t0)
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # a new screen: draw everything
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.22. Passo 4 **(modificato)**: la fine di `new_game`, da `TITLE` a `PLAY`.

Attenzione

Ogni transizione della macchina chiede un disegno completo; ogni passo ne chiede uno parziale. Dimenticarne una è l'errore tipico di questa tecnica: lo schermo mostra uno stato che non è più vero.

Eseguite: Verm1 finito

Lo stesso gioco che c'era dopo il passo 3, pixel per pixel (Figure 22.1b e 22.1c). Se vedete scie o buchi, un cambiamento non è stato registrato: usate il debugger.

22.5 Passo 5: misurare

PRG32-QT è ora usato come uno strumento: contare prima di credere.

Prima e dopo, con i numeri

1. Eseguite il gioco finito in modalità debug e avviate una partita.
2. Leggete la telemetria durante dieci secondi di gioco: istruzioni completate, frame al secondo, frame in ritardo.
3. Confrontate con i numeri annotati per la parte 3 all'inizio del capitolo.
4. Lasciate crescere il verme fino a venti celle in entrambe le versioni e leggete di nuovo: un numero cresce con il verme, l'altro no.
5. Mettete un breakpoint sulla prima istruzione di `draw_changes`: si ferma una volta per passo, non una volta per frame.
6. Scrivete i numeri nei vostri appunti: vanno nella descrizione del gioco nel Capitolo 23.

I numeri deliberatamente non sono stampati qui: vanno misurati. Un'avvertenza nel leggerli: la telemetria conta le istruzioni della cartuccia, non quelle del runtime che riempie i rettangoli, così che il risparmio reale sulla scheda è maggiore di quanto il conteggio mostri; lì ogni pixel viaggia anche su SPI verso il display.

22.5.1 Seguire una chiamata con il debugger

La convenzione di chiamata del Capitolo 18 si può guardare istruzione per istruzione.

1. Fermatevi su `call to_decimal` dentro `draw_score`, e leggete `a0`, `a1`, `ra` e `sp`.
2. Fate un passo: il PC salta dentro `to_decimal`, e `ra` contiene ora l'indirizzo dell'istruzione dopo la chiamata.
3. Proseguite passo-passo: cambiano solo i registri `t` e `a0`, e `sp` non si muove mai. È una funzione foglia.
4. Alla `ret` il PC prende il valore di `ra`: il programma è tornato in `draw_score`.
5. Fate lo stesso con `call draw_text`: `sp` scende di 16, `ra` è memorizzato a `12(sp)`, e ritorna.

Nel listato del debugger `call` compare come `jal`, o come `auipc` seguita da `jalr`, e `ret` come `c.jr ra`.

22.6 Verificate di aver capito

D22.1 `to_decimal` riceve `a0` = 205. Quali byte memorizza, dallo scostamento 0 al 2?

A. `02 00 05`
B. `32 30 35`
C. `35 30 32`
D. `cd 00 00`

D22.2 A quale punteggio il verme fa un passo ogni 4 frame?

A. 4
B. 8
C. 16
D. 32

D22.3 `dirty` vale 0 quando viene chiamata `verm1_draw`. Quante istruzioni macchina esegue?

A. 1
B. 5
C. circa 50
D. migliaia

D22.4 Perché `to_decimal` può fare a meno di uno stack frame?

A. è corta
B. non chiama funzioni e usa solo registri `a` e `t`
C. non ha argomenti
D. i frame sono facoltativi

22.7 Fatelo vostro

Esercizio 22.1 (su PRG32)

1. Una pausa: il pulsante **B** ferma il gioco e scrive `PAUSE`; **B** di nuovo lo riprende. Un quarto stato.
2. Una melodia nella schermata del titolo: una tabella di note in `.rodata`, una nota ogni 15 frame.
3. Cibo bonus: un pezzo ogni cinque è giallo, vale tre punti, e scompare dopo 100 frame.
4. I vostri colori con `prg32_palette_set`, o le vostre regole: questo ora è il vostro gioco.

Risultato atteso. La versione pubblicata nel Capitolo 23 dovrebbe avere qualcosa di vostro. Cambiate anche la stringa del titolo. (Suggerimenti: la pausa è uno stato `PAUSED` con transizioni sul fronte di **B**, `0x20`, e `dirty` = 2 a entrambe le transizioni. Alla melodia servono un indice e un contatore di frame in `.data`; la tabella si legge come `dx`.)

22.8 Le funzioni che non sono cambiate

Otto parti del file sono quelle della parte 3 e restano come sono. Sono ristampate qui perché questo capitolo contenga l'intero programma.

```
verm1_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    sw   s1, 4(sp)
    call prg32_input_read
    mv   s0, a0                 # s0 = buttons held in this frame
    la   t0, prev_btn
    lw   t1, 0(t0)              # t1 = buttons held one frame ago
```

```
    sw   s0, 0(t0)
    not  t1, t1
    and  s1, s0, t1             # s1 = buttons pressed just now
```

Listato 22.23. Invariato: `verm1_update`, pulsanti e fronti.

```
new_game:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, grid
    li   t1, COLS*ROWS
1:  sb   zero, 0(t0)            # empty the grid, one byte per cell
    addi t0, t0, 1
    addi t1, t1, -1
    bnez t1, 1b
    li   t1, 0                  # i = 0, 1, 2: three cells in row 12
2:  la   t0, body_x
    add  t0, t0, t1
    addi t2, t1, 3
    sb   t2, 0(t0)              # columns 3, 4, 5
    la   t0, body_y
    add  t0, t0, t1
    li   t3, 12
    sb   t3, 0(t0)
    la   t0, grid+12*COLS
    add  t0, t0, t2
    li   t3, 1
    sb   t3, 0(t0)              # mark the cell as occupied
    addi t1, t1, 1
    li   t3, 3
    blt  t1, t3, 2b
```

Listato 22.24. Invariato: `new_game`, la mappa e il verme.

```
play_update:                    # a0 = buttons held
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, dir
    lw   t1, 0(t0)              # t1 = direction of the last step
    la   t0, want
    lw   t2, 0(t0)              # t2 = wanted direction, so far
    andi t3, a0, BTN_RIGHT
    beqz t3, 1f
    li   t2, DIR_RIGHT
1:  andi t3, a0, BTN_DOWN
    beqz t3, 2f
    li   t2, DIR_DOWN
2:  andi t3, a0, BTN_LEFT
    beqz t3, 3f
    li   t2, DIR_LEFT
3:  andi t3, a0, BTN_UP
    beqz t3, 4f
    li   t2, DIR_UP
4:  xor  t3, t2, t1             # opposite directions differ in bit 1 only
```

```
    li   t4, 2
    beq  t3, t4, 5f             # a U-turn: ignore the request
    sw   t2, 0(t0)
5:
```

Listato 22.25. Invariato: `play_update`, la guida.

```
step:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)              # s0 = column of the new head
    sw   s1, 4(sp)              # s1 = row of the new head
    sw   s2, 0(sp)              # s2 = 0 when the worm eats
    la   t0, want
    lw   t1, 0(t0)
    la   t0, dir
    sw   t1, 0(t0)              # the wanted direction is now the direction
    slli t1, t1, 2              # table offset = direction * 4 bytes
    la   t0, dx
    add  t0, t0, t1
    lw   t2, 0(t0)              # t2 = dx[direction]
    la   t0, dy
    add  t0, t0, t1
    lw   t3, 0(t0)              # t3 = dy[direction]
```

Listato 22.26. Invariato: `step`, la ricerca nella tabella.

```
place_food:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
1:  li   a0, 0
    li   a1, COLS-1
    call prg32_random_number
    mv   s0, a0                 # s0 = a column
    li   a0, 0
    li   a1, ROWS-1
    call prg32_random_number    # a0 = a row
    li   t1, COLS
    mul  t2, a0, t1
    add  t2, t2, s0
    la   t0, grid
    add  t0, t0, t2
    lbu  t3, 0(t0)
    bnez t3, 1b                 # on the worm: try again
    la   t0, food_x
    sw   s0, 0(t0)
    la   t0, food_y
    sw   a0, 0(t0)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.27. Invariato: `place_food`.

```
draw_worm:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   s0, 0                  # s0 = i: segment 0 is the head
1:  la   t0, length
    lw   t1, 0(t0)
    bge  s0, t1, 3f
    la   t0, head_i
    lw   t1, 0(t0)
    sub  t1, t1, s0
    andi t1, t1, 255            # ring index of segment i
    la   t0, body_x
    add  t0, t0, t1
    lbu  a0, 0(t0)
    la   t0, body_y
    add  t0, t0, t1
    lbu  a1, 0(t0)
    li   a2, BODY
    bnez s0, 2f
    li   a2, GREEN              # the head is brighter
2:  call draw_cell
    addi s0, s0, 1
    j    1b
3:  lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.28. Invariato: `draw_worm`, usata solo da `draw_full`.

```
draw_cell:                      # a0 = column, a1 = row, a2 = colour
    addi sp, sp, -16
    sw   ra, 12(sp)
    slli a0, a0, 3              # x = column * 8
    slli a1, a1, 3
    addi a1, a1, TOP            # y = row * 8 + 8
    mv   a4, a2                 # the colour is the fifth argument
    li   a2, CELL
    li   a3, CELL
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
```

Listato 22.29. Invariato: `draw_cell`.

```
.section .rodata                # constants: they stay in the cartridge file
title:     .asciz "VERM1"
press_a:   .asciz "PRESS A"
game_over: .asciz "GAME OVER"
dx:        .word  1, 0, -1,  0  # right, down, left, up
dy:        .word  0, 1,  0, -1
```

Listato 22.30. Invariato: `.rodata`.

22.9 Dove siamo, e che cosa viene dopo

Verm1 è finito: circa seicento righe di assembly RISC-V, tutte scritte da voi. È cominciato come tre funzioni che ritornano, ed è cresciuto un passo eseguibile alla volta. Dentro c'è ogni capitolo del libro: numeri, logica, stato, memoria, chiamate, pixel, dispositivi.

Un programma che solo il suo autore può eseguire è finito a metà. Il prossimo capitolo gli dà un nome, un'icona, i crediti, un bundle per due architetture, e il Cartridge Store.

Parte VI

Pubblicazione e prospettive

CAPITOLO 23

Verm1 sul Cartridge Store

"I veri artisti consegnano."
— Steve Jobs, alla squadra del Macintosh, 1983

In questo capitolo

Alla fine di questo capitolo avrete

- il vostro Verm1 compilato per due architetture, il kit ESP32-C6 e QEMU, dallo stesso sorgente;
- un manifesto che dà al gioco un'identità: id, titolo, versione, autori, licenza;
- un'icona, e un colophon con crediti e comandi;
- un bundle, un solo file ZIP, controllato e pronto per il Cartridge Store;
- un'idea chiara di ciò che succede dopo la pubblicazione, e di ciò di cui siete responsabili.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI impacchettare e pubblicare; la cartuccia portabile.
DA UTILIZZARE OGGI gli strumenti di PRG32 (si usa, non si studia, in questo corso).
DA COMPRENDERE PIÙ AVANTI niente.

Questo è l'ultimo capitolo del progetto, ed è di nuovo un tutorial: alla fine avrete un bundle, `verm1-1.0.0.zip`, controllato e pronto per essere sottoposto. Tutto fino al caricamento è stato eseguito sul gioco di riferimento dai test del materiale del corso; il caricamento stesso è un atto personale compiuto con un token personale, e per questo libro non è stato eseguito.

23.1 Perché pubblicare?

Un programma che gira solo sul computer del suo autore è un esercizio; uno che altri possono installare è un prodotto. Pubblicare impone le domande che un esercizio permette di saltare: come si chiama, chi lo ha fatto, su che cosa gira, che cosa possono farne gli altri. Il Cartridge Store (Figura 23.1) è lo scaffale di PRG32: giochi e programmi che si installano sul kit, su QEMU e su PRG32-QT. Ogni cartuccia che vi si trova è stata scritta da qualcuno

che è partito da dove siete partiti voi, e la vostra può stare accanto a esse. Lo Store è a https://store.prg32.uniparthenope.it, e PRG32-QT ha un proprio browser dello Store.

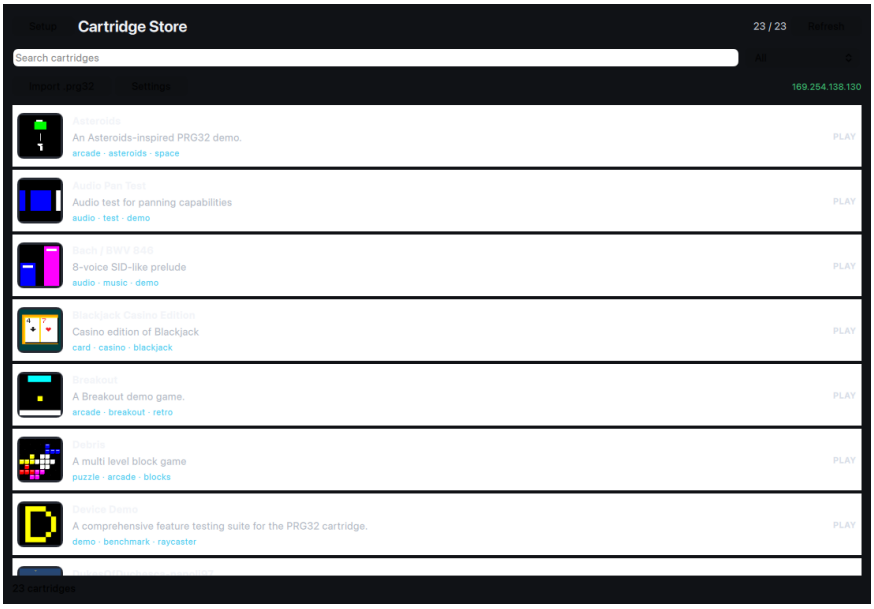


Figura 23.1. Il Cartridge Store, visto da PRG32-QT.

Il percorso dal sorgente allo scaffale ha sei stazioni (Figura 23.2). Le prime quattro avvengono sul computer dell'autore: niente lo lascia prima del comando che pubblica.

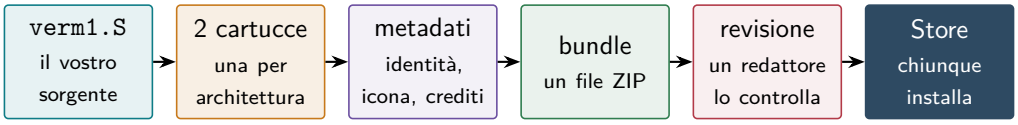


Figura 23.2. Dal sorgente allo scaffale.

23.1.1 Prima di impacchettare: è finito?

Il debugger viene usato un'ultima volta, come strumento di qualità (Tabella 23.1). Una cartuccia che non è mai stata eseguita su un bersaglio non dovrebbe essere pubblicata per quel bersaglio.

23.2 Passo 1: un sorgente, due cartucce

```
cd ~/PRG32          # your PRG32 folder, toolchain active

python3 -m prg32 cartridge build ~/verm1/verm1.S --portable \
    --entry-prefix verm1 --name verm1 \
    --architecture esp32c6 --out ~/verm1/verm1-esp32c6.prg32

python3 -m prg32 cartridge build ~/verm1/verm1.S --portable \
    --entry-prefix verm1 --name verm1 \
    --architecture qemu --out ~/verm1/verm1-qemu.prg32
```

Listato 23.1. Compilare per il kit e per QEMU.

Tabella 23.1. Il gioco è finito?

Controllo	Come	Con
Parte, si gioca e finisce	giocate tre partite intere, perdete in tre modi diversi	PRG32-QT
Nessun frame è in ritardo	telemetria durante una partita lunga: i frame in ritardo restano a zero	PRG32-QT, modalità debug
La memoria è coerente	mettete in pausa a caso e confrontate l'anello, la mappa e lo schermo	PRG32-QT, vista della memoria
Gira sulla seconda architettura	compilate per QEMU, caricate, giocate	QEMU
Gira sull'hardware reale	caricate su un kit, giocate con il joystick e l'altoparlante veri	il kit, se ne avete uno
È vostro	un titolo, una regola o un aspetto che il riferimento non ha	voi

Il Listato 23.1 è il comando di ogni capitolo con un'opzione in più: `--architecture` nomina il bersaglio del file. I nomi seguono una regola, `gioco-architettura.prg32`. Il compilatore di cartucce risponde con il nome, l'indirizzo di caricamento `0x40800000` e la dimensione del codice: Verm1 finito è di circa 6 kilobyte (6 216 byte per il gioco di riferimento).

```
python3 -m prg32 cartridge summary ~/verm1/verm1-esp32c6.prg32

{
  ...
  "import_model": "abi-table",
  ...
}
```

Listato 23.2. È portabile?

Il comando `summary` stampa ciò che il file dichiara di sé (Listato 23.2). Il valore `abi-table` di `import_model` significa che la cartuccia raggiunge il runtime attraverso una tabella di funzioni, la ABI di PRG32, di cui `prg32_gfx_rect` e le altre sono elementi. Una cartuccia di questo tipo non dipende da una particolare compilazione del firmware: è *portabile*, e lo Store accetta cartucce portabili. Una cartuccia non portabile è collegata agli indirizzi di una sola immagine del firmware, e si rompe al successivo aggiornamento. Ripetete il controllo sul file per QEMU.

23.3 Passo 2: un'identità

23.3.1 `manifest.json`: chi è questo gioco?

Create il file del Listato 23.3 in `~/verm1` con un editor di testo. (Nel listato la descrizione è tagliata; il file completo del gioco di riferimento è nell'Appendice D.)

```
{
  "abi": "prg32-metadata-1.0",
  "id": "it.uniparthenope.arc12.verm1",
  "title": "Verm1",
  "version": "1.0.0",
```

```
  "summary": "A hungry worm on a 40 by 24 grid: a tribute to Snake.",
  "description": "Verm1 (say \"vermone\") is the course-long project of ...",
  "authors": [{"name": "Your Name"}],
  "license": "MIT",
  "tags": ["arcade", "snake", "assembly", "education"],
  "assets": {"icon": "icon.png"},
  "architectures": [
    {"id": "esp32c6", "file": "verm1-esp32c6.prg32"},
    {"id": "qemu", "file": "verm1-qemu.prg32"}
  ]
}
```

Listato 23.3. `manifest.json`.

La riga `abi` dice quale versione del formato dei metadati è questa. Gli altri campi sono impegni (Tabella 23.2).

Tabella 23.2. I campi del manifesto, e a che cosa impegnano il loro autore.

Campo	Regola	Per Verm1
`id`	per consuetudine un nome di dominio rovesciato; non cambia mai	uno per autore: metteteci il vostro nome
`version`	major.minor.patch; una versione pubblicata non viene mai sostituita	1.0.0, poi 1.0.1 per una correzione
`title`	ciò che il giocatore legge	Verm1, o il nome della vostra variante
`authors`	le persone che lo hanno scritto	voi, e chiunque abbia lavorato con voi
`license`	che cosa possono farne gli altri	MIT, se volete che altri imparino da esso
`tags`	come verrà trovato	arcade, snake, assembly, education

Attenzione

Trattate un id e una versione, una volta pubblicati, come permanenti. Un errore si corregge con una nuova versione, non con una sostituzione. E due autori non possono pubblicare lo stesso id: ciascuno usa un id proprio, per esempio `it.uniparthenope.arc12.`*cognome*`.verm1`.

23.3.2 Un'icona

L'icona è un piccolo PNG che rappresenta il gioco sullo scaffale (Figura 23.3). L'icona di riferimento è di 64 per 64 pixel: un verme e un pezzo di cibo, nei colori del gioco. La pixel art si addice a una macchina di 320 per 200 pixel: disegnatela in grande, cella per cella, con un qualsiasi editor che salvi PNG, oppure disegnatela con un programma, come è stata fatta l'icona di riferimento. Salvatela come `icon.png` accanto al manifesto, che la nomina sotto `assets`. Un'icona presa dal web è il lavoro di qualcun altro: disegnate la vostra.

23.3.3 `colophon.json`: crediti e comandi

```
{
  "abi": "prg32-colophon-1.0",
  "title": "Verm1",
```

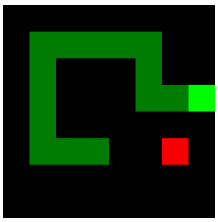


Figura 23.3. `icon.png`, ingrandita.

```
  "subtitle": "A tribute to Snake, in RISC-V assembly",
  "version": "1.0.0",
  "release_date": "2026-12-18",
  "developer": {"name": "Your Name"},
  "authors": [
    {"role": "game design", "name": "Your Name"},
    {"role": "programming", "name": "Your Name"}
  ],
  "license": "MIT",
  "copyright": "Copyright 2026 Your Name",
  "acknowledgements": [
    "Giuseppe Coviello, reference implementation of Verm1",
    "ARC12 Computer Architecture, University of Naples Parthenope",
    "The PRG32 community"
  ],
  "controls": [
    {"input": "JOYSTICK", "action": "Steer the worm"},
    {"input": "A", "action": "Start, and return to the title after game over"}
  ],
  "start_prompt": "Press A to play"
}
```

Listato 23.4. `colophon.json`.

Colophon è la parola dei tipografi per la nota alla fine di un libro che dice chi lo ha fatto. Qui è la pagina dei crediti della cartuccia (Listato 23.4): ruoli e nomi, dando credito a tutti; i comandi, cioè che cosa fa ciascun ingresso, per il giocatore; i ringraziamenti. Sostituite dappertutto `Your Name`, e impostate la data di rilascio. Il primo ringraziamento resta: un gioco che parte dall'implementazione di riferimento di Verm1, contributo di Giuseppe Coviello, lo dice.

23.3.4 Un omaggio non è una copia

Verm1 è un omaggio a Snake: l'idea di un verme che cresce è vecchia quanto *Blockade* (1976) ed è libera per tutti. Il nome, il codice, l'icona e i suoni sono originali: niente è stato preso da un altro gioco. Le regole di un gioco non sono protette; il codice, le immagini, la musica e i nomi lo sono. Prima di pubblicare, chiedetevi per ogni pezzo: l'ho scritto o disegnato io? Se no, la sua licenza mi permette di pubblicarlo, e ho dato credito? I nomi e i personaggi dei giochi commerciali sono marchi, e non vanno usati. La licenza scelta dice agli altri che cosa possono fare con l'opera: la MIT permette di studiarla, modificarla e condividerla, conservando il nome dell'autore.

La regola

Sopra c'è il vostro nome. Pubblicate solo ciò che firmereste.

23.4 Passo 3: la cartuccia porta con sé i suoi documenti

```
# metadata.json = manifest.json WITHOUT the "assets" and "architectures"
    entries

python3 -m prg32 store attach-metadata ~/verm1/verm1-esp32c6.prg32 \
    --metadata ~/verm1/metadata.json --icon ~/verm1/icon.png \
    --colophon ~/verm1/colophon.json --architecture esp32c6 \
    --out ~/verm1/verm1-esp32c6.prg32

python3 -m prg32 store attach-metadata ~/verm1/verm1-qemu.prg32 \
    --metadata ~/verm1/metadata.json --icon ~/verm1/icon.png \
    --colophon ~/verm1/colophon.json --architecture qemu \
    --out ~/verm1/verm1-qemu.prg32
```

Listato 23.5. `metadata.json`, e come allegarlo.

Il file `metadata.json` è una copia del manifesto senza le due voci `assets` e `architectures`. Il comando `attach-metadata` scrive titolo, icona e crediti *dentro* la cartuccia, una volta per file e con la sua architettura (Listato 23.5): una cartuccia copiata a mano dice ancora che cosa è. Lo strumento risponde, per il gioco di riferimento, `legacy=6316 blocks=3`: i tre blocchi sono i metadati, l'icona e il colophon, accodati dopo il programma, che di per sé non viene toccato.

Attenzione

Dare il manifesto completo a `--metadata` fa fallire lo strumento, a causa della voce `assets`.

```
python3 -m prg32 store inspect-metadata ~/verm1/verm1-esp32c6.prg32

{
  ...
    "id": "it.uniparthenope.arc12.verm1",
    "license": "MIT",
    "runtime": { "architecture": "esp32c6", ... },
    "summary": "A hungry worm on a 40 by 24 grid: a tribute to Snake.",
    "title": "Verm1",
    "version": "1.0.0"
  },
  ...
  "trailer_present": true,
  "unknown_blocks": []
}
```

Listato 23.6. Rileggere i metadati (uscita ridotta).

Concetto

Non fidatevi mai di una scrittura che non avete riletto. È l'abitudine di tutto il libro: la vista della memoria del debugger, e ora questo.

Controllate l'id, il titolo, la versione e l'architettura (Listato 23.6); `trailer_present` dice che i documenti sono nel file. Poi eseguite ancora una volta la cartuccia: deve funzionare ancora.

23.5 Passo 4: il bundle

```
python3 -m prg32 store pack-bundle \
    --manifest ~/verm1/manifest.json --out ~/verm1/verm1-1.0.0.zip

Bundle: ~/verm1/verm1-1.0.0.zip
  manifest.json                  0.7 KB
  icon.png                       0.1 KB
  verm1-esp32c6.prg32            7.8 KB
  verm1-qemu.prg32               7.8 KB
Total: 16.4 KB
```

Listato 23.7. Impacchettare il bundle (uscita reale per il gioco di riferimento).

Lo strumento legge il manifesto e raccoglie i file che esso nomina (Listato 23.7), che devono essere nella stessa cartella del manifesto. Un file mancante o con il nome sbagliato lo ferma qui, e non allo Store. Il nome del bundle porta la versione. L'intero gioco, per due macchine, è di 16 kilobyte.

Eseguite: provate il bundle, non la vostra cartella

1. Decomprimete `verm1-1.0.0.zip` in una cartella vuota.
2. In PRG32-QT importate `verm1-esp32c6.prg32` da quella cartella ed eseguitelo: titolo, icona e crediti devono essere i vostri.
3. Caricate `verm1-qemu.prg32` su QEMU e giocate una partita.
4. Se avete un kit, caricate il file per l'ESP32-C6 e giocate con il joystick vero.
5. Chiedete a un collega di fare lo stesso su un altro computer: il primo utente che non siete voi.

Risultato atteso. Il gioco gira dai file del bundle, su ogni bersaglio dichiarato, su un computer che non è il vostro. Provare i file impacchettati, e non quelli della cartella di lavoro, coglie l'errore classico: un bundle costruito prima dell'ultima correzione.

23.6 Passo 5: la pubblicazione

```
# 1. sign in to the Store and create a token on the Tokens page
# 2. keep it in the environment of your shell: never in a file, never in Git
export PRG32_STORE_URL="https://store.prg32.uniparthenope.it"
export PRG32_STORE_TOKEN="paste-your-token-here"

# 3. submit
```

```
python3 -m prg32 store publish-bundle ~/verm1/verm1-1.0.0.zip \
    --store-url "$PRG32_STORE_URL" --token "$PRG32_STORE_TOKEN"
```

Listato 23.8. Pubblicare: un token e un comando.

Per pubblicare, l'autore accede allo Store, crea un *token* nella pagina *Tokens*, lo tiene nell'ambiente della shell e sottopone il bundle (Listato 23.8).

Il token siete voi

Chi ha il token pubblica a vostro nome. È un segreto: non va nel sorgente, in un file, in un repository o in una schermata. Se viene perso o mostrato per errore, revocatelo e createne uno nuovo.

23.6.1 Che cosa succede dopo

In attesa.
Il bundle è memorizzato come proposta: non è ancora sullo scaffale.

Revisione.
Un redattore controlla che si installi, che giri, che sia descritto onestamente e che non contenga niente che non sia dell'autore.

Pubblicato.
Il gioco compare nello Store, per le architetture dichiarate.

Un errore?
Correggetelo, portate la versione a 1.0.1, impacchettate e sottoponete di nuovo: la 1.0.0 resta com'era.

La moderazione protegge i giocatori e gli autori. Un rifiuto arriva con una motivazione, e gli si risponde con una nuova versione.

23.6.2 La lista di controllo per la sottomissione

Una lista di controllo non è burocrazia: è il modo in cui piloti e chirurghi evitano gli errori che fanno gli esperti.

1. Il gioco è stato giocato fino alla fine su ogni architettura del manifesto.
2. `cartridge summary` dice `import_model: abi-table` per ogni file.
3. L'id è vostro e unico; la versione è nuova; titolo, sommario e descrizione dicono che cosa è il gioco.
4. Autori, licenza e colophon sono completi; ogni pezzo è vostro o accreditato.
5. `inspect-metadata` mostra l'id, la versione e l'architettura giusti in ogni cartuccia.
6. Il bundle è stato decompresso e provato su un altro computer.
7. Il token non è in alcun file, in alcuna schermata e in alcun repository.

23.7 Verificate di aver capito

D23.1 Avete pubblicato la versione 1.0.0 e trovato un errore. Che cosa fate?

A. caricate di nuovo la 1.0.0

B. pubblicate la 1.0.1
C. cambiate l'id
D. chiedete che il vecchio file venga modificato

D23.2 Perché `metadata.json` non contiene la voce `architectures`?

A. è facoltativa
B. i metadati vanno dentro una cartuccia, che ha una sola architettura
C. QEMU non la supporta
D. sarebbe troppo lunga

D23.3 Dove va tenuto il token dello Store?

A. in `verm1.S`, come commento
B. nel manifest
C. in una variabile d'ambiente della vostra shell
D. nel `README` del repository

23.8 Il vostro rilascio

Esercizio 23.1 (l'elaborato del progetto)

1. Scegliete il vostro id e il vostro titolo, e scrivete il manifesto e il colophon per la vostra variante di Verm1.
2. Disegnate la vostra icona.
3. Compilate, allegate, ispezionate, impacchettate: producete il vostro bundle, e superate la lista di controllo con un collega come revisore.
4. Se volete il vostro gioco sullo scaffale, pubblicatelo.

Risultato atteso. Un bundle del vostro gioco, rivisto da un collega rispetto alla lista di controllo. La pubblicazione è facoltativa e personale; il bundle, con il sorgente della vostra variante, è l'elaborato del progetto, e l'esame orale può partire da una qualsiasi delle sue righe.

23.9 Che cosa avete costruito

Un gioco di seicento righe di assembly, a partire da tre funzioni che ritornavano soltanto. Con esso, ogni strato della macchina: bit e numeri, porte e tabelle, stato e memoria, il processore, lo schermo, i dispositivi. Un metodo: un passo eseguibile alla volta, e uno sguardo dentro ogni volta che qualcosa sorprende. E l'ultimo passo, che ne fa ingegneria: un prodotto con un nome, una versione e un autore.

Il prossimo gioco sta a voi sceglierlo. Il prossimo capitolo mostra dove le stesse istruzioni girano in grande scala.

CAPITOLO 24

Oltre un solo core: RISC-V superscalare, HPC, GPU e FPGA

"Se doveste arare un campo, che cosa preferireste usare: due buoi robusti o 1024 polli?"
— attribuita a Seymour Cray

In questo capitolo

Alla fine di questo capitolo saprete

- scrivere l'equazione delle prestazioni e dire quale fattore attacca ciascuna tecnica;
- spiegare come un processore superscalare fuori ordine esegue più istruzioni per ciclo, e nominare core RISC-V che lo fanno;
- leggere un ciclo scritto con l'estensione vettoriale di RISC-V;
- descrivere un supercalcolatore, calcolare uno speed-up con la legge di Amdahl, e dire perché il limite è l'energia;
- dire in che cosa sono bravi una GPU e una FPGA, e che cosa costa ciascuna al programmatore.

Stato di apprendimento di questo capitolo

DA COMPRENDERE OGGI l'equazione delle prestazioni; i processori superscalari; le istruzioni vettoriali; HPC e legge di Amdahl; acceleratori GPU e FPGA.

DA UTILIZZARE OGGI il programma completo, gli strumenti di PRG32 (si usa, non si studia, in questo corso).

DA COMPRENDERE PIÙ AVANTI niente.

Questo capitolo conclusivo è uno sguardo in avanti. Parte da un ciclo di Verm1 e lo segue su quattro tipi di macchina: le stesse istruzioni, su macchine che ne eseguono molte insieme. Non fa parte della prova intercorso. I listati vettoriali RISC-V sono stati provati su QEMU; i frammenti CUDA e Verilog sono forme standard da libro di testo che per questo libro non sono state compilate. Le cifre sui supercalcolatori sono arrotondate e vengono dalle liste TOP500 dal 2022 al 2025.

24.1 Riscaldamento sulla macchina

```
    la   t0, grid
    li   t1, COLS*ROWS
1:  sb   zero, 0(t0)            # one byte per turn of the loop
    addi t0, t0, 1
    addi t1, t1, -1
    bnez t1, 1b
```

Listato 24.1. Il ciclo che svuota la mappa, dalla `new_game` di Verm1.

Il Listato 24.1 pone a zero 960 byte: quattro istruzioni per byte, 3 840 istruzioni in tutto, un byte alla volta e un'istruzione alla volta. Nel caso migliore un'istruzione per ciclo di clock e 160 milioni di cicli al secondo sul kit: circa 24 microsecondi. A nessuno serve che questo ciclo sia più veloce; ma è abbastanza piccolo da poter essere seguito su ogni macchina di questo capitolo. Quanti modi ci sono per renderlo più veloce?

24.1.1 L'equazione delle prestazioni

Teorema 24.1 (Equazione delle prestazioni)

Il tempo che un programma impiega è il prodotto di tre numeri,

$$T = N \times CPI \times t,$$

dove N è il numero di istruzioni eseguite, CPI il numero medio di cicli di clock per istruzione e t la durata di un ciclo di clock.

Per il ciclo precedente sul kit: $N = 3\,840$, circa 1 ciclo ciascuna, 6,25 nanosecondi per ciclo. Ogni fattore può essere attaccato: *meno istruzioni*, con algoritmi migliori, istruzioni più ricche, vettori; *meno cicli per istruzione*, con le pipeline e l'esecuzione superscalare; *un ciclo più breve*, con transistor più veloci e pipeline più profonde. E quando un processore non basta: più processori che lavorano insieme. Ogni macchina di questo capitolo è una risposta diversa alla stessa equazione (Patterson e Hennessy, 2020).

24.1.2 Perché non semplicemente un clock più veloce?

Per trent'anni ogni nuova generazione di transistor è stata più piccola, più veloce, e non più calda per millimetro quadrato. Intorno al 2005 questo è finito: transistor più piccoli arrivano ancora, ma un clock più veloce significa più calore di quanto un chip possa smaltire (Capitolo 10), e da allora le frequenze di clock sono rimaste a pochi gigahertz. La legge di Moore, più transistor, è andata avanti; la regola che teneva costante la densità di potenza, nota come scaling di Dennard, è finita. I transistor in più sono andati altrove: processori più larghi, più core, unità vettoriali, acceleratori. Le prestazioni vengono ora dal fare molte cose insieme: dal *parallelismo*. I 160 MHz del kit sono lenti per scelta: costano milliwatt.

24.2 Processori superscalari

Tabella 24.1. Da un'istruzione alla volta a molte insieme.

Processore	Come funziona	Istr. per ciclo	Esempio
Multiciclo	un'istruzione si completa prima che cominci la successiva	meno di 1	il modello del Capitolo 18
Con pipeline	più istruzioni in volo, ciascuna in uno stadio diverso	al più 1	l'ESP32-C6 del kit: quattro stadi
Superscalare, in ordine	due o più pipeline fianco a fianco	fino a 2 o più	piccoli core applicativi
Superscalare, fuori ordine	le istruzioni partono appena i loro operandi sono pronti	fino a 4, 6, 8, ...	portatili, telefoni, server

24.2.1 Da una alla volta a molte insieme

Il Capitolo 19 ha introdotto la pipeline e nominato i processori superscalari in una riga; questo paragrafo apre quella riga (Tabella 24.1). La *larghezza* di un processore è il numero di istruzioni che può avviare in un ciclo. Con due pipeline fianco a fianco e i cinque stadi classici, sei istruzioni si completano in sette cicli (Tabella 24.2); a una sola pipeline ne servono dieci, e a un processore senza pipeline trenta.

Tabella 24.2. Due pipeline fianco a fianco: sei istruzioni in sette cicli, se niente si mette di mezzo.

ciclo	1	2	3	4	5	6	7
i_1	IF	ID	EX	MEM	WB		
i_2	IF	ID	EX	MEM	WB		
i_3		IF	ID	EX	MEM	WB	
i_4		IF	ID	EX	MEM	WB	
i_5			IF	ID	EX	MEM	WB
i_6			IF	ID	EX	MEM	WB

24.2.2 Che cosa si mette di mezzo: le dipendenze

```
    add  t0, t1, t2        # i1
    add  t3, t0, t4        # i2 needs the t0 of i1: it must wait
    lw   t5, 0(t6)         # i3 is independent: it could go now
    beqz t3, skip          # i4: which way? known only after i2
    addi t5, t5, 1         # i5: executed, or not?
skip:
```

Listato 24.2. Dipendenze tra istruzioni vicine.

Questo è il caso ideale, ed è raro. Nel Listato 24.2 l'istruzione i_2 non può essere eseguita accanto a i_1, perché ne usa il risultato: una *dipendenza sui dati*. i_3 non dipende da nessuno: perché dovrebbe aspettare? E dopo il salto i_4 il processore non sa che cosa viene dopo: una *dipendenza di controllo*. Il codice reale è pieno di catene di questo tipo, e la sola larghezza non basta. Un processore superscalare che lavora in ordine si blocca a i_2, e tutto ciò che sta dietro aspetta. Il rimedio è lasciare che i_3 sorpassi.

24.2.3 L'esecuzione fuori ordine

1. **Prelievo e decodifica, in ordine.** Più istruzioni per ciclo entrano in una finestra di decine o centinaia di istruzioni.
2. **Ridenominazione.** Ogni risultato riceve un registro fisico nuovo, così che due usi del nome `t0` non collidono più.
3. **Esecuzione quando è pronta.** Un'istruzione parte appena i suoi operandi esistono, qualunque sia la sua posizione nel programma.
4. **Conferma, in ordine.** I risultati diventano ufficiali nell'ordine del programma: visto da fuori, niente ha mai sorpassato.

L'algoritmo è quello di Tomasulo, progettato nel 1967 per l'unità in virgola mobile dell'IBM System/360 Model 91; il reorder buffer che conferma in ordine è arrivato negli anni Ottanta. Il modello del programmatore, quello imparato in questo libro, è conservato esattamente: è questo il contratto dell'insieme di istruzioni.

24.2.4 La ridenominazione dei registri: trentadue nomi, molti registri

```
# as written                  # as executed, after renaming
    lw   t0, 0(a0)                lw   p41, 0(a0)
    addi t0, t0, 1                addi p42, p41, 1
    sw   t0, 0(a0)                sw   p42, 0(a0)
    lw   t0, 4(a0)                lw   p43, 4(a0)      # a new t0
    addi t0, t0, 1                addi p44, p43, 1
    sw   t0, 4(a0)                sw   p44, 4(a0)
```

Listato 24.3. Lo stesso codice come è scritto, e come è eseguito dopo la ridenominazione.

Nel Listato 24.3 la seconda metà riusa il *nome* `t0`, non il suo valore: una *falsa dipendenza*. Una volta ridenominate, le due metà non condividono niente, e possono essere eseguite nello stesso momento su registri diversi. (I numeri da `p41` a `p44` sono inventati.) I trentadue registri noti al programmatore sono nomi; il chip ne ha molti di più, alcune centinaia in un core grande.

24.2.5 Indovinare il futuro: la predizione dei salti

Un processore largo incontra un salto ogni poche istruzioni e non può aspettare di conoscerne l'esito. Ne *predice* la direzione dalla storia di quel salto, e continua a eseguire lungo l'ipotesi: un'ipotesi giusta non costa niente; una sbagliata butta via il lavoro speculativo e ricomincia. Il ciclo del riscaldamento salta indietro 959 volte e prosegue una volta: un predittore lo impara nei primi giri. I predittori moderni hanno ragione molto più di nove volte su dieci.

Speculazione e sicurezza

L'esecuzione speculativa ha un lato oscuro. Nel 2018 gli attacchi chiamati Spectre e Meltdown hanno mostrato che il lavoro buttato via può lasciare comunque nella cache tracce che rivelano segreti. I trucchi per le prestazioni sono anche questioni di sicurezza.

Tabella 24.3. Alcuni core RISC-V. L'elenco invecchia in fretta.

Core	Da	Tipo
core HP dell'ESP32-C6	Espressif	in ordine, quattro stadi, un'istruzione per ciclo: il kit
BOOM	University of California, Berkeley	fuori ordine, open source, per ricerca e didattica
XiangShan	Accademia Cinese delle Scienze	fuori ordine, open source, alte prestazioni
XuanTie C910	T-Head (Alibaba)	fuori ordine, largo tre; rilasciato come open source
Performance P550	SiFive	fuori ordine, largo tre, tredici stadi; venduto come progetto
Veyron	Ventana Micro Systems	fuori ordine, per i centri di calcolo

24.2.6 RISC-V diventa superscalare

Poiché l'insieme di istruzioni è aperto, chiunque può costruire un core di questo tipo (Tabella 24.3), e diversi sono pubblicati come codice sorgente che uno studente può leggere: non ha precedenti tra le architetture commerciali.

Definizione 24.1 (Architettura e microarchitettura)

L'*architettura* è il contratto: le istruzioni, i registri, che cosa fa ciascuna istruzione. La *microarchitettura* è un modo di onorarlo: stadi, larghezza, predittori, cache.

La `add` scritta per un microcontrollore a 160 MHz è la `add` di un core per server, e Verm1 girerebbe senza modifiche su uno qualsiasi dei core della tabella, dato uno schermo. È per questo che un insieme di istruzioni imparato una volta è una conoscenza che dura. La distinzione organizza l'intero libro: i Capitoli da 2 a 22 hanno insegnato l'architettura, e il kit è una piccola microarchitettura.

24.3 Vettori: un'istruzione, molti dati

24.3.1 La tassonomia di Flynn

Tabella 24.4. La tassonomia di Flynn (1972).

Classe	Istruzioni	Dati	Esempio
SISD	un flusso	un flusso	il kit: un core, un dato alla volta
SIMD	un flusso	molti insieme	istruzioni vettoriali; una GPU, in prima approssimazione
MISD	molti	un flusso	rara: sistemi ridondanti che votano
MIMD	molti	molti	processori multicore, cluster, supercalcolatori

La classificazione della Tabella 24.4 è grossolana, ed è ancora il primo vocabolario del calcolo parallelo.

24.3.2 Lo stesso ciclo, con i vettori

```
    la   t0, grid
    li   t1, COLS*ROWS
    vsetvli t2, t1, e8, m1, ta, ma  # elements are bytes
    vmv.v.i v0, 0                   # a register full of zeros
1:  vsetvli t2, t1, e8, m1, ta, ma  # t2 = bytes of this turn
    vse8.v  v0, (t0)                # store t2 bytes at once
    add  t0, t0, t2
    sub  t1, t1, t2
    bnez t1, 1b
```

Listato 24.4. Svuotare la mappa con l'estensione vettoriale.

L'estensione V di RISC-V, ratificata nel 2021, aggiunge i *registri vettoriali*, larghi molti byte (Listato 24.4). L'istruzione `vsetvli` chiede: quanti dei byte che restano stanno in un registro? La risposta torna in `t2`. Poi `vse8.v` memorizza `t2` byte con una sola istruzione. Con registri di 128 bit il ciclo tratta 16 byte per giro: 60 giri invece di 960.

Il codice non dice mai quanto sono larghi i registri: è *agnostico rispetto alla lunghezza del vettore*, e lo stesso binario usa 128, 256 o 1 024 bit. L'estensione non è sul kit, dato che l'ESP32-C6 è RV32IMAC; il listato è stato provato su QEMU con registri di 128, 256, 512 e 1 024 bit.

24.3.3 Una somma, corsia per corsia

```
    la   t0, numbers
    li   t1, 16                 # sixteen words to add
    vsetvli t2, zero, e32, m1, ta, ma
    vmv.v.i v1, 0               # v1 = partial sums, one per lane
1:  vsetvli t2, t1, e32, m1, tu, ma
    vle32.v v2, (t0)            # load t2 words
    vadd.vv v1, v1, v2          # add them, lane by lane
    slli t3, t2, 2
    add  t0, t0, t3
    sub  t1, t1, t2
    bnez t1, 1b
    vsetvli t2, zero, e32, m1, ta, ma
    vmv.s.x v3, zero
    vredsum.vs v3, v1, v3       # add the lanes together
    vmv.x.s a0, v3
```

Listato 24.5. La somma di sedici parole con i vettori.

Il Listato 24.5 è la somma parallela dei Capitoli 4 e 20, in RISC-V. `vle32.v` carica più parole insieme; `vadd.vv` somma registro a registro, *corsia* per corsia (*lane*); alla fine `vredsum.vs` somma tra loro le corsie. Con i sedici numeri da 1 a 16 il risultato è 136. (L'impostazione `tu`, *tail undisturbed*, conserva le somme parziali delle corsie che l'ultimo giro non usa.)

24.4 Calcolo ad alte prestazioni

24.4.1 Che cos'è un supercalcolatore?

Un supercalcolatore non è un solo enorme processore. Sono migliaia di calcolatori ordinari, i *nodi*, che lavorano allo stesso problema (Figura 24.1). Ogni nodo ha uno o due processori con decine di core, spesso più GPU, e moltissima memoria. Una rete molto veloce unisce i nodi, e un file system parallelo li serve tutti. La velocità si misura in operazioni in virgola mobile al secondo, FLOPS, su un problema standard: un enorme sistema di equazioni lineari (il benchmark chiamato HPL, *High-Performance Linpack*). La lista TOP500 classifica le macchine più veloci due volte l'anno dal 1993.

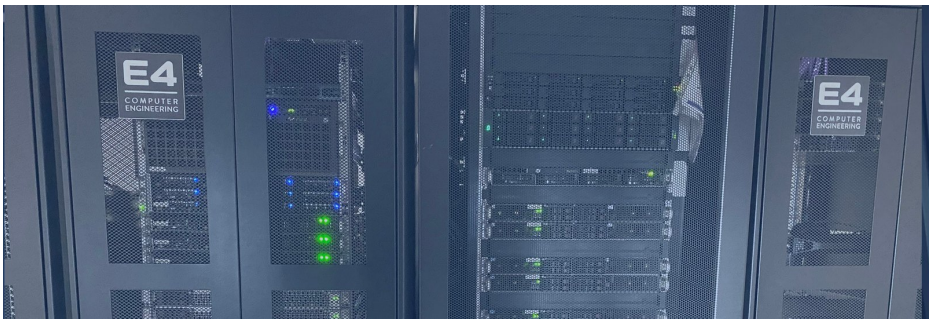


Figura 24.1. Armadi di nodi delle Research Computing Facilities dell'Università di Napoli «Parthenope»: la struttura di un supercalcolatore.

Tabella 24.5. Ordini di grandezza (cifre HPL arrotondate dalle liste TOP500; la cifra per il portatile è un ordine di grandezza).

Macchina	**Dove**	**Operazioni in virgola mobile al secondo**
ESP32-C6	il kit	nessuna unità in virgola mobile: virgola fissa, come in Verm1
Un portatile	la vostra borsa	circa 10^{11}
Leonardo (2022)	CINECA, Bologna	circa $2{,}4 \times 10^{17}$
Frontier (2022)	Oak Ridge, USA	oltre 10^{18}: la prima macchina exascale
El Capitan (2024)	Livermore, USA	circa $1{,}7 \times 10^{18}$
JUPITER (2025)	Jülich, Germania	circa 10^{18}: la prima macchina exascale europea

Exa è 10^{18}. Sette ordini di grandezza separano un portatile da una macchina exascale (Tabella 24.5): circa cinquant'anni di legge di Moore, comprati invece con il parallelismo.

24.4.2 La legge di Amdahl

Teorema 24.2 (Legge di Amdahl)

Se una frazione p del lavoro di un programma può essere svolta in parallelo, lo speed-up ottenuto con n processori è

$$S(n) = \frac{1}{(1-p) + \dfrac{p}{n}}, \qquad \text{e} \qquad \lim_{n\to\infty} S(n) = \frac{1}{1-p}.$$

Dimostrazione. Si prenda come 1 il tempo su un processore. La parte seriale impiega $1 - p$ qualunque sia il numero di processori; la parte parallela, divisa tra n, impiega p/n. Lo speed-up è il rapporto tra i due tempi. □

La frazione seriale non si riduce, per quanti processori si aggiungano (Amdahl, 1967). Con $p = 0{,}9$ lo speed-up non può mai superare 10, nemmeno con un milione di processori (Tabella 24.6): mille processori su un programma parallelo al 90% lo rendono meno di dieci volte più veloce. Prima di comprare processori, misurate p. La stessa legge vale per qualsiasi miglioramento che si applichi solo a una parte del lavoro: una GPU più veloce non aiuta la parte che gira sulla CPU.

Tabella 24.6. La legge di Amdahl in numeri: lo speed-up.

Frazione parallela p	$n = 10$	$n = 100$	$n = 1000$	n illimitato
0,50	1,8	2,0	2,0	2
0,90	5,3	9,2	9,9	10
0,99	9,2	50	91	100
0,999	9,9	91	500	1000

Come verifica, $p = 0{,}9$ e $n = 100$ danno $1/(0{,}1 + 0{,}009) = 9{,}17$. La risposta di Gustafson, nel 1988, fu che con più processori si risolvono problemi più grandi, e che in un problema più grande la parte parallela di solito cresce più in fretta di quella seriale.

24.4.3 Due modi di dividere il lavoro

Con la *memoria condivisa*, più core di un nodo vedono la stessa memoria, e il programma gira come *thread* che leggono e scrivono le stesse variabili. È facile cominciare: OpenMP aggiunge una riga sopra un ciclo. Il pericolo sono due thread che scrivono la stessa variabile. Con la *memoria distribuita* ogni nodo ha la propria memoria e niente è condiviso; il programma gira come *processi* che si scambiano messaggi, e MPI, la *Message Passing Interface*, è lo standard. Scala fino a un milione di core, e il programmatore ne paga il prezzo. Le applicazioni reali usano entrambi: MPI tra i nodi, thread e GPU dentro ogni nodo. I modelli meteorologici e marini del Dipartimento funzionano in questo modo.

24.4.4 Il limite è l'energia

Una macchina exascale assorbe circa venti megawatt, il consumo di una piccola città (Frontier ha raggiunto il suo exaflop a circa 21 MW): circa 50 miliardi di operazioni per joule. L'elettricità della sua vita costa quanto la macchina. Quindi la cifra che conta sono le operazioni per watt, che ha la sua classifica, la Green500, guidata dalle GPU e da altri acceleratori: l'hardware specializzato fa di più con ogni joule. La stessa ragione mette nel kit un piccolo core RISC-V: milliwatt. L'efficienza energetica è il filo che lega un microcontrollore a un supercalcolatore.

24.4.5 RISC-V e il supercalcolo europeo

Le macchine più grandi di oggi usano processori x86 o Arm con acceleratori GPU. L'Europa progetta i propri processori e acceleratori su RISC-V: la European Processor Initiative, e DARE, il progetto nominato nel Capitolo 0 (EuroHPC Joint Undertaking, 2025). Monte Cimone, costruito nel 2022 all'Università di Bologna con CINECA, è un cluster HPC completo su processori RISC-V; il Barcelona Supercomputing Center progetta e fabbrica core

RISC-V propri. Un insieme di istruzioni aperto permette a un continente di costruire ciò che altrimenti dovrebbe comprare.

24.5 Acceleratori GPU

24.5.1 Due modi di spendere i transistor

Una CPU ha pochi core grandi, alcune decine, ciascuno fuori ordine, con predittori e grandi cache: è costruita per finire *un* compito il prima possibile, ed è a suo agio con i salti e con i dati irregolari. Una GPU ha migliaia di core piccoli e semplici, che eseguono tutti lo stesso codice su dati diversi: è costruita per finire un milione di compiti al secondo, ed è a suo agio con gli array regolari: pixel, matrici, griglie. Sono i buoi e i polli di Cray, e la risposta dipende dal campo. La GPU non è più veloce in nessuna cosa singola; fa più cose insieme.

24.5.2 Come un programma usa una GPU

1. **Preparare.** Il programma sulla CPU, l'*host*, costruisce i suoi array in memoria centrale.
2. **Copiare dentro.** I dati viaggiano verso la memoria della GPU: un costo reale, in tempo.
3. **Lanciare.** Una piccola funzione, il *kernel*, gira in migliaia di thread, uno per elemento dei dati.
4. **Copiare fuori.** I risultati tornano indietro, e l'host prosegue.

La GPU è un dispositivo, come quelli del Capitolo 21: la CPU le invia dati e comandi attraverso un driver. I modelli di programmazione sono CUDA, OpenCL e SYCL, e direttive come OpenACC e l'offload di OpenMP.

```
// the kernel: this function runs once in every thread
__global__ void clear(unsigned char *grid, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;   // who am I?
    if (i < n)
        grid[i] = 0;                                 // my cell, and only mine
}

// on the host: 4 blocks of 256 threads cover the 960 cells
clear<<<(960 + 255) / 256, 256>>>(grid, 960);
```

Listato 24.6. La mappa di Verm1 con un thread per cella (CUDA C, nella sua forma usuale da libro di testo; non compilato per questo libro).

Nel Listato 24.6 non c'è alcun ciclo: ci sono 960 thread. Ogni thread calcola il proprio indice e fa uno store. I thread arrivano a blocchi, e gli ultimi non trovano niente da fare. Per 960 byte le due copie costano molto più del lavoro, e il punto è serio: una GPU conviene quando i dati sono grandi e il lavoro per elemento è reale.

24.5.3 Quando conviene una GPU?

Conviene quando la stessa operazione si applica a moltissimi dati, in modo indipendente: immagini, matrici, le griglie di una simulazione, reti neurali; e quando i dati possono restare sulla GPU per molti passi invece di viaggiare a ognuno. Non conviene quando il lavoro è

una catena di decisioni, ciascuna dipendente dalla precedente. E vale di nuovo la legge di Amdahl: ciò che resta sulla CPU fissa il limite. Una simulazione meteorologica aggiorna ogni cella di una griglia tridimensionale a partire dalle sue vicine, milioni di volte: la stessa forma della mappa di Verm1, un po' più grande. L'addestramento delle reti neurali di oggi è, quasi interamente, aritmetica matriciale su GPU.

24.6 Acceleratori FPGA

24.6.1 Un chip che diventa il circuito che descrivete

Definizione 24.2 (FPGA)

Una *FPGA* (*field-programmable gate array*) è un chip fatto di un mare di piccoli blocchi programmabili: *tabelle di look-up*, ciascuna capace di essere una qualsiasi funzione di pochi ingressi; flip-flop, uno accanto a ogni tabella; e fili e interruttori che possono unire qualsiasi blocco a qualsiasi altro. Un file di configurazione, il *bitstream*, imposta ogni tabella e ogni interruttore.

Caricate un altro bitstream, e lo stesso chip è un altro circuito. La prima FPGA, la Xilinx XC2064, fu venduta nel 1985; oggi vanno da pochi euro a schede per centri di calcolo. Una FPGA non esegue un programma: *diventa* un circuito. Niente viene prelevato o decodificato, e tutte le porte lavorano nello stesso momento.

24.6.2 Una tabella di look-up è una tabella di verità in memoria

a	b	memorizzato per AND	memorizzato per XOR	memorizzato per NOT a
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	0	0

Gli ingressi sono l'indirizzo, e quattro bit memorizzati scelgono la funzione: cambiare i bit cambia la porta. Il Capitolo 8 diceva che una tabella di verità definisce una funzione, e il Capitolo 16 che una ROM indirizzata dagli ingressi realizza qualsiasi funzione. Una tabella di look-up è quella ROM, con quattro-sei ingressi e da sedici a sessantaquattro bit; il flip-flop accanto a essa è quello del Capitolo 13. Una FPGA è questo libro, ripetuto centomila volte su un chip.

24.6.3 La regola dell'inversione a U di Verm1, come hardware

```
// Lecture 12: opposite directions differ in bit 1 only
module uturn (
    input  wire [1:0] dir,      // direction of the last step
    input  wire [1:0] want,     // direction asked by the player
    output wire       refuse    // 1 = this would be a U-turn
);
    assign refuse = (dir ^ want) == 2'b10;
endmodule
```

Listato 24.7. La regola dell'inversione a U in Verilog (non sintetizzata per questo libro).

Verilog, come VHDL, è un *linguaggio di descrizione dell'hardware*: descrive circuiti, non passi (Listato 24.7). L'istruzione `assign` è un filo, sempre vera: non c'è un prima e non c'è un dopo. Il circuito è fatto di due porte XOR e un AND con un ingresso invertito,

$$refuse = (d_1 \oplus w_1) \cdot \overline{(d_0 \oplus w_0)}.$$

Sul kit questa regola richiedeva tre istruzioni e tre cicli (Capitolo 12); qui le porte rispondono in una frazione di nanosecondo, in continuazione.

24.6.4 Dalla descrizione al circuito



Figura 24.2. Da una descrizione a un circuito.

Ogni tentativo richiede da minuti a ore (Figura 24.2): più lento che compilare una cartuccia, molto più veloce che fabbricare un chip. Lo stesso flusso, con una fabbrica alla fine invece di un bitstream, produce un *ASIC*, un chip fatto per un solo scopo. Le FPGA sono il luogo in cui i chip si provano prima di fabbricarli.

24.6.5 Anche un processore è un circuito

Un core RISC-V è una descrizione in un linguaggio hardware, e può essere caricato su una FPGA. Questi *soft core* sono liberi da leggere e da usare: PicoRV32, VexRiscv, NEORV32, Rocket. Un core della classe di quello del kit sta in una FPGA piccola ed economica, e accanto a esso si può mettere un circuito proprio per l'unica operazione che un programma ripete di più: un acceleratore su misura. RISC-V riserva spazio nella sua codifica per istruzioni personalizzate, con cui raggiungere tali circuiti.

Il cerchio si chiude

Le porte del Capitolo 8 costruiscono il sommatore del Capitolo 9, i registri del Capitolo 14 e il processore del Capitolo 18; e quel processore è un file che si può leggere. Dalle porte a un processore, e ritorno: ora conoscete ogni strato intermedio.

24.6.6 Quattro modi di calcolare

La generalità si paga in energia; l'efficienza si paga in sforzo (Tabella 24.7). Un moderno sistema su un chip li ha tutti e quattro: un telefono contiene core CPU, una GPU, e circuiti fissi per il video e la radio.

24.7 Ritorno alla macchina

Le prime tre righe della Tabella 24.8 sono lo stesso insieme di istruzioni; la prima e la terza sono state eseguite per questo libro. Il lavoro è lo stesso dappertutto; ciò che cambia è quanto di esso avviene nello stesso momento. E cambiano i costi: alla prima riga servono un chip da due euro e quattro righe di assembly; all'ultima serve un progetto hardware.

Tabella 24.7. Quattro modi di calcolare.

	CPU	**GPU**	**FPGA**	**ASIC**
Che cos'è	pochi core complessi	migliaia di core semplici	un circuito che si configura	un circuito fatto per un solo lavoro
Si programma in	qualsiasi linguaggio	kernel: CUDA, OpenCL, SYCL	Verilog, VHDL, HLS	gli stessi, poi una fabbrica
Forte in	tutto, moderatamente	dati regolari in massa	flussi, bassa latenza	una cosa, alla perfezione
Lavoro per joule	basso	alto	alto	il più alto
Sforzo d'uso	basso	medio	alto	molto alto
Cambiare idea	ricompilare	ricompilare	un nuovo bitstream	un nuovo chip

Tabella 24.8. Svuotare la mappa di Verm1, 960 celle, su cinque macchine.

Macchina	**Come**	**Passi nel tempo**
Il kit: ESP32-C6	un byte per giro, un'istruzione per ciclo	3 840 cicli
Core RISC-V fuori ordine	lo stesso binario; più istruzioni per ciclo, il salto predetto	una frazione di quei cicli
RISC-V con vettori, 128 bit	16 byte per giro	60 giri
GPU	960 thread, uno per cella	un lancio, più due copie
FPGA	960 flip-flop con un reset comune	un ciclo di clock

Concetto

La domanda dell'ingegnere non è mai quale macchina sia la più veloce, ma quale basti.

24.8 Il programma di questo capitolo

I listati di questo capitolo sono parti di un solo programma, `code/l24.S` nel repository del corso, stampato per intero nel Listato 24.8. I test automatici del materiale del corso lo compilano e lo eseguono sull'esecutore senza interfaccia di PRG32-QT, con ingressi programmati, e controllano il frame; la Figura 24.3 è il frame scritto dal test.

Sullo schermo.
Un quadrato verde: i 960 byte della mappa, riempiti di uni da init, sono stati tutti posti a zero dal ciclo di quattro istruzioni della lezione. Un quadrato rosso vorrebbe dire che qualche byte è sopravvissuto.

Che cosa premere.
Niente: il programma va da solo.

```
# ARC12 Computer Architecture - Lecture 24: Beyond One Core
# l24.S: the loop that empties the map of Verm1, one byte at a time
#
```

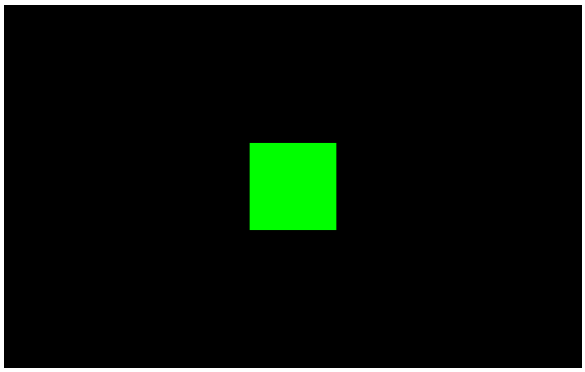


Figura 24.3. `124.S` su PRG32-QT.

```
# On the screen: a green square: the 960 bytes of the map, filled with ones by init,
# were all set to zero by the four-instruction loop of the lecture. A red square would
# mean that some byte survived.
# Build: python3 -m prg32 cartridge build l24.S --portable \
#            --entry-prefix l24 --name l24 --out l24.prg32
.option norelax

.equ BLACK, 0x0000
.equ GREEN, 0x07e0
.equ RED,   0xf800
.equ CELLS, 960              # 40 columns by 24 rows

.section .text
.global l24_init
.global l24_update
.global l24_draw

l24_init:                    # fill the map with ones, so that there is work to do
    la   t0, grid
    li   t1, CELLS
    li   t2, 1
1:  sb   t2, 0(t0)
    addi t0, t0, 1
    addi t1, t1, -1
    bnez t1, 1b
    ret

l24_update:
    la   t0, grid            # the loop of the lecture
    li   t1, CELLS
1:  sb   zero, 0(t0)         # one byte ...
    addi t0, t0, 1           # ... the next address ...
    addi t1, t1, -1          # ... one fewer to go ...
    bnez t1, 1b              # ... 960 times: 3840 instructions

    la   t0, grid            # count the bytes that are not zero
    li   t1, CELLS
    li   t3, 0
```

```
2:  lbu  t2, 0(t0)
    add  t3, t3, t2
    addi t0, t0, 1
    addi t1, t1, -1
    bnez t1, 2b
    la   t0, left
    sw   t3, 0(t0)
    ret

l24_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    la   t0, left
    lw   t1, 0(t0)
    li   a4, GREEN
    beqz t1, 1f
    li   a4, RED
1:  li   a0, 136
    li   a1, 76
    li   a2, 48
    li   a3, 48
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

.section .data
left: .word 1
grid: .space CELLS
```

Listato 24.8. Il programma completo del capitolo, `code/l24.S`.

Per eseguirlo su PRG32-QT i comandi sono quelli del Listato 24.9; il Paragrafo B.8 dà gli stessi passi per QEMU e per la scheda.

```
# 1. start PRG32-QT on this computer (it listens on port 8080)

# 2. in the PRG32 folder, with ESP-IDF active: build the cartridge
python3 -m prg32 cartridge build ~/arc12/code/l24.S --portable \
    --entry-prefix l24 --name l24 --out l24.prg32

# 3. send it to PRG32-QT and start it
python3 -m prg32 esp32c6 upload-and-run l24.prg32 \
    --url http://localhost:8080 --slot cart0

# 4. look, press, compare with the previous slide
```

Listato 24.9. Compilare `l24.S` ed eseguirlo su PRG32-QT.

24.9 Verificate di aver capito

D24.1 Un programma è parallelo all'80%. Qual è il massimo speed-up che un numero qualunque di processori può dare?

A. 4
B. 5
C. 8
D. 80

D24.2 Che cosa elimina la ridenominazione dei registri?

A. i salti
B. il bisogno di uno stack
C. le false dipendenze tra usi dello stesso nome di registro
D. i cache miss

D24.3 Con registri vettoriali a 256 bit, quanti giri servono al ciclo vettoriale per i 960 byte?

A. 4
B. 30
C. 60
D. 960

D24.4 Una tabella di look-up a due ingressi memorizza, per gli ingressi 00, 01, 10, 11, i bit 0, 1, 1, 1. Quale porta è?

A. AND
B. OR
C. XOR
D. NAND

24.10 Esercizi

Esercizio 24.1 (prima su carta, poi sulla macchina)

1. Compilate la tabella di Amdahl per $p = 0{,}95$ e $n = 10, 100, 1000$.
2. Disegnate il diagramma della pipeline di un processore largo tre per nove istruzioni indipendenti con cinque stadi.
3. Ridenominate i registri del codice di `step` che sposta e fa rientrare la testa in Verm1, parte 2: quali istruzioni potrebbero essere eseguite nello stesso ciclo?
4. Scrivete la tabella di verità memorizzata in una tabella di look-up a quattro ingressi per $\mathit{refuse} = (\mathit{dir} \oplus \mathit{want}) = 2$.
5. Compilate il test `vector_qemu.S` del materiale del corso ed eseguitelo su QEMU con registri vettoriali di 128 e di 512 bit: perché lo stesso binario funziona?

Risultato atteso. Amdahl con $p = 0{,}95$: 6,9, 16,8, 19,6. Nove istruzioni su un processore largo tre: sette cicli. La tabella di look-up ha quattro uni su sedici elementi, per (dir,want) = (0,2), (1,3), (2,0), (3,1). QEMU si avvia con `qemu-system-riscv32 -machine virt -cpu rv32,v=true,vlen=128`.

24.11 Glossario

Inglese	Italiano
superscalar	superscalare
register renaming	ridenominazione dei registri
out-of-order execution	esecuzione fuori ordine
branch prediction	predizione dei salti
speed-up	accelerazione (speed-up)
node	nodo
shared memory	memoria condivisa
message passing	scambio di messaggi
accelerator	acceleratore
kernel (GPU)	kernel
look-up table	tabella di look-up
bitstream	bitstream
hardware description language	linguaggio di descrizione dell'hardware
soft core	soft core

24.12 Conclusioni e letture

Un'equazione, tre fattori: ogni macchina veloce ne attacca uno, o moltiplica i processori. I core superscalari fuori ordine rispettano il contratto dell'insieme di istruzioni ed eseguono molte istruzioni insieme. Vettori, GPU e cluster sfruttano i dati che possono essere elaborati in modo indipendente, e la legge di Amdahl dice fin dove si arriva. Le FPGA trasformano una descrizione in un circuito: la logica di questo libro, in grande scala.

Il libro finisce qui: da un bit a un supercalcolatore, con un solo insieme di istruzioni.

Letture. Patterson e Hennessy (2020), capitoli 4 e 6; Hennessy e Patterson (2019); https://top500.org; la specifica vettoriale di RISC-V su https://riscv.org.

Parte VII

Appendici

APPENDICE A

Riferimento di RISC-V

Questa appendice è un riferimento completo dell'architettura RISC-V così come è usata in questo libro: l'insieme di istruzioni RV32IMAC dell'ESP32-C6, i registri di controllo e stato del modo macchina, le convenzioni dell'assemblatore e la convenzione di chiamata. Segue le specifiche ratificate (RISC-V International, 2024c,d,a,b), che prevalgono in caso di dubbio. Dappertutto, $x[r]$ indica il contenuto del registro r, $M[a]$ la memoria all'indirizzo a, $\mathrm{sext}(\cdot)$ l'estensione in segno a 32 bit e $\mathrm{zext}(\cdot)$ l'estensione con zeri; l'aritmetica è modulo 2^{32}.

A.1 Il nome di un insieme di istruzioni

Un insieme di istruzioni RISC-V prende il nome da una base e da un elenco di estensioni. La base dà la larghezza dei registri; ogni lettera aggiunge un gruppo di istruzioni (Tabella A.1). RV32IMAC è quindi la base a 32 bit con moltiplicazione e divisione, istruzioni atomiche e istruzioni compresse. Dal 2019 le istruzioni per i registri di controllo e stato e l'istruzione `fence.i` sono formalmente estensioni separate, Zicsr e Zifencei, che anche l'ESP32-C6 realizza.

A.2 I registri

L'hardware dà un significato speciale solo a `x0`, che si legge sempre come zero e ignora le scritture. Tutti gli altri ruoli della Tabella A.2 sono convenzioni. Le istruzioni compresse con un campo registro di 3 bit possono nominare solo `x8`–`x15`, cioè `s0`, `s1` e `a0`–`a5`. Il program counter non è un registro generico: è letto da `auipc`, `jal` e dai salti condizionati, e scritto dai salti.

A.3 I formati delle istruzioni

Tutte le istruzioni dell'insieme di base sono lunghe 32 bit e sono memorizzate in ordine little-endian; i due bit meno significativi di un'istruzione a 32 bit sono `11`. I sei formati sono mostrati nella Figura A.1.

Immediati. In ogni formato il bit 31 dell'istruzione è il bit di segno dell'immediato, che è sempre esteso in segno a 32 bit.

- **I**: 12 bit, intervallo $-2048 \ldots 2047$.

Tabella A.1. Basi ed estensioni di RISC-V.

Nome	Significato	Che cosa aggiunge	ESP32-C6
RV32I	base intera, 32 bit	32 registri di 32 bit; 40 istruzioni	sì
RV32E	base embedded	come RV32I con 16 registri	–
RV64I	base intera, 64 bit	registri e indirizzi di 64 bit	–
M	moltiplicazione, divisione	`mul`, `mulh`, `mulhsu`, `mulhu`, `div`, `divu`, `rem`, `remu`	sì
A	atomiche	`lr.w`, `sc.w` e le operazioni atomiche in memoria	sì
F, D	virgola mobile	registri `f0`–`f31`; IEEE 754 a precisione singola e doppia	no
C	compresse	codifiche a 16 bit delle istruzioni più comuni	sì
Zicsr	accesso ai CSR	`csrrw`, `csrrs`, `csrrc` e le forme immediate	sì
Zifencei	barriera sulle istruzioni	`fence.i`	sì
B (Zba, Zbb, Zbs)	manipolazione dei bit	scorrimenti con somma, conteggi, rotazioni, operazioni su un singolo bit	no
V	vettori	registri `v0`–`v31` di lunghezza variabile (Capitolo 24)	no

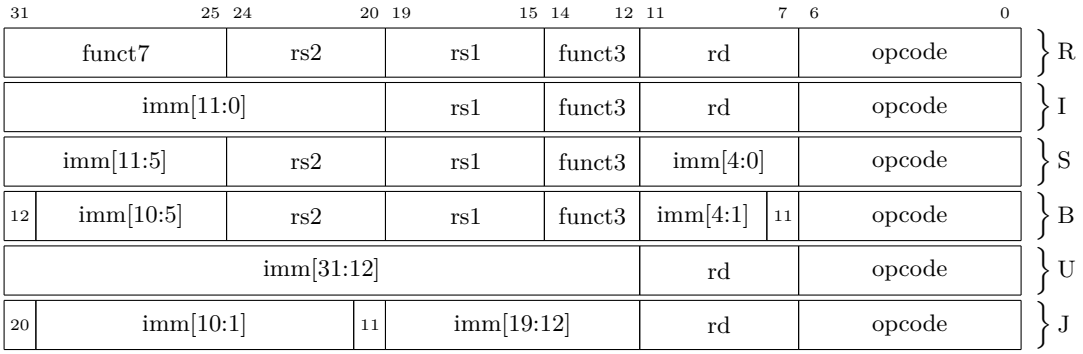


Figura A.1. I sei formati delle istruzioni a 32 bit.

- **S**: 12 bit, divisi tra i bit 31–25 e 11–7, così che `rs1` e `rs2` restano al loro posto.
- **B**: 13 bit, di cui il bit 0 è sempre zero e non è memorizzato; lo scostamento è un multiplo di 2 nell'intervallo $-4096 \ldots 4094$, relativo all'indirizzo del salto.
- **U**: i 20 bit superiori; i 12 inferiori sono zero.
- **J**: 21 bit, di cui il bit 0 è zero e non memorizzato; lo scostamento è un multiplo di 2 nell'intervallo ± 1 MiB.

A.4 RV32I: l'insieme di istruzioni intero di base

La Tabella A.4 elenca le quaranta istruzioni di RV32I. I confronti con segno leggono i registri in complemento a due; «u» segna le forme senza segno.

Tabella A.2. I registri interi e i loro ruoli nella convenzione di chiamata standard; l'ultima colonna dice chi preserva il registro attraverso una chiamata.

Registro	Nome ABI	Ruolo	Preservato da
`x0`	`zero`	zero cablato	–
`x1`	`ra`	indirizzo di ritorno	chiamante
`x2`	`sp`	puntatore allo stack	chiamato
`x3`	`gp`	puntatore globale	– (da non modificare)
`x4`	`tp`	puntatore al thread	– (da non modificare)
`x5`–`x7`	`t0`–`t2`	temporanei	chiamante
`x8`	`s0` / `fp`	registro salvato / puntatore al frame	chiamato
`x9`	`s1`	registro salvato	chiamato
`x10`–`x11`	`a0`–`a1`	argomenti / valori di ritorno	chiamante
`x12`–`x17`	`a2`–`a7`	argomenti	chiamante
`x18`–`x27`	`s2`–`s11`	registri salvati	chiamato
`x28`–`x31`	`t3`–`t6`	temporanei	chiamante
`pc`	–	program counter	–

Tabella A.4. Le istruzioni di RV32I.

Assembly	Fmt	opcode / f3 / f7	Effetto
Costanti e indirizzi			
`lui rd, imm`	U	`0110111`	$x[rd] = imm \ll 12$
`auipc rd, imm`	U	`0010111`	$x[rd] = pc + (imm \ll 12)$
Aritmetica e logica con un immediato			
`addi rd, rs1, imm`	I	`0010011 / 000`	$x[rd] = x[rs1] + \text{sext}(imm)$
`slti rd, rs1, imm`	I	`0010011 / 010`	$x[rd] = 1$ se $x[rs1] < \text{sext}(imm)$ (con segno), altrimenti 0
`sltiu rd, rs1, imm`	I	`0010011 / 011`	lo stesso, confronto senza segno
`xori rd, rs1, imm`	I	`0010011 / 100`	$x[rd] = x[rs1] \oplus \text{sext}(imm)$
`ori rd, rs1, imm`	I	`0010011 / 110`	$x[rd] = x[rs1] \mid \text{sext}(imm)$
`andi rd, rs1, imm`	I	`0010011 / 111`	$x[rd] = x[rs1]$ & $\text{sext}(imm)$
`slli rd, rs1, sh`	I	`0010011 / 001 / 0000000`	$x[rd] = x[rs1] \ll sh$, $0 \le sh \le 31$
`srli rd, rs1, sh`	I	`0010011 / 101 / 0000000`	scorrimento logico a destra: entrano zeri
`srai rd, rs1, sh`	I	`0010011 / 101 / 0100000`	scorrimento aritmetico a destra: entra il bit di segno
Aritmetica e logica tra registri			
`add rd, rs1, rs2`	R	`0110011 / 000 / 0000000`	$x[rd] = x[rs1] + x[rs2]$
`sub rd, rs1, rs2`	R	`0110011 / 000 / 0100000`	$x[rd] = x[rs1] - x[rs2]$
`sll rd, rs1, rs2`	R	`0110011 / 001 / 0000000`	scorrimento a sinistra di $x[rs2]$ mod 32
`slt rd, rs1, rs2`	R	`0110011 / 010 / 0000000`	$x[rd] = 1$ se $x[rs1] < x[rs2]$ (con segno), altrimenti 0
`sltu rd, rs1, rs2`	R	`0110011 / 011 / 0000000`	lo stesso, senza segno
`xor rd, rs1, rs2`	R	`0110011 / 100 / 0000000`	or esclusivo bit a bit

Assembly	**Fmt**	**opcode / f3 / f7**	**Effetto**
`srl rd, rs1, rs2`	R	`0110011 / 101 / 0000000`	scorrimento logico a destra di $x[rs2] \bmod 32$
`sra rd, rs1, rs2`	R	`0110011 / 101 / 0100000`	scorrimento aritmetico a destra di $x[rs2] \bmod 32$
`or rd, rs1, rs2`	R	`0110011 / 110 / 0000000`	or bit a bit
`and rd, rs1, rs2`	R	`0110011 / 111 / 0000000`	and bit a bit
Load e store			
`lb rd, imm(rs1)`	I	`0000011 / 000`	$x[rd] = \text{sext}(M_8[x[rs1] + \text{sext}(imm)])$
`lh rd, imm(rs1)`	I	`0000011 / 001`	lo stesso, 16 bit, esteso in segno
`lw rd, imm(rs1)`	I	`0000011 / 010`	lo stesso, 32 bit
`lbu rd, imm(rs1)`	I	`0000011 / 100`	8 bit, esteso con zeri
`lhu rd, imm(rs1)`	I	`0000011 / 101`	16 bit, esteso con zeri
`sb rs2, imm(rs1)`	S	`0100011 / 000`	$M_8[x[rs1] + \text{sext}(imm)] = x[rs2]_{7:0}$
`sh rs2, imm(rs1)`	S	`0100011 / 001`	lo stesso, i 16 bit bassi
`sw rs2, imm(rs1)`	S	`0100011 / 010`	lo stesso, 32 bit
Salti condizionati (la destinazione è $pc + \text{sext}(imm)$)			
`beq rs1, rs2, label`	B	`1100011 / 000`	salta se $x[rs1] = x[rs2]$
`bne rs1, rs2, label`	B	`1100011 / 001`	salta se $x[rs1] \neq x[rs2]$
`blt rs1, rs2, label`	B	`1100011 / 100`	salta se $x[rs1] < x[rs2]$, con segno
`bge rs1, rs2, label`	B	`1100011 / 101`	salta se $x[rs1] \geq x[rs2]$, con segno
`bltu rs1, rs2, label`	B	`1100011 / 110`	salta se $x[rs1] < x[rs2]$, senza segno
`bgeu rs1, rs2, label`	B	`1100011 / 111`	salta se $x[rs1] \geq x[rs2]$, senza segno
Salti			
`jal rd, label`	J	`1101111`	$x[rd] = pc + 4$; $pc = pc + \text{sext}(imm)$
`jalr rd, imm(rs1)`	I	`1100111 / 000`	$t = pc + 4$; $pc = (x[rs1] + \text{sext}(imm)) \,\&\, \sim 1$; $x[rd] = t$
Ordinamento della memoria e ambiente			
`fence`	I	`0001111 / 000`	ordina gli accessi alla memoria come visti da altri hart e dispositivi
`ecall`	I	`1110011 / 000`, imm 0	environment call: solleva un'eccezione (causa 8, 9 o 11)
`ebreak`	I	`1110011 / 000`, imm 1	breakpoint: solleva un'eccezione (causa 3)

Osservazioni.

- Non c'è un'istruzione per sottrarre un immediato: lo fa `addi` con una costante negativa. Non c'è nemmeno una `nop`: è `addi x0, x0, 0`.
- Non ci sono flag. Overflow e riporto si calcolano con `slt` e `sltu` quando servono (Capitolo 5).

Tabella A.3. Gli opcode principali usati da RV32IMAC (bit da 6 a 0).

Opcode	Nome	Formato	Istruzioni
`0110111`	LUI	U	`lui`
`0010111`	AUIPC	U	`auipc`
`1101111`	JAL	J	`jal`
`1100111`	JALR	I	`jalr`
`1100011`	BRANCH	B	`beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`
`0000011`	LOAD	I	`lb`, `lh`, `lw`, `lbu`, `lhu`
`0100011`	STORE	S	`sb`, `sh`, `sw`
`0010011`	OP-IMM	I	`addi`, `slti`, `sltiu`, `xori`, `ori`, `andi`, scorrimenti
`0110011`	OP	R	operazioni registro-registro; l'estensione M
`0001111`	MISC-MEM	I	`fence`, `fence.i`
`1110011`	SYSTEM	I	`ecall`, `ebreak`, `mret`, `wfi`, istruzioni CSR
`0101111`	AMO	R	l'estensione A

- Una costante di 32 bit si carica con `lui` seguita da `addi`. Poiché `addi` estende in segno il suo immediato, quando il bit 11 della costante vale 1 la parte superiore va aumentata di uno: l'assemblatore lo fa quando espande `li`, e gli operatori di rilocazione `%hi` e `%lo` lo fanno per gli indirizzi.
- Un'istruzione la cui codifica è tutta zeri o tutta uni è illegale per definizione: un salto in una memoria cancellata o vuota viene colto subito.
- Con l'estensione C le istruzioni sono allineate a 2 byte; senza, a 4, e un salto a un indirizzo non allineato solleva un'eccezione.

A.5 L'estensione M: moltiplicazione e divisione

Tabella A.5. L'estensione M: formato R, opcode `0110011`, funct7 `0000001`.

Assembly	funct3	Effetto
`mul rd, rs1, rs2`	`000`	i 32 bit bassi del prodotto (gli stessi per operandi con e senza segno)
`mulh rd, rs1, rs2`	`001`	i 32 bit alti del prodotto con segno × con segno
`mulhsu rd, rs1, rs2`	`010`	i 32 bit alti del prodotto con segno × senza segno
`mulhu rd, rs1, rs2`	`011`	i 32 bit alti del prodotto senza segno × senza segno
`div rd, rs1, rs2`	`100`	quoziente con segno, arrotondato verso zero
`divu rd, rs1, rs2`	`101`	quoziente senza segno
`rem rd, rs1, rs2`	`110`	resto con segno; ha il segno del dividendo
`remu rd, rs1, rs2`	`111`	resto senza segno

La divisione non solleva mai un'eccezione. Una divisione per zero dà un quoziente di tutti uni (-1, o $2^{32}-1$ senza segno) e un resto uguale al dividendo. L'unico overflow con segno, $-2^{31}/-1$, dà il quoziente -2^{31} e il resto 0. Un programma che deve rilevare una divisione per zero esamina il divisore con un salto.

A.6 L'estensione A: istruzioni atomiche

L'estensione A serve ai programmi in cui più thread di esecuzione condividono la memoria. Tutte le sue istruzioni hanno formato R, opcode `0101111` e, per le parole, funct3 `010`; i cinque bit più significativi selezionano l'operazione (Tabella A.6).

Tabella A.6. L'estensione A (RV32).

Assembly	funct5	Effetto
`lr.w rd, (rs1)`	00010	load riservato: $x[rd] = M[x[rs1]]$, e l'indirizzo è riservato
`sc.w rd, rs2, (rs1)`	00011	store condizionato: memorizza $x[rs2]$ solo se la prenotazione vale ancora; $x[rd] = 0$ in caso di successo, diverso da zero in caso di fallimento
`amoswap.w rd, rs2, (rs1)`	00001	atomicamente: $x[rd] = M[x[rs1]]$; $M[x[rs1]] = x[rs2]$
`amoadd.w rd, rs2, (rs1)`	00000	atomicamente: $x[rd] = M$; $M = M + x[rs2]$
`amoxor.w`, `amoand.w`, `amoor.w`	00100, 01100, 01000	lo stesso con xor, and, or
`amomin.w`, `amomax.w`	10000, 10100	lo stesso con minimo, massimo con segno
`amominu.w`, `amomaxu.w`	11000, 11100	lo stesso con minimo, massimo senza segno

A.7 L'estensione C: istruzioni compresse

L'estensione C dà una codifica a 16 bit alle istruzioni più frequenti. Ogni istruzione compressa si espande esattamente in un'istruzione a 32 bit, così che l'estensione non aggiunge niente a ciò che il processore sa fare: rende i programmi più corti di circa un quarto. L'assemblatore sceglie da solo la forma corta ogni volta che gli operandi lo consentono; un disassemblatore la mostra con il prefisso `c.` (Tabella A.7). Un'istruzione a 16 bit si riconosce dai suoi due bit meno significativi, che sono diversi da `11`.

Tabella A.7. Le istruzioni compresse di RV32C (senza virgola mobile).

Compressa	Si espande in	Restrizioni
`c.addi4spn rd', n`	`addi rd', sp, n`	n multiplo di 4 diverso da zero, fino a 1020
`c.lw rd', n(rs1')`	`lw rd', n(rs1')`	n multiplo di 4, fino a 124
`c.sw rs2', n(rs1')`	`sw rs2', n(rs1')`	come sopra
`c.nop`	`addi x0, x0, 0`	
`c.addi rd, n`	`addi rd, rd, n`	$-32 \le n \le 31$, $n \neq 0$
`c.jal label`	`jal ra, label`	solo RV32; ± 2 KiB
`c.li rd, n`	`addi rd, x0, n`	$-32 \le n \le 31$
`c.addi16sp n`	`addi sp, sp, n`	n multiplo di 16 diverso da zero, $-512 \ldots 496$
`c.lui rd, n`	`lui rd, n`	$rd \neq$ `x0`, `x2`; $n \neq 0$
`c.srli rd', sh`	`srli rd', rd', sh`	
`c.srai rd', sh`	`srai rd', rd', sh`	
`c.andi rd', n`	`andi rd', rd', n`	$-32 \le n \le 31$

Compressa	Si espande in	Restrizioni
`c.sub rd', rs2'`	`sub rd', rd', rs2'`	
`c.xor`, `c.or`, `c.and`	`xor`, `or`, `and rd', rd', rs2'`	
`c.j label`	`jal x0, label`	±2 KiB
`c.beqz rs1', label`	`beq rs1', x0, label`	±256 byte
`c.bnez rs1', label`	`bne rs1', x0, label`	±256 byte
`c.slli rd, sh`	`slli rd, rd, sh`	
`c.lwsp rd, n(sp)`	`lw rd, n(sp)`	n multiplo di 4, fino a 252
`c.swsp rs2, n(sp)`	`sw rs2, n(sp)`	come sopra
`c.jr rs1`	`jalr x0, 0(rs1)`	$rs1 \neq$ `x0`; `c.jr ra` è `ret`
`c.jalr rs1`	`jalr ra, 0(rs1)`	
`c.mv rd, rs2`	`add rd, x0, rs2`	
`c.add rd, rs2`	`add rd, rd, rs2`	
`c.ebreak`	`ebreak`	

I registri scritti `rd'`, `rs1'`, `rs2'` sono nominati da 3 bit e possono essere solo `x8`–`x15`. Questo spiega ciò che mostra il debugger di PRG32-QT: una `addi t3, t3, 1` diventa `c.addi t3, 1`, una `ret` diventa `c.jr ra`, e il salvataggio di `ra` a `12(sp)` diventa `c.swsp ra, 12(sp)`.

A.8 Pseudoistruzioni

L'assemblatore accetta un certo numero di mnemonici che non sono istruzioni e li espande (Tabella A.8).

Tabella A.8. Le pseudoistruzioni dell'assemblatore RISC-V.

Pseudoistruzione	Espansione	Significato
`nop`	`addi x0, x0, 0`	nessuna operazione
`li rd, imm`	`addi rd, x0, imm`, oppure `lui` + `addi`	carica una costante
`la rd, symbol`	`auipc rd, hi` + `addi rd, rd, lo`	carica un indirizzo, relativo al PC
`lw rd, symbol`	`auipc rd, hi` + `lw rd, lo(rd)`	carica una variabile globale
`sw rs, symbol, rt`	`auipc rt, hi` + `sw rs, lo(rt)`	memorizza una variabile globale, attraverso `rt`
`mv rd, rs`	`addi rd, rs, 0`	copia un registro
`not rd, rs`	`xori rd, rs, -1`	complemento bit a bit
`neg rd, rs`	`sub rd, x0, rs`	complemento a due
`seqz rd, rs`	`sltiu rd, rs, 1`	pone a 1 se zero
`snez rd, rs`	`sltu rd, x0, rs`	pone a 1 se diverso da zero
`sltz rd, rs`	`slt rd, rs, x0`	pone a 1 se negativo
`sgtz rd, rs`	`slt rd, x0, rs`	pone a 1 se positivo
`beqz rs, label`	`beq rs, x0, label`	salta se zero
`bnez rs, label`	`bne rs, x0, label`	salta se diverso da zero
`blez rs, label`	`bge x0, rs, label`	salta se ≤ 0
`bgez rs, label`	`bge rs, x0, label`	salta se ≥ 0
`bltz rs, label`	`blt rs, x0, label`	salta se < 0

Pseudoistruzione	Espansione	Significato
`bgtz rs, label`	`blt x0, rs, label`	salta se > 0
`bgt rs, rt, label`	`blt rt, rs, label`	salta se maggiore, con segno
`ble rs, rt, label`	`bge rt, rs, label`	salta se minore o uguale, con segno
`bgtu rs, rt, label`	`bltu rt, rs, label`	salta se maggiore, senza segno
`bleu rs, rt, label`	`bgeu rt, rs, label`	salta se minore o uguale, senza segno
`j label`	`jal x0, label`	salto incondizionato
`jal label`	`jal ra, label`	salta e collega
`jr rs`	`jalr x0, 0(rs)`	salta a un registro
`jalr rs`	`jalr ra, 0(rs)`	chiamata attraverso un registro
`ret`	`jalr x0, 0(ra)`	ritorno da una funzione
`call symbol`	`auipc ra, hi` + `jalr ra, lo(ra)`	chiama una funzione (il linker può accorciarla in `jal`)
`tail symbol`	`auipc t1, hi` + `jalr x0, lo(t1)`	salta a una funzione senza collegare
`csrr rd, csr`	`csrrs rd, csr, x0`	legge un CSR
`csrw csr, rs`	`csrrw x0, csr, rs`	scrive un CSR
`csrs csr, rs`	`csrrs x0, csr, rs`	pone a 1 dei bit in un CSR
`csrc csr, rs`	`csrrc x0, csr, rs`	azzera dei bit in un CSR
`csrwi`, `csrsi`, `csrci`	`csrrwi`, `csrrsi`, `csrrci` con `rd` = `x0`	lo stesso con un immediato di 5 bit

A.9 L'assemblatore: direttive, etichette ed espressioni

L'assemblatore del corso è l'assemblatore GNU della toolchain RISC-V di ESP-IDF. Un file sorgente ha l'estensione `.S`, il che significa che passa prima attraverso il preprocessore C: `#include` e `#define` sono disponibili, e un `#` comincia un commento solo quando non può essere una direttiva del preprocessore. I commenti si possono scrivere anche come in C, tra `/*` e `*/`.

Etichette. Un nome seguito da due punti è un'etichetta e sta per l'indirizzo di ciò che segue. Un'etichetta fatta di un solo numero è *locale* e può essere definita molte volte: `1b` si riferisce all'etichetta `1:` più vicina *all'indietro* (*backwards*), e `1f` a quella più vicina *in avanti* (*forwards*).

Espressioni. Dovunque sia attesa una costante si può scrivere un'espressione, con gli operatori del C, che viene calcolata dall'assemblatore: `COLS*ROWS`, `grid+12*COLS`, `~0x10`, `1 << 4`. I numeri sono decimali, esadecimali (`0x`), binari (`0b`) o caratteri (`'A'`).

Operatori di rilocazione. `%hi(sym)` e `%lo(sym)` danno i 20 bit superiori e i 12 inferiori di un indirizzo assoluto, già corretti per l'estensione in segno della parte inferiore; `%pcrel_hi(sym)` e `%pcrel_lo(label)` fanno lo stesso per un indirizzo relativo al PC, e sono ciò che usano `la` e `call`.

Tabella A.9. Le direttive usate in questo libro.

Direttiva	Effetto
`.section .text` (o `.text`)	ciò che segue è codice eseguibile
`.section .rodata`	dati di sola lettura: stringhe, tabelle
`.section .data` (o `.data`)	variabili inizializzate
`.section .bss` (o `.bss`)	variabili inizializzate a zero, che non occupano spazio nel file
`.global name` (o `.globl`)	rende l'etichetta visibile fuori dal file
`.equ NAME, value` (o `.set`)	dà un nome a una costante; non si usa memoria
`.byte`, `.half`, `.word`	memorizzano valori di 8, 16, 32 bit, separati da virgole
`.ascii "text"`	memorizza i caratteri di una stringa
`.asciz "text"` (o `.string`)	lo stesso, seguiti da un byte a zero
`.space n` (o `.zero n`)	riserva n byte posti a zero
`.balign n`	avanza al successivo multiplo di n byte
`.align n`, `.p2align n`	avanza al successivo multiplo di 2^n byte
`.option norelax`	vieta al linker di accorciare le sequenze (usata in ogni cartuccia PRG32)
`.option push`, `.option pop`	salvano e ripristinano le opzioni
`.option rvc`, `.option norvc`	consentono o vietano le istruzioni compresse
`.macro name args ... .endm`	definiscono una macro
`.include "file"`	inserisce un altro file sorgente

A.10 La convenzione di chiamata

La convenzione di chiamata standard per RV32 senza virgola mobile (la ABI chiamata ILP32) è la seguente (RISC-V International, 2024a).

1. **Argomenti.** I primi otto argomenti di una funzione viaggiano in `a0`–`a7`, in ordine; gli argomenti ulteriori vanno sullo stack. Un valore di 64 bit occupa due registri, la parola bassa per prima.
2. **Risultati.** Il risultato torna in `a0` (e `a1` per 64 bit).
3. **Registri salvati dal chiamante.** `ra`, `t0`–`t6` e `a0`–`a7` possono essere distrutti da qualsiasi chiamata. Un chiamante a cui servono i loro valori dopo la chiamata li salva da sé.
4. **Registri salvati dal chiamato.** `sp` e `s0`–`s11` devono avere, al ritorno, il valore che avevano alla chiamata. Una funzione che li usa li salva all'ingresso e li ripristina all'uscita.
5. **Lo stack.** Cresce verso gli indirizzi bassi; `sp` punta all'ultima parola occupata ed è multiplo di 16 a ogni chiamata. Una funzione può usare la memoria sotto `sp` solo dopo aver abbassato `sp`.
6. **`gp` e `tp`** non vanno modificati dal codice ordinario.

Il Listato A.1 è lo schema di una funzione che chiama altre funzioni e usa due registri salvati; a una funzione *foglia* che usa solo registri `t` e `a` non serve niente di tutto questo.

```
my_function:
    addi sp, sp, -16        # prologue: a frame of 16 bytes
    sw   ra, 12(sp)         # the return address
```

```
    sw   s0, 8(sp)          # the saved registers that will be used
    sw   s1, 4(sp)

    mv   s0, a0             # values that must survive the calls
    call other_function     # may destroy ra, t0-t6, a0-a7
    add  a0, a0, s0         # the result goes in a0

    lw   ra, 12(sp)         # epilogue, in reverse order
    lw   s0, 8(sp)
    lw   s1, 4(sp)
    addi sp, sp, 16
    ret
```

Listato A.1. Il frame di una funzione non foglia.

Su PRG32 i tre punti di ingresso di una cartuccia sono funzioni ordinarie che seguono questa convenzione, e così tutte le funzioni del runtime (Appendice B).

A.11 Privilegio, registri di controllo e stato, trap

A.11.1 Modi di privilegio

Tabella A.10. I modi di privilegio.

Livello	Nome	Esegue	ESP32-C6
0	utente (U)	le applicazioni	sì
1	supervisore (S)	il kernel di un sistema operativo	no
3	macchina (M)	il firmware; l'unico modo obbligatorio	sì

A.11.2 Le istruzioni CSR (Zicsr)

I registri di controllo e stato formano uno spazio di indirizzi separato di 12 bit, 4 096 registri. Sono letti e scritti in modo atomico da sei istruzioni di formato I, opcode `1110011`, il cui campo immediato è l'indirizzo del CSR (Tabella A.11).

Tabella A.11. Le istruzioni CSR.

Assembly	funct3	Effetto
`csrrw rd, csr, rs1`	001	$t = CSR$; $CSR = x[rs1]$; $x[rd] = t$
`csrrs rd, csr, rs1`	010	$t = CSR$; $CSR = t \mid x[rs1]$; $x[rd] = t$ (pone a 1 dei bit)
`csrrc rd, csr, rs1`	011	$t = CSR$; $CSR = t \,\&\, \sim x[rs1]$; $x[rd] = t$ (azzera dei bit)
`csrrwi rd, csr, uimm`	101	come `csrrw`, con un immediato senza segno di 5 bit
`csrrsi rd, csr, uimm`	110	come `csrrs`, con l'immediato
`csrrci rd, csr, uimm`	111	come `csrrc`, con l'immediato

A.11.3 I registri del modo macchina

A.11.4 Ingresso in una trap e ritorno

Quando una trap viene accolta nel modo macchina l'hardware compie, in un solo passo, le azioni seguenti.

Tabella A.12. I principali CSR del modo macchina.

Indirizzo	Nome	Contenuto
`0x300`	`mstatus`	stato: MIE (bit 3), abilitazione globale delle interruzioni; MPIE (bit 7), il suo valore prima della trap; MPP (bit 12–11), il modo prima della trap
`0x301`	`misa`	la base e le estensioni realizzate
`0x304`	`mie`	abilitazioni delle interruzioni: MSIE (bit 3), MTIE (bit 7), MEIE (bit 11)
`0x305`	`mtvec`	vettore delle trap: indirizzo base (bit 31–2) e modo (bit 1–0): 0 diretto, 1 vettorizzato
`0x340`	`mscratch`	una parola a uso del gestore delle trap
`0x341`	`mepc`	l'indirizzo dell'istruzione che è stata interrotta, o che ha causato l'eccezione
`0x342`	`mcause`	bit 31: 1 per un'interruzione, 0 per un'eccezione; gli altri bit: il codice della causa
`0x343`	`mtval`	informazione aggiuntiva: l'indirizzo che ha causato l'errore, o l'istruzione illegale
`0x344`	`mip`	interruzioni in attesa: MSIP (bit 3), MTIP (bit 7), MEIP (bit 11)
`0xB00`, `0xB02`	`mcycle`, `minstret`	contatori dei cicli di clock e delle istruzioni completate (i 32 bit bassi)
`0xF11`–`0xF14`	`mvendorid`, `marchid`, `mimpid`, `mhartid`	identificazione del costruttore, dell'architettura, dell'implementazione e del thread hardware

1. `mepc` = l'indirizzo dell'istruzione interrotta (per un'interruzione) o dell'istruzione che ha causato l'eccezione.
2. `mcause` = la causa; `mtval` = l'informazione aggiuntiva.
3. In `mstatus`: MPIE = MIE; MIE = 0; MPP = il modo corrente.
4. Il modo diventa M.
5. $pc = base$ di `mtvec`; oppure, nel modo vettorizzato e per un'interruzione, $pc = base + 4 \times causa$.

L'istruzione `mret` (codifica `0x30200073`) ritorna: il modo diventa quello in MPP, MIE = MPIE, MPIE = 1, e $pc =$ `mepc`. L'istruzione `wfi` (`0x10500073`), *wait for interrupt*, permette al processore di fermarsi finché un'interruzione è in attesa.

Un'interruzione del modo macchina viene accolta quando il suo bit è a 1 sia in `mip` sia in `mie` e, se il processore è nel modo macchina, MIE vale 1. Dopo un'eccezione, un gestore che vuole riprendere *dopo* l'istruzione che l'ha causata (per esempio dopo una `ecall`) deve sommare 4, o 2 per un'istruzione compressa, a `mepc`.

A.12 Codificare e decodificare a mano

Per decodificare una parola di 32 bit: scrivetela in binario; leggete i bit 6–0 e trovate il formato nella Tabella A.3; tagliate i campi come nella Figura A.1; trovate l'istruzione con

Tabella A.13. I valori standard di `mcause`.

Interruzioni (bit 31 = 1)		**Eccezioni** (bit 31 = 0)	
Codice	**Causa**	**Codice**	**Causa**
1	interruzione software di supervisore	0	indirizzo di istruzione non allineato
3	interruzione software di macchina	1	errore di accesso a un'istruzione
5	interruzione del timer di supervisore	2	istruzione illegale
7	interruzione del timer di macchina	3	breakpoint
9	interruzione esterna di supervisore	4	indirizzo di load non allineato
11	interruzione esterna di macchina	5	errore di accesso in load
≥ 16	definite dalla piattaforma	6	indirizzo di store non allineato
		7	errore di accesso in store
		8	environment call dal modo U
		9	environment call dal modo S
		11	environment call dal modo M
		12	page fault su un'istruzione
		13	page fault in load
		15	page fault in store

`funct3` (e `funct7`) nella Tabella A.4 o A.5; ricostruite l'immediato ed estendetelo in segno; traducete i numeri dei registri con la Tabella A.2.

Per codificare, si procede nel verso opposto. Per esempio `sw a0, 8(sp)` è uno store di formato S con `rs2` = `a0` = 10, `rs1` = `sp` = 2, `funct3` = 010 e l'immediato $8 = 0000000\,01000_2$:

$$\underbrace{0000000}_{imm[11:5]}\ \underbrace{01010}_{rs2}\ \underbrace{00010}_{rs1}\ \underbrace{010}_{funct3}\ \underbrace{01000}_{imm[4:0]}\ \underbrace{0100011}_{opcode} = \texttt{0x00A12423}.$$

La Tabella A.14 raccoglie le codifiche incontrate nel libro.

Tabella A.14. Codifiche incontrate nel libro (forme non compresse).

Istruzione	**Codifica**	**Istruzione**	**Codifica**
`addi t1, t1, 1`	`0x00130313`	`sub t0, t1, t2`	`0x407302B3`
`lw t1, 0(t0)`	`0x0002A303`	`sub t1, t0, t1`	`0x40628333`
`sw t1, 0(t0)`	`0x0062A023`	`addi t0, t0, 10`	`0x00A28293`
`add t2, t0, t1`	`0x006283B3`	`sw a0, 8(sp)`	`0x00A12423`
`ret`	`0x00008067`	`nop`	`0x00000013`
`ecall`	`0x00000073`	`ebreak`	`0x00100073`
`mret`	`0x30200073`	`wfi`	`0x10500073`

A.13 Idiomi

La Tabella A.15 raccoglie le brevi sequenze che ricorrono nei programmi di questo libro.

Tabella A.15. Idiomi comuni.

Scopo	**Codice**
esaminare il bit n di `t0`	`srli t1, t0, n` ; `andi t1, t1, 1`
porre a 1, azzerare, invertire dei bit con una maschera m	`ori t0, t0, m` ; `andi t0, t0, ~m` ; `xori t0, t0, m`
moltiplicare per 2^k; divisione senza e con segno per 2^k	`slli` ; `srli` ; `srai` (che arrotonda verso $-\infty$)
elemento i di un array di parole a `t0`	`slli t1, t1, 2` ; `add t0, t0, t1` ; `lw a0, 0(t0)`
`a0` e `a1` sono uguali? (1 o 0)	`xor a0, a0, a1` ; `seqz a0, a0`
limitare `t2` a $[0, max]$	`bgez t2, 1f` ; `li t2, 0` ; `1: ble t2, t3, 2f` ; `mv t2, t3` ; `2:`
indice modulo una potenza di due n	`andi t0, t0, n-1`
overflow senza segno di `add t2, t0, t1`	`sltu t3, t2, t0`
fronte di un segnale: ora e non prima	`not t1, t1` ; `and s1, s0, t1`
virgola fissa Q24.8: a intero; prodotto	`srai a0, t1, 8` ; `mul` poi `srai ..., 8`
da cifra decimale a carattere ASCII	`addi t0, t0, 48`
un ciclo contato	`li t1, n` ; `1: ...` ; `addi t1, t1, -1` ; `bnez t1, 1b`

APPENDICE B

Riferimento di PRG32

Questa appendice è un riferimento della piattaforma PRG32 così come è usata nel libro: il contratto tra una cartuccia e il runtime, l'interfaccia di programmazione, il file della cartuccia, gli strumenti a riga di comando, i tre host e il Cartridge Store. È stata compilata dai sorgenti e dalla documentazione dei repository nell'ottobre 2026 (Montella et al., 2026a; Montella, 2026; PRG32 Project, 2026); la piattaforma evolve, e i repository prevalgono su questo testo. Il manuale di laboratorio della piattaforma è il volume di accompagnamento (Montella et al., 2026b).

B.1 La piattaforma in breve

Tabella B.1. La piattaforma PRG32 in numeri.

Processore	Espressif ESP32-C6: un core RISC-V, RV32IMAC, fino a 160 MHz, modo macchina e modo utente, nessuna virgola mobile
Memoria	512 KB di SRAM, 320 KB di ROM, flash esterna
Display	TFT ILI9341, 320 × 240 pixel, su SPI
Viewport di gioco	320 × 200 pixel; due bande di stato di 20 pixel sopra e sotto
Frame buffer	8 bit per pixel (indici di palette), con una palette di 256 colori RGB565
Griglia di testo	40 × 25 caratteri di 8 × 8 pixel
Tile	8 × 8 pixel; un playfield di 64 × 32 tile su 2 livelli
Ingresso	joystick (4 direzioni) e pulsanti A, B, START/SELECT, per uno o due giocatori
Suono	amplificatore I2S (MAX98357A); note, campioni, tracce
Rete	Wi-Fi: access point `PRG32` o stazione su una rete esistente
Cartuccia	un file `.prg32`, caricato a `0x40800000`; 4 slot di 131 072 byte in flash
RAM della cartuccia	64 KiB per impostazione predefinita (esistono profili da 32 e da 128 KiB)
Host	la scheda ESP32-C6; QEMU di Espressif (ESP32-C3 emulato); PRG32-QT
Licenza	MIT

Tabella B.2. I repository dell'organizzazione `riscv-prg32` su GitHub.

Repository	Contenuto
`PRG32`	il firmware residente, gli strumenti per le cartucce (`python3 -m prg32`), gli esempi, la documentazione
`PRG32-QT`	l'host per desktop e dispositivi mobili, con l'interprete RV32IMAC e il debugger
`PRG32-Construction-Kit`	uno studio per costruire giochi con blocchi e sprite
`Playing-with-RISC-V_PRG32_textbook`	il manuale di laboratorio, con i suoi laboratori verificati

B.2 Il contratto

Una cartuccia fornisce tre funzioni, i cui nomi sono un prefisso scelto dal programmatore seguito da `_init`, `_update` e `_draw`; il prefisso si passa al compilatore di cartucce con `--entry-prefix`.

`prefix_init`
: è chiamata una volta, all'avvio della cartuccia. Prepara lo stato.

`prefix_update`
: è chiamata una volta per frame. Legge l'ingresso e cambia lo stato.

`prefix_draw`
: è chiamata una volta per frame, dopo `update`. Trasforma lo stato in un'immagine.

Le tre funzioni seguono la convenzione di chiamata standard di RISC-V (Appendice A), devono ritornare, e non devono disabilitare le interruzioni né toccare i registri di controllo e stato. I registri non conservano i loro valori tra una chiamata e la successiva: tutto ciò che deve durare vive nelle sezioni `.data` o `.bss` della cartuccia. Una funzione che non ritorna mai blocca il frame: PRG32-QT segnala «instruction budget exhausted», e sulla scheda un watchdog segnala lo stallo.

La tabella ABI. Una cartuccia *portabile* non contiene gli indirizzi delle funzioni del runtime. Quando il runtime entra in `init`, `update` o `draw` passa in `a0` un puntatore a una tabella di puntatori a funzione, la tabella ABI; piccoli stub aggiunti dal compilatore di cartucce memorizzano quel puntatore e permettono al sorgente di mantenere la forma familiare `call prg32_gfx_clear`. La tabella ha un numero magico (`PABI`), una versione, un hash e un insieme di bit di funzionalità (audio, Wi-Fi, multigiocatore, metriche, tastiera, mappa di tile, platform, sprite); una cartuccia dichiara nella sua intestazione l'hash e le funzionalità che richiede, e un host la accetta se può onorarli. Le nuove funzioni vengono sempre e solo accodate alla tabella, così che una vecchia cartuccia continua a funzionare su un nuovo host.

```
# mygame.S
.option norelax

.equ BLACK, 0x0000

.section .text
.global mygame_init
.global mygame_update
```

```
.global mygame_draw

mygame_init:                # once: set up the state
    ret

mygame_update:              # every frame: input, state
    ret

mygame_draw:                # every frame: state to picture
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret

.section .rodata            # constants: strings, tables
.section .data              # variables
```

Listato B.1. Lo scheletro di una cartuccia PRG32 in assembly.

B.3 Schermo, colori e ingresso

Coordinate. L'origine è l'angolo in alto a sinistra della viewport di gioco; x cresce verso destra da 0 a 319 e y cresce verso il basso da 0 a 199. Ciò che si disegna fuori dalla viewport viene tagliato.

Colori. Un colore è un valore di 16 bit in RGB565: 5 bit di rosso (bit 15–11), 6 di verde (bit 10–5), 5 di blu (bit 4–0). Le costanti predefinite sono nella Tabella B.3.

Tabella B.3. Le costanti di colore (`PRG32_COLOR_*`).

BLACK	WHITE	RED	GREEN	BLUE	YELLOW	CYAN	MAGENTA
`0x0000`	`0xffff`	`0xf800`	`0x07e0`	`0x001f`	`0xffe0`	`0x07ff`	`0xf81f`

Pulsanti. `prg32_input_read` restituisce una parola con un bit per pulsante, 1 quando è premuto (Tabella B.4). START e SELECT sono due nomi dello stesso ingresso.

Tabella B.4. Le maschere dei pulsanti (`PRG32_BTN_*` e `PRG32_P2_BTN_*`).

	LEFT	RIGHT	UP	DOWN	A	B	START
Giocatore 1: bit	0	1	2	3	4	5	6
Giocatore 1: maschera	`0x0001`	`0x0002`	`0x0004`	`0x0008`	`0x0010`	`0x0020`	`0x0040`
Giocatore 2: bit	8	9	10	11	12	13	14
Giocatore 2: maschera	`0x0100`	`0x0200`	`0x0400`	`0x0800`	`0x1000`	`0x2000`	`0x4000`

B.4 L'interfaccia di programmazione

Le funzioni che una cartuccia portabile può chiamare sono gli elementi della tabella ABI: 139 nella versione 1.6. Le tabelle che seguono ne danno i prototipi C, come dichiarati in `components/prg32/include/prg32.h`. In assembly gli argomenti vanno, in ordine, in `a0`, `a1`, …, e il risultato torna in `a0`; una chiamata può distruggere ogni registro `t` e `a` e `ra`.

B.4.1 Tempo, ingresso, numeri casuali

Prototipo	Effetto
`uint32_t prg32_ticks_ms(void)`	millisecondi dall'avvio
`uint32_t prg32_input_read(void)`	i pulsanti di entrambi i giocatori, un bit ciascuno
`uint32_t prg32_input_read_player(uint8_t player)`	i pulsanti di un giocatore
`uint32_t prg32_input_read_menu(void)`	i pulsanti come li leggono i menu
`void prg32_input_wait_released(uint32_t mask)`	aspetta finché i pulsanti della maschera sono rilasciati
`uint32_t prg32_controller_read(void)`	lo stato grezzo dei controller
`uint32_t prg32_random_number(uint32_t min, uint32_t max)`	un numero casuale uniforme in $[min, max]$, estremi inclusi; restituisce min se $max \leq min$

B.4.2 Grafica

Prototipo	Effetto
`void prg32_gfx_clear(uint16_t color)`	riempie la viewport con un colore
`void prg32_gfx_pixel(int x, int y, uint16_t color)`	disegna un pixel
`void prg32_gfx_rect(int x, int y, int w, int h, uint16_t color)`	disegna un rettangolo pieno
`void prg32_gfx_text8(int x, int y, const char *s, uint16_t fg, uint16_t bg)`	scrive una stringa terminata da zero con il font 8×8, sul suo colore di sfondo
`void prg32_gfx_present(void)`	invia il frame al display (il runtime lo fa dopo `draw`)
`void prg32_gfx_set_fullscreen(int enabled)`	usa l'intero display 320×240
`int prg32_gfx_fullscreen_enabled(void)`	se lo schermo intero è attivo
`int prg32_gfx_snapshot_row_rgb565(int y, uint16_t *out, size_t pixels)`	copia una riga del frame come RGB565
`void prg32_palette_set(uint8_t index, uint16_t rgb565)`	cambia un elemento della palette: ogni pixel con quell'indice cambia al frame successivo
`uint16_t prg32_palette_get(uint8_t index)`	legge un elemento della palette
`void prg32_gfx_clear_indexed(uint8_t index)`	come `clear`, con un indice di palette
`void prg32_gfx_pixel_indexed(int x, int y, uint8_t index)`	come `pixel`, con un indice di palette
`void prg32_gfx_rect_indexed(int x, int y, int w, int h, uint8_t index)`	come `rect`, con un indice di palette

B.4.3 Console, bande di stato, schermate di apertura

Prototipo	Effetto
`void prg32_console_clear(void)`	cancella la console di testo
`void prg32_console_putc(int ch)`	scrive un carattere
`void prg32_console_write(const char *s)`	scrive una stringa
`void prg32_console_hex32(uint32_t value)`	scrive una parola come otto cifre esadecimali
`void prg32_band_set_mode(uint8_t band, prg32_band_mode_t mode)`	sceglie che cosa mostra una banda di stato (banda 0 in alto, 1 in basso)
`void prg32_band_set_text(uint8_t band, const char *text)`	scrive un testo in una banda
`void prg32_band_set_game_info(const char *text)`	scrive la riga di informazioni del gioco
`void prg32_band_log(const char *message)`	accoda un messaggio al log della banda
`void prg32_band_set_colors(uint8_t band, uint16_t fg, uint16_t bg)`	imposta i colori di una banda
`void prg32_splash_show_game(const char *title, const char *subtitle, uint32_t duration_ms, uint16_t bg, uint16_t fg, uint16_t accent)`	mostra una schermata del titolo per un certo tempo
`void prg32_debug_overlay_draw(int enabled, int x, int y, uint32_t input_mask, uint32_t frame)`	disegna la maschera di ingresso e il numero di frame

Sulla scheda e in QEMU la console è disegnata sullo schermo; PRG32-QT la raccoglie nel suo log. Le altre funzioni dei gruppi `prg32_band_*` e `prg32_splash_*` della tabella (`band_mode`, `band_mode_name`, `band_use_default_colors`, `band_load_config`, `band_save_config`, `splash_draw`, `splash_show`, `splash_draw_game`, `splash_show_default`) sono varianti di queste.

B.4.4 Suono

Prototipo	Effetto
`void prg32_audio_note(uint8_t channel, uint8_t instrument, uint8_t note, uint8_t volume, uint32_t duration_ms)`	suona una nota MIDI (0–127; 69 è il La4, 440 Hz; 72 è il Do5) per una durata
`void prg32_audio_note_off(uint8_t channel)`	silenzia un canale
`void prg32_audio_notes(uint8_t channel, uint8_t instrument, uint8_t volume, const prg32_midi_note_t *notes, size_t count)`	suona una sequenza di note
`void prg32_buzzer_tone(uint32_t hz, uint32_t ms, uint16_t duty)`	un tono di una data frequenza sul buzzer
`void prg32_buzzer_play_notes(const prg32_note_t *notes, size_t count)`	una sequenza di toni
`void prg32_buzzer_sample_u8(const uint8_t *samples, size_t count, uint32_t sample_rate)`	riproduce campioni PCM a 8 bit senza segno centrati su 128

Le chiamate audio sono sicure quando non c'è un'uscita disponibile: non fanno niente. Il runtime possiede l'inizializzazione dell'audio e il volume principale. La tabella contiene anche l'interfaccia polifonica (`prg32_audio_note_on`, `note_on_pan`, `play_sample`, `play_sample_pan`, `play_track`, `stop_track`, `stop_channel`, `stop_all`, `set_tempo`, `set_master_volume`, `set_channel_volume`, `set_channel_pan`), descritta in `docs/software/audio_polyphony.md`.

B.4.5 Tile, playfield e piattaforme

Prototipo	Effetto
`void prg32_tile_clear(uint16_t color)`	cancella lo schermo dei tile
`void prg32_tile_define(uint8_t id, const uint8_t *bitmap8x8, uint16_t fg, uint16_t bg)`	definisce un tile da 8 byte, uno per riga
`void prg32_tile_put(uint8_t tx, uint8_t ty, uint8_t id)`	mette un tile in una cella della griglia 40×25
`void prg32_tile_present(void)`	disegna lo schermo dei tile
`void prg32_playfield_clear(uint8_t layer, uint8_t tile_id)`	riempie un livello del playfield 64×32
`void prg32_playfield_put(uint8_t layer, uint8_t tx, uint8_t ty, uint8_t id)`	mette un tile nel playfield
`uint8_t prg32_playfield_get(uint8_t layer, uint8_t tx, uint8_t ty)`	legge un tile del playfield
`void prg32_playfield_scroll(uint8_t layer, int x, int y)`	imposta la posizione di scorrimento di un livello
`void prg32_playfield_scroll_by(uint8_t layer, int dx, int dy)`	sposta la posizione di scorrimento
`void prg32_playfield_parallax(uint8_t layer, int x_q8, int y_q8)`	imposta la velocità di un livello rispetto alla camera, in virgola fissa Q8 ($256 = 1$)
`void prg32_playfield_camera(int x, int y)`	sposta la camera
`void prg32_playfield_draw(uint8_t layer, int transparent_zero)`	disegna un livello
`void prg32_playfield_draw_dual(void)`	disegna entrambi i livelli
`void prg32_platform_tile_flags(uint8_t tile_id, uint8_t flags)`	segna un tile come solido, piattaforma, pericolo o oggetto da raccogliere
`int prg32_platform_solid_at(uint8_t layer, int pixel_x, int pixel_y)`	se un pixel è dentro un tile solido
`void prg32_platform_actor_init(prg32_platform_actor_t *actor, uint8_t layer, int x, int y, int w, int h)`	prepara un attore di un gioco a piattaforme
`uint16_t prg32_platform_actor_step(prg32_platform_actor_t *actor, uint32_t input_mask, int move_speed, int jump_speed, int gravity, int max_fall)`	un passo di movimento con gravità e collisioni; restituisce indicatori di contatto
`void prg32_platform_camera_follow(const prg32_platform_actor_t *actor, int deadzone_x, int deadzone_y)`	fa sì che la camera segua un attore

B.4.6 Sprite

Prototipo	Effetto
`void prg32_sprite_draw_8x8(int x, int y, const uint8_t *bits, uint16_t fg, uint16_t bg)`	disegna uno sprite a un bit di 8×8
`void prg32_sprite_draw_16x16(int x, int y, const uint16_t *rgb565)`	disegna uno sprite RGB565 di 16×16
`void prg32_sprite_draw_24x24(int x, int y, const uint16_t *rgb565)`	disegna uno sprite RGB565 di 24×24
`void prg32_sprite_draw_frame(int x, int y, int w, int h, const uint16_t *frames, uint32_t frame, uint16_t transparent)`	disegna un frame di un'animazione, con un colore trasparente

Prototipo	Effetto
`uint32_t prg32_sprite_anim_frame(uint32_t now_ms, uint32_t frame_count, uint32_t frame_ms)`	quale frame di un'animazione tocca ora
`void prg32_sprite_draw_indexed(int x, int y, const prg32_indexed_sprite_t *sprite, uint32_t frame)`	disegna uno sprite a 1, 2, 4 o 8 bit per pixel con la propria palette
`void prg32_sprite_draw_bitplanes(int x, int y, const prg32_indexed_sprite_t *sprite, uint32_t frame)`	lo stesso, da piani di bit
`int prg32_sprite_hitbox(int ax, int ay, int aw, int ah, int bx, int by, int bw, int bh)`	se due rettangoli si sovrappongono

B.4.7 Punteggi, ingresso di testo e altri servizi

Prototipo	Effetto
`int prg32_score_submit(const char *game, const char *player, uint32_t score)`	registra un punteggio nella tabella locale
`int prg32_score_submit_current_player(const char *game, uint32_t score)`	lo stesso, per il giocatore corrente
`int prg32_score_count(const char *game)`	quanti punteggi sono registrati
`int prg32_score_get(const char *game, int index, prg32_score_t *out_score)`	legge un punteggio registrato
`int prg32_scoreboard_show(const char *game, const char *title)`	mostra la tabella dei punteggi migliori
`int prg32_score_sync_remote(void)`	sincronizza i punteggi con lo Store
`int prg32_text_input(char *buffer, size_t capacity, const char *title)`	chiede al giocatore un testo con una tastiera sullo schermo

Gli elementi restanti della tabella servono la tastiera sullo schermo (`prg32_keyboard_*`), il nome del giocatore (`prg32_score_player_*`), gli slot delle cartucce (`prg32_cart_*`), lo stato del Wi-Fi (`prg32_wifi_*`), i giochi multigiocatore (`prg32_multiplayer_*`) e la misura delle prestazioni (`prg32_perf_*`); sono documentati in `docs/software` e `docs/measurement` del repository.

B.5 Il file della cartuccia

Un file `.prg32` è un'intestazione seguita dal codice e dai dati del programma, opzionalmente seguiti da un trailer di metadati. I campi di più byte sono little-endian.

Il primo gruppo di campi della Tabella B.12 è l'intestazione originale, che i vecchi strumenti sanno ancora leggere; il secondo gruppo la estende per le cartucce portabili. Il comando `cartridge summary` li stampa tutti.

Limiti. La memoria necessaria (`mem_size`) deve stare nella RAM per le cartucce dell'host: 64 KiB nel profilo predefinito, 32 KiB nel profilo «classroom», 128 KiB in un profilo opzionale della scheda. L'opzione `--cart-ram-kib` del compilatore di cartucce imposta il limite rispetto al quale controllare.

Scrivere cartucce portabili. Una cartuccia portabile può essere caricata a indirizzi diversi su host diversi. Il codice deve quindi essere relativo alla posizione, cosa che l'assem-

Tabella B.12. L'intestazione di una cartuccia (`prg32_cart_header_v2_t`).

Campo	Dimensione	Contenuto
`magic`	4 byte	i caratteri `PRG2`
`abi_major`, `abi_minor`	2 + 2	la versione del formato della cartuccia
`header_size`	2	la lunghezza dell'intestazione: distingue le due versioni
`flags`	2	blocco audio (bit 0), multigiocatore (1), tabella ABI (2), rilocabile (3)
`load_addr`	4	`0x40800000`
`code_size`	4	i byte di codice e dati inizializzati nel file
`mem_size`	4	la memoria necessaria in esecuzione: codice, dati e `.bss`
`init_offset`, `update_offset`, `draw_offset`	4 + 4 + 4	le posizioni dei tre punti di ingresso
`payload_crc32`	4	una somma di controllo del contenuto
`name`	32	il nome della cartuccia
`abi_hash`	4	l'hash della tabella ABI per cui è stata compilata
`required_features`, `optional_features`	4 + 4	i bit di funzionalità necessari, e quelli usati se presenti
`isa_flags`	4	le estensioni dell'insieme di istruzioni usate
`relocation_offset`, `relocation_count`	4 + 4	riservati alla rilocazione
`import_model`	4	1 = `abi-table` (portabile); 0 = importazioni assolute del vecchio tipo

blatore garantisce per `la`, `call` e i salti; una cartuccia non dovrebbe memorizzare indirizzi di codice o di stringhe in tabelle inizializzate di puntatori in `.data`.

Il trailer dei metadati. Il comando `store attach-metadata` accoda dopo il programma un trailer, marcato `PRG32META`, fatto di blocchi: `META` (i metadati, formato `prg32-metadata-1.0`), `ICON` (l'icona), `COLO` (il colophon, formato `prg32-colophon-1.0`) e, opzionalmente, `SCRN` (una schermata) e `SIGN` (una firma). Il programma non viene modificato.

B.6 Gli strumenti a riga di comando

Tutti gli strumenti sono sottocomandi di un solo modulo Python, eseguito dalla cartella del repository di PRG32 con ESP-IDF attivo (Capitolo 1):

```
python3 -m prg32 <command> [subcommand] [options]
```

Ogni comando accetta `--help`. Su Windows l'interprete si chiama `python`. Comandi e sottocomandi possono essere abbreviati a qualsiasi prefisso non ambiguo: `python3 -m prg32 c b` è `cartridge build`. I comandi dello Store leggono `--store-url` e `--token` dalla riga di comando oppure, quando mancano, dal file `~/.prg32/config.json`; un indirizzo senza schema è inteso come `https://`.

Tabella B.13. I comandi di `python3 -m prg32`.

Comando	Effetto
`doctor`	controlla la toolchain: `idf.py`, il compilatore, le partizioni
`runtime --url URL`	stampa lo stato di un host in esecuzione
`cartridge build SRC`	compila una cartuccia portabile da un file `.S` o `.c`. Opzioni: `--out`, `--name`, `--entry-prefix`, `--portable`, `--architecture {esp32c6,qemu}`, `--cart-ram-kib`, `--multiplayer`, `--audio-block`, `--required-feature`, `--optional-feature`, `--build-dir`
`cartridge summary FILE`	stampa l'intestazione di una cartuccia
`esp32c6 upload FILE`	carica una cartuccia su un host via HTTP (`--url`, `--slot cart0`…`cart3`)
`esp32c6 run`	avvia una cartuccia memorizzata
`esp32c6 upload-and-run FILE`	entrambe le cose
`esp32c6 build`, `flash`, `build-and-flash`	compilano il firmware della scheda e lo scrivono attraverso USB
`esp32c6 erase-flash`, `reset`	cancellano la flash; cancellano e riscrivono
`esp32c6 screenshot`, `memory`, `performance`	una schermata, un'analisi della memoria, i risultati del test di prestazioni
`qemu build`, `run`, `build-and-run`	compilano il firmware per QEMU e avviano l'emulatore
`qemu upload FILE`	mette una cartuccia nell'immagine flash di QEMU (con QEMU chiuso)
`store attach-metadata FILE`	accoda o sostituisce il trailer dei metadati (`--metadata`, `--icon`, `--colophon`, `--architecture`, `--out`)
`store inspect-metadata FILE`	stampa il trailer di una cartuccia
`store pack-bundle`	impacchetta il bundle ZIP nominato da un manifesto (`--manifest`, `--out`)
`store publish-bundle ZIP`	sottopone un bundle (`--store-url`, `--token`)
`store list`, `download`, `discover`	elencano e scaricano cartucce; trovano gli Store sulla rete locale
`store set-url`, `clear-url`	impostano lo Store usato dalle compilazioni locali
`wifi set`, `wifi clear`	le credenziali e il modo Wi-Fi della prossima compilazione del firmware (`--ssid`, `--password`, `--mode`)
`abi gen`, `abi check`	generano e verificano gli header della tabella ABI
`debug --url URL ACTION`	controlla il debugger di PRG32-QT: `state`, `enable`, `disable`, `pause`, `step`, `resume`, `speed`
`test`	esegue i test degli strumenti

```
# build
python3 -m prg32 cartridge build mygame.S --portable \
    --entry-prefix mygame --name mygame --out mygame.prg32

# run on PRG32-QT, on this computer
python3 -m prg32 esp32c6 upload-and-run mygame.prg32 \
    --url http://localhost:8080 --slot cart0

# run on the board, joined to its Wi-Fi network
```

```
python3 -m prg32 esp32c6 upload-and-run mygame.prg32 \
    --url http://192.168.4.1 --slot cart0

# run on QEMU
python3 -m prg32 qemu upload mygame.prg32
python3 -m prg32 qemu run
```

Listato B.2. Il ciclo di ogni capitolo.

B.7 Gli host

B.7.1 La scheda

Tabella B.14. Il cablaggio di riferimento del kit: i piedini predefiniti del firmware (`components/prg32/Kconfig`, ottobre 2026).

Display (SPI)		**Giocatore 1 (interruttori verso massa)**	
GPIO7	MOSI	GPIO18	LEFT
GPIO2	MISO	GPIO19	RIGHT
GPIO6	SCLK	GPIO3	UP
GPIO10	CS (selezione del chip)	GPIO13	DOWN
GPIO8	DC (dato/comando)	GPIO21	A
GPIO9	RST (reset)	GPIO22	B
GPIO5	BL (retroilluminazione)	GPIO20	START / SELECT
Audio (I2S)		**Altro**	
GPIO4	BCLK (clock di bit)	—	LED RGB sulla scheda (disabilitato)
GPIO11	LRC / WS (selezione di parola)	USB	firmware, console
GPIO23	DIN (dati)		

I piedini sono opzioni del firmware (`idf.py menuconfig`), e sono cambiati nell'ottobre 2026 con il nuovo prototipo del circuito stampato: DC e RST del display sono passati da GPIO1 e GPIO0 a GPIO8 e GPIO9, e il LED RGB sulla scheda, che è collegato a GPIO8, è stato disabilitato. Al momento in cui scriviamo le tabelle di `docs/hardware/hardware.md` mostrano ancora il cablaggio precedente: quando un display resta nero, confrontate i fili con le opzioni del firmware che è stato caricato.

Ogni pulsante unisce il suo piedino a massa, e il firmware abilita il pull-up interno del piedino e inverte il livello (Capitolo 10). All'avvio, la scheda crea la rete Wi-Fi `PRG32` (password `prg32game`) e risponde a `http://192.168.4.1`; con `wifi set --mode sta` si unisce invece a una rete esistente. Per impostazione predefinita il firmware parte dal menu *Setup*, dal quale si esegue una cartuccia memorizzata (*Run Cartridge*, poi lo slot); un firmware compilato con `PRG32_BOOT_SETUP_MODE=0` avvia direttamente la cartuccia memorizzata.

B.7.2 QEMU

L'emulatore di Espressif esegue il firmware reale su un ESP32-C3 emulato, con una finestra per il display. La tastiera è letta dal terminale che lo ha lanciato. Una cartuccia viene messa nell'immagine flash mentre l'emulatore è chiuso; come la scheda, l'emulatore parte dal menu *Setup*.

B.7.3 PRG32-QT

PRG32-QT è un'applicazione scritta con Qt 6 che interpreta RV32IMAC e offre a una cartuccia la stessa tabella ABI del firmware. Ascolta sulla porta 8080 come una scheda, importa file `.prg32`, e sfoglia lo Store.

Tabella B.15. PRG32-QT: tasti, profili e debugger.

Tasti	frecce o **W A S D**: joystick; **Z** o **J**: A; **X** o **K**: B; **Enter** o **Space**: Start/Select; **Escape**: ritorno a Setup. I controller di gioco sono riconosciuti
Profili di prestazioni	*Accurate* (temporizzato come l'ESP32-C6, il predefinito), *Optimal* (30 frame al secondo), *Unlimited*
Modalità debug	*Settings*, *Run cartridges in RISC-V debug mode*
Esecuzione	**Pause**, **Resume**, **Step** (un'istruzione): tre pulsanti di trasporto disegnati come su un lettore video; velocità da 0.01x a 4x. Il pulsante con l'insetto accende e spegne il debug a livello di istruzione
Viste	**Init**, **Update**, **Draw**, **PC**: il listato di ciascuna funzione, o al program counter
Breakpoint	un clic sul margine del listato; **Resume** ignora una volta il breakpoint corrente
Stato	i 32 registri; 128 byte di memoria a un indirizzo scritto nel campo *Memory*
Telemetria	frame, istruzioni completate, frame al secondo, frame in ritardo, la maschera di ingresso
Errori	«instruction budget exhausted» (un frame che non finisce); «unsupported CSR» (un accesso a un registro di controllo)

B.8 Provare un programma sui tre host

Lo stesso file sorgente dà una cartuccia per ogni host. Questo paragrafo è la guida passo per passo che la presentazione del codice di ogni lezione ripete con il nome del proprio programma; qui il programma è `code/l05.S` del Capitolo 5, il cui prefisso di ingresso è `l05`. Si suppone che il repository del corso sia in `~/arc12` (`git clone https://github.com/riscv-prg32/computer-architecture-with-prg32 ~/arc12`), e ogni comando va dato dalla cartella del repository di PRG32, in un terminale in cui ESP-IDF è attivo (Capitolo 1).

Tabella B.16. Come ogni host è coperto dai test automatici del materiale del corso.

Host	Che cosa fanno i test
PRG32-QT	ogni programma è compilato, eseguito sull'esecutore senza interfaccia con ingressi programmati, e i pixel del suo frame sono controllati
QEMU	ogni programma è compilato per l'architettura `qemu` e caricato nell'immagine flash dell'emulatore; l'immagine non è controllata, perché l'emulatore ha bisogno della sua finestra
la scheda	ogni programma è compilato per l'architettura `esp32c6`; non è eseguito: sulla macchina di prova non c'è una scheda

B.8.1 Su PRG32-QT

1. Avviate PRG32-QT sul computer. Ascolta sulla porta 8080, come una scheda; la sua schermata *Setup* mostra l'indirizzo da usare se `localhost` non risponde.
2. Compilate la cartuccia.
3. Inviatela a PRG32-QT e avviatela.
4. Guardate lo schermo e premete i tasti: frecce o **W A S D** per il joystick, **Z** o **J** per A, **X** o **K** per B, **Enter** per Start, **Escape** per tornare a Setup.
5. Per guardare dentro, abilitate *Run cartridges in RISC-V debug mode* in *Settings* ed eseguite di nuovo la cartuccia.

```
python3 -m prg32 cartridge build ~/arc12/code/l05.S --portable \
    --entry-prefix l05 --name l05 --out l05.prg32
python3 -m prg32 esp32c6 upload-and-run l05.prg32 \
    --url http://localhost:8080 --slot cart0
```

Listato B.3. I passi 2 e 3 su PRG32-QT.

Senza la toolchain, una cartuccia compilata da qualcun altro si importa in PRG32-QT come file e si avvia con *Run Cartridge*.

B.8.2 Su QEMU

1. Una volta: compilate il firmware per l'emulatore. Richiede alcuni minuti.
2. Compilate la cartuccia per l'architettura `qemu`.
3. Con QEMU chiuso, caricate la cartuccia nell'immagine flash.
4. Avviate l'emulatore. Si apre sul menu *Setup*: scegliete *Run Cartridge*, poi `cart0`.
5. Scrivete nel *terminale* che ha lanciato QEMU, non nella finestra con l'immagine: frecce o **W A S D**, **Z** o **J** per A, **X** o **K** per B, **Enter** o **Space** per Select.

```
python3 -m prg32 qemu build
python3 -m prg32 cartridge build ~/arc12/code/l05.S --portable \
    --entry-prefix l05 --name l05 --architecture qemu --out l05-qemu.prg32
python3 -m prg32 qemu upload l05-qemu.prg32
python3 -m prg32 qemu run
```

Listato B.4. I passi da 1 a 4 su QEMU.

B.8.3 Sulla scheda

1. Una volta per scheda: collegatela con un cavo USB e scrivete il firmware.
2. La scheda crea la rete Wi-Fi `PRG32` (password `prg32game`) e risponde a `192.168.4.1`: collegate il computer a quella rete.
3. Compilate la cartuccia per l'architettura `esp32c6`.
4. Inviatela sul Wi-Fi e avviatela.
5. Usate il joystick e i pulsanti del kit.

```
python3 -m prg32 esp32c6 build-and-flash
python3 -m prg32 cartridge build ~/arc12/code/l05.S --portable \
```

```
    --entry-prefix l05 --name l05 --architecture esp32c6 \
    --out l05-esp32c6.prg32
python3 -m prg32 esp32c6 upload-and-run l05-esp32c6.prg32 \
    --url http://192.168.4.1 --slot cart0
```

Listato B.5. I passi 1, 3 e 4 sulla scheda.

B.8.4 I programmi delle lezioni

La Tabella B.17 elenca i programmi della cartella `code`. I capitoli tutorial non vi hanno un programma proprio: il loro programma è Verm1, nella cartella `verm1` (Appendice D).

Tabella B.17. Il programma di ogni lezione, e il suo prefisso di ingresso.

Cap.	File	Mostra	Cap.	File	Mostra
0	`l00.S`	un primo programma	11	`l11.S`	un multiplexer in software
1	`l01.S`	un'immagine di prova	13	`l13.S`	una macchina a stati
2	`l02.S` (`first`)	caricare, modificare, memorizzare	14	`l14.S`	seriale e parallelo
3	`l03.S`	il programma memorizzato	15	`l15.S`	byte e parole
4	`l04.S`	un array	16	`l16.S`	ciò che sopravvive a un riavvio
5	`l05.S`	il complemento a due	18	`l18.S`	caricare, calcolare, memorizzare
6	`l06.S`	la virgola fissa	19	`l19.S`	le istruzioni come numeri
8	`l08.S`	maschere e De Morgan	20	`l20.S`	un pixel
9	`l09.S`	il sommatore completo	21	`l21.S`	il polling del joystick
10	`l10.S`	un pulsante, un bit	24	`l24.S`	il ciclo sulla mappa

B.9 Il Cartridge Store

Lo Store è un servizio che cataloga, rivede e distribuisce cartucce; l'istanza pubblica è a https://store.prg32.uniparthenope.it. Una proposta è un *bundle*: un file ZIP che contiene un manifesto, un'icona e una cartuccia per architettura (Capitolo 23).

Il file `metadata.json` allegato a ogni cartuccia è il manifesto senza `assets` e `architectures`. Il *colophon* (`colophon.json`, formato `prg32-colophon-1.0`) contiene i crediti: `title`, `subtitle`, `version`, `release_date`, `developer`, `authors` con il loro `role`, `license`, `copyright`, `acknowledgements`, `controls` (coppie `input` e `action`) e `start_prompt`.

Il percorso di una proposta. Un bundle sottoposto con un token personale è *in attesa* finché un redattore non lo ha rivisto; quando è approvato è *pubblicato* per le architetture dichiarate. Lo Store conserva anche le tabelle dei punteggi, che una cartuccia raggiunge attraverso le funzioni `prg32_score_*`.

Tabella B.18. I campi di `manifest.json` (formato `prg32-metadata-1.0`).

Campo	Contenuto
`abi`	il formato dei metadati: `prg32-metadata-1.0`
`id`	un identificatore unico e permanente, per consuetudine un nome di dominio rovesciato
`title`	il nome che il giocatore legge
`version`	*major.minor.patch*; una versione pubblicata non viene mai sostituita
`summary`, `description`	una riga per lo scaffale; un paragrafo
`authors`	un elenco di oggetti con un `name`
`license`	la licenza, per esempio `MIT`
`tags`	parole con cui la cartuccia viene trovata
`assets`	i file del bundle che non sono cartucce: `icon`
`architectures`	un elenco di coppie `id` (`esp32c6`, `qemu`) e `file`

Attenzione

Il token dello Store è una credenziale personale. Si tiene in una variabile d'ambiente della shell (`PRG32_STORE_TOKEN`), mai in un file, in un repository o in una schermata.

B.10 Quando qualcosa non funziona

I sintomi incontrati più spesso durante l'installazione sono nella Tabella 1.5. Altri tre riguardano i programmi.

Il compilatore di cartucce dice che manca un punto di ingresso.
Il prefisso passato a `--entry-prefix` non è quello del sorgente, o manca una riga `.global`.

Il gioco si blocca, o torna al menu.
Una funzione non ritorna: un ciclo che non finisce mai, o un `ra` che non è stato salvato prima di una chiamata e ripristinato dopo.

Un valore cambia «da solo».
È stato tenuto in un registro `t` o `a` attraverso una chiamata, o `sp` non è stato ripristinato: controllate il prologo e l'epilogo, istruzione per istruzione, nel debugger.

APPENDICE C

Riferimento di Git e GitHub

La piattaforma di questo libro, gli strumenti, il manuale di accompagnamento e il libro stesso vivono su GitHub, e lì dovrebbe vivere anche il vostro progetto. Questa appendice è un riferimento di Git, il programma che registra la storia di un progetto, e di GitHub, il servizio che lo ospita: i concetti, i comandi di ogni giorno, il flusso di lavoro con cui si contribuisce a PRG32, e il modo in cui si conserva e si rilascia un progetto come Verm1. Non presuppone alcuna esperienza. Le fonti autorevoli sono Chacon e Straub (2014) e la documentazione di GitHub (GitHub, 2026).

C.1 Perché il controllo di versione

Il Capitolo 7 ha dato un'istruzione che ogni tutorial ha ripetuto: *tenete una copia di ogni parte finita*. Una cartella piena di file chiamati `verm1_part2_ok.S` e `verm1_final_really.S` è la forma artigianale del controllo di versione. Git fa la stessa cosa come si deve: ricorda ogni stato registrato dei file, dice esattamente che cosa è cambiato tra due stati, e ritorna a uno qualsiasi di essi.

Concetto

Git registra istantanee di un progetto, chiamate *commit*; ciascuna è uno stato etichettato al quale si può tornare. **GitHub** ospita in linea un repository Git, così che il lavoro ha una copia di sicurezza, può essere condiviso e può essere rivisto. Insieme trasformano «credo di aver rotto qualcosa» in «fammi vedere esattamente che cosa ho cambiato».

C.2 Concetti

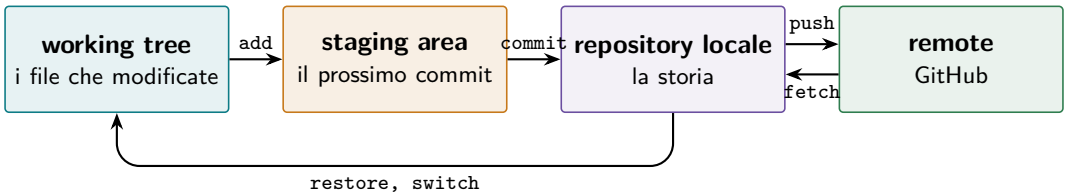


Figura C.1. I quattro luoghi in cui può trovarsi una modifica, e i comandi che la spostano.

Tabella C.1. Il vocabolario di Git e GitHub.

Termine	Significato
repository	un progetto con tutta la sua storia: la contiene la cartella `.git` dentro il progetto
working tree	i file come sono ora nella cartella
staging area (index)	le modifiche scelte per il prossimo commit
commit	un'istantanea registrata, con un autore, una data, un messaggio e un identificatore unico (un hash di 40 cifre esadecimali, di solito abbreviato a 7)
branch	un nome mobile per una linea di commit; quello predefinito è `main`
`HEAD`	il commit su cui è basato il working tree
tag	un nome fisso per un commit, tipicamente una versione: `v1.0.0`
remote	una copia del repository altrove, tipicamente su GitHub; il nome predefinito è `origin`
clone	una copia locale completa di un repository remoto
fork	una copia del repository di qualcun altro sotto il proprio account GitHub
push / fetch / pull	inviare commit a un remote / riceverli / riceverli e fonderli
merge	unire le storie di due branch
pull request (PR)	una proposta, su GitHub, di fondere un branch in un altro, con una discussione e una revisione
issue	una segnalazione o un compito associato a un repository su GitHub
release	un tag pubblicato su GitHub con note e, opzionalmente, file da scaricare

Una modifica viaggia attraverso quattro luoghi (Figura C.1). Viene fatta nel working tree; `git add` la sceglie per il prossimo commit; `git commit` la registra nel repository locale; `git push` la invia al remote. Niente lascia il computer prima del push.

C.3 Installare e configurare

Git si installa con il gestore di pacchetti del sistema (`brew install git` su macOS, `sudo apt install git` su Debian e Ubuntu) oppure, su Windows, con *Git for Windows* da https://git-scm.com; è già stato installato nel Capitolo 1. Si configura una volta per computer (Listato C.1).

```
git --version
git config --global user.name  "Your Name"
git config --global user.email "your.address@studenti.uniparthenope.it"
git config --global init.defaultBranch main
git config --list
```

Listato C.1. Configurare Git, una volta.

Un account GitHub si crea su https://github.com. Per far parlare Git con GitHub senza scrivere una password a ogni push, il modo più semplice è il client ufficiale a riga di comando, `gh`, il cui comando `gh auth login` apre un browser e memorizza la credenziale nel portachiavi del sistema; l'alternativa è una chiave SSH (`ssh-keygen -t ed25519`), la cui metà *pubblica* si incolla nelle impostazioni dell'account.

Attenzione

Una chiave privata, una password e un token non si mettono mai in un commit, non si incollano in un file del progetto e non si mostrano in una schermata. Questo vale per il token del Cartridge Store esattamente come per le credenziali di GitHub.

C.4 Ottenere un repository

Ci sono due modi di ottenere un repository: copiarne uno esistente, o trasformare una cartella in un repository (Listato C.2).

```
# 1. clone an existing repository: the platform, the debugger, the manual
git clone https://github.com/riscv-prg32/PRG32
git clone https://github.com/riscv-prg32/PRG32-QT
git clone https://github.com/riscv-prg32/Playing-with-RISC-V_PRG32_textbook

# 2. start a new repository in the folder of your game
cd ~/verm1
git init
git add verm1.S
git commit -m "Verm1 part 1: a head that crosses the world"
```

Listato C.2. Clonare la piattaforma; avviare un proprio progetto.

Per aggiornare un clone rispetto al remote: `git pull`.

C.5 Il ciclo di ogni giorno

Tre comandi coprono la maggior parte dell'uso quotidiano: `status`, per vedere che cosa è cambiato; `add`, per scegliere che cosa tenere; `commit`, per registrarlo con un messaggio (Listato C.3).

```
git status                      # what has changed?
git diff                        # exactly which lines?
git add verm1.S                 # choose the changes to record
git diff --staged               # what is about to be recorded?
git commit -m "Part 2: steering, with the U-turn rule as one XOR"
git log --oneline               # the history, one line per commit
```

Listato C.3. Ispezionare, preparare, registrare.

Un commit dopo ogni *Eseguite*

Il metodo di questo libro è un passo eseguibile alla volta. Fate un commit a ogni punto di controllo in cui il programma funziona e si comporta come dichiarato: allora ogni rottura è, nel peggiore dei casi, il lavoro di un passo da ispezionare o da annullare.

Un messaggio di commit è una nota per sé stessi nel futuro. La sua prima riga dice all'imperativo che cosa fa il commit, in una cinquantina di caratteri: «Add the edge detector for

button A», non «stuff» o «fix». Se serve altro, segue una riga vuota e poi una spiegazione del *perché*.

C.5.1 Che cosa non registrare: `.gitignore`

I file generati non appartengono a un repository: possono essere ricostruiti dai sorgenti, e cambiano a ogni compilazione. Un file di nome `.gitignore` elenca i modelli che Git deve ignorare (Listato C.4).

```
# build products
build/
*.o
*.elf
*.prg32
*.zip

# editors and operating systems
.vscode/
.DS_Store
Thumbs.db

# secrets: never
.env
*.token
```

Listato C.4. Un `.gitignore` per un progetto di cartuccia.

Se tenere sotto controllo di versione la cartuccia compilata è una questione di scelta; la risposta usuale è no, perché il posto giusto per essa è una release (Paragrafo C.10).

C.6 Guardare la storia

```
git log --oneline --graph --all     # all the branches, as a graph
git log -p verm1.S                  # the changes made to one file
git show 3f2a9c1                    # one commit: message and changes
git diff v1.0.0 HEAD                # from a version to now
git blame verm1.S                   # who wrote each line, and when
git bisect start                    # find the commit that broke something
```

Listato C.5. Leggere la storia.

`git bisect` merita una parola. Dato un commit in cui il programma funzionava e uno in cui non funziona, propone il commit a metà strada, chiede se quello è buono o cattivo, e ripete: una ricerca binaria nella storia, che trova il commit colpevole tra mille in dieci passi.

Tabella C.2. Come annullare, dal meno al più drastico.

Situazione	Comando
scartare le modifiche di un file dall'ultimo commit	`git restore verm1.S`
togliere un file dalla staging area, conservandone le modifiche	`git restore --staged verm1.S`
correggere il messaggio, o aggiungere un file dimenticato, dell'ultimo commit non ancora inviato	`git commit --amend`
riportare un file a com'era in un commit più vecchio	`git restore --source 3f2a9c1 verm1.S`
cancellare l'effetto di un commit già pubblicato, con un nuovo commit	`git revert 3f2a9c1`
mettere da parte il lavoro per un momento, e riprenderlo	`git stash` ; `git stash pop`
guardare un vecchio stato senza cambiare niente	`git switch --detach 3f2a9c1`

C.7 Annullare

Attenzione

I comandi che riscrivono la storia, come `git reset --hard` e `git push --force`, distruggono lavoro, il proprio e quello degli altri. La storia che è stata inviata si corregge con `git revert`, che aggiunge un commit, e mai riscrivendola.

C.8 Branch

Un *branch* (ramo) è una linea di lavoro indipendente. Prima di una modifica rischiosa, per esempio il disegno incrementale del Capitolo 22, si crea un branch: se l'esperimento fallisce il branch si scarta e la linea principale resta intatta; se riesce viene fuso (Listato C.6).

```
git switch -c feature/pause-state   # create a branch and move to it
# ... edit, build, run, commit, as many times as needed ...
git switch main                     # back to the main line
git merge feature/pause-state       # bring the work in
git branch -d feature/pause-state   # the branch is no longer needed

git branch                          # list the branches
git switch -                        # go back to the previous branch
```

Listato C.6. Lavorare su un branch.

Conflitti. Se due branch hanno modificato le stesse righe, il merge si ferma e Git segna il punto nel file:

```
<<<<<<< HEAD
.equ SPEED_MIN, 32
=======
.equ SPEED_MIN, 40
>>>>>>> feature/levels
```

Il conflitto si risolve modificando il file finché dice ciò che si vuole, togliendo le tre righe di marcatura, e poi eseguendo `git add` e `git commit`. `git merge --abort` rinuncia e ritorna allo stato precedente al merge.

C.9 Lavorare con GitHub

C.9.1 Pubblicare un repository

```
# with the GitHub client: create the repository and push, in one step
gh repo create verm1-yourname --public --source . --push

# or: create an empty repository on github.com, then
git remote add origin https://github.com/yourname/verm1-yourname.git
git push -u origin main

# afterwards
git push            # send new commits
git pull            # receive the commits made elsewhere
git remote -v       # where does this repository point?
```

Listato C.7. Pubblicare un repository locale su GitHub.

C.9.2 I file di un buon repository

Il repository di PRG32 è un modello delle convenzioni (Tabella C.3): i file che orientano chi arriva e rendono un progetto facile da valutare.

Tabella C.3. I file che un repository dovrebbe contenere.

File	Contenuto
`README.md`	che cos'è il progetto, una schermata, come compilarlo ed eseguirlo, i comandi
`LICENSE`	che cosa possono farne gli altri: MIT per PRG32 e per i programmi di questo libro
`.gitignore`	che cosa non va registrato
`CITATION.cff`	come citare il progetto in una relazione o in una tesi
`CONTRIBUTING.md`	come proporre una modifica
`CHANGELOG.md`	che cosa è cambiato da una versione alla successiva
`.github/workflows/`	i controlli automatici eseguiti da GitHub Actions

```
# Verm1 Deluxe

A hungry worm on a 40 by 24 grid: a tribute to Snake, written in RISC-V
assembly for the PRG32 platform. My variant adds walls and a pause.

![The game](screenshot.png)

## Build and run

    python3 -m prg32 cartridge build verm1.S --portable \
        --entry-prefix verm1 --name verm1 --out verm1.prg32
```

```
    python3 -m prg32 esp32c6 upload-and-run verm1.prg32 \
        --url http://localhost:8080 --slot cart0

## Controls

Joystick: steer. A: start. B: pause.

## Credits and licence

Written by Your Name for ARC12, University of Naples "Parthenope".
MIT License: see LICENSE.
```

Listato C.8. Un `README.md` per una variante di Verm1.

I file con l'estensione `.md` sono scritti in Markdown: una riga che comincia con `#` è un titolo, `**bold**` è grassetto, una riga rientrata di quattro spazi è codice, e `[testo](indirizzo)` è un collegamento.

C.9.3 Fork e pull request

Non si può fare push su un repository di cui non si è manutentori. Il modo di proporre una modifica al progetto di qualcun altro è la *pull request* (Figura C.2, Listato C.9).



Figura C.2. Il percorso di un contributo.

```
gh repo fork riscv-prg32/PRG32 --clone       # fork, and clone the fork
cd PRG32
git switch -c docs/cartridge-workflow-notes  # a branch from main

# ... make the smallest change that proves the idea; run the checks ...

git add docs/
git commit -m "Clarify the cartridge workflow for first-time users"
git push -u origin docs/cartridge-workflow-notes
gh pr create --fill                          # open the pull request

# keep the fork up to date with the original
git remote add upstream https://github.com/riscv-prg32/PRG32.git
git fetch upstream
git switch main
git merge upstream/main
```

Listato C.9. Un contributo a PRG32, dal fork alla pull request.

Le regole del repository di PRG32, nel suo `CONTRIBUTING.md`, sono scritte prima di tutto per gli studenti: scegliere uno scopo chiaro; creare un branch da `main`; fare la modifica più piccola che dimostri l'idea; aggiornare la documentazione quando cambiano comportamento, comandi, percorsi o interfacce; eseguire i controlli; e aprire una pull request che dica qual

era il *contesto*, quale *metodo* è stato seguito e come la modifica è stata *validata*, su QEMU o sull'hardware. I nomi dei branch dicono il tipo di lavoro: `lab/...`, `game/...`, `docs/...`

La revisione. Una pull request mostra le modifiche riga per riga, e chiunque può commentare una riga. Un revisore approva, o chiede modifiche; l'autore risponde inviando altri commit sullo stesso branch, che aggiornano la pull request. La lista di controllo del Capitolo 23, usata da un collega sul vostro bundle, è una revisione.

C.9.4 Issue

Una *issue* segnala un difetto o propone un compito. Una buona segnalazione di un difetto contiene ciò che è stato fatto (il comando esatto), ciò che ci si aspettava, ciò che è successo (l'*intero* messaggio, come testo), e l'ambiente: il sistema operativo, la versione degli strumenti, l'host. Un commit o una pull request il cui messaggio contiene `Fixes #12` chiude la issue 12 quando viene fuso.

C.9.5 GitHub Actions: controlli che girano da soli

Un file in `.github/workflows` descrive un lavoro che GitHub esegue a ogni push o pull request, su una macchina pulita: tipicamente, compilare il progetto ed eseguirne i test. Il workflow del repository di PRG32 compila le cartucce incluse con lo stesso strumento a riga di comando che si usa in locale, per l'ESP32-C6 e per QEMU, e le espone come file da scaricare. Un segno verde accanto a un commit significa che i controlli sono passati; una croce rossa significa che qualcosa che funzionava sul computer dell'autore non funziona su uno pulito, che è esattamente ciò che si vuole sapere prima di un merge.

```yaml
name: build
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build and test
        run: ./scripts/build-and-test.sh
```

Listato C.10. Lo schema di un file di workflow (YAML). Il passo di compilazione dipende dal progetto: per una cartuccia richiede la toolchain ESP-IDF.

C.10 Versioni, tag e release

Il Capitolo 23 ha dato a Verm1 una versione, 1.0.0, con la regola che una versione pubblicata non viene mai sostituita. La numerazione si chiama *versionamento semantico*: *major.minor.patch*, dove il patch cresce per una correzione, il minor per una nuova funzione che non rompe niente, e il major per una modifica non compatibile. In Git una versione è un *tag*, e su GitHub un tag diventa una *release*, alla quale si possono allegare file (Listato C.11).

```
git tag -a v1.0.0 -m "Verm1 1.0.0: first release"
git push origin v1.0.0

# publish a release on GitHub and attach the bundle of the Store
```

```
gh release create v1.0.0 verm1-1.0.0.zip \
    --title "Verm1 1.0.0" --notes "First release: the finished game."

git tag                  # list the versions
git switch --detach v1.0.0   # look at the sources of a version
```

Listato C.11. Etichettare la versione 1.0.0 e rilasciare il bundle.

Il tag garantisce che i sorgenti da cui è stato costruito un bundle pubblicato si possano sempre ritrovare: lega il file che i giocatori installano alle righe che l'autore ha scritto.

C.11 Questo libro e Overleaf

I sorgenti di questo libro sono una cartella di file LaTeX che si compila su Overleaf (caricate la cartella come file ZIP, o importate il repository; il documento principale è `main.tex` e il compilatore è pdfLaTeX). Overleaf può essere collegato a un repository GitHub, con il quale scambia commit; e ogni progetto Overleaf è esso stesso un repository Git che si può clonare, così che il testo può essere modificato in un browser e versionato allo stesso modo di un programma. Le slide del corso sono generate da sorgenti conservati alla stessa maniera, e le correzioni sono benvenute come pull request.

C.12 Citare il software

Il software usato in una relazione, in una tesi o in un articolo si cita, come qualsiasi altra fonte. Un repository che contiene un file `CITATION.cff` mostra su GitHub un pulsante, *Cite this repository*, che produce il riferimento nei formati usuali. Il repository di PRG32 ne ha uno, e la sua voce nella bibliografia di questo libro (Montella et al., 2026a) è il modello da seguire.

C.13 Riepilogo dei comandi

Tabella C.4. I comandi di Git e del client GitHub usati in questa appendice.

Comando	Effetto
`git config --global user.name "..."`	impostare il nome dell'autore
`git init`	fare della cartella corrente un repository
`git clone URL`	copiare un repository remoto
`git status`	che cosa è cambiato
`git diff` ; `git diff --staged`	le modifiche non ancora preparate; quelle preparate
`git add FILE` ; `git add -p`	preparare un file; preparare una parte delle modifiche
`git commit -m "message"`	registrare un'istantanea
`git log --oneline --graph`	la storia
`git show COMMIT`	un commit
`git restore FILE`	scartare le modifiche di un file
`git restore --staged FILE`	togliere un file dalla staging area
`git revert COMMIT`	annullare un commit con un nuovo commit
`git stash` ; `git stash pop`	mettere da parte il lavoro; riprenderlo

Comando	Effetto
`git switch -c NAME`	creare un branch e spostarvisi
`git switch NAME`	spostarsi su un branch
`git merge NAME`	fondere un branch in quello corrente
`git branch` ; `git branch -d NAME`	elencare i branch; cancellarne uno
`git remote -v` ; `git remote add NAME URL`	elencare i remote; aggiungerne uno
`git fetch` ; `git pull`	ricevere commit; ricevere e fondere
`git push` ; `git push -u origin NAME`	inviare commit; il primo push di un branch
`git tag -a v1.0.0 -m "..."`	dare un nome a una versione
`git bisect start` / `good` / `bad`	trovare il commit che ha introdotto un difetto
`gh auth login`	accedere a GitHub dal terminale
`gh repo create NAME --source . --push`	creare un repository su GitHub dalla cartella corrente
`gh repo fork OWNER/REPO --clone`	fare il fork di un repository e clonare il fork
`gh pr create` ; `gh pr status` ; `gh pr checkout N`	aprire una pull request; vederne lo stato; provarne una in locale
`gh issue create` ; `gh issue list`	aprire una issue; elencarle
`gh release create TAG FILES`	pubblicare una release

Punto di controllo

Ora dovreste saper clonare la piattaforma, registrare il vostro lavoro passo per passo con messaggi significativi, sperimentare su un branch, annullare una modifica indesiderata, pubblicare il vostro repository, proporre una modifica con una pull request e rilasciare una versione del vostro gioco. Queste operazioni bastano per un primo anno di corso, e per un primo contributo a un progetto aperto.

APPENDICE D

Verm1: i sorgenti completi

Questa appendice stampa per intero i quattro programmi di riferimento del progetto e i file del bundle dello Store. L'implementazione di riferimento di Verm1 è un contributo di Giuseppe Coviello; i capitoli tutorial sono costruiti su di essa. Sono i file che i test automatici del materiale del corso compilano, eseguono sull'esecutore senza interfaccia di PRG32-QT e giocano con un giocatore programmato; i listati dei capitoli tutorial sono regioni di questi stessi file. Se il vostro programma si rifiuta di funzionare all'inizio di un tutorial, ripartite dal file della parte precedente (Tabella D.1).

Tabella D.1. I programmi di riferimento.

File	Capitolo	Righe	Il gioco a questo punto
`verm1_part1.S`	7	78	una griglia, una testa che si muove da sola, un mondo senza bordi
`verm1_part2.S`	12	231	la guida, la regola dell'inversione a U, il cibo, un punteggio a tacche, un suono
`verm1_part3.S`	17	435	un corpo che cresce, le collisioni, tre stati, un punteggio migliore
`verm1_part4.S`	22	565	cifre, livelli, due suoni, disegno incrementale: il gioco finito

Ogni parte si compila e si esegue con gli stessi due comandi:

```
python3 -m prg32 cartridge build verm1_part4.S --portable \
    --entry-prefix verm1 --name verm1 --out verm1.prg32
python3 -m prg32 esp32c6 upload-and-run verm1.prg32 \
    --url http://localhost:8080 --slot cart0
```

D.1 Parte 1: una testa che attraversa il mondo

```
# Verm1 (say "vermone"), a tribute to Snake - part 1: a head that crosses the
    world
# ARC12 Computer Architecture, A.Y. 2026/27 - RISC-V assembly for PRG32
# Reference implementation contributed by Giuseppe Coviello
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
```

```
.equ GREEN, 0x07e0
.equ CELL,  8               # a cell is 8 x 8 pixels
.equ COLS,  40              # 320 / 8 cells across
.equ ROWS,  24              # 192 / 8 cells down
.equ TOP,   8               # the top 8 pixels are the title line
.equ SPEED_Q8, 40           # 40/256 of a cell per frame (Q24.8)

.section .text
.global verm1_init
.global verm1_update
.global verm1_draw
verm1_init:
    la   t0, head_x
    li   t1, 5
    sw   t1, 0(t0)              # the head starts in column 5 ...
    la   t0, head_y
    li   t1, 12
    sw   t1, 0(t0)              # ... of row 12
    la   t0, step_q8
    sw   zero, 0(t0)            # no fraction of a step yet
    ret
verm1_update:
    la   t0, step_q8
    lw   t1, 0(t0)
    addi t1, t1, SPEED_Q8       # add a fraction of a cell
    li   t2, 256
    blt  t1, t2, 2f             # less than one whole cell: wait
    sub  t1, t1, t2             # one cell done: keep the remainder
    sw   t1, 0(t0)

    la   t0, head_x
    lw   t3, 0(t0)
    addi t3, t3, 1              # one cell to the right
    li   t4, COLS
    blt  t3, t4, 1f
    li   t3, 0                  # past the right edge: back to column 0
1:  sw   t3, 0(t0)
    ret
2:  sw   t1, 0(t0)
    ret
verm1_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 0                  # x
    li   a1, 0                  # y
    la   a2, title              # address of the string
    li   a3, WHITE
    li   a4, BLACK
    call prg32_gfx_text8
    la   t0, head_x
    lw   a0, 0(t0)
    slli a0, a0, 3              # x = column * 8
    la   t0, head_y
    lw   a1, 0(t0)
```

```
    slli a1, a1, 3
    addi a1, a1, TOP            # y = row * 8 + 8
    li   a2, CELL               # width
    li   a3, CELL               # height
    li   a4, GREEN
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
.section .rodata
title:   .asciz "VERM1"

.section .data
head_x:  .word 5                # column of the head, 0 .. COLS-1
head_y:  .word 12               # row of the head, 0 .. ROWS-1
step_q8: .word 0                # fraction of a step, in 1/256
```

Listato D.1. `verm1_part1.S`.

D.2 Parte 2: la guida, il cibo e un punteggio

```
# Verm1 (say "vermone"), a tribute to Snake - part 2: steering and food
# ARC12 Computer Architecture, A.Y. 2026/27 - RISC-V assembly for PRG32
# Reference implementation contributed by Giuseppe Coviello
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
.equ GREEN, 0x07e0
.equ RED,   0xf800
.equ CELL,  8               # a cell is 8 x 8 pixels
.equ COLS,  40
.equ ROWS,  24
.equ TOP,   8
.equ SPEED_Q8, 40           # 40/256 of a cell per frame
.equ BTN_LEFT,  0x01        # one bit per button
.equ BTN_RIGHT, 0x02
.equ BTN_UP,    0x04
.equ BTN_DOWN,  0x08
.equ DIR_RIGHT, 0           # a direction is a 2-bit code:
.equ DIR_DOWN,  1           #   bit 0 = vertical
.equ DIR_LEFT,  2           #   bit 1 = backwards
.equ DIR_UP,    3

.section .text
.global verm1_init
.global verm1_update
.global verm1_draw
verm1_init:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, head_x
    li   t1, 5
    sw   t1, 0(t0)
    la   t0, head_y
```

```
    li   t1, 12
    sw   t1, 0(t0)
    la   t0, dir
    sw   zero, 0(t0)            # moving right
    la   t0, want
    sw   zero, 0(t0)
    la   t0, step_q8
    sw   zero, 0(t0)
    la   t0, score
    sw   zero, 0(t0)
    call place_food
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
verm1_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    call prg32_input_read       # a0 = one bit per button
    la   t0, dir
    lw   t1, 0(t0)              # t1 = direction of the last step
    la   t0, want
    lw   t2, 0(t0)              # t2 = wanted direction, so far
    andi t3, a0, BTN_RIGHT
    beqz t3, 1f
    li   t2, DIR_RIGHT
1:  andi t3, a0, BTN_DOWN
    beqz t3, 2f
    li   t2, DIR_DOWN
2:  andi t3, a0, BTN_LEFT
    beqz t3, 3f
    li   t2, DIR_LEFT
3:  andi t3, a0, BTN_UP
    beqz t3, 4f
    li   t2, DIR_UP
4:  xor  t3, t2, t1             # opposite directions differ in bit 1 only
    li   t4, 2
    beq  t3, t4, 5f             # a U-turn: ignore the request
    sw   t2, 0(t0)              # remember it for the next step
5:
    la   t0, step_q8
    lw   t1, 0(t0)
    addi t1, t1, SPEED_Q8       # add a fraction of a cell
    li   t2, 256
    blt  t1, t2, 6f             # less than one whole cell: wait
    sub  t1, t1, t2
    sw   t1, 0(t0)
    call step                   # one whole cell: move
    j    7f
6:  sw   t1, 0(t0)
7:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret
step:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, want
```

```
90      lw   t1, 0(t0)
91      la   t0, dir
92      sw   t1, 0(t0)              # the wanted direction is now the direction
93      slli t1, t1, 2              # table offset = direction * 4 bytes
94      la   t0, dx
95      add  t0, t0, t1
96      lw   t2, 0(t0)              # t2 = dx[direction]: -1, 0 or +1
97      la   t0, dy
98      add  t0, t0, t1
99      lw   t3, 0(t0)              # t3 = dy[direction]
100     la   t0, head_x
101     lw   t4, 0(t0)
102     add  t4, t4, t2
103     bgez t4, 1f
104     li   t4, COLS-1             # off the left edge: wrap
105 1:  li   t5, COLS
106     blt  t4, t5, 2f
107     li   t4, 0                  # off the right edge: wrap
108 2:  sw   t4, 0(t0)              # t4 = new column
109     la   t0, head_y
110     lw   t6, 0(t0)
111     add  t6, t6, t3
112     bgez t6, 3f
113     li   t6, ROWS-1
114 3:  li   t5, ROWS
115     blt  t6, t5, 4f
116     li   t6, 0
117 4:  sw   t6, 0(t0)              # t6 = new row
118     la   t0, food_x
119     lw   t1, 0(t0)
120     xor  t1, t1, t4             # 0 if same column
121     la   t0, food_y
122     lw   t2, 0(t0)
123     xor  t2, t2, t6             # 0 if same row
124     or   t1, t1, t2             # 0 only if both are equal
125     bnez t1, 5f                 # not on the food
126     la   t0, score
127     lw   t1, 0(t0)
128     addi t1, t1, 1
129     sw   t1, 0(t0)
130     call place_food
131     call eat_sound
132 5:  lw   ra, 12(sp)
133     addi sp, sp, 16
134     ret
135 place_food:
136     addi sp, sp, -16
137     sw   ra, 12(sp)
138     li   a0, 0
139     li   a1, COLS-1
140     call prg32_random_number    # a0 = a column, 0 .. 39
141     la   t0, food_x
142     sw   a0, 0(t0)
143     li   a0, 0
144     li   a1, ROWS-1
145     call prg32_random_number    # a0 = a row, 0 .. 23
```

```
    la   t0, food_y
    sw   a0, 0(t0)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
eat_sound:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0                  # channel
    li   a1, 0                  # instrument
    li   a2, 72                 # note: C5
    li   a3, 255                # volume
    li   a4, 60                 # milliseconds
    call prg32_audio_note
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
verm1_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 0
    li   a1, 0
    la   a2, title
    li   a3, WHITE
    li   a4, BLACK
    call prg32_gfx_text8
    li   s0, 0                  # s0 = i: one mark per point
1:  la   t0, score
    lw   t1, 0(t0)
    bge  s0, t1, 2f
    slli a0, s0, 2
    addi a0, a0, 56             # x = 56 + 4 * i
    li   a1, 1
    li   a2, 3
    li   a3, 6
    li   a4, WHITE
    call prg32_gfx_rect
    addi s0, s0, 1
    j    1b
2:
    la   t0, food_x
    lw   a0, 0(t0)
    la   t0, food_y
    lw   a1, 0(t0)
    li   a2, RED
    call draw_cell
    la   t0, head_x
    lw   a0, 0(t0)
    la   t0, head_y
    lw   a1, 0(t0)
    li   a2, GREEN
    call draw_cell
    lw   s0, 8(sp)
```

```
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
draw_cell:                      # a0 = column, a1 = row, a2 = colour
    addi sp, sp, -16
    sw   ra, 12(sp)
    slli a0, a0, 3              # x = column * 8
    slli a1, a1, 3
    addi a1, a1, TOP            # y = row * 8 + 8
    mv   a4, a2                 # the colour is the fifth argument
    li   a2, CELL
    li   a3, CELL
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
.section .rodata
title:   .asciz "VERM1"
dx:      .word  1, 0, -1,  0    # right, down, left, up
dy:      .word  0, 1,  0, -1

.section .data
head_x:  .word 5
head_y:  .word 12
dir:     .word 0                # direction of the last step
want:    .word 0                # direction asked by the player
step_q8: .word 0
score:   .word 0
food_x:  .word 20
food_y:  .word 12
```

Listato D.2. `verm1_part2.S`.

D.3 Parte 3: un corpo, regole e stati

```
# Verm1 (say "vermone"), a tribute to Snake - part 3: a body, rules and states
# ARC12 Computer Architecture, A.Y. 2026/27 - RISC-V assembly for PRG32
# Reference implementation contributed by Giuseppe Coviello
.option norelax
.equ BLACK, 0x0000          # RGB565 colours
.equ WHITE, 0xffff
.equ GREEN, 0x07e0          # the head
.equ BODY,  0x03e0          # the body: a darker green
.equ RED,   0xf800          # the food
.equ CELL,  8               # a cell is 8 x 8 pixels
.equ COLS,  40
.equ ROWS,  24
.equ TOP,   8
.equ SPEED_Q8, 40           # 40/256 of a cell per frame
.equ BTN_LEFT,  0x01
.equ BTN_RIGHT, 0x02
.equ BTN_UP,    0x04
.equ BTN_DOWN,  0x08
.equ BTN_A,     0x10
```

```
.equ DIR_RIGHT, 0           # bit 0 = vertical, bit 1 = backwards
.equ DIR_DOWN,  1
.equ DIR_LEFT,  2
.equ DIR_UP,    3
.equ TITLE, 0               # the three states of the game
.equ PLAY,  1
.equ OVER,  2
.section .text
.global verm1_init
.global verm1_update
.global verm1_draw

verm1_init:
    la   t0, state
    sw   zero, 0(t0)            # start on the title screen
    la   t0, prev_btn
    sw   zero, 0(t0)
    ret
verm1_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    sw   s1, 4(sp)
    call prg32_input_read
    mv   s0, a0                 # s0 = buttons held in this frame
    la   t0, prev_btn
    lw   t1, 0(t0)              # t1 = buttons held one frame ago
    sw   s0, 0(t0)
    not  t1, t1
    and  s1, s0, t1             # s1 = buttons pressed just now
    la   t0, state
    lw   t1, 0(t0)              # t1 = present state
    li   t2, PLAY
    beq  t1, t2, u_play
    andi t3, s1, BTN_A          # TITLE or OVER: wait for a new press of A
    beqz t3, u_done
    li   t2, TITLE
    beq  t1, t2, u_start
    sw   t2, 0(t0)              # OVER --A--> TITLE
    j    u_done
u_start:
    call new_game               # TITLE --A--> PLAY
    j    u_done
u_play:
    mv   a0, s0
    call play_update
u_done:
    lw   s1, 4(sp)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
new_game:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, grid
```

```
    li   t1, COLS*ROWS
1:  sb   zero, 0(t0)            # empty the grid, one byte per cell
    addi t0, t0, 1
    addi t1, t1, -1
    bnez t1, 1b
    li   t1, 0                  # i = 0, 1, 2: three cells in row 12
2:  la   t0, body_x
    add  t0, t0, t1
    addi t2, t1, 3
    sb   t2, 0(t0)              # columns 3, 4, 5
    la   t0, body_y
    add  t0, t0, t1
    li   t3, 12
    sb   t3, 0(t0)
    la   t0, grid+12*COLS
    add  t0, t0, t2
    li   t3, 1
    sb   t3, 0(t0)              # mark the cell as occupied
    addi t1, t1, 1
    li   t3, 3
    blt  t1, t3, 2b
    la   t0, head_i
    li   t1, 2
    sw   t1, 0(t0)              # the head is entry 2 of the ring
    la   t0, length
    li   t1, 3
    sw   t1, 0(t0)
    la   t0, dir
    sw   zero, 0(t0)
    la   t0, want
    sw   zero, 0(t0)
    la   t0, step_q8
    sw   zero, 0(t0)
    la   t0, score
    sw   zero, 0(t0)
    call place_food
    la   t0, state
    li   t1, PLAY
    sw   t1, 0(t0)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
play_update:                    # a0 = buttons held
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, dir
    lw   t1, 0(t0)              # t1 = direction of the last step
    la   t0, want
    lw   t2, 0(t0)              # t2 = wanted direction, so far
    andi t3, a0, BTN_RIGHT
    beqz t3, 1f
    li   t2, DIR_RIGHT
1:  andi t3, a0, BTN_DOWN
    beqz t3, 2f
    li   t2, DIR_DOWN
2:  andi t3, a0, BTN_LEFT
```

```
    beqz t3, 3f
    li   t2, DIR_LEFT
3:  andi t3, a0, BTN_UP
    beqz t3, 4f
    li   t2, DIR_UP
4:  xor  t3, t2, t1             # opposite directions differ in bit 1 only
    li   t4, 2
    beq  t3, t4, 5f             # a U-turn: ignore the request
    sw   t2, 0(t0)
5:
    la   t0, step_q8
    lw   t1, 0(t0)
    addi t1, t1, SPEED_Q8       # add a fraction of a cell
    li   t2, 256
    blt  t1, t2, 6f             # less than one whole cell: wait
    sub  t1, t1, t2
    sw   t1, 0(t0)
    call step                   # one whole cell: move
    j    7f
6:  sw   t1, 0(t0)
7:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret
step:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)              # s0 = column of the new head
    sw   s1, 4(sp)              # s1 = row of the new head
    sw   s2, 0(sp)              # s2 = 0 when the worm eats
    la   t0, want
    lw   t1, 0(t0)
    la   t0, dir
    sw   t1, 0(t0)              # the wanted direction is now the direction
    slli t1, t1, 2              # table offset = direction * 4 bytes
    la   t0, dx
    add  t0, t0, t1
    lw   t2, 0(t0)              # t2 = dx[direction]
    la   t0, dy
    add  t0, t0, t1
    lw   t3, 0(t0)              # t3 = dy[direction]
    la   t0, head_i
    lw   t5, 0(t0)              # t5 = ring index of the head
    la   t0, body_x
    add  t0, t0, t5
    lbu  s0, 0(t0)              # one byte: the column of the head
    add  s0, s0, t2
    bgez s0, 1f
    li   s0, COLS-1             # off the left edge: wrap
1:  li   t4, COLS
    blt  s0, t4, 2f
    li   s0, 0
2:  la   t0, body_y
    add  t0, t0, t5
    lbu  s1, 0(t0)
    add  s1, s1, t3
    bgez s1, 3f
```

```
    li   s1, ROWS-1
3:  li   t4, ROWS
    blt  s1, t4, 4f
    li   s1, 0
4:
    la   t0, food_x
    lw   t1, 0(t0)
    xor  t1, t1, s0
    la   t0, food_y
    lw   t2, 0(t0)
    xor  t2, t2, s1
    or   s2, t1, t2            # s2 = 0 if the new head is on the food
    la   t0, length
    lw   t1, 0(t0)
    bnez s2, 5f                # not eating: the tail moves on
    li   t2, 255
    bge  t1, t2, 5f            # the ring is full: no more growth
    addi t1, t1, 1
    sw   t1, 0(t0)             # eating: one cell longer, the tail stays
    j    6f
5:  la   t0, head_i
    lw   t2, 0(t0)
    sub  t2, t2, t1
    addi t2, t2, 1
    andi t2, t2, 255           # ring index of the tail
    la   t0, body_x
    add  t0, t0, t2
    lbu  t3, 0(t0)
    la   t0, body_y
    add  t0, t0, t2
    lbu  t4, 0(t0)
    li   t5, COLS
    mul  t4, t4, t5
    add  t4, t4, t3            # cell number of the tail = row * 40 + column
    la   t0, grid
    add  t0, t0, t4
    sb   zero, 0(t0)           # the tail cell is free again
6:  li   t5, COLS
    mul  t4, s1, t5
    add  t4, t4, s0            # cell number of the new head
    la   t0, grid
    add  t0, t0, t4
    lbu  t1, 0(t0)
    beqz t1, 7f                # a free cell: go on
    la   t0, state
    li   t1, OVER
    sw   t1, 0(t0)             # the worm bit itself: PLAY --> OVER
    la   t0, score
    lw   t1, 0(t0)
    la   t0, best
    lw   t2, 0(t0)
    ble  t1, t2, 9f
    sw   t1, 0(t0)             # a new best score
    j    9f
7:  li   t1, 1
    sb   t1, 0(t0)             # occupy the new cell
```

```
    la   t0, head_i
    lw   t1, 0(t0)
    addi t1, t1, 1
    andi t1, t1, 255            # the ring wraps after entry 255
    sw   t1, 0(t0)
    la   t0, body_x
    add  t0, t0, t1
    sb   s0, 0(t0)              # write the new head into the ring
    la   t0, body_y
    add  t0, t0, t1
    sb   s1, 0(t0)
    bnez s2, 9f
    la   t0, score
    lw   t1, 0(t0)
    addi t1, t1, 1
    sw   t1, 0(t0)
    call place_food
    call eat_sound
9:  lw   s2, 0(sp)
    lw   s1, 4(sp)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
place_food:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
1:  li   a0, 0
    li   a1, COLS-1
    call prg32_random_number
    mv   s0, a0                 # s0 = a column
    li   a0, 0
    li   a1, ROWS-1
    call prg32_random_number    # a0 = a row
    li   t1, COLS
    mul  t2, a0, t1
    add  t2, t2, s0
    la   t0, grid
    add  t0, t0, t2
    lbu  t3, 0(t0)
    bnez t3, 1b                 # on the worm: try again
    la   t0, food_x
    sw   s0, 0(t0)
    la   t0, food_y
    sw   a0, 0(t0)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
eat_sound:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, 0                  # channel
    li   a1, 0                  # instrument
    li   a2, 72                 # note: C5
```

```
    li   a3, 255                # volume
    li   a4, 60                 # milliseconds
    call prg32_audio_note
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
verm1_draw:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 0
    li   a1, 0
    la   a2, title
    call draw_text
    la   t0, state
    lw   t1, 0(t0)
    li   t2, TITLE
    bne  t1, t2, 1f
    li   a0, 132                # TITLE: only an invitation
    li   a1, 96
    la   a2, press_a
    call draw_text
    j    2f
1:  call draw_score             # PLAY and OVER: the whole scene
    call draw_worm
    la   t0, food_x
    lw   a0, 0(t0)
    la   t0, food_y
    lw   a1, 0(t0)
    li   a2, RED
    call draw_cell
    la   t0, state
    lw   t1, 0(t0)
    li   t2, OVER
    bne  t1, t2, 2f
    li   a0, 124
    li   a1, 96
    la   a2, game_over
    call draw_text
2:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret

draw_text:                      # a0 = x, a1 = y, a2 = address of the string
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a3, WHITE
    li   a4, BLACK
    call prg32_gfx_text8
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
draw_worm:
    addi sp, sp, -16
    sw   ra, 12(sp)
```

```
356     sw   s0, 8(sp)
357     li   s0, 0                  # s0 = i: segment 0 is the head
358 1:  la   t0, length
359     lw   t1, 0(t0)
360     bge  s0, t1, 3f
361     la   t0, head_i
362     lw   t1, 0(t0)
363     sub  t1, t1, s0
364     andi t1, t1, 255            # ring index of segment i
365     la   t0, body_x
366     add  t0, t0, t1
367     lbu  a0, 0(t0)
368     la   t0, body_y
369     add  t0, t0, t1
370     lbu  a1, 0(t0)
371     li   a2, BODY
372     bnez s0, 2f
373     li   a2, GREEN              # the head is brighter
374 2:  call draw_cell
375     addi s0, s0, 1
376     j    1b
377 3:  lw   s0, 8(sp)
378     lw   ra, 12(sp)
379     addi sp, sp, 16
380     ret
381 draw_score:
382     addi sp, sp, -16
383     sw   ra, 12(sp)
384     sw   s0, 8(sp)
385     li   s0, 0                  # s0 = i: one mark per point
386 1:  la   t0, score
387     lw   t1, 0(t0)
388     bge  s0, t1, 2f
389     slli a0, s0, 2
390     addi a0, a0, 56             # x = 56 + 4 * i
391     li   a1, 1
392     li   a2, 3
393     li   a3, 6
394     li   a4, WHITE
395     call prg32_gfx_rect
396     addi s0, s0, 1
397     j    1b
398 2:  lw   s0, 8(sp)
399     lw   ra, 12(sp)
400     addi sp, sp, 16
401     ret
402 draw_cell:                      # a0 = column, a1 = row, a2 = colour
403     addi sp, sp, -16
404     sw   ra, 12(sp)
405     slli a0, a0, 3              # x = column * 8
406     slli a1, a1, 3
407     addi a1, a1, TOP            # y = row * 8 + 8
408     mv   a4, a2                 # the colour is the fifth argument
409     li   a2, CELL
410     li   a3, CELL
411     call prg32_gfx_rect
```

```
412     lw   ra, 12(sp)
413     addi sp, sp, 16
414     ret
415 .section .rodata                # constants: they stay in the cartridge file
416 title:     .asciz "VERM1"
417 press_a:   .asciz "PRESS A"
418 game_over: .asciz "GAME OVER"
419 dx:        .word  1, 0, -1,  0  # right, down, left, up
420 dy:        .word  0, 1,  0, -1
421 .section .data                  # variables: they live in RAM
422 state:     .word TITLE
423 prev_btn:  .word 0              # the buttons one frame ago
424 dir:       .word 0
425 want:      .word 0
426 step_q8:   .word 0
427 score:     .word 0
428 best:      .word 0              # lost when the power goes
429 food_x:    .word 0
430 food_y:    .word 0
431 head_i:    .word 0              # ring index of the head
432 length:    .word 0              # number of cells of the worm
433 body_x:    .space 256           # the ring: one byte per segment
434 body_y:    .space 256
435 grid:      .space COLS*ROWS     # one byte per cell: 1 = worm
```

Listato D.3. `verm1_part3.S`.

D.4 Parte 4: il gioco finito

```
1  # Verm1 (say "vermone"), a tribute to Snake - part 4: the finished game
2  # ARC12 Computer Architecture, A.Y. 2026/27 - RISC-V assembly for PRG32
3  # Reference implementation contributed by Giuseppe Coviello
4  .option norelax
5  .equ BLACK, 0x0000          # RGB565 colours
6  .equ WHITE, 0xffff
7  .equ GREEN, 0x07e0          # the head
8  .equ BODY,  0x03e0          # the body: a darker green
9  .equ RED,   0xf800          # the food
10 .equ CELL,  8               # a cell is 8 x 8 pixels
11 .equ COLS,  40
12 .equ ROWS,  24
13 .equ TOP,   8
14 .equ SPEED_MIN, 32          # 32/256 of a cell per frame at the start
15 .equ SPEED_MAX, 96          # the worm never goes faster than this
16 .equ BTN_LEFT,  0x01
17 .equ BTN_RIGHT, 0x02
18 .equ BTN_UP,    0x04
19 .equ BTN_DOWN,  0x08
20 .equ BTN_A,     0x10
21 .equ DIR_RIGHT, 0           # bit 0 = vertical, bit 1 = backwards
22 .equ DIR_DOWN,  1
23 .equ DIR_LEFT,  2
24 .equ DIR_UP,    3
25 .equ TITLE, 0               # the three states of the game
```

```
.equ PLAY,  1
.equ OVER,  2
.section .text
.global verm1_init
.global verm1_update
.global verm1_draw

verm1_init:
    la   t0, state
    sw   zero, 0(t0)            # start on the title screen
    la   t0, prev_btn
    sw   zero, 0(t0)
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # draw the whole title screen, once
    ret
verm1_update:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)
    sw   s1, 4(sp)
    call prg32_input_read
    mv   s0, a0                 # s0 = buttons held in this frame
    la   t0, prev_btn
    lw   t1, 0(t0)              # t1 = buttons held one frame ago
    sw   s0, 0(t0)
    not  t1, t1
    and  s1, s0, t1             # s1 = buttons pressed just now
    la   t0, state
    lw   t1, 0(t0)              # t1 = present state
    li   t2, PLAY
    beq  t1, t2, u_play
    andi t3, s1, BTN_A          # TITLE or OVER: wait for a new press of A
    beqz t3, u_done
    li   t2, TITLE
    beq  t1, t2, u_start
    sw   t2, 0(t0)              # OVER --A--> TITLE
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # a new screen: draw everything
    j    u_done
u_start:
    call new_game               # TITLE --A--> PLAY
    j    u_done
u_play:
    mv   a0, s0
    call play_update
u_done:
    lw   s1, 4(sp)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
new_game:
    addi sp, sp, -16
    sw   ra, 12(sp)
```

```
    la   t0, grid
    li   t1, COLS*ROWS
1:  sb   zero, 0(t0)            # empty the grid, one byte per cell
    addi t0, t0, 1
    addi t1, t1, -1
    bnez t1, 1b
    li   t1, 0                  # i = 0, 1, 2: three cells in row 12
2:  la   t0, body_x
    add  t0, t0, t1
    addi t2, t1, 3
    sb   t2, 0(t0)              # columns 3, 4, 5
    la   t0, body_y
    add  t0, t0, t1
    li   t3, 12
    sb   t3, 0(t0)
    la   t0, grid+12*COLS
    add  t0, t0, t2
    li   t3, 1
    sb   t3, 0(t0)              # mark the cell as occupied
    addi t1, t1, 1
    li   t3, 3
    blt  t1, t3, 2b
    la   t0, head_i
    li   t1, 2
    sw   t1, 0(t0)              # the head is entry 2 of the ring
    la   t0, length
    li   t1, 3
    sw   t1, 0(t0)
    la   t0, dir
    sw   zero, 0(t0)
    la   t0, want
    sw   zero, 0(t0)
    la   t0, step_q8
    sw   zero, 0(t0)
    la   t0, score
    sw   zero, 0(t0)
    call place_food
    la   t0, state
    li   t1, PLAY
    sw   t1, 0(t0)
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # a new screen: draw everything
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
play_update:                    # a0 = buttons held
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, dir
    lw   t1, 0(t0)              # t1 = direction of the last step
    la   t0, want
    lw   t2, 0(t0)              # t2 = wanted direction, so far
    andi t3, a0, BTN_RIGHT
    beqz t3, 1f
    li   t2, DIR_RIGHT
```

```
1:  andi t3, a0, BTN_DOWN
    beqz t3, 2f
    li   t2, DIR_DOWN
2:  andi t3, a0, BTN_LEFT
    beqz t3, 3f
    li   t2, DIR_LEFT
3:  andi t3, a0, BTN_UP
    beqz t3, 4f
    li   t2, DIR_UP
4:  xor  t3, t2, t1             # opposite directions differ in bit 1 only
    li   t4, 2
    beq  t3, t4, 5f             # a U-turn: ignore the request
    sw   t2, 0(t0)
5:
    la   t0, score
    lw   t3, 0(t0)
    slli t3, t3, 2
    addi t3, t3, SPEED_MIN      # speed = 32 + 4 * score, in 1/256 of a cell
    li   t4, SPEED_MAX
    ble  t3, t4, 8f
    mv   t3, t4                 # ... but never more than 96
8:  la   t0, step_q8
    lw   t1, 0(t0)
    add  t1, t1, t3             # add a fraction of a cell
    li   t2, 256
    blt  t1, t2, 6f             # less than one whole cell: wait
    sub  t1, t1, t2
    sw   t1, 0(t0)
    call step                   # one whole cell: move
    j    7f
6:  sw   t1, 0(t0)
7:  lw   ra, 12(sp)
    addi sp, sp, 16
    ret
step:
    addi sp, sp, -16
    sw   ra, 12(sp)
    sw   s0, 8(sp)              # s0 = column of the new head
    sw   s1, 4(sp)              # s1 = row of the new head
    sw   s2, 0(sp)              # s2 = 0 when the worm eats
    la   t0, want
    lw   t1, 0(t0)
    la   t0, dir
    sw   t1, 0(t0)              # the wanted direction is now the direction
    slli t1, t1, 2              # table offset = direction * 4 bytes
    la   t0, dx
    add  t0, t0, t1
    lw   t2, 0(t0)              # t2 = dx[direction]
    la   t0, dy
    add  t0, t0, t1
    lw   t3, 0(t0)              # t3 = dy[direction]
    la   t0, head_i
    lw   t5, 0(t0)              # t5 = ring index of the head
    la   t0, body_x
    add  t0, t0, t5
    lbu  s0, 0(t0)              # one byte: the column of the head
```

```
194     la   t0, old_x
195     sw   s0, 0(t0)              # remember where the head was
196     add  s0, s0, t2
197     bgez s0, 1f
198     li   s0, COLS-1             # off the left edge: wrap
199 1:  li   t4, COLS
200     blt  s0, t4, 2f
201     li   s0, 0
202 2:  la   t0, body_y
203     add  t0, t0, t5
204     lbu  s1, 0(t0)
205     la   t0, old_y
206     sw   s1, 0(t0)
207     add  s1, s1, t3
208     bgez s1, 3f
209     li   s1, ROWS-1
210 3:  li   t4, ROWS
211     blt  s1, t4, 4f
212     li   s1, 0
213 4:
214     la   t0, food_x
215     lw   t1, 0(t0)
216     xor  t1, t1, s0
217     la   t0, food_y
218     lw   t2, 0(t0)
219     xor  t2, t2, s1
220     or   s2, t1, t2             # s2 = 0 if the new head is on the food
221     la   t0, length
222     lw   t1, 0(t0)
223     bnez s2, 5f                 # not eating: the tail moves on
224     li   t2, 255
225     bge  t1, t2, 5f             # the ring is full: no more growth
226     addi t1, t1, 1
227     sw   t1, 0(t0)              # eating: one cell longer, the tail stays
228     la   t0, erase_on
229     sw   zero, 0(t0)            # ... so there is no cell to erase
230     j    6f
231 5:  la   t0, head_i
232     lw   t2, 0(t0)
233     sub  t2, t2, t1
234     addi t2, t2, 1
235     andi t2, t2, 255            # ring index of the tail
236     la   t0, body_x
237     add  t0, t0, t2
238     lbu  t3, 0(t0)
239     la   t0, body_y
240     add  t0, t0, t2
241     lbu  t4, 0(t0)
242     la   t0, erase_x
243     sw   t3, 0(t0)              # remember the cell the tail leaves
244     la   t0, erase_y
245     sw   t4, 0(t0)
246     la   t0, erase_on
247     li   t5, 1
248     sw   t5, 0(t0)
249     li   t5, COLS
```

```
    mul  t4, t4, t5
    add  t4, t4, t3             # cell number of the tail = row * 40 + column
    la   t0, grid
    add  t0, t0, t4
    sb   zero, 0(t0)            # the tail cell is free again
6:  li   t5, COLS
    mul  t4, s1, t5
    add  t4, t4, s0             # cell number of the new head
    la   t0, grid
    add  t0, t0, t4
    lbu  t1, 0(t0)
    beqz t1, 7f                 # a free cell: go on
    call game_is_over           # the worm bit itself
    j    9f
7:  li   t1, 1
    sb   t1, 0(t0)              # occupy the new cell
    la   t0, head_i
    lw   t1, 0(t0)
    addi t1, t1, 1
    andi t1, t1, 255            # the ring wraps after entry 255
    sw   t1, 0(t0)
    la   t0, body_x
    add  t0, t0, t1
    sb   s0, 0(t0)              # write the new head into the ring
    la   t0, body_y
    add  t0, t0, t1
    sb   s1, 0(t0)
    la   t0, dirty
    li   t1, 1
    sw   t1, 0(t0)              # the screen must follow: a few cells
    bnez s2, 9f
    la   t0, score
    lw   t1, 0(t0)
    addi t1, t1, 1
    sw   t1, 0(t0)
    call place_food
    li   a2, 72                 # a short high note
    li   a4, 60
    call play_note
9:  lw   s2, 0(sp)
    lw   s1, 4(sp)
    lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
game_is_over:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, state
    li   t1, OVER
    sw   t1, 0(t0)              # PLAY --> OVER
    la   t0, dirty
    li   t1, 2
    sw   t1, 0(t0)              # a new screen: draw everything
    la   t0, score
    lw   t1, 0(t0)
```

```
306     la   t0, best
307     lw   t2, 0(t0)
308     ble  t1, t2, 1f
309     sw   t1, 0(t0)              # a new best score
310 1:  li   a2, 36                 # a long low note
311     li   a4, 400
312     call play_note
313     lw   ra, 12(sp)
314     addi sp, sp, 16
315     ret
316 place_food:
317     addi sp, sp, -16
318     sw   ra, 12(sp)
319     sw   s0, 8(sp)
320 1:  li   a0, 0
321     li   a1, COLS-1
322     call prg32_random_number
323     mv   s0, a0                 # s0 = a column
324     li   a0, 0
325     li   a1, ROWS-1
326     call prg32_random_number    # a0 = a row
327     li   t1, COLS
328     mul  t2, a0, t1
329     add  t2, t2, s0
330     la   t0, grid
331     add  t0, t0, t2
332     lbu  t3, 0(t0)
333     bnez t3, 1b                 # on the worm: try again
334     la   t0, food_x
335     sw   s0, 0(t0)
336     la   t0, food_y
337     sw   a0, 0(t0)
338     lw   s0, 8(sp)
339     lw   ra, 12(sp)
340     addi sp, sp, 16
341     ret
342 play_note:                      # a2 = MIDI note, a4 = milliseconds
343     addi sp, sp, -16
344     sw   ra, 12(sp)
345     li   a0, 0                  # channel
346     li   a1, 0                  # instrument
347     li   a3, 255                # volume
348     call prg32_audio_note
349     lw   ra, 12(sp)
350     addi sp, sp, 16
351     ret
352 verm1_draw:
353     la   t0, dirty
354     lw   t1, 0(t0)
355     beqz t1, 3f                 # nothing changed: nothing to draw
356     sw   zero, 0(t0)
357     addi sp, sp, -16
358     sw   ra, 12(sp)
359     li   t2, 2
360     beq  t1, t2, 1f
361     call draw_changes           # one step: a few cells
```

```
    j    2f
1:  call draw_full              # a new screen: everything
2:  lw   ra, 12(sp)
    addi sp, sp, 16
3:  ret
draw_changes:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, erase_on
    lw   t1, 0(t0)
    beqz t1, 1f
    la   t0, erase_x
    lw   a0, 0(t0)
    la   t0, erase_y
    lw   a1, 0(t0)
    li   a2, BLACK
    call draw_cell              # the cell the tail has left
1:  la   t0, old_x
    lw   a0, 0(t0)
    la   t0, old_y
    lw   a1, 0(t0)
    li   a2, BODY
    call draw_cell              # the old head is now body
    call draw_head
    call draw_food
    call draw_score
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
draw_full:
    addi sp, sp, -16
    sw   ra, 12(sp)
    li   a0, BLACK
    call prg32_gfx_clear
    li   a0, 0
    li   a1, 0
    la   a2, title
    call draw_text
    call draw_score
    la   t0, state
    lw   t1, 0(t0)
    li   t2, TITLE
    bne  t1, t2, 1f
    li   a0, 132                # TITLE: only an invitation
    li   a1, 96
    la   a2, press_a
    call draw_text
    j    2f
1:  call draw_worm              # PLAY and OVER: the whole scene
    call draw_food
    la   t0, state
    lw   t1, 0(t0)
    li   t2, OVER
    bne  t1, t2, 2f
    li   a0, 124
    li   a1, 96
```

```
418     la   a2, game_over
419     call draw_text
420 2:  lw   ra, 12(sp)
421     addi sp, sp, 16
422     ret
423 draw_head:
424     addi sp, sp, -16
425     sw   ra, 12(sp)
426     la   t0, head_i
427     lw   t1, 0(t0)
428     la   t0, body_x
429     add  t0, t0, t1
430     lbu  a0, 0(t0)
431     la   t0, body_y
432     add  t0, t0, t1
433     lbu  a1, 0(t0)
434     li   a2, GREEN
435     call draw_cell
436     lw   ra, 12(sp)
437     addi sp, sp, 16
438     ret
439 draw_food:
440     addi sp, sp, -16
441     sw   ra, 12(sp)
442     la   t0, food_x
443     lw   a0, 0(t0)
444     la   t0, food_y
445     lw   a1, 0(t0)
446     li   a2, RED
447     call draw_cell
448     lw   ra, 12(sp)
449     addi sp, sp, 16
450     ret
451 draw_text:                      # a0 = x, a1 = y, a2 = address of the string
452     addi sp, sp, -16
453     sw   ra, 12(sp)
454     li   a3, WHITE
455     li   a4, BLACK
456     call prg32_gfx_text8
457     lw   ra, 12(sp)
458     addi sp, sp, 16
459     ret
460 draw_worm:
461     addi sp, sp, -16
462     sw   ra, 12(sp)
463     sw   s0, 8(sp)
464     li   s0, 0                  # s0 = i: segment 0 is the head
465 1:  la   t0, length
466     lw   t1, 0(t0)
467     bge  s0, t1, 3f
468     la   t0, head_i
469     lw   t1, 0(t0)
470     sub  t1, t1, s0
471     andi t1, t1, 255            # ring index of segment i
472     la   t0, body_x
473     add  t0, t0, t1
```

```
    lbu  a0, 0(t0)
    la   t0, body_y
    add  t0, t0, t1
    lbu  a1, 0(t0)
    li   a2, BODY
    bnez s0, 2f
    li   a2, GREEN              # the head is brighter
2:  call draw_cell
    addi s0, s0, 1
    j    1b
3:  lw   s0, 8(sp)
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
draw_score:
    addi sp, sp, -16
    sw   ra, 12(sp)
    la   t0, score
    lw   a0, 0(t0)
    la   a1, score_txt
    call to_decimal             # the score, as three characters
    li   a0, 56
    li   a1, 0
    la   a2, score_txt
    call draw_text
    la   t0, best
    lw   a0, 0(t0)
    la   a1, best_txt+5         # after the letters "BEST "
    call to_decimal
    li   a0, 256
    li   a1, 0
    la   a2, best_txt
    call draw_text
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
to_decimal:                     # a0 = value 0..999, a1 = address of 3 chars
    li   t0, 10
    remu t1, a0, t0             # the units: the remainder of value / 10
    addi t1, t1, 48             # 48 is the ASCII code of '0'
    sb   t1, 2(a1)
    divu a0, a0, t0
    remu t1, a0, t0             # the tens
    addi t1, t1, 48
    sb   t1, 1(a1)
    divu a0, a0, t0
    remu t1, a0, t0             # the hundreds
    addi t1, t1, 48
    sb   t1, 0(a1)
    ret                         # a leaf function: no stack frame needed
draw_cell:                      # a0 = column, a1 = row, a2 = colour
    addi sp, sp, -16
    sw   ra, 12(sp)
    slli a0, a0, 3              # x = column * 8
    slli a1, a1, 3
    addi a1, a1, TOP            # y = row * 8 + 8
```

```
    mv   a4, a2                # the colour is the fifth argument
    li   a2, CELL
    li   a3, CELL
    call prg32_gfx_rect
    lw   ra, 12(sp)
    addi sp, sp, 16
    ret
.section .rodata               # constants: they stay in the cartridge file
title:     .asciz "VERM1"
press_a:   .asciz "PRESS A"
game_over: .asciz "GAME OVER"
dx:        .word  1, 0, -1,  0  # right, down, left, up
dy:        .word  0, 1,  0, -1
.section .data                 # variables: they live in RAM
state:     .word TITLE
prev_btn:  .word 0             # the buttons one frame ago
dir:       .word 0
want:      .word 0
step_q8:   .word 0
score:     .word 0
best:      .word 0             # lost when the power goes
food_x:    .word 0
food_y:    .word 0
head_i:    .word 0             # ring index of the head
length:    .word 0             # number of cells of the worm
body_x:    .space 256          # the ring: one byte per segment
body_y:    .space 256
grid:      .space COLS*ROWS    # one byte per cell: 1 = worm
dirty:     .word 2             # 0 nothing to draw, 1 a step, 2 everything
old_x:     .word 0             # where the head was before the step
old_y:     .word 0
erase_on:  .word 0             # 1 if the tail left a cell
erase_x:   .word 0
erase_y:   .word 0
score_txt: .asciz "000"        # filled in by to_decimal
best_txt:  .asciz "BEST 000"
```

Listato D.4. `verm1_part4.S`.

D.5 I file del bundle dello Store

I tre file che seguono sono quelli del Capitolo 23, per il gioco di riferimento. In un vostro bundle, l'identificatore, il titolo e ogni occorrenza di `Your Name` vengono sostituiti; il ringraziamento per l'implementazione di riferimento resta, come per qualsiasi lavoro che parte da quello di qualcun altro.

```
{
  "abi": "prg32-metadata-1.0",
  "id": "it.uniparthenope.arc12.verm1",
  "title": "Verm1",
  "version": "1.0.0",
  "summary": "A hungry worm on a 40 by 24 grid: a tribute to Snake.",
  "description": "Verm1 (say \"vermone\") is the course-long project of ARC12
    Computer Architecture at the University of Naples Parthenope. Steer the
```

```
    worm with the joystick, eat the red food, and do not bite yourself. Written
     from scratch in RISC-V assembly.",
  "authors": [{"name": "Your Name"}],
  "license": "MIT",
  "tags": ["arcade", "snake", "assembly", "education"],
  "assets": {"icon": "icon.png"},
  "architectures": [
    {"id": "esp32c6", "file": "verm1-esp32c6.prg32"},
    {"id": "qemu", "file": "verm1-qemu.prg32"}
  ]
}
```

Listato D.5. `manifest.json`.

```
{
  "abi": "prg32-metadata-1.0",
  "id": "it.uniparthenope.arc12.verm1",
  "title": "Verm1",
  "version": "1.0.0",
  "summary": "A hungry worm on a 40 by 24 grid: a tribute to Snake.",
  "description": "Verm1 (say \"vermone\") is the course-long project of ARC12
    Computer Architecture at the University of Naples Parthenope. Steer the
    worm with the joystick, eat the red food, and do not bite yourself. Written
     from scratch in RISC-V assembly.",
  "authors": [
    {
      "name": "Your Name"
    }
  ],
  "license": "MIT",
  "tags": [
    "arcade",
    "snake",
    "assembly",
    "education"
  ]
}
```

Listato D.6. `metadata.json`: il manifesto senza `assets` e `architectures`.

```
{
  "abi": "prg32-colophon-1.0",
  "title": "Verm1",
  "subtitle": "A tribute to Snake, in RISC-V assembly",
  "version": "1.0.0",
  "release_date": "2026-12-18",
  "developer": {"name": "Your Name"},
  "authors": [
    {"role": "game design", "name": "Your Name"},
    {"role": "programming", "name": "Your Name"}
  ],
  "license": "MIT",
  "copyright": "Copyright 2026 Your Name",
  "acknowledgements": [
```

```
    "Giuseppe Coviello, reference implementation of Verm1",
    "ARC12 Computer Architecture, University of Naples Parthenope",
    "The PRG32 community"
  ],
  "controls": [
    {"input": "JOYSTICK", "action": "Steer the worm"},
    {"input": "A", "action": "Start, and return to the title after game over"}
  ],
  "start_prompt": "Press A to play"
}
```

Listato D.7. `colophon.json`.

APPENDICE E

Risposte alle domande

Ogni capitolo tematico e ogni capitolo tutorial termina con domande a scelta multipla nello stile della prova intercorso (paragrafo *Verificate di aver capito*). Seguono le risposte: la lettera, il testo dell'opzione giusta, e una riga di spiegazione. Prima rispondete, e poi leggete: una domanda di cui si è letta la risposta non insegna niente.

Capitolo 1: L'installazione

D1.1 **B.** *ESP-IDF non è attivo in questo terminale.* L'attivazione dura quanto il terminale: eseguite export.sh, oppure aprite ESP-IDF PowerShell.

D1.2 **C.** *PRG32-QT.* PRG32-QT interpreta la cartuccia e ha un debugger integrato; il compilatore serve solo per compilare le cartucce.

D1.3 **C.** *chiama il runtime attraverso la stessa tabella di funzioni su ogni host.* Una cartuccia portabile dipende soltanto dall'ABI, la tabella di funzioni del runtime, che ogni host fornisce.

D1.4 **B.** *traduce il sorgente assembly in codice macchina.* L'assembly è testo per le persone; il processore legge numeri. L'assemblatore traduce il primo nei secondi; il runtime di un host esegue poi il risultato.

Capitolo 2: Primo contatto

D2.1 **B.** *L'indirizzo del contatore.* la carica l'indirizzo dell'etichetta; lw legge poi in t1 il valore memorizzato a quell'indirizzo.

D2.2 **C.** *ra, perché call lo sovrascrive.* call scrive in ra l'indirizzo di ritorno. Le chiamate della funzione chiamata lo sovrascriverebbero: una funzione non foglia salva ra sullo stack e lo ripristina prima di ret.

D2.3 **D.** *44.* 300 = 256 + 44 = 1 0010 1100 in binario. AND con 1111 1111 conserva gli otto bit bassi: 0010 1100 = 44.

D2.4 **B.** *31: un sedici più quindici.* 0x indica un numero esadecimale. 1 vale un sedici e F vale quindici unità: 16 + 15 = 31, cioè 1 1111 in binario.

Capitolo 3: Storia della computazione

D3.1 **B.** *Programma e dati stanno nella stessa memoria.* Il concetto di programma memorizzato: le istruzioni sono dati in memoria, prelevati ed eseguiti uno dopo l'altro.

D3.2 **C.** *Il suo insieme di istruzioni è uno standard aperto, libero da realizzare.* RISC-V è un'architettura a standard aperto; chiunque può progettare e vendere un processore RISC-V senza licenza sull'architettura. Non è stata la prima RISC (RISC di Berkeley, MIPS e ARM sono precedenti).

D3.3 **C.** *Ada Lovelace.* Ada Lovelace, per le sue note del 1843 sulla macchina analitica.

Capitolo 4: Informazione digitale

D4.1 **D.** *256.* Un byte sono 8 bit: $2^8 = 256$ configurazioni, per esempio i naturali da 0 a 255.

D4.2 **B.** *magenta.* 0xF81F = 11111 000000 11111: rosso e blu al massimo, verde a zero: magenta.

D4.3 **C.** *5.* Quattro caratteri più il byte di terminazione a zero.

D4.4 **C.** *0xCD.* RISC-V è little-endian: il byte meno significativo, 0xCD, va all'indirizzo più basso.

Capitolo 5: Gli interi

D5.1 **B.** *-4.* Si complementa e si somma uno: 0000 0011 + 1 = 0000 0100 = 4, quindi la configurazione vale -4. (252 è la sua lettura senza segno.)

D5.2 **B.** *100 + 28.* L'intervallo è [-128, 127]. 100 + 27 = 127 ci sta; 100 + 28 = 128 no: due positivi danno una configurazione negativa (1000 0000 = -128).

D5.3 **C.** *t2 = 1, t3 = 0.* Con segno: -1 < 1 è vero, quindi t2 = 1. Senza segno: 4 294 967 295 < 1 è falso, quindi t3 = 0.

D5.4 **A.** *0,101.* 0,625 × 2 = 1,25 (1); 0,25 × 2 = 0,5 (0); 0,5 × 2 = 1,0 (1): 0,101 = 1/2 + 1/8.

Capitolo 6: I reali

D6.1 **D.** *Eccesso k.* In eccesso k, X = x + k: a un numero maggiore corrisponde sempre una configurazione maggiore. Per questo codifica gli esponenti della virgola mobile, che vanno confrontati in fretta.

D6.2 **B.** *1,5.* 0x180 = 384; 384 / 256 = 1,5. Parte intera 0x01, frazione 0x80 = 128/256 = 0,5.

D6.3 **B.** *3,0.* 0100 0000 0100 0000 ...: S = 0, E = 1000 0000 = 128, F = 100...: $1{,}1_2 \times 2^{128-127} = 1{,}5 \times 2 = 3{,}0$.

D6.4 **C.** *infinito.* Tutti uni nell'esponente: infinito se la frazione è zero, NaN altrimenti.

Capitolo 7: Verm1, parte 1

D7.1 **B.** *4.* 64/256 = 1/4 di cella per frame: una cella intera ogni 4 frame.

D7.2 **C.** *(80, 32).* x = 10 × 8 = 80; y = 3 × 8 + 8 = 32.

D7.3 **B.** *draw chiama altre funzioni, che sovrascrivono ra.* Una chiamata scrive in ra l'indirizzo di ritorno. Una funzione che fa chiamate deve mettere al sicuro sullo stack il proprio ra; a una funzione foglia non serve.

Capitolo 8: Algebra di Boole: operatori e proprietà

D8.1 **C.** *x = 1, y = 1.* NAND è il complemento di AND: vale 0 solo quando entrambi gli ingressi valgono 1.

D8.2 **B.** $\overline{a}+b$. $\overline{a \cdot \overline{b}} = \overline{a} + \overline{\overline{b}} = \overline{a} + b$, usando l'involuzione sul secondo termine.

D8.3 **C.** *0x0000000A.* 0x5A = 0101 1010; AND con 0000 1111 conserva il nibble basso: 1010 = 0xA.

D8.4 **B.** *ori t0, t0, 0x20.* OR con una maschera: x + 1 = 1 sul bit 5, x + 0 = x altrove. xori lo invertirebbe; addi propagherebbe un riporto nel bit 6 se il bit fosse già a 1.

Capitolo 9: Algebra di Boole: funzioni e minimizzazione

D9.1 **D.** *256.* $2^{2^3} = 2^8 = 256$.

D9.2 **B.** $A\,\overline{B}\,C$. La riga 5 è 101: $A = 1$, $B = 0$, $C = 1$, quindi il mintermine è $A\,\overline{B}\,C$.

D9.3 **B.** *due variabili.* Un gruppo di 2^k celle elimina k variabili: quattro celle, due variabili.

D9.4 **C.** *almeno due ingressi valgono 1.* C_{out} è la funzione di maggioranza: $AB + C_{in}(A \oplus B) = AB + AC_{in} + BC_{in}$. Un numero dispari di 1 descrive invece il bit di somma S.

Capitolo 10: Elettronica

D10.1 **B.** *260 Ω.* R = (3,3 - 2) / 0,005 = 260 Ω.

D10.2 **B.** *il pMOS conduce e l'uscita vale 1.* Un gate basso chiude il pMOS, che collega l'uscita all'alimentazione; l'nMOS è aperto.

D10.3 **C.** *6.* Un NAND (4) seguito da un invertitore (2).

D10.4 **B.** *Perché il piedino legga un 1 definito quando il pulsante è rilasciato.* Senza, il piedino rilasciato sarebbe flottante; il pull-up lo lega a 3,3 V, e l'interruttore premuto prevale portandolo a massa.

Capitolo 11: Macchine combinatorie

D11.1 **B.** *5.* $\lceil \log_2 26 \rceil = 5$, perché $2^4 = 16 < 26 \leq 32 = 2^5$. Restano sei parole non assegnate: il codice è incompleto.

D11.2 **C.** *8 uscite, una a 1.* $2^3 = 8$ uscite, e ne è attiva esattamente una: la rappresentazione decodificata (one-hot) della parola in ingresso.

D11.3 **B.** *3.* n linee di selezione scelgono tra 2^n ingressi: 3 linee per 8 ingressi.

D11.4 **C.** *non rileva l'errore.* Due alterazioni lasciano invariata la parità. Un solo bit di parità rileva soltanto un numero dispari di errori.

Capitolo 12: Verm1, parte 2

D12.1 **C.** *0x0a.* 0x08 OR 0x02 = 0x0a: un bit per pulsante.

D12.2 **B.** *DOWN (1).* 3 XOR 1 = 2: DOWN è l'opposto di UP.

D12.3 **B.** *ogni elemento della tabella è di 4 byte.* Gli elementi sono parole: l'elemento k sta allo scostamento 4k.

D12.4 **C.** *t1 e t4 sono uguali.* XOR vale 1 dove i bit differiscono: nessuna differenza, tutti zeri.

Capitolo 13: Macchine sequenziali

D13.1 **B.** *dal solo stato presente.* Moore: w va da S a U. In una macchina di Mealy l'uscita dipende dallo stato e dall'ingresso.

D13.2 **C.** *Q resta a 1.* S = R = 0 è la condizione di mantenimento: la retroazione conserva il valore memorizzato.

D13.3 **C.** *Il flip-flop cambia solo su un fronte del clock.* Il latch è trasparente per tutto il livello attivo del clock; il flip-flop campiona D solo sul fronte.

D13.4 **B.** *4.* $\lceil \log_2 10 \rceil = 4$ variabili di stato, con il codice binario di lunghezza minima della Capitolo 11. Ne servirebbero dieci con un codice one-hot.

Capitolo 14: Registri e bus

D14.1 **B.** *Perché solo la sorgente abilitata piloti le linee.* Più sorgenti condividono le stesse linee; tutte tranne una devono essere elettricamente scollegate (alta impedenza), altrimenti due uscite si contenderebbero il filo.

D14.2 **C.** *32.* Un bit per impulso: n impulsi per n bit. Un trasferimento parallelo richiede un impulso e n fili.

D14.3 **C.** *8192.* $2^{10} = 1024$ parole di 8 bit: 8192 bit, cioè 1 KiB.

Capitolo 15: La memoria

D15.1 **C.** *64 KiB.* $2^{16} = 65\,536$ locazioni di un byte: 64 KiB, lo spazio di indirizzi di un Commodore 64.

D15.2 **D.** *0x01.* Little-endian: il byte meno significativo, 0x01, sta all'indirizzo più basso.

D15.3 **A.** *Perché i suoi condensatori perdono la carica.* Ogni bit è una carica in un condensatore che si disperde; un circuito interno alla memoria riscrive periodicamente ogni cella.

D15.4 **B.** *Con sw all'indirizzo di un registro del dispositivo.* RISC-V non ha istruzioni di I/O: i dispositivi sono mappati in memoria e si raggiungono con normali load e store.

Capitolo 16: La memoria permanente

D16.1 **D.** *Flash.* La flash è l'evoluzione a blocchi della EEPROM. Alla EPROM serve la luce ultravioletta; ROM e PROM non si possono cancellare.

D16.2 B. *Il dato viene prelevato dalla memoria centrale e copiato nella cache, con i suoi vicini.* L'accesso è semplicemente più lento: il blocco che contiene l'indirizzo viene caricato nella cache, scommettendo che presto serviranno indirizzi vicini.

D16.3 B. *La latenza.* La latenza è l'attesa che il settore arrivi sotto la testina; il tempo di posizionamento dipende dal movimento del braccio.

D16.4 C. *una transizione tra pit e land.* Un cambio di riflettività, in un verso o nell'altro, è un 1; nessun cambio è uno 0.

Capitolo 17: Verm1, parte 3

D17.1 C. *254, 255, 0, 1.* Dalla coda, head_i - length + 1 = -2, cioè 254 dopo AND 255, fino alla testa: 254, 255, 0, 1.

D17.2 B. *il byte 127.* $3 \times 40 + 7 = 127$.

D17.3 B. *la partita comincia una volta.* Solo il frame in cui A scende dà s1 = 0x10: un fronte, una transizione.

D17.4 B. *perché la testa possa entrare nella cella lasciata dalla coda.* Nello stesso passo la coda si sposta: quella cella è libera per la testa.

Capitolo 18: Il processore

D18.1 B. *il registro di stato con i flag.* RISC-V non ha codici di condizione: i salti confrontano direttamente i registri.

D18.2 C. *solo load e store.* È un'architettura load-store: lw, lh, lb, sw, sh, sb e le forme senza segno.

D18.3 B. *t0 è stato inserito sullo stack.* Il puntatore scende di una parola e il valore è memorizzato sulla nuova cima: un push.

D18.4 B. *accumulatore := accumulatore + [y].* L'accumulatore è l'operando implicito e la destinazione.

Capitolo 19: Istruzioni e interruzioni

D19.1 B. *i sette meno significativi.* I bit da 6 a 0, in ogni formato: il processore sa sempre dove guardare per prima cosa.

D19.2 C. *4.* Gli indirizzi contano i byte e un'istruzione è lunga quattro byte.

D19.3 C. *mepc.* mepc, scritto dall'hardware alla trap e usato da mret. mtvec contiene l'indirizzo del gestore.

D19.4 C. *viene prelevata un'istruzione illegale.* È causata dall'istruzione in esecuzione, in modo sincrono; le altre vengono dall'esterno, in un momento qualunque.

Capitolo 20: I bus e la scheda video

D20.1 C. *921 600 byte.* 307 200 pixel × 3 byte = 921 600 byte.

D20.2 B. *100 MB/s.* 2 byte × 50 milioni di trasferimenti al secondo = 100 MB/s.

D20.3 C. *la posizione del suo colore in una palette.* Il byte è un indice; il colore vero si cerca in una tabella di 256 voci, la palette.

D20.4 B. *L'uso dei processori grafici per calcoli non grafici.* General-purpose computing on graphics processing units: calcolo generico su unità di elaborazione grafica.

Capitolo 21: Ingresso e uscita

D21.1 B. *Trasforma una combinazione di riga e colonna in un codice di tasto.* Da una posizione decodificata nella matrice dei tasti a un codice compatto di n bit, come il codificatore della Capitolo 11.

D21.2 B. *tre celle con filtri rosso, verde e blu.* Tre sub-pixel, ciascuno una cella di cristalli liquidi dietro un filtro colorato; un TFT in ognuno ne conserva lo stato.

D21.3 B. *I pixel sono pilotati uno alla volta e nel frattempo devono mantenere il proprio stato.* Persistenza: senza un transistor per pixel, le celle rispondono lentamente e conservano male il proprio stato mentre si scandiscono le altre.

D21.4 C. *polling.* Il programma interroga periodicamente il dispositivo. Con le interruzioni sarebbe il dispositivo a chiamare il programma.

Capitolo 22: Verm1, parte 4

D22.1 B. *32 30 35.* I caratteri '2', '0', '5': 0x32, 0x30, 0x35. Le unità vanno allo scostamento 2.

D22.2 B. *8.* Un passo ogni 4 frame è 64/256: $32 + 4 \times \text{score} = 64$, quindi score = 8.

D22.3 B. *5.* la sono due istruzioni, auipc e addi; poi lw, beqz e ret.

D22.4 B. *non chiama funzioni e usa solo registri a e t.* Una funzione foglia che tocca solo registri caller-saved non ha nulla da salvare.

Capitolo 23: Il Cartridge Store

D23.1 B. *pubblicate la 1.0.1.* Una versione pubblicata si considera permanente: una correzione è una nuova versione con lo stesso id.

D23.2 B. *i metadati vanno dentro una cartuccia, che ha una sola architettura.* L'elenco dei file appartiene al bundle; una singola cartuccia dichiara la propria architettura, data sulla riga di comando.

D23.3 C. *in una variabile d'ambiente della vostra shell.* Un token è una credenziale: non va mai in un file condiviso o sotto controllo di versione.

Capitolo 24: Oltre un solo core

D24.1 B. *5.* Con n illimitato, S = 1 / (1 - p) = 1 / 0,2 = 5.

D24.2 C. *le false dipendenze tra usi dello stesso nome di registro.* Due valori indipendenti che condividono un nome ricevono registri fisici diversi e si possono calcolare nello stesso momento.

D24.3 B. *30.* 256 bit sono 32 byte per giro: 960 / 32 = 30. Il codice non cambia.

D24.4 B. *OR.* L'uscita vale 1 ogni volta che almeno un ingresso vale 1: OR.

APPENDICE F

Glossario inglese–italiano

La documentazione tecnica, i nomi delle istruzioni e buona parte della letteratura sono in inglese. Questo glossario raccoglie, in ordine alfabetico del termine inglese, i termini dei glossari che chiudono i capitoli, con il capitolo in cui ciascuno è introdotto.

Inglese	**Italiano**	**Cap.**
accelerator	acceleratore	24
access time	tempo di accesso	15
accumulator	accumulatore	18
active matrix	matrice attiva	21
address	indirizzo	2
address / data bus	bus indirizzi / bus dati	15
addressing mode	modo di indirizzamento	19
aligned word	parola allineata	15
analysis / synthesis	analisi / sintesi	8
arithmetic logic unit	unità aritmetico-logica	3
array	vettore (array)	4
backlight	retroilluminazione	21
base (radix)	base	5
bit / byte / word	bit / byte / parola	4
bitstream	bitstream	24
branch	salto condizionato	2
branch prediction	predizione dei salti	24
breadboard	basetta sperimentale (breadboard)	10
breakpoint	punto di interruzione	1
bus	bus	14
bus width	larghezza del bus	20
byte order	ordinamento dei byte	15
byte order (endianness)	ordinamento dei byte	4
cache hit / miss	successo / fallimento di cache	16
calling convention	convenzione di chiamata	2
canonical form	forma canonica	9
capacitor	condensatore	10
cardinality	cardinalità	11
carry / borrow	riporto / prestito	5

Inglese	**Italiano**	**Cap.**
carry in / carry out	riporto entrante / uscente	9
cartridge	cartuccia	1, 2
cathode-ray tube	tubo a raggi catodici	21
chip select	selezione del chip	14
clause, product term	clausola, termine prodotto	9
clock period	periodo di clock	13
code word	parola codice	11
colour depth	profondità di colore	20
combinational network	rete combinatoria	11
comparator	comparatore	11
complement	complemento (negazione)	8
computer	calcolatore, elaboratore	3
computer science / informatics	informatica	3
context	contesto	19
control unit	unità di controllo	3, 18
controller	controller (unità di controllo della periferica)	20
current	corrente	10
decoder / encoder	decodificatore / codificatore	11
dedicated / common bus	bus dedicato / comune	14
denormalised	denormalizzato	6
device driver	driver di periferica	20, 21
don't care	condizione di indifferenza	9
dot-matrix / ink-jet printer	stampante ad aghi / a getto d'inchiostro	21
drum / toner	tamburo / toner	21
duality	dualità	8
emulator	emulatore	1
enable	abilitazione	11
exception / trap	eccezione / trap	19
excess-k (biased)	eccesso-k	6
exponent	esponente	6
factor, sum term	fattore, termine somma	9
feedback	retroazione	13
fetch / decode / execute	prelievo / decodifica / esecuzione	19
finite-state machine	automa a stati finiti	13
firmware	firmware	1, 16
fixed / variable length	lunghezza fissa / variabile	11
fixed point	virgola fissa	6
floating input	ingresso flottante	10
floating point	virgola mobile	6
gate, source, drain	gate, source, drain	10
general / special register	registro generico / speciale	18
ground	massa	10
half adder / full adder	semisommatore / sommatore completo	9
hardware description language	linguaggio di descrizione dell'hardware	24
hexadecimal	esadecimale	4
hidden bit	bit nascosto	6
high impedance	alta impedenza	14

Inglese	Italiano	Cap.
high-performance computing	calcolo ad alte prestazioni	3
host	macchina ospite	1
immediate	immediato (costante)	2
instruction set	insieme (set) di istruzioni	3, 18
integrated circuit	circuito integrato	10
interpreter	interprete	1
interrupt	interruzione	19
interrupt latency	latenza di interruzione	19
interrupt service routine	routine di servizio dell'interruzione	19
involution	involuzione	8
Karnaugh map	mappa di Karnaugh	9
kernel (GPU)	kernel	24
keyboard / key	tastiera / tasto	21
label	etichetta	2
latch / flip-flop	latch / flip-flop (bistabile)	13
latency	tempo di latenza	16
level- / edge-triggered	sensibile al livello / al fronte	13
linked list	lista concatenata	4
liquid crystals	cristalli liquidi	21
literal	letterale	9
load / store	caricare / memorizzare	2
load, strobe	caricamento, strobe	14
logic circuit	circuito logico	8
logic gate	porta logica	8
logic level	livello logico	10
logical product (AND)	prodotto logico (congiunzione)	8
logical sum (OR)	somma logica (disgiunzione)	8
look-up table	tabella di look-up	24
loom	telaio	3
machine code	codice macchina	2
machine learning	apprendimento automatico	4
machine register	registro di macchina	18
magnetic core	nucleo magnetico (di ferrite)	15
main memory	memoria centrale	15
mainframe	mainframe, grande elaboratore	3
mantissa / fraction	mantissa	6
mask	maschera	8
mass storage	memoria di massa	16
memory-mapped I/O	I/O mappato in memoria	15
message passing	scambio di messaggi	24
minterm / maxterm	mintermine / maxtermine	9
most significant bit	bit più significativo	5
motherboard	scheda madre	16
multiplexer / demultiplexer	multiplexer / demultiplexer	11
natural number	numero naturale	5
nesting	annidamento	19
next-state function	funzione stato prossimo	13

Inglese	**Italiano**	**Cap.**
node	nodo	24
noise margin	margine di rumore	10
non-volatile	non volatile	16
normalisation	normalizzazione	6
Not a Number (NaN)	non un numero (NaN)	6
one's complement	complemento a uno (alla base diminuita)	6
operand	operando	19
operation code (opcode)	codice operativo	19
out-of-order execution	esecuzione fuori ordine	24
overflow	overflow (traboccamento)	5
palette	tavolozza (palette)	20
parity bit	bit di parità	11
peripheral	periferica	20
pipeline stage	stadio della pipeline	19
pointing device	periferica di puntamento	21
polling	interrogazione ciclica	21
positional notation	notazione posizionale	4
present / next state	stato presente / prossimo	13
privileged instruction	istruzione privilegiata	18
product of sums (POS)	prodotto di somme (tipo S)	9
program counter	contatore di programma	13, 18
propagation delay	ritardo di propagazione	10, 11
pull-up resistor	resistenza di pull-up	10
punched card	scheda perforata	3
quantisation	quantizzazione	4
read / write enable	abilitazione lettura / scrittura	15
redundant code	codice ridondante	11
refresh	rinfresco (refresh)	15
register	registro	2, 13, 14
register renaming	ridenominazione dei registri	24
remainder	resto	5
resistor / resistance	resistore / resistenza	10
resolution	risoluzione	20
return address	indirizzo di ritorno	2
ripple-carry adder	sommatore a propagazione del riporto	9
rising / falling edge	fronte di salita / di discesa	13
rounding	arrotondamento	6
runtime	ambiente di esecuzione	1
sampling	campionamento	4
seek time	tempo di posizionamento	16
semiconductor, doping	semiconduttore, drogaggio	10
sequential network	rete sequenziale	13
serial / parallel transfer	trasferimento seriale / parallelo	14
shared memory	memoria condivisa	24
shift	scorrimento (shift)	5
shift register	registro a scorrimento	14
sign and magnitude	segno e modulo	5

Inglese	Italiano	Cap.
sign extension	estensione del segno	5
signed integer	numero relativo (intero con segno)	5
single / double precision	singola / doppia precisione	6
soft core	soft core	24
solid-state disk	disco a stato solido	16
speed-up	accelerazione (speed-up)	24
stack	pila (stack)	2
stack	pila	4
stack pointer	puntatore alla pila	2
stack pointer	puntatore allo stack	18
state table / diagram	tabella / diagramma degli stati	13
status register, flags	registro di stato, indicatori	18
stored program	programma memorizzato	3
string	stringa	4
sum of products (SOP)	somma di prodotti (tipo P)	9
superscalar	superscalare	24
supervisor / user state	stato supervisore / utente	18
switching network	rete di commutazione	8
tape library	libreria a nastri	16
three-state (tri-state)	a tre stati (tristate)	14
to build	compilare (costruire)	1
to clamp	limitare a un intervallo	5
to flash	scrivere nella memoria flash	1
to mask an interrupt	mascherare un'interruzione	19
to set / clear / toggle	porre a 1 / azzerare / invertire	8
toolchain	catena di strumenti di sviluppo	1
track / sector / block	traccia / settore / blocco	16
transcoder	transcodificatore	11
transfer rate	velocità di trasferimento	20
tree	albero	4
truth table	tabella di verità	8
two's complement	complemento a due (alla base)	5
video card	scheda video	20
video memory	memoria video	20
volatile / persistent	volatile / persistente	15
voltage	tensione	10
word gate	porta di parola	14

Bibliografia

Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pagine 483–485, 1967.

George Boole. *An Investigation of the Laws of Thought*. Walton and Maberly, London, 1854.

Giacomo Bucci. *Calcolatori elettronici: architettura e organizzazione*. McGraw-Hill Education, 4 edizione, 2017.

Paul E. Ceruzzi. *A History of Modern Computing*. MIT Press, 2 edizione, 2003.

Scott Chacon e Ben Straub. *Pro Git*. Apress, 2 edizione, 2014. https://git-scm.com/book.

Espressif Systems. ESP32-C6 technical reference manual, 2025. https://www.espressif.com/en/support/documents/technical-documents.

Espressif Systems. ESP-IDF programming guide, 2026. https://docs.espressif.com/projects/esp-idf/.

EuroHPC Joint Undertaking. Advancing European Sovereignty in HPC with RISC-V: the DARE Project. Press release, European High-Performance Computing Joint Undertaking, 2025. https://eurohpc-ju.europa.eu/.

Michael J. Flynn. Some computer organizations and their effectiveness. *IEEE Transactions on Computers*, C-21(9):948–960, 1972.

GitHub. GitHub docs, 2026. https://docs.github.com.

David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.

Richard W. Hamming. Error detecting and error correcting codes. *Bell System Technical Journal*, 29(2):147–160, 1950.

John L. Hennessy e David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6 edizione, 2019.

IEEE. IEEE standard for floating-point arithmetic. IEEE Std 754-2019, Institute of Electrical and Electronics Engineers, 2019.

Maurice Karnaugh. The map method for synthesis of combinational logic circuits. *Transactions of the American Institute of Electrical Engineers, Part I*, 72(5):593–599, 1953.

George H. Mealy. A method for synthesizing sequential circuits. *Bell System Technical Journal*, 34(5):1045–1079, 1955.

Raffaele Montella. PRG32-QT: the portable C++20/Qt 6 host and debugger for PRG32 cartridges. https://github.com/riscv-prg32/PRG32-QT, 2026. Version 0.4.1, MIT License.

Raffaele Montella, Ivan Cafiero, e Simone Boscaglia. PRG32: Platform for RISC-V Gaming Development (32-bit). Software repository, University of Naples "Parthenope", 2026a. https://github.com/riscv-prg32/PRG32.

Raffaele Montella, Ivan Cafiero, e Simone Boscaglia. *Playing with RISC-V: Assembly, C, and Retro Games on the PRG32 Platform*. University of Naples "Parthenope", 2026b. https://github.com/riscv-prg32/Playing-with-RISC-V_PRG32_textbook.

Edward F. Moore. Gedanken-experiments on sequential machines. In *Automata Studies*, pagine 129–153. Princeton University Press, 1956.

Gordon E. Moore. Cramming more components onto integrated circuits. *Electronics*, 38(8): 114–117, 1965.

David A. Patterson e John L. Hennessy. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, 2 edizione, 2020.

PRG32 Project. PRG32 Cartridge Store: catalogue, review, scores, metrics and multiplayer service. https://github.com/riscv-prg32/CartridgeStore, 2026. MIT License.

RISC-V International. RISC-V ABIs specification (ELF psabi). Ratified specification, 2024a. https://github.com/riscv-non-isa/riscv-elf-psabi-doc.

RISC-V International. RISC-V assembly programmer's manual, 2024b. https://github.com/riscv-non-isa/riscv-asm-manual.

RISC-V International. The RISC-V Instruction Set Manual. Technical specification, RISC-V International, 2024c. https://riscv.org/technical/specifications/.

RISC-V International. The RISC-V instruction set manual, volume II: Privileged architecture. Ratified specification, 2024d. https://riscv.org/specifications/ratified/.

Claude E. Shannon. A symbolic analysis of relay and switching circuits. *Transactions of the American Institute of Electrical Engineers*, 57(12):713–723, 1938.

William Stallings. *Computer Organization and Architecture: Designing for Performance*. Pearson, 11 edizione, 2021.

Andrew S. Tanenbaum e Todd Austin. *Structured Computer Organization*. Pearson, 6 edizione, 2013.

Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 1936.

John von Neumann. First draft of a report on the EDVAC. Rapporto tecnico, Moore School of Electrical Engineering, University of Pennsylvania, 1945.